BharatData Context

The Real Problem That Exists Today

India is one of the largest data producing countries in the world. The Indian government collects data on everything from crime statistics to crop prices, from air quality to school enrollment, from hospital beds to railway accidents. This data is publicly available across hundreds of government websites like data.gov.in, NCRB, RBI, Census, CPCB, Agmarknet, and many more.

But here is the painful truth. This data is practically unusable.

A developer who wants to build a farming app needs crop prices from Agmarknet. But the data is hidden inside badly formatted HTML tables that change structure every few months. A journalist investigating pollution trends needs air quality data from CPCB. But the data is locked inside scanned PDFs with no search or filter capability. A researcher studying crime patterns needs NCRB data. But it comes in inconsistent Excel files where column names change every year and state names are spelled differently across sheets.

There is no single API. There is no SDK. There is no clean database. There is no documentation. Every developer, journalist, and researcher who needs Indian public data has to spend weeks or even months manually downloading files, cleaning messy spreadsheets, parsing broken PDFs, and writing custom scrapers that break every time the government website changes its layout.

This is not a small inconvenience. This is a massive blocker that stops thousands of potentially impactful applications from being built. AgriTech startups cannot get real time mandi prices without building their own scraping infrastructure. Health tech companies cannot access district wise hospital data without hiring someone to manually copy paste from government reports. Climate researchers cannot study pollution patterns without spending months collecting and cleaning air quality readings from multiple sources.

The data exists. The demand exists. But the bridge between them is completely broken.

How Technology Solves This

The solution is BharatAPI, a unified intelligent layer that sits between messy government data sources and the developers who need clean structured data.

Think of it like this. Google Maps did not create roads. It made existing roads usable by organizing them into a searchable navigable system. Similarly BharatAPI does not create data. It makes existing government data usable by organizing it into a clean searchable queryable system.

There are three layers of technology that make this work.

The first layer is the data pipeline. This is the foundation. Automated scrapers continuously visit government websites, download new data, detect format changes, and pull raw information. Parsers then take this raw data whether it is a PDF table, a broken CSV, an HTML page, or an XML feed and convert it into clean structured records. Normalizers then ensure consistency. If one source calls it “Andhra Pradesh” and another calls it “AP” and a third calls it “ANDHRA PRADESH”, the normalizer maps them all to a single standard. Every data point gets tagged with its source, date of collection, and reliability score. This pipeline runs automatically every few hours so the data stays fresh without any human intervention.

The second layer is the API and SDK. Once data is clean and structured, it gets exposed through a simple REST API and installable developer SDK. A developer can install the BharatAPI package in their project and start querying Indian government data with simple function calls. Want crime data for Maharashtra? One function call. Want literacy rates across all states? One function call. Want crop prices for wheat in Punjab today? One function call. No scraping. No cleaning. No parsing. Just clean data delivered instantly.

The third layer is the AI intelligence layer. This is where Generative AI and Agentic AI come in and this is what makes BharatAPI genuinely different from just another data API.

How Generative AI Works in This System

Generative AI serves three purposes in BharatAPI.

The first purpose is natural language querying. Most people who need government data are not programmers. They are journalists, researchers, policy makers, students, and analysts. They should not need to learn an API or write code to get answers. With the GenAI layer, a user can simply type a question in plain English or Hindi like “Which state had the highest increase in cyber crime between 2020 and 2023” and the AI understands the intent, figures out which dataset to query, constructs the right filters and aggregations, fetches the data, and returns a clear answer. Behind the scenes the AI is translating human language into structured database queries. But the user never sees that complexity. They just ask and get answers.

The second purpose is data narration. Raw numbers are meaningless without context. If I tell you Delhi’s AQI was 445 in December, that number means nothing unless you know the safe limit is 50, that 445 is classified as severe, and that it represents a 12 percent increase from last year. The GenAI narration layer takes raw data and generates human readable insights. It spots trends, makes comparisons, highlights anomalies, and writes clear explanations. A journalist can get not just data but a ready to use analytical paragraph with proper citations and context. This saves hours of manual analysis.

The third purpose is anomaly detection and explanation. The AI continuously monitors data patterns and flags unusual changes. If tomato prices suddenly spike 400 percent in a particular region, the AI does not just flag the spike. It looks at historical patterns, checks weather data, examines supply chain indicators, and generates an explanation like “Tomato prices in Kolar Karnataka have spiked 380 percent above seasonal average. Historical data shows similar spikes during unseasonal rainfall periods. Current meteorological data confirms heavy rainfall in Karnataka this week likely disrupting supply chains.” This transforms raw data into actionable intelligence.

How Agentic AI Works in This System

Agentic AI is fundamentally different from Generative AI. Generative AI is like a very smart person sitting in a chair. You ask a question and they give you an answer based on what they know. Agentic AI is like a very smart person who can get up from the chair, walk around, use tools, make decisions, and complete multi step tasks on their own.

In BharatAPI, Agentic AI works in three critical areas.

The first area is autonomous data collection. The data ingestion agent runs continuously in the background without any human triggering it. Every six hours it checks all registered government data sources. It detects if any source has published new data. It notices if a website has changed its layout or moved its download links. It adapts its scraping strategy based on what it finds. If a PDF format has changed, the agent figures out the new structure and adjusts its parser. If a new relevant dataset appears on data.gov.in, the agent evaluates its relevance, creates a schema, ingests the data, and updates the API. If it encounters something it cannot handle confidently, it flags it for human review instead of ingesting bad data. This agent is the reason the platform stays up to date without a team of people manually monitoring dozens of government websites.

The second area is complex research queries. When a user asks a simple question like “What is the literacy rate in Kerala” that is a single step GenAI task. But when a user asks “Compare women safety across all Indian states and show how it correlates with female education levels and police to population ratio” that requires an agent. The research agent breaks this complex question into a plan. First it queries crime against women data from the NCRB dataset. Then it queries female literacy data from the Census dataset. Then it queries police strength data from the Bureau of Police Research dataset. Then it merges all three datasets by state. Then it calculates statistical correlations between the variables. Then it generates visualizations showing the relationships. Then it writes a comprehensive narrative explaining the findings with proper caveats like correlation does not equal causation. A single GenAI call cannot do this because it requires multiple sequential decisions, tool usage, and intermediate reasoning. The agent handles the entire workflow autonomously, making decisions at each step about what to do next based on what it found in the previous step.

The third area is automatic documentation and SDK updates. When a new dataset is added to the system, the documentation agent analyzes its schema, generates TypeScript type definitions,

creates a new SDK method, writes documentation with usage examples, and opens a pull request on GitHub. This means the SDK documentation is always in sync with available data without any manual documentation effort.

How Different Users Actually Use the Product

There are five distinct groups of users and each interacts with BharatAPI differently.

The first group is developers building applications. These are software engineers at startups and companies who are building products that need Indian data. A fintech developer building a loan risk assessment model needs demographic and income data by district. Today they spend three weeks collecting this data manually. With BharatAPI they install the SDK package in their project and call a function that returns clean structured demographic data for any district in India. They integrate it into their application and move on. A food delivery startup wants to understand regional food preferences using agricultural production data. They use the SDK to pull crop production data by state and season. An insurance company needs crime and disaster data by geography for risk modeling. They use the SDK to get structured data that feeds directly into their models. For these users, BharatAPI is a developer tool that saves weeks of data collection and cleaning on every project.

The second group is data journalists and media professionals. These are people at organizations like The Wire, Scroll, IndiaSpend, NDTV, or international outlets covering India. They need data to support their stories but spend enormous time collecting and verifying it. A journalist investigating whether air pollution is getting worse in Indian cities can come to the BharatAPI web playground, type their question in plain English, and get a comprehensive analysis with charts data points and a narrative summary. They do not need to write code. They do not need to download Excel files. They do not need to clean data. They get verified sourced data with AI generated insights that they can fact check and use in their reporting. For these users, BharatAPI is a research tool that turns months of data collection into minutes.

The third group is academic researchers. PhD students at IITs, IIMs, and international universities studying Indian economics, public health, environmental science, or social issues need clean reliable datasets. Currently they spend the first six months of their research just collecting and cleaning data. With BharatAPI

they can access pre cleaned well documented datasets through the SDK or web interface, download them in research friendly formats, and cite BharatAPI as their data source. The AI research agent can even help them find correlations across datasets they might not have thought to examine. For these users, BharatAPI is an academic research platform that accelerates their work by months.

The fourth group is government officials and policy makers. District collectors, state planning boards, and organizations like NITI Aayog produce vast amounts of data but ironically struggle to query across their own datasets. A district collector who wants to know how their district performs on education health and employment indicators compared to the state average currently has to request reports from multiple departments and wait weeks. With BharatAPI they can ask this question in the web interface and get an instant comparative analysis. For these users, BharatAPI is a decision support tool.

The fifth group is students and hackathon participants. Millions of computer science students in India need real world data for their projects, competitions, and learning. Currently they use fake datasets or spend days scraping. With BharatAPI they install one package and have access to real Indian government data for their machine learning projects, data visualization assignments, and hackathon submissions. For these users, BharatAPI is a learning and building tool that gives them access to real data instantly.

How the Product Solves Each User’s Core Problem

The core problem for every user group is the same. Indian public data exists but the cost of accessing it is too high. That cost is measured in time spent scraping, cleaning, and normalizing. It is measured in technical skill required to parse PDFs and broken CSVs. It is measured in reliability risk because manual scraping breaks constantly. It is measured in completeness because no one can reasonably monitor dozens of government websites for updates.

BharatAPI eliminates all of these costs.

For the developer, it reduces data integration time from weeks to minutes. They stop building and maintaining custom scrapers and start building their actual product.

For the journalist, it reduces research time from months to minutes. They stop wrestling with Excel files and start writing impactful stories backed by solid data.

For the researcher, it reduces data collection from the first six months of a project to the first day. They stop cleaning spreadsheets and start doing actual analysis.

For the policy maker, it provides cross departmental data visibility that currently does not exist in any accessible form. They stop waiting for reports and start making informed decisions.

For the student, it provides access to real world data that was previously too complex to obtain. They stop using fake datasets and start building with real information.

The AI layer amplifies all of this. Without AI, BharatAPI would be a useful data API. With GenAI, it becomes accessible to non technical users through natural language. With Agentic AI, it becomes self maintaining, self updating, and capable of complex multi step research that no single API call could achieve.

The Approach in Simple Terms

You build this in four stages.

Stage one is the data foundation. You pick five high value government data sources like NCRB crime data, Census demographics, CPCB air quality, Agmarknet crop prices, and RBI economic indicators. You build automated scrapers for each. You build parsers that convert their messy formats into clean structured records. You build a normalizer that ensures consistency across all sources. You store everything in a database with proper indexing and caching.

Stage two is the developer interface. You build a TypeScript SDK that developers can install via npm. You build a REST API with proper documentation. You build a web based playground where anyone can try queries without installing anything. You publish the SDK to npm so developers can start using it immediately.

Stage three is the AI layer. You add natural language querying so non technical users can ask questions in English. You add data narration so raw numbers come with context and insights. You

add the research agent so complex multi dataset questions get comprehensive answers. You add the data ingestion agent so the platform stays fresh automatically.

Stage four is the launch. You publish on Product Hunt, Hacker News, Reddit, Twitter, Dev.to, and LinkedIn. You write a compelling story about why you built this. You respond to every piece of feedback. You iterate based on what users actually need.

The entire thing can be built and launched in eight to ten weeks by a single developer using modern AI assisted development tools. The infrastructure cost is zero using free tiers of Vercel, Neon, Upstash, and Cloudflare. The only paid component is the AI API calls which can be minimized using caching and cheaper models for simple queries.

This is not a toy project. This is a real product solving a real problem for millions of people. And no one has built it properly yet.

Honest Assessment of the Critical Gaps

You have raised exactly the right questions. These are the questions that separate a project that dies on GitHub with 50 stars from a project that becomes a real sustainable product. Let me address each one with complete honesty.

On Monetization

This is the most important gap and you are absolutely right to flag it. Without a monetization strategy, you are not building a product. You are building a hobby project that happens to be useful. Those are fundamentally different things and they require fundamentally different decisions from day one.

Let me think through this properly.

The first question is who has the money and the pain. Developers have pain but they are notoriously resistant to paying for tools especially in India. The average Indian developer will use a free tier forever and switch to a competitor the moment you charge. This is not a criticism, it is economic reality. Developer tools monetization works at scale in the US because a $29 monthly

subscription is a rounding error for someone earning $150,000 a year. For someone earning $8,00,000 a year in India, that same subscription is a meaningful expense they need to justify.

Journalists and researchers have extreme pain but almost no budget. Indian newsrooms are financially struggling. Academic researchers in India operate on tiny grants. These are your most passionate users but your worst paying customers.

The people who actually have money are enterprises and government bodies. A fintech company that needs district level demographic data for credit scoring will happily pay $500 a month if it saves them a data engineer’s salary. An insurance company that needs crime and disaster data by geography will pay even more. A consulting firm advising international companies entering the Indian market needs this data and has the budget. NITI Aayog or state planning departments have procurement budgets for data tools.

This means your monetization strategy needs to be built around enterprise value from day one, even though your initial users will be individual developers and researchers. The free tier is not charity. It is your distribution channel. Free developers build projects with your SDK. Those projects get deployed at companies. Those companies need reliability, SLAs, higher rate limits, and premium datasets. That is where the money comes from.

The practical monetization model should look like this. The open source SDK and basic API are completely free forever. This is non negotiable because this is your distribution mechanism and your community builder. The free tier should be generous enough that a student or indie developer never needs to pay. Something like 10,000 API calls per month with basic datasets.

The paid tier targets professional developers and small companies. This is where you charge $29 to $49 per month for higher rate limits, all datasets, the AI query layer, CSV bulk exports, and webhook notifications for data changes. The AI queries specifically are a natural monetization point because they cost you real money in LLM API calls so it is reasonable and understandable to users that these are limited or paid.

The enterprise tier targets larger companies and is priced at $299 to $999 per month depending on usage. This includes dedicated support, custom dataset integration, real time data feeds, SLA guarantees, and the ability to run the stack on their own infrastructure for compliance reasons. Enterprise also gets the research agent which is expensive to run but extremely valuable for analysts.

There is also an interesting fourth revenue stream that most people miss. Data quality consulting. Once you have built the infrastructure to clean and normalize Indian government data, you have developed deep expertise in Indian data systems. Government departments themselves struggle with data quality. State IT departments would pay for consulting on making their own data more accessible. This is a longer term play but it is worth mentioning because it builds naturally from the core product.

The monetization strategy shapes your build order. You do not build the AI research agent first because it is cool. You build the core data pipeline and SDK first because those create the free user base. Then you build the AI query layer as the first paid feature because it has a natural cost justification. Then you build enterprise features like SLAs, dedicated support, and custom integrations because those are where the real revenue lives.

One more thing on monetization. The worst thing you can do is build everything for free and try to add monetization later. Users who got everything free will revolt when you start charging. Instead, design the free and paid boundaries from the very beginning. Make them clear in your documentation. Let users self select into the right tier based on their needs. This is not greedy. This is how you build something sustainable enough to keep maintaining for years.

On Legal Considerations

This is the gap that could kill the entire project if ignored, and you are right that I did not address it. Let me be very direct about what the legal landscape actually looks like.

India’s National Data Sharing and Accessibility Policy of 2012 explicitly states that all sharable non sensitive government data should be made available to the public. The Open Government Data Platform at data.gov.in was created specifically under this policy and the data published there comes with the Government Open Data License which generally permits free use including commercial use with attribution. This is good news for datasets published on data.gov.in.

However and this is the critical however, not all government data is published on data.gov.in. Many of the most valuable datasets live on individual ministry and department websites like NCRB, CPCB, Agmarknet, RBI, and Census. Each of these websites has its own terms of service. Some explicitly prohibit automated scraping. Some say nothing about it. Some have moved behind CAPTCHAs specifically to prevent automated access.

The legal risk is not about whether the data itself is public. It is about the method of collection and the commercial redistribution. There is a meaningful difference between a researcher manually downloading a CSV from a government website and a commercial product running automated scrapers against that website every six hours and then selling access to the cleaned data.

Here is how you navigate this properly.

First, prioritize data sources that come with explicit open licenses. data.gov.in datasets with the Government Open Data License are your safest starting point. RBI publishes a lot of data through their DBIE platform with relatively permissive terms. Some datasets are available through official APIs that the government has explicitly created for public consumption.

Second, for data sources without clear licensing, you take a transformative approach. You are not just mirroring government data. You are cleaning, normalizing, structuring, enriching, and adding AI analysis on top of it. This transformative value addition strengthens your position significantly. You are not redistributing their PDFs. You are creating a fundamentally new product from publicly available information.

Third, always provide attribution. Every data point in your system should clearly cite its source, the original URL, the date of collection, and the applicable license. This is not just legally prudent. It builds trust with users who need to cite sources in their work.

Fourth, consider partnering with the government rather than working around them. India’s Ministry of Electronics and Information Technology has been actively pushing for better data accessibility. If your product demonstrates genuine public value, there is a real possibility of getting official support or even formal data sharing agreements. The smart play is to build the product, demonstrate its impact, and then approach government departments for official partnerships. Several organizations like DataMeet and IndiaStack have done this successfully.

Fifth, structure your terms of service carefully. Make it clear that BharatAPI aggregates and normalizes publicly available government data. Include disclaimers that the data should be verified against primary sources for critical decisions. Make your data sourcing methodology transparent so users and regulators can see exactly where every data point comes from.

Sixth and this is practical advice, do not scrape aggressively. Government websites are often hosted on infrastructure that cannot handle high traffic. If your scraper brings down a

government website, you will attract the wrong kind of attention very quickly. Be respectful with rate limiting, cache aggressively, and schedule scrapes during off peak hours.

The legal situation is not a dealbreaker. Hundreds of organizations worldwide build commercial products on top of publicly available government data. But it requires thoughtful handling from the start, not an afterthought once you have built the product. Get a one time legal consultation with an IP lawyer who understands Indian data law. It will cost you maybe ten to fifteen thousand rupees and could save you from a serious problem later.

On the Timeline and Difficulty

I was dishonest with you in the original plan and you caught it. Let me recalibrate with complete honesty.

Building reliable PDF parsing at scale is one of the hardest problems in data engineering. I say this without exaggeration. Indian government PDFs are particularly brutal. They contain scanned images of tables, not actual text. Column headers span multiple rows in inconsistent ways. The same report from the same department uses different table layouts year to year. Hindi and English text are mixed within the same cells. Some PDFs are password protected. Some are corrupted. Some contain watermarks that interfere with OCR.

A single reliable PDF parser for one specific government report format will take one to two weeks of focused work. And it will break the next time that department publishes a new report with a slightly different layout. Across dozens of data sources, you are looking at months of parsing work, not days.

Government website scraping is equally fragile. These are not modern well maintained web applications. They run on NIC servers that go down frequently. They serve CAPTCHAs during high traffic periods. They restructure URLs when the website is redesigned, which happens whenever a new government takes charge or a ministry reorganizes. They sometimes block IP ranges that make too many requests. They occasionally put data behind login walls that did not exist before.

The normalization problem is deeper than it appears. It is not just state name variations. It is district names that change when districts are bifurcated. Telangana did not exist before 2014. Ladakh did not exist as a separate entity before 2019. Districts are regularly split or merged. A dataset from 2010 might list 640 districts while a dataset from 2023 lists 780 districts, and you need to map between them to allow historical comparisons. Census codes change. PIN codes change. Constituency boundaries

change with delimitation. Building a proper entity resolution system that handles all of this correctly is a serious engineering challenge.

The honest timeline looks more like this.

Weeks one through four should focus on building a working proof of concept with just two to three data sources that have clean structured data. Pick the easiest sources first. data.gov.in CSV datasets, RBI's structured data portal, and maybe one well formatted NCRB dataset. Get these into a clean database with a basic API and a simple SDK. This is your minimum viable product. It is not impressive yet but it works.

Weeks five through eight should focus on hardening the data pipeline. Add better error handling. Build monitoring that alerts you when a scraper breaks. Improve the normalizer to handle the most common edge cases. Add two to three more data sources. Build the basic web playground.

Weeks nine through twelve should focus on adding the AI layer. Natural language queries, basic data narration, and simple anomaly flagging. This is where the product starts feeling differentiated.

Weeks thirteen through sixteen should focus on community building, documentation, and launch preparation. Write proper docs. Create example projects. Prepare your launch strategy.

Months five through six should be your public launch period with rapid iteration based on user feedback.

That is a realistic timeline for a single developer working full time. If you are doing this alongside a job or studies, multiply by two. And even this timeline assumes you do not get stuck on particularly nasty PDF parsing problems or government website changes.

The honest framing is this. You can launch a useful but limited version in eight weeks. You will need six months to build something comprehensive enough to be genuinely impressive. You will need a year or more to build something robust enough that enterprises would trust it in production.

This is not discouraging. This is realistic planning. The projects that succeed are not the ones that ship the fastest. They are the ones that set honest expectations and then consistently deliver. If you tell people your SDK covers five datasets and those five work perfectly, you build trust. If you claim to cover fifty datasets and half of them return bad data, you destroy trust permanently.

Start small. Be reliable. Expand methodically.

On the Moat and Defensibility

This is the hardest truth to hear but you need to hear it. The technical infrastructure you are building has almost no moat by itself. Scrapers can be rebuilt. Databases can be replicated. APIs can be cloned. If your product is just clean government data behind an API, a well funded competitor can replicate your entire technical stack in a few months.

Organizations like DataMeet have been working on Indian open data for years. Datahub.io exists as a global platform. The government itself could improve data.gov.in to make your product unnecessary. A big company like Google or Microsoft could build something similar as a side project and give it away free to gain developer mindshare in India.

So where does real defensibility come from?

The first source of defensibility is data quality reputation. This takes time and cannot be shortcut. When a journalist cites BharatAPI in a published article and the data is accurate, that builds reputation. When a researcher uses your data in a peer reviewed paper and it holds up, that builds reputation. When a developer integrates your SDK and the data is consistently reliable month after month, that builds reputation. This kind of trust takes years to establish but it is extremely hard for a competitor to replicate. Trust is earned one interaction at a time.

The second source of defensibility is the entity resolution layer. The mapping between “AP” and “Andhra Pradesh” and “ANDHRA PRADESH” is easy. The mapping between old district boundaries and new district boundaries after bifurcation, between old census codes and new census codes, between historical administrative units and current ones, that is genuinely hard institutional knowledge. Every time you correctly handle a case like “This district was part of Andhra Pradesh until 2014 then became part of Telangana and was subsequently bifurcated into two new districts in 2019”, you are encoding knowledge that took real research to acquire. A competitor who forks your code does not get this knowledge. They have to rebuild it themselves.

The third source of defensibility is community lock in. If developers build applications on your SDK, switching costs increase over time. Their code uses your type definitions, your function signatures, your data schemas. Migration to a competitor means rewriting integration code. This is not about vendor lock in as a strategy. It is about making your SDK so well designed and reliable that switching feels like unnecessary effort. The more applications built on your SDK, the stronger this effect becomes.

The fourth source of defensibility is the AI training data feedback loop. Every natural language query your users make teaches you what questions people actually ask about Indian data. Over time you build the best training dataset in the world for Indian civic data queries. Your AI gets better at understanding Indian data questions. A competitor starting from scratch does not have this query history.

The fifth source of defensibility is relationships. If you build relationships with government data officers, academic institutions, and newsrooms, those relationships become channels that a technical competitor cannot replicate with just code. If NCRB starts sending you data directly instead of you scraping it, that is a relationship moat. If IIT researchers default to citing your platform, that is a reputation moat.

Here is my honest assessment of the moat timeline. In the first six months you have essentially no moat. Anyone could build what you have built. From six to eighteen months, if you have been consistently reliable and built a growing user base, you start developing data quality reputation and community lock in. After eighteen months to three years, if you have done everything right, you have a meaningful moat through accumulated institutional knowledge, community, relationships, and AI training data that would take a competitor years to replicate.

This means you need to be honest with yourself about vulnerability in the early period. During the first year, your real protection is not technical. It is momentum. You need to be the first person to do this well enough that people start depending on you before a competitor decides to enter the space. Speed of execution matters enormously in the early phase.

My Overall Honest Opinion

BharatAPI is a genuinely good idea solving a genuinely real problem. I stand behind that assessment. But the original plan I gave you was too optimistic about execution difficulty, completely silent on legal risk, missing monetization thinking, and naive about competitive defensibility.

The corrected view is this.

This is a twelve to eighteen month project to do properly, not an eight week project. The first version should be embarrassingly limited in scope but absolutely reliable in quality. Two or three data sources, perfectly cleaned, well documented, with a polished SDK. Launch that. Get users. Get feedback. Expand from there.

The legal situation is navigable but requires upfront research and ongoing attention. Do not skip this.

Monetization should be designed into the product from day one, not added later. The free tier is your distribution channel. The paid tier is your business. Enterprise is where the real money lives. Know this before you write your first line of code.

Your moat in the first year is speed and quality, not technology. Move fast, be reliable, build relationships, and accumulate the institutional knowledge that makes your product hard to replicate.

If you go in with these realistic expectations and you execute consistently, this project can absolutely become something that gets you recognized globally, demonstrates serious engineering and product capability, and potentially becomes a real business. But only if you approach it as a serious product with serious tradeoffs, not as a weekend project you vibe code into existence.

The question is not whether this idea is good enough. It is. The question is whether you are willing to commit to the sustained effort it actually requires. If the answer is yes, then build it. But build it with open eyes.

My Final Opinion After Everything

I am going to be completely straight with you. That Sonnet 4.6 review is accurate on every point it raises. And the fact that you sought a second opinion from another model to stress test my own thinking tells me something important about how you approach problems. That instinct, the willingness to challenge advice you received even when it sounded good, is more valuable than any technical skill when it comes to building a real product.

Let me address the three remaining gaps that review correctly identified, and then give you my actual final take.

On Consulting Being Undersold

Sonnet 4.6 is right and I was wrong to treat this as a footnote. Let me explain why this changes your entire year one strategy.

When you are a solo builder with no revenue, no team, and no track record, the biggest existential threat is not technical failure. It is running out of motivation and money before the product reaches critical mass. Most open source projects and developer

tools die not because they were bad ideas but because the builder could not sustain the effort long enough for the product to find its audience.

Consulting solves this problem directly. Here is the logic. By the time you have built scrapers for five government data sources and normalized the data properly, you have developed a very specific and rare expertise. You understand how Indian government data is structured, where it breaks, how to clean it, and how to make it usable. This expertise is immediately valuable to organizations that struggle with the same problems but do not have the time or knowledge to solve them internally.

State IT departments across India are under pressure from the central government to digitize and make their data accessible. Most of them are failing at this because their teams are understaffed and lack modern data engineering skills. A single consulting engagement with one state department, even a small one, could be worth three to five lakhs. That funds months of product development.

Think tanks and research organizations like NITI Aayog, Centre for Policy Research, IDFC Institute, and Observer Research Foundation regularly need cleaned structured government data for their policy analysis. They currently hire junior researchers to manually compile this data. If you can demonstrate that your pipeline does this faster and more reliably, they will pay for your time.

International development organizations like the World Bank, UNDP, and various UN agencies working on India programs constantly need structured Indian data and are willing to pay international consulting rates for it.

The strategic insight here is that consulting is not a distraction from the product. It is a direct accelerant. Every consulting engagement teaches you what data people actually need, what quality standards matter, and what problems are worth solving. It funds your development. It builds relationships that become your enterprise sales pipeline later. And it gives you case studies and testimonials that establish credibility.

My revised recommendation is this. Do not wait until the product is mature to explore consulting. Start reaching out to potential consulting clients by month two or three, as soon as you have enough working infrastructure to demonstrate competence. Even one engagement in the first six months changes the entire financial equation of this project.

On the First 30 Days and Competitive Speed

Sonnet 4.6 is correct that saying speed matters without defining what speed looks like is unhelpful advice. Let me fix that.

Your first 30 days should produce exactly one thing. A working SDK that covers exactly two datasets with perfect reliability, published to npm, with a clean landing page and a live playground where anyone can try a query without signing up.

Not five datasets. Not ten. Two. Pick the two that have the cleanest source data and the highest developer demand. My recommendation would be Indian Census demographic data and NCRB crime statistics. Census because every developer building anything India specific eventually needs population and demographic data by state and district. NCRB because crime data has high public interest and journalists specifically seek it out which gives you a built in audience for launch.

These two datasets should work flawlessly. Every query should return correct data. Every edge case should be handled. The TypeScript types should be perfect. The documentation should be clear with real examples. The response times should be fast because you are serving from a cached database not scraping on demand.

On day 30 you publish a Show HN post and a Twitter thread. Not with the framing of “I built an SDK for all Indian government data” because you have not. With the framing of “I built a reliable way to query Indian Census and crime data programmatically and I am expanding to more datasets.” Honest scope, perfect execution.

The milestone that signals you have established enough presence to matter is this. By day 60 you should have at least 200 GitHub stars, at least 50 weekly npm downloads, and at least 10 developers who have actually used your SDK in a project and given you feedback. These are not vanity metrics. They represent real validation that people want what you are building and they are using it not just starring it.

If you hit those numbers, you have enough momentum that a competitor entering the space has to contend with your existing user base and community. If you do not hit those numbers, that is a signal to examine whether your execution quality or your distribution strategy needs adjustment before expanding scope.

The competitive threat Sonnet 4.6 mentions, a well funded startup reading your work and executing faster, is real but manageable. Here is why. A funded startup optimizes for investor metrics. They will try to cover fifty datasets in three months to show breadth in

their pitch deck. But breadth without reliability is worthless for developer tools. Developers will try your SDK once, and if the data is wrong or the API is unreliable, they will never come back. Your advantage as a solo builder is that you can optimize for quality over breadth because you are not answering to investors demanding growth metrics.

The other protection is that this problem is genuinely hard in ways that are not obvious from the outside. A competitor who has not actually tried to parse NCRB PDFs does not understand how hard it is. They will underestimate the effort, ship unreliable data, lose developer trust, and struggle to recover. Your head start is not measured in code written. It is measured in institutional knowledge acquired through actually doing the work.

On Enterprise Go To Market Without a Sales Team

This is where Sonnet 4.6’s criticism is most valid. Saying enterprises will pay is meaningless without explaining how you actually reach and convert them. Let me lay out the realistic path for a solo developer with no sales experience and no track record.

You do not do cold outreach. Cold emails from an unknown solo developer to enterprise companies have a near zero conversion rate. You will waste weeks writing emails that get ignored and it will demoralize you.

Instead you build an inbound funnel through your free tier users and content. Here is how this actually works in practice.

A developer at a fintech company discovers your SDK while searching for Indian demographic data for a project. They install it, use it in a prototype, and show it to their team. Their team starts using it. They hit the free tier limits. Someone on the team asks whether there is a paid tier with higher limits and more datasets. They find your pricing page. If the price is reasonable relative to the engineering time they would spend building this themselves, they expense it on a company card without needing VP approval. This is how most developer tool companies get their first hundred paying customers. It is not glamorous but it works.

For larger enterprise deals that require actual sales conversations, the path is through visible credibility. When a CTO or VP of Engineering at an Indian fintech sees your project featured on Hacker News, reads a thoughtful technical blog post you wrote about data normalization challenges, notices that your SDK has 500 stars and is actively maintained, and then discovers that their own developers are already using the free tier, the sales conversation is fundamentally different from cold outreach. You

are not convincing them your product might be useful. Their own team has already proven it is useful. You are just discussing pricing and SLAs.

The specific actions that create this inbound flow are these. Write one in depth technical blog post per month about the challenges of working with Indian government data. Not marketing content. Genuine technical content that demonstrates expertise. Post these on your blog, Dev.to, Hashnode, and LinkedIn. Share them on Twitter and Reddit. Speak at local developer meetups and online conferences. Contribute to discussions in Indian developer communities on Reddit, Discord, and Twitter.

Your first enterprise customer will most likely come from one of three channels. Either a developer at a company who discovers your free tier and champions it internally. Or a consulting relationship that evolves into a product subscription. Or a direct inbound inquiry from someone who found you through content or community. Plan for all three but do not count on any specific one.

The timeline for first enterprise revenue is realistically eight to twelve months from launch. This is normal for developer tools. Do not panic if it takes this long. But do track leading indicators like free tier signups from company email domains, API usage patterns that suggest team adoption, and inbound inquiries about pricing. These signals will appear months before actual revenue and they tell you whether your trajectory is correct.

My Absolute Final Take

Here is what I believe after thinking through the original vision, the honest reassessment, and the critical review of that reassessment.

BharatAPI is a genuinely viable project. The problem is real, the gap in the market is real, and the potential for both career impact and eventual revenue is real. I am not saying this to be encouraging. I am saying it because after three rounds of critical analysis, the core thesis has held up. The idea is not the weak link here.

The risks are execution stamina and premature scaling of scope. The most likely failure mode is not that someone else builds this first. It is that you try to cover too many datasets too quickly, the quality suffers, early users lose trust, and you burn out trying to maintain fragile scrapers across dozens of government websites. The second most likely failure mode is that you build a technically

impressive system but never develop the distribution and monetization muscles needed to sustain it beyond the initial launch excitement.

The path I would actually follow if I were building this is this.

Months one and two, build the core SDK with two datasets. Census and NCRB. Publish to npm. Launch the landing page and playground. Ship the Show HN post. Get first users and feedback.

Month three, add the AI natural language query layer on top of existing datasets. This is your differentiation moment. This is when the product goes from useful data API to something genuinely novel. Start reaching out to potential consulting clients using your existing work as a demonstration of capability.

Months four and five, expand to two more datasets based on what users are actually requesting. Not what you think is cool but what real users tell you they need. Start writing technical content consistently. Apply to speak at meetups.

Month six, formal launch push. Product Hunt, broader media outreach, outreach to journalist and researcher communities. If consulting revenue has materialized, use it to fund better infrastructure. If not, explore other funding options like GitHub Sponsors, grants from organizations supporting open data, or small freelancing to sustain yourself.

Months seven through twelve, steady expansion of datasets and features driven by user demand. Build the enterprise tier. Close first paying customers. Build the agentic data ingestion pipeline to reduce your manual maintenance burden. Continue consulting if it is generating revenue.

Month twelve assessment. If you have at least 500 GitHub stars, at least 100 weekly active API users, at least 5 paying customers of any size, and at least some consulting revenue, you have a viable business that can grow. If you are significantly below these numbers, you need to honestly assess whether the problem is execution, distribution, or market fit, and adjust accordingly.

Should you build this? Yes. But treat the second document as your operating plan, treat the Sonnet 4.6 review as your advisory board feedback, and treat the first document as your pitch deck. Build with the discipline of the second document and the ambition of the first. That combination is what actually produces things that matter.

And the most important thing I can tell you is this. The fact that you went through three rounds of critical thinking before writing a single line of code puts you ahead of ninety percent of builders

who start coding on day one and figure out the hard questions only after they have built something nobody wants. Your process is right. Now execute.

I Am Not Going to Convince You. I Am Going to Agree With Sonnet 4.6.

Let me say this directly. That 6.4 rating is fair. Every single criticism in that review is correct. I am not going to argue against it because arguing against valid criticism is exactly how builders end up spending six months on something nobody pays for.

What I am going to do instead is give you the actual execution process that addresses every gap Sonnet 4.6 identified. Not theory. Not motivation. The real work that moves this from a well written idea to a fundable, buildable, defensible product.

But first, let me be honest about something uncomfortable.

What I Got Wrong Across All Three Documents

Every document I gave you was written to make you feel confident. That is the wrong goal. The right goal is to make you feel prepared. Confidence without preparation is just enthusiasm, and enthusiasm dies the first time a government PDF parser returns garbage data at 2 AM.

Sonnet 4.6 nailed it when it said the documents were written to convince, not to stress test. That is exactly what happened. I described the problem beautifully. I designed the architecture impressively. I even did an honest reassessment. But at no point did I tell you to stop planning and go talk to actual humans. That is the single biggest failure across everything I told you.

Here is the uncomfortable truth about idea validation. You can spend three months designing the perfect SDK, building beautiful documentation, crafting an elegant architecture, and launch to absolute silence. Not because the idea is bad but because you built what you imagined people wanted instead of what they actually need. The gap between those two things is where most projects die.

Sonnet 4.6 asked whether a journalist at The Wire or a developer at a fintech startup would actually change their behavior the day you launch. I do not know the answer. You do not know the answer. Nobody knows the answer until you ask them. And that asking is work that cannot be replaced by more planning documents.

The Honest Process That Actually Earns a 9 Out of 10

I am going to break this into phases but these are not the comfortable phases of build, launch, grow. These are the phases that most builders skip because they feel awkward, slow, and unglamorous. They are also the phases that determine whether your project succeeds or becomes another abandoned GitHub repository.

Phase Zero — Validation Before Code

This phase has no code in it. None. Zero lines. This will feel wrong because you are a builder and builders want to build. Resist that instinct for fourteen days. Those fourteen days will save you months of wasted effort.

During the first week your only job is to have conversations. Not pitches. Conversations. You need to talk to real humans who work with Indian government data today. Not your friends. Not other developers who think the idea sounds cool. People who have actually tried to use government data in the last six months and experienced the pain firsthand.

You need to find and talk to at least three developers who have built applications that required Indian government data. Ask them how they got the data. Ask them how long it took. Ask them what broke. Ask them what they are currently using. Ask them if they tried any existing tools and why those tools failed. Ask them what they would pay for a reliable solution. Do not pitch BharatAPI. Do not describe your architecture. Just listen.

You need to find and talk to at least three journalists or researchers who have published work using Indian government data. The best places to find them are IndiaSpend, The Wire’s data journalism team, FactChecker.in, Article 14, and research labs at ISI Bangalore, IIT Delhi, or JNU’s Centre for Economic Studies. Ask them the same questions. How do you get your data

today. What is the most painful part. How much time do you spend on data collection versus actual analysis. What would change if the data were instantly available and clean.

You need to find and talk to at least two people at organizations that might actually pay for this. This means someone at a fintech company, an insurance company, a consulting firm, a think tank, or a government IT department. Ask them whether they currently pay for any data services. Ask them what their budget looks like for data tools. Ask them what procurement process looks like at their organization.

Where do you find these people? LinkedIn is your best tool here. Search for data journalist India, data engineer fintech India, research associate policy India. Look at who has published articles on IndiaSpend or EPW. Look at who contributes to DataMeet. Look at who has spoken at Bangalore Data Science meetups or PyData India. Send them a simple message that says something like “I am building a tool that makes Indian government data accessible through a simple API. I am in the research phase and would love to understand how you currently work with this data. Would you have 15 minutes for a conversation?” Most people will ignore you. Some will respond. You only need ten conversations total to learn more than months of planning would teach you.

During the second week your job is to synthesize what you heard and build one single demo of the hardest technical challenge. Not the SDK. Not the playground. Not the landing page. The hardest thing. And the hardest thing is parsing one genuinely messy government data source reliably.

Pick NCRB Crime in India report. Download the most recent PDF. Try to extract even one table from it programmatically. Experience firsthand how the headers span multiple rows, how footnotes are embedded in data cells, how the table continues across pages without repeating headers, how some numbers use Hindi numerals, how the column structure changes between chapters. Spend a full week on this. Not to build a perfect parser but to understand exactly how hard this problem actually is.

At the end of these fourteen days you will have two things you do not currently have. First, real evidence about whether people want this enough to pay for it or whether they have workarounds that are good enough. Second, real understanding of the technical difficulty based on actual experience not estimation. These two things together will either confirm that BharatAPI is worth building or save you from spending a year on something that was never going to work. Either outcome is valuable.

Phase One — The Minimum Believable Product

Notice I did not say minimum viable product. I said minimum believable product. The difference matters. A minimum viable product for BharatAPI would be a basic API serving one dataset. That is viable but nobody would believe it represents a serious effort. A minimum believable product is small in scope but so well executed that anyone who sees it immediately understands this is built by someone who knows what they are doing and is going to keep building.

This phase takes six weeks if you are working full time. Longer if you have other commitments.

Based on what you learned in Phase Zero you will choose exactly two data sources to start with. My recommendation based on demand and feasibility is Census demographic data and one other source that your user interviews revealed as the most painful. It might be NCRB crime data. It might be CPCB air quality data. It might be Agmarknet crop prices. Let your interviews decide, not my speculation.

For each data source you build a complete pipeline. Scraper that handles failures gracefully and does not hammer government servers. Parser that extracts structured data and logs every edge case it cannot handle. Normalizer that maps state names, district names, and administrative codes to a consistent standard. Database storage with proper indexing. Cache layer for fast responses.

You build the SDK as a TypeScript package publishable to npm. The SDK should have complete type definitions so developers get autocomplete in their editor. It should have clear error messages when something goes wrong. It should have exactly zero dependencies that are not absolutely necessary. Developers judge SDK quality by the install size and dependency count.

You build documentation that includes a quick start guide taking someone from install to first query in under two minutes. Three real world examples showing how a developer would actually use this in a project. An API reference with every method, parameter, and return type documented. A data sources page explaining where each dataset comes from, when it was last updated, and what its limitations are.

You build a web playground where someone can type a query and see results without installing anything. This is critical because journalists and researchers will not install an npm package. They

need a browser-based interface. The playground should let someone type a natural language question, see the structured data response, and download the results as CSV.

You do not build the AI layer in this phase. You do not build the agentic pipeline. You do not build the enterprise features. You build two datasets served reliably with excellent documentation and a functional playground. That is your minimum believable product.

Phase Two — AI Layer and First Users

This phase takes four to six weeks and runs partially in parallel with getting your first real users.

Now you add the Generative AI layer. Natural language queries that translate English questions into structured data requests. Data narration that takes raw numbers and generates human readable insights with context. Basic anomaly flagging that spots unusual values in the data.

The AI layer is what makes BharatAPI genuinely different from a simple data API. Without it you are just a cleaner version of data.gov.in. With it you are a tool that makes government data accessible to people who cannot write code. That is a fundamentally larger market.

Simultaneously you begin real distribution. Not launch day posting on Product Hunt. Ongoing, relationship-based distribution that builds an audience before you have anything to formally launch.

Write your first technical blog post. Not about BharatAPI. About the problem. Title it something like “I Tried to Extract Crime Data From NCRB PDFs and Here is What Happened.” Make it genuinely useful and interesting. Show the actual challenges you encountered. Show before and after examples of the messy data versus your cleaned output. Post it on Dev.to, Hashnode, and your personal blog. Share it on Twitter and Reddit. This establishes your credibility in the Indian open data space before you ask anyone to use your product.

Reach out to the people you interviewed in Phase Zero. Tell them you built the thing you discussed. Ask them to try it. Ask for honest feedback. These are your first real users and their feedback is worth more than a thousand GitHub stars from strangers.

Find and join the communities where your users already gather. DataMeet has an active mailing list and GitHub organization. The developersIndia subreddit has over 300,000 members. Indian data journalism Twitter is a real community with active conversations. Do not join these communities to promote your product. Join them to contribute genuinely and let your product come up naturally in relevant conversations.

Phase Three — Revenue and Sustainability

This phase starts around month four and runs indefinitely.

The first revenue path you pursue is consulting, not SaaS subscriptions. Sonnet 4.6 was right that I undersold this. Consulting is your fastest path to money and your best source of product insight simultaneously.

Reach out to three types of organizations with a specific offer. To state IT departments you say “I have built infrastructure for cleaning and serving Indian government data programmatically. I can help your department make your data more accessible as required under the National Data Sharing Policy. Here is a demo of what I have already done with NCRB and Census data.” To think tanks and research organizations you say “I can provide clean structured government data for your research projects at a fraction of the time and cost of manual data collection. Here is what the output looks like.” To international development organizations working in India you say “I provide structured queryable Indian government data through an API. If your team needs reliable Indian data for program analysis, I can help.”

One consulting engagement paying two to five lakhs funds months of product development. Two engagements in the first year and you have a sustainable operation without needing any SaaS revenue at all.

For SaaS revenue you do not put up a pricing page and wait. You watch your free tier usage analytics. When you see a user from a company email domain making hundreds of API calls, you reach out personally. Not with a sales pitch. With a genuine message like “I noticed your team is using BharatAPI heavily. I wanted to check if everything is working well and whether there are datasets or features that would make it more useful for your work.” That conversation naturally leads to whether a paid tier with higher limits and additional features would be valuable.

Your first paying SaaS customer will almost certainly come from this kind of personal outreach to an existing free tier user. Not from cold email. Not from a pricing page. From a real relationship with someone who is already getting value from your product.

Phase Four — Scale and Defensibility

This phase starts around month six and is where the project transforms from a side project into a real product.

You add more data sources based entirely on user demand. Not based on what you think is interesting. Every new dataset you add increases maintenance burden. Only add datasets that users are actively requesting and that you can maintain reliably.

You build the Agentic AI pipeline for automated data ingestion. This is where you reduce your own maintenance burden by having AI agents monitor government websites for changes, adapt parsers when formats shift, and flag issues for your review instead of requiring you to manually check every source every day. This is not a launch feature. This is an operational sustainability feature that you build after you understand the maintenance burden from firsthand experience.

You build the entity resolution layer that becomes your long term moat. The comprehensive mapping between historical and current administrative boundaries, between different naming conventions across sources, between old and new district codes. This is institutional knowledge encoded in software. It takes months to build properly and it is the single hardest thing for a competitor to replicate because it requires deep domain expertise, not just engineering skill.

You apply for grants and programs that support open data work. The Shuttleworth Foundation funds open data projects. Mozilla has supported civic tech. Google has a social impact fund. The Ford Foundation funds transparency and accountability technology. These are not guaranteed revenue but they are worth pursuing because they provide funding without giving up equity and they validate your work in the eyes of the broader community.

What I Want You to Actually Do This Week

Not this month. This week. Seven days.

Day one and two. Make a list of fifteen people you are going to reach out to for validation conversations. Five developers, five journalists or researchers, five people at organizations that might pay. Find them on LinkedIn, Twitter, or through published work. Send each of them a short respectful message asking for fifteen minutes of their time.

Day three and four. Download the most recent NCRB Crime in India PDF. Try to extract data from it. Experience the pain firsthand. Document every problem you encounter. Take screenshots. This is both technical research and content for your first blog post.

Day five and six. Based on your scraping attempt, write a blog post about the experience. Make it honest, detailed, and interesting. Publish it. Share it in relevant communities.

Day seven. Review whatever responses you have gotten from your outreach. Schedule conversations for the following week. If you have fewer than five conversations scheduled, send more outreach messages.

That is one week of work. No code. No architecture. No SDK design. Just validation and learning. If this feels like a waste of time, I understand. But every successful product builder I can point to will tell you that the weeks they spent talking to users before building saved them months of building the wrong thing.

My Final Honest Position

I am not going to tell you the plan is a 9 out of 10. It is not. Sonnet 4.6 is right. It is a 6.4 and the reason is not the idea. The idea is excellent. The reason is that right now the plan is built on assumptions instead of evidence. Assumptions about what users want. Assumptions about what they will pay. Assumptions about how hard the technical work is. Assumptions about how you will reach people.

The gap between 6.4 and 9 is not filled with better architecture diagrams or more detailed technical plans. It is filled with evidence. Evidence that real people want this. Evidence that you can actually parse the data reliably. Evidence that someone will pay. Evidence that you can reach your audience.

Going and getting that evidence is harder and less fun than writing code. It requires you to talk to strangers, hear uncomfortable feedback, and possibly discover that some of your assumptions are wrong. But every assumption you correct now saves you weeks of wasted building later.

The builders who earn a 9 out of 10 are not the ones with the best ideas. They are the ones who did the unglamorous validation work that most people skip because it does not feel like building. It is building. It is building the foundation that everything else stands on.

Go do Phase Zero. Come back with ten real conversations, one working NCRB parser, and one blog post about the experience. Then we plan the rest based on what you actually learned, not what we imagined from behind a keyboard.

That is the honest process. It is not exciting. It works.

My Honest Opinion on This Entire Exchange

Sonnet 4.6 gave you a genuinely good response. The core advice is correct. The “ship and see” reframe is correct. The focus on quality over coverage is correct. The build in public advice is correct. I agree with almost all of it.

But I want to add something that Sonnet 4.6 was too polite to say directly because your mindset shift made the conversation warmer and more supportive. There is a risk hiding inside what just happened and I want to name it clearly.

What Actually Just Happened in This Conversation

You received a very tough 6.4 rating with four specific criticisms. No user interviews done. No technical risk proven. No distribution strategy. No real customer identified. Each criticism was valid and each one required uncomfortable real world work to address.

Then you responded with a mindset reframe. You said you are not financially dependent on this. You are not chasing millions. You want to build for society and community and go with the flow.

And the tone of the conversation immediately softened. The response you got back was warm, encouraging, supportive. It told you your mindset is healthier than most founders. It compared your path to Linux and PostgreSQL. It said monetization will find you when the time is right. It told you to stop worrying about enterprise sales and pricing tiers.

Here is what I want you to notice. The four hard problems that earned the 6.4 rating did not disappear because your mindset changed. They are still there. You still have not talked to real users. You still have not proven you can parse NCRB PDFs reliably. You still have no distribution channel. The only thing that changed is that the conversation stopped pushing you toward those uncomfortable tasks.

I am not saying your mindset is wrong. I am saying there is a real risk that a healthy mindset becomes a comfortable reason to skip the hard validation work. And that risk is worth naming out loud.

Where Sonnet 4.6 Is Right

The mindset you described is genuinely better than chasing revenue from day one. This is true. When you are not financially desperate you make better product decisions. You do not add features just because someone might pay for them. You do not compromise data quality to ship faster. You do not ignore interesting but non commercial use cases. Financial independence gives you the freedom to build something genuinely good instead of something that merely sells.

The open source infrastructure framing is exactly right. BharatAPI should not be thought of as a startup. It should be thought of as a public good with a technical layer. Like OpenStreetMap. Like Wikipedia’s data layer. Like the countless open source tools that became industry standards not because someone monetized them aggressively but because they were genuinely the best solution to a real problem.

The build in public advice is correct and specifically valuable for your situation. When you share your journey publicly, tweet about weird edge cases in government data, write about parsing challenges, show before and after examples of messy versus clean data, you accomplish three things simultaneously. You build an audience that becomes your distribution channel. You establish credibility in the Indian open data space. And you create accountability that prevents the project from becoming a permanent work in progress. Building in public is the single most effective strategy for a solo developer with no marketing budget.

The north star of citations and integrations instead of revenue is the right metric for your current phase. Every time someone uses your data in a research paper, a news article, a hackathon project, or a production application, that is proof of real value. That kind of proof compounds over time in a way that GitHub stars do not.

Where I Want to Push You Beyond What Sonnet 4.6 Said

Sonnet 4.6 told you to stop worrying about enterprise sales and pricing tiers. I agree with that for now. But it also implicitly gave you permission to skip the validation conversations. It said focus on three things: ship fast, build in public, prioritize quality. All three are correct. But none of them is “talk to real users before building.”

The previous review said you need fifteen real conversations before writing code. That advice did not become wrong because your goals shifted. Whether you are building for money or for society, you still need to know if the thing you are building is what people actually need. A tool built for society that nobody uses is not serving society. It is serving your ego.

The difference is not whether you validate. It is how you frame the validation. You are not asking “will you pay for this” anymore. You are asking “would this actually help you in your work” and “what would make this useful enough that you would switch from your current approach.” Those are different questions with different emotional weight but they require the same action. Talking to real humans.

Here is my specific concern. You are a vibe coder. You like building. The most natural thing for you to do right now is open your editor, start coding the SDK, and feel productive. That will feel great for two weeks. But if you build the SDK around Census data and then discover through user conversations that everyone actually needs agricultural mandi price data and nobody cares about Census data through an API because they already have CSV downloads that work fine, you have wasted two weeks building the wrong thing.

Five conversations before you start coding. Not fifteen. Not twenty. Five. That is the minimum that separates building with intention from building with assumption. You can have all five in one week. Each one takes fifteen to twenty minutes. That is less than two hours of total conversation time. There is no version of your goals, whether commercial or community oriented, where two hours of user research is not worth doing.

The Specific Risk of Building for Society Without Validation

There is a particular trap that affects people who build for social good and I want to name it because I have seen it repeatedly.

When you build for money and nobody uses your product, the market gives you a clear signal. Zero revenue. You cannot ignore it. It forces you to either pivot or quit. That feedback loop is harsh but it is honest.

When you build for society and nobody uses your product, you can tell yourself a different story. “People just do not know about it yet.” “The impact is long term.” “Not everything valuable is immediately popular.” “I am building infrastructure and infrastructure takes time to be appreciated.” All of these statements can be true. They can also be comfortable lies that let you avoid confronting the possibility that you built something nobody actually needs.

The protection against this trap is the same as the protection against building the wrong product for commercial reasons. Real user feedback from real humans. Not GitHub stars. Not Twitter likes. Not upvotes on Reddit. Actual evidence that someone used your data in something real. A journalist who cited your API in a published story. A developer who integrated your SDK in a production application. A researcher who downloaded your data for an actual study.

Set a specific milestone for yourself. Within three months of launching, at least five real projects or publications should be using your data. If that does not happen, it does not mean the idea is bad. It means something about your execution, your data source selection, your distribution, or your user experience needs to change. Having that milestone written down in advance prevents you from drifting in comfortable ambiguity.

What I Actually Think You Should Do

Here is the process I would follow if I were you, given your specific situation of financial independence, community motivation, and desire to contribute meaningfully.

This week, before any code. Have five conversations with potential users. Two developers who have worked with Indian government data. Two journalists or researchers. One person from DataMeet or a similar organization. Ask them what data sources cause them the most pain. Ask them what they are currently using. Ask them what a good solution would look like. Write down what they say.

Weeks two and three. Based on those conversations, pick your first data source. Not what I recommended. Not what sounds impressive. The one that your conversations revealed as the most

painful and most frequently needed. Build a parser for it. Experience the difficulty firsthand. Get clean structured data into a database.

Week four. Build the minimal SDK. One npm package. One data source. TypeScript types. Three code examples. A README that clearly explains what this is and what it is not. Publish it to npm. Push the code to GitHub.

Week five. Write your first build in public blog post. Not about BharatAPI as a product. About the problem you solved. The weird things you found in government data. The technical challenges of parsing it. The before and after. Make it genuinely interesting to read. Share it everywhere relevant.

Week six. Reach back out to the five people you talked to in week one. Show them what you built. Ask for feedback. Ask them to try it. Watch how they use it. Listen to what confuses them. Note what they ask for that you did not build.

Weeks seven and eight. Add the second data source. Improve the SDK based on feedback. Add the natural language query feature as your AI contribution. This is where the project starts feeling genuinely differentiated.

Week eight, day seven. Your first real launch moment. Hacker News Show HN post. Twitter thread. Reddit post in developersIndia. Dev.to article. Not as a massive launch event. As a simple “I built this thing, here is what it does, try it if you work with Indian data.”

After launch. Build in public continuously. Add data sources based on demand. Respond to every GitHub issue. Write about your journey. Let the community tell you what to build next.

The AI natural language query layer is your genuine contribution to the AI industry. A domain specific AI system that understands Indian civic data, translates between English, Hindi, and structured queries, and provides contextual analysis of government statistics. That is a real and meaningful contribution. Not because it uses the latest model but because it solves a domain specific problem that general purpose AI tools handle poorly.

My Final Honest Position

Your mindset is good. Sonnet 4.6 is right about that. Building without financial pressure, for community and society, with willingness to let traction guide your decisions, is a genuinely healthy approach that produces better products than anxiety driven development.

But good mindset plus no validation equals a well intentioned project that nobody uses. Good mindset plus five real conversations before coding equals a well intentioned project that actually solves problems people have. The difference between those two outcomes is five conversations. That is it. Two hours of your time.

Do not let the comfort of a good mindset become a reason to skip the uncomfortable work of talking to strangers. The building is the fun part. The conversations are the important part. Do both.

And then ship it. Put it in the world. See what happens. Respond to what you learn. That is the process. It is not complicated. It is just hard to do consistently. But you have the right foundation for it.

Go build BharatAPI. But talk to five humans first.

BharatAPI — One Execution Strategy

Before Everything Else — The Five Conversations

This is non-negotiable. Before you open your code editor, before you create a GitHub repo, before you design a logo or pick a domain name, you talk to five real humans.

You are going to find two developers who have worked with Indian government data in the last year. You are going to find two journalists or researchers who have published anything using Indian statistics. You are going to find one person connected to the Indian open data community, ideally someone from DataMeet or a similar group.

You find them on LinkedIn by searching for terms like “data journalism India” or “data engineer civic tech” or “research analyst policy India.” You find them on Twitter by looking at who is posting about Indian open data, who is complaining about government PDF formats, who is sharing analysis based on NCRB or Census data. You find them by looking at bylines on IndiaSpend, Article 14, FactChecker.in, or EPW. You find them by checking who has contributed to DataMeet’s GitHub repositories.

You send each person a message that is short, respectful, and honest. Something like this. “Hi, I am a developer exploring the idea of building an open source SDK that makes Indian government data accessible through simple API calls. I am in the research phase and trying to understand how people currently work with this data. Would you have 15 minutes this week for a quick conversation? I am not selling anything, just trying to learn.”

Most people will not respond. That is normal. Send twenty messages to get five conversations. It takes two days of effort to send the messages and one week to complete the conversations.

In each conversation you ask four questions and then you shut up and listen.

How do you currently get Indian government data when you need it for your work?

What is the most painful or time consuming part of that process?

Have you tried any existing tools or services for this and what was your experience?

If a tool existed that gave you clean queryable government data through a simple API or web interface, what data sources would you want it to cover first?

You write down every answer. You do not pitch your idea. You do not describe your architecture. You do not show mockups. You listen. When they finish talking, you thank them and ask if they would be willing to try an early version when you have something built. Most will say yes. Now you have your first five beta testers.

After five conversations you will know three things you do not currently know. Which data sources people actually need most urgently, not which ones you assumed they need. What their current workarounds look like and how good or bad those workarounds are. Whether the pain is acute enough that people will actually change their behavior to use your tool.

This takes one week. It changes everything that follows.

Week Two and Three — Prove the Hard Thing

Based on your five conversations you now know which data source to start with. If your conversations did not reveal a clear winner, default to NCRB crime data because it has the highest combination of public interest, journalistic demand, and technical difficulty. Proving you can handle NCRB means you can handle almost anything else.

Go to the NCRB website. Download the most recent Crime in India annual report PDF. Open it. Look at the tables. Try to understand the structure. Notice how table headers sometimes span three rows. Notice how some tables continue across pages without repeating the header. Notice how footnotes are embedded inside data cells. Notice how state names are sometimes abbreviated and sometimes not. Notice how the format is slightly different from the previous year’s report.

Now build a parser for it. Not a perfect parser. A working parser that handles the most common table structures in that specific report. Use whatever tools work. PDF extraction libraries, OCR if needed for scanned pages, LLM assistance for ambiguous cases. Your goal is to extract at least ten major tables from the report into clean structured JSON with proper column names, state names normalized, and numeric values correctly parsed.

This will be harder than you expect. Significantly harder. That is the point. You need to experience this difficulty firsthand so you understand exactly what you are building and so you can write and speak about it authentically later.

While building the parser, document every problem you encounter. Take screenshots of particularly messy table layouts. Note which edge cases you solved and which ones you punted on. Write down the decisions you made, like how you handled merged cells or what you did when a footnote appeared inside a numeric field. This documentation becomes your first blog post and establishes your credibility in the space.

By the end of week three you should have one data source fully parsed, cleaned, normalized, and stored in a PostgreSQL database. The data should be queryable. State names should be consistent. District names should be mapped correctly. Years should be properly tagged. Data types should be correct. Numbers should be numbers, not strings.

If you cannot get this working reliably for one data source in two weeks, that is important information. It tells you the scope of the technical challenge and it forces you to adjust your overall timeline accordingly. Better to learn this now than after you have promised users a ten-dataset SDK.

Week Four — Build the SDK

Now you build the thing developers will actually install. A TypeScript package that wraps your database in a clean, well-typed API.

The SDK should do exactly one thing at this point. Let a developer query your cleaned dataset with simple function calls and get back structured, typed data. Nothing else. No AI. No natural language. No playground. Just clean data access.

The SDK needs to have complete TypeScript type definitions so that when a developer uses it in VS Code they get autocomplete showing exactly what fields are available, what the return types look like, and what parameters each function accepts. This is not optional polish. This is the core developer experience that determines whether anyone uses your SDK more than once.

The SDK needs exactly three code examples in the README. A minimal example showing installation and a single query. A slightly more complex example showing filtering and sorting. A real world example showing how someone might use this data in an actual application, like calculating crime rate trends across states.

The SDK needs a clear, honest README that says what data is available, where it comes from, when it was last updated, what its limitations are, and what is coming next. Do not oversell. If you have one dataset, say you have one dataset. Developers respect honesty about scope far more than they respect ambitious promises.

Publish it to npm. Push the code to GitHub with a proper MIT license. Make sure the repo has a clean structure, a contributing guide, and issue templates. First impressions matter on GitHub and developers judge project quality by repository hygiene.

This takes one week of focused work.

Week Five — First Content and Second Dataset

You now have a working SDK with one dataset published on npm and GitHub. This is the moment you start building in public.

Write your first blog post. The topic is not BharatAPI. The topic is the problem. Title it something like “What I Found When I Tried to Parse 10 Years of Indian Crime Data Programmatically.” Write about the actual challenges you faced. Show the messy PDF tables. Show your cleaned output. Share specific examples of inconsistencies you found in government data. Make it interesting to read even for someone who will never use your SDK.

Post it on Dev.to, Hashnode, and your personal site. Share it on Twitter with relevant screenshots or comparisons. Post it in the developersIndia subreddit. Share it in DataMeet channels if you have access.

This post serves three purposes. It establishes your credibility as someone who actually understands Indian government data challenges. It attracts the exact audience who might use your SDK. And it creates a piece of permanent content that will continue bringing people to your project through search for months and years.

Simultaneously, start building your second data source. Pick this based on what your user conversations revealed. If multiple people mentioned needing agricultural price data, build the Agmarknet parser. If people mentioned air quality, build the CPCB parser. If people mentioned economic indicators, build the RBI data pipeline. Let demand drive your choices, not your assumptions.

Week Six and Seven — AI Layer

This is where BharatAPI becomes something genuinely different from a simple data API.

Add a natural language query interface. A user should be able to type “which state had the highest murder rate in 2022” and get back the correct structured data. Behind the scenes you are using an LLM to parse the intent, identify the dataset, construct the query parameters, execute the query against your database, and return the results.

This is your AI contribution. Not a generic chatbot. A domain-specific AI system that understands Indian civic data terminology, knows the difference between IPC and SLL crimes, understands

that “UP” means Uttar Pradesh, knows that Telangana data only exists from 2014 onward, and can translate between casual human questions and precise data queries.

Add a data narration layer. When the system returns raw numbers, it should also generate a brief human-readable insight. Not a long essay. Two or three sentences that give context. Something like “Maharashtra reported the highest number of cyber crimes at 8,249 cases in 2022, a 23 percent increase from 2021. This is consistent with the state’s larger internet-using population and higher digital transaction volume.”

Build a simple web playground. A single page where someone can type a question, see the structured data response, read the AI-generated narrative, and download the results as CSV. This is critical because your journalist and researcher audience will not install npm packages. They need a browser interface.

The playground does not need to be beautiful. It needs to work reliably. A clean input box, a results table, a narrative paragraph, and a download button. That is enough.

Week Eight — Real Launch

You now have two clean datasets, a working SDK on npm, a natural language query interface, a web playground, one published blog post, and five beta testers from your initial conversations.

Reach out to your five beta testers first. Tell them the tool is ready. Ask them to try it for their actual work, not as a favor but as a real test. Watch what happens. Do they understand how to use it? Do they find the data they need? Do they encounter errors? What do they ask for that you did not build? This feedback is more valuable than anything that happens on launch day.

Then launch publicly. Not as a massive event. As a simple, honest announcement.

Post on Hacker News with a Show HN title. Something like “Show HN: BharatAPI — Open source SDK for querying Indian government data with AI.” Write a brief comment explaining what it does, why you built it, and what data is currently available. Be honest about the current scope.

Post a Twitter thread. Start with the problem. Show a screenshot of a messy government PDF next to your clean API output. Explain what the SDK does. Show a code snippet of installation and a simple query. Link to the playground for non-developers. Link to the GitHub repo.

Post on Reddit in developersIndia, programming, and webdev. Each post should be tailored to that community’s interests, not a copy paste across all three.

Write a Dev.to article that is longer and more detailed than your Twitter thread. Walk through the architecture. Show real examples. Explain your AI approach. This becomes a permanent reference that people find through search.

Post on LinkedIn with a different angle. Focus on the impact story. Why Indian public data matters. What problems better data access could solve. How this could help journalists, researchers, and policy makers. LinkedIn reaches a different audience than Twitter and Reddit and that audience includes potential consulting clients and enterprise users.

This is not a launch day. This is launch week. Space out your posts across five to seven days so each one gets individual attention rather than all competing with each other.

Month Three — Community and Second Content Wave

If your launch went well you should have some GitHub stars, some npm downloads, some playground usage, and some feedback. If it went poorly you should have specific information about what did not resonate and what needs to change.

Regardless of launch results, you now enter a sustained building phase. This is where most projects die because the excitement of launch fades and the daily work of maintenance, bug fixes, and incremental improvement feels unglamorous. This is where your mindset advantage matters. Because you are not chasing revenue metrics, you can focus on genuine quality improvement without panicking about growth numbers.

Add your third and fourth data sources based on what users are asking for. Check your GitHub issues. Look at playground query logs to see what questions people are asking that your system cannot answer. Those unanswered questions tell you exactly what data sources to add next.

Write your second and third blog posts. Topics that work well at this stage include deep dives into specific data findings that surprised you, technical posts about your normalization approach and entity resolution challenges, and analysis posts that use your own tool to answer interesting questions about Indian data. Each post brings new people to the project and establishes deeper credibility.

Start responding to and engaging with the Indian open data community more actively. Comment on DataMeet discussions. Respond to journalists on Twitter when they post about data challenges. Offer to help when someone publicly struggles with government data. Do not promote your tool. Just be helpful. The tool comes up naturally when it is relevant.

If anyone has used your data in a published article, research paper, or project, highlight it. With their permission, feature it on your website. These case studies are the most powerful form of social proof because they show real impact, not just theoretical potential.

Month Four to Six — AI Deepening and The Consulting Question

By month four you should have a working product with four to six data sources, an active GitHub repository, some community engagement, and hopefully a few real world use cases.

This is when you deepen the AI layer. Add the anomaly detection that spots unusual patterns in data and generates explanations. Add cross-dataset analysis capability so the research agent can answer questions that require combining information from multiple sources. Add support for Hindi language queries if your user base includes non-English speakers.

This is also when you start exploring consulting if you want to. Not as a pivot. As a natural extension of the expertise you have built. You now genuinely understand Indian government data systems better than almost anyone in the tech community. That expertise is valuable to organizations struggling with data quality and accessibility.

You do not need to formally start a consulting practice. Simply let it be known through your content and community presence that you are available for data-related projects. Write a blog post about the state of Indian government data accessibility. Mention specific challenges faced by different ministries and departments. When someone from a think tank or government body reaches out asking for help, have a conversation about what you could do for them.

One consulting engagement in months four through six changes everything. It gives you revenue to fund infrastructure costs as your data grows. It gives you deep relationships with organizations that become long term users and advocates. It gives

you inside knowledge about what government data challenges look like from the institutional side. And it gives you a case study that dramatically strengthens your credibility.

Month Six — The Assessment Point

This is the most important moment in your project’s life. At the six month mark you stop and honestly assess where you are.

You look at five specific indicators.

How many real projects or publications are using your data? If the answer is fewer than five, something about your data source selection, quality, or distribution needs to change.

How many developers are actively using the SDK on a weekly basis? Not stars. Not total installs. Weekly active usage. If this number is below twenty, your developer experience or data coverage needs improvement.

How many people are using the web playground regularly? If journalists and researchers are not coming back after their first visit, the natural language interface is not good enough or the data they need is not available.

Is anyone asking you for features you have not built? Demand pull is the strongest signal that your product has real value. If people are asking for more datasets, more query capabilities, or integration options, you are on the right track.

Has anyone spontaneously recommended your tool to someone else? Word of mouth is the ultimate validation. If even one person has told a colleague “you should try BharatAPI for that,” your product has crossed the threshold from interesting project to genuinely useful tool.

Based on these indicators you make one of three decisions. If most indicators are positive, you expand scope and start thinking about sustainability through consulting or sponsorships or eventually a paid tier. If indicators are mixed, you focus on improving the weak areas before expanding. If most indicators are negative, you have an honest conversation with yourself about whether the approach needs to fundamentally change or whether this is not the right project for you right now.

Month Six Onward — Let Reality Guide You

From this point forward your strategy is not predetermined. It is guided by what you have learned in the first six months. The beauty of your financial independence is that you do not need to force growth. You can let the project evolve naturally based on real usage patterns and real feedback.

If the developer community has adopted the SDK, you focus on expanding data sources, improving reliability, and making the SDK the default way anyone accesses Indian government data programmatically.

If journalists and researchers are the primary users, you focus on the web interface, the AI narrative generation, and making the tool accessible to non-technical users.

If consulting opportunities have emerged, you balance product development with consulting work, using each to strengthen the other.

If the AI community is interested in your work, you focus on publishing your normalized datasets openly, writing about your AI approaches, and contributing to conversations about domain-specific AI systems.

The path reveals itself through doing, not through planning. Your first six months of planning and building create the foundation. Everything after that is responsive to reality.

The Things That Must Be True Throughout

Across all of these phases, three principles stay constant.

Data quality is sacred. Every number in your system must be traceable to its government source. Every dataset must clearly state when it was last updated and what its limitations are. If you are not confident in a data point, do not serve it. One wrong number cited in a published article destroys your reputation permanently. Quality is not a feature. It is the foundation everything else stands on.

Transparency is your brand. Your scraping methodology, your normalization decisions, your data sources, your update frequency, your known limitations, all of it should be documented and public. When you make a mistake, which you will,

acknowledge it publicly and fix it. The open data community values transparency above everything else. Being open about your process builds more trust than having a perfect product.

Consistency beats intensity. Publishing one blog post per month for twelve months is more valuable than publishing five posts in week one and then going silent. Responding to GitHub issues within 48 hours every time is more valuable than responding instantly sometimes and ignoring them other times. Adding one well-tested dataset per month is more valuable than adding five in a rush. Sustainable consistent effort over twelve months produces better results than intense effort for three months followed by burnout.

What Success Looks Like After One Year

If you execute this strategy with discipline and consistency, twelve months from now you will have a working open source SDK covering eight to twelve Indian government data sources. You will have a web playground that journalists and researchers actually use. You will have an AI query layer that understands Indian civic data better than any general purpose tool. You will have a GitHub repository with real community engagement. You will have a body of technical content that establishes you as a domain expert. You will have real world citations and integrations that prove your tool delivers genuine value.

Whether that turns into a business, a career opportunity, a community leadership position, or simply a project you are proud of depends on factors you cannot predict today. And that is fine. Because the work itself is worth doing regardless of which outcome materializes.

But it all starts with five conversations this week. Go have them.

My Honest Opinion on This Strategy and Skipping Conversations

The strategy document is solid. It is the most grounded and executable plan that has come out of this entire conversation. The phase structure is clean. The exit conditions are specific. The principle of finishing one phase before starting the next is correct and important.

But you want to skip Phase 1, the fifteen conversations. Let me be completely honest about what that means, why I partially understand your reasoning, and what you should do instead.

Why You Want to Skip It and Why That Is Partially Valid

I can guess why you want to skip conversations. You are a student. Cold messaging journalists at IndiaSpend or researchers at IITs feels awkward and intimidating. You have no track record to reference. You are worried people will ignore you or think you are wasting their time. The whole process feels like a barrier between you and the thing you actually want to do which is build.

Here is the part where you are not entirely wrong. The problem you are solving is not a hypothesis that needs validation through interviews. Nobody needs to confirm that Indian government PDFs are a mess. Nobody needs to tell you that NCRB data is hard to extract. That pain is publicly documented everywhere. Journalists complain about it on Twitter. Researchers write about it in methodology sections of their papers. Developers rant about it on Reddit. DataMeet has entire GitHub repositories dedicated to cleaning government data that prove the problem exists.

This is fundamentally different from someone building a new social app who needs to validate whether anyone wants yet another feed. Your problem is proven. The pain is documented. The gap is visible.

Many successful open source tools were never validated through formal user interviews before the first version shipped. The creator of SQLite did not interview fifteen database users before building it. The people behind cURL did not conduct user research. They had a problem, they built a solution, they shipped

it, and they iterated based on what happened next. For open source infrastructure tools, shipping fast and iterating on real feedback is a legitimate validation path.

So I am not going to tell you that your project will fail if you skip formal interviews. It will not necessarily fail.

What You Actually Lose By Skipping It

But I want you to understand clearly what you are trading away.

You lose dataset prioritization insight. Right now you are guessing that NCRB crime data is the right first dataset. That guess is probably reasonable but it is still a guess. Maybe every developer you would have talked to would have said “I actually need real time mandi prices from Agmarknet way more than I need crime data.” Maybe every journalist would have said “CPCB air quality data is what I spend the most time cleaning because I need it for monthly pollution stories not annual crime stories.” Without those conversations you are choosing based on your own intuition which may or may not match what the market actually needs most urgently.

You lose early advocates. The fifteen people you would have talked to become your first users, your first feedback givers, your first sharers. They feel invested because you asked for their input before building. When you launch, they show up because they were part of the journey. Without those conversations your launch is a cold start. You are posting on Reddit and Twitter hoping strangers care. That works sometimes but it is harder and less reliable than having fifteen people already waiting for what you built.

You lose language. The specific words people use to describe their pain become your marketing copy, your README introduction, your blog post hooks. “I spent three weeks just cleaning the NCRB PDF before I could start my actual analysis” is a quote from a real person that resonates with other people who have felt the same frustration. Without conversations you write your own copy which will be less compelling because it comes from your imagination instead of someone else’s lived experience.

What I Actually Recommend Instead of Skipping Entirely

Do not do fifteen formal conversations. But do not do zero validation either. There is a middle path that gives you eighty percent of the insight with twenty percent of the effort and none of the awkwardness of cold outreach.

Spend two days doing passive validation research. Not talking to anyone. Just reading.

Go to Twitter and search for phrases like “NCRB data” or “government data India” or “data.gov.in” or “Indian open data.” Read what people are saying. Screenshot the complaints. Note which datasets people mention most often. Note which pain points come up repeatedly.

Go to the developersIndia subreddit and search for posts about government data or civic tech or data science projects using Indian data. Read the comments. See what people struggled with. See what they wished existed.

Go to DataMeet’s GitHub organization. Look at their repositories. See which government datasets they have tried to clean. Read their issues and discussions. See where they got stuck. See what they gave up on.

Go to Google Scholar and search for recent papers that cite Indian government statistics. Read the methodology sections. See how researchers describe their data collection process. Note the pain they mention. Note which datasets appear most frequently in academic work.

Go to IndiaSpend and Article 14 and look at their data journalism pieces. See which government data sources they use most often. See if they have written anything about the difficulty of accessing that data.

This takes two days. No cold messages. No awkward conversations. No waiting for people to respond. Just you, a browser, and a notebook. At the end of two days you will have a much clearer picture of which dataset to start with, what pain points are most acute, and what language people use to describe the problem.

Then start building. And here is the critical addition. When you launch in Phase 3, you treat that launch as your real validation moment. You watch what happens. You read every comment. You respond to every GitHub issue. You pay attention to which features people ask for and which they ignore. You notice who

uses the playground and what queries they type. That real world usage data becomes your ongoing user research, replacing the conversations you skipped.

This approach works. But it only works if you ship fast enough to get real feedback while your motivation is high, and if you actually pay attention to that feedback instead of just building what you already planned.

My Validation of Each Phase

Let me go through the strategy phase by phase and tell you what is right, what needs adjustment, and what is missing.

Phase 0 is correct but should be shortened to three days not a full week. Pick NCRB crime data as your first dataset. Your quality bar should be written in a single paragraph not a lengthy document. Make your first public post on Twitter or LinkedIn saying you are starting this project and why. Do the two days of passive research I described above. Phase 0 should end by day three. Do not let it become a planning trap where you spend weeks preparing to prepare.

Phase 2 is the core of your project and the strategy describes it well. The build order of ingester then parser then normalizer then storage then SDK then playground is exactly right. The emphasis on fault tolerance and confidence scoring in the parser is exactly right. The advice to keep the SDK to five functions maximum initially is exactly right.

One thing to add to Phase 2. Build the normalizer reference file as a separate publicly accessible JSON file in your repo from day one. This file that maps “AP” to “Andhra Pradesh” and handles district name variations will become one of the most valuable artifacts of your entire project. Other developers will use it independently of your SDK. Researchers will reference it. It will get contributions from the community. Treat it as a first class product, not a utility buried in your code.

One more addition to Phase 2. Set a hard time limit. Give yourself four weeks maximum for Phase 2, not an open ended timeline. If the NCRB parser is not perfect after four weeks, ship it anyway with clear documentation about what works and what does not work yet. A shipped imperfect parser that says “these 15 tables are reliably parsed, these 5 are still in progress” is infinitely more valuable than an unshipped perfect parser that nobody knows exists.

Phase 3 is where the strategy needs the most adjustment given that you are skipping formal conversations.

Since you will not have fifteen pre-existing relationships to leverage at launch, your content strategy becomes even more important. The blog post about your experience parsing NCRB data is not optional. It is your single most important distribution asset. Write it well. Make it genuinely interesting. Include specific examples with screenshots of messy PDF tables next to your clean output. Make a reader think “this person really understands this problem” not “this person is promoting their product.”

The exit condition of fifty organic users is ambitious for a first launch without pre-existing relationships. Adjust it to twenty-five real users in the first month. If you hit twenty-five people who found you organically and actually used the SDK or playground, that is strong signal. If you cannot reach twenty-five in a month, something about your content distribution or product experience needs to change before you move forward.

Add one specific action to Phase 3 that is missing. Create a GitHub Discussions section in your repo and actively seed it with interesting questions. Things like “Which dataset should we add next” or “How should we handle the Telangana bifurcation in historical crime data.” These discussions attract community participation and make your repo feel alive and collaborative rather than a solo project.

Phase 4 is correct in principle. The AI layer build order of natural language queries then data narration then anomaly detection is the right sequence. The advice to add the second dataset based on user demand not your own curiosity is exactly right.

One addition. When you build the natural language query layer, keep a log of every query that fails or produces wrong results. This failure log becomes your most important AI improvement tool. Review it weekly. Each failed query teaches you something about how people actually phrase questions about Indian data that you would never have anticipated.

Phase 5 is correct and I have nothing to add. The advice to stop directing and start responding is the right posture for a mature open source project.

What Is Missing From the Entire Strategy

There are three things not covered in any phase that you need to think about.

Data freshness and maintenance burden. The strategy talks about building the ingester and parser but does not address what happens three months after launch when the NCRB publishes a new report in a slightly different format and your parser breaks. Or when data.gov.in changes their URL structure. Or when a government website goes down for two weeks during a ministry reorganization.

You need a monitoring system from day one. Not something complex. A simple health check that runs daily, tries to access each data source, and alerts you if something has changed or broken. A GitHub Action that runs your parser against the latest source and opens an issue automatically if the output does not match expected structure. This is boring infrastructure work but it is what separates a project that stays alive from one that silently rots.

Plan to spend twenty percent of your ongoing time on maintenance, not new features. This is the tax you pay for building on top of sources you do not control. Accept it from the start.

Legal minimum. You do not need a lawyer on day one. But you do need three things. A clear attribution line on every data point that credits the original government source. A terms of use page on your website that says your data is aggregated from public government sources and should be verified against primary sources for critical use cases. And rate limiting on your scrapers that ensures you never make more than one request per second to any government website. These three things cost you nothing and protect you from the most likely legal and reputational risks.

Your own skill development tracking. You said one of your goals is showcasing skills for job opportunities. The strategy does not address how the project demonstrates specific skills that employers care about. Think about this consciously. Your data pipeline demonstrates backend engineering and data engineering skills. Your SDK design demonstrates API design and developer experience thinking. Your AI layer demonstrates practical LLM application skills. Your content and community building demonstrates communication skills. When you eventually put this on your resume or portfolio, frame it in terms of these specific skills with specific evidence. Not “I built BharatAPI” but “I designed and built a TypeScript SDK that processes and

normalizes Indian government data from PDF sources, handling entity resolution across 780 districts and serving structured data through a type-safe API used by X developers.”

My Final Take on the Strategy

The strategy is good. It is specific enough to be actionable and flexible enough to adapt as you learn. The single rule of finishing each phase before starting the next is the most important principle in the entire document and if you follow nothing else, follow that.

The decision to skip formal user conversations is a calculated risk that I think is acceptable given your situation, but only if you replace it with the passive research I described and only if you ship fast enough that real user feedback replaces the conversations you did not have. If you skip conversations and also take six months to ship, you get the worst of both worlds. No validation and no product. Speed becomes even more important when you skip upfront research.

My concrete recommendation for your immediate next steps.

Today, create the GitHub repository. Write the README with a clear description of what BharatAPI will be and what it is not yet. Make your first public post announcing what you are building and why.

Tomorrow and the day after, do the passive research. Two days reading Twitter, Reddit, DataMeet, academic papers, and data journalism sites. Write down what you learn about which datasets people need most and what specific pain points they describe.

Day four, download the NCRB Crime in India PDF and start building the parser. You are now in Phase 2. Build with intensity and focus. Ship in four weeks.

That is the plan. It is good enough. Go execute it.

My Honest Opinion on This Roadmap

The structure is clean. The milestone format is clear. The phase progression is logical. This is a well-organized document. But there are six specific problems I see that will cause you real trouble during execution if you do not address them now.

Problem One — Phase 1 Contradicts What You Told Me

You explicitly said you want to skip the conversation part. But this roadmap includes Phase 1 with fifteen conversations as a hard dependency for Phase 2. That means according to your own roadmap you cannot start building until all fifteen conversations are done.

You will not do those fifteen conversations. I know this because you already told me you do not want to. And a roadmap you will not follow is worse than no roadmap because it creates guilt and paralysis. Every day you spend not doing Phase 1 you will also not start Phase 2 because your own document says Phase 2 depends on Phase 1 being complete.

Here is what to do. Replace Phase 1 entirely with what we already agreed on. Two days of passive research. Reading Twitter, Reddit, DataMeet GitHub repos, academic papers, IndiaSpend articles. No conversations. No cold messages. Just reading and noting what datasets people complain about most and what language they use to describe the pain. Then confirm your dataset choice and move straight into Phase 2.

The roadmap should reflect what you will actually do, not what sounds thorough on paper.

Problem Two — No Timelines Anywhere

This roadmap has twenty-nine milestones across six phases but not a single date or duration attached to any of them. Without time boundaries, phases expand to fill whatever time you have. Phase 2 especially will eat months of your life if you do not put a deadline on it because parser perfection is an infinite rabbit hole.

Attach specific durations to each phase.

Phase 0 should take three days. Not a week. Three days. Pick dataset, write quality bar, make first public post, create GitHub repo. Done.

Phase 1, which you are replacing with passive research, should take two days.

Phase 2 should take four weeks maximum with a hard deadline. If the parser is not perfect after four weeks, ship it anyway with clear documentation about what works and what is still in progress. A shipped imperfect parser is infinitely more valuable than an unshipped perfect one.

Phase 3 should take four weeks of active distribution work after Phase 2 ships.

Phase 4 should take three to four weeks.

Phase 5 is ongoing and has no end date, which is correct.

Write these durations directly into your roadmap. Then put the actual calendar dates next to them based on when you plan to start. A milestone that says “Build PDF parser, deadline July 15” is ten times more actionable than “Build PDF parser, not started.”

Problem Three — No Failure Conditions

Every phase has an exit condition for success but none of them say what happens if you fail to meet that condition. What happens if you build the parser and it only reliably handles eight out of twenty tables? What happens if you launch and get twelve organic users instead of fifty? What happens if nobody opens a single GitHub issue?

Without failure conditions you will either keep pushing toward an unrealistic goal until you burn out or you will quietly lower the bar without acknowledging it. Both are bad.

Add a simple failure protocol to each phase. If Phase 2 parser handles fewer than ten tables reliably after four weeks, ship what works and document what does not. If Phase 3 produces fewer than twenty-five organic users after four weeks, write a second blog post with a different angle and try different distribution channels before moving to Phase 4. If Phase 4 AI layer produces wrong answers more than fifteen percent of the time, do not ship it until accuracy improves.

These do not need to be elaborate. One sentence per phase. But they force you to think about what “not working” looks like before you are emotionally invested in the outcome.

Problem Four — Three Critical Things Are Missing Entirely

The roadmap covers building and distribution but completely ignores three things that will bite you during execution.

First, data freshness monitoring. Your roadmap describes building the ingester and parser but says nothing about what happens after they are built. Government websites change. PDFs move. URLs break. You need a health check system from day one. A simple GitHub Action that runs daily, checks if your data sources are still accessible, and opens an issue automatically if something breaks. Add this as a milestone inside Phase 2. It takes half a day to build and it prevents the silent data rot that kills data projects.

Second, legal minimum. Not a full legal strategy. Three specific things. An attribution line on every data point crediting the original government source. A terms of use page on your website stating that data is aggregated from public government sources and should be verified against primary sources for critical decisions. Rate limiting on your scrapers to never exceed one request per second to any government server. Add these as a milestone inside Phase 2. They take a few hours and protect you from the most likely problems.

Third, your portfolio narrative. You said one of your goals is showcasing skills for job opportunities. But nowhere in this roadmap do you plan when and how you will document what you built in portfolio-ready format. Add a milestone after Phase 3 or Phase 4 where you write a proper case study of your own project. Not the README. A separate document or blog post that says “I designed and built a data pipeline that processes NCRB PDF reports into structured queryable data, handling entity resolution across 780 districts, serving it through a type-safe TypeScript SDK used by X developers.” That framing is what gets interviews.

Problem Five — Phase 3 Exit Condition Is Unrealistic Without Phase 1

You removed conversations but kept the Phase 3 exit condition of fifty organic users. The original strategy assumed you would have fifteen warm contacts who would use the product and share it with their networks. Without those fifteen people, reaching fifty organic users purely through content and community posts is significantly harder.

Lower the Phase 3 exit condition to twenty-five real users. Twenty-five people who found your SDK or playground organically, used it at least once, and are not your personal friends. If you hit twenty-five, that is genuine traction for a solo open source project with no pre-existing audience. If you hit fifty, celebrate. But do not make fifty the gate that blocks you from starting Phase 4.

Also add one specific milestone to Phase 3 that compensates for the missing warm contacts. Find three to five developers on Twitter or Reddit who have publicly complained about Indian government data access in the last six months. Reply to their old posts or DM them with your tool. These are not cold contacts. These are people who have already expressed the exact pain you are solving. They are far more likely to try your tool than random people who see a Show HN post.

Problem Six — Product Hunt Is Too Late

The roadmap puts Product Hunt in Phase 5 after the AI layer is complete. That means your first major public launch happens four to five months into the project. That is too late.

Product Hunt and Hacker News launches are most effective when you are early and scrappy, not when you are polished and complete. The developer community on these platforms loves discovering projects in their early stages and watching them grow. A launch that says “I built a clean API for NCRB crime data and I am expanding to more datasets” will get more genuine engagement than a launch that says “here is a complete platform with AI and five datasets” because the first version invites participation and the second version just asks for consumption.

Move your Hacker News Show HN post to the end of Phase 3, not Phase 5. Keep Product Hunt for Phase 5 if you want a second launch moment. But get on Hacker News early while the project is young enough that the community feels like they are discovering something, not being marketed to.

The Revised Roadmap I Would Actually Follow

Phase 0 takes three days. Pick NCRB. Write quality bar. Create GitHub repo. Make first public post.

Phase 1 takes two days. Passive research only. Read Twitter, Reddit, DataMeet, academic papers. Confirm dataset choice. No conversations.

Phase 2 takes four weeks with a hard deadline. Build ingester, parser, normalizer, database, SDK, playground. Add health monitoring. Add legal basics. Ship on npm. Deploy playground. The SDK works or it does not by the end of week four.

Phase 3 takes four weeks. Contact five people who publicly complained about Indian data. Write the problem-first blog post. Post in communities. Submit Show HN. Reach for twenty-five organic users. Write portfolio case study.

Phase 4 takes three to four weeks. Build natural language queries, data narration, anomaly detection. Add second dataset based on user demand. Write technical blog post about AI approach.

Phase 5 is ongoing. Product Hunt launch. Identify strongest user segment. Double down. Explore first money conversation when the signal is clear.

Total timeline from start to Phase 4 complete is roughly three and a half to four months if you are working consistently. That is honest and achievable.

My Bottom Line

This roadmap is eighty percent of the way there. The structure is good. The phase logic is sound. The milestone format is clear. But it needs timelines, failure conditions, the three missing elements I described, a realistic Phase 1 replacement, an adjusted Phase 3 target, and an earlier Hacker News launch.

Make those six changes and you have a roadmap you can actually follow from start to finish without getting stuck, burning out, or silently abandoning milestones that do not fit reality.

Then stop refining the roadmap and start Phase 0. Today. The roadmap is a tool, not the product. The product is BharatAPI. Go build it.

Setu + Data Identity — Revised Name Suggestions

You are right. Setu alone makes people ask “what is this?” You need someone to hear the name and immediately think “this is about data” without needing an explanation. The Indian identity should be the flavor, not the barrier. Let me rethink this with that constraint.

My New Top Pick — DataSetu

I know I mentioned this earlier with some hesitation but after thinking about it again with your specific concern in mind, DataSetu is actually the strongest choice. Here is why I changed my position.

The word Data is universally understood by every developer, journalist, and researcher on the planet. There is zero ambiguity about what this product relates to. The word Setu adds the Indian identity and the bridge metaphor. Together they say exactly what the product is. A bridge to data. Indian data specifically.

It is eight letters which is slightly longer than ideal but still completely workable. `npm install datasetu` reads naturally. `datasetu.dev` or `datasetu.io` as a domain is clean. In code it looks good: `import { DataSetu } from 'datasetu'`. In conversation it flows: “I used DataSetu to pull crime statistics.” In a blog post title it works: “DataSetu: Bridging the Gap in Indian Public Data.” On a resume it communicates instantly what the project does without needing further explanation.

The word order matters. DataSetu is better than SetuData because English speakers naturally put the modifier first. Data is what it is about. Setu is how it works. DataSetu. Data bridge. Done.

Strong Alternatives With Data Clarity

DataKosh combines Data with Kosh which means treasury or repository in Sanskrit. The meaning is “treasury of data” which perfectly describes what you are building. A curated, organized, valuable collection of Indian public data. It is eight letters, same as DataSetu. `npm install datakosh` works well. The brand angle is

“India’s data treasury.” It sounds premium and implies that the data inside is valuable and well-organized, not raw and messy. This is my second choice.

DataDhara combines Data with Dhara which means flow or stream in Sanskrit. The concept is a continuous flowing stream of clean data. It connects beautifully to your real-time pipeline that continuously ingests and updates government data. Nine letters is slightly long but still manageable. `npm install datadhara` reads well. The brand angle is “a continuous stream of clean Indian data.” The flowing water metaphor also implies cleansing which is exactly what your normalizer does to messy government data.

DataNetra combines Data with Netra which means eye in Sanskrit. The meaning is “the eye on data” or “seeing through data.” It connects to the AI insight layer of your product because netra implies not just seeing data but understanding it. Nine letters. `npm install datanetra` works. The brand angle is “see clearly through Indian public data.”

DataVani combines Data with Vani which means voice or speech in Sanskrit. This connects directly to the natural language query feature because vani implies speaking to your data and having it speak back. Eight letters. `npm install datavani` is clean. The brand angle is “give voice to India’s data.” This is particularly strong if the AI conversational layer becomes your primary differentiator.

A Different Approach — Indian Word That Already Means Data

Ankura means data point or digit in Sanskrit. It also means sprout or beginning which gives it a secondary meaning of growth and new beginnings. Six letters, distinctive, pronounceable globally. `npm install ankura` is clean. The brand angle is “Indian data, from the ground up.” The risk is that the word is not widely known so you still need to explain it, which is the problem you had with Setu alone.

Sankhya means number or data in Sanskrit. It is also the name of one of the oldest schools of Indian philosophy which gives it intellectual weight. Seven letters. `npm install sankhya` works well. The brand angle is “India by the numbers.” The risk is pronunciation difficulty for non-Indian developers.

Names I Considered But Rejected

DataSutra sounds interesting because sutra means thread or formula, but the word sutra has acquired connotations in Western culture that would distract from your product entirely. Avoid this.

DataYantra where yantra means machine or instrument sounds technical but yantra is eleven letters total and feels heavy to type repeatedly.

DataPravah where pravah means flow is too similar to DataDhara but less distinctive and harder to pronounce for non-Hindi speakers.

JanData where jan means people was in my earlier list but it sounds like an NGO rather than a developer tool which works against your goal of being taken seriously as a technical product.

Quick Comparison

Name	Letters	Meaning	Clarity	Global Pronunciation	Brand Feel
DataSetu	8	Data Bridge	Instant	Easy	Technical + Indian
DataKosh	8	Data Treasury	Instant	Easy	Premium + Curated
DataDhara	9	Data Stream	Instant	Medium	Flowing + Clean
DataNetra	9	Data Eye	Instant	Medium	Insightful + AI
DataVani	8	Data Voice	Instant	Easy	Conversational + AI
Ankura	6	Data Point	Low	Easy	Fresh + Minimal
Sankhya	7	Numbers	Medium	Hard	Academic + Deep

My Final Recommendation

Go with **DataSetu**.

It solves your exact concern. The word Data makes the purpose instantly clear to any person on the planet. The word Setu adds Indian identity and a meaningful metaphor. Together they form a name that needs no explanation. A developer sees DataSetu and

immediately understands “this is a data tool with Indian roots.” A journalist sees DataSetu and thinks “this is about accessing data.” A recruiter sees DataSetu on your resume and understands what you built without clicking through.

Check availability today for `datasetu.dev` or `datasetu.io` as your domain. Check npm for the package name `datasetu`. If the exact package name is taken use `@datasetu/core` as a scoped package.

Pick the name. Buy the domain. Move to Phase 0. The name is decided. Now build the thing behind it.

DataSetu — Professional Documentation Index

Why This Matters

A well-documented project communicates three things before anyone reads a single line of your code. This person is serious. This person thinks clearly. This person builds things that last. Every recruiter, contributor, user, and potential collaborator will judge your project by the quality and completeness of its documentation before they ever look at your code. These documents are not bureaucracy. They are the difference between a side project and a professional product.

Complete Document List

Doc No.	Document Name	Purpose	Audience
00	Execution Strategy	Internal roadmap with phases, milestones, timelines, failure protocols	You (private)
01	Project Overview & Vision	What DataSetu is, why it exists, where it is going	Everyone

Doc No.	Document Name	Purpose	Audience
02	Product Requirements Document	What exactly you are building in v1 with scope boundaries	You + Contributors
03	Technical Architecture Document	System design, data flow, infrastructure decisions	You + Contributors + Recruiters
04	Data Source Registry	Every data source catalogued with status, format, update frequency, legal standing	You + Users + Contributors
05	SDK Design Specification	API surface design, function signatures, type definitions, naming conventions	You + Contributors
06	Data Schema & Normalization Guide	How raw data becomes clean data, canonical mappings, entity resolution rules	You + Contributors + Researchers
07	AI Layer Specification	How GenAI and Agentic AI components work, prompt strategies, guardrails	You + Contributors + AI Community
08	Brand & Identity Guidelines	Name, logo, colors, voice, tone, how to talk about DataSetu	You + Contributors + Media
09	README	First thing anyone sees on GitHub,	Everyone

Doc No.	Document Name	Purpose	Audience
		installation to first query in 2 minutes	
10	Contributing Guide	How to contribute code, data mappings, bug reports, feature requests	Contributors
11	Code of Conduct	Community behavior standards	Contributors + Community
12	License Document	Legal terms for using and contributing to the project	Everyone
13	Security Policy	How to report vulnerabilities, what security measures exist	Users + Security Researchers
14	Terms of Use	Legal terms for using the API and data	Users
15	Privacy Policy	What data you collect from users, how you handle it	Users
16	Data Attribution & Legal Compliance	Source attribution standards, government data licensing, scraping policies	You + Users + Legal
17	Changelog	Version history with what changed, what broke, what was added	Users + Contributors
18	Public Roadmap	User-facing version of where the	Users + Community

Doc No.	Document Name	Purpose	Audience
		project is headed	
19	Content & Distribution Strategy	Blog post plan, community engagement plan, launch checklist	You (private)
20	Portfolio Case Study	Your personal narrative of building DataSetu for job applications	Recruiters + You

---

Document Details

Document 00 — Execution Strategy

Status: Complete
Audience: You (private, not published)
Format: Internal document

This is already done. It contains your phased roadmap, milestones, timelines, exit conditions, and failure protocols. This document stays private. It is your operating manual. Everything else flows from it.

Document 01 — Project Overview & Vision

Audience: Everyone — developers, journalists, researchers, recruiters, potential contributors, media
Format: Markdown file in repo root or dedicated docs site page
Length: 2-3 pages maximum

This document answers five questions and nothing else.

What is DataSetu. One paragraph. A unified open source SDK and API that makes Indian government public data accessible, clean, and queryable. No buzzwords. No hype. Just clarity.

Why does DataSetu exist. Two paragraphs. The problem stated concretely with specific examples of the pain. Government data locked in PDFs. Inconsistent formats across sources. No unified API. Weeks of cleaning before any analysis can begin.

Who is DataSetu for. One paragraph listing the primary audiences. Developers building applications with Indian data. Journalists and researchers who need clean statistics. Students and hackathon participants who need real datasets.

What does DataSetu do today. One paragraph listing current capabilities honestly. Which datasets are available. What the SDK can do. What the playground offers. No promises about future features in this section.

Where is DataSetu going. One paragraph about the direction. More datasets driven by community demand. AI-powered natural language queries. Anomaly detection. Always honest about what is built versus what is planned.

Document 02 — Product Requirements Document

Audience: You and contributors
Format: Markdown in a `/docs/internal/` folder or a Notion page
Length: 3-5 pages

This document defines exactly what you are building in version 1 and more importantly what you are not building. It prevents scope creep which is the number one killer of solo developer projects.

It should contain the following sections.

v1 Scope — Exact list of features included in the first public release. Which datasets. Which SDK functions. What the playground does. What the API endpoints are. Be exhaustive and specific.

Explicitly Out of Scope for v1 — Features you are intentionally not building yet. AI layer. Multiple datasets. User accounts. Paid tiers. Authentication. Write these down so you do not get tempted to add them mid-build.

User Stories — Five to eight specific scenarios written as “As a [type of user], I want to [action], so that [outcome].” These keep you focused on real use cases not abstract features.

Acceptance Criteria — How you know each feature is done. Not “parser works” but “parser extracts all table data from NCRB Chapter 2 with >95% accuracy verified against manual spot checks of 20 random cells.”

Technical Constraints — Free tier infrastructure only. Single developer. Must work offline for SDK users. Response time under 2 seconds for API calls.

Dependencies & Risks — Government website might change format. PDF parser might not handle all table types. Free tier database might hit storage limits.

Document 03 — Technical Architecture Document

Audience: You, contributors, and recruiters
Format: Markdown with diagrams in `/docs/architecture/`
Length: 4-6 pages with diagrams

This is one of the most important documents for your career goal. A well-written architecture document demonstrates systems thinking which is the skill that separates senior engineers from junior ones.

It should contain the following sections.

System Overview — A high level diagram showing all components and how data flows between them. Raw source to ingester to parser to normalizer to database to API to SDK to user. One diagram that tells the whole story.

Component Details — Each component described in one to two paragraphs. What it does. Why you chose this approach over alternatives. What technology it uses and why.

Data Flow — Step by step walkthrough of what happens when raw data enters the system and what happens when a user makes a query. Two separate flows documented clearly.

Infrastructure — What runs where. Database on Neon. API on Vercel or Cloudflare Workers. Cron jobs on GitHub Actions. Why each choice was made. What the free tier limits are. What happens when you hit those limits.

Error Handling Strategy — What happens when a scraper fails. What happens when a parser encounters an unexpected format. What happens when the database is unreachable. How errors are logged and surfaced.

Security Considerations — How API keys are managed. How rate limiting works. How you prevent abuse of the free playground.

Future Scalability — How the architecture supports adding new datasets without restructuring. How the system would scale if usage grows beyond free tiers.

Document 04 — Data Source Registry

Audience: You, users, contributors, researchers
Format: Markdown table in `/docs/data-sources/` with one detailed page per source
Length: One summary table plus one page per data source

This is the document that researchers and journalists will reference most. It tells them exactly where every number in your system comes from.

The summary table should list every data source with these columns. Source name. Source URL. Data format (PDF, CSV, HTML, API). Update frequency (annual, monthly, real-time). Date of last ingestion. Coverage period (what years are available). Legal status (open license, public access, terms unclear). Current status (active, planned, deprecated).

For each active data source, create a dedicated page containing the following. Full name and description. Direct URL to the original data. What government body publishes it. What license or terms apply. What fields are extracted. Known data quality issues (missing years, inconsistent formatting, known errors in source). How your normalizer handles edge cases specific to this source. Sample of raw versus cleaned data showing the transformation.

Document 05 — SDK Design Specification

Audience: You, contributors, and advanced users
Format: Markdown in `/docs/sdk-design/`
Length: 3-4 pages

This document defines your API surface before you build it. It is the contract between your SDK and every developer who uses it. Changing it later breaks their code so getting it right upfront matters.

It should contain the following sections.

Design Principles — The rules that govern every API decision. Consistency: similar operations should have similar signatures. Predictability: a developer should guess correctly how a function works before reading docs. Minimalism: do not add a function unless multiple users need it. Type safety: every input and output is fully typed.

Naming Conventions — How you name functions, parameters, and return types. Camel case versus snake case. How dataset modules are named. How filter parameters are structured.

Function Signatures — Every public function listed with its name, parameters, return type, and a one-line description. This is not full documentation. It is the shape of the API.

Error Handling Contract — How errors are returned. Custom error types. Error codes. What the user can expect when something goes wrong.

Versioning Strategy — How you version the SDK. Semantic versioning rules. What constitutes a breaking change. How deprecations are communicated.

Document 06 — Data Schema & Normalization Guide

Audience: You, contributors, researchers who need to understand your data transformations
Format: Markdown in `/docs/data-schema/` with the canonical JSON reference file
Length: 2-3 pages plus the canonical mappings file

This document explains how raw messy government data becomes clean structured DataSetu data. It is the intellectual core of your project and the part that is hardest for competitors to replicate.

It should contain the following sections.

Canonical Entity Mappings — Reference to your `canonical-mappings.json` file. Explanation of how state name variations are resolved. How district name variations are handled. How historical boundary changes (Telangana bifurcation, new districts, Ladakh separation) are mapped.

Data Type Standards — How numbers are stored (always integers or floats, never strings). How dates are formatted (ISO 8601). How null or missing values are represented. How Indian number formatting (lakhs, crores) is converted to standard notation.

Confidence Scoring — How confidence scores are assigned. What a score of 1.0 means versus 0.7 versus 0.3. When a data point is flagged for manual review versus automatically included.

Normalization Pipeline — Step by step explanation of what transformations are applied to raw data and in what order. Field mapping. Type conversion. Name normalization. Deduplication. Confidence scoring. Source attribution attachment.

Document 07 — AI Layer Specification

Audience: You, contributors, AI community
Format: Markdown in `/docs/ai-layer/`
Length: 3-4 pages (write this during Phase 4, not before)

This document is written during Phase 4 when you actually build the AI components. Do not write it in advance because your AI approach will change based on what you learn during Phases 2 and 3.

It should contain the following sections when written.

Natural Language Query Engine — How user questions are parsed. What LLM is used and why. How the prompt is structured. How structured queries are generated from natural language. How ambiguous queries are handled. Examples of queries that work well and queries that fail.

Data Narration — How raw data is transformed into human-readable insights. What prompt template is used. How hallucination is prevented. How source attribution is maintained in generated text.

Anomaly Detection — How baseline patterns are established. What thresholds trigger anomaly flags. How false positive rates are managed. How anomalies are explained in human terms.

Guardrails — What prevents the AI from returning fabricated data. What happens when the AI is uncertain. How confidence is communicated to users.

Cost Management — Which queries use AI and which are direct database queries. How caching reduces API costs. What the per-query cost looks like at different scales.

Document 08 — Brand & Identity Guidelines

Audience: You, contributors, anyone writing about DataSetu
Format: Markdown in `/docs/brand/` with logo files
Length: 1-2 pages

This is short but important for consistency.

Name — DataSetu. Capital D, capital S. Never Datasetu or datasetu in prose (though lowercase in code and package names is fine). Never abbreviated to DS.

Tagline — One line that appears everywhere. Something like “Clean, queryable Indian public data” or “The bridge to India’s public data.” Pick one and use it consistently.

Logo — Keep it simple. You can create this with any free tool. A clean wordmark is better than a bad icon. Include SVG and PNG versions in the repo.

Voice and Tone — How DataSetu communicates. Honest about limitations. Technical but accessible. Never hype. Never corporate. The tone of a knowledgeable friend who happens to be an engineer.

Colors and Typography — Pick two colors and one font. Use them everywhere. Website, docs, playground, social media. Consistency builds recognition.

Document 09 — README

Audience: Everyone — this is the front door of your entire project
Format: `README.md` in repo root
Length: One screen of essential information plus expandable sections

This is the most important document in your entire project. More people will read your README than every other document combined. It must accomplish five things in this exact order.

First three seconds: the reader understands what DataSetu is. First thirty seconds: the reader knows how to install it. First two minutes: the reader has made their first query. First five minutes: the reader understands the scope and limitations. First ten minutes: the reader knows how to contribute.

Document 10 — Contributing Guide

Audience: Potential contributors
Format: `CONTRIBUTING.md` in repo root
Length: 2-3 pages

It should cover the following. How to set up the development environment. How to report bugs with a good bug report template. How to suggest new data sources. How to contribute canonical mappings (this is the easiest entry point for new contributors). How to submit code changes. Your review process and response time commitment. What kind of contributions you are actively looking for.

Document 11 — Code of Conduct

Audience: Community
Format: `CODE_OF_CONDUCT.md` in repo root
Length: 1 page

Use the Contributor Covenant which is the industry standard. It is available at contributor-covenant.org. Do not write your own. Using the standard one signals that you understand open source norms.

Document 12 — License Document

Audience: Everyone
Format: `LICENSE` file in repo root
Length: Standard license text

Use MIT License. It is the most permissive and widely understood open source license. It allows anyone to use your code for any purpose including commercial use. This maximizes adoption which is your primary goal. Do not use GPL or AGPL unless you have a specific reason to restrict commercial use.

Document 13 — Security Policy

Audience: Security researchers and users
Format: `SECURITY.md` in repo root
Length: Half a page

State how to report security vulnerabilities (email, not public issue). State your response time commitment. State what versions are supported. This is standard for any professional open source project and its absence is noticed by experienced developers.

Document 14 — Terms of Use

Audience: API and playground users
Format: Page on your website
Length: 1-2 pages in plain language

Not a legal wall of text. Clear, readable terms that state the following. DataSetu aggregates publicly available Indian government data. Data should be verified against primary sources for critical decisions. The service is provided as-is with no uptime guarantees. Rate limits exist and abuse will result in access restriction. Attribution to original government sources is required when publishing.

Document 15 — Privacy Policy

Audience: Users
Format: Page on your website
Length: 1 page in plain language

State what you collect (API usage analytics, playground queries). State what you do not collect (personal information, payment details since there are none). State whether you use cookies. State whether you share data with third parties. Keep it honest and minimal because you are probably collecting very little.

Document 16 — Data Attribution & Legal Compliance

Audience: You, users, anyone concerned about legal standing
Format: Markdown in `/docs/legal/`
Length: 2-3 pages

This document addresses the legal considerations discussed earlier in our conversation.

Attribution Standards — How every data point credits its government source. The format of attribution strings. How users should cite DataSetu in their own work.

Government Data Licensing — Overview of the Government Open Data License. Which of your sources are covered by it. Which sources have different or unclear terms.

Scraping Ethics — Your rate limiting policy. Your approach to sources that block automated access. Your commitment to not overloading government servers.

Disclaimer — DataSetu is not affiliated with any government body. Data is provided for informational purposes. Users are responsible for verifying data against primary sources for critical applications.

Document 17 — Changelog

Audience: Users and contributors
Format: `CHANGELOG.md` in repo root
Length: Grows with every release

Follow the Keep a Changelog format from keepachangelog.com. Every version entry has sections for Added, Changed, Deprecated, Removed, Fixed, and Security. Start this from your very first release. It shows professionalism and respect for users who depend on your tool.

Document 18 — Public Roadmap

Audience: Users and community
Format: GitHub Projects board or a dedicated page on your docs site
Length: Living document

This is the public-facing version of your private execution strategy. It shows what you are working on now, what is coming next, and what is being considered. It does not include timelines (those are internal). It does not include failure protocols (those are internal). It shows direction and invites community input on priorities.

Use three columns. Now (actively being built). Next (committed to building soon). Considering (might build based on demand). Let users vote or comment on items in the Considering column.

Document 19 — Content & Distribution Strategy

Audience: You (private, not published)
Format: Internal document
Length: 2-3 pages

This is your private plan for how you create content and reach people.

Blog Post Calendar — Plan your first five blog posts with titles, target publish dates, and target platforms. Post one is the problem-first NCRB parsing story. Post two is a technical deep-dive into your normalization approach. Post three is an interesting finding from the data itself. Post four is the AI layer technical explanation. Post five is a retrospective on building DataSetu.

Community Engagement Plan — Which communities you will be active in. How often you will post. What kind of contributions you will make beyond promoting your own project.

Launch Checklist — Every platform and action for your Phase 3 launch. Hacker News title and comment. Reddit posts with subreddit-specific framing. Twitter thread structure. LinkedIn post angle.

Document 20 — Portfolio Case Study

Audience: Recruiters and hiring managers
Format: Blog post or dedicated portfolio page
Length: 2-3 pages

This is written after Phase 3 when you have real metrics and real usage to reference.

Problem Statement — What problem exists and why it matters. Two paragraphs maximum.

Technical Approach — What you built and why you made the architectural decisions you made. Focus on tradeoffs, not just what you chose but what you rejected and why.

Engineering Highlights — The hardest technical challenges you solved. PDF parsing edge cases. Entity resolution across historical boundary changes. AI guardrails against hallucination. Pick three and go deep.

Impact Metrics — Real numbers. GitHub stars, npm downloads, weekly active users, number of real-world citations or integrations. Only include numbers you can verify.

Skills Demonstrated — Explicitly map your work to skills employers care about. Data engineering. API design. TypeScript. AI/LLM integration. Open source community building. Technical writing.

Document Creation Priority

Priority	Document	Create During	Reason
1	Doc 00 — Execution Strategy	Already Done	Foundation for everything
2	Doc 01 — Project Overview & Vision	Phase 0	Clarifies your thinking before building
3	Doc 02 — Product Requirements	Phase 0	Prevents scope creep during Phase 2
4	Doc 09 — README	Phase 2	Must exist before

Priority	Document	Create During	Reason
			publishing to npm
5	Doc 12 — License	Phase 2	Must exist before publishing to npm
6	Doc 05 — SDK Design Specification	Phase 2 Start	Design API before coding it
7	Doc 06 — Data Schema & Normalization	Phase 2 Mid	Document as you build the normalizer
8	Doc 04 — Data Source Registry	Phase 2 End	Document after first source is complete
9	Doc 03 — Technical Architecture	Phase 2 End	Document after architecture stabilizes
10	Doc 10 — Contributing Guide	Phase 3 Start	Must exist before inviting community
11	Doc 11 — Code of Conduct	Phase 3 Start	Must exist before inviting community
12	Doc 13 — Security Policy	Phase 3 Start	Professional standard for public repos
13	Doc 17 — Changelog	Phase 3 Start	Begin tracking from first public release
14	Doc 14 — Terms of Use	Phase 3 Start	Must exist before playground goes public
15	Doc 15 — Privacy Policy	Phase 3 Start	Must exist before playground goes public
16	Doc 16 — Data Attribution & Legal	Phase 3 Start	Must exist before public data access

Priority	Document	Create During	Reason
17	Doc 08 — Brand & Identity	Phase 3 Start	Needed for consistent public presence
18	Doc 19 — Content Strategy	Phase 3 Start	Guides your distribution efforts
19	Doc 18 — Public Roadmap	Phase 3 Mid	Publish after initial community forms
20	Doc 07 — AI Layer Specification	Phase 4	Write when you build it not before
21	Doc 20 — Portfolio Case Study	Phase 3 End	Write when you have real metrics

---

One Final Note

Do not try to write all twenty documents before you start building. That is a procrastination trap disguised as professionalism. Follow the priority order above. Create each document when it becomes necessary for the phase you are in. A perfect documentation suite with no product behind it impresses nobody. A working product with documentation that grows alongside it impresses everyone.

Start with Document 01 and Document 02 today. They take a few hours combined and they will sharpen your thinking before you write your first line of code.

Finding the Right Available Name

Since datasetu is taken on npm, we need to find something that is actually available. Let me think about this differently.

Before I suggest names, here is what you should check for each option before committing.

Check npm availability by going to npmjs.com/package/[name]. Check domain availability for .dev, .io, and .sh extensions. Check GitHub for organization or username availability. Check Twitter/X for handle availability. All four need to work together for a professional brand.

My Revised Suggestions

OpenSetu — Open communicates open source and open data simultaneously. Setu still carries the bridge metaphor. Eight letters. The word Open is universally understood by developers and signals that this is a community project not a commercial product. `npm install opensetu` might be available. `opensetu.dev` as a domain. In code: `import { OpenSetu } from 'opensetu'`. The brand angle is “the open bridge to India’s public data.” This is my top pick because Open has strong positive associations in the developer community and it immediately tells people this is free and community driven which aligns perfectly with your goals.

SetuData — Reverse of DataSetu. If datasetu is taken, setudata might not be. Same meaning, same clarity, different word order. Eight letters. `npm install setudata`. The slight advantage of this order is that Setu comes first which makes the Indian identity more prominent. The slight disadvantage is that English speakers more naturally say “data bridge” than “bridge data” but honestly both work fine.

SanskritData or SanData — Completely different direction. SanData is seven letters, short, clean, and the prefix San could be interpreted as Sanskrit or Sankhya (numbers) or simply as a clean modern prefix. `npm install sandata`. This has the advantage of being distinctive and unlikely to be taken anywhere.

DataSarathi — Sarathi means charioteer or guide in Sanskrit. The most famous sarathi is Krishna who guided Arjuna. The metaphor is perfect. DataSetu guides you through the complexity of Indian public data. Ten letters is slightly long but the word is beautiful and deeply meaningful. `npm install datasarathi`. The brand angle is “your guide through India’s public data.”

NitiData — Niti means policy or guidance in Sanskrit. It directly connects to the policy and governance nature of the data you are serving. Also subtly references NITI Aayog which is India’s premier policy think tank. Eight letters. `npm install nitidata`. The brand angle is “policy data, made accessible.”

JanSetu — Jan means people in Hindi. JanSetu means “people’s bridge.” Seven letters. It communicates that this is a bridge built for the public to access public data. `npm install jansetu`. The brand angle is “the people’s bridge to public data.” Earlier I

rejected JanData because it sounded like an NGO but JanSetu with the bridge metaphor sounds more like infrastructure which is what you are building.

Dhara — Just the single word. It means flow or stream. Six letters. Short, clean, elegant. `npm install dhara`. It does not have "data" in the name which was your concern with Setu alone. But it is short enough and distinctive enough that it could work as a standalone brand with a tagline that provides context. "Dhara — streaming clean Indian public data." The risk is the same as Setu alone, people might not immediately associate it with data.

SevaSetu — Seva means service or selfless work. SevaSetu means "bridge of service." Eight letters. It communicates the social good mission of your project beautifully. `npm install sevasetu`. The brand angle is "serving India through open data." This name is particularly strong because it signals that the project exists to serve the community not to make money which aligns exactly with your stated goals.

Quick Comparison

Name	Letters	Meaning	Data Clarity	Available (Check)	Brand Feel
OpenSetu	8	Open Bridge	High (Open = open data)	Check npm	Community + Indian
SetuData	8	Bridge to Data	High	Check npm	Indian + Technical
SanData	7	Sacred/ Sanskrit Data	Medium	Likely available	Minimal + Clean
DataSarathi	11	Data Guide	High	Likely available	Meaningful + Deep
NitiData	8	Policy Data	High	Likely available	Governance + Serious
JanSetu	7	People's Bridge	Medium	Likely available	Civic + Infrastructure
Dhara	5	Flow/ Stream	Low alone	Check npm	Elegant + Short
SevaSetu	8	Bridge of Service	Medium	Likely available	Mission + Service

---

What I Want You To Do Right Now

Do not spend more than one hour on this. Naming is important but it is also one of the easiest places to get stuck in an infinite loop of second-guessing.

Take the top three names that feel right to you from this list. Open three browser tabs. Check npm availability for each. Check domain availability for each. Check GitHub organization availability for each.

The first name that is available across all three platforms is your name. Pick it. Register the npm package name immediately by publishing an empty placeholder package. Buy the domain. Create the GitHub organization or repo.

Then close the naming question permanently and start Document 01. The name is a container. What matters is what you put inside it.

Bharat Data — Honest Assessment

Yes. This works. Let me tell you why it is actually a strong choice and then the few things you need to verify and decide.

Why Bharat Data Works

The word Bharat is officially recognized as the name of India in the Indian Constitution. It carries national identity without being obscure. Unlike Sanskrit words that need explanation, every Indian knows what Bharat means and most international developers have enough context to connect it to India. It is not a word that needs a translation or a tagline to make sense.

The word Data makes the purpose instantly clear to anyone on the planet. A developer sees Bharat Data and immediately understands “this is about Indian data.” A journalist understands it. A recruiter understands it. A researcher understands it. There is zero ambiguity about what this project does.

Together the two words say exactly what you are building. India's data, made accessible. No metaphors needed. No Sanskrit translations required. No tagline necessary to explain the name. It is direct, honest, and clear which matches the personality of your project.

The name also carries a subtle but real sense of national pride and civic purpose. You are not just building a data tool. You are building Bharat's data infrastructure. That framing resonates with Indian developers, with Indian media, with government officials, and with the broader Indian tech community. It positions the project as something that belongs to the country, not just to you.

The Naming Decision — One Word or Two

You need to decide how to format it because this affects everything from package names to domains to code imports.

Option 1: bharatdata (one word, lowercase) This is the cleanest option for technical use. `npm install bharatdata` is simple. `import { BharatData } from 'bharatdata'` reads well in code. `bharatdata.dev` or `bharatdata.io` as a domain is clean. GitHub repo name `bharatdata` works. This is my recommendation for the package name and domain regardless of how you write it in prose.

Option 2: Bharat Data (two words, capitalized in prose) In your README, blog posts, landing page, and all human-facing text, write it as two words with capitals. Bharat Data. This reads more naturally in English and looks more like a proper brand name than a single compressed word. Most developer tools do this. GitHub is one word in code but two words conceptually. YouTube is one word but everyone understands it as two concepts.

My recommendation: Use `bharatdata` as one word for all technical contexts — npm package name, domain, GitHub repo, import statements. Use Bharat Data as two words in all human-facing contexts — README title, blog posts, landing page, social media. This is exactly how most professional projects handle it.

What You Need to Check Right Now

Go check these five things before committing.

Check `npmjs.com/package/bharatdata` to see if the npm package name is available. If it is, publish an empty placeholder package immediately to reserve it. If it is taken, check `@bharatdata/core` as a scoped package alternative.

Check domain availability for `bharatdata.dev` which is the best option for a developer tool. If taken, check `bharatdata.io`, `bharatdata.sh`, `getbharatdata.dev`, or `bharatdata.in`. The `.in` domain is actually interesting here because it reinforces the Indian identity.

Check GitHub for the organization name `bharatdata` or the repo name `bharatdata`. If you can create a GitHub organization called bharatdata, do it now. It makes your project look established and it lets you use scoped packages like `@bharatdata/core` on npm.

Check Twitter/X for the handle `@bharatdata` or `@bharatdatadev`. Having a matching social handle matters for brand consistency.

Check Google to make sure there is no existing well-known project or company called Bharat Data that you would be competing with for search visibility. A quick search will tell you if the namespace is clear.

One Potential Concern and Why It Is Manageable

The word Bharat has been used in many government and semi-government initiatives. BharatPe, Bharat Bill Payment System, Digital Bharat, Bharat Net, and others. There is a small risk that someone might initially mistake your project for an official government initiative. This is manageable through two simple measures. First, include a clear disclaimer on your website and README stating that Bharat Data is an independent open source project and is not affiliated with any government body. Second, your open source nature, GitHub presence, and developer-focused branding will quickly make it obvious that this is a community project not a government portal.

This concern is minor and should not stop you from using the name.

How It Looks in Practice

In your README header: **Bharat Data — Clean, queryable Indian public data**

In npm install instructions: `npm install bharatdata`

In code:

```
import { BharatData } from 'bharatdata'
const bd = new BharatData()
const crimes = await bd.crime.getByState('Maharashtra', 2023)
```

In a blog post title: “I Built Bharat Data Because Indian Government Data Shouldn’t Be This Hard to Access”

In a Hacker News post: “Show HN: Bharat Data — Open source SDK for Indian government data”

On your resume: “Built Bharat Data, an open source SDK serving clean Indian government data to X developers”

It works everywhere. It reads naturally everywhere. It needs no explanation anywhere.

My Final Take

Bharat Data is a strong name. It is clear, meaningful, proud, and immediately understandable. It does not require a Sanskrit dictionary or a tagline to explain itself. It says exactly what the project is. India’s data, made accessible.

Go check availability across npm, domain, GitHub, and Twitter right now. If it is available, take it. Register everything today. Then close the naming question permanently and move to Document 01.

You have spent enough time on the name. The name is good. Now build the thing it represents.

Bharat Data — Professional Documentation Index (Updated)

Important Rule

Write each document when its phase arrives. Not before. A document written before you have built anything is fiction. A document written after you have built something is knowledge. The priority list at the bottom ensures you create each document at the moment it becomes necessary and not a day earlier.

Complete Document List

Doc No.	Document Name	Purpose	Audience	Create During
00	Execution Strategy	Internal roadmap with phases, milestones, timelines, failure protocols	You (private)	Already Done
00a	Project Journal	Running log of decisions, problems, surprises — 5 minutes per session	You (private)	Phase 2 Day 1 onwards
01	Project Overview & Vision	Outward-facing narrative of what Bharat Data is and why it matters	Everyone	Phase 0
02	Product Requirements Document	Internal operational spec defining v1 scope, acceptance criteria, constraints	You + Contributors	Phase 0
03	Technical Architecture Document	System design, data flow, infrastructure decisions, tradeoffs	You + Contributors + Recruiters	Phase 2 End
03a		How parser accuracy,		

Doc No.	Document Name	Purpose	Audience	Create During
	Testing Strategy	normalization correctness, and data quality are verified	You + Contributors	Phase 2 End
04	Data Source Registry	Every data source catalogued with status, format, frequency, legal standing	You + Users + Contributors	Phase 2 End
05	SDK Design Specification	API surface design, function signatures, type definitions, naming conventions	You + Contributors	Phase 2 Start
06	Data Schema & Normalization Guide	How raw data becomes clean data, canonical mappings, entity resolution rules	You + Contributors + Researchers	Phase 2 Mid
07	AI Layer Specification	How GenAI and Agentic AI components work, prompt strategies, guardrails	You + Contributors + AI Community	Phase 4
08a	Brand Minimums Checklist	Canonical name format, tagline, disclaimer — minimum needed before README	You	Phase 2 Day 1
08	Brand & Identity Guidelines	Full brand document with logo, colors, voice,	You + Contributors + Media	Phase 3 Start

Doc No.	Document Name	Purpose	Audience	Create During
		tone, usage rules		
09	README	First thing anyone sees on GitHub, installation to first query in 2 minutes	Everyone	Phase 2
10	Contributing Guide	How to contribute code, data mappings, bug reports, feature requests	Contributors	Phase 3 Start
11	Code of Conduct	Community behavior standards	Contributors + Community	Phase 3 Start
12	License Document	Legal terms for using and contributing to the project	Everyone	Phase 2
13	Security Policy & Incident Response	How to report vulnerabilities plus what happens when bad data is served	Users + Security Researchers	Phase 3 Start
14	Terms of Use	Legal terms for using the API and data	Users	Phase 3 Start
15	Privacy Policy	What data you collect from users, how you handle it	Users	Phase 3 Start
16	Data Attribution & Legal Compliance	Source attribution standards, government data licensing, scraping policies	You + Users + Legal	Phase 3 Start
17	Changelog	Version history with	Users + Contributors	Phase 3 Start

Doc No.	Document Name	Purpose	Audience	Create During
		what changed, what broke, what was added		
18	Public Roadmap	User-facing direction document accessible to non-GitHub users	Users + Community	Phase 3 Mid
19	Content & Distribution Strategy	Blog post plan, community engagement plan, launch checklist	You (private)	Phase 2 Week 2
20	Portfolio Case Study	Your personal narrative of building Bharat Data for job applications	Recruiters + You	Phase 3 End

---

Document Details

Document 00 — Execution Strategy

Status: Complete **Audience:** You (private, not published) **Format:** Internal document **Create During:** Already Done **Time to Write:** Done

This is your phased roadmap with milestones, timelines, exit conditions, and failure protocols. This document stays private. It is your operating manual. Everything else flows from it.

Document 00a — Project Journal

Audience: You (private, never published directly) **Format:** Simple text file or Notion page at `/docs/internal/JOURNAL.md` **Length:** Grows daily, 3-5 bullet points per session **Create During:** Phase 2 Day 1, maintained throughout entire project

This is not a formal document. It is a running log that you write for five minutes at the end of every build session. You write down what you worked on, what decision you made and why, what surprised you, what broke, and what you would do differently.

This journal feeds three critical documents later. Your Technical Architecture Document uses it to explain why you chose certain approaches over alternatives. Your Portfolio Case Study uses it for the specific vivid details that make technical writing compelling. Your blog posts use it for the real stories that attract readers.

The details that make your writing stand out cannot be reconstructed from memory months later. “Today pdfplumber failed on Table 14 because the header spans three merged rows and the footnote marker sits inside a numeric cell” is the kind of entry that becomes a fascinating paragraph in a blog post. Without the journal that detail is lost forever.

Write it every session. Five minutes. No formatting required. Just raw notes.

Document 01 — Project Overview & Vision

Audience: Everyone — developers, journalists, researchers, recruiters, media **Format:** Markdown at `/docs/PROJECT_OVERVIEW.md` **Length:** 2 pages maximum **Create During:** Phase 0 **Time to Write:** 2 hours

This is purely an outward-facing narrative document. No technical details. No user stories. No scope lists. No acceptance criteria. Those belong in Document 02. This document tells the story of why Bharat Data exists and who it is for.

What is Bharat Data. One paragraph. An open source project that makes Indian government public data accessible, clean, and queryable. Not a product description. A statement of purpose.

Why does Bharat Data exist. Two paragraphs. The problem stated through concrete human stories. A journalist who spends three weeks cleaning NCRB data before writing a single word of analysis. A developer who builds a custom scraper for every project because no unified data access layer exists. A researcher who wastes the first six months of their PhD just collecting and normalizing census data. Real pain, described in human terms.

Who is Bharat Data for. One paragraph. Developers, journalists, researchers, students, policy analysts. Listed simply without elaboration.

What Bharat Data believes. One paragraph. Public data should be publicly accessible. Clean data is a public good. Open source is the right model for civic infrastructure. Data quality matters more than data quantity.

Where Bharat Data is going. One paragraph. More datasets driven by community demand. AI-powered querying for non-technical users. A canonical reference for Indian administrative geography. Honest about current state versus future direction.

This document should read like a manifesto, not a spec sheet. It is the document a journalist quotes when writing about your project. It is the document a conference organizer reads when deciding whether to invite you to speak.

Document 02 — Product Requirements Document

Audience: You and contributors (internal operational document) **Format:** Markdown at `/docs/internal/PRD.md` **Length:** 3-5 pages **Create During:** Phase 0 **Time to Write:** 3 hours

This is purely an internal operational spec. No narrative. No mission statements. No storytelling. Those belong in Document 01. This document defines what you are building, what you are not building, and how you know when each part is done.

v1 Scope — Included:

One dataset: NCRB Crime in India (or whichever passive research confirms).

Data pipeline: Ingester, parser, normalizer, confidence scorer.

Storage: PostgreSQL on Neon free tier with full data lineage.

SDK: TypeScript package on npm with five query functions.

API: REST endpoints matching SDK functions.

Playground: Web-based text input returning structured results with CSV download.

Monitoring: Daily health check on data sources with automated issue creation.

Attribution: Source URL, collection date, and confidence score on every data point.

v1 Scope — Explicitly Excluded:

AI natural language queries. Multiple datasets. User accounts or authentication. Paid tiers or subscription logic. Mobile application. Data visualization or charting. Webhook notifications. Email alerts. Dashboard analytics. Rate limiting by user (implement global rate limiting only). Any form of user-generated content.

Write these exclusions down so you cannot talk yourself into adding them mid-build.

User Stories:

As a developer, I want to install a package and query Indian crime data by state and year so that I can build a data application without spending weeks collecting data.

As a journalist, I want to visit a web playground and ask about crime statistics so that I can get verified numbers for my article without downloading PDF reports.

As a researcher, I want to download clean structured crime data as CSV so that I can use it in statistical analysis without manual data cleaning.

As a student, I want to access real Indian government data through a simple API so that I can use it in my college project instead of fake datasets.

As a contributor, I want to add state or district name variations to a public mappings file so that I can help improve data quality for everyone.

Acceptance Criteria:

Parser extracts all major tables from NCRB report with greater than 95 percent accuracy verified against manual spot checks of 20 random cells.

SDK returns fully typed responses for all five functions with zero any types in the public API surface.

Playground returns results for any valid query in under 3 seconds.

Every data point includes source URL, collection date, and confidence score in its response.

README enables a new user to go from npm install to first successful query in under 5 minutes, verified by testing with one person unfamiliar with the project.

Health monitoring detects a source change or outage and creates a GitHub issue within 24 hours.

Technical Constraints:

All infrastructure on free tiers. Single developer, no team. SDK works offline after installation. API response time under 2 seconds. Database fits within Neon free tier 500MB limit. Scrapers never exceed one request per second to any government server.

Dependencies and Risks:

NCRB website might change format, move URLs, or go offline temporarily. PDF parser might fail on certain table structures. Free tier database might hit storage limits as historical data grows. Government body might object to automated access. npm package name might conflict with existing packages.

Document 03 — Technical Architecture Document

Audience: You, contributors, recruiters **Format:** Markdown with diagrams at `/docs/architecture/ARCHITECTURE.md` **Length:** 4-6 pages with diagrams **Create During:** Phase 2 End **Time to Write:** 4 hours

Write this after your architecture stabilizes. Document what you actually built, not what you planned to build. Your Project Journal (Doc 00a) provides the raw material for this document.

System Overview — High level diagram showing all components and data flow. Government data source to ingester to parser to normalizer to confidence scorer to database to API layer to SDK to end user. One diagram that tells the complete story.

Component Details — Each component in one to two paragraphs covering what it does, what technology it uses, why you chose this approach, and what alternatives you considered and rejected. The “rejected alternatives” part is what makes this document valuable to recruiters because it demonstrates architectural thinking.

Data Flow — Ingestion Path — Step by step walkthrough. Source is checked for updates via hash comparison. Raw file is downloaded and stored with metadata. Parser extracts structured records with confidence annotations. Normalizer applies canonical mappings. Records are validated against expected ranges. Clean records are stored in Postgres with full lineage chain.

Data Flow — Query Path — Step by step walkthrough. User calls SDK function or hits API endpoint. Request is validated. Rate limiter checks. Cache is checked. Database query is executed with filters. Results are enriched with source attribution. Response is cached and returned.

Infrastructure Map — What runs where and why. Database on Neon. API on Vercel or Cloudflare Workers. Cron jobs on GitHub Actions. File storage on Cloudflare R2. What each free tier provides. What limits apply. What happens when limits are reached.

Error Handling Strategy — What happens when each component fails. Scraper failure: log, retry with backoff, alert via GitHub issue after three consecutive failures. Parser failure: flag record with low confidence, never serve unvalidated data silently. Database unreachable: return cached response if available, return service unavailable error with estimated recovery time if not.

Security Considerations — Rate limiting implementation. Input sanitization. Playground abuse prevention. How environment variables and API keys are managed.

Scalability Path — How adding a new dataset works without restructuring existing code. How the system handles 10x or 100x traffic growth. What changes at 100 users versus 1000 versus 10000. Which free tier limits you will hit first and what the migration plan is.

Document 03a — Testing Strategy

Audience: You, contributors **Format:** Markdown at `/docs/architecture/TESTING.md` **Length:** 2-3 pages **Create During:** Phase 2 End **Time to Write:** 2 hours

For a data product, testing is uniquely critical because wrong data served with high confidence is actively worse than no data at all. A developer who gets incorrect crime statistics from your SDK and uses them in a published application has been harmed by your product. This document defines how you prevent that.

Parser Accuracy Testing:

How accuracy is measured. Select 20 random cells from the original PDF. Manually verify the value. Compare against parser output. Calculate accuracy percentage. Minimum threshold is 95 percent. Below 95 percent the parser output for that table is not served through the API.

How edge cases are catalogued. Every parser failure is logged with the specific table, row, column, and reason for failure. These edge cases form a regression test suite that grows over time.

How regression testing works. When you modify the parser, all previously catalogued edge cases are re-tested automatically. A parser change that fixes one table but breaks another is caught before deployment.

Normalization Verification:

How canonical mappings are tested. Every entry in canonical-mappings.json has at least one test case verifying correct resolution. New mapping entries require a corresponding test case in the same PR.

How boundary changes are verified. Historical data for states affected by bifurcation (Telangana, Ladakh, new districts) is verified against known totals. If Andhra Pradesh 2013 total plus Telangana 2014 total do not approximately equal the original combined Andhra Pradesh 2013 total, something is wrong.

Production Data Quality Monitoring:

Daily automated checks that run against the live database. Record counts per state and year should remain stable unless a new ingestion has occurred. Statistical outliers (a value more than 3 standard deviations from historical mean for that field) are flagged for manual review. Any data point with confidence score below 0.5 is excluded from API responses and logged for investigation.

End-to-End Query Testing:

A set of 10 known queries with expected results that run as part of the CI pipeline. If any query returns unexpected results after a deployment, the deployment is flagged.

Document 04 — Data Source Registry

Audience: You, users, contributors, researchers **Format:** Markdown at `/docs/data-sources/REGISTRY.md` plus one page per source in `/docs/data-sources/` **Length:** Summary table plus one detailed page per active source **Create During:** Phase 2 End **Time to Write:** 2 hours

Summary Table Columns:

Source name. Source URL. Publishing government body. Data format (PDF, CSV, HTML, API). Update frequency (annual, monthly, real-time). Coverage period (years available). Date of last Bharat Data ingestion. Legal status (GODL covered, public access, terms unclear). Current status in Bharat Data (active, in progress, planned, deprecated).

Per-Source Detail Page:

Full official name and description of the dataset. Direct URL to original data on government website. Which ministry or department publishes it. What license or terms of use apply. Complete list of fields extracted with data types. Known data quality issues present in the original government source. How Bharat Data normalizer handles edge cases specific to this source. Sample comparison showing raw government data alongside cleaned Bharat Data output. Ingestion history showing dates of all data pulls and any format changes detected.

Document 05 — SDK Design Specification

Audience: You, contributors, advanced users **Format:** Markdown at `/docs/sdk/DESIGN.md` **Length:** 3-4 pages **Create During:** Phase 2 Start (before writing SDK code) **Time to Write:** 3 hours

Write this before you code the SDK. Design the API surface on paper first. Changing function signatures after developers depend on them breaks their code and erodes trust.

Design Principles:

Consistency: Similar operations have similar signatures across all dataset modules.

Predictability: A developer should correctly guess how a function works before reading docs.

Minimalism: No function added unless multiple real users need it.

Type safety: Every input and output fully typed. Zero any in public API surface.

Attribution by default: Every response includes source information automatically.

Fail loudly: Errors are clear, specific, and suggest resolution.

Naming Conventions:

Package name: `bharatdata`

Import: `import { BharatData } from 'bharatdata'`

Client: `const bd = new BharatData()`

Dataset modules: `bd.crime`, `bd.census`, `bd.agriculture`

Query methods: `bd.crime.getByState()`, `bd.crime.getByYear()`

All method names camelCase. All parameter keys camelCase. All response keys camelCase.

v1 Function Signatures:

`bd.crime.getByState(state, year)` — Crime data for a specific state and year.

`bd.crime.getByYear(year)` — Crime data for all states in a given year.

`bd.crime.getByCategory(category, options)` — Crime data filtered by crime category with optional state and year filters.

`bd.crime.compare(states, years)` — Comparative data across multiple states and years.

`bd.crime.search(query)` — Flexible search across all crime data fields.

Response Shape:

Every response returns an object with three keys. `data` containing the actual array of records. `meta` containing source URL, publishing body, collection date, total record count, average confidence score, and Bharat Data version. `query` containing an echo of the request parameters for debugging.

Error Handling Contract:

Custom error classes: `BharatDataError` (base), `SourceNotFoundError`, `InvalidQueryError`, `RateLimitError`, `DataUnavailableError`.

Every error includes human-readable message, machine-readable error code, and a suggested resolution string.

Versioning Strategy:

Semantic versioning. Major bump for breaking changes to function signatures or response shapes. Minor bump for new functions, new datasets, or new optional parameters. Patch bump for bug fixes, data updates, and documentation improvements.

Document 06 — Data Schema & Normalization Guide

Audience: You, contributors, researchers **Format:** Markdown at `/docs/data-schema/NORMALIZATION.md` plus canonical JSON at `/data/canonical-mappings.json` **Length:** 2-3 pages plus the mappings file **Create During:** Phase 2 Mid **Time to Write:** 2 hours

Canonical Entity Mappings:

Reference to `/data/canonical-mappings.json` in the repository. This public version-controlled file contains all state name variations mapped to standard forms, all district name variations mapped to standard forms, historical boundary changes with effective dates, and cross-source administrative code mappings.

Example entry: “AP”, “Andhra Pradesh”, “ANDHRA PRADESH”, “A.P.”, “Andhra” all resolve to canonical form “Andhra Pradesh” with state code “AP” and Census 2011 code “28”.

Historical Boundary Handling:

Telangana: Data before 2014 is attributed to unified Andhra Pradesh. Data from 2014 onward is split between Andhra Pradesh and Telangana. Cross-period queries return both with a boundary change annotation.

Ladakh: Data before 2019 is attributed to Jammu & Kashmir. Data from 2019 onward separates Ladakh as a union territory.

District bifurcations: Parent-child relationships tracked with effective dates. Queries for a parent district after bifurcation return the sum of child districts with an annotation.

Data Type Standards:

All numeric values stored as numbers, never strings. All dates in ISO 8601 format. Null or missing values represented as null, never zero, empty string, or dash. Indian number formatting (lakhs, crores) converted to standard integers in data fields with original formatting preserved in metadata.

Confidence Scoring:

1.0: Extracted from clean structured source with automated validation against known totals. 0.8: Extracted from PDF with high parser confidence and manual spot-check verification. 0.5: Extracted but parsing ambiguity exists. Served with a warning annotation. Below 0.5: Flagged for manual review. Not served through API until verified.

Normalization Pipeline Order:

Step 1: Raw extraction from source file. Step 2: Character encoding normalization (UTF-8). Step 3: Field mapping to canonical schema. Step 4: State and district name resolution via canonical mappings. Step 5: Data type conversion and validation. Step 6: Deduplication against existing records. Step 7: Confidence scoring based on extraction quality. Step 8: Source attribution attachment. Step 9: Range validation against historical bounds. Step 10: Storage with full lineage metadata.

Document 07 — AI Layer Specification

Audience: You, contributors, AI community **Format:** Markdown at `/docs/ai/AI_SPECIFICATION.md` **Length:** 3-4 pages **Create During:** Phase 4 **Time to Write:** 3 hours

Do not write this until you are building the AI layer. Your approach will change based on what you learn in Phases 2 and 3.

Natural Language Query Engine:

Which LLM is used and why. How the system prompt is structured to understand Indian civic data terminology. How user questions are parsed into structured database queries. How ambiguous queries are handled — clarification questions versus best-guess with confidence annotation. How questions about data outside Bharat Data coverage are handled. Ten examples of successful queries. Five examples of failed queries with explanations of why they failed and how you plan to address them.

Data Narration:

How raw query results become human-readable insights. Prompt template used for narrative generation. How hallucination is prevented by grounding every statement in actual returned data. How source citations are maintained in generated text. How the system avoids editorializing or adding opinions to factual summaries.

Anomaly Detection:

How statistical baselines are established per dataset, per field, per geographic unit. What thresholds trigger anomaly flags. How seasonal patterns are accounted for. How false positive rates are managed. How detected anomalies are explained in human terms with historical context.

Guardrails:

What prevents the AI from fabricating data points not present in the database. What happens when the AI is uncertain — explicit uncertainty communication to users. What queries are refused entirely. How the system handles politically sensitive queries about the data.

Cost Management:

Which user interactions trigger LLM API calls versus direct database queries. How response caching reduces costs. Per-query cost at current usage. Cost projection at 10x and 100x current usage. Fallback behavior when API budget is exhausted.

Query Failure Log:

How every failed or incorrect AI query is logged with the original question, the generated query, the returned result, and what was wrong. How this log is reviewed weekly. How the community can report AI errors through a simple feedback mechanism.

Document 08a — Brand Minimums Checklist

Audience: You **Format:** Simple checklist at `/docs/internal/BRAND_MINIMUMS.md` **Length:** Half a page **Create During:** Phase 2 Day 1 **Time to Write:** 20 minutes

This is the minimum brand information you need before writing your README. The full brand document comes later in Phase 3.

Name format decided: Bharat Data in prose. `bharatdata` in code and package names.

Tagline decided: "Clean, queryable Indian public data" — use this in README, npm description, GitHub repo description, and social media bio.

Disclaimer language decided: "Bharat Data is an independent open source project. It is not affiliated with or endorsed by any Indian government body."

GitHub repo description decided: One line that appears in search results.

npm package description decided: One line that appears on npmjs.com.

That is all you need for Phase 2. Logo, colors, typography, and full voice guidelines come in Document 08 during Phase 3.

Document 08 — Brand & Identity Guidelines

Audience: You, contributors, anyone writing about Bharat Data **Format:** Markdown at `/docs/brand/BRAND.md` with assets in `/docs/brand/assets/` **Length:** 1-2 pages **Create During:** Phase 3 Start **Time to Write:** 1 hour

Name Usage:

Full name in prose: Bharat Data (two words, both capitalized). In code and packages: `bharatdata` (one word, lowercase). Never BharatData, bharatData, Bharatdata, BHARATDATA, or BD. Always use the full name.

Tagline:

“Clean, queryable Indian public data” — used consistently on website, README, social media, npm, and all public materials.

Disclaimer:

Present on website footer, README, and API documentation: “Bharat Data is an independent open source project. It is not affiliated with or endorsed by any Indian government body.”

Voice and Tone:

Honest about limitations. Technical but accessible. Never hype, never corporate, never defensive. The tone of a knowledgeable peer who respects your time. When uncertain, say so. When wrong, correct publicly and promptly.

Logo:

Clean wordmark. No complex icons or illustrations. SVG and PNG versions stored in `/docs/brand/assets/`. Used consistently on README, website, social media, and documentation.

Colors and Typography:

Two brand colors and one font. Defined here with hex codes. Used consistently across website, documentation, playground, and social graphics.

Document 09 — README

Audience: Everyone **Format:** `README.md` in repo root **Length:** One screen of essentials plus expandable sections **Create During:** Phase 2 Week 4 **Time to Write:** 3 hours

The README accomplishes five things in order.

First 3 seconds: Reader understands what Bharat Data is. Project name, tagline, one-sentence description.

First 30 seconds: Reader sees installation and a minimal code example. `npm install bharatdata` followed by a three-line query returning real data.

First 2 minutes: Reader knows what datasets are available, what functions exist, and where to try the playground without installing anything.

First 5 minutes: Reader understands scope, limitations, data sources with attribution, and what is planned next.

First 10 minutes: Reader knows how to contribute, where to report issues, and where to find full documentation.

Structure:

Project name and tagline. One-paragraph description. Badges for npm version, GitHub stars, license, and build status. Quick start with install command and minimal code example. Link to live playground for non-developers. Available datasets table with source attribution. API reference summary linking to full docs. Data sources section with links to government sources. Contributing link. License.

Document 10 — Contributing Guide

Audience: Potential contributors **Format:** `CONTRIBUTING.md` in repo root **Length:** 2-3 pages **Create During:** Phase 3 Start **Time to Write:** 2 hours

Step by step local development environment setup. How to report bugs using the issue template (expected behavior, actual behavior, reproduction steps, environment). How to suggest new data sources using the data source request template (URL, format, frequency, why it matters). How to contribute canonical mappings — the easiest entry point for first-time contributors with explicit instructions. How to submit code changes (fork, branch naming, commit messages, PR description standards). Your review process and commitment to respond within 48 hours. What contributions you are actively seeking right now, updated periodically.

Document 11 — Code of Conduct

Audience: Community **Format:** `CODE_OF_CONDUCT.md` in repo root **Length:** 1 page **Create During:** Phase 3 Start **Time to Write:** 15 minutes

Adopt the Contributor Covenant from contributor-covenant.org. Do not write a custom one. The standard signals open source literacy. Customize only the enforcement contact email address.

Document 12 — License Document

Audience: Everyone **Format:** `LICENSE` file in repo root **Length:** Standard license text **Create During:** Phase 2 Day 1 **Time to Write:** 10 minutes

MIT License. Most permissive and widely understood. Allows commercial use, modification, distribution. Maximizes adoption. Aligns with your community-first mission.

Document 13 — Security Policy & Incident Response

Audience: Security researchers, users **Format:** `SECURITY.md` in repo root **Length:** 1 page **Create During:** Phase 3 Start **Time to Write:** 45 minutes

Vulnerability Reporting:

Report via dedicated email, not public GitHub issues. Acknowledgment within 48 hours. Fix timeline communicated within one week. Credit given to reporters in changelog unless they prefer anonymity. Currently supported versions listed.

Incident Response Protocol:

What constitutes a data incident: serving incorrect data, serving data attributed to the wrong source, serving data with inflated confidence scores, or any data that could mislead users if used in published work.

Detection: Automated monitoring catches statistical anomalies. User reports via GitHub issues or email. Manual review during routine quality checks.

Immediate response: Affected data is flagged or pulled from API within 4 hours of detection. A GitHub issue is opened publicly documenting the incident.

Communication: Users who queried the affected data in the past 7 days are notified if contact information is available. A notice is posted on the website and README if the incident is significant.

Remediation: Root cause analysis. Parser or normalizer fix. Re-ingestion of affected data. All corrected data re-verified against source.

Post-mortem: Public write-up within one week explaining what happened, why, and what was changed to prevent recurrence.

Document 14 — Terms of Use

Audience: API and playground users **Format:** Page at `bharatdata.dev/terms` **Length:** 1-2 pages in plain language **Create During:** Phase 3 Start **Time to Write:** 1 hour

Bharat Data aggregates publicly available Indian government data and serves it in structured format. Data should be verified against primary government sources for any critical decision, published research, or legal proceeding. The service is provided as-is without uptime or accuracy guarantees. Rate limits exist and are documented at the API reference page. Systematic abuse including excessive automated requests will result in access restriction. When using Bharat Data in published work, attribution to both Bharat Data and the original government data source is required. Bharat Data is not a government service and makes no claim of official authority.

Document 15 — Privacy Policy

Audience: Users **Format:** Page at `bharatdata.dev/privacy` **Length:** 1 page in plain language **Create During:** Phase 3 Start **Time to Write:** 1 hour

What is collected: API endpoint usage counts, playground query text for service improvement, standard web analytics (page views, referral sources). What is not collected: personal identifying information, email addresses (unless voluntarily provided), payment information, location data. Cookie usage: only essential cookies if any. Third-party sharing: none. Data retention: usage analytics retained for 12 months then deleted. Playground queries retained for 6 months for AI improvement then deleted. User rights: email to request deletion of any data associated with your usage.

Document 16 — Data Attribution & Legal Compliance

Audience: You, users, legal **Format:** Markdown at `/docs/legal/ATTRIBUTION.md` **Length:** 2-3 pages **Create During:** Phase 3 Start **Time to Write:** 2 hours

Attribution Standards:

Every Bharat Data API response includes the original government source URL, the publishing body name, the date the data was collected by Bharat Data, and the applicable license identifier. Users publishing analysis or applications based on Bharat Data must cite both Bharat Data and the original government source. Suggested citation format provided.

Government Data Licensing:

Overview of the Government Open Data License (GODL) India and what it permits. Which Bharat Data sources are published under GODL. Which sources are publicly accessible but have separate or unclear terms. Bharat Data's approach to sources without explicit open licensing: treat as publicly available information with full attribution, cease use immediately if the publishing body requests it.

Scraping Ethics:

Maximum one request per second to any single government server. No scraping during known high-traffic periods for government services (tax filing deadlines, exam result days). Immediate cessation of scraping if any government body requests it. No circumvention of CAPTCHAs, login walls, or access controls. Robots.txt directives respected.

Independence Disclaimer:

Bharat Data is an independent open source project created and maintained by the community. It is not affiliated with, endorsed by, or connected to any Indian government ministry, department, agency, or body. All data is sourced from publicly available government publications and is provided for informational and research purposes only. Users bear full responsibility for verifying accuracy against primary sources for any critical application.

Document 17 — Changelog

Audience: Users and contributors **Format:** `CHANGELOG.md` in repo root **Length:** Grows with every release **Create During:** Phase 3 Start (first public release) **Time to Write:** 30 minutes initial setup, 15 minutes per release

Follow the Keep a Changelog format. Every version entry includes sections for Added, Changed, Deprecated, Removed, Fixed, and Security. Only include sections that have entries for that release. Start from your first public release and never skip an entry. Date every entry. Link every entry to the corresponding git tag.

Document 18 — Public Roadmap

Audience: Users and community including non-GitHub users **Format:** `ROADMAP.md` in repo root plus a page at `bharatdata.dev/roadmap` **Length:** Living document **Create During:** Phase 3 Mid **Time to Write:** 1 hour initial, updated monthly

This is a Markdown file, not a GitHub Projects board. Journalists, researchers, and policy analysts who use Bharat Data may not have GitHub accounts. The roadmap must be accessible to everyone through the website.

Three sections.

Now — What is actively being built this month. Specific items with brief descriptions.

Next — What is committed for the following one to two months. Less specific but directionally clear.

Considering — What might be built based on community demand. Users can signal interest by opening or reacting to GitHub issues tagged with “roadmap” label.

No dates in the public roadmap. Dates are internal only. The roadmap communicates direction and invites participation without creating expectations you cannot guarantee.

Document 19 — Content & Distribution Strategy

Audience: You (private, not published) **Format:** Internal document at `/docs/internal/CONTENT_STRATEGY.md` **Length:** 2-3 pages **Create During:** Phase 2 Week 2 **Time to Write:** 2 hours

This is moved earlier than originally planned because your first blog post should be drafted during Phase 2 while the parsing pain is fresh and specific. By Phase 3 launch week you want the post edited, polished, and ready to publish on day one — not being written under time pressure.

Blog Post Plan:

Post 1 (Draft during Phase 2, publish Phase 3 launch): “I Tried to Extract Clean Crime Data From NCRB PDFs. Here Is What Broke.” Problem-first story with screenshots of messy data versus clean output. Link to Bharat Data only at the end.

Post 2 (Publish Phase 3 Week 3): “How I Built a Data Normalizer That Handles 780 Indian Districts and Their 47 Name Variations.” Technical deep-dive into the canonical mappings challenge. Shows engineering depth.

Post 3 (Publish Phase 3 Week 5): “What NCRB Crime Data Actually Reveals About [Specific Interesting Finding].” Use your own tool to answer a genuinely interesting question. Demonstrates the product by using it.

Post 4 (Publish Phase 4 Week 2): “Building an AI That Answers Questions About Indian Government Data Without Making Things Up.” Technical AI deep-dive about prompt engineering and guardrails.

Post 5 (Publish Month 6): “Six Months of Building Bharat Data: What Worked, What Broke, What Surprised Me.” Retrospective using material from your Project Journal.

Community Engagement Plan:

Which communities: DataMeet mailing list and Slack, r/developersIndia, Indian data journalism Twitter, relevant Discord servers. Engagement frequency: Present daily, contribute genuinely, promote only when relevant. Contribution beyond self-promotion: Answer questions about data access, share interesting government data findings, help other developers with scraping challenges.

Launch Checklist:

Hacker News: Show HN title, prepared first comment explaining the project. Reddit: Separate posts for r/developersIndia, r/programming, r/webdev with community-specific framing. Twitter/X: Thread with screenshots, code snippets, and playground link. LinkedIn: Impact-focused post about civic data accessibility. Dev.to: Full technical walkthrough article.

Document 20 — Portfolio Case Study

Audience: Recruiters, hiring managers, you **Format:** Blog post or dedicated portfolio page **Length:** 2-3 pages **Create During:** Phase 3 End **Time to Write:** 3 hours (fast because your Project Journal provides all the raw material)

This document is written at Phase 3 End but it is built from notes you have been collecting since Phase 2 Day 1 in your Project Journal. Without the journal this takes a week of trying to remember details. With the journal it takes three hours of synthesis.

Problem Statement:

The Indian government data accessibility gap described in two paragraphs. Specific enough to be compelling. Concrete examples of who is affected and how.

What I Built:

Bharat Data: an open source TypeScript SDK that ingests, parses, normalizes, and serves Indian government data through a type-safe API with a web playground for non-technical users. One paragraph.

Technical Decisions:

Three architecture decisions explained with full tradeoff analysis. Why you chose this parser approach over alternatives. Why the normalizer is designed as a public canonical reference file. Why the SDK API surface is shaped the way it is. Focus on the reasoning process not just the final choice. This is what demonstrates senior-level thinking to recruiters.

Hardest Problems Solved:

Three specific engineering challenges described with enough detail to be interesting. PDF table extraction with inconsistent multi-row headers and embedded footnotes. Entity resolution across decades of administrative boundary changes. Confidence scoring that accurately reflects extraction reliability.

AI Integration (after Phase 4):

How the natural language query layer translates human questions into database queries. How hallucination is prevented. How graceful failure works when the AI cannot answer. What you learned about prompt engineering for domain-specific applications.

Impact Metrics:

GitHub stars. npm weekly downloads. Real-world usage examples — published articles, research citations, developer projects. Community contributors. Specific numbers only, no estimates.

Skills Demonstrated:

Listed explicitly for recruiter scanning. Data engineering and pipeline architecture. TypeScript and API design. PDF parsing and data extraction. Entity resolution and data normalization. LLM integration and prompt engineering. Open source project management and community building. Technical writing and developer documentation.

Creation Priority and Timeline

Priority	Doc No.	Document Name	Create During	Time to Write
1	00			Done

Priority	Doc No.	Document Name	Create During	Time to Write
		Execution Strategy	Already Done	
2	01	Project Overview & Vision	Phase 0 Day 1	2 hours
3	02	Product Requirements Document	Phase 0 Day 2	3 hours
4	08a	Brand Minimums Checklist	Phase 2 Day 1	20 minutes
5	12	License (MIT)	Phase 2 Day 1	10 minutes
6	00a	Project Journal	Phase 2 Day 1 onwards	5 min per session
7	05	SDK Design Specification	Phase 2 Day 1	3 hours
8	06	Data Schema & Normalization Guide	Phase 2 Week 2	2 hours
9	19	Content & Distribution Strategy	Phase 2 Week 2	2 hours
10	09	README	Phase 2 Week 4	3 hours
11	04	Data Source Registry	Phase 2 Week 4	2 hours
12	03	Technical Architecture Document	Phase 2 Week 4	4 hours
13	03a	Testing Strategy	Phase 2 Week 4	2 hours
14	10	Contributing Guide	Phase 3 Day 1	2 hours
15	11	Code of Conduct	Phase 3 Day 1	15 minutes
16	13	Security Policy & Incident Response	Phase 3 Day 1	45 minutes
17	17	Changelog	Phase 3 Day 1	30 minutes

Priority	Doc No.	Document Name	Create During	Time to Write
18	14	Terms of Use	Phase 3 Day 1	1 hour
19	15	Privacy Policy	Phase 3 Day 1	1 hour
20	16	Data Attribution & Legal Compliance	Phase 3 Day 1	2 hours
21	08	Brand & Identity Guidelines	Phase 3 Day 1	1 hour
22	18	Public Roadmap	Phase 3 Week 2	1 hour
23	07	AI Layer Specification	Phase 4 Week 1	3 hours
24	20	Portfolio Case Study	Phase 3 Week 4	3 hours

Summary

Category	Count	Total Time
Phase 0 Documents	2	5 hours
Phase 2 Documents	10	21 hours + ongoing journal
Phase 3 Documents	10	12 hours
Phase 4 Documents	1	3 hours
Already Complete	1	Done
Total	**24**	**~38 hours across all phases**

Honest Answer About Google Antigravity IDE

I need to be straightforward with you here. I do not have reliable information about a Google product specifically called “Antigravity IDE.” I do not want to make up details about a tool I am not confident about because giving you wrong information about IDE configuration, plugin setup, and MCP connections would waste your time and could break your workflow.

There are a few possibilities for what you might be referring to.

What You Might Actually Mean

Firebase Studio (formerly Project IDX) — Google's cloud-based AI-powered development environment that runs in the browser with Gemini AI integration. This is Google's most prominent AI-assisted IDE offering.

Jules by Google — Google's AI coding agent that can work on tasks autonomously. This is more of an agentic coding tool than a full IDE.

Gemini integration in VS Code or Android Studio — Google's AI assistant integrated into existing IDEs.

A very recently launched product — Something released after my knowledge cutoff that I genuinely do not have information about.

What I Need From You

Tell me which of these you are referring to, or share a link to the tool you mean. Once I know exactly which tool you are using, I can give you a properly detailed execution strategy with the correct plugin names, MCP configurations, and workflow optimizations specific to that tool.

If this is a brand new tool I do not have information about, tell me what features it has and I will design the workflow around those capabilities.

What I Can Give You Right Now

While you clarify the IDE, here is what I can provide immediately that applies regardless of which AI-powered IDE you use.

The Vibe Coding Workflow for Bharat Data

No matter which AI IDE you choose, the efficient vibe coding approach for this project follows the same pattern.

The core principle of vibe coding for a data project like Bharat Data is this. You describe what you want in natural language. The AI generates the code. You review the output for

correctness. You test against real data. You iterate. The key difference from regular vibe coding is that your project deals with real government data where wrong output is not just a bug but a trust-destroying event. This means your review step is more important than in a typical web app.

What works well with AI coding assistance for this project:

Generating boilerplate — SDK structure, API route handlers, TypeScript type definitions, database schema migrations, test file scaffolding. AI handles this faster than you can type.

Writing parsers with guidance — You show the AI a screenshot or text sample of the NCRB PDF table structure and describe the edge cases. The AI generates a parser. You test it against real data. You feed failures back to the AI for fixes. This iterative loop is where vibe coding shines.

Building the normalizer — You describe the rules ("AP maps to Andhra Pradesh, all these variations should resolve to one canonical form") and the AI generates the mapping logic and the reference JSON file.

Creating documentation — You describe what a document should contain and the AI drafts it. You edit for accuracy and voice. This is where AI saves the most time for your 24-document plan.

Writing tests — You describe what correct output looks like and the AI generates test cases. This is especially valuable for parser accuracy tests where you need many specific assertions.

What does not work well with AI coding assistance for this project:

Understanding PDF structure — The AI cannot look at a government PDF and understand its visual layout. You need to examine the PDF yourself and describe the structure to the AI in words.

Data quality judgment — The AI cannot tell you whether a parsed number is plausible for Indian crime statistics. You need domain knowledge for this. A value of 50,000 murders in a small district should trigger your suspicion even if the parser extracted it correctly from the source.

Government website behavior — The AI does not know that NCRB takes their website down during report preparation season or that certain government servers block requests after 10 PM IST. You learn this through experience.

MCP Connections That Are Useful Regardless of IDE

MCP (Model Context Protocol) servers that would help with Bharat Data specifically.

Filesystem MCP — Lets the AI read and write files in your project directory. Essential for any coding workflow.

PostgreSQL MCP — Lets the AI query your database directly, inspect schemas, and write migrations. Very useful for the data pipeline work.

GitHub MCP — Lets the AI create issues, read PRs, manage your repository. Useful for automating parts of your open source management workflow.

Fetch/Web MCP — Lets the AI make HTTP requests to test your API endpoints and potentially examine government data source pages.

Memory/Knowledge MCP — Lets the AI remember context across sessions about your project decisions, data schemas, and normalizer rules. Particularly useful for a long-running project like Bharat Data where consistency across sessions matters.

Tell Me Your IDE and I Will Build the Full Strategy

Reply with one of these and I will give you the complete updated execution strategy with exact plugin names, MCP configurations, extension recommendations, and vibe coding workflow specific to that tool.

Option A: Firebase Studio (Project IDX)

Option B: Jules by Google

Option C: VS Code with Gemini

Option D: Something else (share the name or link)

I would rather give you the right answer after a one-message clarification than give you a detailed wrong answer right now. Which tool are you using?

Bharat Data — Updated Execution Strategy with Antigravity IDE

IDE Setup — Complete Before Phase 0

This section is your one-time setup. Do it once, do it right, never think about it again.

Install and Configure Antigravity

Download from antigravity.google/download. Sign in with your Google account. When it asks your development approach, select review-driven development.

Go to Settings and configure your models. Set Claude Opus 4.6 as your primary model for architecture decisions, complex reasoning, parser logic, and anything touching the data pipeline. Set Gemini 3 Pro as your secondary model for boilerplate generation, documentation drafting, repetitive code, and test scaffolding. Set Gemini 3 Flash for quick inline completions, comment generation, and simple edits where speed matters more than depth.

The model switching capability is your single biggest advantage in Antigravity. Use the right model for each task. Complex pipeline logic gets Claude Opus. Boilerplate SDK structure gets Gemini Pro. Quick formatting fixes get Gemini Flash. This discipline saves you both money and quality.

Set your default interaction mode to Planning mode globally. Never use Fast mode for anything in the Bharat Data project. You are building a data product where wrong decisions compound silently. Planning mode generates artifacts of the agent’s thinking — task lists, implementation plans, walkthroughs — giving you review gates before any code is written. Fast mode skips these gates and that is unacceptable for a project where serving wrong data destroys your reputation permanently.

Install MCP Servers

Access the MCP Store from Editor View through the dropdown menu and select MCP Servers.

Install these six MCP servers in this order.

Sequential Thinking MCP — Install first. This is the most critical MCP for Bharat Data. It forces the agent to reason through edge cases step by step before writing code. Without it the agent will generate parsers that handle the happy path beautifully and fail silently on the messy government PDF edge cases that are your entire problem domain. Every pipeline component should be built with Sequential Thinking active.

PostgreSQL MCP or Neon MCP — Search for PostgreSQL or Neon in the MCP Store. This connects your agent directly to your database so it can inspect schemas, generate migrations, write queries, and verify that stored data matches expected structure. You will use this constantly during Phase 2 when building the storage layer and during testing when spot-checking parser output.

GitHub MCP — Connects your agent to your GitHub repository. The agent can create branches, commit code, open pull requests, manage issues, and update your changelog. This saves enormous time during Phase 3 when you are managing documentation, community contributions, and release management simultaneously.

Filesystem MCP — Lets the agent read and write files directly in your project directory. Essential for the normalizer workflow where the agent needs to read canonical-mappings.json, check for unmapped entities, and update the file. Also essential for the Project Journal where the agent can append session notes.

Context7 MCP — Provides current documentation for your technology stack. Install this so the agent has accurate docs for pdfplumber, Hono, Drizzle ORM, Neon SDK, and npm publishing workflows without hallucinating outdated API signatures. This prevents the frustrating situation where the agent generates code using a deprecated function because its training data is stale.

Brave Search or Fetch MCP — Lets the agent make web requests to verify that government URLs are still live before building scrapers around them. Also useful when the agent needs to check current NCRB website structure or verify data.gov.in API endpoints. If this MCP is unavailable or unreliable in Antigravity, your fallback is checking URLs manually in a browser.

Honest note about MCP reliability. Antigravity uses Google’s own extension system alongside MCP. Google-native MCPs work well. Third-party MCPs are inconsistent. If a specific MCP fails to install or behaves unreliably, do not let it block your progress. Use the terminal or a browser for that specific task and move on. No MCP failure should stop your build momentum.

Install Extensions

Since Antigravity is built on a VS Code fork, most VS Code extensions work. Install these grouped by purpose.

For the data pipeline work:

Python and Pylance for pdfplumber scripts and data cleaning utilities. Thunder Client as a lightweight REST client for testing your API endpoints without leaving the IDE. SQLTools with the PostgreSQL driver for direct database querying from the IDE sidebar so you can spot-check parsed data without switching to a separate database tool.

For TypeScript SDK development:

ESLint configured to enforce your zero-any rule automatically. This is not optional. Your SDK Design Specification requires zero any types in the public API surface. ESLint catches violations before they reach your repository. Prettier for consistent formatting across the entire codebase. TypeScript Hero for auto-organizing imports. npm Intellisense for autocomplete on package names.

For your 24 documents:

Markdown All in One for live preview, automatic table of contents generation, and link validation across your documentation files. markdownlint to catch formatting errors before publishing. These two extensions together ensure your documentation looks professional without manual formatting effort.

For data inspection:

Rainbow CSV to color-code CSV columns when spot-checking parser output against original sources. This is genuinely useful when you are manually verifying 20 random cells from a parsed NCRB table. JSON Crack to visualize complex JSON structures like your canonical-mappings.json which will grow large as you add district name variations.

For version control and project management:

GitLens for full git history and blame annotations. Essential for tracking when a specific parser change introduced a regression. GitHub Actions extension to view and manage your scraper cron jobs directly from the IDE without switching to the GitHub web interface.

Create Skills Before Writing Any Code

Skills are Antigravity's most powerful feature for a long-running project like Bharat Data. They are persistent context files stored in your repository that instruct every agent session to follow your project's specific rules automatically.

Create a `.agents/skills/` folder in your project root and add these five skill files.

Skill 1 — PDF Parser skill at `.agents/skills/pdf-parser/SKILL.md`

Name this pdf-parser. Description: Extracts structured tables from NCRB government PDF reports. Rules: Always use pdfplumber for text extraction. Always assign a confidence score between 0.0 and 1.0 to every extracted row. Never output a null value without an explicit low-confidence flag. Cross-check row totals against column totals where both exist in the original table. When a table spans multiple pages, detect continuation and merge. When headers span multiple rows, resolve to flat column names. Log every parsing failure with table number, page number, and failure reason.

Skill 2 — Data Normalizer skill at `.agents/skills/normalizer/SKILL.md`

Name this normalizer. Description: Normalizes Indian state and district names to canonical forms using the Bharat Data reference file. Rules: Always read data/canonical-mappings.json before any normalization operation. Always log unmapped entities to a separate file for manual review. Never silently drop records that cannot be normalized. Never modify canonical-mappings.json without explicit approval. When encountering a new name variation, add it to an unmapped-entities.json staging file, not directly to canonical mappings. Handle Telangana bifurcation by attributing pre-2014 data to unified Andhra Pradesh and post-2014 data to separate states.

Skill 3 — SDK Conventions skill at `.agents/skills/sdk-conventions/SKILL.md`

Name this sdk-conventions. Description: TypeScript SDK design rules for Bharat Data. Rules: Zero any types in any public-facing function signature, parameter, or return type. Use camelCase for all method names, parameter keys, and response keys. Every API response must include three top-level keys: data (array of records), meta (sourceUrl, collectionDate, recordCount, confidence, bharatDataVersion), and query (echo of request parameters). Maximum 5 public functions in v1. Follow the signatures defined in docs/sdk/DESIGN.md exactly. Custom error

classes for all error conditions: BharatDataError, SourceNotFoundError, InvalidQueryError, RateLimitError, DataUnavailableError.

Skill 4 — Data Quality skill at `.agents/skills/data-quality/SKILL.md`

Name this data-quality. Description: Data quality standards for the Bharat Data pipeline. Rules: Every record must have a confidence score, source URL, and collection date. Records with confidence below 0.5 are stored in the database but excluded from API responses and flagged for manual review. Never serve unvalidated data through the API. Wrong data with an explicit low-confidence flag is acceptable. Wrong data served confidently is a critical failure. All numeric values stored as numbers, never strings. All dates in ISO 8601. Null values represented as null, never zero, empty string, or dash.

Skill 5 — Project Context skill at `.agents/skills/project-context/SKILL.md`

Name this project-context. Description: Overall Bharat Data project context and constraints. Rules: All infrastructure must use free tiers only. Neon for PostgreSQL. Vercel or Cloudflare Workers for API hosting. GitHub Actions for cron jobs. No authentication or user accounts in v1. No AI natural language queries in v1. MIT License. Scrapers must never exceed one request per second to any government server. The project is an open source public good, not a commercial product. Data quality matters more than feature count. One perfectly reliable dataset is better than five unreliable ones.

These five skills mean every agent session in every phase automatically follows your established rules. You will never need to re-explain your quality bar, naming conventions, or infrastructure constraints. The agent reads these skills at the start of every session and applies them to every decision.

Updated Execution Strategy — Phase by Phase

Phase 0 — Clarity Before Code

Goal: Make three critical decisions before touching your keyboard. **Duration:** 3 days **Failure Protocol:** If indecision blocks progress past Day 3, default to NCRB dataset and proceed. **Antigravity Mode:** Editor View only. Do not use Manager View agents for this phase.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Choose one primary dataset	Not Started	You	None	No AI needed. Your decision.
Define written quality bar	Not Started	You	Dataset selected	Use AI sidebar to review your quality bar for completeness
Set up public build log	Not Started	You	Quality bar defined	No AI needed. Manual post on your chosen platform
Initialize GitHub repository	Not Started	You	Dataset selected	Use Manager View once to scaffold the monorepo structure

Day 1 — Dataset Selection and Quality Bar

Choose NCRB Crime in India as your primary dataset unless your passive research in Phase 1 changes this. Write your quality bar on paper or in a text file. Every data point must include source URL, collection date, and confidence score. State names normalized to canonical forms. Numbers verified against original source. A journalist can trace any number back to the original PDF page within two minutes.

Do not use the AI agent for this. These are judgment calls that require your thinking, not generated text.

Day 2 — Public Build Log

Make your first public post on Twitter, LinkedIn, or your blog. Explain what you are building and why. This is your accountability mechanism. Do not use the AI to write this post. Write it in your own voice. Authenticity matters more than polish for a first post.

Day 3 — Repository Initialization

Now use Antigravity Manager View for the first time. Give it this prompt:

“Create a monorepo structure for Bharat Data using Turborepo with the following packages: packages/pipeline (Python, for data ingestion and parsing), packages/sdk (TypeScript, publishable to npm as bharatdata), packages/api (TypeScript, Hono framework on Cloudflare Workers), packages/web (Next.js, for playground and documentation site). Add a docs folder with subfolders for architecture, sdk, data-sources, data-schema, ai, brand, legal, and internal. Add a data folder for canonical-mappings.json. Add standard files: README.md, LICENSE (MIT), CONTRIBUTING.md stub, CODE_OF_CONDUCT.md stub, SECURITY.md stub, CHANGELOG.md stub. Initialize git. Do not write any application code. Only create the structure with placeholder files.”

Review the generated structure against your documentation index. Every document should have a clear home. Commit this structure to GitHub.

Phase 0 Exit Condition: Dataset chosen. Quality bar written. First public post live. GitHub repo initialized with correct structure. Skills folder created with all five skills. MCP servers installed and verified working.

Phase 1 — Passive Research and Validation

Goal: Validate demand through documented research without cold outreach. **Duration:** 2 days **Failure Protocol:** If research yields conflicting signals, stick with NCRB and let real usage guide iteration. **Antigravity Mode:** Editor View with AI sidebar for research organization only.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Scan developer and open data communities	Not Started	You	Phase 0 complete	Use Brave Search MCP to find recent discussions
Review journalism	Not Started	You	Community scan done	Manual browser

Milestone	Status	Owner	Dependencies	Antigravity Usage
and academic sources				research. AI summarizes findings.
Document pain points and exact language	Not Started	You	Sources reviewed	Use AI sidebar to organize notes into categories
Confirm or adjust dataset choice	Not Started	You	Documentation complete	Your decision. No AI delegation.

Day 1 — Community and Source Scanning

Open your browser and read. Search Twitter for “NCRB data”, “government data India”, “data.gov.in frustration”. Search r/developersIndia for posts about government data. Browse DataMeet GitHub repositories to see which datasets they have attempted to clean. Read methodology sections of recent Google Scholar papers citing Indian government statistics.

If Brave Search MCP is working, you can ask the agent to search for “developers complaining about Indian government data access 2024 2025” and summarize findings. But verify every claim the agent surfaces by checking the original source yourself.

Day 2 — Documentation and Decision

Compile your findings into a one-page document. Use the AI sidebar in Editor View to help organize your raw notes into categories: most-mentioned datasets, most-described pain points, exact quotes that describe the frustration. This organized output feeds your README copy and blog post later.

Confirm or adjust your dataset choice. This is your decision, not the agent’s.

Phase 1 Exit Condition: Research documented. Pain points catalogued with exact language from real people. Dataset choice confirmed. You have specific quotes and complaints that prove the problem exists without needing formal interviews.

Phase 2 — Build the Hardest Thing

Goal: One dataset, reliably ingested, with a working SDK on npm and a live playground. **Duration:** 4 weeks maximum (hard deadline) **Failure Protocol:** If parser handles fewer than 10

tables reliably by Week 4, ship working subset with transparent documentation and iterate post-launch. **Antigravity Mode:** Manager View with Planning mode for all pipeline components. Parallel agents for independent tasks.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Create Brand Minimums Checklist (Doc 08a)	Not Started	You	Phase 1 complete	Editor View. Write yourself.
Add MIT License file (Doc 12)	Not Started	You	Phase 1 complete	Manager View. 1 minute task.
Start Project Journal (Doc 00a)	Not Started	You	Phase 2 starts	Editor View. Manual. 5 min per session.
Write SDK Design Specification (Doc 05)	Not Started	You	Phase 1 complete	AI sidebar to review, not to write.
Build the data ingester	Not Started	You	Doc 05 complete	Manager View + Sequential Thinking MCP
Build the PDF parser	Not Started	You	Ingester working	Manager View + Claude Opus + Sequential Thinking
Build the normalizer and reference file	Not Started	You	Parser producing raw records	Manager View + Filesystem MCP
Write Data Schema Guide (Doc 06)	Not Started	You	Normalizer working	AI sidebar to draft from your decisions
Draft Content Strategy (Doc 19)	Not Started	You	Week 2	Editor View. AI assists with blog

Milestone	Status	Owner	Dependencies	Antigravity Usage
				post outline.
Set up Postgres database schema	Not Started	You	Normalizer working	Manager View + PostgreSQL MCP
Add data freshness monitoring	Not Started	You	Database live	Manager View. GitHub Actions cron.
Implement legal minimums	Not Started	You	Schema complete	Editor View. Manual additions.
Publish TypeScript SDK to npm	Not Started	You	Monitoring and legal in place	Manager View for boilerplate. Manual review of types.
Deploy REST API and web playground	Not Started	You	SDK published	Manager View for scaffolding. Manual testing.
Write README (Doc 09)	Not Started	You	Playground live	AI drafts. You heavily edit for voice and accuracy.
Write Data Source Registry (Doc 04)	Not Started	You	First source complete	AI drafts from your parser notes. You verify.
Write Architecture Document (Doc 03)	Not Started	You	Architecture stable	AI drafts from Journal. You verify every claim.
Write Testing Strategy (Doc 03a)	Not Started	You	Tests established	AI drafts. You verify against actual test setup.

Week 1 — Foundation and Ingester

Day 1 setup tasks. Create Doc 08a Brand Minimums in Editor View. Write it yourself in twenty minutes. Name format: Bharat Data in prose, bharatdata in code. Tagline: Clean, queryable Indian public data. Disclaimer: Bharat Data is an independent open source project not affiliated with any government body. Add the MIT License file. Start your Project Journal file at docs/internal/JOURNAL.md.

Write Doc 05 SDK Design Specification. Use the AI sidebar to review your function signatures for consistency but write the design decisions yourself. This document is your API contract and it must reflect your thinking, not the agent's defaults.

Day 2-3 build the ingester. Open Manager View. Give this prompt with Sequential Thinking MCP active:

"Build a data ingester for Bharat Data. The ingester must visit the NCRB Crime in India report page, detect if new data is available by comparing file hashes, download the PDF report, and store the raw file with metadata including source URL, download timestamp, file hash, and format type. The ingester must be fault tolerant. It must never crash silently. Every failure must be logged with enough detail to diagnose the next morning. It must respect rate limiting of maximum one request per second. It must handle the case where the NCRB website is temporarily down by retrying with exponential backoff up to 3 attempts. Read the Skills in .agents/skills/ before starting."

The agent will generate a task list and implementation plan as artifacts in Planning mode. Review every item. Check that rate limiting is included. Check that error logging is specific not generic. Check that the retry logic has a maximum attempt count. Only after you approve the plan does the agent write code.

After the agent produces the ingester code, test it manually. Run it against the actual NCRB website. Verify that it downloads the correct PDF. Verify that the metadata is stored correctly. Verify that running it twice does not duplicate the download. Write in your Project Journal what happened.

Day 4-5 build the PDF parser. This is the hardest component in the entire project. Switch your model to Claude Opus 4.6 explicitly for this task. Give this prompt with Sequential Thinking MCP active:

"Build a PDF parser for NCRB Crime in India report. Before writing any code, analyze the structure challenges: tables with headers spanning multiple rows, tables continuing across pages without repeated headers, footnotes embedded inside data cells, Hindi text mixed with English, state names in inconsistent formats, numbers using Indian formatting with lakhs and crores.

Use pdfplumber. Build a confidence scoring system where every extracted row gets a score between 0.0 and 1.0. Never output wrong data silently. If a cell cannot be parsed confidently, assign low confidence and flag it. Read the Skills before starting.”

The agent will produce a plan. This plan will likely underestimate the difficulty. That is expected. Review it and add these explicit requirements: handle the case where a table header uses merged cells, handle the case where a footnote marker appears inside a numeric cell, handle the case where page breaks split a table mid-row.

After the code is generated, test it manually against the actual NCRB PDF. Pick 20 random cells across different tables. Compare the parser output against the original PDF values. Record accuracy. If below 95 percent, feed the specific failures back to the agent with the exact table, page, and cell that failed. Iterate until accuracy meets your threshold or until you have documented which tables work and which need more work.

Write detailed notes in your Project Journal. These notes become your first blog post and your portfolio case study later.

Week 2 — Normalizer, Schema, and Content Planning

Day 1-2 build the normalizer. Use Manager View with Filesystem MCP active:

“Build the Bharat Data normalizer. Create a canonical-mappings.json file at data/canonical-mappings.json containing all known variations of Indian state names mapped to canonical forms. Include at minimum: full names, common abbreviations, all-caps variants, and names with and without diacritical marks. Build the normalizer module that reads this file and resolves any input state or district name to its canonical form. When the normalizer encounters an unrecognized name, it must not silently drop the record. It must log the unmapped name to data/unmapped-entities.json and pass the record through with a low confidence flag. Handle Telangana bifurcation: pre-2014 data attributed to unified Andhra Pradesh, post-2014 data separated. Read the Skills before starting.”

Run a parallel agent simultaneously to begin writing Doc 06 Data Schema and Normalization Guide based on the normalizer design you just approved.

Day 3 draft your Content Strategy (Doc 19). Switch to Editor View. Use the AI sidebar to help outline your first blog post based on the parsing challenges you encountered in Week 1. The title should be something like “I Tried to Extract Clean Crime Data

From NCRB PDFs and Here Is What Broke.” Outline the structure now. Draft the opening paragraphs while the pain is fresh and specific. You will finish and polish this post before Phase 3 launch.

Day 4-5 set up the database. Use Manager View with PostgreSQL MCP active:

“Design and create the PostgreSQL schema for Bharat Data on Neon. The schema must support multiple datasets in the future without restructuring. Every record must store: canonical data fields, raw source reference URL, collection date, confidence score, normalizer version that processed it, and ingestion batch ID. Create indexes for the most common query patterns: by state, by year, by category, and by state plus year combination. The schema must fit within Neon’s free tier 500MB storage limit. Read the Skills before starting.”

After the schema is created, use the PostgreSQL MCP to verify the table structure directly from the IDE. Run the full pipeline manually: ingester downloads PDF, parser extracts records, normalizer cleans them, records are inserted into the database. Query the database and spot-check 10 records against the original PDF.

Week 3 — SDK, API, Monitoring, and Legal

Day 1-2 build data freshness monitoring. Use Manager View:

“Create a GitHub Action workflow that runs daily at 6 AM UTC. It checks if the NCRB data source URL is still accessible. It runs the parser against the most recently downloaded PDF and compares output structure against the expected schema. If the URL returns a non-200 status or the parser output structure has changed, it automatically opens a GitHub issue with the label data-source-alert including the specific failure details. Read the Skills before starting.”

Day 2 implement legal minimums. This is a manual task in Editor View. Add source attribution fields to every database record if not already present. Create a simple terms page. Add rate limiting to your scraper configuration file ensuring maximum one request per second. These are small additions but they must exist before any public launch.

Day 3-4 publish the TypeScript SDK. Use Manager View with Gemini 3 Pro (this is structured boilerplate work where speed matters):

“Build the Bharat Data TypeScript SDK following the design specification in docs/sdk/DESIGN.md exactly. The package name is bharatdata. Implement exactly these five functions: getByState, getByYear, getByCategory, compare, and search. All under the

bd.crime namespace. Every function must return the standard response shape with data, meta, and query keys. Zero any types anywhere in the public API. Include custom error classes: BharatDataError, SourceNotFoundError, InvalidQueryError, RateLimitError, DataUnavailableError. Create a package.json ready for npm publishing. Read the Skills before starting."

After generation, manually inspect every TypeScript type definition. Run the TypeScript compiler in strict mode. If any any types exist, fix them before publishing. Test every function against your live database. Verify that response shapes match Doc 05 exactly. Then publish to npm.

Day 5 deploy the REST API and playground. Run two parallel agents. First agent builds the API:

"Build a REST API using Hono framework deployable to Cloudflare Workers. Create endpoints that mirror the SDK functions: GET /api/crime/state/:state, GET /api/crime/year/:year, GET /api/crime/category/:category, GET /api/crime/compare, GET /api/crime/search. Add global rate limiting. Add CORS headers. Every response includes source attribution. Read the Skills before starting."

Second agent builds the playground simultaneously:

"Build a minimal web playground for Bharat Data using Next.js. Single page. Text input at the top. Results displayed as a table below. CSV download button. No login. No signup. No authentication. A visitor must be able to type a query and see real data within 30 seconds of landing on the page. Read the Skills before starting."

Test both manually. Visit the playground URL. Type a query. Verify data appears. Download the CSV. Open it in a spreadsheet. Verify the numbers match your database.

Week 4 — Documentation Sprint and Quality Gate

This week you write the remaining Phase 2 documents and perform your final quality check before Phase 3.

Run three parallel agents for documentation.

Agent 1 drafts Doc 09 README using your brand minimums, SDK design spec, and current project state as context. You will heavily edit this draft for voice, accuracy, and the 3-second/30-second/2-minute reading framework.

Agent 2 drafts Doc 04 Data Source Registry from your parser implementation notes and Project Journal entries about the NCRB source.

Agent 3 drafts Doc 03 Technical Architecture Document from your Project Journal, your actual code structure, and your infrastructure setup.

You write Doc 03a Testing Strategy yourself based on how you have actually been testing throughout Weeks 1-3. The agent can help format it but the testing approach must reflect your real practice not a theoretical framework.

Quality gate before leaving Phase 2. Give your playground URL to someone who knows nothing about Bharat Data. Watch them use it without helping. If they get confused, that is a UX bug. If they get wrong data, that is a critical bug. If they say “that is actually useful” without prompting, you are ready.

Phase 2 Exit Condition: npm package published and installable. Playground live at a public URL. One complete dataset in the database with normalized records, source attribution, and confidence scores. Ingester runs automatically. Monitoring creates GitHub issues on source changes. All Phase 2 documents written. README reviewed by at least one person who is not you.

Phase 3 — Real Users and Early Distribution

Goal: 25 real users who found you organically. **Duration:** 4 weeks **Failure Protocol:** If fewer than 25 users after 4 weeks, publish a second technical post with different angle and test new distribution channels before advancing. **Antigravity Mode:** Parallel agents for documentation. Editor View for content writing.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Write Contributing Guide (Doc 10)	Not Started	You	Phase 2 complete	Manager View parallel agent
Add Code of Conduct (Doc 11)	Not Started	You	Phase 2 complete	Manager View. 5 minute task.
Write Security Policy (Doc 13)	Not Started	You	Phase 2 complete	AI drafts. You add incident response section.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Create Changelog (Doc 17)	Not Started	You	First release tagged	Manager View generates from git history
Write Terms of Use (Doc 14)	Not Started	You	Playground live	AI drafts. You review for accuracy.
Write Privacy Policy (Doc 15)	Not Started	You	Playground live	AI drafts. You verify data collection claims.
Write Attribution and Legal (Doc 16)	Not Started	You	Phase 2 complete	AI drafts. You verify government license details.
Create Brand Guidelines (Doc 08)	Not Started	You	Brand minimums exist	AI drafts. You define voice and tone.
Contact 5 public data complainers	Not Started	You	Playground live	No AI. Manual outreach.
Publish problem-first blog post	Not Started	You	Post drafted in Phase 2	Editor View to polish. You write final version.
Post in DataMeet and dev communities	Not Started	You	Blog post live	No AI. Manual community engagement.
Submit Show HN launch	Not Started	You	Community posts live	No AI. Write your own HN post.
Create Public Roadmap (Doc 18)	Not Started	You	Initial traction visible	AI generates from your internal strategy
Hit 25 organic users	Not Started	Community	All distribution active	No AI. Organic growth.
Write Portfolio	Not Started	You	Traction data available	AI drafts from Journal. You

Milestone	Status	Owner	Dependencies	Antigravity Usage
Case Study (Doc 20)				verify metrics.

Week 1 Day 1 — Documentation Blitz

This is the most efficient use of parallel agents in the entire project. These documents are independent of each other and can be generated simultaneously.

Launch five parallel agents in Manager View using Gemini 3 Pro for all of them since these are structured documents not complex reasoning tasks.

Agent 1: “Generate a CONTRIBUTING.md file for Bharat Data. Include local development setup instructions, bug report template, data source suggestion template, canonical mappings contribution guide, code change submission process, and commit message standards. Read the project README and Skills for context.”

Agent 2: “Generate a SECURITY.md file for Bharat Data. Include vulnerability reporting process via email, 48-hour acknowledgment commitment, supported versions, and an incident response protocol covering detection of bad data, immediate response within 4 hours, user notification, remediation, and public post-mortem within one week.”

Agent 3: “Generate Terms of Use for Bharat Data. Plain language, not legal jargon. Cover: data is aggregated from public government sources, should be verified for critical decisions, service provided as-is, rate limits exist, attribution required, project is independent from government.”

Agent 4: “Generate a Privacy Policy for Bharat Data. Plain language. Cover: API usage analytics collected, playground queries retained for 6 months for improvement, no personal information collected, no cookies beyond essential, no third-party sharing.”

Agent 5: “Generate Data Attribution and Legal Compliance documentation for Bharat Data. Cover: attribution standards for every data point, overview of GODL India license, scraping ethics with rate limiting policy, and independence disclaimer.”

Review all five outputs. Verify that the Terms of Use and Privacy Policy accurately reflect what you actually collect and how your system works. Verify that the Attribution document correctly describes government licensing. These documents must be factually correct because they create legal obligations.

Add Code of Conduct using the Contributor Covenant. One agent, two minutes.

Generate the first Changelog entry from your git history using the GitHub MCP.

Create Doc 08 Brand Guidelines by expanding your Brand Minimums Checklist. Add voice and tone section, logo specifications, and color choices.

Week 1-2 — Content and Distribution

Finish polishing your blog post that you drafted during Phase 2 Week 2. Use Editor View with the AI sidebar to improve clarity and flow but write the final version yourself. Your authentic voice describing real frustration with government PDFs is more compelling than any AI-polished prose.

Find five developers or journalists who have publicly complained about Indian government data access on Twitter or Reddit in the last six months. Reply to their posts or DM them with your live playground link. These are warm contacts because they have already expressed the exact pain you solve.

Publish the blog post on Dev.to, Medium, or Hashnode. Share on Twitter with screenshots showing messy PDF versus clean Bharat Data output. Post in r/developersIndia with genuine framing, not promotional language. Share in DataMeet community channels.

Week 2-3 — Show HN and Community

Submit your Hacker News Show HN post. Write the title and first comment yourself. Do not use AI for this. Authenticity matters on Hacker News and AI-generated posts are increasingly recognized and dismissed.

Reach out to two college hackathon organizers to offer Bharat Data as a data resource for participants.

Week 3-4 — Roadmap and Case Study

Create Doc 18 Public Roadmap as a ROADMAP.md file. Use the AI to generate the initial structure from your internal execution strategy but remove all dates and failure protocols. The public version shows direction without commitments.

Write Doc 20 Portfolio Case Study. Give your Project Journal to the agent and ask it to draft a case study following the structure defined in your documentation index. Then heavily edit the output because the specific details and authentic reflections are what make a case study compelling. The agent provides structure but you provide substance.

Phase 3 Exit Condition: Twenty-five organic users. At least two GitHub issues from strangers. At least one real-world usage of your data. All Phase 3 documents published. Blog post live and shared. Show HN submitted.

Phase 4 — Add AI Intelligence Layer

Goal: Add GenAI on top of a product with real users and real feedback. **Duration:** 3-4 weeks **Failure Protocol:** If AI returns incorrect data more than 15 percent of the time, disable auto-execution and add explicit confirmation steps until accuracy improves. **Antigravity Mode:** Manager View with Claude Opus 4.6 for all AI architecture decisions. Planning mode mandatory.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Build natural language query engine	Not Started	You	Phase 3 complete	Manager View + Claude Opus + Sequential Thinking
Add data narration layer	Not Started	You	NL engine working	Manager View + Claude Opus
Build anomaly detection system	Not Started	You	Narration live	Manager View + Sequential Thinking
Add second dataset based on demand	Not Started	You	Demand signal clear	Manager View. Full pipeline repeat.
Write AI Layer Specification (Doc 07)	Not Started	You	AI layer live	AI drafts from implementation. You verify.
Write AI technical deep-dive blog post	Not Started	You	Doc 07 complete	Editor View. You write. AI reviews.

Week 1-2 — Natural Language Query Engine

Switch to Claude Opus 4.6 explicitly. This is the most architecturally complex component in Bharat Data and Gemini 3 Pro will take shortcuts that cost you quality in prompt engineering and edge case handling.

Use Manager View with Sequential Thinking MCP active:

“Build a natural language query engine for Bharat Data. The engine receives a plain English question like ‘which state had the highest murder rate in 2022’ and translates it into a structured database query. Use the Vercel AI SDK with your chosen LLM provider. The system prompt must understand Indian civic data terminology: IPC versus SLL crimes, state name variations, that Telangana data only exists from 2014 onward, that UP means Uttar Pradesh. Build graceful failure: if the query cannot be understood, return a clear message saying so instead of guessing incorrectly. Never return wrong data confidently. If the AI is uncertain about interpretation, ask for clarification. Read the Skills before starting.”

After generation, create a test suite of 30 natural language queries ranging from simple (“crime in Maharashtra 2022”) to complex (“compare cyber crime trends across top 5 states over last 3 years”) to ambiguous (“which state is most dangerous”). Test every query. Record which ones succeed, which fail, and why they fail. Keep a query failure log. This log becomes your most important AI improvement tool and the basis for your technical blog post.

Week 2-3 — Narration and Anomaly Detection

Build the data narration layer that generates two to three sentence human-readable summaries for query results. Build the anomaly detection system with conservative thresholds. Both use Manager View with Claude Opus.

Week 3-4 — Second Dataset and Documentation

Add the second dataset based on what users requested in GitHub issues and playground queries. Run the full pipeline again: ingester, parser, normalizer, database insertion, SDK function addition, API endpoint, playground update. This is a complete repeat of Phase 2’s pipeline work but faster because your infrastructure exists.

Write Doc 07 AI Layer Specification. The agent drafts from your implementation and your query failure log. You verify every technical claim against how the system actually behaves.

Write your AI technical blog post in Editor View. This post covers your prompt engineering decisions, your failure modes, and what you learned about building domain-specific AI. Write it yourself with AI sidebar review for clarity.

Phase 4 Exit Condition: Natural language queries work reliably for your primary dataset. Data narration is live. Anomaly detection is active with conservative thresholds. Second dataset is in the system. AI Layer Specification is published. Technical blog post is live.

Phase 5 — Follow the Signal and Scale

Goal: Stop directing and start responding to real usage patterns. **Duration:** Ongoing **Failure Protocol:** If engagement plateaus for 60 days, pause new features and conduct retrospective. **Antigravity Mode:** Mixed. Manager View for new features. Editor View for community management.

Milestone	Status	Owner	Dependencies	Antigravity Usage
Product Hunt launch	Not Started	You	Phase 4 complete	No AI. Write your own maker story.
Identify top user segment	Not Started	You	Sufficient usage data	AI helps analyze usage patterns if data is available
Double down on top use case	Not Started	You	Segment identified	Manager View for feature development
Open first sustainability conversation	Not Started	You	Strong engaged base	No AI. Personal conversations.

Product Hunt launch happens now, not earlier. You have real users, real feedback, a polished AI layer, and a complete documentation suite. Write your maker story yourself. Respond to every comment personally.

From this point forward, use Antigravity Manager View for new feature development based on user requests. Use parallel agents to handle multiple feature requests simultaneously when they are independent. Continue using your Project Journal. Continue the quality gate of manual spot-checking after every pipeline change.

Phase 5 has no exit condition. This is called maintaining a product.

The Vibe Coding Session Pattern

Every build session in Antigravity follows this exact sequence regardless of which phase you are in.

Step 1 Context Load. Start every Manager View session by telling the agent: “Read the Skills in .agents/skills/, read docs/internal/PRD.md, and read the current state of the component I am working on.” This takes thirty seconds and prevents the agent from contradicting your established decisions.

Step 2 One Feature Per Session. Never ask the agent to build multiple pipeline components in a single session. One session, one component. Ingester is one session. Parser is one session. Normalizer is one session. Mixing them produces code that works individually but integrates poorly.

Step 3 Review Artifacts Before Execution. In Planning mode the agent generates task lists and implementation plans before writing code. Read every item. If anything contradicts your PRD, your quality bar, or your Skills, comment on the artifact. The agent incorporates your feedback without restarting.

Step 4 Parallel Agents for Independent Tasks. When two tasks share no dependencies, run them as parallel agents. Parser and its unit tests can run in parallel. Documentation for different documents can run in parallel. API endpoints and playground UI can run in parallel. Never parallelize tasks that depend on each other.

Step 5 Manual Testing After Every Session. Run the output yourself. For pipeline components, check 10 random data points against the original source. For SDK functions, call each one and verify the response shape. For the playground, use it as a real user would. Do not trust the agent’s claim that tests pass. Verify yourself.

Step 6 Commit to GitHub. After every successful session, commit and push. This gives you a clean rollback point if a future session introduces a regression. GitLens extension helps you track which session introduced which change.

Step 7 Journal Entry. Five minutes. What the agent decided. What you accepted or rejected. What surprised you. What broke. What you learned. This feeds every document you write later.

Summary

Phase	Milestones	Duration	Antigravity Mode	Primary Model
IDE Setup	N/A	Day 0	Configuration	N/A
Phase 0	4	3 days	Editor View mostly	Your brain
Phase 1	4	2 days	Editor View with sidebar	Gemini Pro for research
Phase 2	18	4 weeks	Manager View heavy	Claude Opus for pipeline. Gemini Pro for boilerplate.
Phase 3	15	4 weeks	Parallel agents for docs. Editor for content.	Gemini Pro for docs. Your voice for content.
Phase 4	6	3-4 weeks	Manager View with Claude Opus	Claude Opus for all AI work
Phase 5	4	Ongoing	Mixed based on task	Match model to task complexity
Total	**51**	**~3.5-4 months**		

The single rule remains unchanged. Finish each phase completely before starting the next. Antigravity makes you faster. It does not make you immune to the consequences of skipping steps. Use it as your accelerator. Keep yourself as the quality gate.

BharatData — Complete Knowledge Base Files

Below are all the markdown files you need to create in your project. I am giving you the exact file path and the complete content for each file. Create them in order.

File 1: `.agents/FEEDING_PROMPT.md`

This is the master prompt you paste into Antigravity at the start of every session.

```md
# BharatData — Master Context Prompt

You are working on BharatData, an open source TypeScript SDK and
        REST API that makes Indian government public data
        accessible, clean, and queryable.

## Before doing anything, read these files in order:

1. `.agents/PROJECT_CONTEXT.md` — Complete project overview,
        goals, constraints, and audience
2. `.agents/PROBLEM_DEFINITION.md` — The exact problem we are
        solving and why it matters
3. `docs/internal/PRD.md` — Product Requirements Document with v1
        scope, acceptance criteria, and explicit exclusions
4. `docs/internal/QUALITY_STANDARDS.md` — Non-negotiable quality
        rules for every component
5. `docs/sdk/DESIGN.md` — SDK API surface design, function
        signatures, naming conventions
6. `docs/internal/DATA_PIPELINE_SPEC.md` — Pipeline architecture
        for ingestion, parsing, normalization, storage
7. `docs/data-schema/NORMALIZATION.md` — How raw data becomes
        clean data, canonical mappings, entity resolution
8. `docs/data-sources/NCRB_CRIME.md` — First data source details,
        known challenges, parsing requirements
9. `docs/internal/BRAND_MINIMUMS.md` — Name format, tagline,
        disclaimer language
10. Read all Skills in `.agents/skills/
        ` — Project-wide rules that apply to every task

## Critical Rules:

- ALWAYS use Planning mode. Generate task list and implementation
        plan before writing code.
- NEVER use Fast mode for any BharatData task.
- NEVER introduce `any` types in TypeScript. Zero tolerance.
- NEVER silently produce wrong data. Wrong data with a low-
        confidence flag is acceptable. Wrong data served
        confidently is a critical failure.
- NEVER exceed one request per second to any government server.
- NEVER add features outside v1 scope defined in PRD.md without
        explicit approval.
- ALWAYS include source attribution (sourceUrl, collectionDate,
        confidence) on every data record.
- ALWAYS read the relevant Skill file before starting a task.
- ALWAYS commit to GitHub after completing each task successfully.

## Models to use:
```

- Claude Opus 4.6 — for pipeline logic, parser code, architecture decisions, AI layer, anything complex
- Gemini 3 Pro — for boilerplate, documentation drafting, SDK scaffolding, test generation
- Gemini 3 Flash — for inline completions, quick edits, formatting fixes

Current Phase:

Update this line as you progress:
Currently in: Phase 0 — Clarity Before Code

File 2: .agents/PROJECT_CONTEXT.md

BharatData — Complete Project Context

What is BharatData

BharatData is an open source project that makes Indian government public data accessible, clean, and queryable through a TypeScript SDK, REST API, and web playground.

The npm package name is `bharatdata`. The project name in prose is "Bharat Data" (two words, both capitalized). In code and package references it is always `bharatdata` (one word, lowercase).

Why BharatData Exists

India has over 600,000 public datasets spread across hundreds of government websites including data.gov.in, NCRB, RBI, Census, CPCB, Agmarknet, and many more. This data is practically unusable because:

- Data is locked in scanned PDFs, broken CSVs, and inconsistent HTML tables
- No unified API exists across government data sources
- No SDK exists for developers to query Indian public data programmatically
- Column names and formats change between years and reports
- State and district names are spelled differently across sources
- No documentation exists for most data sources
- Government websites change structure frequently, breaking any custom scrapers

Developers spend weeks scraping and cleaning before using a single data point. Journalists spend months collecting statistics that should take minutes. Researchers waste the first six months of their projects just getting data into usable format.

Who BharatData is For

Primary Audiences

Developers building applications with Indian data — Fintech developers needing demographic data for credit scoring. AgriTech startups needing crop price data. InsurTech companies needing crime and disaster data for risk modeling. Any developer whose application requires structured Indian government statistics.

Data journalists and media professionals — Reporters at IndiaSpend, The Wire, Scroll, Article 14, NDTV, and international outlets covering India. They need verified government statistics for data-driven stories and currently spend enormous time collecting and cleaning data manually.

Academic researchers — PhD students and professors at IITs, IIMs, JNU, ISI, and international universities studying Indian economics, public health, environmental science, or social issues. They spend the first months of research projects just collecting and normalizing data.

Students and hackathon participants — Millions of CS students who need real-world data for projects, competitions, and learning. Currently using fake datasets because real government data is too hard to access.

Government officials and policy makers — District collectors, state planning boards, and organizations like NITI Aayog who produce data but struggle to query across their own datasets.

How Each Audience Uses BharatData

Developers install the npm package and call functions like `bd.crime.getByState('Maharashtra', 2023)` to get clean structured data in their applications.

Journalists visit the web playground, type a question in plain English, and get verified data with source citations they can use in their reporting without writing code.

Researchers use the SDK or playground to access pre-cleaned datasets, download them as CSV, and cite BharatData as their data source.

Students install the package and have immediate access to real Indian data for their ML projects, visualization assignments, and hackathon submissions.

Project Goals

Primary Goals (in priority order)

1. Build something genuinely useful for the developer community and society
2. Create a high-quality open source project that demonstrates professional engineering skills
3. Contribute meaningfully to the AI industry through domain-specific data and AI applications
4. Build a portfolio piece that showcases data engineering, API design, TypeScript, and AI integration skills

What This Project is NOT

- This is NOT a startup chasing revenue from day one
- This is NOT an enterprise product with paid tiers (yet)
- This is NOT trying to cover all 600,000 government datasets immediately
- This is NOT affiliated with or endorsed by any Indian government body
- This is NOT a commercial product. It is an open source public good.

Builder Context

The builder is a student and developer who is not financially dependent on this project. The primary motivation is building for societal impact and developer community contribution, not revenue. Monetization will be explored later if the project gains traction, but it is explicitly not a v1 concern.

The builder uses vibe coding methodology with AI-assisted development through Google Antigravity IDE.

Technical Constraints

- All infrastructure must use free tiers only
- Neon for PostgreSQL (free tier: 500MB storage)
- Vercel or Cloudflare Workers for API hosting (free tier)
- GitHub Actions for cron jobs and CI/CD (free tier)
- Cloudflare R2 for file storage if needed (free tier: 10GB)
- Single developer, no team
- MIT License for maximum adoption
- Scrapers must never exceed 1 request per second to any government server

Success Metrics

Phase 3 (Launch) targets:
- 25+ organic users (not from personal outreach)
- 2+ GitHub issues from strangers
- 1+ real-world usage (article, project, or analysis using BharatData data)

6-month targets:
- 5+ real projects or publications using BharatData data
- 20+ weekly active SDK/API users
- Regular playground usage from non-developer audience
- At least one spontaneous recommendation (word of mouth)

12-month targets:
- 500+ GitHub stars
- 100+ weekly npm downloads
- Recognized in Indian open data community
- Multiple real-world citations in published work

File 3: `.agents/PROBLEM_DEFINITION.md`

BharatData — Problem Definition

The Core Problem

Indian government public data exists in abundance but is practically unusable for developers, journalists, researchers, and citizens.

Specific Pain Points

Pain Point 1: Data is Locked in Hostile Formats

The NCRB publishes its annual Crime in India report as a PDF containing roughly 500 tables across 400+ pages. Table headers are inconsistent year to year. Some tables span two pages with the header only on the first page. Some cells contain footnotes embedded inside the data. Some numbers use Indian number formatting with lakhs and crores. Some tables are in Hindi. The 2020 report was delayed due to COVID and has structural differences from surrounding years.

The Census publishes data in a mix of Excel files, PDFs, and HTML tables. Column names change between census years. Geographic coding systems differ from other government data sources.

CPCB publishes air quality data through a web portal that uses JavaScript rendering, making it resistant to simple scraping. Historical data is in PDFs.

Agmarknet publishes daily mandi crop prices in HTML tables that change structure periodically and use session-based navigation that breaks automated access.

Pain Point 2: No Unified Access Layer

Each government data source has its own website, its own format, its own update schedule, and its own quirks. A developer building an application that needs crime data AND demographic data AND agricultural prices must build three separate scrapers, three separate parsers, and three separate normalization systems. There is no unified API, no SDK, no single point of access.

Pain Point 3: Entity Resolution Nightmare

The same state is called "AP", "Andhra Pradesh", "ANDHRA PRADESH", "A.P.", and "Andhra" across different sources. Districts are spelled differently. Districts are bifurcated (one becomes two) without consistent mapping. Telangana did not exist before 2014. Ladakh became a separate entity in 2019. A dataset from 2010 might list 640 districts while 2023 lists 780 districts, and there is no standard mapping between old and new boundaries.

Pain Point 4: No Quality Assurance

Government data sources contain errors. Numbers that do not add up. Categories that change definitions between years. Missing data filled with dashes, zeros, or blank cells inconsistently. There is no metadata about data quality, no confidence scoring, no indication of which numbers are reliable and which are questionable.

Pain Point 5: No Documentation

Most government data portals have no API documentation, no data dictionaries, no schema definitions, no changelog of when data was updated or what changed. A developer must reverse-engineer the data structure by examining the files directly.

Who Feels This Pain

Developers: "I spent three weeks building a scraper for NCRB data. It broke the next month when they redesigned the website. I gave up and used estimates instead."

Journalists: "I need crime statistics for a story due tomorrow. The data exists on the NCRB website but it is in a 400-page PDF and I need numbers from five different tables cross-referenced by state."

Researchers: "The first six months of my PhD were spent just collecting and cleaning census data. My actual research did not begin until month seven."

Students: "I want to use real Indian data for my machine learning project but I cannot access it without spending weeks on data collection. I ended up using a Kaggle dataset about housing prices in Boston instead."

These quotes are synthesized from public complaints found on Twitter, Reddit (r/developersIndia), DataMeet community discussions, and academic paper methodology sections. They represent documented, recurring frustration.

What a Solution Looks Like

A developer installs a package with `npm install bharatdata`, calls `bd.crime.getByState('Maharashtra', 2023)`, and gets back clean, typed, attributed data in milliseconds. No scraping. No parsing. No cleaning. No normalization. Just data.

A journalist visits a web playground, types "which state had the highest cyber crime increase between 2020 and 2023", and gets back a clear answer with source citations, ready to use in their article.

A researcher downloads five years of normalized crime data as CSV with consistent state names, consistent column definitions, and confidence scores indicating data reliability.

That is what BharatData builds.

Why No One Has Solved This Yet

Several organizations have attempted partial solutions:

- **DataMeet** is an Indian open data community that has cleaned specific datasets. But their work is volunteer-driven, inconsistent, and not packaged as a developer tool.
- **data.gov.in** is the government's own open data portal. But the data quality is poor, updates are inconsistent, and there is no unified API across datasets.
- **Kaggle datasets** contain some Indian government data but they are static snapshots, often years out of date, with no update mechanism.

No one has built a comprehensive, maintained, developer-friendly SDK with automated ingestion, systematic normalization, confidence scoring, and source attribution. That gap is what BharatData fills.

File 4: `docs/internal/PRD.md`

```
# BharatData — Product Requirements Document

**Version:** 1.0
**Status:** Active
**Last Updated:** [Update when you create this file]

## v1 Scope — Included

### Data Pipeline
- Automated ingester for NCRB Crime in India PDF reports
- PDF parser with table extraction and confidence scoring
- Data normalizer with canonical state and district name mapping
- PostgreSQL storage with full data lineage on every record
- Daily health monitoring with automated GitHub issue creation on source changes

### SDK
- TypeScript package published to npm as `bharatdata`
- 5 query functions under the `bd.crime` namespace
- Fully typed responses with zero `any` types
- Source attribution on every response (sourceUrl, collectionDate, confidence)
- Custom error classes with human-readable messages

### API
- REST endpoints mirroring SDK functions
- Global rate limiting
- CORS headers for browser access
- JSON responses matching SDK response shape

### Web Playground
- Single page web application
- Text input for queries
- Results displayed as a table
- CSV download button
- No login or signup required
- Must return real data within 30 seconds of page load

### Monitoring
- Daily GitHub Action checking data source accessibility
```

- Parser output structure validation
- Automated issue creation on detected changes

Legal Minimums
- Source attribution on every data record
- Terms of Use page
- Rate limiting on scrapers (max 1 req/sec)
- Independence disclaimer

Documentation
- README with 5-minute quick start
- SDK API reference
- Data Source Registry for NCRB
- Contributing Guide
- Changelog

v1 Scope — Explicitly Excluded

The following are intentionally NOT part of v1. Do not build, plan, or suggest any of these:

- AI natural language queries (Phase 4)
- AI data narration (Phase 4)
- AI anomaly detection (Phase 4)
- Multiple datasets beyond NCRB (Phase 4)
- User accounts or authentication
- Paid tiers or subscription logic
- Mobile application
- Data visualization or charting libraries
- Webhook notifications
- Email alerts
- Dashboard analytics
- User-specific rate limiting (use global only)
- GraphQL API
- WebSocket connections
- Any form of user-generated content
- Social features (comments, sharing, likes)
- Admin panel
- Multi-tenancy

User Stories

US-01: Developer queries crime data
As a developer, I want to install a package and query Indian crime data by state and year so that I can build a data application without spending weeks collecting data.

Acceptance: `npm install bharatdata` works. `bd.crime.getByState('Maharashtra', 2023)` returns correct, typed data within 2 seconds. Response includes source attribution.

US-02: Journalist uses playground
As a journalist, I want to visit a web playground and look up crime statistics so that I can get verified numbers for my article without downloading PDF reports.

Acceptance: Playground loads. User types a state name and year. Results appear in under 3 seconds. CSV download works. Source URL is visible for verification.

US-03: Researcher downloads data
As a researcher, I want to download clean structured crime data as CSV so that I can use it in my statistical analysis without manual data cleaning.

Acceptance: Playground CSV download contains all queried records with proper column headers, consistent state names, and source attribution columns.

US-04: Student accesses real data
As a student, I want to access real Indian government data through a simple API so that I can use it in my college project instead of fake datasets.

Acceptance: SDK documentation is clear enough for a student with basic TypeScript knowledge to make their first query within 5 minutes.

US-05: Contributor adds mappings
As a contributor, I want to add state or district name variations to a public mappings file so that I can help improve data quality for everyone.

Acceptance: `CONTRIBUTING.md` explains the canonical mappings contribution process. The `canonical-mappings.json` file is clearly structured. A contributor can add an entry without understanding the rest of the codebase.

Acceptance Criteria — Technical

AC-01: Parser accuracy
Parser extracts all major tables from NCRB Crime in India report with >95% accuracy verified against manual spot checks of 20 random cells across different tables and years.

AC-02: Type safety
SDK returns fully typed responses for all 5 functions. TypeScript compiler in strict mode produces zero errors. No `any` types in any public API surface. Users get full autocomplete in VS Code / Antigravity.

AC-03: Response time
Playground returns results for any valid query in under 3 seconds. API endpoints respond in under 2 seconds. SDK function calls (which hit the API) complete in under 3 seconds on a standard internet connection.

AC-04: Source attribution
Every data point returned through SDK, API, or playground includes: original government source URL, BharatData collection date, confidence score (0.0 to 1.0), and normalizer version.

AC-05: Documentation usability
README enables a new user to go from `npm install bharatdata` to first successful data query in under 5 minutes. Verified by testing with at least one person unfamiliar with the project.

AC-06: Monitoring responsiveness
Health monitoring detects a source URL change or outage and creates a GitHub issue within 24 hours with specific failure details.

AC-07: Data freshness
When NCRB publishes new annual data, BharatData ingests and serves it within 2 weeks of publication.

Technical Constraints

- All infrastructure on free tiers (Neon 500MB, Vercel/CF Workers, GitHub Actions)
- Single developer — no team, no code review from others
- SDK must work offline after installation (types and client code, obviously requires network for data)
- API response time under 2 seconds for all endpoints
- Database must fit within 500MB storage limit
- Scrapers never exceed 1 request per second to any government server
- No external dependencies in SDK unless absolutely necessary (minimize install size)

Dependencies and Risks

Risk	Likelihood	Impact	Mitigation
NCRB changes PDF format	High (happens periodically)	Parser breaks	Monitoring alerts. Parser designed for adaptability.
NCRB website goes offline	Medium	Cannot update data	Serve cached data. Alert users via status page.
Free tier storage limit reached	Medium (at scale)	Cannot add more historical data	Optimize storage. Consider paid tier migration.
npm package name conflict	Low	Branding confusion	Already checked availability.

| Government objects to scraping | Low | Must cease scraping that source | Full compliance. Attribution. Rate limiting. Alternative sources. |
| Parser produces wrong data | Medium | Trust destruction | Confidence scoring. Manual spot checks. Never serve low-confidence data silently. |

File 5: docs/internal/QUALITY_STANDARDS.md

```
# BharatData — Quality Standards

These standards are non-negotiable. Every component, every commit,
        every release must meet these requirements. No exceptions.

## Data Quality Rules

### Rule 1: Every record must have attribution
No data point exists in BharatData without: original government
        source URL, the specific page or table reference, the date
        BharatData collected it, and a confidence score between
        0.0 and 1.0.

### Rule 2: Never serve wrong data confidently
If the parser is uncertain about an extracted value, it MUST
        assign a low confidence score. Records with confidence
        below 0.5 are stored but NOT served through the API. They
        are flagged for manual review. A journalist publishing
        wrong numbers from BharatData would permanently destroy
        the project's reputation.

### Rule 3: Never silently drop records
If a record cannot be parsed or normalized, it must be logged with
        the specific failure reason. It must NOT be silently
        discarded. The unmapped-entities.json file collects all
        entities that could not be resolved. These are reviewed
        and either added to canonical mappings or documented as
        known gaps.

### Rule 4: Numbers are numbers, never strings
All numeric values are stored as integers or floats. Never as
        strings. "1,234" is stored as 1234. "12.5%" is stored as
        12.5 with a metadata flag indicating percentage. "N/A" or
        "-" or "" are stored as null with a metadata flag
        indicating missing data.

### Rule 5: Dates are ISO 8601
All dates stored as ISO 8601 format (YYYY-MM-DD). Year-only data
        stored as YYYY. No "FY 2022-23" format in the database —
        convert to the appropriate year with a metadata note about
        fiscal year mapping.
```

Rule 6: Indian number formatting is converted
Lakhs and crores in source data are converted to standard integers. "1,23,456" becomes 123456. "2.5 crore" becomes 25000000. Original formatting preserved in a metadata field if needed for display.

Rule 7: Null means null
Missing values are represented as null. Never as 0 (which could be a valid data point). Never as empty string. Never as "-" or "N/A". These source representations are converted to null during normalization with a flag indicating the original representation.

Code Quality Rules

Rule 8: Zero `any` types
The TypeScript SDK must have zero `any` types in any public-facing function signature, parameter, or return type. Internal utility functions may use `any` only when absolutely necessary and must include a comment explaining why.

Rule 9: Every function has error handling
No function should throw an unhandled exception. All errors are caught and returned as typed custom error objects (BharatDataError, SourceNotFoundError, InvalidQueryError, RateLimitError, DataUnavailableError).

Rule 10: Every function is documented
Every public SDK function has a JSDoc comment with description, parameter explanations, return type description, example usage, and thrown error types.

Pipeline Quality Rules

Rule 11: Ingester is fault tolerant
The ingester never crashes silently. Every failure is logged with: timestamp, source URL, HTTP status code, error message, retry count. It retries with exponential backoff up to 3 attempts. After 3 failures it creates a GitHub issue and stops.

Rule 12: Parser validates output
After parsing a table, the parser cross-checks row totals against column totals where both exist in the source. If they do not match within a 1% tolerance, the entire table is flagged with reduced confidence.

Rule 13: Normalizer logs everything
Every name resolution is logged: input value, canonical output, mapping source (canonical-mappings.json entry). Every unmapped entity is logged to unmapped-entities.json with the source table and context.

Rule 14: Database records are immutable

Once a record is stored with a specific ingestion batch ID, it is never modified in place. Updates create new records with new batch IDs. Historical records are preserved for audit trail.

Scraping Ethics Rules

Rule 15: Maximum 1 request per second
Never exceed 1 HTTP request per second to any single government server. Implement explicit delays between requests.

Rule 16: Respect robots.txt
Check and respect robots.txt directives on all government websites.

Rule 17: No CAPTCHA circumvention
If a government website presents a CAPTCHA, do not attempt to bypass it. Document the limitation and find an alternative data access method.

Rule 18: Immediate compliance
If any government body requests cessation of automated access, comply immediately. Document the situation and find alternative data sources.

Release Quality Rules

Rule 19: Manual spot check before every release
Before any npm publish or API deployment, manually verify 10 random data points against the original government source. If any are incorrect, do not release.

Rule 20: Changelog entry for every release
No release without a corresponding CHANGELOG.md entry documenting what changed.

Rule 21: Version bump follows semver
Major version: breaking changes to function signatures or response shapes.
Minor version: new functions, new datasets, new optional parameters.
Patch version: bug fixes, data updates, documentation improvements.

File 6: docs/sdk/DESIGN.md

BharatData — SDK Design Specification

Package name: bharatdata
Language: TypeScript
Registry: npm

License: MIT

Design Principles

1. Consistency
Similar operations have similar signatures across all dataset modules. If `bd.crime.getByState()` takes `(state, year)`, then future `bd.census.getByState()` also takes `(state, year)`.

2. Predictability
A developer should correctly guess how a function works before reading documentation. Method names describe exactly what they return. Parameter names describe exactly what they filter.

3. Minimalism
No function is added unless multiple real users need it. v1 has exactly 5 functions. Each one earns its place.

4. Type Safety
Every input and output is fully typed. Zero `any` in the public API surface. Users get full autocomplete in their IDE.

5. Attribution by Default
Every response includes source attribution automatically. The user never needs to request it. It is always there.

6. Fail Loudly
Errors are clear, specific, and suggest resolution. Never return empty results for an invalid query. Throw a typed error that tells the user exactly what went wrong and what to do about it.

Installation and Setup

```typescript
// Installation
npm install bharatdata

// Basic usage
import { BharatData } from 'bharatdata';

const bd = new BharatData({
  // No API key required for v1
  // Optional configuration
  baseUrl: 'https://api.bharatdata.dev', // default
  timeout: 10000, // default 10s
});
```

Naming Conventions

- Package name: `bharatdata` (lowercase, one word)
- Import: `import { BharatData } from 'bharatdata'`
- Client instantiation: `const bd = new BharatData()`
- Dataset modules: `bd.crime`, `bd.census`, `bd.agriculture` (future)
- Query methods: `bd.crime.getByState()`, `bd.crime.getByYear()`
- All method names: camelCase
- All parameter keys: camelCase
- All response keys: camelCase

v1 Function Signatures

bd.crime.getByState(state, year?)

Returns crime data for a specific state. Optionally filtered by year.

```
interface GetByStateParams {
  state: string;      // State name (any known variation accepted)
  year?: number;      // Optional: specific year
}

const result = await bd.crime.getByState('Maharashtra', 2023);
```

bd.crime.getByYear(year)

Returns crime data for all states in a given year.

```
interface GetByYearParams {
  year: number;       // Four-digit year
}

const result = await bd.crime.getByYear(2023);
```

bd.crime.getByCategory(category, options?)

Returns crime data filtered by crime category with optional state and year filters.

```
interface GetByCategoryParams {
  category: string;           // Crime category (e.g., 'murder',
        'theft', 'cyber-crime')
  options?: {
    state?: string;           // Optional: filter by state
    year?: number;            // Optional: filter by year
    yearRange?: [number, number]; // Optional: filter by year
        range
  };
}
```

```
const result = await bd.crime.getByCategory('cyber-crime', {
  state: 'Karnataka',
  yearRange: [2019, 2023]
});
```

bd.crime.compare(states, years)

Returns comparative data across multiple states and/or years.

```
interface CompareParams {
  states: string[];    // Array of state names
  years: number[];     // Array of years
  category?: string;   // Optional: specific crime category
}

const result = await bd.crime.compare(
  ['Maharashtra', 'Karnataka', 'Tamil Nadu'],
  [2021, 2022, 2023]
);
```

bd.crime.search(query)

Flexible search across all crime data fields.

```
interface SearchParams {
  query: string;              // Search term
  filters?: {
    state?: string;
    year?: number;
    category?: string;
  };
  limit?: number;             // Default: 50, Max: 500
  offset?: number;            // Default: 0
}

const result = await bd.crime.search('cyber', {
  filters: { year: 2023 },
  limit: 10
});
```

Standard Response Shape

Every function returns this exact shape:

```
interface BharatDataResponse<T> {
  data: T[];                    // Array of records
  meta: {
    source: {
```

```
      name: string;             // e.g., "National Crime Records
        Bureau"
      url: string;              // Direct URL to original data
      reportName: string;       // e.g., "Crime in India 2023"
    };
    collectionDate: string;     // ISO 8601: when BharatData
        ingested this
    recordCount: number;        // Number of records returned
    totalAvailable: number;     // Total matching records (for
        pagination)
    confidence: {
      average: number;          // Average confidence across
        returned records
      minimum: number;          // Lowest confidence in result set
    };
    bharatDataVersion: string;  // SDK version
    normalizerVersion: string;  // Normalizer version used
  };
  query: {                     // Echo of request parameters
    function: string;           // Which function was called
    params: Record<string, unknown>; // Parameters passed
    executedAt: string;         // ISO 8601 timestamp
  };
}
```

Individual Record Shape (Crime Data)

```
interface CrimeRecord {
  id: string;                   // BharatData internal ID
  state: string;                // Canonical state name
  stateCode: string;            // Standard state code
  district?: string;            // Canonical district name (if
        available)
  year: number;                 // Four-digit year
  category: string;             // Crime category
  subcategory?: string;         // Crime subcategory (if
        available)
  totalCases: number;           // Total reported cases
  rate?: number;                // Rate per lakh population (if
        available)
  chargesheeted?: number;       // Cases chargesheeted (if
        available)
  convicted?: number;           // Cases with conviction (if
        available)
  source: {
    url: string;                // Original government source URL
    page?: number;              // Page number in source document
    table?: string;             // Table identifier in source
  };
  confidence: number;           // 0.0 to 1.0
```

```
  collectionDate: string;       // ISO 8601
}
```

Error Handling

Custom Error Classes

```
class BharatDataError extends Error {
  code: string;
  suggestion: string;
}

class SourceNotFoundError extends BharatDataError {
  // When requested state/category/year does not exist in data
  code: 'SOURCE_NOT_FOUND';
  suggestion: 'Check available states with
        bd.crime.getAvailableStates()';
}

class InvalidQueryError extends BharatDataError {
  // When parameters are malformed
  code: 'INVALID_QUERY';
  suggestion: string; // Specific to the error
}

class RateLimitError extends BharatDataError {
  // When API rate limit is exceeded
  code: 'RATE_LIMIT';
  retryAfter: number; // Seconds until retry
  suggestion: 'Wait {retryAfter} seconds before retrying';
}

class DataUnavailableError extends BharatDataError {
  // When the API or database is temporarily unavailable
  code: 'DATA_UNAVAILABLE';
  suggestion: 'The service is temporarily unavailable. Try again
        in a few minutes.';
}
```

Error Response Shape

```
interface BharatDataErrorResponse {
  error: {
    code: string;
    message: string;
    suggestion: string;
    timestamp: string;
  };
}
```

Versioning Strategy

Semantic Versioning (semver):

- **Major** (X.0.0): Breaking changes to function signatures, response shapes, or removal of functions
- **Minor** (0.X.0): New functions, new datasets, new optional parameters, new response fields
- **Patch** (0.0.X): Bug fixes, data updates, documentation improvements, performance improvements

Pre-1.0: The API surface may change between minor versions.
Post-1.0: Breaking changes only in major versions.

File 7: `docs/internal/DATA_PIPELINE_SPEC.md`

```markdown

# BharatData — Data Pipeline Specification

## Pipeline Overview

The data pipeline transforms messy government data into clean, queryable records. It has four stages that run in sequence: Ingest → Parse → Normalize → Store.
```

Government Source (PDF/CSV/HTML) │ ▼ ┌─────────────────┐ │ INGESTER │ Downloads raw files with metadata │ │ Detects updates via hash comparison │ │ Fault tolerant with retry logic └─────────────────┘ │ ▼ ┌─────────────────┐ │ PARSER │ Extracts structured records from raw files │ │ Assigns confidence scores per row │ │ Logs all parsing failures └─────────────────┘ │ ▼ ┌─────────────────┐ │ NORMALIZER │ Resolves name variations to canonical forms │ │ Converts data types (strings to numbers, dates) │ │ Handles historical boundary changes └─────────────────┘ │ ▼ ┌─────────────────┐ │ STORAGE │ Stores records in PostgreSQL with full lineage │ │ Indexes for common query patterns │ │ Caches frequently accessed data └─────────────────┘

```markdown

## Stage 1: Ingester

### Purpose
Automatically visit government data sources, detect if new data is available, download raw files, and store them with complete metadata.

### Behavior
```

- Runs on a schedule via GitHub Actions cron job
- Before downloading, compares file hash against last known hash
- If hash has changed (new data available), downloads the file
- If hash is same (no update), skips and logs "no update"
- Stores raw file in a designated storage location
- Records metadata: source URL, download timestamp, file hash, file size, format type, HTTP status code

Fault Tolerance
- Retries failed downloads with exponential backoff: wait 1 minute, then 5 minutes, then 30 minutes
- Maximum 3 retry attempts per source per run
- After 3 consecutive failures, creates a GitHub issue with label `data-source-alert`
- Never crashes silently — every failure is logged with full context

Rate Limiting
- Maximum 1 HTTP request per second to any government server
- Explicit `sleep(1000)` between requests
- Respects robots.txt directives

Output
For each successful ingestion:
```json
{
  "sourceId": "ncrb-crime-in-india",
  "sourceUrl": "https://ncrb.gov.in/...",
  "downloadedAt": "2025-01-15T06:00:00Z",
  "fileHash": "sha256:abc123...",
  "fileSize": 15234567,
  "format": "pdf",
  "httpStatus": 200,
  "rawFilePath": "/data/raw/ncrb/crime-in-india-2023.pdf",
  "previousHash": "sha256:def456...",
  "isNewData": true
}
```

Stage 2: Parser

Purpose

Extract structured data records from raw files (PDFs, CSVs, HTML tables) with confidence scoring on every extracted value.

NCRB PDF Parsing Challenges

These are the specific challenges the parser must handle for NCRB Crime in India reports:

1. **Multi-row headers**: Table headers often span 2-3 rows with merged cells. The parser must detect header rows and merge them into flat column names.

2. **Cross-page tables**: Tables continue across page breaks without repeating headers. The parser must detect table continuation and apply the header from the first page.

3. **Embedded footnotes**: Footnote markers (, #, @) appear inside data cells (e.g., “1,234”). The parser must strip footnote markers and store them as metadata.

4. **Mixed languages**: Some table headers are in Hindi, some in English, some in both. The parser must handle mixed-language content.

5. **Indian number formatting**: Numbers may appear as “1,23,456” (Indian notation) or “1,234,567” (international). Both must be parsed correctly to integers.

6. **Inconsistent state names**: States may be abbreviated, fully spelled, or in ALL CAPS. The parser outputs raw extracted names; the normalizer handles mapping.

7. **Empty or merged cells**: Some cells are empty, contain “-”, “NA”, or are merged with adjacent cells. Each must be handled explicitly, not silently dropped.

8. **Varying table structures**: Different chapters of the same report may use different table structures. The parser must adapt per table, not assume a single structure.

Confidence Scoring

Every extracted row receives a confidence score:

- **1.0**: All cells in the row parsed cleanly. Row total matches sum of component values (where verifiable). No ambiguity.
- **0.8**: All cells parsed but some minor ambiguity (e.g., footnote marker stripped, number format conversion applied). Cross-check passed.
- **0.6**: Most cells parsed but one or more cells had parsing difficulty. Values plausible but not independently verified.
- **0.4**: Significant parsing uncertainty. Multiple cells ambiguous. Row included but flagged.
- **0.2**: Major parsing issues. Row extracted but likely contains errors. Not served via API.

- **0.0**: Row could not be parsed. Stored for debugging only. Never served.

Output

For each parsed row:

```
{
  "rawValues": {
    "state": "ANDHRA PRADESH",
    "year": "2023",
    "murder": "1,234",
    "theft": "45,678"
  },
  "parsedValues": {
    "state": "ANDHRA PRADESH",
    "year": 2023,
    "murder": 1234,
    "theft": 45678
  },
  "confidence": 0.8,
  "sourceTable": "Table 1.1",
  "sourcePage": 23,
  "parsingNotes": ["Indian number format converted", "Footnote
        marker * stripped from murder cell"]
}
```

Stage 3: Normalizer

Purpose

Transform parsed records into canonical form using standardized names, codes, and data types.

Operations (in order)

1. **Character encoding**: Ensure UTF-8 encoding for all text
2. **State name resolution**: Map raw state names to canonical forms using `data/canonical-mappings.json`
3. **District name resolution**: Map raw district names to canonical forms (where applicable)
4. **Historical boundary handling**: Apply Telangana/Ladakh/ district bifurcation rules
5. **Data type enforcement**: All numbers to integers/floats, all dates to ISO 8601, all missing values to null
6. **Indian number format conversion**: “1,23,456” → 123456, “2.5 crore” → 25000000
7. **Deduplication**: Check for duplicate records from overlapping ingestion runs

8. **Confidence adjustment**: Reduce confidence if normalization required significant assumptions
9. **Source attribution attachment**: Add source URL, collection date, normalizer version
10. **Range validation**: Flag values outside historically plausible ranges

Unmapped Entity Handling

When the normalizer encounters a state or district name not in `canonical-mappings.json`: - The record is NOT dropped - The record is passed through with the raw name intact - The unmapped name is logged to `data/unmapped-entities.json` - Confidence is reduced by 0.2 - The unmapped-entities.json file is reviewed periodically and new mappings are added

Output

```
{
  "canonicalValues": {
    "state": "Andhra Pradesh",
    "stateCode": "AP",
    "censusCode": "28",
    "year": 2023,
    "murder": 1234,
    "theft": 45678
  },
  "normalization": {
    "stateInput": "ANDHRA PRADESH",
    "stateResolution": "canonical-mappings.json#AP",
    "dataTypeConversions": ["murder: string→number", "theft:
        string→number"],
    "normalizerVersion": "1.0.0"
  },
  "confidence": 0.8,
  "source": {
    "url": "https://ncrb.gov.in/...",
    "page": 23,
    "table": "Table 1.1",
    "collectionDate": "2025-01-15T06:00:00Z"
  }
}
```

Stage 4: Storage

Database

PostgreSQL on Neon free tier (500MB storage limit).

Schema Design Principles

- Support multiple datasets without restructuring
- Every record traceable to its source
- Optimized for common query patterns (by state, by year, by category)
- Historical records preserved (no in-place updates)

Core Tables

`data_sources`: Registry of all government data sources - id, name, url, format, update_frequency, last_ingested_at, status

`ingestion_runs`: Log of every ingestion execution - id, source_id, started_at, completed_at, status, records_extracted, records_stored, file_hash

`crime_records`: Normalized crime data - id, ingestion_run_id, state, state_code, district, year, category, subcategory, total_cases, rate_per_lakh, chargesheeted, convicted, acquitted, confidence, source_url, source_page, source_table, collection_date, normalizer_version, created_at

`normalization_log`: Record of every normalization decision - id, record_id, field, input_value, output_value, mapping_source, timestamp

`unmapped_entities`: Names that could not be resolved - id, entity_type (state/district), raw_value, source_table, source_page, first_seen_at, resolved (boolean)

Indexes

- `crime_records(state, year)` — most common query pattern
- `crime_records(year)` — year-based queries
- `crime_records(category, year)` — category filters
- `crime_records(state, category, year)` — compound filter
- `crime_records(confidence)` — filter out low-confidence records

Storage Estimate

NCRB Crime in India: ~500 tables × ~35 states × ~10 years = ~175,000 records At ~500 bytes per record = ~87.5 MB. Well within 500MB free tier.

Monitoring

Daily Health Check (GitHub Action)

Runs at 06:00 UTC daily: 1. Check if each data source URL returns HTTP 200 2. Compare current page hash against last known hash 3. If URL is unreachable: create GitHub issue with `data-source-alert` label 4. If hash has changed: create GitHub issue with `data-source-updated` label 5. Run parser against most recent downloaded file and compare output schema against expected schema 6. If schema drift detected: create GitHub issue with `parser-attention` label

Data Quality Check (GitHub Action)

Runs weekly: 1. Query record counts by state and year — should be stable unless new ingestion occurred 2. Check for statistical outliers (values > 3 standard deviations from historical mean) 3. Verify confidence score distribution — alert if average confidence drops below 0.7 4. Check for records with null values in required fields 5. Report results as a GitHub issue with `quality-report` label

---

File 8: `docs/data-schema/NORMALIZATION.md`

```markdown

# BharatData — Data Schema & Normalization Guide

## Canonical Mappings File

Location: `data/canonical-mappings.json`

This is one of the most valuable artifacts of the BharatData project. It is a public, version-controlled JSON file that maps every known variation of Indian state and district names to a single canonical form. Other developers can use this file independently of the BharatData SDK.

### Structure

```json
{
  "version": "1.0.0",
  "lastUpdated": "2025-01-15",
  "states": {
    "Andhra Pradesh": {
      "canonical": "Andhra Pradesh",
```
```

```
      "code": "AP",
      "censusCode": "28",
      "variations": [
        "AP", "A.P.", "ANDHRA PRADESH", "Andhra", "andhra
pradesh",
        "Andhra pradesh", "ANDHRA", "Andhra Pr."
      ],
      "notes": "Post-2014 refers to residuary Andhra Pradesh after
Telangana bifurcation",
      "boundaryChanges": [
        {
          "date": "2014-06-02",
          "type": "bifurcation",
          "description": "Telangana carved out as separate state",
          "affectedDistricts": ["Hyderabad", "Warangal",
"Karimnagar", "..."]
        }
      ]
    },
    "Telangana": {
      "canonical": "Telangana",
      "code": "TG",
      "censusCode": "36",
      "variations": [
        "TG", "TELANGANA", "Telengana", "telangana"
      ],
      "notes": "Created June 2, 2014. Data before 2014 is part of
unified Andhra Pradesh.",
      "existsFrom": "2014-06-02"
    }
  },
  "districts": {
    "Mumbai": {
      "canonical": "Mumbai",
      "state": "Maharashtra",
      "variations": ["MUMBAI", "Bombay", "BOMBAY", "Mumbai City"],
      "notes": "Renamed from Bombay. Pre-1995 data may use
Bombay."
    }
  }
}
```


How the Normalizer Uses This File

1. Receive raw state name from parser (e.g., “ANDHRA PRADESH”)
2. Convert to lowercase for comparison: “andhra pradesh”
3. Search through all states’ variations arrays for a case-insensitive match
4. If found: return the canonical name, code, and census code

5. If NOT found: log to unmapped-entities.json, pass through with raw name, reduce confidence by 0.2

Historical Boundary Changes

Telangana Bifurcation (June 2, 2014)

- **Before 2014**: All data is attributed to unified “Andhra Pradesh” with combined state code
- **2014 onwards**: Separate entries for “Andhra Pradesh” (residuary) and “Telangana”
- **Cross-period queries**: When a user queries Andhra Pradesh across 2012-2016, the response includes a `boundaryChange` annotation explaining that pre-2014 data includes Telangana territory
- **District mapping**: Districts like Hyderabad, Warangal, Karimnagar that moved to Telangana are mapped with effective dates

Ladakh Separation (October 31, 2019)

- **Before 2019**: Ladakh data is part of “Jammu & Kashmir”
- **2019 onwards**: Separate entries for “Jammu & Kashmir” (UT) and “Ladakh” (UT)
- **Note**: Both changed from State to Union Territory status simultaneously

District Bifurcations

Districts in India are regularly split. When this happens: - Parent district record includes a `bifurcatedInto` field with child district names and effective date - Child district records include a `bifurcatedFrom` field with parent district name - Queries for a parent district after bifurcation return combined data from child districts with an annotation - Historical queries correctly attribute data to the parent district for pre-bifurcation years

Data Type Standards

Source Format	BharatData Format	Example
“1,23,456” (Indian notation)	Integer: 123456	totalCases: 123456
“1,234,567” (international)	Integer: 1234567	totalCases: 1234567
“12.5%”		

Source Format	BharatData Format	Example
	Float: 12.5 + type flag	rate: 12.5, rateType: “percentage”
“2.5 crore”	Integer: 25000000	population: 25000000
“1.5 lakh”	Integer: 150000	cases: 150000
“2023-24” (fiscal year)	Integer: 2023 + flag	year: 2023, yearType: “fiscal_start”
“N/A”, “-”, “”,”NA”	null + flag	value: null, originalValue: “N/A”
“2023” (year)	Integer: 2023	year: 2023
“15-Jan-2024” (date)	String: “2024-01-15” (ISO)	date: “2024-01-15”

Confidence Scoring

Score	Meaning	API Behavior
1.0	Extracted from clean structured source. Automated cross-check validation passed.	Served normally
0.8	Extracted from PDF with high parser confidence. Minor formatting conversions applied. Spot-check verified.	Served normally
0.6	Extracted but some parsing ambiguity. Values plausible but not independently verifiable.	Served with warning annotation
0.5	Threshold for API serving. Below this, data is stored but not returned to users.	Threshold
0.4	Significant parsing uncertainty.	Stored only. Not served via API.

Score	Meaning	API Behavior
	Multiple ambiguous cells.	
0.2	Major parsing issues. Likely contains errors.	Stored for debugging. Not served.
0.0	Could not be parsed at all.	Stored as failed parse record. Not served.

Normalization Pipeline — Step by Step

```
Step 1:  Raw text extraction from source file
Step 2:  Character encoding normalization to UTF-8
Step 3:  Field mapping to canonical schema (column name → standard
field name)
Step 4:  State name resolution via canonical-mappings.json
Step 5:  District name resolution via canonical-mappings.json
Step 6:  Historical boundary annotation (Telangana, Ladakh,
bifurcations)
Step 7:  Data type conversion (strings to numbers, dates to ISO
8601)
Step 8:  Indian number format conversion (lakhs/crores to standard
integers)
Step 9:  Null value standardization (N/A, -, "" → null with
original preserved)
Step 10: Deduplication against existing records in database
Step 11: Confidence scoring based on extraction quality and
normalization steps applied
Step 12: Source attribution attachment (URL, page, table,
collection date)
Step 13: Range validation (flag values outside historically
plausible bounds)
Step 14: Final record assembly with full metadata


---

## File 9: `docs/data-sources/NCRB_CRIME.md`

```markdown

# Data Source: NCRB Crime in India

## Source Information

| Field | Value |
|---|---|
| **Full Name** | Crime in India — Annual Statistical Report |
```
```

Publishing Body	National Crime Records Bureau (NCRB), Ministry of Home Affairs, Government of India
Source URL	https://ncrb.gov.in/crime-in-india.html
Data Format	PDF (primary), some tables also in Excel
Update Frequency	Annual (typically published 8-12 months after the reference year)
Coverage Period	1953 - present (BharatData targets 2015-latest initially)
Geographic Coverage	All states and union territories of India; some tables have district-level data
Legal Status	Published by Government of India. Available on public government website. No explicit open data license on NCRB website. Attribution required.
BharatData Status	Primary — first dataset for v1

Why This Source First

1. **High demand**: Crime data is cited in hundreds of news articles annually. Journalists at IndiaSpend, The Wire, Scroll, and NDTV regularly reference NCRB statistics.
2. **High pain**: The data is locked in a 400+ page PDF with ~500 tables. No API. No structured download option.
3. **Technical challenge**: Parsing this PDF demonstrates real engineering skill. Multi-row headers, cross-page tables, embedded footnotes, mixed languages.
4. **Broad interest**: Crime statistics interest developers (building analytics), journalists (reporting), researchers (studying patterns), students (ML projects), and citizens (understanding safety).

Report Structure

The NCRB Crime in India report is organized into volumes and chapters:

- **Volume I**: Covers IPC (Indian Penal Code) crimes — murder, theft, robbery, kidnapping, assault, etc.
- **Volume II**: Covers SLL (Special & Local Laws) crimes — NDPS Act, Arms Act, Gambling Act, etc.
- **Volume III**: Covers crimes against specific groups — women, children, SC/ST, senior citizens

Each volume contains multiple chapters, each with multiple tables.

Key Tables to Extract (v1 Priority)

Table ID	Description	Complexity
Table 1.1	IPC crimes — state-wise total cognizable crimes	Medium
Table 1.2	IPC crimes — disposal of cases by police	Medium
Table 1.3	IPC crimes — disposal of cases by courts	Medium

Table 3.1	Murder — state-wise cases and rates	Low
Table 4.1	Kidnapping & Abduction — state-wise	Low
Table 5.1	Crimes against women — state-wise	Medium
Table 9A.1	Cyber crimes — state-wise	Low
Table 15.1	Economic offences — state-wise	Medium
Chapter summary tables	Each chapter has summary aggregations	Varies

Known Data Quality Issues in Source

1. **State name inconsistency**: "D & N Haveli" vs "Dadra & Nagar Haveli" vs "DNH" across different tables in the same report.
2. **Telangana transition**: 2014 report has both combined AP data and separate TS data in different tables.
3. **Footnote overload**: Some tables have 5-10 footnotes with important contextual information embedded in data cells.
4. **Category redefinition**: Some crime categories changed definition between years (e.g., what counts as "cyber crime" expanded over time).
5. **Delhi dual status**: Delhi appears both as a UT and as a city in different tables. Data may be counted differently.
6. **Zero vs null ambiguity**: Some states show "0" for a crime category. It's unclear whether this means zero reported cases or no data available.
7. **2020 data anomalies**: COVID-19 affected reporting in 2020. Lower crime numbers may reflect reduced reporting rather than reduced crime.

Parsing Challenges Specific to NCRB PDFs

Challenge 1: Multi-row headers

Sl. | State/UT | Murder | Kidnapping & Abduction |
No. | | | Of Women | Of Children | Total |

|—-|————-|——-|————-|————-|————|

Headers span 2-3 rows with merged cells. Parser must detect and merge these into flat column names like "kidnapping_of_women", "kidnapping_of_children", "kidnapping_total".

Challenge 2: Cross-page table continuation
A table starting on page 23 continues on page 24 without repeating the header row. Parser must detect that page 24's table is a continuation and apply page 23's headers.

Challenge 3: Embedded footnotes
Cell value appears as "1,234*" where * references a footnote at the bottom of the page. Parser must strip the marker and parse 1234 as the numeric value, storing the footnote reference as metadata.

Challenge 4: Summary rows
Tables include "Total" and "Grand Total" rows that are aggregations, not data. Parser must identify and flag these as summary rows, not include them as state-level data.

Challenge 5: State groupings
Some tables group states by region (Northern, Southern, etc.) with sub-headers. Parser must handle these hierarchical groupings.

Expected Output Fields

After parsing and normalization, each NCRB crime record should contain:

state: string (canonical) stateCode: string district: string | null (when available) year: number category: string (e.g., “murder”, “theft”, “cyber-crime”) subcategory: string | null ipcOrSll: “IPC” | “SLL” totalCases: number caseRate: number | null (per lakh population) casesChargesheeted: number | null casesPendingInvestigation: number | null casesConvicted: number | null casesAcquitted: number | null casesPendingTrial: number | null

---

File 10: `docs/internal/BRAND_MINIMUMS.md`

BharatData — Brand Minimums Checklist

These are the minimum brand decisions needed before writing the README. The full brand document (Doc 08) comes in Phase 3.

Name Format

- In prose: **Bharat Data** (two words, both capitalized)
- In code: `bharatdata` (one word, lowercase)
- npm package: `bharatdata`
- GitHub repo: `bharatdata`
- Domain: `bharatdata.dev` (preferred) or `bharatdata.io`
- Never: BharatData, bharatData, Bharatdata, BHARATDATA, or "BD"

Tagline

"Clean, queryable Indian public data"

Use this everywhere: README, npm description, GitHub repo description, social media bio, website header.

Disclaimer

Include on website footer, README, and API documentation:

"Bharat Data is an independent open source project. It is not affiliated with or endorsed by any Indian government body."

GitHub Repository Description

"Open source SDK for clean, queryable Indian government data. TypeScript. REST API. Web playground."

npm Package Description

"Clean, queryable Indian government data. TypeScript SDK with full types, source attribution, and confidence scoring."

Social Media Bio

"Building Bharat Data — making Indian government data accessible for developers, journalists, and researchers. Open source. 🇮🇳"

File 11: `docs/internal/EXECUTION_STRATEGY.md`

BharatData — Execution Strategy

Status: Active
Visibility: Private (not published)

Phase Overview

Phase	Duration	Goal	Exit Condition
Phase 0	3 days	Decisions and direction	Dataset chosen, quality bar written, repo initialized, first public post live
Phase 1	2 days	Passive validation	Research documented, dataset confirmed
Phase 2	4 weeks (hard deadline)	Core engineering	npm package published, playground live, monitoring active
Phase 3	4 weeks	Organic distribution	25 organic users, 2+ GitHub issues from strangers
Phase 4	3-4 weeks	AI differentiation	NL queries working, narration live, second dataset added
Phase 5	Ongoing	Scale and sustainability	Product Hunt launch, segment identified, sustainability explored

Phase 0 — Clarity Before Code (3 days)

Failure Protocol: Default to NCRB if indecision persists past Day 3.

```
| Milestone | Dependencies |
|---|---|
| Choose primary dataset (NCRB) | None |
| Define written quality bar | Dataset selected |
| Set up public build log | Quality bar defined |
| Initialize GitHub repository | Dataset selected |

## Phase 1 — Passive Research (2 days)

**Failure Protocol:** If conflicting signals, stick with NCRB.

| Milestone | Dependencies |
|---|---|
| Scan developer and open data communities | Phase 0 complete |
| Review journalism and academic sources | Community scan done |
| Document pain points with exact language | Sources reviewed |
| Confirm dataset choice | Documentation complete |

## Phase 2 — Build the Hardest Thing (4 weeks max)

**Failure Protocol:** If parser handles <10 tables reliably by
        Week 4, ship working subset with documentation.

### Week 1: Foundation
| Milestone | Dependencies |
|---|---|
| Brand Minimums (Doc 08a) | Phase 1 complete |
| MIT License (Doc 12) | Phase 1 complete |
| Start Project Journal (Doc 00a) | Phase 2 start |
| SDK Design Spec (Doc 05) | Phase 1 complete |
| Build data ingester | Doc 05 complete |
| Build PDF parser | Ingester working |

### Week 2: Normalization and Schema
| Milestone | Dependencies |
|---|---|
| Build normalizer and canonical-mappings.json | Parser producing
        records |
| Write Data Schema Guide (Doc 06) | Normalizer working |
| Draft Content Strategy (Doc 19) | Week 2 start |
| Set up Postgres schema | Normalizer working |

### Week 3: SDK, API, Monitoring
| Milestone | Dependencies |
|---|---|
| Add data freshness monitoring | Database live |
| Implement legal minimums | Schema complete |
| Publish TypeScript SDK to npm | Monitoring in place |
| Deploy REST API and web playground | SDK published |

### Week 4: Documentation and Quality Gate
```

Milestone	Dependencies
Write README (Doc 09)	Playground live
Write Data Source Registry (Doc 04)	First source complete
Write Architecture Document (Doc 03)	Architecture stable
Write Testing Strategy (Doc 03a)	Tests established
Quality gate: external person tests playground	All above complete

Phase 3 — Real Users (4 weeks)

Failure Protocol: If <25 users after 4 weeks, publish second blog post and test new channels.

Week 1: Documentation and Launch Prep

Milestone	Dependencies
Contributing Guide (Doc 10)	Phase 2 complete
Code of Conduct (Doc 11)	Phase 2 complete
Security Policy (Doc 13)	Phase 2 complete
Changelog (Doc 17)	First release tagged
Terms of Use (Doc 14)	Playground live
Privacy Policy (Doc 15)	Playground live
Attribution & Legal (Doc 16)	Phase 2 complete
Brand Guidelines (Doc 08)	Brand minimums exist

Week 2-3: Distribution

Milestone	Dependencies
Contact 5 public data complainers	Playground live
Publish problem-first blog post	Post drafted in Phase 2
Post in DataMeet and dev communities	Blog post live
Submit Show HN	Community posts live

Week 3-4: Assessment and Portfolio

Milestone	Dependencies
Public Roadmap (Doc 18)	Initial traction
Hit 25 organic users	All distribution active
Portfolio Case Study (Doc 20)	Traction data available

Phase 4 — AI Layer (3-4 weeks)

Failure Protocol: If AI returns incorrect data >15% of the time, disable auto-execution.

Milestone	Dependencies
Natural language query engine	Phase 3 complete
Data narration layer	NL engine working
Anomaly detection	Narration live
Second dataset (user-driven)	Demand signal clear

```
| AI Layer Spec (Doc 07) | AI layer live |
| AI technical blog post | Doc 07 complete |

## Phase 5 — Follow the Signal (Ongoing)

**Failure Protocol:** If engagement plateaus 60 days, pause
        features and retrospect.

| Milestone | Dependencies |
|---|---|
| Product Hunt launch | Phase 4 complete |
| Identify top user segment | Usage data sufficient |
| Double down on top use case | Segment identified |
| First sustainability conversation | Strong engaged base |

## Single Rule

Finish each phase completely before starting the next. Ship
        imperfectly on deadline rather than perfectly never.
```

File 12: docs/internal/CONTENT_STRATEGY.md

```
# BharatData — Content & Distribution Strategy

**Status:** Private (not published)

## Blog Post Plan

### Post 1: The Problem Story
**Title:** "I Tried to Extract Clean Crime Data From NCRB PDFs.
        Here Is What Broke."
**Draft during:** Phase 2 Week 2 (while pain is fresh)
**Publish:** Phase 3 launch week
**Platform:** Dev.to + Medium + personal blog
**Content:** Problem-first narrative. Show messy PDF screenshots.
        Show specific parsing failures. Show before/after
        comparison of raw data vs clean Bharat Data output. Link
        to project only at the end.
**Goal:** Establish credibility. Attract people who have felt this
        pain.

### Post 2: The Technical Deep-Dive
**Title:** "How I Built a Data Normalizer That Handles 780 Indian
        Districts and Their 47 Name Variations"
**Publish:** Phase 3 Week 3
**Platform:** Dev.to + Hashnode
**Content:** Technical walkthrough of canonical-mappings.json.
        Telangana bifurcation handling. District name resolution.
        The engineering decisions behind the normalization
        pipeline.
```

Goal: Demonstrate engineering depth. Attract contributors.

Post 3: The Data Story
Title: "What NCRB Crime Data Actually Reveals About [Specific Finding]"
Publish: Phase 3 Week 5
Platform: Medium + LinkedIn
Content: Use Bharat Data to answer a genuinely interesting question. Present the findings with proper visualization and context. Demonstrate the product by using it.
Goal: Attract journalists and researchers. Show real-world utility.

Post 4: The AI Story
Title: "Building an AI That Answers Questions About Indian Government Data Without Making Things Up"
Publish: Phase 4 Week 2
Platform: Dev.to + Hashnode
Content: Prompt engineering decisions. Guardrails against hallucination. Query failure analysis. What works and what does not.
Goal: AI community credibility. Technical depth showcase.

Post 5: The Retrospective
Title: "Six Months of Building Bharat Data: What Worked, What Broke, What Surprised Me"
Publish: Month 6
Platform: All platforms
Content: Honest retrospective using Project Journal material. Metrics. Lessons. What you would do differently.
Goal: Build in public narrative. Community trust.

Community Engagement Plan

Communities to Be Active In
- DataMeet mailing list and Slack
- r/developersIndia (330,000+ members)
- Indian data journalism Twitter
- Dev.to Indian developer community
- Relevant Discord servers

Engagement Rules
- Be present daily in at least one community
- Contribute genuinely — answer questions, share insights, help others
- Promote Bharat Data only when directly relevant to a conversation
- Never spam. Never cross-post the same message to multiple communities.

Launch Checklist (Phase 3 Week 2-3)

Hacker News

- Title: "Show HN: Bharat Data — Open source SDK for querying Indian government data"
- First comment: Brief honest explanation of what it does, current scope, and future direction
- Respond to every comment personally

Reddit
- r/developersIndia: "I built an open source SDK for accessing NCRB crime data programmatically. Here's what I learned about Indian government PDFs."
- r/programming: Technical angle focused on the PDF parsing challenge
- r/webdev: SDK and API design angle

Twitter/X
- Thread format: Problem → Screenshots of messy data → Solution → Code example → Playground link → GitHub link
- Tag relevant accounts if appropriate (DataMeet, IndiaSpend)

LinkedIn
- Impact angle: "Indian public data should be accessible to everyone. Here's what I'm building to make that happen."
- Career angle for recruiter visibility

Dev.to
- Full technical walkthrough article (repurpose blog post 1 with more code examples)

File 13: docs/internal/LONG_TERM_VISION.md

BharatData — Long Term Vision

This document is aspirational. Nothing here is committed. It guides direction.

Year 1 Vision (Months 1-12)

Data Coverage
- NCRB Crime in India (v1 — Phase 2)
- Second dataset based on user demand (Phase 4)
- Third and fourth datasets by month 8-10
- Target: 4-6 reliable datasets by end of year 1

Most Likely Second Dataset Candidates (based on known demand)
1. **CPCB Air Quality Index data** — High journalist demand, environmental research applications
2. **Agmarknet Mandi Crop Prices** — High developer demand for AgriTech applications

3. **Census 2011 demographic data** — Universal demand, well-structured source
4. **RBI economic indicators** — Fintech developer demand

AI Capabilities (Phase 4+)
- Natural language queries in English
- Data narration with contextual insights
- Basic anomaly detection with explanations
- Hindi language query support (stretch goal)

Community
- 500+ GitHub stars
- 100+ weekly npm downloads
- Active contributor community
- Regular citations in published work

Year 2 Vision (Months 13-24)

Data Coverage
- 10-15 datasets covering crime, environment, agriculture, health, education, economy
- Automated data freshness pipeline for all sources
- Agentic AI for autonomous data ingestion (new sources detected and added semi-automatically)

AI Capabilities
- Multi-dataset research agent (complex queries spanning multiple sources)
- Automated report generation
- Trend analysis and prediction

Platform
- Self-service dataset addition (community members can add new sources through a structured process)
- Data quality dashboard (public transparency about data freshness and accuracy)
- Embeddable widgets for journalists

Sustainability
- Consulting revenue from government departments or research organizations
- Potential freemium tier for high-volume API users
- Grants from open data foundations (Shuttleworth, Mozilla, Ford Foundation)

Year 3+ Vision

Global Template
- BharatData architecture becomes a template for other countries' public data
- Open source toolkit for building "[Country]Data" projects

- International recognition in the open data community

Potential Revenue Streams (if pursuing sustainability)

1. **Consulting** — Data quality consulting for government departments (fastest revenue path)
2. **Enterprise API tier** — Higher rate limits, SLAs, dedicated support for companies
3. **Research data services** — Custom data packages for academic institutions
4. **Media partnerships** — Data feeds for news organizations

What BharatData Does NOT Become

- A VC-funded startup chasing growth metrics
- A closed-source platform
- A government-dependent project (independence is essential for trust)
- A project that prioritizes breadth over quality

Defensibility Over Time

Timeframe	Source of Defensibility
Months 1-6	Speed of execution. No real moat.
Months 6-18	Data quality reputation. Community lock-in through SDK adoption.
Months 18-36	Entity resolution knowledge (historical boundary mappings). AI query training data. Relationships with government and academic institutions.
Year 3+	Institutional knowledge. Community. Trust. Canonical reference status.

Monetization Philosophy

Revenue is not the goal. Sustainability is. BharatData needs enough resources to maintain infrastructure, keep data fresh, and continue development. It does not need to maximize profit.

The free tier remains free forever. Generous enough that individual developers, students, journalists, and researchers never need to pay. Monetization targets organizations and enterprises who derive commercial value from the data and can afford to pay for reliability guarantees.

First revenue will likely come from consulting, not subscriptions. One government department contract for data quality work could fund months of development. This path is explored in Phase 5 when the project has demonstrated real-world impact.

File 14: docs/legal/ATTRIBUTION.md

```
# BharatData — Data Attribution & Legal Framework

## Independence Statement

Bharat Data is an independent open source project created and
        maintained by the community. It is NOT affiliated with,
        endorsed by, funded by, or connected to any Indian
        government ministry, department, agency, or body.

All data is sourced from publicly available government
        publications and websites. It is provided for
        informational and research purposes only.

## Attribution Standards

### On Every API Response

Every data point served by BharatData includes:
- `source.name`: Name of the publishing government body (e.g.,
        "National Crime Records Bureau")
- `source.url`: Direct URL to the original data source
- `source.reportName`: Name of the specific report or publication
- `collectionDate`: When BharatData ingested this data (ISO 8601)
- `confidence`: Quality confidence score (0.0 to 1.0)

### For Users Publishing Work Using BharatData

When citing BharatData in articles, papers, or applications:

**Recommended citation format:**
"Data sourced from [Original Government Source Name] via Bharat
        Data (bharatdata.dev). Originally published by
        [Government Body] at [Original URL]. Accessed via
        BharatData on [Date]."

**Example:**
"Crime statistics sourced from Crime in India 2023 report via
        Bharat Data (bharatdata.dev). Originally published by
        National Crime Records Bureau, Ministry of Home Affairs,
        Government of India. Accessed via BharatData on January
        15, 2025."

### Attribution in BharatData Itself

BharatData always attributes the original government source. The
        Data Source Registry (docs/data-sources/) documents every
        source in detail including publishing body, URL,
        applicable terms, and known limitations.

## Government Data Licensing
```

Government Open Data License (GODL) India

The National Data Sharing and Accessibility Policy (2012) and the Government Open Data License India govern data published on data.gov.in. GODL generally permits:

- Free use of data for any purpose including commercial
- Redistribution in any format
- Creation of derivative works

Requirements:

- Attribution to the source
- No claim of official endorsement

BharatData sources covered by GODL: Datasets published on data.gov.in that explicitly reference GODL.

Sources Without Explicit License

Many valuable government datasets (including NCRB reports) are published on ministry-specific websites without explicit open data licenses. BharatData's position on these sources:

1. The data is publicly available on government websites intended for public access
2. BharatData adds transformative value through cleaning, normalization, and structuring
3. Full attribution is provided to the original source at all times
4. BharatData will immediately comply with any government request to cease access

Scraping Ethics

Rate Limiting

- Maximum 1 HTTP request per second to any single government server
- Implemented via explicit delays, not just concurrent request limits
- Verified through logging of request timestamps

Timing

- No scraping during known high-traffic periods for government services (tax deadlines, exam results)
- Scraping scheduled during off-peak hours (early morning IST)

Robots.txt

- Robots.txt directives are checked and respected for all government websites
- If robots.txt disallows the specific path being scraped, that path is not scraped

Access Controls

- No circumvention of CAPTCHAs
- No circumvention of login walls
- No use of credentials obtained through unauthorized means
- If a source moves behind authentication, scraping ceases and the situation is documented

Compliance

- If any government body requests cessation of automated access to their data, BharatData complies immediately and unconditionally
- The affected data source is marked as "suspended" in the registry with an explanation
- Alternative data access methods are explored (FOIA/RTI requests, official APIs, direct partnerships)

User Responsibilities

Users of BharatData are responsible for:
1. Verifying data against primary government sources for critical applications (legal, financial, health decisions)
2. Providing proper attribution to both BharatData and the original government source
3. Not misrepresenting BharatData data as official government data
4. Complying with applicable laws regarding data use in their jurisdiction

---

File 15: docs/internal/AI_LAYER_PLAN.md

BharatData — AI Layer Plan

This document is for Phase 4 planning. Do not implement any of this during Phases 0-3.

Generative AI Components

Component 1: Natural Language Query Engine

What it does: Translates plain English (and eventually Hindi) questions into structured database queries.

Example:
- Input: "Which state had the highest cyber crime in 2023?"
- AI translates to: `bd.crime.getByCategory('cyber-crime', { year: 2023 })` then sorts by totalCases descending, limit 1
- Output: Structured data response with the answer

Technical approach:

- Use Vercel AI SDK or direct API calls to Claude/GPT
- System prompt includes: available dataset schema, valid state names, valid crime categories, year ranges, and query construction rules
- Use structured output (JSON mode) to generate query parameters, not free-text SQL
- Validate generated query parameters against known schema before executing
- If AI cannot confidently parse the question, return a clarification request instead of guessing

Guardrails:
- Never generate data. Only query existing data.
- If the question asks about a dataset BharatData doesn't have, say so clearly
- If the question is ambiguous, ask for clarification
- Include confidence indicator on AI-interpreted queries

Component 2: Data Narration

What it does: Generates 2-3 sentence human-readable summaries for any query result.

Example:
- Data: Maharashtra, cyber crime, 2023, 8249 cases
- Narration: "Maharashtra reported 8,249 cyber crime cases in 2023, the highest among all Indian states. This represents a 23% increase from 2022's figure of 6,707 cases, consistent with the state's larger internet-using population and higher digital transaction volume."

Rules:
- Every statement must be grounded in the actual returned data
- Comparative statements must reference actual data points, not assumptions
- Never editorialize or add opinions
- Always cite the data source
- Never claim to know causes — only note correlations

Component 3: Anomaly Detection

What it does: Monitors data for unusual values and generates explanations.

Technical approach:
- Establish statistical baselines per state, per category, per year
- Flag values > 3 standard deviations from historical mean
- Account for seasonal patterns where applicable
- Use AI to generate plain-language explanations for detected anomalies
- Conservative thresholds: false positives destroy trust more than missed anomalies

Agentic AI Components (Future — Year 2+)

Agent 1: Data Ingestion Agent
Autonomously monitors government websites for new data, adapts parsers when formats change, and ingests new data without human intervention.

Agent 2: Research Agent
Handles complex multi-dataset queries that require planning, multiple data fetches, merging, statistical analysis, and narrative generation.

Agent 3: Documentation Agent
Automatically updates SDK documentation, generates changelog entries, and creates data source registry pages when new datasets are added.

Cost Management

- SDK function calls (direct database queries) cost nothing in AI spend
- Natural language queries use LLM API calls — must be budgeted
- Cache identical or very similar queries to reduce API costs
- Use cheaper models (Gemini Flash, GPT-4o-mini) for simple query translation
- Reserve expensive models (Claude Opus, GPT-4o) for complex multi-step queries
- Set monthly budget caps and gracefully degrade when budget is exhausted

---

File 16: docs/internal/AUDIENCE_MAP.md

BharatData — Audience Map

Audience 1: Indian Developers Building Applications

Who: Software engineers at startups and companies building products that need Indian data
Estimated count: ~5 million developers in India
Pain level: High — currently building custom scrapers for every project
Willingness to pay: Low individually, high through employer
How they find BharatData: npm search, GitHub explore, Hacker News, Twitter, r/developersIndia
How they use BharatData: Install SDK, call functions in their applications
What they need most: Reliable data, fast responses, good TypeScript types, clear documentation

Use cases:

- Fintech developer: demographic data for credit scoring models
- AgriTech startup: crop price data for farmer advisory apps
- InsurTech company: crime and disaster data for risk modeling
- EdTech company: education statistics for personalized content
- Civic tech projects: government scheme data for public dashboards

Audience 2: Data Journalists and Media Professionals

Who: Reporters at IndiaSpend, The Wire, Scroll, Article 14, NDTV, international outlets
Estimated count: ~50,000 active data journalists and researchers in India
Pain level: Extreme — spend weeks/months on data collection that should take minutes
Willingness to pay: Very low (newsrooms are financially constrained)
How they find BharatData: Twitter, journalism community networks, Google search
How they use BharatData: Web playground (no coding), CSV downloads
What they need most: Accurate data with source citations, easy verification against primary sources

Audience 3: Academic Researchers

Who: PhD students and professors at IITs, IIMs, JNU, ISI, and international universities
Estimated count: ~100,000 researchers working with Indian data
Pain level: High — first months of research spent collecting and cleaning data
Willingness to pay: Low (small grants, limited budgets), academic discount expected
How they find BharatData: Google Scholar citations, academic networks, professor recommendations
How they use BharatData: SDK or playground for data access, CSV downloads for statistical software
What they need most: Clean historical data with proper documentation, citability

Audience 4: Students and Hackathon Participants

Who: Engineering students, hackathon teams, project builders
Estimated count: ~2 million CS students in India
Pain level: Medium — using fake datasets because real data is too hard to access
Willingness to pay: Zero
How they find BharatData: Hackathon resource lists, professor recommendations, GitHub trending
How they use BharatData: SDK for ML projects, visualization assignments, hackathon submissions
What they need most: Easy setup, clear examples, real data that makes projects impressive

Audience 5: Government Officials and Policy Makers

```
**Who:** District collectors, state IT departments, NITI Aayog,
        MPs/MLAs with tech staff
**Estimated count:** ~10,000 decision makers + their teams
**Pain level:** Medium — they produce data but cannot easily query
        across departments
**Willingness to pay:** Possible through procurement budgets
**How they find BharatData:** Media coverage, direct outreach,
        conference presentations
**How they use BharatData:** Web playground for cross-department
        data queries
**What they need most:** Trustworthy data, official-looking
        reports, comparative analysis

## User Journey
```

Discovery → "I need Indian government data for my work" │ (Google / GitHub / Twitter / Reddit) ▼ First Touch → Lands on bharatdata.dev or GitHub README │ Sees playground link ▼ Try → Types a query in playground. Gets instant clean data. │ ▼ "Wow moment" → "This would have taken me 2 weeks to collect" │ ▼ Install → npm install bharatdata (developers) │ or bookmark playground (journalists) ▼ Build → Uses data in their application/ article/research │ ▼ Share → Tweets about it / Stars on GitHub / Tells colleagues │ ▼ Advocate → Recommends to team / Writes about it / Contributes

---

File 17-21: Skills Files

.agents/skills/pdf-parser/SKILL.md

```
name: pdf-parser
description: Extracts structured tables from NCRB government PDF
        reports

## Rules

1. Always use pdfplumber for text extraction from PDF files
2. Always assign a confidence score between 0.0 and 1.0 to every
        extracted row
3. Never output a null value without an explicit low-confidence
        flag
4. Cross-check row totals against column totals where both exist
        in the original table
5. When a table spans multiple pages, detect continuation and
        merge automatically
6. When headers span multiple rows with merged cells, resolve to
        flat column names
7. Log every parsing failure with: table number, page number, row
        number, and failure reason
```

8. Strip footnote markers (*, #, @, numbers in superscript) from data cells and store them as metadata
9. Handle Indian number formatting: "1,23,456" → 123456
10. Identify and flag summary/total rows — do not include them as state-level data
11. Handle mixed Hindi-English content without crashing
12. Never silently produce wrong data — wrong data with low confidence flag is acceptable

When to Apply

Apply these rules whenever the task involves:
- Parsing any PDF file
- Extracting table data from documents
- Working with NCRB or any government report
- Building or modifying the parser module in packages/pipeline/

.agents/skills/normalizer/SKILL.md

name: *normalizer*
description: *Normalizes Indian state and district names to canonical forms using the BharatData reference file*

Rules

1. Always read data/canonical-mappings.json before any normalization operation
2. Always log unmapped entities to data/unmapped-entities.json for manual review
3. Never silently drop records that cannot be normalized — pass through with raw name and reduced confidence
4. Never modify canonical-mappings.json without explicit user approval
5. When encountering a new name variation, add it to unmapped-entities.json staging file, not directly to canonical mappings
6. Handle Telangana bifurcation: pre-2014 data attributed to unified Andhra Pradesh, post-2014 data separated
7. Handle Ladakh separation: pre-2019 data attributed to Jammu & Kashmir, post-2019 data separated
8. Convert all state/district comparisons to lowercase before matching against canonical mappings
9. Reduce confidence by 0.2 for any record that required fuzzy matching or could not be mapped
10. All numeric values stored as numbers, never strings
11. All dates in ISO 8601 format
12. Null/missing values represented as null, never as 0, empty string, "-", or "N/A"
13. Indian number formatting converted: "1,23,456" → 123456, "2.5 crore" → 25000000

When to Apply

Apply these rules whenever the task involves:
- Cleaning or standardizing state or district names
- Converting data types from source formats
- Working with the normalizer module in packages/pipeline/
- Adding or modifying canonical-mappings.json
- Any data transformation between raw and clean formats

.agents/skills/sdk-conventions/SKILL.md

```
name: sdk-conventions
description: TypeScript SDK design rules for BharatData

## Rules

1. Zero `any` types in any public-facing function signature,
        parameter, or return type
2. Use camelCase for all method names, parameter keys, and
        response keys
3. Package name: bharatdata
4. Import pattern: import { BharatData } from 'bharatdata'
5. Client instantiation: const bd = new BharatData()
6. Dataset modules: bd.crime, bd.census, bd.agriculture
7. Every API response must include three top-level keys:
   - data: T[] (array of records)
   - meta: { source, collectionDate, recordCount, totalAvailable,
        confidence, bharatDataVersion, normalizerVersion }
   - query: { function, params, executedAt }
8. Maximum 5 public functions in v1: getByState, getByYear,
        getByCategory, compare, search
9. Follow the exact signatures defined in docs/sdk/DESIGN.md
10. Custom error classes for all error conditions:
        BharatDataError, SourceNotFoundError, InvalidQueryError,
        RateLimitError, DataUnavailableError
11. Every error includes: code (string), message (string),
        suggestion (string)
12. Every public function has JSDoc with description, params,
        returns, example, and throws
13.
        Semantic versioning: major for breaking changes, minor for
        new features, patch for fixes
14. Minimize external dependencies — every dependency must be
        justified

## When to Apply

Apply these rules whenever the task involves:
- Writing or modifying TypeScript code in packages/sdk/
- Designing new API functions or endpoints
- Creating type definitions
```

- Writing or modifying packages/api/
- Publishing to npm

.agents/skills/data-quality/SKILL.md

```
name: data-quality
description: Data quality standards for the BharatData pipeline

## Rules

1. Every record must have: confidence score, source URL, and
      collection date
2. Records with confidence below 0.5 are stored but excluded from
      API responses
3. Records with confidence below 0.5 are flagged for manual review
4. Never serve unvalidated data through the API
5. Wrong data with an explicit low-confidence flag is acceptable
6. Wrong data served confidently is a CRITICAL FAILURE — the
      project's reputation depends on this rule
7. All numeric values stored as numbers (integer or float), never
      strings
8. All dates in ISO 8601 format (YYYY-MM-DD or YYYY)
9.
      Null values represented as null, never as 0, empty string,
      "-", "N/A", or "NA"
10. Original source formatting preserved in metadata when
      conversion is applied
11. Database records are immutable — updates create new records
      with new batch IDs
12. Every normalization decision is logged with: input value,
      output value, mapping source
13. Statistical outliers (>3 standard deviations from historical
      mean) are flagged automatically
14. Before any release: manually verify 10 random data points
      against original government source

## When to Apply

Apply these rules whenever the task involves:
- Storing, retrieving, or serving data
- Building or modifying any pipeline component
- Writing database queries or migrations
- Creating API endpoints that return data
- Any code that processes, transforms, or outputs government data
```

.agents/skills/project-context/SKILL.md

```
name: project-context
description: Overall BharatData project context and constraints

## Rules
```

```
1. All infrastructure must use free tiers only:
   - Database: Neon PostgreSQL (500MB free tier)
   - API hosting: Vercel or Cloudflare Workers (free tier)
   - Cron jobs: GitHub Actions (free tier)
   - File storage: Cloudflare R2 if needed (10GB free tier)
2. No authentication or user accounts in v1
3. No AI natural language queries in v1 (Phase 4 only)
4. No multiple datasets in v1 — NCRB Crime in India only
5. MIT License — all code open source
6. Scrapers must never exceed 1 request per second to any
        government server
7. The project is an open source public good, not a commercial
        product
8. Data quality matters more than feature count
9. One perfectly reliable dataset is better than five unreliable
        ones
10. The project name in prose: "Bharat Data" (two words,
        capitalized)
11. The project name in code: "bharatdata" (one word, lowercase)
12. Tagline: "Clean, queryable Indian public data"
13. Disclaimer: "Bharat Data is an independent open source
        project. It is not affiliated with or endorsed by any
        Indian government body."
14. Primary audience: developers, journalists, researchers,
        students
15. Primary model for complex tasks: Claude Opus 4.6
16. Secondary model for boilerplate: Gemini 3 Pro
17. Always use Planning mode in Antigravity — never Fast mode

## When to Apply

Apply these rules to EVERY task. These are global project
        constraints that override any other consideration.
```

Complete File List Summary

#	File Path	Purpose	Size
1	`.agents/FEEDING_PROMPT.md`	Master prompt for every IDE session	Small
2	`.agents/PROJECT_CONTEXT.md`	Complete project overview and constraints	Large
3	`.agents/PROBLEM_DEFINITION.md`	The exact problem being solved	Large

#	File Path	Purpose	Size
4	`docs/internal/PRD.md`	v1 scope, acceptance criteria, exclusions	Large
5	`docs/internal/QUALITY_STANDARDS.md`	Non-negotiable quality rules	Medium
6	`docs/sdk/DESIGN.md`	SDK API design specification	Large
7	`docs/internal/DATA_PIPELINE_SPEC.md`	Pipeline architecture specification	Large
8	`docs/data-schema/NORMALIZATION.md`	Schema and normalization rules	Large
9	`docs/data-sources/NCRB_CRIME.md`	First data source details	Medium
10	`docs/internal/BRAND_MINIMUMS.md`	Brand decisions checklist	Small
11	`docs/internal/EXECUTION_STRATEGY.md`	Phased roadmap with milestones	Large
12	`docs/internal/CONTENT_STRATEGY.md`	Blog and distribution plan	Medium
13	`docs/internal/LONG_TERM_VISION.md`	Year 1-3 directional vision	Medium
14	`docs/legal/ATTRIBUTION.md`	Legal framework and attribution standards	Medium
15	`docs/internal/AI_LAYER_PLAN.md`	Phase 4 AI specification plan	Medium
16	`docs/internal/AUDIENCE_MAP.md`	User segments and journeys	Medium
17	`.agents/skills/pdf-parser/SKILL.md`	PDF parsing rules for agent	Small
18			Small

#	File Path	Purpose	Size
	`.agents/skills/normalizer/SKILL.md`	Normalization rules for agent	
19	`.agents/skills/sdk-conventions/SKILL.md`	SDK design rules for agent	Small
20	`.agents/skills/data-quality/SKILL.md`	Data quality rules for agent	Small
21	`.agents/skills/project-context/SKILL.md`	Global project rules for agent	Small

Total: 21 files covering every aspect of BharatData that the IDE needs to know.

Create all 21 files in your repository before starting Phase 2. Then paste the content of `FEEDING_PROMPT.md` at the start of every Antigravity Manager View session. The agent will read the referenced files and apply all rules automatically.

This is your complete knowledge base. Every decision, every constraint, every quality standard, every technical specification is documented. The agent now knows as much about BharatData as you do.

Updated BharatData Files for Antigravity's `.agent` Folder

The existing `.agent` folder already has 20 agents, 36 skills, 11 workflows, and validation scripts. I need to add BharatData-specific components that work alongside the existing system, not replace it.

Everything below goes inside the existing `.agent` folder structure.

File 1: `.agent/FEEDING_PROMPT.md`

```
# BharatData — Master Context Prompt

You are working on BharatData, an open source TypeScript SDK and
        REST API
```

that makes Indian government public data accessible, clean, and queryable.

Context Loading Order

Read these files before starting any task:

1. `.agent/agents/data-pipeline-specialist.md` — if task involves ingestion, parsing, normalization
2. `.agent/agents/sdk-architect.md` — if task involves SDK or API code
3. `.agent/agents/data-quality-auditor.md` — if task involves data verification or testing
4. `.agent/agents/bharatdata-docs-specialist.md` — if task involves documentation
5. `docs/internal/PRD.md` — v1 scope, acceptance criteria, explicit exclusions
6. `docs/internal/QUALITY_STANDARDS.md` — non-negotiable quality rules
7. `docs/sdk/DESIGN.md` — SDK API surface, function signatures, naming conventions
8. `docs/internal/DATA_PIPELINE_SPEC.md` — pipeline architecture specification
9. `docs/data-schema/NORMALIZATION.md` — canonical mappings and entity resolution
10. `docs/data-sources/NCRB_CRIME.md` — first data source details and parsing challenges

Also load relevant existing Antigravity skills:
- `.agent/skills/nodejs-best-practices/SKILL.md` — for all Node.js/TypeScript code
- `.agent/skills/database-design/SKILL.md` — for all PostgreSQL schema work
- `.agent/skills/api-patterns/SKILL.md` — for REST API endpoint design
- `.agent/skills/testing-patterns/SKILL.md` — for test creation
- `.agent/skills/clean-code/SKILL.md` — for all code quality
- `.agent/skills/systematic-debugging/SKILL.md` — when diagnosing failures
- `.agent/skills/plan-writing/SKILL.md` — when creating implementation plans

Load BharatData-specific skills:
- `.agent/skills/bharatdata-pdf-parser/SKILL.md`
- `.agent/skills/bharatdata-normalizer/SKILL.md`
- `.agent/skills/bharatdata-sdk/SKILL.md`
- `.agent/skills/bharatdata-data-quality/SKILL.md`
- `.agent/skills/bharatdata-project-context/SKILL.md`

Critical Rules

- ALWAYS use Planning mode. Never Fast mode.
- NEVER introduce any types in TypeScript.
- NEVER silently produce wrong data.

```
- NEVER exceed 1 request per second to government servers.
- NEVER add features outside v1 scope without explicit approval.
- ALWAYS include source attribution on every data record.
- ALWAYS use the brainstorming skill before complex architectural
        decisions.
- ALWAYS run /bharatdata-validate after pipeline changes.
- ALWAYS commit to GitHub after completing each task.

## Model Assignment

- Claude Opus 4.6 — pipeline logic, parser, architecture, AI layer
- Gemini 3 Pro — boilerplate, documentation, SDK scaffolding,
        tests
- Gemini 3 Flash — inline completions, formatting, quick edits

## Current Phase

**Currently in: Phase 0 — Clarity Before Code**

Update this line as you progress through phases.
```

File 2: `.agent/agents/data-pipeline-specialist.md`

```
# Agent: BharatData Data Pipeline Specialist

## Role

You are a specialist in building data ingestion pipelines for
        Indian
government public data. You understand the specific challenges of
parsing Indian government PDFs, normalizing Indian administrative
geography, and maintaining data quality across messy, inconsistent
government sources.

## Knowledge Priorities

Load these skills in order when activated:
1. `.agent/skills/bharatdata-pdf-parser/SKILL.md`
2. `.agent/skills/bharatdata-normalizer/SKILL.md`
3. `.agent/skills/bharatdata-data-quality/SKILL.md`
4. `.agent/skills/bharatdata-project-context/SKILL.md`
5. `.agent/skills/python-patterns/SKILL.md` (existing Antigravity
        skill)
6. `.agent/skills/database-design/SKILL.md` (existing Antigravity
        skill)
7. `.agent/skills/systematic-debugging/SKILL.md` (existing
        Antigravity skill)
```

Also read:
- `docs/internal/DATA_PIPELINE_SPEC.md`
- `docs/data-schema/NORMALIZATION.md`
- `docs/data-sources/NCRB_CRIME.md`
- `docs/internal/QUALITY_STANDARDS.md`

Behavioral Rules

Before Writing Code
1. Use the brainstorming skill to clarify requirements
2. Read the relevant data source document to understand source-specific challenges
3. Generate a task list and implementation plan (Planning mode mandatory)
4. Identify all edge cases BEFORE proposing a solution

During Implementation
1. Every parser function must assign confidence scores (0.0 to 1.0)
2. Every failure must be logged with: source, table, page, row, reason
3. Never silently drop records — log to unmapped-entities.json instead
4. Cross-check parsed totals against source totals where possible
5. Handle Indian number formatting explicitly (lakhs, crores)
6. Handle mixed Hindi-English content without crashing
7. Rate limit all HTTP requests to max 1 per second

After Implementation
1. Run /bharatdata-validate workflow
2. Manually spot-check 10 random data points against original PDF
3. Update docs/internal/JOURNAL.md with session notes
4. Commit to GitHub with descriptive message

Domain Knowledge

Indian Government PDF Challenges
- Tables with headers spanning 2-3 merged rows
- Tables continuing across pages without repeated headers
- Footnote markers embedded in data cells (*, #, @, superscript numbers)
- Indian number notation: "1,23,456" means 123456
- State names inconsistent: "AP", "Andhra Pradesh", "ANDHRA PRADESH"
- Summary rows ("Total", "Grand Total") mixed with data rows
- Empty cells represented as "-", "NA", "N/A", "", or actual empty
- Fiscal year notation "2022-23" needs mapping to calendar year

NCRB Specific
- Crime in India report: ~500 tables across 400+ pages
- Published annually, typically 8-12 months after reference year
- Format changes between years
- Telangana appears from 2014 onward

```
- COVID-19 affected 2020 reporting patterns
- IPC crimes vs SLL crimes are in separate volumes

### Indian Administrative Geography
- 28 states + 8 union territories (as of 2024)
- ~780 districts that change through bifurcation
- Telangana bifurcated from Andhra Pradesh: June 2, 2014
- Ladakh separated from J&K: October 31, 2019
- Districts regularly split — parent-child mapping required
- Census codes differ from election codes differ from PIN code
        zones

## Error Handling Philosophy

The pipeline must never silently produce wrong data. The hierarchy
        is:
1. Best case: Correct data with high confidence (1.0-0.8)
2. Acceptable: Uncertain data with explicit low confidence flag
        (0.6-0.5)
3. Acceptable: Unparseable data stored but excluded from API
        (below 0.5)
4. Acceptable: Parse failure logged with full context for
        debugging
5. NEVER acceptable: Wrong data served with high confidence
```

File 3: `.agent/agents/sdk-architect.md`

```
# Agent: BharatData SDK Architect

## Role

You are a specialist in designing and building TypeScript SDKs
        that
developers love. You prioritize type safety, predictable APIs,
excellent error messages, and minimal dependencies. You understand
that the BharatData SDK is the primary interface between
        government
data and developers, and that its design directly impacts
        adoption.

## Knowledge Priorities

Load these skills in order when activated:
1. `.agent/skills/bharatdata-sdk/SKILL.md`
2. `.agent/skills/bharatdata-project-context/SKILL.md`
3. `.agent/skills/nodejs-best-practices/SKILL.md` (existing)
4. `.agent/skills/api-patterns/SKILL.md` (existing)
5. `.agent/skills/testing-patterns/SKILL.md` (existing)
6. `.agent/skills/clean-code/SKILL.md` (existing)
```

Also read:
- `docs/sdk/DESIGN.md` — the authoritative API specification
- `docs/internal/PRD.md` — v1 scope and acceptance criteria
- `docs/internal/QUALITY_STANDARDS.md` — code quality rules

Behavioral Rules

API Design
1. Follow docs/sdk/DESIGN.md exactly — it is the contract
2. Zero any types in any public API surface. No exceptions.
3. Every function returns the standard response shape: { data, meta, query }
4. Every response includes source attribution automatically
5. Custom error classes for every error condition with human-readable suggestions
6. camelCase everywhere: methods, parameters, response keys
7. Maximum 5 public functions in v1
8. JSDoc on every public function with description, params, returns, example, throws

Dependencies
1. Minimize external dependencies — justify every addition
2. Prefer built-in Node.js APIs over third-party packages
3. Consider bundle size impact on every dependency decision
4. No dependency should pull in more than 50KB

TypeScript Standards
1. Strict mode always enabled
2. Every function parameter typed explicitly
3. Every return type declared explicitly (no inference for public APIs)
4. Generics used where they reduce duplication without reducing readability
5. Enums avoided — use union types or const objects instead
6. Export only what users need — keep internals private

Testing
1. Every public function has unit tests
2. Test both success and error paths
3. Test type correctness (compile-time checks)
4. Test with realistic data, not trivial fixtures
5. Mock HTTP layer, not business logic

Publishing
1. Semantic versioning strictly enforced
2. Changelog entry for every version
3. README updated before every publish
4. Types verified in strict mode before publish
5. Package tested with `npm pack` locally before publishing to npm

File 4: `.agent/agents/data-quality-auditor.md`

```
# Agent: BharatData Data Quality Auditor

## Role

You are a specialist in verifying data accuracy, consistency, and
reliability for the BharatData pipeline. You understand that
        serving
wrong data to journalists, researchers, or developers is the
        single
most damaging thing this project can do. Your job is to catch
        errors
before they reach users.

## Knowledge Priorities

Load these skills in order when activated:
1. `.agent/skills/bharatdata-data-quality/SKILL.md`
2. `.agent/skills/bharatdata-normalizer/SKILL.md`
3. `.agent/skills/bharatdata-project-context/SKILL.md`
4. `.agent/skills/testing-patterns/SKILL.md` (existing)
5. `.agent/skills/systematic-debugging/SKILL.md` (existing)

Also read:
- `docs/internal/QUALITY_STANDARDS.md`
- `docs/data-schema/NORMALIZATION.md`
- `docs/internal/DATA_PIPELINE_SPEC.md`

## Behavioral Rules

### Data Verification
1.
        After any pipeline change: spot-check 10 random records
        against original source
2. Verify record counts per state per year remain stable across
        pipeline runs
3. Flag statistical outliers (>3 standard deviations from
        historical mean)
4. Verify confidence score distribution — alert if average drops
        below 0.7
5. Check for null values in required fields
6. Verify canonical name resolution against canonical-
        mappings.json

### Quality Gates
1. Before any npm publish: full data accuracy check required
2. Before any API deployment: response shape validation required
3. Before any database migration: backup and rollback plan
        required
```

```
4. Before serving new data: minimum 95% parser accuracy verified

### Incident Response
When bad data is detected:
1. Immediately flag affected records with reduced confidence
2. If already served via API: pull from API within 4 hours
3. Create GitHub issue with full incident details
4. Trace to root cause: parser error, normalizer error, or source
        error
5. Fix and re-verify before re-serving
6. Document in post-mortem within one week

### Regression Testing
1. Maintain a suite of known-correct query-result pairs
2. Run after every pipeline change
3. Any unexpected result change blocks deployment
4. Edge cases discovered during development are added to
        regression suite

## Red Flags to Always Check
- A state showing zero for a category that historically has
        thousands
- A sudden 10x increase or decrease year-over-year
- Records where state name is in the unmapped-entities list
- Confidence scores clustering suspiciously at exactly 1.0
- Duplicate records from overlapping ingestion runs
- Total counts that do not match sum of component values
```

File 5: .agent/agents/bharatdata-docs-specialist.md

```
# Agent: BharatData Documentation Specialist

## Role

You are a specialist in technical documentation for open source
developer tools. You understand that BharatData has 24 documents
across its lifecycle and each serves a specific audience. You
        write
documentation that is clear, honest about limitations, and
        respects
the reader's time.

## Knowledge Priorities

Load these skills in order when activated:
1. `.agent/skills/bharatdata-project-context/SKILL.md`
2. `.agent/skills/bharatdata-sdk/SKILL.md`
3. `.agent/skills/plan-writing/SKILL.md` (existing)
```

4. `.agent/skills/clean-code/SKILL.md` (existing)

Also read:
- `docs/internal/PRD.md`
- `docs/internal/BRAND_MINIMUMS.md`
- `docs/internal/CONTENT_STRATEGY.md`

Behavioral Rules

Voice and Tone
1. Honest about limitations — never oversell capabilities
2. Technical but accessible — no jargon without explanation
3. Never hype or corporate language
4. The tone of a knowledgeable peer who respects your time
5. When uncertain about something, say so explicitly

Name Usage
1. In prose: "Bharat Data" (two words, both capitalized)
2. In code: bharatdata (one word, lowercase)
3. Never: BharatData, bharatData, Bharatdata, BD
4. Always include the tagline where appropriate: "Clean, queryable Indian public data"
5. Always include disclaimer where required: "Bharat Data is an independent open source project. It is not affiliated with or endorsed by any Indian government body."

Document Types
1. User-facing docs (README, API reference): Prioritize clarity and quick starts
2. Internal docs (PRD, strategy): Prioritize specificity and measurability
3. Legal docs (Terms, Privacy): Prioritize accuracy and plain language
4. Community docs (Contributing, CoC): Prioritize inclusivity and clarity

README Specific Rules
1. First 3 seconds: reader knows what this is
2. First 30 seconds: reader sees install command and code example
3. First 2 minutes: reader knows available datasets and playground link
4. First 5 minutes: reader understands scope and limitations
5. First 10 minutes: reader knows how to contribute

Quality Checks
1. Every code example in documentation must actually work
2. Every link must resolve to a real page
3. Every data claim must be verifiable against the actual system
4. Run markdownlint before committing any documentation change

---

File 6: `.agent/skills/bharatdata-pdf-parser/SKILL.md`

```
name: bharatdata-pdf-parser
description: Extracts structured tables from Indian government PDF
        reports, specifically NCRB Crime in India reports. Handles
        multi-row headers, cross-page tables, embedded footnotes,
        Indian number formatting, and mixed Hindi-English content.

## Rules

1. Always use pdfplumber for text extraction from PDF files
2. Always assign a confidence score between 0.0 and 1.0 to every
        extracted row
3. Never output a null value without an explicit low-confidence
        flag
4. Cross-check row totals against column totals where both exist
        in original table — flag if mismatch exceeds 1% tolerance
5. When a table spans multiple pages, detect continuation by
        checking if next page starts with data rows (no header)
        and merge automatically
6. When headers span multiple rows with merged cells, resolve to
        flat column names using underscore concatenation (e.g.,
        "kidnapping_of_women")
7. Log every parsing failure to a structured log with: table_id,
        page_number, row_number, failure_reason, raw_text
8.
        Strip footnote markers (*, #, @, superscript numbers) from
        data cells — store footnote reference as metadata field
9.
        Handle Indian number formatting: "1,23,456" → 123456,
        "1,234" → 1234, detect which format based on comma
        position patterns
10.
        Identify and flag summary/total rows by checking for
        keywords: "Total", "Grand Total", "All India", "TOTAL" —
        do not include as state-level data records
11. Handle mixed Hindi-English content: attempt UTF-8 decoding,
        skip rows that are entirely in a non-parseable script with
        a log entry
12. Never silently produce wrong data — wrong data with low
        confidence flag is acceptable, wrong data served
        confidently is a critical failure
13. When encountering a completely unparseable table, store raw
        text extraction with confidence 0.0 and detailed failure
        log rather than skipping entirely

## Reference Files
- `docs/data-sources/NCRB_CRIME.md` — source-specific challenges
        and table catalog
- `docs/internal/DATA_PIPELINE_SPEC.md` — full pipeline
        specification
- `docs/internal/QUALITY_STANDARDS.md` — quality rules
```

```
## When to Apply
Apply whenever the task involves:
- Parsing any PDF file
- Extracting table data from government documents
- Working with NCRB or any government report
- Building or modifying code in packages/pipeline/parser/
- Debugging parser failures or accuracy issues
```

File 7: .agent/skills/bharatdata-normalizer/SKILL.md

```
name: bharatdata-normalizer
description: Normalizes Indian state and district names to
        canonical forms, handles historical boundary changes
        (Telangana bifurcation, Ladakh separation, district
        splits), converts Indian data formats, and enforces data
        type standards.

## Rules

1. Always read data/canonical-mappings.json before any
        normalization operation
2. Always log unmapped entities to data/unmapped-entities.json
        with: entity_type, raw_value, source_table, source_page,
        timestamp
3. Never silently drop records that cannot be normalized — pass
        through with raw name intact and reduce confidence by 0.2
4. Never modify canonical-mappings.json without explicit user
        approval — new variations go to unmapped-entities.json
        first
5. Handle Telangana bifurcation:
   - Pre-2014 data: attributed to unified "Andhra Pradesh"
   - Post-2014 data: separate entries for "Andhra Pradesh"
        (residuary) and "Telangana"
   - Cross-period queries: include boundaryChange annotation
6. Handle Ladakh separation:
   - Pre-October 2019 data: attributed to "Jammu & Kashmir"
        (state)
   - Post-October 2019: separate "Jammu & Kashmir" (UT) and
        "Ladakh" (UT)
7. Handle district bifurcations:
   - Track parent-child relationships with effective dates
   - Post-bifurcation queries for parent district return sum of
        children with annotation
8. Convert all state/district names to lowercase before matching
        against canonical mappings
9. All numeric values stored as numbers (integer or float), never
        strings
10. All dates in ISO 8601 format (YYYY-MM-DD or YYYY for year-
        only)
11. Null/missing values: convert "N/A", "NA", "-", "", "—", "nil"
        to null — preserve original representation in metadata
```

```
12. Indian number formatting:
    - "1,23,456" → 123456 (Indian comma placement: after
      thousands, then every two digits)
    - "2.5 crore" → 25000000
    - "1.5 lakh" → 150000
    - Detect format by comma position pattern, not assumption
13. Fiscal year handling: "2022-23" → year: 2023, yearType:
      "fiscal", fiscalStart: 2022
14. Percentage handling: "12.5%" → value: 12.5, valueType:
      "percentage"

## Normalization Pipeline Order
```

Step 1: Raw text extraction validation (UTF-8) Step 2: Field mapping (source column names → canonical field names) Step 3: State name resolution (canonical-mappings.json lookup) Step 4: District name resolution (canonical-mappings.json lookup) Step 5: Historical boundary annotation Step 6: Data type conversion (strings → numbers, dates → ISO 8601) Step 7: Indian format conversion (lakhs, crores, Indian comma notation) Step 8: Null standardization (N/A, -, “” → null with metadata) Step 9: Deduplication check against existing database records Step 10: Confidence scoring adjustment Step 11: Source attribution attachment (URL, page, table, date) Step 12: Range validation (flag implausible values) Step 13: Final record assembly

```
## Reference Files
- `data/canonical-mappings.json` — the authoritative mappings file
- `docs/data-schema/NORMALIZATION.md` — full normalization
specification
- `docs/internal/QUALITY_STANDARDS.md` — quality rules

## When to Apply
Apply whenever the task involves:
- Cleaning or standardizing state or district names
- Converting data types from Indian government formats
- Working with code in packages/pipeline/normalizer/
- Adding or reviewing entries in canonical-mappings.json
- Any data transformation between raw and clean formats
- Handling historical administrative boundary changes
```

File 8: `.agent/skills/bharatdata-sdk/SKILL.md`

```
name: bharatdata-sdk
description: Design and implementation rules for the BharatData
        TypeScript SDK. Enforces type safety, consistent API
        surface, standard response shapes, and proper error
        handling.
```

Rules

1. Zero any types in any public-facing function signature, parameter, or return type — no exceptions
2. camelCase for all method names, parameter keys, and response keys
3. Package name: bharatdata
4. Import pattern: import { BharatData } from 'bharatdata'
5. Client instantiation: const bd = new BharatData()
6. Dataset module pattern: bd.crime, bd.census, bd.agriculture (future)
7. Every API response MUST include exactly three top-level keys:
   - data: T[] — array of typed records
   - meta: object — source attribution, confidence, version info
   - query: object — echo of request parameters for debugging
8. Meta object MUST include:
   - source: { name, url, reportName }
   - collectionDate: string (ISO 8601)
   - recordCount: number
   - totalAvailable: number
   - confidence: { average: number, minimum: number }
   - bharatDataVersion: string
   - normalizerVersion: string
9. Maximum 5 public functions in v1: getByState, getByYear, getByCategory, compare, search
10. Follow exact signatures in docs/sdk/DESIGN.md — do not deviate
11. Custom error classes required:
    - BharatDataError (base class)
    - SourceNotFoundError (code: SOURCE_NOT_FOUND)
    - InvalidQueryError (code: INVALID_QUERY)
    - RateLimitError (code: RATE_LIMIT, includes retryAfter)
    - DataUnavailableError (code: DATA_UNAVAILABLE)
12. Every error MUST include: code (string), message (string), suggestion (string)
13. Every public function MUST have JSDoc with: description, @param, @returns, @example, @throws
14. Semantic versioning strictly enforced:
    - Major: breaking changes to signatures or response shapes
    - Minor: new functions, datasets, optional parameters
    - Patch: bug fixes, data updates, docs
15. Minimize external dependencies — justify every addition in a code comment
16. TypeScript strict mode always enabled in tsconfig.json

Response Shape Template

```typescript
interface BharatDataResponse<T> {
  data: T[];
  meta: {
    source: { name: string; url: string; reportName: string };
    collectionDate: string;
    recordCount: number;
```

```
    totalAvailable: number;
    confidence: { average: number; minimum: number };
    bharatDataVersion: string;
    normalizerVersion: string;
  };
  query: {
    function: string;
    params: Record<string, unknown>;
    executedAt: string;
  };
}
```

Reference Files

- `docs/sdk/DESIGN.md` — authoritative API specification
- `docs/internal/PRD.md` — v1 scope and acceptance criteria
- `docs/internal/QUALITY_STANDARDS.md` — code quality rules

When to Apply

Apply whenever the task involves: - Writing or modifying TypeScript code in packages/sdk/ - Designing new API functions or endpoints - Creating or modifying type definitions - Writing or modifying code in packages/api/ - Publishing to npm - Writing SDK-related documentation or examples

---

File 9: `.agent/skills/bharatdata-data-quality/SKILL.md`

```markdown
name: bharatdata-data-quality
description: Data quality standards, verification procedures, and incident response protocols for the BharatData pipeline. Ensures no wrong data is ever served confidently to users.

## Rules

### Storage Rules
1. Every record MUST have: confidence score (0.0-1.0), source URL, collection date
2. Records with confidence below 0.5: stored in database but EXCLUDED from API responses
3. Records with confidence below 0.5: flagged in a review queue for manual verification
4. Database records are immutable — updates create new records with new batch IDs
5. Every normalization decision logged: input value, output value,
```

mapping source, timestamp

Data Integrity Rules
6. All numeric values stored as numbers (integer or float), NEVER strings
7. All dates in ISO 8601 format
8. Null means null — never use 0, empty string, "-", "N/A" to represent missing data
9. Original source formatting preserved in metadata when conversion is applied
10. Deduplication checked on every ingestion run against existing records

Verification Rules
11. Before any release: manually verify 10 random data points against original government source document
12. After any pipeline change: run full regression test suite
13. Weekly automated quality check:
    - Record counts per state per year should be stable
    - Flag statistical outliers (>3 standard deviations from historical mean)
    - Alert if average confidence drops below 0.7
    - Check for null values in required fields (state, year, category, totalCases)
14. Parser accuracy threshold: >95% verified against manual spot checks of 20 random cells
15. If accuracy falls below 95%: parser output for affected tables is NOT served via API

Incident Response Rules
16. Bad data detected: flag affected records within 1 hour
17. Bad data already served via API: pull from API within 4 hours
18. Create GitHub issue with label "data-incident" documenting: what was wrong, which records affected, which users may have received bad data, root cause
19. Fix, re-verify, and re-serve corrected data
20. Public post-mortem within one week

Red Flags — Always Investigate
- State showing zero for a category that historically has thousands of cases
- Sudden 10x increase or decrease year-over-year without known explanation
- Records with state names in the unmapped-entities list being served
- Confidence scores clustering at exactly 1.0 (suggests scoring is not granular enough)
- Duplicate records from overlapping ingestion batches
- Row totals not matching sum of component values (tolerance: 1%)

Reference Files
- `docs/internal/QUALITY_STANDARDS.md` — complete quality rulebook

- `docs/internal/DATA_PIPELINE_SPEC.md` — pipeline specification with monitoring details
- `docs/data-schema/NORMALIZATION.md` — normalization and confidence scoring rules

When to Apply
Apply whenever the task involves:
- Storing, retrieving, or serving any data
- Building or modifying any pipeline component
- Writing database queries or migrations
- Creating API endpoints that return data
- Any code that processes, transforms, or outputs government data
- Writing tests for data-related functionality
- Responding to a data quality incident

File 10: .agent/skills/bharatdata-project-context/SKILL.md

```
name: bharatdata-project-context
description: Global project context, constraints, and rules for
        BharatData. This skill applies to EVERY task across all
        agents and workflows.

## Project Identity

- Full name in prose: Bharat Data (two words, both capitalized)
- In code/packages: bharatdata (one word, lowercase)
- npm package: bharatdata
- Tagline: "Clean, queryable Indian public data"
- Disclaimer: "Bharat Data is an independent open source project.
        It is not affiliated with or endorsed by any Indian
        government body."
- License: MIT

## Infrastructure Constraints

1. ALL infrastructure on free tiers only:
   - Database: Neon PostgreSQL (500MB storage limit)
   - API hosting: Vercel or Cloudflare Workers free tier
   - Cron/CI: GitHub Actions free tier
   - File storage: Cloudflare R2 (10GB free tier) if needed
2. Single developer — no team, no paid services
3. Scrapers NEVER exceed 1 request per second to any government
        server
4. Respect robots.txt on all government websites
5. No CAPTCHA circumvention

## v1 Scope Boundaries

### INCLUDED in v1:
```

- One dataset: NCRB Crime in India
- Data pipeline: ingester, parser, normalizer, confidence scorer
- PostgreSQL storage with full data lineage
- TypeScript SDK (5 functions, npm package)
- REST API
- Web playground (no login)
- Daily health monitoring
- Source attribution on every record

EXCLUDED from v1 — do NOT build or suggest:
- AI natural language queries
- AI data narration or anomaly detection
- Multiple datasets (only NCRB in v1)
- User accounts or authentication
- Paid tiers or subscriptions
- Mobile app
- Data visualization/charting
- Webhooks, email alerts, dashboards
- GraphQL, WebSockets
- Admin panel, multi-tenancy

Primary Audiences
1. Developers building applications with Indian data
2. Data journalists needing verified government statistics
3. Academic researchers studying Indian public policy
4. Students needing real data for projects
5. Government officials needing cross-department data views

Quality Philosophy
- Data quality matters more than feature count
- One perfectly reliable dataset is better than five unreliable ones
- Wrong data served confidently is the worst possible failure
- Transparency about limitations builds trust

Development Rules
- Always use Planning mode in Antigravity — never Fast mode
- Primary model: Claude Opus 4.6 (complex tasks)
- Secondary model: Gemini 3 Pro (boilerplate, docs)
- Quick edits: Gemini 3 Flash
- Commit to GitHub after every completed task
- Write Project Journal entry after every build session

Reference Files
- `docs/internal/PRD.md` — complete v1 specification
- `docs/internal/QUALITY_STANDARDS.md` — non-negotiable quality rules
- `docs/internal/EXECUTION_STRATEGY.md` — phased roadmap
- `docs/internal/LONG_TERM_VISION.md` — year 1-3 direction

```
## When to Apply
ALWAYS. This skill applies to every task in the BharatData project
        regardless of which agent or workflow is active. These are
        global constraints that override any other consideration.
```

File 11: .agent/workflows/bharatdata-validate.md

```
# Workflow: /bharatdata-validate

description: Runs data quality validation checks across the
        BharatData pipeline. Use after any pipeline change, before
        any release, or as a periodic health check.

## Steps

### Step 1: Parser Accuracy Check
- Select 10 random records from the most recent ingestion run
- For each record, display: state, year, category, totalCases,
        confidence, sourceTable, sourcePage
- Prompt user: "Please verify these 10 records against the
        original PDF. Mark any incorrect values."
-
        If any incorrect values found: flag as parser regression,
        do not proceed to release

### Step 2: Confidence Distribution Check
- Query database for confidence score distribution of most recent
        ingestion
- Display: count of records at each confidence level (1.0, 0.8,
        0.6, 0.4, 0.2, 0.0)
- Alert if average confidence < 0.7
- Alert if more than 10% of records have confidence < 0.5

### Step 3: Completeness Check
- Query record counts by state for the most recent year
- Compare against expected state count (36 states + UTs)
- Flag any missing states
- Flag any states with suspiciously low record counts

### Step 4: Consistency Check
- Compare record counts by state for the two most recent years
- Flag any state where count changed by more than 50%
- Flag any state where a category dropped to zero from a non-zero
        value

### Step 5: Schema Validation
```

```
-
        Verify all required fields are non-null: state, stateCode,
        year, category, totalCases, confidence, source.url,
        collectionDate
- Verify all numeric fields contain numbers not strings
- Verify all dates are valid ISO 8601
- Verify all confidence scores are between 0.0 and 1.0

### Step 6: API Response Check
- Call each of the 5 SDK functions with test parameters
- Verify response shape matches docs/sdk/DESIGN.md specification
- Verify meta.source fields are populated
- Verify query echo is correct
- Verify response time < 3 seconds for each

### Step 7: Report
- Generate summary report with pass/fail for each step
- If all pass: "Validation complete. Safe to release."
- If any fail: "Validation FAILED. Do not release. Issues listed
        above."
```

File 12: .agent/workflows/bharatdata-ingest.md

```
# Workflow: /bharatdata-ingest

description: Runs the full data ingestion pipeline for a specified
        data source. Downloads raw data, parses it, normalizes it,
        and stores it in the database.

## Prerequisites
- Database connection configured (Neon PostgreSQL)
- Data source document exists in docs/data-sources/
- canonical-mappings.json is up to date

## Steps

### Step 1: Pre-flight Check
- Verify database connection is active
- Verify target data source URL is accessible (HTTP 200)
- Check robots.txt compliance for the URL
- Verify rate limiter is configured (max 1 req/sec)
- Load relevant data source document from docs/data-sources/

### Step 2: Download
- Fetch raw file from source URL
- Calculate file hash (SHA-256)
- Compare against last known hash in data_sources table
- If unchanged: log "No new data" and exit
```

```
- If changed: store raw file with metadata (URL, timestamp, hash,
        size)

### Step 3: Parse
- Load appropriate parser for the source format (PDF/CSV/HTML)
- Load bharatdata-pdf-parser skill if PDF
- Extract structured records with confidence scores
- Log all parsing failures to structured log
- Display parsing summary: total tables found, rows extracted,
        average confidence, failures

### Step 4: Normalize
- Load bharatdata-normalizer skill
- Read canonical-mappings.json
- Run normalization pipeline (13 steps as defined in
        NORMALIZATION.md)
- Log all unmapped entities to unmapped-entities.json
- Display normalization summary: records normalized, unmapped
        entities found, confidence adjustments

### Step 5: Store
- Insert normalized records into database with full lineage
        metadata
- Create ingestion_run record with statistics
- Log normalization decisions to normalization_log table
- Display storage summary: records inserted, duplicates skipped

### Step 6: Post-ingestion Validation
- Automatically trigger /bharatdata-validate workflow (Steps 2-5
        only)
- Display validation results
- If validation fails: warn user but do not roll back (data is
        stored with confidence flags)

### Step 7: Update Registry
- Update last_ingested_at in data_sources table
- Update docs/data-sources/ registry page with new ingestion date
- Prompt user to commit changes to GitHub
```

File 13: `.agent/workflows/bharatdata-add-source.md`

```
# Workflow: /bharatdata-add-source

description: Guided process for adding a new government data
        source to BharatData. Creates the data source document,
        designs the parser, builds the normalizer mapping, and
        integrates into the existing pipeline.
```

Prerequisites
- v1 pipeline is working with at least one source
- User has identified the new data source URL and format
- Addition is driven by user demand (check GitHub issues)

Steps

Step 1: Source Assessment
Ask the user:
1. What is the data source name and URL?
2. What format is the data in? (PDF / CSV / HTML / API)
3. How often is it updated? (Annual / Monthly / Daily / Real-time)
4. What government body publishes it?
5. What license or terms apply?
6. Why is this source being added? (Link to GitHub issue or user request)

Step 2: Feasibility Analysis
- Visit the source URL and assess:
  - Is the data actually accessible?
  - What is the format complexity? (structured CSV vs messy PDF)
  - Does robots.txt allow automated access?
  - Is the data behind a login or CAPTCHA?
- Rate feasibility: Easy / Medium / Hard / Not Feasible
- If Not Feasible: document why and suggest alternatives

Step 3: Documentation
- Create a new file at docs/data-sources/[SOURCE_NAME].md following the template in docs/data-sources/NCRB_CRIME.md
- Document: full name, URL, publishing body, format, update frequency, coverage period, legal status, known challenges
- List the key tables/fields to extract
- Document known data quality issues

Step 4: Schema Extension
- Design the database table for the new dataset following existing patterns
- Ensure the schema is consistent with existing tables (same metadata fields)
- Verify the addition fits within Neon 500MB storage limit
- Generate migration script

Step 5: Parser Development
- Build parser specific to this source's format
- Apply all rules from bharatdata-pdf-parser skill (if PDF) or equivalent
- Include confidence scoring
- Test against real source data
- Document parser-specific challenges

Step 6: Normalizer Update

```
- Add any new state/district name variations to canonical-
        mappings.json
- Update unmapped-entities.json with any new unmapped names
- Verify existing normalization rules apply to new source

### Step 7: SDK Extension
- Add new dataset module (e.g., bd.census, bd.agriculture)
        following existing patterns
- Same response shape, same error handling, same naming
        conventions
- Add TypeScript types for new record shape
- Update README with new dataset

### Step 8: Integration Test
- Run /bharatdata-ingest for the new source
- Run /bharatdata-validate for the new data
- Verify SDK functions return correct data for new source
- Verify playground can query new source
```

File 14: `.agent/workflows/bharatdata-publish.md`

```
# Workflow: /bharatdata-publish

description: Pre-publish checklist and procedure for releasing a
        new version of the BharatData npm package.

## Steps

### Step 1: Pre-publish Validation
- Run /bharatdata-validate — must pass completely
- Run TypeScript compiler in strict mode — zero errors required
- Run ESLint — zero errors required (warnings acceptable)
- Run all unit tests — 100% pass required
- Verify zero any types in public API surface using grep

### Step 2: Version Decision
Ask the user:
- What type of change is this? (Breaking / Feature / Fix)
- Breaking → major version bump
- Feature → minor version bump
- Fix → patch version bump
- Confirm new version number with user

### Step 3: Documentation Update
- Update CHANGELOG.md with new version entry (Added, Changed,
        Fixed sections)
- Update version in package.json
- Update README if any API changes
```

```
- Update docs/sdk/DESIGN.md if any signature changes
- Verify all code examples in README still work

### Step 4: Final Check
- Run npm pack locally — inspect package contents
- Verify package size is reasonable
- Verify no unnecessary files included
- Test installation from packed file in a clean directory

### Step 5: Publish
- npm publish
- Create git tag for the version
- Push tag to GitHub
- Create GitHub release with changelog entry

### Step 6: Post-publish
- Verify package is live on npmjs.com
- Test npm install bharatdata@[new version] in a clean directory
- Run a basic query to verify the published package works
- Announce on relevant channels if significant update
```

File 15: .agent/scripts/bharatdata_quality_check.py

```python
"""
BharatData Quality Check Script

Runs automated data quality checks against the live database.
Designed to be executed by GitHub Actions on a weekly schedule
or manually via the /bharatdata-validate workflow.

Usage:
    python .agent/scripts/bharatdata_quality_check.py --db-url
        $DATABASE_URL

Output:
    JSON report with pass/fail status for each check.
    Exit code 0 if all checks pass, 1 if any check fails.
"""

import argparse
import json
import sys
from datetime import datetime

def check_required_fields(cursor, table_name="crime_records"):
    """Verify no null values in required fields."""
    required = ["state", "state_code", "year", "category",
        "total_cases",
```

```
                "confidence", "source_url", "collection_date"]
    issues = []
    for field in required:
        cursor.execute(
            f"SELECT COUNT(*) FROM {table_name} WHERE {field} IS
        NULL"
        )
        null_count = cursor.fetchone()[0]
        if null_count > 0:
            issues.append({
                "field": field,
                "null_count": null_count,
                "severity": "critical"
            })
    return {
        "check": "required_fields",
        "passed": len(issues) == 0,
        "issues": issues
    }

def check_confidence_distribution(cursor,
        table_name="crime_records"):
    """Check confidence score distribution and flag if average is
        too low."""
    cursor.execute(f"""
        SELECT
            AVG(confidence) as avg_conf,
            MIN(confidence) as min_conf,
            COUNT(*) FILTER (WHERE confidence < 0.5) as
        low_conf_count,
            COUNT(*) as total_count
        FROM {table_name}
    """)
    row = cursor.fetchone()
    avg_conf, min_conf, low_conf_count, total_count = row

    issues = []
    if avg_conf and avg_conf < 0.7:
        issues.append({
            "message": f"Average confidence {avg_conf:.2f} is
        below 0.7 threshold",
            "severity": "warning"
        })
    if total_count and low_conf_count / total_count > 0.1:
        issues.append({
            "message": f"{low_conf_count}/{total_count} records
        ({low_conf_count/total_count*100:.1f}%) have confidence <
        0.5",
            "severity": "warning"
        })

    return {
        "check": "confidence_distribution",
```

```
        "passed": len(issues) == 0,
        "average_confidence": float(avg_conf) if avg_conf else
        None,
        "low_confidence_percentage": float(low_conf_count /
        total_count * 100) if total_count else 0,
        "issues": issues
    }

def check_completeness(cursor, table_name="crime_records",
        expected_states=36):
    """Check if all expected states are present for the most
        recent year."""
    cursor.execute(f"""
        SELECT year, COUNT(DISTINCT state) as state_count
        FROM {table_name}
        WHERE year = (SELECT MAX(year) FROM {table_name})
        GROUP BY year
    """)
    row = cursor.fetchone()
    if not row:
        return {
            "check": "completeness",
            "passed": False,
            "issues": [{"message": "No data found in database",
        "severity": "critical"}]
        }

    year, state_count = row
    issues = []
    if state_count < expected_states:
        cursor.execute(f"""
            SELECT DISTINCT state FROM {table_name} WHERE year =
        {year}
        """)
        present_states = {r[0] for r in cursor.fetchall()}
        issues.append({
            "message": f"Only {state_count}/{expected_states}
        states found for year {year}",
            "severity": "warning",
            "year": year
        })

    return {
        "check": "completeness",
        "passed": len(issues) == 0,
        "year": year,
        "states_found": state_count,
        "states_expected": expected_states,
        "issues": issues
    }

def check_statistical_outliers(cursor,
        table_name="crime_records"):
```

```
    """Flag values that are statistical outliers (>3 std dev from
        mean)."""
    cursor.execute(f"""
        SELECT state, year, category, total_cases,
               AVG(total_cases) OVER (PARTITION BY state,
        category) as avg_cases,
               STDDEV(total_cases) OVER (PARTITION BY state,
        category) as std_cases
        FROM {table_name}
        WHERE confidence >= 0.5
    """)
    outliers = []
    for row in cursor.fetchall():
        state, year, category, total, avg, std = row
        if std and std > 0 and abs(total - avg) > 3 * std:
            outliers.append({
                "state": state,
                "year": year,
                "category": category,
                "value": total,
                "mean": round(float(avg), 1),
                "std_dev": round(float(std), 1),
                "deviations": round(abs(total - avg) / std, 1),
                "severity": "warning"
            })

    return {
        "check": "statistical_outliers",
        "passed": len(outliers) == 0,
        "outlier_count": len(outliers),
        "outliers": outliers[:10],
        "issues": outliers[:10]
    }

def check_data_types(cursor, table_name="crime_records"):
    """Verify data type integrity."""
    issues = []

    cursor.execute(f"""
        SELECT COUNT(*) FROM {table_name}
        WHERE confidence < 0 OR confidence > 1
    """)
    bad_confidence = cursor.fetchone()[0]
    if bad_confidence > 0:
        issues.append({
            "message": f"{bad_confidence} records have confidence
        outside 0.0-1.0 range",
            "severity": "critical"
        })

    cursor.execute(f"""
        SELECT COUNT(*) FROM {table_name}
```

```
        WHERE year < 1950 OR year > 2030
    """)
    bad_years = cursor.fetchone()[0]
    if bad_years > 0:
        issues.append({
            "message":
        f"{bad_years} records have implausible year values",
            "severity": "critical"
        })

    cursor.execute(f"""
        SELECT COUNT(*) FROM {table_name}
        WHERE total_cases < 0
    """)
    negative_cases = cursor.fetchone()[0]
    if negative_cases > 0:
        issues.append({
            "message": f"{negative_cases} records have negative
        total_cases",
            "severity": "critical"
        })

    return {
        "check": "data_types",
        "passed": len(issues) == 0,
        "issues": issues
    }

def run_all_checks(db_url):
    """Run all quality checks and return combined report."""
    try:
        import psycopg2
        conn = psycopg2.connect(db_url)
        cursor = conn.cursor()
    except Exception as e:
        return {
            "timestamp": datetime.utcnow().isoformat(),
            "status": "FAILED",
            "error": f"Database connection failed: {str(e)}",
            "checks": []
        }

    checks = [
        check_required_fields(cursor),
        check_confidence_distribution(cursor),
        check_completeness(cursor),
        check_statistical_outliers(cursor),
        check_data_types(cursor),
    ]

    cursor.close()
    conn.close()
```

```
    all_passed = all(c["passed"] for c in checks)

    return {
        "timestamp": datetime.utcnow().isoformat(),
        "status": "PASSED" if all_passed else "FAILED",
        "checks_run": len(checks),
        "checks_passed": sum(1 for c in checks if c["passed"]),
        "checks_failed": sum(1 for c in checks if not
        c["passed"]),
        "checks": checks
    }

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="BharatData
        Quality Check")
    parser.add_argument("--db-url", required=True,
        help="PostgreSQL connection URL")
    parser.add_argument("--output", default=None, help="Output
        file path (default: stdout)")
    args = parser.parse_args()

    report = run_all_checks(args.db_url)

    output = json.dumps(report, indent=2)

    if args.output:
        with open(args.output, "w") as f:
            f.write(output)
    else:
        print(output)

    sys.exit(0 if report["status"] == "PASSED" else 1)
```

Updated File Summary

#	File Path	Type	Purpose
1	`.agent/FEEDING_PROMPT.md`	Prompt	Master context for every IDE session
2	`.agent/agents/data-pipeline-specialist.md`	Agent	Pipeline ingestion, parsing, normalization
3	`.agent/agents/sdk-architect.md`	Agent	TypeScript SDK design and implementation

#	File Path	Type	Purpose
4	`.agent/agents/data-quality-auditor.md`	Agent	Data verification and incident response
5	`.agent/agents/bharatdata-docs-specialist.md`	Agent	Documentation across 24 documents
6	`.agent/skills/bharatdata-pdf-parser/SKILL.md`	Skill	PDF parsing rules for government reports
7	`.agent/skills/bharatdata-normalizer/SKILL.md`	Skill	Indian name normalization and data conversion
8	`.agent/skills/bharatdata-sdk/SKILL.md`	Skill	SDK design rules and TypeScript standards
9	`.agent/skills/bharatdata-data-quality/SKILL.md`	Skill	Data quality enforcement and incident response
10	`.agent/skills/bharatdata-project-context/SKILL.md`	Skill	Global project context and constraints
11	`.agent/workflows/bharatdata-validate.md`	Workflow	7-step data quality validation procedure
12	`.agent/workflows/bharatdata-ingest.md`	Workflow	Full ingestion pipeline execution
13	`.agent/workflows/bharatdata-add-source.md`	Workflow	Guided process for adding new data sources
14	`.agent/workflows/bharatdata-publish.md`	Workflow	npm publish checklist and procedure
15	`.agent/scripts/bharatdata_quality_check.py`	Script	Automated database quality checks

Integration with existing Antigravity `.agent` folder:

Component	Existing (Antigravity)	Added (BharatData)	Total
Agents	20	4	24
Skills	36	5	41
Workflows	11	4	15
Scripts	2 master + 18 skill-level	1	3 master + 18 skill-level

These 15 BharatData-specific files sit alongside the existing 69 Antigravity components. The BharatData agents reference both BharatData-specific skills and existing Antigravity skills like nodejs-best-practices, database-design, api-patterns, testing-patterns, clean-code, and systematic-debugging. This means you get the full power of Antigravity's built-in engineering standards plus BharatData's domain-specific rules.

The existing `docs/` folder files from the previous response remain unchanged. Together with these `.agent` folder additions, the IDE now has complete knowledge of the BharatData project across every dimension: architecture, data pipeline, SDK design, quality standards, normalization rules, legal framework, audience, content strategy, and long-term vision.

BharatData — Document Creation Sequence

Phase 0 — Clarity Before Code (Day 1-3)

Priority	Doc	Document Name	Day	Time
1	00	Execution Strategy	Day 1	3 hours
2	01	Project Overview and Vision	Day 1	2 hours
3	02	Product Requirements Document	Day 2	3 hours
4	00c	Audience Map	Day 2	1 hour
5	00d	Long-Term Vision	Day 3	1 hour

Phase 1 — Passive Research (Day 4-5)

Priority	Doc	Document Name	Day	Time
6	00c	Audience Map (update with research findings)	Day 5	30 minutes

Phase 2 — Build the Hardest Thing (Week 1-4)

Week 1

Priority	Doc	Document Name	Day	Time
7	27	Feeding Prompt	Week 1 Day 1	1 hour
8	28	Agent Definitions	Week 1 Day 1	2 hours
9	09a	Brand Minimums Checklist	Week 1 Day 1	20 minutes
10	20	License (MIT)	Week 1 Day 1	10 minutes
11	00a	Project Journal (start)	Week 1 Day 1	5 minutes per session ongoing
12	05a	TypeScript SDK Design Specification	Week 1 Day 2	3 hours
13	05b	Python SDK Design Specification	Week 1 Day 2	3 hours

Week 2

Priority	Doc	Document Name	Day	Time
14	06			2 hours

Priority	Doc	Document Name	Day	Time
		Data Schema and Normalization Guide	Week 2 Day 1	
15	00b	Content and Distribution Strategy	Week 2 Day 2	2 hours
16	08	Data Pipeline Specification	Week 2 Day 3	3 hours
17	25	Quality Standards	Week 2 Day 4	2 hours

Week 3

Priority	Doc	Document Name	Day	Time
18	29	Workflow Definitions	Week 3 Day 1	2 hours
19	14	Data Dictionary	Week 3 Day 3	2 hours

Week 4

Priority	Doc	Document Name	Day	Time
20	10	README	Week 4 Day 1	3 hours
21	04	Data Source Registry	Week 4 Day 2	2 hours
22	03	Technical Architecture Document	Week 4 Day 3	4 hours
23	03a	Testing Strategy	Week 4 Day 3	2 hours
24	11	TypeScript SDK Reference Documentation	Week 4 Day 4	3 hours
25	12	Python SDK Reference Documentation	Week 4 Day 4	3 hours
26	13	API Reference	Week 4 Day 5	2 hours

Phase 3 — Real Users and Distribution (Week 5-8)

Week 5 Day 1 — Documentation Blitz

Priority	Doc	Document Name	Day	Time
27	16	Contributing Guide	Week 5 Day 1	2 hours
28	17	Code of Conduct	Week 5 Day 1	15 minutes
29	24	Security Policy and Incident Response	Week 5 Day 1	45 minutes
30	18	Changelog	Week 5 Day 1	30 minutes
31	21	Terms of Use	Week 5 Day 1	1 hour
32	22	Privacy Policy	Week 5 Day 1	1 hour
33	23	Data Attribution and Legal Compliance	Week 5 Day 1	2 hours
34	09b	Brand and Identity Guidelines	Week 5 Day 2	1 hour
35	15	Quick Start Guides	Week 5 Day 2	3 hours

Week 6-7

Priority	Doc	Document Name	Day	Time
36	19	Public Roadmap	Week 6	1 hour

Week 8

Priority	Doc	Document Name	Day	Time
37	26	Portfolio Case Study	Week 8	3 hours

Phase 4 — AI Layer (Week 9-12)

Priority	Doc	Document Name	Day	Time
38	07	AI Layer Specification	Week 10	3 hours
39	11	TypeScript SDK Reference (update for AI features)	Week 11	1 hour
40	12	Python SDK Reference (update for AI features)	Week 11	1 hour
41	13	API Reference (update for AI endpoints)	Week 11	1 hour
42	15	Quick Start Guides (update for AI queries)	Week 11	1 hour

---

Ongoing — Throughout All Phases

Doc	Document Name	Frequency
00a	Project Journal	Every build session (5 minutes)
18	Changelog	Every release (15 minutes)
19	Public Roadmap	Monthly update (30 minutes)
04	Data Source Registry	Every new source added (1 hour)
10	README	Every major feature change (30 minutes)

Summary

Phase	Documents Created	Total Time
	5	10 hours

Phase	Documents Created	Total Time
Phase 0 (Day 1-3)		
Phase 1 (Day 4-5)	1 update	30 minutes
Phase 2 Week 1	7	9.5 hours
Phase 2 Week 2	4	9 hours
Phase 2 Week 3	2	4 hours
Phase 2 Week 4	7	19 hours
Phase 3 Week 5	9	11.5 hours
Phase 3 Week 6-7	1	1 hour
Phase 3 Week 8	1	3 hours
Phase 4 Week 10-11	5	7 hours
Total	**42 document actions**	**~75 hours across 12 weeks**

Three Rules for Document Creation

Rule 1: Never write a document before its scheduled phase. A document written before you have built anything is fiction. A document written after you have built something is knowledge. The sequence above ensures every document is written when you have the real experience to make it accurate.

Rule 2: Never let documentation block building. If a document is taking longer than its estimated time, write what you know now and mark the rest as TODO. Ship the incomplete document and update it later. A partial document that exists is better than a perfect document that blocks your next milestone.

Rule 3: The Project Journal is the most important document. Five minutes after every build session. No formatting required. Just raw notes about what happened. Every other document you write will be better because the Journal captured the details while they were fresh.

BharatData — The Complete Project Document

The Country That Drowns in Its Own Data

India is not a country that lacks data. India is a country that is drowning in data it cannot use.

Every single day across the length and breadth of the Indian subcontinent, an enormous machinery of data collection grinds forward. Police stations in 780 districts file First Information Reports that eventually flow upward through state police headquarters to the National Crime Records Bureau in New Delhi. Weather stations operated by the India Meteorological Department record temperature, humidity, rainfall, and wind speed readings every few hours from thousands of observation points. Air quality monitoring stations run by the Central Pollution Control Board and State Pollution Control Boards measure particulate matter, nitrogen dioxide, sulfur dioxide, ozone, carbon monoxide, and lead levels in the air of over 300 cities. Thousands of agricultural mandis across the country record wholesale prices for hundreds of crops every morning as farmers and traders negotiate in the pre-dawn hours. Hospitals report disease incidence, bed occupancy, and mortality figures to the Integrated Disease Surveillance Programme. Schools report enrollment, dropout rates, teacher counts, and infrastructure status through the Unified District Information System for Education. Banks report lending figures, deposit rates, and non-performing asset ratios to the Reserve Bank of India. Courts report case filings, pendency, and disposal rates to the National Judicial Data Grid. Railways report passenger counts, freight tonnage, and accident data. The Registrar General records births, deaths, and causes of death across the country.

This is arguably the most comprehensive national data collection effort anywhere in the democratic world. The Indian government, through its central ministries, state departments, autonomous bodies, regulatory authorities, and constitutional commissions, collects data on virtually every aspect of Indian life. This data covers 1.4 billion people across 28 states, 8 union territories, approximately 780 districts, over 6,000 blocks, and more than 600,000 villages. It spans decades of collection. Some datasets go back to independence in 1947. The Census has been conducted every decade since 1881.

If this data were properly accessible, it could power a revolution in how India is governed, how businesses operate, how research is conducted, how journalism is practiced, how citizens engage with their democracy, and how the world understands the most populous nation on earth.

But it is not accessible. It is locked away behind a wall of hostile formats, incompatible systems, broken websites, inconsistent standards, and institutional indifference to the needs of anyone who might actually want to use it.

This is the problem BharatData exists to solve.

The Scale of Indian Government Data

To understand why this problem matters, you must first understand the sheer scale and diversity of Indian government data and the number of institutions that produce it.

At the central government level, there are over 50 ministries and departments, each producing their own data. The Ministry of Home Affairs oversees the National Crime Records Bureau which publishes annual crime statistics, prison statistics, and accidental death and suicide statistics. The Ministry of Health and Family Welfare publishes data through the National Health Mission, the Central Bureau of Health Intelligence, and the National Family Health Survey. The Ministry of Agriculture publishes data through the Directorate of Economics and Statistics, the Agricultural Census, and the Agmarknet portal for daily mandi prices. The Ministry of Education publishes through the All India Survey on Higher Education and the Unified District Information System for Education. The Ministry of Statistics and Programme Implementation publishes through the National Statistical Office which conducts the Periodic Labour Force Survey, the Consumer Expenditure Survey, and the Annual Survey of Industries. The Ministry of Environment publishes through the Central Pollution Control Board, the Forest Survey of India, and the Wildlife Institute of India. The Ministry of Finance publishes through the Reserve Bank of India, the Securities and Exchange Board of India, and the Controller General of Accounts. The Ministry of Commerce publishes trade data through the Directorate General of Commercial Intelligence and Statistics. The Ministry of Labour publishes employment data through the Labour Bureau.

And these are just the major ones. There are dozens more. The Election Commission of India publishes detailed electoral data for every constituency in the country. The Comptroller and Auditor General publishes audit reports for every government department. The National Commission for Scheduled Castes and Scheduled Tribes publishes data on caste-based discrimination and violence. The National Commission for Women publishes data on crimes against women and complaints received. The Law Commission publishes judicial statistics. The National Human Rights Commission publishes data on human rights violations. The

Central Water Commission publishes river flow and flood data. The Geological Survey of India publishes mineral resource data. The Survey of India publishes topographic and geographic data.

At the state level, multiply this complexity by 28 states and 8 union territories, each with their own statistical departments, planning boards, and domain-specific agencies. State crime records bureaus. State pollution control boards. State agricultural departments. State health departments. State education departments. Each producing data in their own formats, on their own schedules, with their own naming conventions and geographic coding systems.

The Open Government Data Platform at data.gov.in alone lists over 600,000 datasets from more than 150 government departments. But listing is not the same as accessibility. The vast majority of these datasets are in formats that are practically unusable for any modern application.

This is not a problem of data scarcity. India may have more government data per capita than any other developing nation. This is a problem of data accessibility. The data exists. The systems to use it do not.

How People Actually Suffer From This Problem

The consequences of inaccessible government data are not abstract. They are concrete, daily, and they affect real people doing real work that matters.

Developers Suffer

A developer at an agricultural technology startup in Bangalore wants to build an app that shows farmers the current wholesale price of their crops across nearby mandis so they can decide where to sell for the best price. This is a straightforward application that could genuinely improve farmer incomes. The data exists. Agmarknet publishes daily prices from thousands of mandis.

But the Agmarknet website uses session-based navigation that breaks automated access. The table structure changes every few months when the website is updated. Crop names are sometimes in Hindi, sometimes in English, sometimes in regional languages, and sometimes in transliterated forms that do not match any standard dictionary. The same crop might be called “Tomato” in one mandi’s listing and “Tamatar” in another. The website goes

down frequently during peak hours. There is no API. There is no bulk download option. There is no documentation about the data format.

The developer spends six weeks building a custom scraper. The scraper breaks two weeks later when the website is redesigned. They rebuild it. It breaks again when the session management changes. They rebuild it again. By the time they have a reliable data pipeline, their startup has burned through three months of runway on data plumbing instead of product development. Their competitor who launched without real data, using static price estimates, has already acquired users.

A fintech developer in Mumbai needs district-level demographic and economic data to build a credit scoring model for rural lending. The data exists across the Census, the Socio-Economic and Caste Census, the Reserve Bank of India’s district-level banking statistics, and the Ministry of Rural Development’s data on poverty indicators. But each source uses different geographic identifiers. The Census uses a census code. The RBI uses a district name that may not match the Census spelling. The SECC uses yet another coding system. Some districts have been bifurcated between the Census year and the RBI data year, so a district that was one entity in 2011 is now two entities and the data cannot be directly compared.

The developer hires a data engineer specifically to build mapping tables between these different geographic systems. This takes two months. Then a new round of district bifurcations is announced and all the mapping tables need to be updated. The developer eventually abandons the idea of using government data and buys expensive proprietary data from a commercial provider instead. The very data that the government collected from these districts at taxpayer expense is unusable by the companies trying to serve those districts.

A health technology developer wants to build a disease surveillance dashboard that combines data from the Integrated Disease Surveillance Programme, the National Vector Borne Disease Control Programme, and the Central Bureau of Health Intelligence. Each publishes data in different formats. IDSP uses weekly reports in PDF format. NVBDCP publishes monthly data on their website in HTML tables. CBHI publishes an annual National Health Profile as a massive PDF with hundreds of tables. The disease categorizations do not match across sources. The geographic levels do not match. The time periods do not align. Building a unified view requires months of manual data harmonization that must be redone every year when new data is published.

These are not edge cases. This is the daily reality for thousands of developers across India who are trying to build applications that serve the Indian market using Indian data. The irony is acute: the government spends enormous resources collecting this data, but the format in which it is published makes it practically useless for the technology sector that could turn it into services for citizens.

Journalists Suffer

A reporter at a major Indian news outlet is investigating the claim made by a state government that crime has decreased under their administration. The reporter needs state-level crime data broken down by category for the last ten years to verify this claim. The data exists. The NCRB publishes it annually in the Crime in India report.

But the Crime in India report is a PDF document of over 400 pages containing roughly 500 tables. The reporter needs data from ten annual reports spanning 2014 to 2023. That is over 4,000 pages and approximately 5,000 tables across ten separate PDF files. Each year’s report has a slightly different structure. The table numbering changes between years. The crime categories are redefined periodically. In 2014, the data for Andhra Pradesh includes Telangana. From 2014 onward, they are separate. The reporter must manually account for this in any trend analysis.

The reporter spends two weeks downloading reports, finding the relevant tables, manually transcribing numbers into a spreadsheet, checking the numbers for obvious errors, normalizing state names across years, adjusting for the Telangana bifurcation, and calculating year-over-year changes. By the time the analysis is complete, the news cycle has moved on. The story runs three weeks late and gets less attention than it deserved. The public is less informed because the data was too hard to access in time.

A data journalist at IndiaSpend wants to create an interactive visualization showing how air quality varies across Indian cities over the course of a year. The data exists. The Central Pollution Control Board maintains a network of continuous air quality monitoring stations. But the CPCB website renders data through JavaScript which makes automated access difficult. Historical data is available only through a clunky web interface that allows downloading one station at a time, one parameter at a time, for one date range at a time. There are over 300 stations. There are six key pollutants. The journalist needs a year of hourly data. Downloading this manually through the web interface would take weeks of clicking through forms. The journalist gives up and writes the story using only the few data points they could gather manually, producing a less comprehensive and less impactful piece than they had envisioned.

A foreign correspondent based in Delhi is writing a story about rural healthcare access in India for an international publication. They need data on the number of primary health centres, community health centres, district hospitals, doctors, nurses, and hospital beds per capita across Indian states. The data exists across publications from the Ministry of Health, the Rural Health Statistics report, the National Health Profile, and state-specific health department reports. But these sources use different definitions of the same metrics, cover different time periods, and are published in different formats. The correspondent has no idea which source to trust, how to reconcile conflicting numbers, or how to adjust for the fact that some states report more completely than others.

They hire a local research assistant to compile the data. The assistant spends a month. The numbers still do not add up because the assistant did not know about the definitional changes that the Ministry of Health made between 2017 and 2019. The story runs with a caveat about data reliability, which undermines its impact.

Researchers Suffer

A PhD candidate at a premier Indian university is studying the relationship between government spending on education and learning outcomes across Indian states over the last two decades. This is important policy research that could influence how hundreds of billions of rupees in education funding are allocated.

The candidate needs education spending data from state budget documents, school enrollment and dropout data from UDISE, learning assessment data from the National Achievement Survey, teacher training data from the Ministry of Education, and demographic data from the Census for population normalization. Each of these sources is published by a different government body, in a different format, using different geographic coding, with different update frequencies.

The first eight months of the three-year PhD programme are spent not on research but on data collection and cleaning. The candidate builds custom scrapers for four different government websites. Three of them break within six months when the websites are redesigned. The state budget data is in PDF format and must be manually transcribed because the tables are scanned images, not searchable text. The UDISE data is in Excel format but the column headers change every year and the school identification codes are not consistent across years. The NAS data uses a sampling methodology that changed in 2017, making pre-2017 and post-2017 data not directly comparable without adjustment.

The candidate’s thesis committee grows frustrated with the slow progress. The candidate’s actual analytical contribution does not begin until month nine. By the time the thesis is complete three years later, a significant portion of the effort was spent on data infrastructure that had to be rebuilt for this single study and will be abandoned afterward. The next PhD student studying a related topic will build the same infrastructure from scratch.

This pattern repeats across thousands of research projects at hundreds of Indian universities every year. Collectively, Indian academia spends millions of person-hours on data collection and cleaning that adds no intellectual value to the research itself.

International researchers face even greater barriers. A professor at a European university studying comparative urbanization patterns between India and Southeast Asia needs Indian district-level urbanization data. They have no familiarity with Indian government websites, no understanding of the administrative geography, and no ability to read Hindi which some data sources require. They settle for using outdated Census 2011 data from a secondary source that may or may not have transcribed it correctly, because accessing the original data is simply too difficult from outside India.

Students Suffer

At any given time, millions of computer science students across India are working on projects, assignments, and competitions. A significant number of them want to use real Indian data. They want to build projects that are relevant to their country, that address real problems, that demonstrate genuine analytical capability.

But real Indian data is inaccessible to them. A student who wants to build a machine learning model for crop yield prediction using real agricultural data would need to navigate Agmarknet’s broken interface, the India Meteorological Department’s outdated data portal, and the Ministry of Agriculture’s scattered publications. This would take weeks, and the student has a submission deadline in ten days.

So the student uses a Kaggle dataset about California housing prices or Boston crime instead. Their project works technically but has no connection to Indian reality. The student learns nothing about Indian data challenges. Their project impresses no one because thousands of other students have used the same generic dataset. The opportunity to build something genuinely relevant and impactful is lost because the data barrier was too high.

During hackathons, this problem is even more acute. Teams have 24 to 48 hours to build a working prototype. Any team that tries to use real Indian government data will spend most of their

hackathon time on data access and cleaning rather than on building their solution. Smart teams have learned to avoid government data entirely. The most innovative solutions to India's real problems never get built because the data needed to prototype them is locked away in unusable formats.

Citizens Suffer

An ordinary citizen who hears a politician claim that their state has the lowest unemployment rate in India has no practical way to verify this claim. The data exists. The Periodic Labour Force Survey publishes state-level unemployment estimates. But accessing and interpreting this data requires navigating the Ministry of Statistics website, downloading a PDF report, finding the right table, understanding the methodology, and knowing the difference between usual principal status and current weekly status definitions of unemployment.

A farmer in Madhya Pradesh who wants to check whether the Minimum Support Price for their crop was actually raised as the government announced has to navigate the Department of Agriculture's website, find the right circular, and compare numbers across years. The information is technically public but practically inaccessible to someone farming in a village with a smartphone and limited data bandwidth.

A parent who wants to compare schools in their district using government quality metrics collected through UDISE cannot do so because the data is not published in any format that allows school-level comparison by a non-technical person. The government collected the data specifically to enable quality assessment, but the publication format defeats the purpose.

A local journalist in a tier-three city who wants to investigate whether their district receives its fair share of central government scheme funding compared to neighbouring districts needs data from the Ministry of Finance's expenditure reports, the Controller General of Accounts, and the relevant scheme's MIS portal. Each is a separate system with separate access methods and separate data formats. The journalist does not have the technical skills or the time to compile this data. The story goes unwritten. Potential misallocation of public funds goes unexamined.

Government Officials Themselves Suffer

Perhaps the most ironic consequence is that government officials themselves cannot easily access their own government's data across departments. A district collector who wants to understand their district's performance across education, health, employment, and infrastructure indicators relative to the state average must request separate reports from the District Education Office, the

Chief Medical Officer, the District Employment Officer, and the Public Works Department. Each report arrives in a different format, at a different time, measuring different things over different periods. A holistic view of the district's development status requires weeks of manual compilation.

The NITI Aayog, India's premier policy think tank, has publicly acknowledged the challenge of data accessibility across government departments. Despite being at the apex of India's planning infrastructure, NITI Aayog struggles to get timely, consistent, comparable data from state governments and central ministries.

If the most powerful planning body in the country cannot easily access cross-departmental government data, what hope does an ordinary developer, journalist, researcher, student, or citizen have?

The Specific Loopholes and Failure Points

The inaccessibility of Indian government data is not caused by a single problem. It is caused by a network of interconnected failure points that reinforce each other.

Loophole 1: No Mandatory Data Standards

There is no binding standard that requires all government bodies to publish data in a consistent, machine-readable format. Each ministry, department, and state government is free to publish data in whatever format they choose. Some publish Excel files. Some publish PDFs. Some publish HTML tables. Some publish scanned images of printed reports. Some publish through web portals with JavaScript-rendered content. Some publish through APIs. Most publish through a combination of all of these, changing formats whenever they redesign their website or when a new officer takes charge.

The Government of India's Open Data Policy of 2012 and the subsequent updates have encouraged machine-readable data publication, but compliance is voluntary and uneven. The data.gov.in portal was created to centralize open government data, but it functions primarily as a listing service. It catalogues over 600,000 datasets but many of them are links to files hosted on other government servers. When those servers go down or restructure their URLs, the data.gov.in links break. The quality of

data on the platform varies enormously. Some datasets are well-structured CSVs. Others are links to scanned PDF documents. Others are links to web pages that no longer exist.

Loophole 2: Geographic Identity Crisis

India does not have a single, consistent, universally used geographic coding system. The Census uses one set of state and district codes. The Election Commission uses another. The PIN code system uses yet another. The NCRB sometimes uses state names and sometimes uses abbreviations that do not match any official standard. The Ministry of Agriculture uses mandi codes that are specific to their system and do not map to any other geographic identifier.

District names are spelled differently across sources. “Bangalore” and “Bengaluru” refer to the same city but appear as different entries in different datasets. “Varanasi” and “Banaras” are the same place. “Pondicherry” and “Puducherry” are the same union territory but appear differently in data published before and after the name change.

More critically, the geographic entities themselves change over time. New states are created. Telangana was carved out of Andhra Pradesh in 2014. Ladakh was separated from Jammu and Kashmir in 2019. Districts are bifurcated regularly. India had 593 districts at the time of Census 2001, 640 at Census 2011, and over 780 today. When a district is split into two, the historical data for the original district cannot be directly compared with data for either of the new districts without knowing which areas went where. This information exists in government gazettes but is not available in any machine-readable format.

This geographic identity crisis means that joining data from two different government sources by location — which is the most fundamental and most common data operation — requires extensive manual mapping work. A developer who wants to combine crime data from NCRB with demographic data from the Census must first build a mapping table between NCRB’s geographic identifiers and Census geographic identifiers. This mapping must account for name variations, code differences, boundary changes, and temporal inconsistencies. Building and maintaining this mapping is a significant engineering effort in itself.

Loophole 3: Temporal Inconsistency

Different government data sources operate on different temporal cycles and reference periods. The NCRB publishes annual crime data for the calendar year (January to December). The government budget operates on the fiscal year (April to March).

The agricultural year varies by crop. The Census is decennial. The NFHS (National Family Health Survey) is conducted every few years on an irregular schedule.

When a researcher wants to correlate crime data with economic data, they face the immediate problem that the crime data is for January-December 2023 while the economic data is for April 2023-March 2024. These are not the same time period. Adjusting for this requires either accessing monthly data (which is often not published) or making assumptions about temporal distribution that may not be valid.

Publication delays compound this problem. The NCRB typically publishes crime data eight to twelve months after the reference year. The Census takes several years to fully process and publish. Some surveys take even longer. At any given point, the most recent available data from different sources may refer to different years, making cross-source comparison misleading if the time gap is not properly accounted for.

Loophole 4: Format Hostility

The most common format for Indian government statistical publications is the PDF. Not a digitally-created PDF with searchable text and accessible tables. Often a PDF that is a scanned image of a printed document, making it completely resistant to any form of automated data extraction. Even digitally-created PDFs use complex table layouts with merged cells, multi-row headers, embedded footnotes, and decorative formatting that breaks standard PDF parsing tools.

The NCRB’s Crime in India report is a case study in format hostility. The report contains approximately 500 tables across 400 pages. Table headers span multiple rows with merged cells. Tables continue across page breaks without repeating the header on the subsequent page. Footnote markers are embedded inside data cells so that a number appears as “1,234*” requiring the parser to distinguish between the data value and the footnote marker. Numbers use Indian formatting conventions. Some tables mix Hindi and English text. Summary rows (“Total”, “Grand Total”) are interspersed with data rows without any programmatic indicator distinguishing them. Every year, the table structure changes slightly.

This is not unique to NCRB. It is the norm across Indian government publications. The assumption underlying government data publication in India is that the consumer of the data is a human being who will read a printed report. The possibility that someone might want to programmatically extract and analyze the data is simply not considered in the publication process.

Loophole 5: Website Fragility

Government websites in India are notoriously fragile. They are typically hosted on National Informatics Centre (NIC) infrastructure that frequently experiences downtime. Websites are redesigned whenever a new government takes charge or whenever a new IT initiative is launched. When a website is redesigned, all existing URLs break. There is rarely any redirect from old URLs to new ones. Bookmarks, scrapers, and external links all become instantly useless.

Some government websites block automated access through CAPTCHAs, IP-based rate limiting, or session-based navigation that requires browser interaction. Some websites require JavaScript execution to render data, making them inaccessible to simple HTTP-based scrapers. Some websites go behind login walls without warning, cutting off public access to data that was previously freely available.

The fragility of government websites means that any data access solution built on scraping is inherently unstable. A scraper that works today may break tomorrow because the website changed its HTML structure, moved to a new server, added a CAPTCHA, or simply went offline. This fragility has discouraged many developers from even attempting to build government data products because the maintenance burden is too high.

Loophole 6: No Feedback Loop

When a government department publishes data with errors, inconsistencies, or in unusable formats, there is no effective mechanism for users to report these issues and have them corrected. The relationship between government data publishers and data consumers is entirely one-directional. The government publishes. The public receives or, more accurately, fails to receive. There is no channel for feedback, no process for reporting data quality issues, no system for requesting format improvements.

The RTI (Right to Information) Act provides a legal mechanism for requesting government data, but it is designed for obtaining specific documents, not for requesting format changes or reporting data quality issues in published statistics. Filing an RTI request to get data that is already publicly available but in an unusable format is technically possible but practically absurd.

This lack of feedback means that data quality issues persist for years. A misspelled district name in an NCRB table might remain uncorrected across multiple annual publications because no one in the publishing chain has any incentive or mechanism to fix it.

What BharatData Is

BharatData is an open source project that builds the missing infrastructure layer between Indian government data and the people who need it.

It is not a website. It is not a dashboard. It is not a visualization tool. It is infrastructure. It is the plumbing that makes data flow from government sources to developers, journalists, researchers, students, and citizens in a form they can actually use.

BharatData does four things.

It collects data automatically from government sources across every ministry, department, and agency that publishes public data. It monitors these sources continuously, detecting when new data is published, when formats change, when websites restructure, and when sources go offline.

It cleans and normalizes this data. It parses hostile PDF formats, broken CSV files, JavaScript-rendered web tables, and inconsistent Excel spreadsheets. It resolves the geographic identity crisis by mapping every state name variation, every district name variation, every administrative code system, and every historical boundary change into a single consistent canonical form. It converts Indian number formats, handles fiscal year versus calendar year discrepancies, standardizes null value representations, and assigns confidence scores to every extracted data point.

It serves this data through developer-friendly interfaces. A TypeScript SDK installable through npm for JavaScript and TypeScript developers. A Python SDK installable through pip for data scientists, machine learning engineers, and researchers. A REST API for developers in any language. A web playground for non-technical users. Every interface returns the same clean, normalized, attributed data.

It adds intelligence through AI. A natural language query engine that lets anyone ask questions about Indian government data in plain English or Hindi without writing code. A data narration layer that transforms raw numbers into contextual human-readable insights. An anomaly detection system that spots unusual patterns in data and generates explanations. A research agent that can answer complex questions spanning multiple datasets by autonomously planning and executing multi-step analyses.

BharatData’s ambition is not to clean up one or two government datasets. Its ambition is to become the single canonical access layer for all Indian government public data. Every ministry. Every department. Every state. Every dataset. Every year. One unified, clean, queryable, AI-powered interface.

In the long term, BharatData aims to go beyond government data. Once the infrastructure for data ingestion, normalization, and serving is established, it can accommodate any structured data relevant to understanding India. NGO data on development indicators. Academic research datasets. Crowdsourced data on public services. Corporate filings and regulatory data. Court judgments and legislative records. Any data that helps people understand India better belongs in BharatData.

BharatData is the vault. The single source of truth for anyone who needs data about India.

How BharatData Solves Each Problem

Solving Format Hostility

BharatData’s data pipeline is specifically engineered to handle the hostile formats that Indian government data comes in. The PDF parser handles multi-row merged headers by detecting header rows through pattern recognition and merging them into flat column names. It handles cross-page table continuations by detecting when a new page starts with data rows instead of header rows. It strips footnote markers from data cells while preserving the footnote reference as metadata. It handles Indian number formatting by detecting the comma placement pattern and converting accordingly. It identifies summary rows by looking for keywords like “Total” and “Grand Total” and flags them as aggregations rather than data records.

For CSV files, the pipeline handles inconsistent delimiters, mixed encodings, embedded newlines within cells, and header rows that change position. For HTML tables, it handles JavaScript-rendered content through headless browser execution when necessary. For Excel files, it handles multiple sheets, merged cells, and formatted numbers that Excel stores differently from how they appear on screen.

Every extracted value is assigned a confidence score. A value extracted cleanly from a well-structured source gets a high confidence score. A value extracted from a messy PDF where the parser had to make assumptions gets a lower confidence score. Values that the parser could not reliably extract are flagged and

excluded from API responses until manually verified. This confidence scoring system means that users always know how much to trust each data point.

Solving the Geographic Identity Crisis

BharatData maintains a canonical geographic reference system that is one of the most valuable artifacts of the entire project. This is a comprehensive mapping file that resolves every known variation of every Indian state and district name to a single canonical form.

Every state has a canonical name, a standard code, and a list of every variation that appears across government sources. “AP”, “A.P.”, “Andhra Pradesh”, “ANDHRA PRADESH”, “Andhra”, and “Andhra Pr.” all resolve to the canonical “Andhra Pradesh” with state code “AP”. This mapping covers all 28 states and 8 union territories with every abbreviation, misspelling, formatting variation, and alternative name that has appeared in any government publication.

District-level mapping is even more comprehensive and more challenging because districts change more frequently than states. The mapping tracks every district with its canonical name, its parent state, all name variations, and critically, its boundary change history. When a district is bifurcated, the mapping records the parent district, the child districts, the effective date, and the geographic areas that moved from parent to children. This allows BharatData to correctly handle historical queries. When a user asks for crime data for a district that was bifurcated in 2018, BharatData can return pre-2018 data attributed to the original district and post-2018 data showing the separate child districts, with annotations explaining the boundary change.

This canonical geographic reference system is published as an open file in the BharatData repository. Other developers can use it independently of the BharatData SDK. Researchers can reference it in their methodology sections. Contributors can improve it by adding new variations they discover. Over time, this file becomes the most comprehensive, most accurate, and most up-to-date mapping of Indian administrative geography available anywhere.

Solving Temporal Inconsistency

BharatData tags every data point with its exact temporal reference. When data comes from a calendar year source, it is tagged with the calendar year. When data comes from a fiscal year source, it is tagged with both the fiscal year designation and the corresponding calendar years. When data comes from a specific survey round, it is tagged with the survey dates.

When users query across sources that use different temporal references, BharatData's metadata makes the temporal basis of each data point explicit. A query that returns crime data for 2023 alongside economic data for FY 2023-24 will clearly indicate that these are different reference periods. The user is never misled into thinking they are comparing contemporaneous data when they are not.

For the AI layer, the research agent understands temporal alignment and automatically accounts for it when performing cross-dataset analysis. If a user asks "How does crime rate correlate with unemployment rate across states?" the agent knows that NCRB crime data is calendar-year-based while PLFS employment data is collected over a July-to-June period, and it adjusts the analysis accordingly or flags the temporal mismatch in its narrative.

Solving Website Fragility

BharatData's ingestion pipeline is designed to be resilient to the inherent fragility of government websites. It does not rely on any single access method for any data source. When a website goes down, BharatData serves the most recent cached data with a clear indication of when it was last updated. When a website changes its URL structure, the monitoring system detects the change and creates an alert so the scraper can be updated. When a website adds new access barriers, the system logs the change and explores alternative access methods.

The monitoring system runs daily health checks on every registered data source. If a source URL returns an error, the system creates an alert. If the content at a URL has changed structure, the system creates a different alert indicating that the parser may need updating. This continuous monitoring means that BharatData detects source changes within 24 hours rather than discovering them weeks later when a user reports incorrect data.

BharatData also stores raw downloaded files with complete metadata. If a government website permanently removes historical data, which happens more often than it should, BharatData has a copy. This archival function is an important secondary benefit of the system. Government data should be permanently available but in practice it is often deleted during website redesigns. BharatData's archive preserves data that might otherwise be lost.

Solving the Feedback Loop

BharatData creates a feedback loop that government data publication currently lacks. When BharatData's parser encounters an error in government data, it is logged and made visible. When

the normalizer encounters an inconsistency between two government sources, it is documented. When a user reports a data quality issue through BharatData's GitHub repository, it becomes a public record that can be tracked and resolved.

Over time, this feedback loop produces a comprehensive catalogue of data quality issues across Indian government publications. This catalogue itself is valuable. It can inform government data publishers about the errors in their publications. It can guide the National Statistical Office in prioritizing data quality improvements. It can help researchers understand the reliability of different data sources.

BharatData does not fix government data problems. But it makes them visible, documented, and quantified. Visibility is the first step toward resolution.

How Technical Users Interact With BharatData

JavaScript and TypeScript Developers

JavaScript and TypeScript developers install BharatData through npm and use it as a standard library in their Node.js applications, React frontends, Next.js backends, and any other JavaScript-based project.

They import the BharatData client, create an instance, and call query functions on dataset modules. Each dataset — crime, census, agriculture, health, education, environment, economy, and so on — is a module on the client object. Each module has query functions that accept parameters like state name, year, category, and district, and return fully typed response objects containing the data, metadata about the source, and confidence information.

The response includes the data records, each fully typed with TypeScript definitions that enable IDE autocomplete. A developer typing `record.` in VS Code or Antigravity sees every available field with its type and description. The response also includes metadata telling the developer exactly where each data point came from: the government source URL, the specific report name, the page and table in the source document, when BharatData collected it, and the confidence score.

Error handling uses custom typed error classes so developers can catch specific error conditions and handle them appropriately. If a requested state does not exist in the data, a specific error is

thrown with a suggestion to check available states. If the API rate limit is exceeded, a specific error is thrown with a retry-after interval.

The TypeScript SDK is designed to have minimal dependencies so it does not bloat the application’s bundle size. It supports tree-shaking so that only the dataset modules actually used are included in the final bundle. It works in Node.js, in the browser, and in edge runtime environments like Cloudflare Workers and Vercel Edge Functions.

Python Developers, Data Scientists, and Machine Learning Engineers

Python developers install BharatData through pip and use it in Jupyter notebooks, data analysis scripts, machine learning pipelines, and research workflows.

The Python SDK is designed specifically for the Python data science ecosystem. Its most important feature is native pandas integration. Every query function can return results as a pandas DataFrame with proper column types, meaningful indexes, and snake_case column names that follow Python conventions. A data scientist can go from import to analysis in three lines of code: import the package, create a client, query data and convert to DataFrame. From that point they use standard pandas, matplotlib, seaborn, scikit-learn, and any other Python data tool they are familiar with.

For machine learning engineers, the Python SDK supports bulk data access with automatic pagination. A researcher requesting ten years of data across all states and all crime categories does not need to write loops or manage page tokens. They call a single function with a year range and get back a complete dataset, streamed in chunks to manage memory for very large requests.

The SDK supports export to all common data science formats. CSV for simple sharing and spreadsheet import. Parquet for efficient columnar storage that preserves types and works with Apache Spark, Dask, and other big data tools. JSON for web applications and API integration. Excel for colleagues who prefer spreadsheets. Each export format includes source attribution metadata in file headers or embedded metadata fields.

For researchers specifically, the SDK includes citation generation. A researcher can call a method that generates a properly formatted academic citation for the data they queried, suitable for inclusion in a methodology section or bibliography. The citation includes the original government source, the BharatData access date, the dataset version, and the specific query parameters used.

Caching is available as an optional feature for researchers and data scientists who run the same analysis repeatedly during iterative development. When caching is enabled, query results are stored locally and served from cache on subsequent identical queries, avoiding redundant API calls. The cache respects data freshness and invalidates automatically when BharatData's underlying data is updated.

Type hints throughout the SDK enable full IDE autocomplete in VS Code with Pylance and in PyCharm. Every function has complete docstrings in NumPy format documenting parameters, return types, examples, and edge cases.

Developers in Other Languages

BharatData's REST API is publicly documented and accessible from any programming language that can make HTTP requests. R developers can use httr2 or curl. Java developers can use any HTTP client. Go developers can use net/http. Ruby developers can use Faraday. Every language's HTTP ecosystem works because the API follows standard REST conventions with JSON responses.

The API documentation provides curl examples for every endpoint, making it straightforward for a developer in any language to understand the request format and response structure. Community-contributed SDKs for other languages are welcome and the Contributing Guide explains how to build and publish one.

How Non-Technical Users Interact With BharatData

The Web Playground

The web playground is a browser-based interface that makes BharatData accessible to everyone without any technical knowledge, any installation, any account creation, or any payment. A journalist on deadline, a student doing homework, a citizen checking a political claim, or a policy analyst preparing a brief can visit the playground URL and start getting answers immediately.

The playground has two modes of interaction.

The first mode is structured querying. The user selects a dataset from a dropdown, selects a state or leaves it blank for all states, selects a year or year range, selects a category if applicable, and

clicks query. The results appear as a sortable, searchable table with every data point linked to its original government source. The user can download the results as CSV with a single click.

The second mode is natural language querying powered by the Generative AI layer. The user types a question in plain English or Hindi in a text box. “Which state had the highest increase in crimes against women between 2019 and 2023?” The AI processes the question, constructs the appropriate queries, fetches the data, and presents the answer as both a data table and a narrative summary. The narrative might read: “Rajasthan reported the highest increase in crimes against women between 2019 and 2023, with total cases rising from 41,550 to 59,204, an increase of 42.5 percent. This was followed by Maharashtra with a 38.2 percent increase and Uttar Pradesh with a 31.7 percent increase. When adjusted for population, however, the rate per lakh tells a different story, with Rajasthan’s rate increasing from 107.6 to 145.3 while Assam’s rate increased from 154.2 to 201.7, making Assam’s per-capita increase steeper despite lower absolute numbers.”

Every statement in the narrative is grounded in actual data from the database. The AI does not speculate about causes, does not editorialize, and does not add information that is not present in the queried data. Every number in the narrative is linked to its source so the user can verify it.

The playground is designed to be the entire product for non-technical users. It is not a demo of the SDK. It is not a documentation tool. It is a first-class product that deserves the same design attention, performance optimization, and usability testing as the SDK itself. A journalist’s experience on the playground determines whether BharatData becomes their go-to tool for government data or whether it becomes another bookmark they never revisit.

Downloadable Reports

For users who need to take data offline, prepare presentations, or share with colleagues who will not visit the playground, BharatData generates downloadable reports. A user can query any data and download the results as a formatted PDF report, a CSV spreadsheet, an Excel workbook, or a JSON file. The PDF report includes the data tables, the AI-generated narrative, source citations, and a methodology note explaining how BharatData collected and processed the data. This report format is designed to be directly shareable with editors, supervisors, colleagues, or audiences.

How Generative AI Works in BharatData

Generative AI in BharatData is not a chatbot. It is not a general-purpose question-answering system. It is a domain-specific intelligence layer that understands Indian civic data and translates between human questions and structured data operations. It is designed to be maximally useful and minimally dangerous.

Natural Language Query Engine

The natural language query engine is the most user-facing AI component. When a user types a question in the playground or through the API, the engine performs a multi-step translation process.

First, it identifies the intent. Is the user asking for specific data? Are they asking for a comparison? Are they asking for a trend? Are they asking about a specific geographic area or a specific time period? The engine uses a domain-specific system prompt that understands Indian civic data terminology. It knows that “UP” means Uttar Pradesh. It knows that “cyber crime” maps to a specific category in the NCRB classification. It knows that “last five years” means the five most recent years available in the data which may not be the literal last five calendar years due to publication delays. It knows that asking about Telangana before 2014 is meaningless because the state did not exist.

Second, it constructs a structured query from the parsed intent. This is not free-text SQL generation. The engine generates a structured query object with specific parameters: dataset, state, year or year range, category, aggregation type, sort order. This structured approach prevents SQL injection, prevents queries on datasets that do not exist, and ensures that every generated query maps to a valid operation on the actual database.

Third, it validates the query before execution. If the generated query asks about a state that does not exist in the data, or a year for which data is not yet available, or a category that does not match any known classification, the engine detects this before hitting the database. Instead of returning empty results or error messages, it provides a helpful response: “I don’t have data for 2025 yet. The most recent available year is 2023. Would you like me to query 2023 instead?”

Fourth, it executes the validated query against the database and returns the results.

Fifth, it optionally generates a narrative summary of the results through the data narration layer.

The engine is designed to fail gracefully at every step. If it cannot parse the intent with confidence, it asks for clarification rather than guessing. If the question is ambiguous, it presents the possible interpretations and asks the user to choose. If the question asks about data outside BharatData's coverage, it says so explicitly and suggests where the user might find that data. It never fabricates data. It never presents uncertain interpretations as confident answers. It never generates numbers that are not in the database.

Data Narration

The data narration layer transforms raw query results into human-readable insights. This is not a summary of the data. It is an analytical narrative that provides context, comparison, and trend information.

When the data shows that Maharashtra had 8,249 cyber crime cases in 2023, the narration layer does not just restate this number. It compares it to other states to establish relative position. It compares it to Maharashtra's own previous years to establish trend. It adjusts for population where rate data is available. It notes any anomalies or unusual patterns. It cites every number it mentions and attributes every comparison to specific data points.

The narration follows strict rules. Every factual statement is grounded in data that was actually returned by the query. No information is added from the AI's training data. No speculation about causes is included. No opinions or value judgments are expressed. The tone is analytical and neutral. The word "significant" is only used in its statistical sense, not as a subjective judgment. Cautions are included when relevant: "Note that lower reported crime numbers may reflect lower reporting rates rather than lower actual crime rates."

This narration layer is specifically designed for journalists who need analytical paragraphs they can use as the factual foundation for their reporting, and for researchers who need quick contextual understanding of their query results before diving into deeper analysis.

Anomaly Detection

The anomaly detection component continuously monitors data for unusual values and patterns. It establishes statistical baselines for each data series — crime rates by state by year, pollution levels by city by month, crop prices by mandi by season — and flags values that deviate significantly from these baselines.

When an anomaly is detected, the Generative AI generates a plain-language explanation. It does not just say “this value is unusual.” It says why it is unusual by referencing the historical baseline, it identifies the magnitude of the deviation, and it provides any contextual information available from other data sources that might explain the anomaly. Seasonal patterns, known events, and historical precedents are all referenced when available.

The anomaly detection is conservative by design. False positives — flagging normal values as anomalous — are worse than false negatives — missing actual anomalies — because false positives erode trust in the system. Thresholds are set high enough that only genuinely unusual values trigger alerts. This conservatism means the system may miss some real anomalies but the ones it does flag are almost certainly worth investigating.

How Agentic AI Works in BharatData

Agentic AI in BharatData operates fundamentally differently from the Generative AI components. While Generative AI responds to user queries one at a time in a single step, Agentic AI works autonomously over multiple steps, making decisions, using tools, and adapting its approach based on what it discovers.

The Data Ingestion Agent

The data ingestion agent is the autonomous system that keeps BharatData’s data current. It runs continuously in the background without any human triggering it and without any human supervising each individual action.

The agent’s cycle begins with perception. It visits every registered government data source and compares the current state with the last known state. Has the NCRB published a new annual report? Has the CPCB updated their air quality portal? Has the Census released new data tables? Has any source changed its URL structure, added new access controls, or modified its data format?

Based on what it perceives, the agent makes decisions. If new data is available at a source, the agent decides to download it. If a source has changed its format, the agent assesses whether the change is minor enough to handle automatically or significant enough to require human attention. If a source is temporarily unavailable, the agent decides to retry later with appropriate backoff. If a new relevant dataset appears on data.gov.in, the agent evaluates its quality and relevance and decides whether to recommend it for inclusion.

The agent then acts on its decisions. It downloads new data. It runs the parser. It feeds the parsed data through the normalizer. It stores the cleaned records in the database with full lineage. If the parser encounters problems, the agent attempts to resolve them. If resolution fails, the agent creates a detailed report of what went wrong and flags it for human review.

After acting, the agent verifies its work. It runs data quality checks on the newly ingested data. It compares record counts against expected values. It checks for statistical outliers. It verifies that the parsed data is consistent with historical patterns. If verification fails, the agent does not push the data to the API. It quarantines it for review.

Finally, the agent reports what it did. A summary of each ingestion cycle is logged: how many sources were checked, how many had new data, how many records were extracted, what quality issues were found, what requires human attention. This report is available through the monitoring dashboard and is also posted as a GitHub issue when significant events occur.

The critical design principle for the ingestion agent is that it should handle routine operations autonomously but escalate novel situations to a human. The agent can handle a source publishing new data in a format it has seen before. It should not autonomously handle a source fundamentally restructuring its data in a way the agent has never encountered. The former is routine maintenance. The latter requires human judgment about how to adapt.

The Research Agent

The research agent handles complex questions that cannot be answered with a single database query. These are questions that require data from multiple sources, multi-step analysis, and synthesis of findings into a coherent narrative.

When a user asks “What is the relationship between air pollution and respiratory disease hospitalizations across Indian states, and how has this relationship changed over the last decade?”, a simple query engine cannot answer this. The research agent can.

The agent begins by planning. It identifies that this question requires air quality data from CPCB, hospitalization data from the Ministry of Health, and population data from the Census for normalization. It identifies the time period as the last ten years. It identifies the geographic level as state-level. It identifies the analytical operations needed: temporal alignment of the three data sources, population normalization, correlation calculation, and trend analysis.

The agent then executes each step of the plan sequentially. It queries air quality data. It queries health data. It queries population data. After each query, it evaluates whether the returned data is sufficient for the analysis. If a particular state is missing from the health data, the agent notes this and decides whether to exclude that state or flag the gap. If the time periods of the data sources do not perfectly align, the agent decides how to handle the mismatch.

After gathering all necessary data, the agent performs the analysis. It calculates correlation coefficients between air quality indices and respiratory hospitalization rates across states. It performs the analysis separately for each year to identify trends. It generates charts showing the relationship and how it has changed over time.

Finally, the agent generates a comprehensive narrative that presents the findings, explains the methodology, cites every data source, notes the limitations, and provides statistical caveats. The output is a complete research brief that a policy analyst could present to a committee or a journalist could use as the factual foundation for a feature story.

The research agent is the most powerful demonstration of how BharatData can serve non-technical users at a level that goes far beyond simple data access. It turns BharatData from a data tool into a research assistant.

The Documentation Agent

The documentation agent handles the automated maintenance of BharatData’s documentation as the system evolves. When a new dataset is added to BharatData, the agent analyzes the new data source’s schema, generates type definitions for both the TypeScript and Python SDKs, creates new query functions following established naming conventions and response shapes, writes documentation with usage examples for both languages, updates the data source registry with the new source’s details, updates the README to reflect the expanded coverage, and generates a changelog entry documenting the addition.

When a user reports a data quality issue, the agent traces the problem through the pipeline, identifies whether the issue originated in the parser, the normalizer, or the source data itself, creates a GitHub issue with a detailed diagnosis, and suggests a fix with a test case to prevent regression.

This agent reduces the documentation and maintenance burden on the solo developer from overwhelming to manageable. Without it, every new dataset would require hours of manual documentation work. With it, the developer reviews and approves the agent's work rather than doing it from scratch.

The Vision of BharatData as India's Data Vault

BharatData's ultimate vision is to become the canonical data layer for India. Not just a tool for accessing a few government datasets. The single, comprehensive, authoritative source that anyone in the world can use to access, understand, and analyze data about India.

This vision unfolds in stages.

In the first stage, BharatData establishes itself as the most reliable way to access the most commonly needed government datasets. Crime statistics from NCRB. Demographic data from the Census. Air quality from CPCB. Agricultural prices from Agmarknet. Economic indicators from RBI. Education statistics from the Ministry of Education. Health data from the Ministry of Health.

In the second stage, BharatData expands to cover the long tail of government data. Election data from the Election Commission. Judicial data from the National Judicial Data Grid. Budget data from the Ministry of Finance. Employment data from the Labour Bureau. Land records from state revenue departments. Company filings from the Ministry of Corporate Affairs. Infrastructure data from the Ministry of Roads and Railways. Water quality data from the Central Water Commission. Forest cover data from the Forest Survey. Mineral resource data from the Geological Survey. And dozens more sources across central and state governments.

In the third stage, BharatData goes beyond government data to include any structured data relevant to understanding India. Academic research datasets that are currently locked in individual researchers' hard drives. NGO data on development indicators that is currently published in isolated reports. Crowdsourced data on public service quality. Media monitoring data tracking government performance. Citizen-reported data on infrastructure quality. Corporate ESG data relevant to India's development.

In the fourth stage, BharatData becomes the platform that enables a new generation of data-driven applications, journalism, research, and governance in India. A developer building any India-focused application starts with BharatData. A journalist researching any India story checks BharatData first. A researcher

studying any India-related topic cites BharatData in their methodology. A government official preparing any policy brief queries BharatData for cross-departmental context.

This is an ambitious vision. It will take years to fully realize. But the architectural decisions made in the first version — the canonical geographic reference system, the confidence scoring framework, the source attribution system, the multi-dataset normalization pipeline, the AI query layer, the dual SDK approach for both TypeScript and Python ecosystems — all of these are designed to scale from one dataset to hundreds. The infrastructure is built for the vault, not just for the first deposit.

The name itself reflects this ambition. Bharat Data. Not “NCRB Data Tool” or “Government PDF Parser.” Bharat Data. India’s data. All of it. Made accessible to everyone.

What Makes BharatData Different

Many projects have attempted to make Indian government data more accessible. DataMeet is a community of civic data enthusiasts who have cleaned individual datasets. Individual developers have published scraped and cleaned datasets on Kaggle and GitHub. Government initiatives like data.gov.in have created portals for data discovery. International organizations like the World Bank and UNDP publish India-specific data through their own platforms.

BharatData is different in five fundamental ways.

First, it is comprehensive by design. It is not a project to clean one dataset. It is infrastructure to clean all datasets. The pipeline, the normalizer, the geographic reference system, the confidence scoring framework — all of these are designed to handle any government data source, not just the first one implemented.

Second, it is developer-first. It is not a website for browsing data. It is an SDK for building with data. The primary interface is a function call, not a web page. This means BharatData’s data can be embedded in applications, automated in pipelines, integrated into research workflows, and used in ways that a browsing-only interface cannot support.

Third, it is AI-powered. The natural language query engine, the data narration layer, the anomaly detection, and the research agent make BharatData accessible to people who cannot write code and enable analyses that would be impractical to perform manually.

Fourth, it is continuously maintained. Government data is not static. It is published on schedules, updated periodically, and retroactively revised. A scraped dataset on Kaggle is a snapshot. BharatData is a living system that continuously ingests new data, updates existing records, and monitors for source changes.

Fifth, it is open source. The code, the data, the canonical mappings, the documentation, and the AI prompts are all publicly available. Anyone can verify how the data was processed. Anyone can contribute improvements. Anyone can build on top of it. This transparency is essential for trust, and trust is essential for a data platform that journalists cite in published articles, researchers cite in academic papers, and developers rely on in production applications.

The Societal Impact

If BharatData succeeds, the impact extends far beyond the individual users who interact with it.

Better journalism becomes possible when reporters can access verified government data in minutes instead of weeks. More data-driven stories get published. More government claims get fact-checked. More citizens are informed about the reality behind political rhetoric.

Better research becomes possible when academics can focus on analysis instead of data collection. Research cycles shorten. More studies are published. Policy recommendations become more evidence-based. The gap between academic understanding and policy action narrows.

Better applications become possible when developers can access real data without building custom infrastructure. More India-specific products get built. More problems get addressed through technology. The Indian technology ecosystem becomes more relevant to Indian reality.

Better governance becomes possible when government officials can see cross-departmental data about their jurisdiction. Decision-making becomes more informed. Resource allocation becomes more evidence-based. Performance monitoring becomes more transparent.

Better civic engagement becomes possible when citizens can access and understand their government’s data. Public discourse becomes more grounded in facts. Political accountability increases. Democracy functions better when the public is informed.

BharatData is a piece of infrastructure. Like a road or a bridge, its value is not in itself but in what it enables. The applications, articles, research papers, policy decisions, and civic conversations that BharatData enables are its true impact. The project succeeds not when it has a million users but when the data flowing through it makes India a better-informed, better-governed, better-understood country.

That is what BharatData is. That is why it exists. That is what it aims to become.

BharatData — The Complete Build Strategy

Starting From Absolute Zero

Before you write a single line of code, before you create a repository, before you design anything, you need to do one thing. You need to understand the data you are going to work with by touching it with your own hands. Not reading about it. Not planning around it. Actually touching it.

Go to the NCRB website right now. Download the most recent Crime in India PDF report. Open it on your screen. Scroll through it. Look at the tables. Try to copy a table from the PDF and paste it into a spreadsheet. Watch what happens. The columns will misalign. The headers will merge into a single cell. The numbers will lose their structure. Footnotes will get mixed into data cells. What was a clean table in the PDF becomes garbage in your spreadsheet.

Now try to do this programmatically. Write a quick Python script using pdfplumber or tabula-py. Point it at one table in the report. See what comes out. It will not be clean. Headers will be wrong. Numbers will be strings. Rows will merge. Cells will be missing. Some tables will parse reasonably well. Others will be completely mangled.

This exercise is not about building a parser. It is about calibrating your understanding of the actual difficulty. Every decision you make from this point forward will be better because you have personally experienced the problem. You will not underestimate the parsing challenge because you have seen it firsthand. You will not overdesign your normalizer because you know exactly what

the raw data looks like. You will not set unrealistic quality targets because you know what 95 percent accuracy actually means when cells have embedded footnotes and merged headers.

Spend two to three days on this. Download reports from multiple government sources. NCRB, Census, CPCB, RBI. Try to extract data from each one. Take notes on every problem you encounter. Take screenshots of particularly messy tables. Write down the exact error messages your parsing tools produce. This is not wasted time. This is the most valuable research you will do for the entire project. These notes become your first blog post, your README introduction, your pitch to users, and your portfolio case study.

After you have done this, you will know something that no amount of planning can teach you. You will know in your bones how hard this problem is and exactly where the difficulty lies. That knowledge is the foundation for everything that follows.

The First Real Decision

Now you need to choose your first data source. Not your only data source. Your first one. The one you will build the entire initial system around.

This decision matters because your first data source determines your first users, your first blog post topic, your first community, and your first feedback loop. Choose wrong and you build something nobody cares about. Choose right and you have an engaged audience from day one.

The criteria for your first data source are these. It must be data that people actively complain about not being able to access. Not data that would theoretically be useful. Data that real people are currently frustrated about. It must be technically challenging enough that solving it demonstrates real engineering skill. A simple CSV cleanup does not impress anyone. A hostile PDF parsing pipeline does. It must have a broad enough audience that journalists, developers, researchers, and students all find it relevant. It must be published by a credible government body so that the data carries authority.

NCRB Crime in India meets all four criteria. Journalists cite it constantly. Researchers study it extensively. Developers want it for applications. Students want it for projects. It is published as a notoriously difficult PDF. And it is produced by a major government bureau under the Ministry of Home Affairs.

But you should verify this choice against what you observed during your hands-on exploration. If you discovered during your research that a different dataset is more painful to access and more frequently needed, choose that one instead. The point is to let evidence drive the decision, not to follow a prescription.

Once you have chosen, commit to it completely. Do not second-guess. Do not keep evaluating alternatives. You will add more sources later. Right now, you need depth on one source, not breadth across many.

Building the Knowledge Foundation

Before you write application code, you need to create the knowledge documents that will guide every decision you make during development. These are not bureaucratic exercises. They are thinking tools. Writing them forces you to make decisions explicitly rather than implicitly, which prevents you from contradicting yourself later and prevents an AI coding assistant from making decisions you did not intend.

Write your Project Overview first. One document that explains what BharatData is, why it exists, and who it is for. Keep it to two pages. Write it in your own voice, not in corporate language. This document is for you as much as anyone else. When you are deep in parser debugging at two in the morning and wondering why you started this project, this document reminds you.

Write your Product Requirements next. This is the most important document in the entire project. It defines what you are building in the first version and more critically what you are not building. List every feature that is included. Then list every feature that is excluded. The excluded list should be longer than the included list. No AI features in the first version. No multiple datasets. No user accounts. No paid tiers. No mobile app. No visualization tools. Write these exclusions down because when you are building and a shiny idea occurs to you, you need a written commitment to yourself that you will not chase it right now.

Write your SDK Design Specification. Both the TypeScript and Python SDKs. Design the API surface on paper before writing code. What functions exist. What parameters they accept. What they return. What the error cases are. Think about this from the perspective of a developer who has never heard of BharatData and just installed the package. What would they expect to type? What would they expect to get back? Make the API so predictable that a developer can guess correctly how a function works before reading the documentation.

Write your Quality Standards. Non-negotiable rules that apply to every piece of code and every piece of data. Every record must have source attribution. No data is served without a confidence score. Wrong data with a low confidence flag is acceptable. Wrong data served confidently is a project-destroying failure. Write these standards down because they are the rules you will be tempted to relax when you are behind schedule, and relaxing them will cost you more than any delay.

Write your Data Pipeline Specification. How data flows from government source to your database. Four stages: ingest, parse, normalize, store. Document each stage's responsibilities, inputs, outputs, and failure modes. This document becomes the blueprint your AI coding assistant follows when generating pipeline code.

Write your Normalization Guide. How you handle state name variations, district name variations, historical boundary changes, Indian number formatting, fiscal year versus calendar year, and null value representations. This is the intellectual core of your project. The canonical mappings file that emerges from this guide will become one of the most valuable artifacts of the entire BharatData ecosystem.

Write your Data Source Document for your first source. Everything you learned during your hands-on exploration. The URL. The format. The publishing body. The update frequency. The specific parsing challenges. Which tables you plan to extract. Known quality issues in the source itself. This document is the reference your parser will be built against.

Configure your IDE. Set up Antigravity with the right models, the right MCP servers, the right extensions, and the right skill files. Create agent definitions for the pipeline specialist, the SDK architect, the data quality auditor, and the documentation specialist. Create workflow definitions for validation, ingestion, publishing, and adding new sources. Create the feeding prompt that loads all your knowledge documents at the start of every session.

All of this work happens before you write a single line of application code. It takes roughly a week of focused effort. It feels slow. It feels like you should be building instead of writing. But every hour you spend on these documents saves you three hours of confusion, rework, and debugging later. The developers who skip this step build faster in week one and slower in every subsequent week because they are constantly discovering decisions they forgot to make and contradictions they did not anticipate.

Building the Data Pipeline

Now you build. And you build in a very specific order because each component depends on the one before it.

Start with the ingester. This is the simplest component but it must be robust from day one because everything downstream depends on it. The ingester visits a government source URL, checks whether new data is available by comparing file hashes, downloads the raw file if it has changed, and stores it with complete metadata including the source URL, download timestamp, file hash, file size, and HTTP status code.

The ingester must handle failure gracefully. Government websites go down. They return error pages instead of data. They block requests that come too fast. Your ingester must retry with exponential backoff. It must log every failure with enough detail to diagnose the problem the next morning. It must never crash silently. If it fails three consecutive times on the same source, it must create a visible alert so you know something is wrong.

Rate limiting is non-negotiable from the start. Never send more than one request per second to any government server. This is not just ethical. It is practical. If your scraper overwhelms a government website and causes an outage, you will attract attention you do not want. Build the rate limiter into the ingester at the foundation level, not as an afterthought.

Test the ingester by running it against your actual data source. Not a mock. The real government website. Verify that it downloads the correct file. Verify that the metadata is complete. Run it twice and verify that it correctly detects no new data on the second run. Run it when the website is slow and verify that the timeout handling works. Run it with your internet disconnected and verify that the error logging captures the failure clearly.

Next, build the parser. This is the hardest component in the entire project. It takes the raw file downloaded by the ingester and extracts structured records from it.

For PDF sources like NCRB, the parser must handle every challenge you discovered during your hands-on exploration. Multi-row headers with merged cells. Tables continuing across pages. Footnote markers embedded in data cells. Indian number formatting. Summary rows mixed with data rows. Mixed Hindi and English text. Your hands-on notes are your reference. Build the parser to handle the specific problems you actually observed, not hypothetical problems.

Every row extracted by the parser must have a confidence score. A row where every cell was extracted cleanly and the row total matches the sum of components gets a high score. A row where

the parser had to strip footnote markers or convert number formats gets a slightly lower score. A row where cells were ambiguous or could not be reliably parsed gets a low score. A row that could not be parsed at all gets a zero and is stored for debugging but never served to users.

Test the parser obsessively. Pick 20 random cells from across different tables in the original PDF. Manually verify each one against the parser output. Calculate your accuracy percentage. If it is below 95 percent, identify the failure patterns and fix them. Retest. Repeat until accuracy exceeds 95 percent on your spot checks.

Document every parsing failure you encounter in your project journal. The specific table, the specific cell, the specific reason the parser failed. These failure notes are the raw material for your blog post, your portfolio case study, and your regression test suite.

Next, build the normalizer. This takes the raw parsed records and transforms them into canonical form. State names are resolved to standard forms using your canonical mappings file. District names are resolved similarly. Numbers are converted from Indian notation to standard integers. Dates are converted to ISO 8601. Null values from various representations (dash, N/A, blank, NA) are converted to actual null with metadata preserving the original representation. Historical boundary changes are annotated.

The normalizer must never silently drop a record. If it encounters a state name or district name that is not in the canonical mappings file, it must pass the record through with the raw name intact, reduce the confidence score, and log the unmapped name to a separate file for review. This unmapped entities file grows over time and is periodically reviewed. New legitimate variations are added to the canonical mappings. Genuine errors in the source data are documented.

Next, set up the database. Design the schema to support multiple datasets without restructuring. Even though you are starting with one dataset, your table design should accommodate future datasets cleanly. Every record in the database must store the canonical data fields, the raw source reference including URL and page and table identifier, the collection date, the confidence score, the normalizer version that processed it, and the ingestion batch identifier.

Create indexes for the query patterns your SDK will use. By state. By year. By category. By state and year together. By confidence score so you can efficiently filter out low-confidence records.

Run the complete pipeline end to end. Ingester downloads the PDF. Parser extracts records. Normalizer cleans them. Records go into the database. Query the database and verify the data looks

correct. Spot check ten random records against the original PDF. This end-to-end test is your first real validation that the system works.

Set up monitoring. A scheduled job that runs daily, checks whether your data source URLs are still accessible, runs the parser against the most recent download and checks for schema drift, and creates alerts if anything has changed. This monitoring system prevents the silent data rot that kills data projects. Without it, you would only discover that a government website redesigned itself weeks later when a user reports wrong data.

Building the Developer Interfaces

With clean data in your database, you now build the interfaces that let people access it.

Build the TypeScript SDK first because TypeScript developers are your most likely early adopters. The SDK is an npm package that wraps your database in clean, typed function calls.

Follow your SDK Design Specification exactly. The function signatures you designed on paper are now implemented in code. Do not deviate from the spec unless you discover during implementation that something in the spec was genuinely wrong, in which case you update the spec first and then implement the updated design.

Every function returns the standard response shape you defined. Data array. Meta object with source attribution. Query echo. Every record includes all the fields you specified including confidence score and source URL. Every error condition throws the appropriate custom error class with a human-readable message and a resolution suggestion.

Write the TypeScript types with extreme care. These types are the primary developer experience of your SDK. A developer using your package in VS Code or Antigravity should see full autocomplete for every field on every response. They should never have to guess what a function returns. They should never encounter an any type. They should see JSDoc comments explaining every parameter and every return type when they hover over a function call.

Test the SDK by writing a small application that uses it. Not unit tests of individual functions, though those matter too. An actual small application that installs the package from a local build,

creates a client, makes several queries, and uses the results. If the developer experience of this small application feels clunky, fix the SDK before publishing.

Publish to npm. Verify that npm install bharatdata works in a clean directory. Verify that the types work. Verify that the first query returns data. This is a real launch moment even though it is not a public announcement. Your package exists on the public npm registry. Anyone can find it and install it.

Build the Python SDK next. This is equally important because data scientists, ML engineers, and researchers are a massive audience for Indian government data. The Python SDK mirrors the TypeScript SDK in structure but adapts to Python conventions.

snake_case everywhere instead of camelCase. Type hints on every function using Python's typing module. Docstrings in NumPy format on every public function. The critical Python-specific feature is pandas DataFrame integration. Every query function should offer a way to get results as a pandas DataFrame with proper column types, meaningful column names, and the source attribution preserved as columns in the DataFrame.

Build export methods. to_csv, to_parquet, to_json, to_excel. These are essential for researchers who need to take data into Stata, SPSS, R, or other analysis tools. Parquet export is particularly important for data science workflows because it preserves types and handles large datasets efficiently.

Build the citation generation method. A researcher should be able to call a function that generates a properly formatted academic citation for the data they queried. This small feature creates enormous goodwill in the academic community because it demonstrates that you understand and respect their workflow.

Publish to PyPI. Verify that pip install bharatdata works. Verify that the pandas integration works in a Jupyter notebook. Verify that a simple query returns a clean DataFrame.

Build the REST API. This is the backend that both SDKs call. It exposes endpoints that mirror the SDK functions. GET endpoints for each query type. JSON responses matching the SDK response shape. Global rate limiting. CORS headers so the playground can call the API from the browser. Error responses matching the SDK error format.

Deploy the API to a free tier hosting service. Vercel or Cloudflare Workers are both good options. Verify that the endpoints work from both SDKs.

Build the web playground. This is the interface for non-technical users. It is a single-page web application with a text input at the top and results displayed below. A user visits the URL, types a query or selects filters, and sees data in a table with a CSV download button. No login. No signup. No account required.

The playground must load fast. The first query must return results within a few seconds. The table must be sortable and searchable. The CSV download must work on every browser. Source attribution must be visible on every data point. The playground is the first thing a journalist or researcher will try. If it is slow, confusing, or broken, they will leave and never return.

Deploy the playground to a public URL. This is another real launch moment. Your product is now live on the internet. Anyone with the URL can access Indian government data through a clean interface.

Writing the README and Documentation

Your README is the single most important piece of content in your entire project. More people will read your README than will read all other documentation combined. More people will form their opinion of BharatData based on the README than based on actually using the product.

The README must accomplish five things in strict order. In the first three seconds, the reader understands what BharatData is. This means the project name, the tagline, and a one-sentence description must be immediately visible without scrolling. In the first thirty seconds, the reader sees how to install and use it. This means npm install bharatdata and pip install bharatdata commands followed by a three-line code example showing a query that returns real data. In the first two minutes, the reader knows what datasets are available and where to try the playground. In the first five minutes, the reader understands the scope, the limitations, and where the data comes from. In the first ten minutes, the reader knows how to contribute.

Write the README yourself. Not with an AI. In your own voice. The AI can help you organize and edit, but the README must sound like a real person wrote it because it did. Authenticity in a README builds trust. Marketing language in a README destroys it.

Write the API reference documentation for both SDKs. Every function. Every parameter. Every return type. Every error case. With code examples that actually work. Test every code example

in your documentation by running it. A code example that does not work is worse than no code example at all because it wastes the developer’s time and damages their trust.

Write quick start guides for each audience. A TypeScript developer quick start. A Python data scientist quick start. A journalist playground quick start. Each guide is one page. Each guide gets the user from zero to their first useful result in under five minutes. Each guide is self-contained. A journalist should never need to read the developer documentation, and a developer should never need to read the journalist guide.

Write the contributing guide. How to set up the development environment. How to report bugs. How to suggest new data sources. How to contribute canonical name mappings, which is the easiest and most valuable contribution path for newcomers. How to submit code changes.

Write the legal documents. Terms of use in plain language. Privacy policy in plain language. Data attribution and legal compliance document explaining your approach to government data licensing, scraping ethics, and independence from government. These are not exciting documents but their absence makes your project look unprofessional and creates legal uncertainty for users who want to cite your data in published work.

Getting Your First Real Users

Everything you have built so far is invisible to the world. Your package is on npm and PyPI but nobody knows it exists. Your playground is live but nobody has the URL. Your GitHub repository exists but nobody has seen it. Now you need to put BharatData in front of real people and learn from their reactions.

You do not start with a big launch. You start with targeted, personal outreach to people who are most likely to care.

Find five people who have publicly complained about Indian government data access in the last six months. These people exist on Twitter, Reddit, and DataMeet forums. They have posted things like “spent two weeks cleaning NCRB data” or “why is government data so hard to access” or “does anyone have clean crime statistics.” These are your ideal first users because they have already expressed the exact pain you solve. They do not need to be convinced that the problem exists. They need to be shown that a solution exists.

Do not send them a marketing message. Send them something personal and specific. Reference their original complaint. Show them exactly how BharatData solves the problem they described. Include a direct link to the playground where they can try it immediately without installing anything. Include a direct link to the npm or PyPI package if they are a developer. Ask them to try it and tell you what breaks. Not what they think about it. What breaks. This framing is important because it sets the expectation that you want honest critical feedback, not polite encouragement.

Most of these people will not respond. Some will try it briefly and not follow up. One or two will actually use it and give you real feedback. Those one or two people are worth more than a thousand Product Hunt upvotes because they are telling you what real users actually need, not what imaginary users might theoretically want.

Write your first blog post. Not about BharatData. About the problem. The title should be something like “I Tried to Extract Clean Crime Data From NCRB PDFs and Here Is What Broke.” Write about your hands-on exploration. Show the messy PDF tables. Show the parsing failures. Show the before and after comparison of raw government data versus clean BharatData output. Make it genuinely interesting to read even for someone who will never use your tool. Link to BharatData at the end, not at the beginning. The post is not an advertisement. It is a story about a real problem. BharatData is mentioned as the thing you built because you got frustrated enough to do something about it.

Publish the blog post on Dev.to and Medium. Share it on Twitter with screenshots comparing messy PDFs to clean API output. Post it on the developersIndia subreddit. Share it in DataMeet community channels. The blog post serves as your introduction to the communities you want to be part of. It establishes your credibility as someone who genuinely understands the problem. People who read it to the end are exactly the people who might use your tool.

Post on Hacker News with a Show HN submission. Write your own title and first comment. Be honest about the current scope. Do not claim to have solved all of Indian government data. Say you have one clean dataset and you are expanding based on demand. Hacker News respects honest scope more than ambitious claims. Respond to every comment. This is not optional. Every comment on a Show HN post is a gift of someone’s attention and leaving it unanswered is disrespectful of that gift.

Reach out to two or three hackathon organizers at Indian colleges. Offer BharatData as a data resource for their participants. Hackathon participants are perfect early users because they are desperate for real data under extreme time

pressure. If BharatData saves a hackathon team from spending their entire 24 hours on data collection, that team becomes an advocate.

Contact one professor at any Indian university who teaches data science, public policy, or statistics. Show them the Python SDK and the playground. Ask if they would be willing to mention it to their students as a data source for projects. One professor recommending BharatData to a class of 60 students can give you more real users in a day than a week of social media posting.

Learning From Users and Deciding What to Build Next

Once you have real users, even just five or ten of them, you enter a fundamentally different mode of building. You stop building what you think is needed and start building what users demonstrate is needed.

Watch how people use the playground. If you have basic analytics, look at what queries people type. What datasets do they ask for that you do not have? What states do they query most? What time periods? What categories? Every query that returns “data not available” is a signal about what to build next.

Read every GitHub issue carefully. Not just for the specific bug or feature request. For the underlying need. A user who reports “the CSV download does not include confidence scores” is telling you that they care about data reliability. A user who asks “can I get this data as a pandas DataFrame” is telling you they are a Python data scientist. A user who asks “where does this number come from” is telling you that source attribution matters to their use case.

When someone uses BharatData in a real project or publication, pay attention. If a journalist publishes a story citing BharatData, read the story carefully. What data did they use? What analysis did they perform? What did they have to do manually that BharatData could have done for them? If a student uses BharatData in a hackathon project, look at their code. Where did they struggle? What workarounds did they build?

Based on this learning, choose your second data source. Not based on what you find interesting. Based on what users are asking for. Check your GitHub issues. Check your playground query logs. Check the feedback from your direct outreach. The data source that comes up most frequently in user requests is your second source. Build the pipeline for it. Run it through the same ingestion, parsing, normalization, and storage stages. Add

SDK functions for it following the same patterns as the first source. Update the playground to support it. Update the documentation.

This user-driven expansion is critical. Every data source you add increases your maintenance burden. If you add a source nobody asked for, you bear the maintenance cost with no user benefit. If you add a source that ten users requested, those ten users become more engaged, more likely to contribute, and more likely to recommend BharatData to others.

Building the AI Layer

After you have at least two datasets in the system and a real user base that is actively using the data, you build the AI layer. Not before. The AI layer built on top of a product with real users will be completely different from the AI layer you would build in isolation, because real user queries teach you what the AI actually needs to understand.

Start with the natural language query engine. This is the highest-impact AI feature because it makes BharatData accessible to non-technical users. A journalist who cannot write code but can type “which state had the most cyber crime last year” gets the same data as a developer calling bd.crime.getByCategory.

The system prompt for the query engine must be deeply domain-specific. It must understand Indian civic data terminology. IPC crimes versus SLL crimes. State abbreviations. District hierarchies. Temporal coverage of each dataset. Crime category names as they appear in NCRB classification. The system prompt is not something you write once and forget. It is something you refine continuously based on query failures.

Keep a log of every query that fails or produces wrong results. Every failed query teaches you something about how people actually phrase questions about Indian data. “Show me crime in UP” fails because the engine does not know that UP means Uttar Pradesh. Add that mapping. “What about last year” fails because the engine does not know which year is “last year” given NCRB’s publication delay. Add that temporal logic. “Compare Mumbai and Delhi” fails because Mumbai is a city within Maharashtra, not a state. Add that geographic hierarchy understanding.

Review this failure log weekly. Each fix makes the engine better. Over months, you build the most sophisticated understanding of Indian civic data queries that exists anywhere. This accumulated query intelligence becomes a significant competitive advantage that cannot be replicated by simply copying your code.

Add the data narration layer after the query engine is working reliably. The narration takes raw query results and generates two to three sentence contextual summaries. This feature is specifically designed for journalists and researchers who need not just numbers but analytical context.

The narration must follow strict grounding rules. Every statement must be based on data that was actually returned by the query. No information from the AI's training data. No speculation about causes. No opinions. Every number cited must match a number in the query results. Source citations must be preserved. When the narration says "Maharashtra reported 8,249 cyber crime cases in 2023," the number 8,249 must appear in the query results and must be traceable to the original NCRB source.

Add anomaly detection after the narration layer is working. The anomaly detector establishes statistical baselines for each data series and flags values that deviate significantly. It generates plain-language explanations for detected anomalies. This feature is conservative by design. False positives destroy trust faster than missed anomalies, so thresholds are set high.

Each AI feature goes through the same cycle. Build it. Test it with real queries from your failure log and your user analytics. Ship it to the playground. Watch how users interact with it. Collect failures. Fix them. Repeat. The AI layer is never done. It is continuously improving based on real usage data.

Growing the Data Coverage

With the AI layer working and a growing user base, you systematically expand data coverage. Each new data source follows the same pipeline pattern: ingest, parse, normalize, store, add SDK functions, update playground, update documentation.

The order of data source addition is always driven by user demand. You might personally find railway data fascinating, but if your users are asking for air quality data, you build the CPCB air quality pipeline first. User demand ensures that every source you add immediately serves real needs and generates engagement.

As you add more sources, the cross-dataset analysis capability becomes increasingly powerful. With crime data and demographic data, users can study crime patterns relative to population characteristics. Add air quality data and they can study environmental health correlations. Add education data and they can study the relationship between literacy and crime. Add

economic data and they can study the impact of employment on crime rates. Each new dataset does not just add one more thing to query. It multiplies the number of questions that can be answered.

The Agentic AI research agent becomes viable when you have three or more datasets. This agent handles complex questions that span multiple datasets, performing the multi-step analysis that no single query can accomplish. “How does air quality correlate with respiratory disease hospitalizations across Indian states?” requires air quality data, health data, and population data. The research agent plans the analysis, queries each dataset, aligns the data temporally and geographically, calculates correlations, generates visualizations, and writes a comprehensive narrative with methodology notes and statistical caveats.

The autonomous data ingestion agent also becomes more valuable as you add sources. Monitoring one data source manually is easy. Monitoring fifteen is a full-time job. The ingestion agent monitors all registered sources, detects new publications, adapts to format changes, ingests new data, runs quality checks, and escalates novel problems to you. This agent is what makes BharatData sustainable as a solo developer project at scale.

Building the Public Identity

As your data coverage grows and your user base expands, you build BharatData’s public identity through consistent, valuable content.

Write a technical blog post about your normalization system. How you handle 780 districts and their name variations. How you track boundary changes. How you resolve the geographic identity crisis across government data sources. This post attracts contributors who want to improve the canonical mappings and establishes your technical credibility in the data engineering community.

Write a data story that uses BharatData to answer a genuinely interesting question about India. Use your own tool to produce an analysis that a journalist would want to publish or a researcher would want to cite. This post demonstrates BharatData’s value by showing it in action, not by describing it in theory.

Write a technical AI post about your natural language query engine. How you handle domain-specific terminology. How you prevent hallucination. What your failure rate is and how you are reducing it. This post establishes credibility in the AI community and attracts developers who are building similar domain-specific AI applications.

Write a retrospective at the six-month mark. What worked. What broke. What surprised you. What you would do differently. Be honest about the failures. The developer community respects honesty about mistakes more than it respects a polished success narrative. This post builds personal brand and community trust simultaneously.

Launch on Product Hunt when you have the AI layer working and multiple datasets available. This is your broad-audience launch moment. You should have real users, real usage data, real testimonials, and a polished product before this launch. Write a genuine maker story. Respond to every comment personally.

Speak at meetups and conferences. Start with local developer meetups. Submit talks about the challenges of Indian government data or about building domain-specific AI applications. These talks position you as a domain expert and bring your project to audiences who will never find it through online channels.

Building Toward Sustainability

When BharatData has consistent usage, engaged users, and visible impact, you begin exploring sustainability. Not because you need revenue to survive. Because you need resources to maintain and grow the platform beyond what a single person can do as a side project.

The first sustainability path to explore is consulting. By the time you have built reliable data pipelines for multiple government sources, you have developed rare expertise. You understand Indian government data systems. You know which sources are reliable and which are not. You know how to clean government PDFs. You know how to normalize Indian administrative geography. This expertise is valuable to organizations that struggle with the same challenges.

State IT departments are under pressure to make their data accessible. Think tanks and research organizations need clean data for policy analysis. International development organizations working in India need reliable Indian data. Any of these could become consulting clients. You are not selling BharatData itself. You are selling the expertise you developed by building it.

One consulting engagement can fund months of development. Two or three engagements in a year make the project financially self-sustaining without requiring any SaaS revenue at all.

The second sustainability path is freemium API access. Your free tier remains generous enough that individual developers, students, journalists, and researchers never need to pay. The paid

tier targets organizations that need higher rate limits, guaranteed uptime, priority support, and premium features like the research agent for complex multi-dataset queries. You do not build the paid tier until you have clear signals that organizations want to pay. Those signals come from usage patterns, not from speculation. When you see company email domains making hundreds of API calls per day, reach out to those companies and ask if a paid tier with higher limits would be useful.

The third sustainability path is grants. Organizations like the Shuttleworth Foundation, Mozilla Foundation, and Ford Foundation fund open data and civic technology projects. India-specific foundations and corporate social responsibility programs also fund technology for social good. These grants provide funding without equity dilution and validate your work in the eyes of the broader open data community.

The fourth sustainability path is government partnerships. If BharatData demonstrates significant public value, there is a real possibility of formal data sharing agreements with government departments. NITI Aayog has expressed interest in better data accessibility. State governments are digitizing their data under national initiatives. A partnership where a government department officially provides data to BharatData, rather than BharatData scraping it, would solve access reliability issues and provide legitimacy that no amount of marketing can buy.

None of these sustainability paths require you to transform BharatData into a traditional SaaS business with aggressive growth metrics and investor expectations. They allow BharatData to remain what it is: an open source public good with enough resources to be maintained well.

Building the Full Platform

As BharatData matures, the web playground evolves into a full platform. Not a SaaS product in the traditional sense. A comprehensive public data platform that serves every audience through the interface most appropriate for them.

The developer portal provides SDK documentation, API reference, code examples, and integration guides for both TypeScript and Python. It includes a sandbox where developers can test queries against live data without installing anything.

The research hub provides tools specifically for academic users. Dataset exploration with statistical summaries. Codebook generation for academic methodology sections. Citation export in

multiple academic formats. Data download in formats compatible with Stata, SPSS, R, and Python analysis tools. Saved query collections for reproducible research.

The media toolkit provides tools specifically for journalists. Pre-built analyses for common story types like state comparisons and year-over-year trends. Embeddable charts and tables that journalists can include in online articles. Fact-check tools that compare political claims against actual government data. Source verification that shows the exact government document every number came from.

The civic dashboard provides tools for citizens. Simple visualizations of key indicators for their state or district. Comparisons against national averages. Trend lines showing whether things are getting better or worse. Plain language explanations of what the numbers mean. This is the most ambitious interface because it must be usable by people with no data literacy.

The data observatory provides tools for monitoring and discovering patterns in government data. Real-time anomaly alerts. Cross-dataset correlation discoveries. Automated trend reports. This is where the Agentic AI capabilities are most visible, proactively surfacing insights that no one explicitly asked for.

Each of these interfaces is a layer on top of the same underlying data infrastructure. They share the same data pipeline, the same database, the same normalization, and the same quality standards. They differ only in how they present data and what tools they provide for working with it.

Collecting Feedback at Every Stage

Feedback collection is not a phase. It is a continuous practice woven into every stage of building BharatData. Here is how you ask for feedback at each stage and what you do with what you learn.

When you first reach out to potential users before launching, you are not asking for feedback on a product. You are asking about their workflow. The question is not “would you use BharatData” because nobody can honestly answer that about a product they have never seen. The question is “how do you currently get government data and what is the most painful part.” Their answers guide your build priorities.

When you share the playground URL with your first five users, you ask a different question. “Try querying crime data for your state. Tell me what breaks.” Not “tell me what you think.” What breaks.

This framing gives you actionable information. “The query returned no results for Chhattisgarh” tells you that your normalizer is not handling that state name correctly. “I could not figure out how to download the data” tells you that your CSV download button is not visible enough.

When you post on Hacker News and Reddit, you are collecting feedback at scale. Read every comment. Categorize them. Feature requests go into a list. Bug reports go into GitHub issues. Technical criticism goes into your architecture review. Emotional reactions tell you how your positioning resonates. A comment that says “I have been waiting for something like this for years” tells you that your problem framing is correct. A comment that says “this is just a wrapper around a CSV file” tells you that your value proposition is not coming through clearly.

When you add analytics to the playground, you are collecting behavioral feedback that people never articulate. The queries they type. The datasets they query most. The states they look at. The year ranges they request. The percentage who download CSV versus just browsing. The percentage who return versus visiting once and never coming back. Each of these metrics tells you something about whether BharatData is genuinely useful or merely interesting.

When you open GitHub Discussions on your repository, you create a permanent channel for feedback. Seed it with questions. “Which government dataset should we add next?” “How should we handle districts that were bifurcated recently?” “What export format would be most useful for your workflow?” These questions invite participation and make users feel like co-builders rather than passive consumers.

When you write blog posts, the comments and shares are feedback. A post about your normalization challenge that gets shared widely tells you that the technical community finds that problem interesting. A post about a data finding that gets picked up by media tells you that the journalistic community values your analysis capability.

When a user builds something with BharatData and shares it publicly, that is the strongest feedback signal. It means they found enough value to invest their own time in building on top of your work. Study what they built. What data did they use? What did they have to work around? What would have made their job easier?

At every stage, the feedback loop is the same. Observe what users do. Ask them what is broken. Prioritize fixes and features based on what you observe and what they report. Build the highest-

demand item. Ship it. Observe again. The product evolves not according to a predetermined roadmap but according to the continuous signal from real usage.

The Strategy in One Sentence

Touch the data with your own hands, build one pipeline perfectly, ship both SDKs and a playground, put it in front of five real people who feel the pain, listen to what breaks, fix it, add what they ask for, write about the journey, let the community guide expansion, add AI when you have enough data and users to make it genuinely useful, grow the platform surface by surface based on which audience engages most deeply, and explore sustainability when the project’s impact is visible enough that organizations want to support it.

That is the entire strategy. Everything else is execution detail.

BharatData — Complete Data Source Registry

Understanding the Landscape

The Indian government publishes data through hundreds of ministries, departments, bureaus, commissions, regulatory bodies, and autonomous institutions at both central and state levels. Not every government data source is worth integrating into BharatData. Some are too small to matter. Some are already available in clean formats. Some are behind impenetrable access barriers. Some are updated so rarely that the maintenance cost exceeds the value.

The sources listed below are selected based on four criteria. First, how many people actively need this data and currently struggle to access it. Second, how painful the current access method is. Third, how broad the potential user base is across developers, journalists, researchers, students, and citizens. Fourth, how feasible the ingestion is given the source format and access method.

Each source is assigned a priority tier. Tier 1 sources are the first to be integrated because they have the highest demand, the most painful access experience, and the broadest audience. Tier 2 sources are integrated after the core platform is proven and the

first users are generating demand signals. Tier 3 sources are integrated based on specific user requests and community demand. Tier 4 sources are aspirational and may require special access arrangements or partnerships.

Tier 1 — Foundation Sources (Build First)

These are the data sources that BharatData must have to be taken seriously as a platform. They represent the most commonly needed, most frequently cited, and most painfully inaccessible government datasets. Each one individually justifies the existence of BharatData.

---

1. National Crime Records Bureau — Crime in India

Publishing Body: National Crime Records Bureau, Ministry of Home Affairs, Government of India

What Data It Contains: Comprehensive annual crime statistics for India covering all Indian Penal Code crimes including murder, kidnapping, robbery, theft, dacoity, riots, crimes against women, crimes against children, crimes against scheduled castes and scheduled tribes, economic offences, cyber crimes, and offences against the state. Also covers Special and Local Laws crimes including NDPS Act violations, Arms Act violations, gambling, and prohibition offences. Includes data on police disposal of cases, court disposal of cases, conviction rates, pendency rates, and chargesheeting rates. Covers state-level and in some tables district-level breakdowns.

Current Format: PDF report of 400 plus pages containing approximately 500 tables. Some supplementary data in Excel format on the NCRB website.

Update Frequency: Annual. Published approximately 8 to 12 months after the reference calendar year.

Coverage Period: Reports available from 1953 to present. BharatData should initially target 2015 to latest.

Why It Matters: Crime data is among the most frequently cited government statistics in Indian media. Every major news outlet references NCRB data multiple times per year. Researchers across criminology, sociology, public policy, gender studies, and urban planning use it extensively. Developers building safety

applications, insurance risk models, and civic dashboards need it. It is also one of the most painful sources to access due to the complex PDF format.

Who Uses This Data: Journalists investigating crime patterns and government claims about law and order. Researchers studying crime trends, criminal justice system efficiency, and social factors correlated with crime. Developers building safety apps, risk models, and civic dashboards. Policy makers evaluating policing effectiveness. Citizens verifying claims about crime in their state. Lawyers and legal researchers studying conviction rates and case pendency.

Integration Priority: First source. Build the entire initial pipeline around this.

---

2. Census of India

Publishing Body: Office of the Registrar General and Census Commissioner, Ministry of Home Affairs, Government of India

What Data It Contains: The most comprehensive demographic dataset for India. Population counts by state, district, and sub-district. Age distribution. Sex ratio. Literacy rates by gender. Urban and rural population split. Scheduled caste and scheduled tribe population. Religious composition. Language data. Migration data. Housing conditions including building materials, drinking water source, lighting source, latrine availability, and drainage. Household assets including ownership of radio, television, computer, phone, bicycle, scooter, and car. Workers and non-workers classification by industry category. Fertility and mortality indicators.

Current Format: Mix of Excel files, PDF tables, and HTML tables distributed across multiple Census of India websites. The primary data tables are available as Excel downloads but with inconsistent formatting across different data categories. Some detailed tables are in PDF. The Census provides a data dissemination portal but it is slow, poorly designed, and frequently unavailable.

Update Frequency: Decennial. Census 2011 is the most recent completed census. Census 2021 was postponed due to COVID-19 and its timeline remains uncertain. Annual population estimates are published by the Registrar General.

Coverage Period: Census data available from 1951 to 2011 in varying levels of detail and format.

Why It Matters: Census data is the foundational demographic dataset for India. Virtually every cross-sectional analysis of India requires census data for population normalization. Crime rates

per lakh population, hospital beds per thousand people, school enrollment ratios, voter turnout percentages — all require census population figures as the denominator. Without clean census data, no other government dataset can be properly analyzed. Census data is also the primary source for district-level development indicators.

Who Uses This Data: Every developer building any India-focused application that needs demographic context. Every researcher studying any aspect of Indian society. Every journalist normalizing any per-capita statistic. Every policy maker assessing district or state development levels. Every data scientist training models on Indian demographic features. Every urban planner studying migration and urbanization.

Integration Priority: Second source. Essential for population normalization of all other datasets.

---

3. Central Pollution Control Board — Air Quality Data

Publishing Body: Central Pollution Control Board, Ministry of Environment, Forest and Climate Change, Government of India. Data also contributed by State Pollution Control Boards.

What Data It Contains: Continuous ambient air quality monitoring data from over 300 stations across India. Measures particulate matter (PM2.5, PM10), nitrogen dioxide (NO2), sulfur dioxide (SO2), ozone (O3), carbon monoxide (CO), ammonia (NH3), and lead (Pb). Provides hourly, daily, and annual averages. Calculates the Air Quality Index (AQI) for each station. Also publishes the National Air Quality Index with city-level daily readings. Historical data available for manual air quality monitoring stations going back decades.

Current Format: Data available through the CPCB online portal which uses JavaScript rendering making it resistant to simple scraping. Historical data in PDF reports. Real-time data through a separate portal. Bulk download is not available through any official channel. Accessing station-level historical data requires navigating a complex web interface one station at a time, one parameter at a time.

Update Frequency: Real-time for continuous monitoring stations. Daily AQI published every day. Annual reports published annually.

Coverage Period: Continuous monitoring data available from approximately 2015 onward for most stations. Manual monitoring data available from the 1990s for some stations.

Why It Matters: Air pollution is India's most pressing environmental health crisis. The WHO estimates that air pollution contributes to over a million premature deaths in India annually. Delhi's air quality crisis receives global media attention every winter. Despite this, accessing clean historical air quality data for analysis is extraordinarily difficult. Journalists writing about pollution trends cannot easily compare cities or track improvement over time. Researchers studying health impacts of pollution cannot easily correlate air quality data with health outcomes. Environmental activists cannot easily verify government claims about air quality improvement.

Who Uses This Data: Environmental journalists covering pollution stories across Indian cities. Health researchers studying respiratory disease correlations with air quality. Climate scientists studying atmospheric chemistry and emissions patterns. Urban planners assessing livability and environmental infrastructure. App developers building air quality alert applications. Environmental lawyers building cases around pollution compliance. Citizens checking the air quality in their city.

Integration Priority: Third source. High public interest, strong journalist demand, clear environmental impact.

---

4. Agmarknet — Agricultural Market Prices

Publishing Body: Directorate of Marketing and Inspection, Ministry of Agriculture and Farmers Welfare, Government of India

What Data It Contains: Daily wholesale prices of agricultural commodities from approximately 7,000 regulated agricultural markets (mandis) across India. Covers over 300 commodities including food grains (wheat, rice, maize), pulses (tur, moong, urad), oilseeds (groundnut, soybean, mustard), fruits (mango, banana, apple), vegetables (tomato, onion, potato), spices (turmeric, chilli, coriander), fibres (cotton, jute), and others. Reports minimum price, maximum price, and modal price for each commodity at each mandi on each trading day. Also provides arrival quantities at some mandis.

Current Format: HTML tables on the Agmarknet website which uses session-based navigation and form submissions. No API. No bulk download. Data accessible only through a clunky web interface that requires selecting state, district, market, commodity, and date range through multiple dropdowns and form submissions. The interface is slow and frequently unavailable.

Update Frequency: Daily. Prices are updated each trading day by mandis that report electronically. Not all mandis report daily and reporting compliance varies significantly across states.

Coverage Period: Historical data available from approximately 2005 onward though completeness varies by mandi and commodity.

Why It Matters: Agriculture is the livelihood of approximately half of India's population. Crop prices directly affect farmer incomes, food prices, inflation, and rural economic health. The government announces Minimum Support Prices for key crops, but actual market prices often diverge significantly from MSP. Access to real-time and historical price data is critical for agricultural policy analysis, farmer advisory services, commodity trading, food security research, and inflation forecasting. Despite this importance, accessing Agmarknet data programmatically is notoriously difficult due to the hostile web interface.

Who Uses This Data: AgriTech startups building farmer advisory and price discovery apps. Commodity traders and analysts tracking price trends. Agricultural economists studying market efficiency and price transmission. Government agricultural departments monitoring MSP compliance. Food security researchers tracking affordability. Journalists investigating food inflation and farmer distress. Farmer producer organizations making selling decisions.

Integration Priority: Fourth source. Extremely high demand from AgriTech developers and agricultural researchers.

---

5. Reserve Bank of India — Economic and Financial Data

Publishing Body: Reserve Bank of India, India's central bank

What Data It Contains: Comprehensive monetary, banking, and economic statistics. Includes money supply data, inflation rates (CPI and WPI), interest rates (repo rate, reverse repo, bank rate), exchange rates, foreign exchange reserves, government securities yields, credit and deposit data for scheduled commercial banks, state-wise banking statistics, industrial production index, balance of payments, external debt, and government finance data. Also publishes the Database on Indian Economy (DBIE) which is a structured data portal with thousands of time series.

Current Format: RBI has one of the better data dissemination practices among Indian government bodies. The DBIE portal provides downloadable Excel and CSV files for many datasets. The RBI website also publishes data in HTML tables, PDFs, and Excel files. However, the DBIE portal is slow, has a complex navigation structure, and bulk download is limited. Many important tables are still in PDF format in the RBI Annual Report and the Handbook of Statistics on the Indian Economy.

Update Frequency: Varies by dataset. Monetary policy rates updated on policy days. Exchange rates daily. Banking statistics monthly and quarterly. Government finance data annually. Key indicators are released on a pre-announced calendar.

Coverage Period: Many time series go back to the 1950s. The DBIE portal covers data from the 1970s onward for most series.

Why It Matters: RBI data is essential for any economic analysis of India. Fintech developers need interest rate and banking data. Economic researchers need monetary and fiscal indicators. Financial journalists need policy rate histories and inflation trends. Investment firms need macroeconomic data for India allocation decisions. Trade economists need balance of payments and exchange rate data. State-level banking statistics are crucial for financial inclusion research.

Who Uses This Data: Fintech developers building lending, investment, and payment products. Economic researchers and analysts at think tanks. Financial journalists covering monetary policy, banking sector health, and economic trends. Investment professionals making India allocation decisions. Central and state government economists doing policy analysis. Students studying macroeconomics and monetary theory with real Indian data.

Integration Priority: Fifth source. High demand from fintech developers and economic researchers. RBI's relatively better data infrastructure makes initial integration somewhat easier.

Tier 2 — Expansion Sources (Build When Core Is Proven)

These sources are integrated after BharatData has a proven pipeline, an established user base, and clear demand signals from Tier 1 users.

---

6. Ministry of Education — Education Statistics

Publishing Body: Multiple bodies under the Ministry of Education including the Department of School Education and Literacy (for UDISE data), the All India Survey on Higher Education (AISHE), and the National Council of Educational Research and Training (NCERT for NAS data)

What Data It Contains: UDISE (Unified District Information System for Education) covers over 1.5 million schools with data on enrollment by grade and gender, dropout rates, teacher counts and qualifications, infrastructure (classrooms, toilets, drinking

water, electricity, computers, internet), mid-day meal coverage, and school management type. AISHE covers all higher education institutions with data on enrollment, faculty, infrastructure, and courses. NAS (National Achievement Survey) provides learning outcome data from sample-based student assessments.

Current Format: UDISE publishes dashboards and downloadable reports in PDF and Excel. State report cards are available. Raw data access is limited. AISHE publishes annual reports in PDF. NAS results are in PDF reports.

Update Frequency: UDISE annual. AISHE annual. NAS periodic.

Coverage Period: UDISE from 2012 onward. AISHE from 2011 onward. NAS various rounds.

Why It Matters: Education is the most debated social sector in India. Every state government claims credit for improving education. Verifying these claims requires accessible education data. Researchers studying education inequality, the impact of mid-day meals, the effect of infrastructure on learning outcomes, and gender gaps in enrollment all need UDISE data. EdTech companies building products for Indian schools need school-level data for market sizing and product design.

Who Uses This Data: Education researchers and policy analysts. EdTech companies building school-focused products. Journalists investigating education quality and access. State and district education planners. International organizations monitoring SDG education targets.

---

7. Ministry of Health — Health Statistics

Publishing Body: Multiple bodies including the Central Bureau of Health Intelligence (CBHI), the National Health Mission (NHM), the National Family Health Survey (NFHS conducted by IIPS Mumbai), and the Integrated Disease Surveillance Programme (IDSP)

What Data It Contains: CBHI publishes the National Health Profile covering hospital infrastructure (primary health centres, community health centres, district hospitals), human resources (doctors, nurses, specialists), disease burden, maternal and child health indicators, and health expenditure. NFHS provides household survey data on fertility, family planning, infant and child mortality, maternal and child health, nutrition, anaemia, HIV awareness, domestic violence, women’s empowerment, and disability. Rural Health Statistics provides facility-level data on health infrastructure in rural areas. IDSP provides weekly disease outbreak reports.

Current Format: CBHI publishes the National Health Profile as a massive PDF. NFHS data is available through a custom data portal and downloadable reports. Rural Health Statistics is in PDF. IDSP weekly reports are in PDF.

Update Frequency: CBHI annual. NFHS every few years. Rural Health Statistics annual. IDSP weekly.

Coverage Period: CBHI data from the 1990s. NFHS from 1992 with four completed rounds. Rural Health Statistics annual from 2005.

Why It Matters: India faces enormous public health challenges. Maternal mortality, child malnutrition, disease outbreaks, shortage of doctors and hospital beds, and healthcare access inequality are all major policy concerns. Clean health data is essential for health policy research, epidemiological studies, healthcare planning, and accountability. The COVID-19 pandemic highlighted how critical accessible health infrastructure data is.

Who Uses This Data: Public health researchers and epidemiologists. Health policy analysts at think tanks. HealthTech companies building healthcare delivery products. Journalists covering healthcare access and disease outbreaks. State health departments doing planning. International health organizations monitoring India programs.

---

8. India Meteorological Department — Weather and Climate Data

Publishing Body: India Meteorological Department, Ministry of Earth Sciences, Government of India

What Data It Contains: Daily weather observations from hundreds of surface observatories across India including temperature (maximum, minimum, mean), rainfall, humidity, wind speed and direction, cloud cover, and sunshine hours. Seasonal and annual rainfall data by state and district. Monsoon tracking data. Cyclone monitoring data. Climate normals and long-term averages. Agricultural meteorological data for crop weather advisories.

Current Format: IMD publishes data through multiple portals with varying levels of accessibility. Historical station data is available through a data portal but requires registration and has limited bulk download capability. Real-time data is available through weather apps and the IMD website. Climatological tables are in PDF format. District-level rainfall data is published in PDF reports.

Update Frequency: Daily observations. Weekly weather summaries. Seasonal forecasts. Annual climate summaries.

Coverage Period: Some station records go back to the 1800s. Comprehensive digital records from the 1950s onward.

Why It Matters: Weather and climate data is fundamental to agriculture, disaster preparedness, urban planning, energy management, and climate research. India's monsoon directly affects the food security and economic wellbeing of hundreds of millions of people. Historical weather data is essential for climate change research, crop yield modeling, flood prediction, and drought monitoring. AgriTech applications need weather data to provide crop advisories. Insurance companies need historical rainfall data for crop insurance products.

Who Uses This Data: Agricultural researchers studying crop-weather relationships. Climate scientists studying Indian monsoon patterns and climate change. AgriTech companies building crop advisory platforms. Insurance companies designing weather-linked products. Disaster management agencies doing flood and drought risk assessment. Urban planners designing climate-resilient infrastructure. Energy companies forecasting demand and supply.

---

9. Ministry of Labour — Employment Data

Publishing Body: Labour Bureau and National Statistical Office, Ministry of Statistics and Programme Implementation (for PLFS), and Ministry of Labour and Employment

What Data It Contains: Periodic Labour Force Survey (PLFS) provides quarterly and annual estimates of labour force participation rate, worker population ratio, unemployment rate, and distribution of workers by industry and occupation, for rural and urban areas, by age group and gender, at state and all-India levels. Also provides data on wages and earnings, conditions of employment, social security coverage, and informal sector employment. Annual Survey of Industries provides factory-level data on employment, wages, output, and capital investment in the organized manufacturing sector. Quarterly Employment Survey covers establishments with 10 or more workers.

Current Format: PLFS publishes quarterly and annual bulletins in PDF format. Unit-level microdata is available for researchers through a request process. Annual Survey of Industries data is available through a data portal with limited bulk download.

Update Frequency: PLFS quarterly bulletins and annual reports. Annual Survey of Industries annual. Quarterly Employment Survey quarterly.

Coverage Period: PLFS from 2017 onward. Earlier employment data from NSSO Employment-Unemployment Rounds (quinquennial from 1972 to 2011).

Why It Matters: Unemployment is one of the most politically charged topics in India. Every government claims credit for job creation and every opposition alleges joblessness. Accurate employment data is essential for verifying these claims. Youth unemployment, female labour force participation, informal sector employment, and rural-urban employment gaps are critical policy issues. Economists, policy makers, and journalists need clean employment data for economic analysis and accountability.

Who Uses This Data: Economists studying labour markets and structural transformation. HR technology companies analyzing skill demand and supply. Policy researchers evaluating employment programs like MGNREGA. Journalists fact-checking government claims about job creation. State planning boards assessing local employment conditions. Students studying labour economics.

---

10. Election Commission of India — Electoral Data

Publishing Body: Election Commission of India

What Data It Contains: Results of all general elections (Lok Sabha) and state assembly elections (Vidhan Sabha) including constituency-wise votes received by each candidate, party affiliation, winner margin, voter turnout, number of electors, and number of votes polled. Also includes data on electoral rolls (number of registered voters by constituency, age group, and gender), election expenditure, and model code of conduct violations.

Current Format: Election results are published on the ECI website in HTML tables. Historical data is available through the ECI statistical reports in PDF. Detailed constituency-level results are downloadable but in formats that require significant cleaning. The ECI has a data portal but it covers limited historical depth.

Update Frequency: After each election. General elections every five years. State elections on varying schedules.

Coverage Period: General election results available from 1951. State election results from various starting years.

Why It Matters: India is the world's largest democracy. Electoral data is fundamental to political science research, election journalism, democratic accountability, and civic engagement. Researchers study voting patterns, incumbency effects, party system dynamics, and democratic representation. Journalists need quick access to historical election data for context during election coverage. Citizens want to understand their representatives' electoral history.

Who Uses This Data: Political scientists studying Indian democracy and electoral behaviour. Election journalists covering campaigns and results. Political parties and campaign strategists analyzing historical patterns. Civic technology organizations building voter information tools. Students studying political science. International observers and researchers studying democratic processes.

---

11. National Judicial Data Grid — Court Statistics

Publishing Body: National Judicial Data Grid, e-Courts Mission Mode Project, Department of Justice, Ministry of Law and Justice

What Data It Contains: Case filing, pendency, and disposal data for district courts and high courts across India. Data is available at the national, state, district, and court-complex level. Includes case type breakdowns (civil, criminal, motor accident, family, labour, consumer), age of pending cases, judge strength versus sanctioned strength, and case clearance rates.

Current Format: The NJDG website provides a dashboard with interactive visualizations but limited downloadable data. Detailed data requires navigating multiple layers of the dashboard. Historical data is not easily accessible. The e-Courts website provides case status search but not statistical data.

Update Frequency: Near real-time for pending case data. Monthly and annual statistical reports.

Coverage Period: NJDG data from approximately 2015 onward. Earlier judicial statistics from the Law Commission and Supreme Court annual reports in PDF.

Why It Matters: India has a massive judicial backlog. Over 40 million cases are pending across the court system. Access to justice is a fundamental right and understanding the scale and patterns of judicial delay is critical for legal reform. Researchers studying justice system efficiency, lawyers assessing likely case timelines, legal tech companies building case management tools, and journalists investigating judicial accountability all need clean judicial statistics.

Who Uses This Data: Legal researchers studying access to justice and judicial efficiency. Legal technology companies building case management and prediction tools. Journalists investigating judicial delays and court performance. Law commissions and judicial reform bodies. Bar associations assessing court workload. Citizens tracking the status of judicial infrastructure.

Tier 3 — Demand-Driven Sources (Build When Users Request)

These sources are integrated based on specific user demand signals from GitHub issues, playground queries, and community feedback. Do not build these proactively. Wait for users to ask for them.

---

12. Ministry of Finance — Union and State Budget Data

What Data It Contains: Union budget expenditure by ministry and department. Revenue receipts by source. Fiscal deficit, revenue deficit, and primary deficit. State budget data covering state-wise expenditure, revenue, and debt. Finance Commission reports on centre-state fiscal transfers.

Who Uses This Data: Economists, fiscal policy researchers, journalists covering budget sessions, development sector analysts tracking social sector spending, and state government planning departments.

---

13. Controller General of Accounts — Government Expenditure

What Data It Contains: Actual government expenditure versus budgeted expenditure by ministry and department. Monthly expenditure tracking. Scheme-wise expenditure for centrally sponsored schemes. State-wise releases and utilization of central funds.

Who Uses This Data: Transparency and accountability researchers, budget watchdog organizations, journalists tracking government spending, and policy analysts evaluating scheme implementation.

---

14. Ministry of Commerce — Trade Data

What Data It Contains: India's merchandise trade data including commodity-wise, country-wise, and port-wise exports and imports. Trade balance data. Special Economic Zone performance data. Foreign Direct Investment inflows by sector and country.

Who Uses This Data: Trade economists, export-import businesses, trade policy researchers, journalists covering economic policy, and investment analysts tracking FDI trends.

---

15. Ministry of Housing and Urban Affairs — Urban Development Data

What Data It Contains: Data from the Smart Cities Mission, Swachh Bharat Mission (urban), PMAY (Pradhan Mantri Awas Yojana) housing scheme, AMRUT (Atal Mission for Rejuvenation and Urban Transformation). City-level data on sanitation, water supply, housing construction, and urban infrastructure.

Who Uses This Data: Urban planners, real estate developers, housing researchers, municipal governance analysts, and journalists covering urbanization.

---

16. Ministry of Road Transport — Vehicle and Accident Data

What Data It Contains: Road accident statistics from the annual Road Accidents in India report published by the Transport Research Wing. State-wise accidents, fatalities, injuries by road type, vehicle type, cause, and time. Vehicle registration data by type and state.

Who Uses This Data: Transport safety researchers, insurance companies, automotive industry analysts, urban mobility planners, and journalists covering road safety.

---

17. Ministry of Women and Child Development — Women and Child Welfare Data

What Data It Contains: Data from Integrated Child Development Services (ICDS) covering anganwadi centres, beneficiaries, supplementary nutrition, immunization. Data on women's helpline calls, shelter homes, working women's hostels. One Stop Centre data for violence against women.

Who Uses This Data: Gender researchers, child welfare organizations, women’s rights advocates, journalists covering gender issues, and state WCD departments.

---

18. Ministry of Rural Development — Rural Development Data

What Data It Contains: MGNREGA (Mahatma Gandhi National Rural Employment Guarantee Act) data including person-days of employment generated, expenditure, number of households benefited, and work completion status. PMAY-Gramin housing data. Socio-Economic and Caste Census data. Rural road connectivity data from PMGSY.

Who Uses This Data: Development economists studying poverty alleviation, rural policy researchers, journalists investigating scheme implementation, and state rural development departments.

---

19. Ministry of Drinking Water and Sanitation — Water and Sanitation Data

What Data It Contains: Jal Jeevan Mission data on functional household tap connections by state, district, and village. Swachh Bharat Mission (Gramin) data on toilet construction, ODF (Open Defecation Free) status, and waste management. Water quality monitoring data.

Who Uses This Data: Public health researchers studying waterborne disease links, water sector companies, WASH (Water, Sanitation, and Hygiene) organizations, journalists covering rural development, and state water departments.

---

20. Forest Survey of India — Forest and Environment Data

What Data It Contains: State of Forest Report published biennially covering forest cover by state and district (very dense, moderately dense, open, scrub), tree cover outside forests, mangrove cover, forest fire data, carbon stock in forests, and biodiversity assessments.

Who Uses This Data: Environmental researchers, climate scientists studying carbon sequestration, conservation organizations, forestry companies, journalists covering deforestation and environmental protection, and state forest departments.

---

21. Central Water Commission — Water Resources Data

What Data It Contains: River flow data from hundreds of gauging stations across India. Reservoir storage levels for major dams. Flood forecasting data. Historical water level and discharge records. Interstate water sharing data.

Who Uses This Data: Hydrologists and water resource engineers, flood risk analysts, dam safety researchers, climate scientists studying precipitation patterns, agriculture researchers studying irrigation, and disaster management agencies.

---

22. Geological Survey of India — Mineral Resource Data

What Data It Contains: Mineral resource inventory for all major minerals mined in India including coal, iron ore, bauxite, manganese, limestone, and rare earths. State-wise mineral production data. Geological mapping data. Groundwater resource assessment.

Who Uses This Data: Mining industry analysts, mineral economists, environmental impact researchers, state mining departments, and energy policy analysts.

---

23. Registrar General of India — Vital Statistics

What Data It Contains: Annual reports on birth registration, death registration, and causes of death through the Medical Certification of Cause of Death system. Infant mortality rate, maternal mortality ratio, fertility rates, and life expectancy estimates through the Sample Registration System.

Who Uses This Data: Demographers studying population dynamics, public health researchers studying mortality patterns, epidemiologists studying cause-of-death trends, and health policy analysts.

Tier 4 — Aspirational Sources (May Require Partnerships)

These sources contain extremely valuable data but may require special access arrangements, government partnerships, or alternative acquisition strategies beyond web scraping.

---

24. Land Records — Digitized Land Ownership Data

What Data It Contains: State-level land record digitization under the Digital India Land Records Modernization Programme (DILRMP). Includes ownership records, survey maps, property registration details, and mutation records. Data availability and digitization status varies enormously across states.

Why It Is Aspirational: Land records are extremely sensitive, vary by state in format and law, and most are behind authentication walls. Integration would likely require state-by-state partnerships.

Who Uses This Data: Real estate developers, property technology companies, legal researchers studying land disputes, agricultural economists studying land tenure, and citizens verifying land ownership.

---

25. Income Tax and GST Statistics

What Data It Contains: Aggregate income tax statistics including number of taxpayers by income bracket, average tax paid, sectoral distribution of taxpayers. GST revenue data by state and tax category. Tax-to-GDP ratio. Tax compliance indicators.

Why It Is Aspirational: Detailed tax microdata is confidential. Only aggregate statistics are publishable. The Central Board of Direct Taxes and Central Board of Indirect Taxes publish some aggregate statistics but in limited detail.

Who Uses This Data: Tax policy researchers, fiscal economists, state finance departments, and economic journalists.

---

26. Aadhaar and Digital Identity Statistics

What Data It Contains: Aadhaar enrollment and authentication statistics. State-wise Aadhaar coverage. Aadhaar-linked benefit transfers (Direct Benefit Transfer data). Digital payment transaction statistics from NPCI (National Payments Corporation of India) including UPI transaction volumes.

Why It Is Aspirational: UIDAI publishes some aggregate Aadhaar statistics. NPCI publishes UPI transaction data. However, detailed microdata is not publicly available for privacy reasons. Integration would focus on published aggregate statistics.

Who Uses This Data: Fintech companies, digital economy researchers, financial inclusion analysts, and identity system researchers.

---

27. Parliament and Legislative Data

What Data It Contains: Rajya Sabha and Lok Sabha proceedings, question hour data, bill tracking (introduced, debated, passed, pending), member attendance and participation data, committee reports, budget session data.

Why It Is Aspirational: Parliament data is technically public through the parliamentary websites but is in very unstructured formats. PRS Legislative Research has done significant work in structuring this data. Integration may involve collaboration with PRS or similar organizations.

Who Uses This Data: Political scientists, legislative researchers, civic technology organizations, democracy watchdogs, journalists covering Parliament, and citizens tracking their representatives.

---

28. RTI Response Datasets

What Data It Contains: Aggregated responses to Right to Information requests filed across government departments. Includes data obtained by journalists and activists that reveals government functioning not available through regular statistical publications.

Why It Is Aspirational: RTI responses are individual documents, not structured datasets. Integration would require curation and structuring of significant RTI-obtained datasets compiled by transparency organizations.

Who Uses This Data: Transparency researchers, anti-corruption organizations, investigative journalists, and governance researchers.

Complete Summary

Priority	Source	Publishing Body	Primary Format	Users
Tier 1-1	Crime Statistics	NCRB	PDF	Journalists, Researchers, Developers, Citizens
Tier 1-2	Census Demographics	Census of India	Excel, PDF, HTML	Everyone (foundation dataset)
Tier 1-3	Air Quality	CPCB	Web Portal, PDF	Environmental journalists, Health researchers, Citizens
Tier 1-4	Agricultural Prices	Agmarknet	HTML (hostile)	AgriTech developers, Agricultural economists, Farmers
Tier 1-5	Economic and Financial	RBI	Excel, PDF, DBIE Portal	Fintech developers, Economists, Financial journalists
Tier 2-6	Education Statistics	Ministry of Education	PDF, Excel, Dashboard	Education researchers, EdTech companies, Journalists
Tier 2-7	Health Statistics	CBHI, NFHS, NHM	PDF, Web Portal	Health researchers, HealthTech companies, Journalists
Tier 2-8	Weather and Climate	IMD	Web Portal, PDF	AgriTech, Climate researchers, Insurance companies
Tier 2-9			PDF	

Priority	Source	Publishing Body	Primary Format	Users
	Employment Data	Labour Bureau, NSO		Economists, Policy researchers, HR Tech companies
Tier 2-10	Electoral Data	Election Commission	HTML, PDF	Political scientists, Election journalists, Civic tech
Tier 2-11	Court Statistics	NJDG	Dashboard	Legal researchers, Legal tech, Judicial reform bodies
Tier 3-12	Budget Data	Ministry of Finance	PDF, Excel	Economists, Fiscal policy researchers, Journalists
Tier 3-13	Government Expenditure	CGA	PDF, HTML	Accountability researchers, Budget watchdogs
Tier 3-14	Trade Data	Ministry of Commerce	Excel, PDF	Trade economists, Export-import businesses
Tier 3-15	Urban Development	MoHUA	Dashboard, PDF	Urban planners, Real estate developers
Tier 3-16	Road Accidents	MoRTH	PDF	Transport researchers, Insurance companies
Tier 3-17	Women and Child Data	MoWCD	PDF, Dashboard	Gender researchers, Women’s rights organizations
Tier 3-18	Rural Development	MoRD	Dashboard, PDF	Development economists, Rural policy researchers
Tier 3-19	Water and Sanitation	MoDWS	Dashboard	Public health researchers,

Priority	Source	Publishing Body	Primary Format	Users
				WASH organizations
Tier 3-20	Forest Cover	FSI	PDF	Environmental researchers, Conservation organizations
Tier 3-21	Water Resources	CWC	Web Portal	Hydrologists, Disaster management agencies
Tier 3-22	Mineral Resources	GSI	PDF	Mining analysts, Energy policy researchers
Tier 3-23	Vital Statistics	RGI	PDF	Demographers, Public health researchers
Tier 4-24	Land Records	State Revenue Depts	Varies by state	PropTech companies, Legal researchers
Tier 4-25	Tax Statistics	CBDT, CBIC	PDF	Tax policy researchers, Fiscal economists
Tier 4-26	Aadhaar and Digital	UIDAI, NPCI	Dashboard	Fintech companies, Digital economy researchers
Tier 4-27	Parliament Data	Lok Sabha, Rajya Sabha	HTML, PDF	Political scientists, Civic tech organizations
Tier 4-28	RTI Datasets	Various via RTI	Unstructured	Transparency researchers, Investigative journalists

Integration Order

Start with NCRB Crime in India. Build the entire pipeline, both SDKs, the API, and the playground around this single source. Get real users. Get feedback.

Add Census demographics second because every other dataset needs population figures for normalization.

Add CPCB air quality third because it has high public interest and serves a different audience segment than crime data, broadening your user base.

Add Agmarknet agricultural prices fourth because it serves the AgriTech developer community which is large and growing in India.

Add RBI economic data fifth because it serves the fintech developer community and economic research community.

After these five Tier 1 sources are stable and serving users, let user demand determine the order of Tier 2 additions. Track GitHub issues, playground query logs, and community feedback. The dataset that users request most frequently is the one you build next.

Tier 3 sources are built only when specific users ask for them. Tier 4 sources are explored when partnerships or alternative access methods become available.

The total number of data sources BharatData can eventually integrate is limited only by the maintenance capacity available. Each source requires ongoing monitoring, parser maintenance, normalizer updates, and quality assurance. Start with five. Grow to ten or twelve in the first year. Reach twenty to twenty-five in the second year. Let the system prove itself at each scale before expanding further.

Stage 1 — Choosing and Understanding Your First Data Source

Before You Do Anything Else

You have already chosen NCRB Crime in India as your first data source based on the selection criteria. That decision is correct and I am not going to reopen it. What you need to do now is not choose again. What you need to do is deeply understand what you are about to work with. You need to touch this data with your own hands. You need to see the mess. You need to feel the pain. You need to know exactly what your pipeline will face before you design it.

This stage has three tasks. Navigate and explore the NCRB ecosystem to understand where all the data lives. Download actual reports and examine them to understand the format challenges. Document everything you find so your future self and your AI coding assistant have a complete reference.

Task 1 — Navigate the NCRB Ecosystem

The NCRB does not publish data in one place. It is spread across multiple sections of their website. You need to visit every relevant section and understand what is available where.

Primary Website

Go to the NCRB main website at https://ncrb.gov.in

This is the homepage of the National Crime Records Bureau. Look at the navigation. Find the publications section. The NCRB publishes several different reports, not just Crime in India. Understanding the full landscape helps you plan for future expansion even though you are starting with one report.

Crime in India Report Page

Go to https://ncrb.gov.in/crime-in-india.html

This is the main page for the Crime in India annual publication. This is your primary data source. On this page you will find links to the most recent reports and archives of older reports. Each year’s report is typically available as a single large PDF or as chapter-wise PDFs.

Look at what years are available. Note the URL pattern for each year’s report. Some years may be on different URL paths because the website has been redesigned multiple times. Note whether

reports are available as full PDFs or chapter-wise downloads. Note whether any Excel or CSV supplementary files are provided alongside the PDFs.

When you visit this page, you may find that the exact URL has changed because NCRB redesigns their website periodically. If the link does not work, go to the main NCRB website and navigate to Publications and then Crime in India. Document the current working URL because it will be different six months from now and your ingester needs to handle this.

Other NCRB Publications

While you are on the NCRB website, note these other publications that you will eventually want to integrate as separate data sources within the crime domain.

Prison Statistics India — published annually with data on prison population, capacity, occupancy, undertrial prisoners, and prison infrastructure by state. Available at the NCRB publications section.

Accidental Deaths and Suicides in India — published annually with data on accidental deaths by cause (road accidents, drowning, fire, poisoning) and suicides by cause, age, gender, and profession. Also in the NCRB publications section.

These are separate reports but they use similar formats and similar geographic conventions as Crime in India. Your parser built for Crime in India will be partially reusable for these reports. Note their existence but do not download or work on them now. They are future expansion sources within the NCRB domain.

Data.gov.in NCRB Datasets

Go to https://data.gov.in and search for “NCRB” or “Crime in India”

The Open Government Data Platform sometimes has NCRB data in CSV or Excel format uploaded separately from the NCRB website. These are often subsets of the full Crime in India report but in cleaner formats. Check what is available here. If there are clean CSV files for specific tables, note them. They could serve as supplementary sources or as validation datasets to cross-check your PDF parser output.

Note which datasets on data.gov.in are actually current and which are years out of date. Many datasets on data.gov.in were uploaded once and never updated. Check the “last updated” date on each dataset.

Task 2 — Download and Examine Actual Reports

This is the most important part of Stage 1. You are going to download real NCRB reports and examine them in detail to understand exactly what your parser will face.

What to Download

Download the Crime in India report for at least three different years. I recommend 2021, 2022, and 2023 if available. If 2023 is not yet published when you do this, download 2020, 2021, and 2022. You need multiple years because the format changes between years and your parser must handle these variations.

For each year, download both the full PDF if available and the chapter-wise PDFs. Sometimes the chapter-wise PDFs are easier to parse because they are smaller and more focused.

If any supplementary Excel files are provided for that year, download those too. They serve as ground truth to validate your parser against.

Create a folder on your machine called ncrb-raw-data with subfolders for each year. Store everything you download with clear names including the year and chapter or volume identifier.

How to Examine the PDFs

Open the most recent Crime in India PDF in a PDF viewer. Not a web browser. A proper PDF viewer like Adobe Acrobat Reader or Okular or Preview on Mac. You need to see the exact rendering of tables including merged cells and multi-row headers.

Go through the report slowly. Do not skim. You are building a mental model of the document structure that will inform every parser decision you make.

What to Look for and Document

Open a new document or text file called ncrb-examination-notes.md. As you go through the PDF, document everything listed below. This document becomes part of your project knowledge base and feeds directly into your Data Source Registry document, your parser design, and your first blog post.

Document Structure. How is the report organized? How many volumes are there? What does each volume cover? How are chapters organized within volumes? Is there a table of contents? Is the table of contents accurate in terms of page numbers? Are there appendices?

Write down the volume and chapter structure. For example Volume 1 covers IPC crimes with chapters on murder, kidnapping, robbery, theft, and so on. Volume 2 covers SLL crimes. Volume 3 covers crimes against specific groups.

Table Identification. How are tables numbered? Is there a consistent numbering scheme like Table 1.1, Table 1.2? Do table numbers reset in each chapter or are they sequential across the entire report? Is there a list of tables anywhere in the report?

Go to five different tables spread across different chapters. For each table, write down the table number, the page number, what data the table contains, how many rows and columns it has, and what the column headers look like.

Header Complexity. This is the most critical thing to examine. Open any table and look at the column headers carefully.

Are the headers a single row or do they span multiple rows? In most NCRB tables, headers span two or three rows with merged cells. For example, the top row might say “Kidnapping and Abduction” spanning three columns, and the second row underneath might say “Of Women”, “Of Children”, “Total” as three separate columns under that merged header.

Document at least five examples of multi-row headers with the exact text in each cell. Draw them out in your notes so you can see the merge pattern. This is what your parser must correctly flatten into single-row column names like “kidnapping_of_women”, “kidnapping_of_children”, “kidnapping_total”.

Cross-Page Tables. Find a table that spans more than one page. This is very common in NCRB reports because tables listing data for all 36 states and union territories are too long for a single page.

Look at how the table continues on the next page. Does the header repeat on the second page? In most NCRB reports it does not. The table just continues with data rows. There is usually a “(Contd.)” or “(Continued)” notation. Document exactly what the continuation looks like including whether there is any text indicating continuation and whether the header repeats or not.

Footnotes. Find tables that have footnotes. Look at how footnotes are marked in the data cells. Common markers are asterisk, hash, at sign, and superscript numbers. Document the exact characters used.

Look at where the footnote text appears. Is it at the bottom of the table? At the bottom of the page? At the end of the chapter? Document examples.

The critical question is whether footnote markers appear inside numeric data cells. If a cell contains “1,234*” your parser must extract 1234 as the value and store the asterisk as a footnote reference. Document every example you find of footnotes inside data cells.

Number Formatting. Look at how numbers are formatted in the tables. Indian notation uses commas differently from international notation. In Indian notation, one lakh twenty-three thousand four hundred fifty-six is written as 1,23,456 with a comma after the thousands position and then every two digits. In international notation, the same number is 123,456 with commas every three digits.

Check whether NCRB uses Indian notation, international notation, or a mix. Check whether decimal numbers appear and how they are formatted. Check whether any cells contain text like “2.5 lakh” or “1.3 crore” instead of raw numbers. Document examples of each formatting style you find.

State and UT Names. Look at how states and union territories are listed in the tables. Write down the exact spelling used for every state and UT as it appears in the tables. Note any abbreviations. Note whether “Andhra Pradesh” includes a note about Telangana for pre-2014 data. Note how union territories are distinguished from states if they are. Note whether “Total” and “Grand Total” rows are labeled and where they appear.

Check whether the state listing order is consistent across tables. Some tables list states alphabetically. Some list them geographically (North, South, East, West). Some list states first and UTs separately at the end. Document the pattern for at least five tables.

Data Categories. What crime categories are covered? Write down the major categories from the table of contents or from scanning the tables. IPC crimes include murder, attempt to murder, culpable homicide, kidnapping and abduction, robbery, dacoity, burglary, theft, riots, criminal breach of trust, cheating, counterfeiting, arson, dowry deaths, assault on women, cruelty by husband, rape, and many more. SLL crimes include NDPS Act, Arms Act, Gambling Act, Excise Act, and many more.

Note whether the category names are consistent across years. NCRB occasionally renames or reclassifies crime categories. Document any differences you notice between the years you downloaded.

Year-to-Year Differences. Compare the same table across your two or three downloaded years. Open Table 1.1 or whatever the first major state-wise crime table is from each year side by side. Note any differences in table structure, column names, state listing order, number formatting, or header layout. These differences are what make multi-year parsing challenging.

Pay special attention to how Telangana appears. In reports before 2014, Telangana does not exist and all data is under Andhra Pradesh. From 2014 onward, Telangana and Andhra Pradesh appear as separate entries. Some transitional year reports may have notes or special handling. Document exactly what you see.

Page Layout and Orientation. Are all pages portrait or are some tables on landscape-oriented pages? Landscape tables are common for wide tables with many columns. Your PDF parser handles portrait and landscape differently. Document which tables use landscape layout.

Are there any pages that contain multiple tables? Are there pages that contain both text paragraphs and tables? Are there any charts or graphs embedded in pages that also contain tables? Document these because your parser needs to distinguish tables from other page content.

Supplementary Data. Beyond the main tables, look for any annexures, appendices, or supplementary tables at the end of chapters or at the end of the report. These sometimes contain more detailed breakdowns (district-level data, monthly data) that are extremely valuable but in even more challenging formats.

Task 3 — Hands-On Parsing Attempt

After you have examined the reports and documented what you found, attempt to extract data from one table programmatically. This is not about building your production parser. This is about experiencing the difficulty firsthand so you calibrate your expectations correctly.

Set Up a Quick Test Environment

Create a simple Python script or Jupyter notebook. Install pdfplumber which is the most capable Python library for extracting tables from PDFs. You will also want tabula-py as a comparison and camelot-py as another option to evaluate.

Install them with pip install pdfplumber tabula-py camelot-py ghostscript. Note that camelot requires ghostscript to be installed on your system separately.

Try Extracting One Table

Pick a relatively simple table from the NCRB report. Something like the state-wise total IPC crimes table which typically has a manageable number of columns and straightforward headers. Note the page number.

Write a script that opens the PDF, goes to that page, and attempts to extract the table using pdfplumber. Look at what comes out. The columns will likely be misaligned. Some header cells will be merged incorrectly. Some numbers will include footnote markers. Some rows will be split across two lines.

Try the same extraction with tabula-py. Compare the results. Try with camelot-py. Compare all three. Note which library handles each challenge best.

Document your results in your project journal. For each library, note what it got right, what it got wrong, and what it could not handle at all. Take screenshots of the original PDF table next to the extracted output.

Try a Hard Table

Now pick a complex table. One with multi-row merged headers, cross-page continuation, and footnotes in data cells. Attempt the same extraction. Watch it fail more dramatically. Document exactly how and why it fails.

This failure documentation is extremely valuable. It tells you precisely what your production parser must handle. It also becomes compelling content for your blog post. Showing a complex government PDF table next to the garbled output from a standard parsing library is a powerful demonstration of the problem BharatData solves.

Calculate Baseline Accuracy

For the simple table you extracted, manually check 20 cells against the original PDF. How many did the parser get right? How many were wrong? What was the error pattern? Were the errors random or systematic? This gives you a baseline accuracy number that your production parser must beat.

Document this accuracy number. It is your starting point. Your production parser must achieve 95 percent or higher on the same check.

Task 4 — Document Your Choice Formally

After completing the exploration and hands-on parsing, write the formal data source document. This is the docs/data-sources/NCRB_CRIME.md file from your documentation plan. It should contain everything you learned during this stage.

The source name, URL, publishing body, format, update frequency, and coverage period. The report structure including volumes, chapters, and table numbering. The specific parsing challenges you identified with examples from your examination. The tables you plan to extract in version one with their table numbers, descriptions, and complexity ratings. The known data quality issues in the source itself. The year-to-year format differences you discovered. The state and UT naming conventions used. The number formatting conventions used. Your baseline parsing accuracy from the hands-on attempt.

This document becomes the authoritative reference that your parser is built against. When your AI coding assistant generates parser code, it reads this document to understand the specific challenges it must handle.

Task 5 — Confirm Your Decision

After all of this exploration, ask yourself one question. Is NCRB Crime in India still the right first source?

If your hands-on exploration confirmed that the data is painful to access, broadly relevant, and technically challenging enough to demonstrate real engineering skill, the answer is yes. Commit and move forward.

If your exploration revealed something unexpected that makes a different source a better choice, such as discovering that NCRB has quietly made all their data available through a clean API that you did not know about which would make it too easy to demonstrate engineering skill, then you have learned something valuable and should adjust your choice accordingly.

In practice, NCRB Crime in India will almost certainly remain the right choice. The hostile PDF format, the broad audience, and the high citation frequency make it the ideal first source for BharatData.

What You Should Have After Completing This Stage

When this stage is complete, you should have these specific artifacts.

A folder on your machine containing downloaded NCRB Crime in India reports for at least three years with both full and chapter-wise PDFs.

A detailed examination notes document covering document structure, header complexity, cross-page tables, footnotes, number formatting, state names, data categories, and year-to-year differences with specific examples from the actual reports.

Screenshots of at least ten different tables showing the variety of formats and challenges across the report.

A test script or notebook showing your hands-on parsing attempt with results from at least two parsing libraries on at least two tables of different complexity levels.

A baseline accuracy measurement from manually checking 20 cells of parser output against the original PDF.

A formal data source document (NCRB_CRIME.md) containing all findings organized for reference during parser development.

A confirmed decision to proceed with NCRB Crime in India as your first data source.

An entry in your project journal documenting what you did, what surprised you, and what you learned.

What Comes After This Stage

With this stage complete, you have the deep firsthand understanding of your data source that separates a thoughtful engineering effort from a naive one. Every parser decision, every normalizer rule, every confidence scoring threshold, every test case you write from this point forward is grounded in real knowledge of real data rather than assumptions about hypothetical challenges.

The next thing you build is the knowledge foundation, the documents that guide your AI coding assistant and define your quality standards. But those documents will be dramatically better because you have actually seen the data they must handle.

Do not rush through this stage. The two or three days you spend here save weeks of wrong assumptions and misguided design decisions later. Explore thoroughly. Document obsessively. Touch the data with your own hands.

Then move forward with confidence because you know exactly what you are building and exactly why it is hard.

Stage 1 — Complete Guide With the NCRB Additional Tables Discovery

---

What You Just Found Changes Everything

This is a genuinely important discovery. The page you found at https://www.ncrb.gov.in/crime-in-india-additional-table?year=2023&category= contains downloadable structured files, not PDFs. Look at the file sizes. The States/UTs files are 11 to 41 KB. The District-wise files are 149 to 693 KB. These are Excel or CSV files, not 400-page PDF reports.

This means NCRB publishes data in two forms. The main Crime in India report is the notorious hostile PDF with 500 tables across 400 pages that we discussed extensively. But alongside that PDF report, NCRB also publishes these “Additional Tables” as separate downloadable structured files covering the same data in a much more accessible format.

This does not eliminate the PDF parsing challenge entirely because the main PDF report contains analysis, context, and some tables not available in the additional tables. But it dramatically changes your starting strategy. Instead of beginning with the hardest possible engineering challenge of parsing hostile PDFs, you can begin with these structured files, build a working product with real data much faster, and then add the PDF parser later to fill gaps.

This is the smart engineering decision. Ship something useful quickly using the easier data. Then tackle the hard PDF parsing to demonstrate engineering depth and fill data gaps. Your users do not care whether you parsed a PDF or read an Excel file. They care whether they can query clean Indian crime data with one function call.

Which Tables to Pick — The Complete Selection

You are looking at roughly 200 files across all the sections on that page. You do not need all of them for Stage 1. You need a strategic subset that gives you the broadest coverage with the least complexity. Here is exactly what to pick, organized by priority.

Priority 1 — States/UTs Section (Pick These First)

The States/UTs section at the bottom of the page contains the most universally useful data. State-level data is what journalists cite, what researchers analyze, what developers query, and what students use in projects. This is your foundation.

From the States/UTs section, download these specific files for 2023.

File 1: State/UT-wise Cases Registered in Police Stations and Cr.PC (156_3) under IPC Crimes During 2023 — 11.96 KB

This is your single most important file. It contains the total IPC crime figures for every state and union territory. When a journalist asks “how much crime does Maharashtra have” this is the data that answers it.

File 2: State/UT-wise Cases Registered In Police Stations and Cr.PC (156_3) under SLL Crimes During 2023 — 11.84 KB

This is the companion to File 1 covering Special and Local Laws crimes like NDPS Act, Arms Act, Gambling Act. Together with File 1, you have comprehensive crime coverage.

File 3: State-UT-wise Cases Registered 156_3 under Crimes against Women during 2023 — 11.99 KB

Crimes against women is one of the most frequently cited and most politically significant crime categories in India. Journalists, researchers, and activists query this data constantly.

File 4: State-UT-wise Cases Registered 156_3 under Crimes against Children during 2023 — 11.91 KB

Child safety data is similarly high-demand for the same audiences.

File 5: State-UT-wise Cases Registered 156_3 under Crimes against SCs during 2023 — 11.75 KB

Caste-based crime data. Critical for social justice research and reporting.

File 6: State-UT-wise Cases Registered 156_3 under Crimes against STs during 2023 — 11.75 KB

Tribal community crime data. Same importance as SC data.

File 7: State-UT-wise Cases Registered 156_3 under Crimes against Senior Citizen during 2023 — 11.87 KB

Senior citizen crime data. Growing importance as India’s population ages.

File 8: Crimehead-wise Cases Registered 156_3 under IPC and SLL during 2023 — 25.47 KB

This is the crime category breakdown. It tells you how many murders, how many thefts, how many cyber crimes across the country. Essential for category-wise analysis.

File 9: State-UT-wise Persons Disposal under IPC Crimes during 2023 — 41.11 KB

This covers the criminal justice pipeline. How many people were arrested, chargesheeted, convicted, acquitted. Essential for understanding police and court efficiency.

File 10: State-UT-wise Persons Disposal under Cyber Crimes during 2023 — 32.77 KB

Cyber crime disposal data. High demand topic because cyber crime is growing rapidly.

File 11: State-UT-wise Property Stolen and Recovered during 2023 — 24.46 KB

Property crime data including the value of stolen and recovered property. Useful for economic crime analysis.

File 12: State-UT-wise Seizures under NDPS Act during 2023 — 28.08 KB

Drug seizure data by state. High interest for drug policy research and journalism.

That is 12 files from the States/UTs section. Total download size is approximately 250 KB. These 12 files give you state-level crime data covering every major crime category, every vulnerable group, and the criminal justice disposal pipeline. This is enough to build a genuinely useful first version of BharatData.

Priority 2 — District-Wise Reports (Pick These Second)

The District Wise Reports section contains district-level breakdowns. This is extremely high-value data because district-level analysis is what researchers and developers need most and what is hardest to access.

Download all 10 district-wise files for 2023.

File 1: Districtwise IPC Crimes_2023 — 693.21 KB

File 2: Districtwise SLL Crimes_2023 — 416.87 KB

File 3: Districtwise Crime against Women_2023 — 261.22 KB

File 4: Districtwise Crime against Children_2023 — 323.47 KB

File 5: Districtwise Crime against SCs_2023 — 225.97 KB

File 6: Districtwise Crime against STs_2023 — 210.37 KB

File 7: Districtwise IPC Crime by Juveniles_2023 — 531.92 KB

File 8: Districtwise SLL Crime by Juveniles_2023 — 338.26 KB

File 9: Districtwise Cyber Crimes_2023 — 227.82 KB

File 10: Districtwise Missing Persons_2023 — 149.16 KB

These 10 files are larger because they contain rows for every district in India, roughly 780 districts each. Total download is approximately 3.4 MB. This district-level data is what will make

BharatData genuinely valuable to researchers and developers because district-level crime data is the most painful to compile manually from the PDF reports.

Priority 3 — Metropolitan Cities Summary (Pick These Third)

The Metropolitan Cities section covers 34 major Indian cities. This is useful for urban analysis and for journalists who want to compare cities.

Download these 3 summary files for 2023.

File 1: 1B.1 IPC Crimes (City-wise) - 2021-2023 — 120.22 KB

File 2: 1B.2 SLL Crimes (City-wise) - 2021-2023 — 120.26 KB

File 3: 1B.3 IPC & SLL Crimes (City-wise) - 2021-2023 — 120.67 KB

These three files give you city-level crime totals for three years. Note that these files already contain multi-year data (2021-2023) which is excellent because it enables trend analysis without needing to download separate files for each year.

What to Skip for Now

Skip the detailed category-wise city files like 1B.4 and 1B.5. Skip the detailed disposal files for each crime category at the city level. Skip the detailed victim breakdowns by age and gender for cities. These contain valuable data but they add complexity without adding proportional user value for your first version. You can add them later when users ask for more detail.

Skip the Mega City section for now. It overlaps significantly with the Metropolitan Cities section and the States/UTs section. Add it later.

Skip the detailed disposal files for specific crime categories at the state level (Files 19-27 in States/UTs like Crime against Women disposal, Crime against Children disposal). You already have the aggregate disposal data from File 9 above. The category-specific disposal data can be added later.

Multi-Year Coverage

The critical next step is to download the same files for previous years. Go to the same page but change the year parameter.

For 2022: https://www.ncrb.gov.in/crime-in-india-additional-table?year=2022&category=

For 2021: https://www.ncrb.gov.in/crime-in-india-additional-table?year=2021&category=

Download the same Priority 1 and Priority 2 files for 2022 and 2021. This gives you three years of data which enables trend analysis, the feature that journalists and researchers value most.

If files for 2020 and 2019 are available, download those too. Five years of data is ideal for trend analysis. But three years is your minimum.

Step-by-Step Guide to Complete Stage 1

Step 1 — Set Up Your Working Directory

Create this folder structure on your machine before downloading anything.

```
bharatdata-raw/
├── ncrb-crime-in-india/
│   ├── 2023/
│   │   ├── states-uts/
│   │   ├── districts/
│   │   ├── metro-cities/
│   │   └── pdf-report/
│   ├── 2022/
│   │   ├── states-uts/
│   │   ├── districts/
│   │   ├── metro-cities/
│   │   └── pdf-report/
│   ├── 2021/
│   │   ├── states-uts/
│   │   ├── districts/
│   │   ├── metro-cities/
│   │   └── pdf-report/
│   └── examination-notes/
│       ├── file-format-notes.md
│       ├── column-header-inventory.md
│       ├── state-name-variations.md
│       ├── district-name-inventory.md
```

```
│        ├── data-quality-issues.md
│        └── screenshots/
```

This structure keeps raw data organized by year and category. The examination-notes folder is where you document everything you discover during your exploration.

Step 2 — Download All Selected Files

Go to the NCRB additional tables page for 2023 at https://www.ncrb.gov.in/crime-in-india-additional-table?year=2023&category=

Download each of the 12 States/UTs files listed in Priority 1 above. Save them in the 2023/states-uts/ folder. Keep the original filenames exactly as NCRB names them. Do not rename them yet. You want to preserve the original naming for documentation.

Download each of the 10 District-wise files listed in Priority 2. Save them in the 2023/districts/ folder.

Download the 3 Metropolitan Cities summary files from Priority 3. Save them in the 2023/metro-cities/ folder.

Repeat for 2022 and 2021 by changing the year in the URL.

Also download the main Crime in India PDF report for 2023 from the main NCRB publications page at https://ncrb.gov.in/crime-in-india.html. Save it in the 2023/pdf-report/ folder. You will not parse this PDF in version one but you need it as a reference to verify your structured file data and to understand what additional data the PDF contains that the structured files do not.

After downloading, verify your file counts. You should have approximately 25 files per year, totaling roughly 75 files across three years plus the PDF reports.

Step 3 — Examine the File Formats

This is the critical exploration step. Open each type of file and document what you find.

Open the first States/UTs file (State/UT-wise IPC Crimes 2023) in a spreadsheet application like Excel, Google Sheets, or LibreOffice Calc. Do not open it in a text editor yet. Look at it as a spreadsheet user would see it.

Document the following in your file-format-notes.md.

File type. What is the actual file extension? Is it .xls, .xlsx, .csv, or something else? The NCRB website does not always make the format clear from the download link. Open the file properties to check.

Header structure. How many rows do the headers occupy? Is it a single header row or are there multiple header rows with merged cells? Are there title rows above the actual data headers? Sometimes NCRB files have a title like “State/UT-wise IPC Crimes During 2023” in the first row, then blank rows, then the actual column headers, then the data. Document the exact row numbers where the title is, where the headers are, and where the data starts.

Column names. Write down every column name exactly as it appears in the file. Note any abbreviations. Note whether column names are identical across the States/UTs files or whether they vary by crime category. This is crucial for your normalizer because you need to map these column names to your canonical schema.

State and UT names. Write down every state and UT name exactly as it appears in the data rows. Compare the spellings across multiple files. Does the same state appear as “Andhra Pradesh” in one file and “ANDHRA PRADESH” in another? Is “D & N Haveli” or “Dadra & Nagar Haveli” or “D&NH” used? Write down every variation you find.

Create a file called state-name-variations.md listing every unique state/UT spelling you encounter across all files. This becomes the seed for your canonical-mappings.json file later.

Data types. Are the numbers stored as actual numbers or as text strings? Sometimes Excel files from government sources store numbers as text which causes problems when you try to do arithmetic on them. Check by clicking on a numeric cell and looking at the cell format. Are there any cells that contain text mixed with numbers? Are there any cells with special characters or symbols?

Null and missing values. How are missing values represented? Check every file for cells that are empty, contain a dash, contain “NA”, contain “N/A”, contain a zero that might actually mean missing, or contain any other placeholder. Document every representation you find.

Summary rows. Look for rows labeled “Total”, “Grand Total”, “All India”, or similar. Where do they appear? At the bottom of the table? Interspersed with data? Are they formatted differently from data rows (bold, different color)?

Footnotes. Do any cells contain footnote markers? Do any files have footnote text at the bottom? Are there any annotation rows between data rows?

Year columns. Some files contain multi-year data (like the Metro Cities files that show 2021-2023). How are the years organized? Are they separate columns? Separate sheets?

Now open one of the District-wise files. These are larger and potentially more complex. Document the same observations. Pay special attention to district names because this is where the geographic identity crisis is most acute. There are roughly 780 districts and the spelling variations will be extensive.

Create a file called district-name-inventory.md listing every unique district name you encounter in the district-wise files along with its parent state. This is a massive but critical exercise. You do not need to complete it exhaustively right now but you need to examine enough districts to understand the scope of the naming challenge.

Now open the PDF report. Compare the data in the PDF tables with the data in the corresponding structured files. Pick one table, say state-wise IPC crimes, and verify that the numbers in the structured file match the numbers in the PDF table. If they match, the structured files are reliable representations of the PDF data. If they do not match, document the discrepancies because that affects which source you trust.

Step 4 — Document Data Quality Issues

As you examine the files, you will encounter issues. Document every single one in your data-quality-issues.md file. Here are the types of issues to look for.

Duplicate entries. Does any state appear twice in the same file? This sounds unlikely but government data sometimes contains duplicate rows.

Impossible values. Are there any negative numbers where only positive numbers make sense? Are there any crime counts that seem impossibly high or impossibly low for a particular state? You need domain knowledge here. If a small UT like Lakshadweep shows 50,000 murders, that is obviously wrong. If Maharashtra shows 0 cyber crimes, that is also obviously wrong.

Inconsistencies across files. Take the total IPC crimes for Maharashtra from the States/UTs IPC file. Now look at the district-wise IPC file and sum up all Maharashtra districts. Do the numbers match? If they do not, document the discrepancy. This kind of cross-file validation is how you catch data quality issues.

Category mismatches. Look at the crime categories in the Crimehead-wise files. Are the category names consistent across files? Does “Cyber Crime” in one file become “Cyber Crimes” (plural) in another? Does “Kidnapping & Abduction” in one file become “Kidnapping and Abduction” (ampersand versus word) in another? Document every naming inconsistency.

Year inconsistencies. If you downloaded files for multiple years, check whether the same state/UT names appear in all years. Telangana should not appear before 2014. Ladakh as a separate UT should not appear before 2019. If they do appear in years where they should not exist, that is a data issue to document.

Step 5 — Create Your Priority Table Inventory

After examining all the files, create a definitive list of which tables you will include in BharatData version one. This is not all the tables you downloaded. This is the subset you will actually parse, normalize, and serve through your SDK.

Your version one table inventory should look like this.

Core State-Level Tables (serve first):

State/UT-wise IPC Crimes — total crime counts by state. Years: 2021, 2022, 2023.

State/UT-wise SLL Crimes — special laws crime counts by state. Years: 2021, 2022, 2023.

State/UT-wise Crimes against Women — gender-based violence by state. Years: 2021, 2022, 2023.

State/UT-wise Crimes against Children — child-related crimes by state. Years: 2021, 2022, 2023.

State/UT-wise Crimes against SCs — caste-based crimes by state. Years: 2021, 2022, 2023.

State/UT-wise Crimes against STs — tribal community crimes by state. Years: 2021, 2022, 2023.

State/UT-wise Cyber Crimes — digital crimes by state. Years: 2021, 2022, 2023.

Crimehead-wise IPC and SLL — crime category breakdown nationally. Year: 2023.

Core District-Level Tables (serve second):

Districtwise IPC Crimes. Year: 2023.

Districtwise Crime against Women. Year: 2023.

Districtwise Cyber Crimes. Year: 2023.

City-Level Tables (serve third):

IPC Crimes City-wise 2021-2023.

IPC and SLL Crimes City-wise 2021-2023.

This gives you state-level data for 7 crime categories across 3 years, district-level data for 3 key categories for 1 year, and city-level summary data for 3 years. That is a comprehensive and genuinely useful dataset for your first version.

Step 6 — Map the Schema

For each table in your inventory, define the canonical schema that BharatData will use. This means deciding what columns your database will have and how the raw file columns map to them.

Open the State/UT-wise IPC Crimes file. Write down every column header exactly as it appears. Then next to each one, write the canonical BharatData field name you want to use. For example.

The raw column “State/UT” becomes your canonical field “state” (string, will be normalized through canonical-mappings.json).

The raw column that contains the year becomes “year” (integer).

The raw column for total cases becomes “total_cases” (integer).

If there are columns for specific sub-categories within IPC, map each one to a descriptive canonical name using snake_case.

Do this mapping for every table type. Keep the mappings consistent. If the “state” column is called “State/UT” in one file and “States/UTs” in another and “STATE/UT” in a third, your mapping should handle all three variations mapping to the single canonical “state” field.

Write all these mappings in a new file called docs/data-schema/FIELD_MAPPINGS.md. This document becomes the reference for your parser code.

Step 7 — Build the Canonical State Mappings Seed

From your state-name-variations.md notes, create the initial version of your data/canonical-mappings.json file. This does not need to be complete yet. It needs to cover every state and UT that appears in the files you downloaded.

For each state, list the canonical name, the standard code, and every variation you found in the data files. For example, if you found that Andhra Pradesh appears as “Andhra Pradesh”, “ANDHRA PRADESH”, and “A.P.” across different files or years, all three go into the variations list.

Pay special attention to union territories which have particularly inconsistent naming. “Dadra and Nagar Haveli and Daman and Diu” was merged from two separate UTs in 2020. Check how this appears across different years. “Delhi” appears variously as “Delhi”, “Delhi UT”, “NCT of Delhi”, and “Delhi (UT)”. “Jammu & Kashmir” appears with an ampersand, with “and”, and sometimes as “J&K”. Document all of these.

For district-level mappings, you will build these incrementally as you work with the district files. Start by examining the district names for two or three states in the district-wise files and noting variations.

Step 8 — Test Automated Reading

Now write a simple script to verify that you can read these files programmatically. This is not your production parser. This is a validation that the files are readable and that your schema mapping is correct.

Open a Python script or Jupyter notebook. Use pandas to read one of the States/UTs Excel files. Check what pandas produces. Does it read the headers correctly? Does it skip the title rows? Are the numbers parsed as numbers or as strings? Are there any encoding issues with state names?

Try reading all 12 States/UTs files for 2023. Check that every file produces a clean DataFrame with correct column types. Note any file that requires special handling like skipping header rows or specifying encoding.

Try reading one district-wise file. These are larger and may have different formatting challenges. Check that all 780-plus district rows parse correctly. Check that state names and district names are readable.

Calculate some basic statistics from the parsed data. Total IPC crimes across all states. Top 5 states by total crime. These sanity checks verify that the data makes sense numerically.

Compare one or two numbers from your parsed output against the original file opened in a spreadsheet. They must match exactly. If they do not, your parsing has an issue that must be resolved before proceeding.

Document the results of this automated reading test. Which files parsed cleanly? Which required special handling? What pandas parameters were needed (header row, skip rows, encoding)? This documentation feeds directly into your production parser design.

Step 9 — Assess What the Structured Files Do Not Cover

Open the main Crime in India PDF report. Go through the table of contents. For each major table or section in the PDF, check whether the same data is available in the structured files you downloaded.

Make a list of data that exists in the PDF but not in the structured files. This might include detailed victim profiles (age, gender, relationship to offender), motive analysis for specific crimes, temporal patterns (month-wise crime distribution), and detailed disposal breakdowns.

This list defines the scope of the PDF parsing challenge that comes later. For version one, you are building with the structured files. But knowing what the PDF contains that the structured files do not helps you plan future expansion and helps you answer user questions about why certain data is not yet available.

Step 10 — Write Your Data Source Document

With all exploration complete, write or update your docs/data-sources/NCRB_CRIME.md document. This is the formal reference document that your entire pipeline is built against.

The document should now reflect your actual findings, not assumptions. It should cover the two data access methods for NCRB data. The main PDF report with its challenges and the Additional Tables page with its structured downloadable files. It should list every file you are including in version one with the exact filename, file size, coverage period, geographic level, and column mapping. It should document every data quality issue you

found. It should document every state name variation and district name variation you discovered. It should document the URL patterns for accessing different years of data.

Step 11 — Confirm Your Decision and Document It

Write a brief confirmation in your project journal. State that you have examined NCRB Crime in India data in detail. State that you discovered the Additional Tables page with structured downloadable files which significantly reduces the initial technical barrier. State that you are proceeding with NCRB as your first source, starting with the structured files and adding PDF parsing later.

Note the specific files you are including in version one. Note the schema mappings you defined. Note the data quality issues you discovered. Note the multi-year coverage you have secured.

What You Should Have After Completing All Steps

When Stage 1 is fully complete, you should have these specific artifacts on your machine and in your repository.

Downloaded data files. Approximately 25 structured files per year for 2021, 2022, and 2023, organized in your folder structure. Plus the PDF reports for reference.

file-format-notes.md. Detailed observations about file types, header structures, data types, null representations, summary row patterns, and footnote conventions across all file types.

column-header-inventory.md. Every column header from every file type, with your canonical field name mapping next to each one.

state-name-variations.md. Every unique state and UT spelling found across all files, organized by canonical state name.

district-name-inventory.md. Initial inventory of district names from at least three states in the district-wise files, with variations noted.

data-quality-issues.md. Every data quality issue discovered during examination including duplicates, impossible values, cross-file inconsistencies, and naming mismatches.

canonical-mappings.json (initial version). The seed file mapping all state/UT name variations to canonical forms, ready for the normalizer.

FIELD_MAPPINGS.md. The complete column-to-canonical-field mapping for every table type in your version one inventory.

Test script or notebook. A working Python script that reads the structured files with pandas and produces clean DataFrames with correct types.

Updated NCRB_CRIME.md. The formal data source document updated with all actual findings from your examination.

Project journal entry. A record of what you did, what you discovered, what surprised you, and what you decided.

Why This Approach Is Better Than Starting With PDFs

Starting with the structured Additional Tables files instead of the hostile PDF gives you three critical advantages.

First, you ship a working product much faster. Parsing Excel files with pandas takes days, not weeks. You can have clean data in your database and a working SDK within a couple of weeks instead of spending a month just on PDF parsing.

Second, your data quality is higher from the start. Structured files have fewer parsing errors than PDF extraction. Your confidence scores will be higher. Your users will trust the data more. You start with a reputation for accuracy rather than starting with a parser that gets 90 percent of cells right and 10 percent wrong.

Third, you still get to demonstrate the hard engineering later. When you add PDF parsing to extract data that is not available in the structured files, you are doing it to expand coverage, not to build the basic product. The PDF parsing becomes a feature addition, not a foundation risk. If the PDF parser takes longer than expected, your product still works because it is built on the structured files.

The blog post you write about this discovery is actually more interesting than a pure PDF-parsing story. The title becomes something like “NCRB Publishes Crime Data in Two Formats and Almost Nobody Knows About the Easy One.” That is a story about data discovery and government data literacy, not just engineering.

It is a story that journalists and researchers will find directly useful because you are telling them about a data access method they may not have known existed.

What Comes Next After Stage 1

With Stage 1 complete, you move into building the knowledge foundation documents (your PRD, quality standards, SDK design spec, pipeline spec) and then building the actual data pipeline. But those documents will be dramatically more specific and more accurate because you have now touched the actual data files, mapped the actual schemas, documented the actual quality issues, and verified that automated reading works.

You are no longer designing in the abstract. You are designing against real data with real quirks that you have personally observed and documented. That is the entire point of Stage 1.

Stage 1 Validation and Stage 2 Complete Guide

Stage 1 Validation

Your Stage 1 execution is genuinely strong. The completion report is professional, the verification matrix shows systematic thinking, and the extra work on Metro City PDF parsing shows initiative that goes beyond what was asked. Let me validate each element and flag the few things that need attention before you fully close Stage 1.

What You Nailed

Your file acquisition is complete and properly organized. 75 files across three years covering state, district, metro city, and PDF sources is exactly the right scope. The fact that you validated 66 out of 66 Excel files with 100 percent automated parsing success means your data foundation is solid.

Your state normalization with 36 entities mapped in states.json is correct. India has 28 states and 8 union territories giving exactly 36 entities. The fact that you have this as a structured JSON file means your normalizer has a working reference from day one.

Your data quality documentation with 133 issues identified and catalogued shows the level of rigor that separates a professional data project from an amateur one. These 133 issues become your regression test cases later. Every one of them is a potential bug that your pipeline must handle correctly.

Your PDF gap analysis identifying the Metro Cities strategy was smart thinking. You recognized that the structured files do not cover everything and you built a plan for the gap instead of ignoring it.

Your field mappings covering 16 table types with raw-to-canonical snake_case mappings means your parser knows exactly how to translate government column names into BharatData schema fields. This is the kind of upfront work that saves weeks of debugging later.

The Metro City PDF parsing producing 9 structured JSON files with verified accuracy is excellent extra work. You have proven that you can handle PDF extraction when needed, which gives you confidence for future PDF-heavy data sources.

What Needs a Small Fix Before Closing Stage 1

There are three items I want you to verify or complete before moving fully into Stage 2. These are small but they matter because Stage 2 documents will reference them.

District name inventory completeness. Your report mentions 36 state entities and 34 metro cities mapped but does not explicitly confirm the district-level name inventory. You downloaded district-wise files containing roughly 780 districts. Did you create the district-name-inventory.md file listing every unique district name from the district-wise files? You do not need to map all 780 districts to canonical forms right now but you need to have examined at least 3-4 states worth of district names and documented the variation patterns you found. If you have done this and it is simply not mentioned in the report, confirm it. If you have not done it, spend one session examining the district names from Maharashtra, Uttar Pradesh, and Tamil Nadu in the district-wise files and documenting the naming patterns. This takes about an hour and it feeds directly into your normalization guide in Stage 2.

Cross-file validation. Your report mentions 133 data quality issues but does not specifically mention whether you cross-validated totals between file types. Did you take the total IPC crimes for any state from the States/UTs file and compare it against the sum of all that state’s districts from the District-wise file? If these numbers match, your data sources are internally

consistent. If they do not match, that is a critical data quality finding that affects your confidence scoring design. Do this check for at least 3 states. It takes 15 minutes.

Year-to-year format consistency check. Did you verify that the same file type from 2021, 2022, and 2023 has the same column structure? Sometimes NCRB adds or removes columns between years. If the 2023 IPC crimes file has 12 columns but the 2021 version has 10 columns because they added two new crime categories, your parser must handle this gracefully. Verify this for at least the States/UTs IPC crimes files across all three years. Document any column differences in your file-format-notes.md.

If all three of these checks are already done and simply not mentioned in your report, add a note confirming them and Stage 1 is fully closed. If any are not done, complete them before starting Stage 2. They take a combined maximum of two hours.

Your Modified Folder Structure

Your folder structure modification is fine. The reorganization into data/raw, data/structured, data/canonical-mappings, Scripts/stage-1, and Docs with subdirectories is clean and logical. The addition of Docs/stages-execution-docs for completion reports is a nice touch for project tracking. This structure will scale well as you add more data sources and pipeline stages.

Stage 2 — Why It Matters

Stage 2 is called the Knowledge Foundation and it is the most misunderstood stage in any software project. Most developers skip it because it feels like paperwork that delays real building. In reality, it is the stage that determines whether your building effort produces a coherent product or a collection of disconnected features that do not fit together.

Here is specifically why each Stage 2 document matters for BharatData.

Why the Project Overview Matters

The Project Overview is the document that every person who encounters BharatData reads first. When a journalist finds your GitHub repository, they read this. When a recruiter looks at your portfolio, they read this. When a potential contributor evaluates whether to invest time in your project, they read this. When you yourself are deep in parser debugging at 2 AM and questioning why you started this, you read this.

Without this document, everyone who encounters BharatData forms their own understanding of what it is. That understanding will be wrong in different ways for different people. A developer might think it is just a data API. A journalist might think it is a visualization tool. A researcher might think it is a static dataset. The Project Overview ensures that everyone starts from the same accurate understanding.

For your AI coding assistant in Antigravity, the Project Overview is the first file loaded in every session. It tells the AI what BharatData is, who it serves, what constraints exist, and what the goals are. Without it, the AI makes assumptions that may contradict your intentions. With it, every piece of code the AI generates is aligned with your project's actual purpose.

Why the PRD Matters

The Product Requirements Document is the single most important document for preventing wasted work. It defines what you are building in version one and explicitly lists what you are not building. Without it, you will inevitably drift. You will start building the SDK and think "I should add a GraphQL endpoint too." You will start building the playground and think "I should add user accounts so people can save their queries." Each of these additions sounds reasonable in isolation but collectively they double your build time and delay your launch by months.

The PRD contains your acceptance criteria. These are the specific measurable conditions that define when each feature is done. Without acceptance criteria, "done" is subjective and you will either ship too early with broken features or never ship because nothing feels complete enough.

For BharatData specifically, the PRD codifies the decision to start with structured files rather than PDFs, which tables are included in version one, what the SDK functions are, what the API endpoints are, and what the playground does. Every engineering decision in Stage 3 (building) is guided by this document.

Why the Quality Standards Matter

BharatData is a data product. The single worst thing that can happen to a data product is serving wrong data confidently. A journalist who publishes an article based on wrong numbers from BharatData will never trust the platform again. A researcher who cites wrong data in a paper suffers professional damage. A developer whose application shows wrong crime statistics loses their users' trust.

The Quality Standards document defines the rules that prevent this. Every record must have a confidence score. Records below a threshold are not served. Numbers must be numbers, not strings. Null means null, not zero. Source attribution is mandatory on every response. These rules sound obvious but without writing them down explicitly, they get violated under time pressure. “I will add confidence scoring later” becomes “I forgot to add confidence scoring and now there are 50,000 records in the database without scores.”

For the AI coding assistant, the Quality Standards document is loaded as a skill file. It means every piece of pipeline code the AI generates automatically includes confidence scoring, source attribution, and null handling because the rules are in the AI’s context, not just in your head.

Why the SDK Design Specification Matters

The SDK is the primary interface between BharatData and developers. Its design determines whether developers love using your tool or abandon it after five minutes of frustration.

Designing the SDK on paper before writing code is critical because changing the API after developers depend on it breaks their applications. If you design bd.crime.getByState() now and a developer builds an app around it, you cannot rename it to bd.crime.fetchState() later without breaking their code. The design spec is your chance to get the API surface right before it becomes a commitment.

The spec also ensures consistency between the TypeScript and Python SDKs. Both must have the same logical structure even though they follow different language conventions. If the TypeScript SDK has bd.crime.getByState() then the Python SDK must have bd.crime.get_by_state(), not bd.crime.fetch_state() or bd.get_crime_by_state(). The spec defines these patterns once and both SDKs follow them.

Why the Pipeline Specification Matters

The pipeline spec is the internal architecture document that describes how data flows from government source to user query response. It specifies the four stages (ingest, parse, normalize, store), what each stage does, what its inputs and outputs are, and how failures are handled.

For a solo developer using an AI coding assistant, this document is essential because it gives the AI a complete blueprint to follow. Without it, the AI generates parser code that does not connect properly to the normalizer, or normalizer code that outputs a

format the database schema does not expect. With the pipeline spec, every component the AI generates fits together because they all follow the same documented architecture.

The pipeline spec also documents your monitoring strategy, your data freshness checks, and your incident response process. These operational details are easy to forget when you are focused on building features but they are what keep the system running reliably after launch.

Why the Agent and IDE Configuration Matters

Your AI coding assistant is only as good as the context you give it. Without proper configuration, the AI generates generic code that does not follow your project's specific rules. With proper configuration, the AI generates code that automatically includes confidence scoring, follows your naming conventions, uses your canonical mappings, and respects your infrastructure constraints.

The feeding prompt, agent definitions, skill files, and workflow definitions you create in Stage 2 are the instructions that transform a generic AI into a BharatData specialist. Every minute you spend on this configuration saves many minutes of correcting AI output during Stage 3.

Stage 2 — Complete Detailed Guide

Step 1 — Project Overview Document

File: docs/PROJECT_OVERVIEW.md

Time: 2-3 hours

Why First: This document establishes the language and framing that every subsequent document uses. If you write the PRD before the Overview, you will find yourself re-explaining what BharatData is inside the PRD. If you write the SDK spec before the Overview, you will embed positioning language in a technical document where it does not belong. The Overview comes first because it defines the vocabulary and the vision that everything else references.

What To Write:

Open a new markdown file. Write this document yourself, not with AI assistance. The Overview must be in your voice because it represents your vision and your commitment. AI can help you edit for clarity afterward but the first draft must come from you.

Start with what BharatData is in one paragraph. Not what it does technically. What it is in the world. An open source project that makes Indian government public data accessible to everyone through developer tools and a web playground. One paragraph. Clear. Direct. No buzzwords.

Then write why BharatData exists in two paragraphs. Draw from the problem understanding you built during Stage 1. You have personally seen the messy NCRB files. You have personally encountered the 133 data quality issues. You have personally experienced the state name inconsistencies. Write about the problem from this firsthand experience, not from abstract theory. “Indian government data is published in formats that are practically unusable” is abstract. “The NCRB publishes crime data for 780 districts but spells district names differently across files from the same year” is concrete. Use the concrete version.

Then write who BharatData is for in one paragraph. Developers who need Indian data in their applications. Data scientists who need clean DataFrames for analysis. Journalists who need verified statistics for reporting. Researchers who need cited data for papers. Students who need real data for projects. Citizens who want to understand their government’s data. List them. Do not elaborate on each one here. That detail lives in the Audience Map document.

Then write what BharatData believes in one paragraph. Public data should be publicly accessible. Data quality matters more than data quantity. Open source is the right model for civic infrastructure. Transparency about limitations builds trust. These are your principles. They guide every decision you make.

Then write where BharatData is going in one paragraph. More datasets driven by community demand. AI-powered natural language queries for non-technical users. A canonical reference for Indian administrative geography. Honest about what is built today versus what is planned.

How to Know It Is Done: Read the document to someone who has never heard of BharatData. If they can explain what it is, why it exists, and who it is for after reading it, the document works. If they have questions about any of those three things, revise until they do not.

Step 2 — Product Requirements Document

File: docs/internal/PRD.md

Time: 3-4 hours

Why Second: The PRD translates the vision from the Overview into engineering specifications. It is the bridge between what you want to build and what you actually build. Every engineering decision in Stage 3 traces back to a line in this document.

What To Write:

This document has seven sections. Each one requires you to make specific decisions and write them down precisely.

Section 1: v1 Scope — Included. List every feature that version one of BharatData includes. Be exhaustive and specific. Do not say "crime data." Say "State/UT-wise IPC crime data for 2021, 2022, and 2023 from NCRB Additional Tables, covering 36 states and UTs with the following fields: [list the exact fields from your FIELD_MAPPINGS.md]."

Based on your Stage 1 work, your v1 scope should include these specific items.

Data coverage: 12 state-level table types across 3 years, 10 district-level table types for 2023, 3 metro city summary tables across 3 years, and 9 metro city PDF-parsed datasets across 3 years.

Data pipeline: Excel file ingester for NCRB Additional Tables, PDF parser for metro city data, normalizer with states.json canonical mappings, PostgreSQL storage with source attribution and confidence scores on every record, daily health monitoring on NCRB source URLs.

TypeScript SDK: npm package bharatdata with typed query functions following the design spec. Specific functions for querying by state, by year, by category, by district, and a comparison function.

Python SDK: PyPI package bharatdata with pandas DataFrame integration, export methods, and type hints.

REST API: Endpoints matching SDK functions with rate limiting and CORS.

Web playground: Browser-based query interface with table display and CSV download, no login required.

Documentation: README, API reference for both SDKs, data source registry, contributing guide.

Section 2: v1 Scope — Explicitly Excluded. This is equally important. List everything you are deliberately not building.

AI natural language queries. AI data narration. AI anomaly detection. User accounts or authentication. Paid tiers. Mobile app. Data visualization or charting. Webhooks or email alerts. GraphQL. WebSocket connections. Admin panel. Multi-tenancy. Dashboard analytics beyond the playground.

Write each exclusion as a clear statement. “AI natural language queries are excluded from v1 and will be built in a later stage after the core data platform has real users.” This prevents you from adding these features impulsively during Stage 3.

Section 3: User Stories. Write 6-8 specific user stories based on your five audience segments. Use the format “As a [user type], I want to [action], so that [outcome].”

Draw these from real scenarios. “As a journalist at IndiaSpend, I want to query state-wise crime against women data for the last three years, so that I can identify which states showed the largest increase for my article.” “As a data science student, I want to install a Python package and get NCRB crime data as a pandas DataFrame, so that I can use real Indian data in my machine learning project instead of the Boston housing dataset.” “As a policy researcher at NITI Aayog, I want to download district-wise IPC crime data as CSV, so that I can analyze crime patterns across 780 districts in Stata.”

Section 4: Acceptance Criteria. For each major feature, write specific measurable conditions that define when it is done. Not “parser works well” but “parser correctly extracts data from all 66 Excel files with 100 percent field mapping accuracy verified by running examine_all.py.” Not “SDK is typed” but “TypeScript compiler in strict mode produces zero errors and the grep command finds zero instances of the any type in any exported function signature.”

Your acceptance criteria should reference your Stage 1 artifacts directly. “Parser handles all 133 documented data quality issues from data-quality-issues.md.” “Normalizer resolves all state name variations documented in state-name-variations.md.” “All 16 table types documented in FIELD_MAPPINGS.md are ingested and queryable.”

Section 5: Technical Constraints. Free tier infrastructure only. Single developer. Both SDKs must work after installation without requiring any API key for v1. Response time under 3 seconds. Database within Neon 500MB limit. Scrapers maximum 1 request per second.

Section 6: Dependencies and Risks. NCRB might change file formats. NCRB might remove the Additional Tables page. Free tier database might hit storage limits. District name normalization might have gaps. State name variations beyond what you documented might exist in user queries.

Section 7: Data Inventory. This section is unique to a data product PRD. List every data file included in v1 with its filename, year, geographic level (state/district/city), crime category, and the canonical BharatData table it maps to. This is essentially your table-inventory.md from Stage 1 formatted into the PRD.

How to Know It Is Done: Every engineering task you will do in Stage 3 should be traceable to a specific line in the PRD. If you find yourself building something in Stage 3 that the PRD does not mention, either the PRD needs updating or you are building something out of scope.

Step 3 — Quality Standards Document

File: docs/internal/QUALITY_STANDARDS.md

Time: 2 hours

Why Third: The quality standards must exist before you write the SDK spec and the pipeline spec because both of those documents reference quality rules. The SDK spec needs to know what confidence scores look like in API responses. The pipeline spec needs to know what confidence thresholds determine whether data is served or quarantined.

What To Write:

This document defines non-negotiable rules organized by category. Draw heavily from your Stage 1 data quality findings. The 133 issues you documented are your evidence base for why each rule exists.

Data quality rules. Every record must have source attribution including the original filename, the government source URL, and the collection date. Every record must have a confidence score between 0.0 and 1.0. Define what each score range means specifically for your data. For NCRB structured files that parsed cleanly with no quality issues, confidence is 1.0. For files where timestamp prefixes in filenames required handling, confidence is

0.95. For PDF-extracted metro city data where you verified accuracy manually, confidence is 0.9. For any record where a data quality issue from your 133-issue log applies, confidence is reduced by a defined amount per issue type.

Define the serving threshold. Records with confidence below 0.5 are stored but not served through the API. This threshold is based on your judgment about what level of uncertainty is acceptable for a data product that journalists might cite.

Define null handling rules based on what you actually found in the NCRB files. You know from Stage 1 exactly how missing values are represented. Write rules that handle each representation you documented.

Define number format rules based on what you found. You know from Stage 1 whether numbers are stored as text or numeric, whether Indian notation is used, and whether percentage symbols appear in cells. Write rules for each case.

Code quality rules. Zero any types in TypeScript. Every function has error handling. Every public function has documentation.

Pipeline quality rules. Ingester logs every failure. Parser validates output against expected schema. Normalizer logs every name resolution and every unmapped entity. Database records are immutable.

Release quality rules. Manual spot check of 10 records before every release. Changelog entry for every release.

How to Know It Is Done: Every data quality issue in your 133-issue log should be addressed by at least one rule in this document. Go through the log and verify.

Step 4 — SDK Design Specification (TypeScript)

File: docs/sdk/DESIGN_TYPESCRIPT.md

Time: 3 hours

Why Fourth: The SDK design must come after the PRD (which defines what data is available) and after the quality standards (which define what responses look like). It must come before the pipeline spec because the pipeline’s output format must match what the SDK expects.

What To Write:

Design the TypeScript SDK API surface on paper. Think about this from the perspective of a developer who just ran npm install bharatdata and is reading your README for the first time.

Client instantiation. How does a developer create a BharatData client? What configuration options exist? For v1 with no authentication, the client creation should be as simple as possible. const bd = new BharatData() with no required parameters.

Dataset modules. The client has a crime property that exposes all crime-related query functions. In the future, bd.census, bd.environment, and bd.agriculture will exist. For v1, only bd.crime exists.

Query functions. Based on your Stage 1 data inventory, design the specific functions. You have state-level data, district-level data, and city-level data. You have data by year, by crime category, and by victim group. Your functions should reflect these natural query dimensions.

bd.crime.getByState() — returns crime data for a specific state, optionally filtered by year and category. This is the most common query. A journalist asking “what is the crime in Maharashtra” uses this.

bd.crime.getByYear() — returns crime data for all states in a given year. A researcher comparing all states uses this.

bd.crime.getByDistrict() — returns crime data for a specific district or all districts in a state. This is your district-level access function. A developer building a district-level dashboard uses this.

bd.crime.getByCategory() — returns crime data filtered by crime category (IPC, SLL, crimes against women, crimes against children, cyber crimes, etc.) with optional state and year filters.

bd.crime.getByCity() — returns crime data for metropolitan cities. This uses your metro city parsed data.

bd.crime.compare() — returns comparative data across multiple states, districts, or cities for side-by-side analysis. A journalist comparing three states uses this.

bd.crime.search() — flexible text search across all crime data.

For each function, write the complete TypeScript signature including parameter types and return types. Design the response shape that every function returns. Data array, meta object with source attribution and confidence, and query echo object.

Error handling. Define custom error classes and when each is thrown.

Versioning. Define your semver strategy.

How to Know It Is Done: Write a small TypeScript file that imports BharatData and makes five different queries covering each function. The code should feel natural and predictable even though the SDK does not exist yet. If you find yourself uncertain about how a function should be called, that is a signal that the design needs refinement.

Step 5 — SDK Design Specification (Python)

File: docs/sdk/DESIGN_PYTHON.md

Time: 2-3 hours

Why Fifth: The Python SDK design mirrors the TypeScript SDK but adapts to Python conventions and the data science ecosystem. Writing it immediately after the TypeScript spec ensures consistency while the design is fresh in your mind.

What To Write:

The Python SDK has the same logical structure as the TypeScript SDK but with Python-specific additions.

Naming conventions. Everything is snake_case. bd.crime.get_by_state() not bd.crime.getByState(). Parameter names are snake_case. Response keys are snake_case.

DataFrame integration. This is the most critical Python-specific feature. Every query function should support returning results as a pandas DataFrame. Design how this works. Does every function return a BharatDataResponse object that has a to_dataframe() method? Or does every function accept a as_dataframe=True parameter? Pick one approach and apply it consistently.

The DataFrame should have proper column types. Numeric columns as int64 or float64, not object. Date columns as datetime64. String columns as string. The index should be meaningful, perhaps the state name for state-level queries or the district name for district-level queries.

Export methods. to_csv(), to_parquet(), to_json(), to_excel(). Design the method signatures including what parameters they accept (file path, whether to include metadata).

Bulk data access. get_all() function that handles pagination transparently for large queries. A researcher requesting all districts across all years should not need to write pagination loops.

Type hints. Every function signature must have complete type annotations. Design the TypedDict definitions for your response shapes.

Docstrings. Define the docstring format. NumPy style is standard for data science packages.

How to Know It Is Done: Write a sample Jupyter notebook cell sequence that imports bharatdata, queries crime data, converts to DataFrame, does basic pandas analysis (groupby, describe), and exports to CSV. The code should feel like natural pandas workflow, not like using a foreign API.

Step 6 — Data Pipeline Specification

File: docs/internal/DATA_PIPELINE_SPEC.md

Time: 3-4 hours

Why Sixth: The pipeline spec comes after the SDK specs because the pipeline's output must match what the SDKs expect to consume. It comes after the quality standards because every pipeline stage must enforce those standards.

What To Write:

Document the four-stage pipeline with specific reference to your Stage 1 findings.

Stage 1: Ingester. For NCRB Additional Tables, the ingester visits the NCRB website, checks for new files, downloads them, and stores them with metadata. Based on your Stage 1 experience, document the specific URL patterns, the file naming conventions including the timestamp prefix issue you found in 2021 files, and the download behavior including any session requirements or rate limiting.

Stage 2: Parser. You now have two parser types based on your Stage 1 work. The Excel parser uses pandas to read the structured files with the specific skiprows logic you developed to handle merged header cells. Document the exact pandas parameters that work for each file type. The PDF parser uses the approach you developed for metro city data. Document how it works and its accuracy characteristics.

Reference your FIELD_MAPPINGS.md for the column-to-canonical-field translations. Reference your examine_all.py script as the validation tool.

Stage 3: Normalizer. Reference your states.json canonical mappings. Document the normalization pipeline steps: state name resolution, data type conversion, null standardization, confidence

scoring, source attribution attachment. Reference the specific data quality issues from your 133-issue log that the normalizer handles.

Stage 4: Storage. Design the PostgreSQL schema based on your actual data. You know exactly what fields exist from your FIELD_MAPPINGS.md. You know how many records to expect from your Stage 1 file examination. Design the tables, indexes, and storage estimates with these real numbers.

Monitoring. Daily health check on NCRB URLs. Weekly data quality check. Automated alerting through GitHub issues.

How to Know It Is Done: Walk through the pipeline mentally with one specific file, say the State/UT-wise IPC Crimes 2023 Excel file. At each stage, you should be able to describe exactly what happens to the data, what checks are performed, and what the output looks like. If any stage is vague, add more detail.

Step 7 — Agent and IDE Configuration

Time: 2-3 hours

What To Create:

Based on everything you wrote in Steps 1-6, create the following files that configure your AI coding assistant.

Feeding Prompt at .agent/FEEDING_PROMPT.md. This is the master context prompt you paste at the start of every Antigravity session. It lists all files the AI must read before starting any task, the critical rules that are never violated, and the model assignment for different task types.

The feeding prompt should reference your actual documents by their actual file paths. Not hypothetical paths. The real paths in your actual folder structure as reorganized in Stage 1.

Agent Definitions. Create four agent definition files. The data pipeline specialist who knows about your NCRB data, your parser quirks from Stage 1, and your normalization rules. The SDK architect who knows your TypeScript and Python design specs. The data quality auditor who knows your 133 documented issues and your confidence scoring rules. The documentation specialist who knows your brand, voice, and document standards.

Each agent definition should reference specific files from your project. The pipeline specialist should reference your examine_all.py script, your FIELD_MAPPINGS.md, your states.json, and your data-quality-issues.md. These are real artifacts that exist in your project, not hypothetical references.

Skill Files. Create five skill files covering PDF parsing, normalization, SDK conventions, data quality, and project context. Each skill encodes the rules from your Quality Standards document in a format the AI can apply automatically during code generation.

Workflow Definitions. Create workflows for data validation, data ingestion, adding a new data source, and publishing to npm/PyPI. Each workflow is a step-by-step procedure the AI follows when triggered.

How to Know It Is Done: Start a new Antigravity session. Paste the feeding prompt. Ask the AI to describe the BharatData project including its data sources, quality standards, and SDK design. If the AI can accurately describe your project based on the loaded context, your configuration is correct. If it gets anything wrong, the relevant document needs clarification.

Step 8 — Normalization Guide

File: docs/data-schema/NORMALIZATION.md

Time: 2 hours

Why This Is Separate From the Pipeline Spec: The normalization guide is a reference document that both the pipeline spec and the SDK specs reference. It documents how raw data becomes clean data at a conceptual level. The pipeline spec documents how the normalizer code works. The normalization guide documents what the normalizer is supposed to achieve.

What To Write:

Document your canonical mappings structure based on your actual states.json file. Show the format. Explain how variations are resolved. Explain how the normalizer handles names not found in the mappings.

Document your data type standards based on what you actually found in the NCRB files during Stage 1. You know exactly how numbers are formatted, how dates appear, how null values are represented. Write the conversion rules for each.

Document your confidence scoring criteria with specific reference to your data sources. Excel files from NCRB Additional Tables with clean parsing get 1.0. PDF-extracted metro city data with verified accuracy gets 0.9. Any record affected by a documented quality issue gets a reduced score. Define the reduction amounts.

Document the historical boundary handling for Telangana and Ladakh. Your states.json should already encode when each entity was created. The normalization guide explains how queries across boundary change dates are handled.

Step 9 — Content Strategy Draft

File: docs/internal/CONTENT_STRATEGY.md

Time: 1-2 hours

Why During Stage 2: Your first blog post should be drafted during Stage 3 while building the pipeline, but the content strategy should be planned during Stage 2 so you know what to capture during the build process. If you wait until after building to plan your content, you will have forgotten the specific details that make blog posts compelling.

What To Write:

Plan your first five blog posts with titles, angles, and what to capture during Stage 3 building.

Post 1 should be about your NCRB data discovery. You found that NCRB publishes data in two formats and the easier one is barely known. That is a genuinely useful finding for journalists and researchers. Plan the structure: problem, discovery, what the data looks like, how to access it, what BharatData does with it.

Post 2 should be a technical deep dive into normalization. 36 states with dozens of name variations. 780 districts with even more variations. Boundary changes over time. Plan what screenshots and examples to capture during Stage 3.

Post 3 should use BharatData to answer an interesting question about Indian crime data. Plan what question to investigate and what data to query.

Plan your community engagement strategy. Which communities to join. How to participate genuinely. When to mention BharatData and when not to.

Step 10 — Brand Minimums Checklist

File: docs/internal/BRAND_MINIMUMS.md

Time: 20 minutes

Why Last: Brand minimums are needed before you write the README in Stage 3 but they are the simplest document in Stage 2. Name format, tagline, disclaimer, package descriptions for npm and PyPI, GitHub repo description. Write them down so they are consistent across everything you create.

Stage 2 Completion Criteria

Stage 2 is complete when all of the following are true.

The Project Overview exists and accurately describes BharatData in your own voice.

The PRD exists with specific v1 scope, explicit exclusions, measurable acceptance criteria, and a complete data inventory referencing your actual Stage 1 files.

The Quality Standards exist with rules that address all 133 documented data quality issues from Stage 1.

Both SDK Design Specifications exist with complete function signatures, response shapes, and error handling contracts.

The Pipeline Specification exists with stage-by-stage architecture referencing your actual parsers, normalizer, and schema from Stage 1.

The Normalization Guide exists with canonical mapping documentation, data type standards, and confidence scoring criteria.

The Agent and IDE configuration is complete with feeding prompt, agent definitions, skill files, and workflow definitions that reference your actual project files.

The Content Strategy is drafted with blog post plans and community engagement approach.

The Brand Minimums checklist is written.

All documents reference each other correctly. The PRD references the quality standards. The SDK specs reference the normalization guide. The pipeline spec references the field mappings. There are no dangling references to documents that do not exist.

When all of these conditions are met, you are ready for Stage 3 where you build the actual data pipeline, both SDKs, the API, and the playground. And you will build them faster and better than if

you had skipped Stage 2 because every engineering decision is guided by documents grounded in your real Stage 1 data exploration.

Document 5: Python SDK Design Specification

```
# BharatData — Python SDK Design Specification

**Document ID:** DOC-05
**Version:** 1.0
**Status:** Active
**Created:** 2025-03-20
**Audience:** Builder, contributors, SDK architect agent
**Package Name:** bharatdata
**Registry:** PyPI
**License:** MIT
**Minimum Python:** 3.9+

---

## Design Philosophy

The BharatData Python SDK is built specifically for the Python
        data science
ecosystem. It serves data scientists working in Jupyter notebooks,
        machine
learning engineers building training pipelines, researchers
        performing
statistical analysis, analysts using pandas and matplotlib, and
        developers
building Python backend services.

The SDK does not simply wrap the REST API in Python function
        calls. It
provides a native Python experience with pandas DataFrames as
        first-class
outputs, snake_case naming throughout, complete type hints for IDE
autocomplete, export methods for common data science formats, and
thoughtful integration with the tools Python data professionals
        already use.

A data scientist who has never heard of BharatData should be able
        to
install the package, open a Jupyter notebook, and have a clean
        pandas
DataFrame of Indian crime data in under 5 minutes. The workflow
        should
```

feel like using any well-designed Python data library, not like using
a foreign API through a thin wrapper.

Design Principles

1. Pythonic Above All

The SDK follows Python conventions everywhere. snake_case for all function
names, parameter names, and dictionary keys. Type hints on every public
function. Context managers where appropriate. Generators for large result
sets. The SDK should feel like it was written by a Python developer for
Python developers, not ported from another language.

2. DataFrames as First-Class Citizens

pandas is the lingua franca of Python data work. Every query function
supports returning results as a pandas DataFrame with proper column types,
meaningful indexes, and clean column names. DataFrame conversion is not
an afterthought bolted onto dictionary responses. It is a core design
consideration that influences every aspect of the response structure.

3. Progressive Complexity

Simple things are simple. Complex things are possible.

```python

# Simple: one line to get data
df = bd.crime.get_by_state("Maharashtra",
        year=2023).to_dataframe()

# Complex: full control over every parameter
result = bd.crime.compare(
    entities=["Maharashtra", "Karnataka", "Tamil Nadu"],
    years=[2021, 2022, 2023],
    category="cyber-crime",
)
df = result.to_dataframe()
df.to_parquet("crime_comparison.parquet")
```

4. Fail Helpfully

Error messages include not just what went wrong but what to do about it. When a state name is not recognized, the error message suggests checking available states and notes that common abbreviations are accepted. When a year is out of range, the error message lists available years.

5. Minimal Required Dependencies

The only required runtime dependency is `requests` for HTTP communication. pandas, pyarrow, and openpyxl are optional extras installed only when the user needs DataFrame conversion, Parquet export, or Excel export respectively. A developer who just needs dictionary responses should not be forced to install the entire pandas ecosystem.

Installation

```python
# Core package (dictionary responses only)
pip install bharatdata

# With pandas DataFrame support
pip install bharatdata[pandas]

# With all export formats (pandas + parquet + excel)
pip install bharatdata[all]

# Development installation
pip install bharatdata[dev]
```

Dependency Groups

```toml
[project]
name = "bharatdata"
version = "0.1.0"
description =
        "Clean, queryable Indian government data. Python SDK with
        pandas integration, source attribution, and confidence
        scoring."
requires-python = ">=3.9"
license = {text = "MIT"}
keywords = ["india", "government-data", "ncrb", "crime-data",
        "open-data", "pandas", "data-science"]

dependencies = [
    "requests>=2.28.0",
]
```

```toml
[project.optional-dependencies]
pandas = [
    "pandas>=1.5.0",
    "numpy>=1.23.0",
]
parquet = [
    "pandas>=1.5.0",
    "numpy>=1.23.0",
    "pyarrow>=10.0.0",
]
excel = [
    "pandas>=1.5.0",
    "numpy>=1.23.0",
    "openpyxl>=3.0.0",
]
all = [
    "pandas>=1.5.0",
    "numpy>=1.23.0",
    "pyarrow>=10.0.0",
    "openpyxl>=3.0.0",
]
dev = [
    "pytest>=7.0.0",
    "pytest-cov>=4.0.0",
    "mypy>=1.0.0",
    "ruff>=0.1.0",
    "responses>=0.23.0",
]
```

Client Instantiation

```python
from bharatdata import BharatData

# Default configuration (no API key needed for v1)
bd = BharatData()

# With optional configuration
bd = BharatData(
    base_url="https://api.bharatdata.dev",  # default
    timeout=10,                             # default 10 seconds
    retries=3,                              # default 3
    cache=False,                            # default False
)

# With local caching for repeated queries during development
bd = BharatData(cache=True, cache_dir="~/.bharatdata/cache")
```

Configuration Parameters

```
class BharatData:
    """
    BharatData client for accessing Indian government public data.

    Parameters
    ----------
    base_url : str, optional
        Base URL for the BharatData API.
        Default: 'https://api.bharatdata.dev'

    timeout : int, optional
        Request timeout in seconds.
        Default: 10

    retries : int, optional
        Number of retry attempts for failed requests.
        Default: 3

    cache : bool, optional
        Whether to cache query results locally for repeated
         queries.
        Useful during iterative analysis in Jupyter notebooks.
        Default: False

    cache_dir : str, optional
        Directory for cached query results.
        Default: '~/.bharatdata/cache'

    cache_ttl : int, optional
        Cache time-to-live in seconds. Cached results older than
         this
        are re-fetched from the API.
        Default: 86400 (24 hours)

    Examples
    --------
    >>> from bharatdata import BharatData
    >>> bd = BharatData()
    >>> result = bd.crime.get_by_state("Maharashtra", year=2023)
    >>> print(result.data[0]["total_cases"])
    """
```

Dataset Modules

The BharatData client exposes dataset-specific modules as properties:

```
bd = BharatData()

bd.crime       # Crime data from NCRB (v1)

# Future modules:

# bd.census        # Census demographic data

# bd.environment   # CPCB air quality, FSI forest cover

# bd.agriculture   # Agmarknet prices, crop data

# bd.economy       # RBI indicators, trade data

# bd.education     # UDISE, AISHE data

# bd.health        # CBHI, NFHS data
```

Each module follows the identical interface pattern. Learning one module means knowing them all.

Query Functions — Crime Module

bd.crime.get_by_state()

```
def get_by_state(
    self,
    state: str,
    *,
    year: int | None = None,
    category: str | None = None,
) -> BharatDataResponse:
    """
    Get crime data for a specific state or union territory.

    Accepts any known name variation for the state. For example,
    "Maharashtra", "MAHARASHTRA", and "MH" all return the same
        data.

    Parameters
    ----------
    state : str
        State or union territory name. Any known variation is
        accepted.
        Use ``bd.crime.get_states()`` to see all available states.

    year : int, optional
        Four-digit year to filter by (e.g., 2023).
        If not provided, returns data for all available years.
```

```
category : str, optional
    Crime category to filter by (e.g., "cyber-crime",
    "crimes-against-women"). Use ``bd.crime.get_categories()``
    to see all available categories.

Returns
-------
BharatDataResponse
    Response object containing:
    - ``data``: list of crime record dictionaries
    - ``meta``: source attribution and confidence metadata
    - ``query``: echo of query parameters
    - ``.to_dataframe()``: convert to pandas DataFrame
    - ``.to_csv(path)``: export to CSV file
    - ``.to_parquet(path)``: export to Parquet file
    - ``.to_json(path)``: export to JSON file
    - ``.to_excel(path)``: export to Excel file

Raises
------
SourceNotFoundError
    If the state name is not recognized and cannot be resolved
    through canonical mappings.
InvalidQueryError
    If year is outside the available range or category is
    invalid.
DataUnavailableError
    If the BharatData API is temporarily unavailable.

Examples
--------
Get all crime data for Maharashtra:

>>> result = bd.crime.get_by_state("Maharashtra")
>>> len(result.data)
36

Get 2023 data as a pandas DataFrame:

>>> df = bd.crime.get_by_state("Maharashtra",
    year=2023).to_dataframe()
>>> df.columns
Index(['state', 'year', 'category', 'total_cases',
    'rate_per_lakh', ...])

Get cyber crime data:

>>> result = bd.crime.get_by_state("Maharashtra", year=2023,
    category="cyber-crime")
>>> print(result.data[0]["total_cases"])
8249
```

```
    Export to CSV:

    >>> bd.crime.get_by_state("UP",
        year=2023).to_csv("up_crime_2023.csv")

    Notes
    -----
    - State name resolution is case-insensitive
    - Common abbreviations are accepted (UP, MP, AP, TN, etc.)
    - Telangana data is available only from 2014 onward
    - Ladakh data is available only from 2019 onward
    """
```

bd.crime.get_by_year()

```
def get_by_year(
    self,
    year: int,
    *,
    category: str | None = None,
) -> BharatDataResponse:
    """
    Get crime data for all states and UTs in a specific year.

    Parameters
    ----------
    year : int
        Four-digit year (e.g., 2023). Use ``bd.crime.get_years()``
        to see all available years.

    category : str, optional
        Crime category to filter by.

    Returns
    -------
    BharatDataResponse
        Response with data for all 36 states and UTs.

    Raises
    ------
    InvalidQueryError
        If year is not in the available range.

    Examples
    --------
    Get all 2023 crime data as DataFrame:

    >>> df = bd.crime.get_by_year(2023).to_dataframe()
    >>> df.groupby("state")
        ["total_cases"].sum().sort_values(ascending=False).head()
```

```
    state
    Uttar Pradesh    637495
    Maharashtra      545862
    ...

    Compare crimes against women across all states:

    >>> df = bd.crime.get_by_year(2023, category="crimes-against-
        women").to_dataframe()
    >>> df.sort_values("total_cases", ascending=False).head(10)
    """
```

bd.crime.get_by_district()

```
def get_by_district(
    self,
    *,
    district: str | None = None,
    state: str | None = None,
    year: int | None = None,
    category: str | None = None,
) -> BharatDataResponse:
    """
    Get crime data at the district level.

    At least one of ``district`` or ``state`` must be provided.
    If only ``state`` is provided, returns data for all districts
    in that state. If only ``district`` is provided, searches for
    the district across all states.

    Parameters
    ----------
    district : str, optional
        District name. Accepts common variations.

    state : str, optional
        State name to filter districts. Useful when district names
        are ambiguous (multiple states may have districts with
        similar names).

    year : int, optional
        Four-digit year. Default: most recent available year.

    category : str, optional
        Crime category to filter by.

    Returns
    -------
    BharatDataResponse
        Response with district-level crime records.
```

```
    Raises
    ------
    InvalidQueryError
        If neither district nor state is provided.
    SourceNotFoundError
        If the district or state is not recognized.

    Examples
    --------
    Get all districts in Maharashtra:

    >>> df =
        bd.crime.get_by_district(state="Maharashtra").to_dataframe()
    >>> len(df)
    36  # Maharashtra has 36 districts

    Get data for a specific district:

    >>> result = bd.crime.get_by_district(district="Pune",
        state="Maharashtra")
    >>> print(result.data[0]["total_cases"])

    Analyze crime across all Indian districts:

    >>> df = bd.crime.get_by_district(state=None, district=None,
        year=2023)
    ... # This raises InvalidQueryError — at least one filter
        required
    >>> df = bd.crime.get_by_district(category="cyber-crime",
        year=2023)
    ... # Gets cyber crime for all districts
    """
```

bd.crime.get_by_city()

```
def get_by_city(
    self,
    city: str,
    *,
    year: int | None = None,
    category: str | None = None,
) -> BharatDataResponse:
    """
    Get crime data for a metropolitan city.

    BharatData covers 34 metropolitan cities as defined by NCRB.

    Parameters
    ----------
    city : str
        City name (e.g., "Mumbai", "Delhi", "Bengaluru").
```

```
        Accepts common variations ("Bangalore" resolves to
        "Bengaluru").

    year : int, optional
        Four-digit year. If not provided, returns all available
        years.

    category : str, optional
        Crime category to filter by.

    Returns
    -------
    BharatDataResponse
        Response with city-level crime records.

    Raises
    ------
    SourceNotFoundError
        If the city is not in the 34 metropolitan cities covered
        by NCRB.

    Examples
    --------
    >>> result = bd.crime.get_by_city("Mumbai", year=2023)
    >>> df = result.to_dataframe()
    >>> print(df[["category", "total_cases"]].head())

    Compare cities over time:

    >>> for city in ["Mumbai", "Delhi", "Bengaluru"]:
    ...     result = bd.crime.get_by_city(city)
    ...     df = result.to_dataframe()
    ...     print(f"{city}: {df.groupby('year')
        ['total_cases'].sum().to_dict()}")
    """
```

bd.crime.get_by_category()

```
def get_by_category(
    self,
    category: str,
    *,
    state: str | None = None,
    year: int | None = None,
    level: str | None = None,
) -> BharatDataResponse:
    """
    Get crime data filtered by crime category.

    Parameters
    ----------
    category : str
```

```
        Crime category identifier. Use
        ``bd.crime.get_categories()``
        to see all available categories with descriptions.

    state : str, optional
        Filter by state or UT.

    year : int, optional
        Filter by year.

    level : str, optional
        Geographic level: "state", "district", or "city".
        Default: "state"

    Returns
    -------
    BharatDataResponse

    Examples
    --------
    Cyber crime across all states in 2023:

    >>> df = bd.crime.get_by_category("cyber-crime",
        year=2023).to_dataframe()
    >>> df.sort_values("total_cases", ascending=False).head()

    Crimes against women at district level:

    >>> df = bd.crime.get_by_category(
    ...     "crimes-against-women",
    ...     state="Maharashtra",
    ...     level="district"
    ... ).to_dataframe()
    """
```

bd.crime.compare()

```python
def compare(
    self,
    entities: list[str],
    *,
    years: list[int] | None = None,
    category: str | None = None,
    level: str | None = None,
) -> BharatDataResponse:
    """
    Compare crime data across multiple states, districts, or
        cities.

    Parameters
    ----------
    entities : list[str]
```

```
    List of entity names to compare (2 to 10 entities).

years : list[int], optional
    List of years to include. If not provided, includes all
    available years.

category : str, optional
    Crime category to filter by.

level : str, optional
    Geographic level: "state", "district", or "city".
    Auto-detected from entity names if not provided.

Returns
-------
BharatDataResponse

Raises
------
InvalidQueryError
    If fewer than 2 or more than 10 entities are provided.

Examples
--------
Compare three states over three years:

>>> result = bd.crime.compare(
...     ["Maharashtra", "Karnataka", "Tamil Nadu"],
...     years=[2021, 2022, 2023]
... )
>>> df = result.to_dataframe()
>>> pivot = df.pivot_table(
...     values="total_cases",
...     index="state",
...     columns="year",
...     aggfunc="sum"
... )
>>> print(pivot)

Compare cyber crime in top 5 states:

>>> df = bd.crime.compare(
...     ["Maharashtra", "Karnataka", "Telangana", "UP",
     "Delhi"],
...     years=[2023],
...     category="cyber-crime"
... ).to_dataframe()
"""
```

bd.crime.search()

```python
def search(
    self,
    query: str,
    *,
    state: str | None = None,
    year: int | None = None,
    category: str | None = None,
    limit: int = 50,
    offset: int = 0,
) -> BharatDataResponse:
    """
    Flexible text search across all crime data fields.

    Parameters
    ----------
    query : str
        Search term to match against state names, district names,
        category names, and other text fields.

    state : str, optional
        Filter results to a specific state.

    year : int, optional
        Filter results to a specific year.

    category : str, optional
        Filter results to a specific category.

    limit : int, optional
        Maximum number of results to return. Default: 50. Max:
        500.

    offset : int, optional
        Number of results to skip for pagination. Default: 0.

    Returns
    -------
    BharatDataResponse

    Examples
    --------
    >>> result = bd.crime.search("cyber", year=2023, limit=20)
    >>> df = result.to_dataframe()
    """
```

bd.crime.get_all()

```python
def get_all(
    self,
```

```
    *,
    year_range: tuple[int, int] | None = None,
    states: list[str] | None = None,
    categories: list[str] | None = None,
    level: str = "state",
) -> BharatDataResponse:
    """
    Get all available crime data matching the filters.

    Handles pagination transparently. For large requests, data is
    streamed in chunks to manage memory.

    Parameters
    ----------
    year_range : tuple[int, int], optional
        Inclusive year range, e.g., (2021, 2023).
        If not provided, returns all available years.

    states : list[str], optional
        List of states to include. If not provided, includes all.

    categories : list[str], optional
        List of categories to include. If not provided, includes
         all.

    level : str, optional
        Geographic level: "state", "district", or "city".
        Default: "state"

    Returns
    -------
    BharatDataResponse
        Complete dataset matching filters. May contain thousands
        of records for district-level requests.

    Examples
    --------
    Get all state-level data for ML training:

    >>> df = bd.crime.get_all(year_range=(2021,
         2023)).to_dataframe()
    >>> print(f"Shape: {df.shape}")
    Shape: (108, 15)  # 36 states × 3 years

    Get all district-level data:

    >>> df = bd.crime.get_all(level="district", year_range=(2023,
         2023)).to_dataframe()
    >>> print(f"Shape: {df.shape}")
    Shape: (780, 15)  # ~780 districts

    Export complete dataset as Parquet for Spark processing:
```

```
    >>> bd.crime.get_all(level="district").to_parquet("ncrb_districts.parquet")

    Notes
    -----
    Large district-level requests may take several seconds.
    Consider using the ``cache=True`` option on the client for
    repeated access to the same data during iterative analysis.
    """
```

Metadata Functions

```
def get_states(self) -> list[StateInfo]:
    """Get list of all available states and union territories."""

def get_districts(self, state: str | None = None) ->
        list[DistrictInfo]:

        """Get list of all available districts, optionally
        filtered by state."""

def get_cities(self) -> list[CityInfo]:
    """Get list of all available metropolitan cities."""

def get_categories(self) -> list[CategoryInfo]:
    """Get list of all available crime categories with
        descriptions."""

def get_years(self) -> list[int]:
    """Get list of all available years."""
```

Response Object

BharatDataResponse Class

```
from __future__ import annotations
from typing import Any, TypedDict
import json

class BharatDataResponse:
    """
    Response object returned by all BharatData query functions.

    Contains the query results, source metadata, and methods for
    converting to pandas DataFrames and exporting to various
        formats.

    Attributes
    ----------
```

```
    data : list[dict]
        List of data record dictionaries.

    meta : ResponseMeta
        Metadata about the data source, confidence, and query.

    query : QueryEcho
        Echo of the original query parameters.

    Examples
    --------
    >>> result = bd.crime.get_by_state("Maharashtra", year=2023)
    >>> result.data[0]["total_cases"]
    545862
    >>> result.meta["confidence"]["average"]
    0.98
    >>> result.to_dataframe().describe()
    """

    def __init__(self, data: list[dict], meta: dict, query: dict):
        self.data = data
        self.meta = meta
        self.query = query

    def __repr__(self) -> str:
        return (
            f"BharatDataResponse("
            f"records={len(self.data)}, "
            f"source='{self.meta['source']['name']}', "
            f"confidence={self.meta['confidence']
        ['average']:.2f})"
        )

    def __len__(self) -> int:
        return len(self.data)

    def __iter__(self):
        return iter(self.data)

    def __getitem__(self, index):
        return self.data[index]

    def to_dataframe(self, include_meta: bool = False):
        """
        Convert response data to a pandas DataFrame.

        Parameters
        ----------
        include_meta : bool, optional
            If True, includes source_url, confidence, and
        collection_date
            as columns in the DataFrame. Default: False.
```

```
    Returns
    -------
    pandas.DataFrame
        DataFrame with proper column types:
        - Numeric columns as int64 or float64
        - String columns as string dtype
        - Year as int64
        - Confidence as float64

    Raises
    ------
    ImportError
        If pandas is not installed. Install with:
        ``pip install bharatdata[pandas]``

    Examples
    --------
    >>> df = result.to_dataframe()
    >>> df.dtypes
    state              string
    year                int64
    category           string
    total_cases         int64
    rate_per_lakh     float64
    confidence        float64
    dtype: object

    >>> df = result.to_dataframe(include_meta=True)
    >>> "source_url" in df.columns
    True
    """
    try:
        import pandas as pd
    except ImportError:
        raise ImportError(
            "pandas is required for DataFrame conversion. "
            "Install it with: pip install bharatdata[pandas]"
        )

    df = pd.DataFrame(self.data)

    # Convert column types
    numeric_columns = [
        "total_cases", "rate_per_lakh", "chargesheeted",
        "convicted", "acquitted", "pending_investigation",
        "pending_trial", "confidence"
    ]
    for col in numeric_columns:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")
```

```
        if "year" in df.columns:
            df["year"] = df["year"].astype("int64")

        string_columns = ["state", "state_code", "district",
        "city",
                          "category", "category_label"]
        for col in string_columns:
            if col in df.columns:
                df[col] = df[col].astype("string")

        if not include_meta:
            meta_cols = ["source_url", "source_file",
        "collection_date"]
            df = df.drop(columns=[c for c in meta_cols if c in
        df.columns],
                         errors="ignore")

        return df

    def to_csv(self, path: str, include_meta: bool = True,
        **kwargs) -> None:
        """
        Export response data to a CSV file.

        Parameters
        ----------
        path : str
            File path for the CSV output.

        include_meta : bool, optional
            If True, includes source attribution columns. Default:
        True.

        **kwargs
            Additional arguments passed to
        ``pandas.DataFrame.to_csv()``.

        Examples
        --------
        >>> result.to_csv("maharashtra_crime_2023.csv")
        >>> result.to_csv("output.csv", index=False,
        encoding="utf-8-sig")
        """
        df = self.to_dataframe(include_meta=include_meta)
        df.to_csv(path, index=False, **kwargs)

    def to_parquet(self, path: str, **kwargs) -> None:
        """
        Export response data to a Parquet file.

        Parquet preserves column types and is efficient for large
        datasets. Ideal for use with Apache Spark, Dask, or other
        big data tools.
```

```
        Parameters
        ----------
        path : str
            File path for the Parquet output.

        **kwargs
            Additional arguments passed to
        ``pandas.DataFrame.to_parquet()``.

        Raises
        ------
        ImportError
            If pyarrow is not installed. Install with:
            ``pip install bharatdata[parquet]``

        Examples
        --------
        >>> result.to_parquet("crime_data.parquet")
        """
        try:
            import pyarrow  # noqa: F401
        except ImportError:
            raise ImportError(
                "pyarrow is required for Parquet export. "
                "Install it with: pip install bharatdata[parquet]"
            )
        df = self.to_dataframe(include_meta=True)
        df.to_parquet(path, index=False, **kwargs)

    def to_json(self, path: str, **kwargs) -> None:
        """
        Export response data to a JSON file.

        Parameters
        ----------
        path : str
            File path for the JSON output.

        Examples
        --------
        >>> result.to_json("crime_data.json")
        """
        output = {
            "data": self.data,
            "meta": self.meta,
            "query": self.query,
        }
        with open(path, "w", encoding="utf-8") as f:
            json.dump(output, f, indent=2, ensure_ascii=False)

    def to_excel(self, path: str, **kwargs) -> None:
```

```python
        """
        Export response data to an Excel file.

        Parameters
        ----------
        path : str
            File path for the Excel output (.xlsx).

        Raises
        ------
        ImportError
            If openpyxl is not installed. Install with:
            ``pip install bharatdata[excel]``

        Examples
        --------
        >>> result.to_excel("crime_data.xlsx")
        """
        try:
            import openpyxl  # noqa: F401
        except ImportError:
            raise ImportError(
                "openpyxl is required for Excel export. "
                "Install it with: pip install bharatdata[excel]"
            )
        df = self.to_dataframe(include_meta=True)
        df.to_excel(path, index=False, **kwargs)

    def to_dict(self) -> dict:
        """
        Return the complete response as a dictionary.

        Returns
        -------
        dict
            Dictionary with 'data', 'meta', and 'query' keys.
        """
        return {
            "data": self.data,
            "meta": self.meta,
            "query": self.query,
        }

    def cite(self, style: str = "apa") -> str:
        """
        Generate an academic citation for the queried data.

        Parameters
        ----------
        style : str, optional
            Citation style: "apa", "mla", "chicago", or "bibtex".
            Default: "apa"
```

```
        Returns
        -------
        str
            Formatted citation string.

        Examples
        --------
        >>> print(result.cite())
        National Crime Records Bureau. (2023). Crime in India
         2023.
        Ministry of Home Affairs, Government of India. Retrieved
         via
        BharatData (https://bharatdata.dev) on 2025-03-20.

        >>> print(result.cite("bibtex"))
        @misc{ncrb2023,
          author = {{National Crime Records Bureau}},
          title = {Crime in India 2023},
          year = {2023},
          publisher = {Ministry of Home Affairs, Government of
         India},
          note = {Retrieved via BharatData (https://
         bharatdata.dev)
                   on 2025-03-20}
        }
        """
```

Type Definitions

```python
from typing import TypedDict, Literal

class CrimeRecord(TypedDict, total=False):
    """A crime data record at the state/UT level."""
    id: str
    state: str
    state_code: str
    year: int
    category: str
    category_label: str
    total_cases: int
    rate_per_lakh: float | None
    chargesheeted: int | None
    convicted: int | None
    acquitted: int | None
    pending_investigation: int | None
    pending_trial: int | None
    source_url: str
    source_file: str
    confidence: float
```

```python
    collection_date: str

class DistrictCrimeRecord(CrimeRecord):
    """A crime data record at the district level."""
    district: str
    boundary_note: str | None

class CityCrimeRecord(CrimeRecord):
    """A crime data record at the metropolitan city level."""
    city: str
    parent_state: str

class SourceMeta(TypedDict):
    name: str
    url: str
    report_name: str
    report_year: int

class ConfidenceMeta(TypedDict):
    average: float
    minimum: float

class ResponseMeta(TypedDict):
    source: SourceMeta
    collection_date: str
    record_count: int
    total_available: int
    confidence: ConfidenceMeta
    bharatdata_version: str
    normalizer_version: str
    available_years: list[int]

class QueryEcho(TypedDict):
    function: str
    params: dict
    executed_at: str

class StateInfo(TypedDict):
    name: str
    code: str
    type: Literal["state", "union_territory"]
    available_years: list[int]

class DistrictInfo(TypedDict):
    name: str
    state: str
    available_years: list[int]

class CityInfo(TypedDict):
    name: str
    state: str
    available_years: list[int]
```

```python
class CategoryInfo(TypedDict):
    id: str
    label: str
    description: str
    levels: list[Literal["state", "district", "city"]]
    available_years: list[int]

CrimeCategory = Literal[
    "ipc-total",
    "sll-total",
    "murder",
    "kidnapping",
    "robbery",
    "theft",
    "dacoity",
    "burglary",
    "riots",
    "crimes-against-women",
    "crimes-against-children",
    "crimes-against-sc",
    "crimes-against-st",
    "crimes-against-senior-citizens",
    "cyber-crime",
    "economic-offences",
    "ndps",
    "missing-persons",
    "juveniles-ipc",
    "juveniles-sll",
    "property-stolen-recovered",
]
```

Error Handling

```python
class BharatDataError(Exception):
    """Base error class for all BharatData errors."""

    def __init__(self, message: str, code: str, suggestion: str):
        super().__init__(message)
        self.code = code
        self.suggestion = suggestion
        self.timestamp = datetime.utcnow().isoformat()

    def __str__(self) -> str:
        return f"{self.args[0]}\nSuggestion: {self.suggestion}"

class SourceNotFoundError(BharatDataError):
    """Raised when a requested entity is not found in
        BharatData."""
```

```
    def __init__(self, entity_type: str, entity_value: str):
        super().__init__(
            message=f'{entity_type} "{entity_value}" not found in
        BharatData.',
            code="SOURCE_NOT_FOUND",
            suggestion=(
                f"Check available {entity_type}s with "
                f"bd.crime.get_{entity_type}s(). "
                f'Common variations are accepted (e.g., "UP" for
        "Uttar Pradesh").'
            ),
        )

class InvalidQueryError(BharatDataError):
    """Raised when query parameters are malformed or invalid."""

    def __init__(self, message: str, suggestion: str):
        super().__init__(
            message=message,
            code="INVALID_QUERY",
            suggestion=suggestion,
        )

class RateLimitError(BharatDataError):
    """Raised when the API rate limit is exceeded."""

    def __init__(self, retry_after: int):
        self.retry_after = retry_after
        super().__init__(
            message=f"Rate limit exceeded. Try again in
        {retry_after} seconds.",
            code="RATE_LIMIT",
            suggestion=f"Wait {retry_after} seconds before
        retrying.",
        )

class DataUnavailableError(BharatDataError):
    """Raised when the BharatData API is temporarily
        unavailable."""

    def __init__(self):
        super().__init__(
            message="BharatData service is temporarily
        unavailable.",
            code="DATA_UNAVAILABLE",
            suggestion="Please try again in a few minutes.",
        )
```

Jupyter Notebook Integration

The BharatDataResponse object provides rich display in Jupyter:

```
class BharatDataResponse:
    def _repr_html_(self) -> str:
        """Rich HTML display for Jupyter notebooks."""
        if not self.data:
            return "<p>No data returned.</p>"

        try:
            df = self.to_dataframe()
            html = f"<p><strong>BharatData Response</strong>: "
            html += f"{len(self.data)} records from "
            html += f"{self.meta['source']['name']}</p>"
            html += f"<p>Confidence: {self.meta['confidence']
        ['average']:.2f} avg</p>"
            html += df.head(20)._repr_html_()
            if len(df) > 20:
                html += f"<p>... and {len(df) - 20} more records</
        p>"
            return html
        except ImportError:
            return repr(self)
```

When a BharatDataResponse is the last expression in a Jupyter cell, it renders as a formatted HTML table with metadata header showing record count, source name, and confidence score. This makes interactive exploration natural and informative.

Caching System

```
class BharatData:
    def __init__(self, cache: bool = False, cache_dir: str =
        "~/.bharatdata/cache",
                 cache_ttl: int = 86400):
        """
        Local caching for repeated queries during development.

        When enabled, query results are stored on disk as JSON
        files.
        Subsequent identical queries return cached results without
        hitting the API. Cache entries expire after cache_ttl
        seconds.

        Cache is invalidated when:
        - The entry is older than cache_ttl
        - The user explicitly calls bd.clear_cache()
        - BharatData detects that upstream data has been updated
          (checked via ETag/Last-Modified headers)
```

```
        Cache keys are SHA-256 hashes of the normalized query
         parameters,
        ensuring that bd.crime.get_by_state("MH") and
        bd.crime.get_by_state("Maharashtra") produce the same
         cache key
        after name resolution.
        """
```

Usage Examples — Complete Workflows

Machine Learning Pipeline

```python
from bharatdata import BharatData
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Initialize client with caching for iterative development
bd = BharatData(cache=True)

# Get 3 years of state-level crime data
df = bd.crime.get_all(
    year_range=(2021, 2023),
    level="state"
).to_dataframe()

# Feature engineering
features = df.pivot_table(
    values="total_cases",
    index=["state", "year"],
    columns="category",
    aggfunc="sum"
).reset_index()

# Train-test split
X = features.drop(columns=["state", "year", "ipc-total"])
y = features["ipc-total"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
        test_size=0.2)

# Train model
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print(f"R² Score: {model.score(X_test, y_test):.3f}")
```

Research Analysis

```python
from bharatdata import BharatData
import matplotlib.pyplot as plt

bd = BharatData()

# Compare cyber crime trends across top states
result = bd.crime.compare(
    ["Maharashtra", "Karnataka", "Telangana", "Uttar Pradesh",
        "Tamil Nadu"],
    years=[2021, 2022, 2023],
    category="cyber-crime"
)
df = result.to_dataframe()

# Create trend visualization
pivot = df.pivot_table(values="total_cases", index="year",
        columns="state")
pivot.plot(kind="line", marker="o", figsize=(10, 6))
plt.title("Cyber Crime Trends: Top 5 States (2021-2023)")
plt.ylabel("Total Cases Reported")
plt.xlabel("Year")
plt.legend(title="State")
plt.tight_layout()
plt.savefig("cyber_crime_trends.png", dpi=150)
plt.show()

# Generate citation
print("\nCitation:")
print(result.cite("apa"))
```

This specification is the authoritative reference for BharatData Python SDK design. All implementation must follow this specification exactly. The Python SDK mirrors the TypeScript SDK in logical structure but follows Python conventions throughout.

Document 6: Data Pipeline Specification

```markdown

# BharatData — Data Pipeline Specification

**Document ID:** DOC-06
**Version:** 1.0
**Status:** Active
**Created:** 2025-03-20
**Audience:** Builder, contributors, data pipeline specialist
```

```
agent
**References:** FIELD_MAPPINGS.md, states.json,
QUALITY_STANDARDS.md

---

## Pipeline Architecture Overview

The BharatData data pipeline transforms raw Indian government data
files
into clean, normalized, queryable records. It operates in four
sequential
stages, each with defined inputs, outputs, quality checks, and
failure
handling procedures.

```mermaid
graph TD
    A[Government Data Sources] -->|"Download files<br/>Rate: max 1
req/sec"| B[Stage 1: Ingester]
    B -->|"Raw files +<br/>metadata"| C[Stage 2: Parser]
    C -->|"Structured records +<br/>confidence scores"| D[Stage 3:
Normalizer]
    D -->|"Clean canonical<br/>records"| E[Stage 4: Storage]
    E -->|"Indexed records"| F[REST API]

    G[Monitoring] -->|"Daily health check"| A
    G -->|"Weekly quality check"| E

    H[states.json] -->|"Canonical mappings"| D
    I[FIELD_MAPPINGS.md] -->|"Column translations"| C

    style A fill:#ff9999
    style B fill:#99ccff
    style C fill:#99ccff
    style D fill:#99ccff
    style E fill:#99ccff
    style F fill:#99ff99
    style G fill:#ffcc99
```
```

Pipeline Principles

1. **Data flows one direction:** Source → Ingester → Parser → Normalizer → Storage
2. **Each stage is independently testable** with defined inputs and outputs
3. **No stage silently drops data.** Failures are logged with full context.
4. **Confidence scores flow through the entire pipeline** from extraction to serving

5. **Source attribution is attached early** and preserved through every stage
6. **The pipeline is idempotent.** Running it twice with the same input produces identical output without duplicate records.

Stage 1: Ingester

Purpose

The ingester is responsible for downloading raw data files from government sources and storing them with complete metadata. It is the only component that communicates with external government servers.

Input

- NCRB Additional Tables page URL: `https://www.ncrb.gov.in/crime-in-india-additional-table?year={year}&category=`
- Year parameter: 2021, 2022, or 2023 (expandable)
- File download links from the page

Output

For each downloaded file:

```
{
  "ingestion_id": "uuid-v4",
  "source_url": "https://www.ncrb.gov.in/...",
  "source_page": "crime-in-india-additional-table?year=2023",
  "downloaded_at": "2025-03-20T06:00:00Z",
  "file_hash_sha256": "abc123...",
  "file_size_bytes": 11960,
  "file_format": "xlsx",
  "original_filename": "State-UT-wise IPC Crimes During
        2023.xlsx",
  "stored_path": "data/raw/ncrb-crime-in-india/2023/states-
        uts/...",
  "http_status": 200,
  "is_new_data": true,
  "previous_hash": null
}
```

Behavior

Normal Flow

```
sequenceDiagram
    participant I as Ingester
    participant N as NCRB Website
    participant S as Local Storage
    participant D as Database

    I->>N: GET additional tables page (year=2023)
    N-->>I: HTML with download links
    I->>I: Extract file download URLs
    loop For each file
        I->>I: Check rate limit (1 req/sec)
        I->>I: sleep(1)
        I->>N: GET file download URL
        N-->>I: File content
        I->>I: Calculate SHA-256 hash
        I->>D: Check if hash matches previous download
        alt Hash is new
            I->>S: Store raw file with metadata
            I->>D: Record ingestion event
        else Hash unchanged
            I->>D: Log "no update" event
        end
    end
```

Fault Tolerance

```
Attempt 1: Request fails
  → Log: timestamp, URL, error type, HTTP status
  → Wait 60 seconds

Attempt 2: Request fails
  → Log: timestamp, URL, error type, HTTP status
  → Wait 300 seconds (5 minutes)

Attempt 3: Request fails
  → Log: timestamp, URL, error type, HTTP status
  → Create GitHub issue with label "data-source-alert"
  → Include: URL, all three error details, last successful
download date
  → STOP retrying for this source
```

Rate Limiting

```python
import time

class RateLimiter:
```

```python
    """Ensures maximum 1 request per second per host."""

    def __init__(self, min_interval: float = 1.0):
        self.min_interval = min_interval
        self._last_request: dict[str, float] = {}

    def wait(self, host: str) -> None:
        """Block until the minimum interval has passed."""
        now = time.monotonic()
        last = self._last_request.get(host, 0)
        elapsed = now - last
        if elapsed < self.min_interval:
            time.sleep(self.min_interval - elapsed)
        self._last_request[host] = time.monotonic()
```

Handling Known NCRB Issues

Based on Stage 1 discovery findings:

1. **Timestamp prefix in 2021 filenames:** Some 2021 files have timestamp prefixes in their filenames that 2022 and 2023 files do not. The ingester strips these prefixes when storing files but preserves the original filename in metadata.

2. **Inconsistent URL patterns:** The NCRB website has been redesigned multiple times. File download URLs may change format between years. The ingester extracts download URLs from the current page HTML rather than constructing them from a pattern.

3. **Session-based access:** Some NCRB pages require maintaining a session. The ingester uses a requests.Session() to maintain cookies across multiple downloads within the same ingestion run.

Stage 2: Parser

Purpose

The parser extracts structured data records from raw downloaded files. It handles both Excel files and PDF files, applying file-type-specific extraction logic and assigning confidence scores to every extracted record.

Input

- Raw files from the ingester (Excel .xlsx/.xls or PDF)
- Field mappings from `docs/data-schema/FIELD_MAPPINGS.md`

- File metadata from the ingestion record

Output

For each extracted record:

```
{
  "raw_values": {
    "State/UT": "ANDHRA PRADESH",
    "Year": "2023",
    "Total Cognizable IPC Crimes": "1,23,456"
  },
  "parsed_values": {
    "state_raw": "ANDHRA PRADESH",
    "year": 2023,
    "total_cases": 123456
  },
  "confidence": 1.0,
  "source_file": "State-UT-wise IPC Crimes During 2023.xlsx",
  "source_sheet": "Sheet1",
  "source_row": 5,
  "parsing_notes": [],
  "field_mapping_version": "1.0"
}
```

Excel Parser

```
graph TD
    A[Open Excel file] --> B[Detect header row]
    B --> C[Map columns to canonical fields]
    C --> D[Extract data rows]
    D --> E{Is row a summary/total?}
    E -->|Yes| F[Flag as summary, skip for data]
    E -->|No| G[Parse each cell]
    G --> H{Cell parsing successful?}
    H -->|Yes| I[Assign high confidence]
    H -->|No| J[Log failure, reduce confidence]
    I --> K[Output parsed record]
    J --> K
```

Header Detection

Based on Stage 1 findings, NCRB Excel files have varying header structures:

```python
def detect_header_row(filepath: str) -> int:
    """
    Detect the actual header row in an NCRB Excel file.

    NCRB files may have:
```

```
    - Title text in rows 0-2
    - Blank rows between title and headers
    - Merged cells in header rows
    - The actual data starting 3-5 rows down

    Strategy: Find the first row where at least 3 cells match
    known column name patterns from FIELD_MAPPINGS.
    """
    known_patterns = [
        "state", "ut", "year", "total", "ipc", "sll",
        "murder", "theft", "crime", "cases", "sl."
    ]
    # ... implementation
```

Column Mapping

The parser uses `FIELD_MAPPINGS.md` to translate raw column names to canonical BharatData field names:

```
FIELD_MAPPINGS = {
    # State/UT identification
    "State/UT": "state_raw",
    "States/UTs": "state_raw",
    "STATE/UT": "state_raw",
    "Sl. No.": "_skip",
    "S.No.": "_skip",
    "Sl.No": "_skip",

    # Crime counts
    "Total Cognizable IPC Crimes": "total_cases",
    "Total IPC Crimes": "total_cases",
    "Total Cases": "total_cases",
    "Incidence": "total_cases",

    # Rates
    "Crime Rate": "rate_per_lakh",
    "Rate of Total Cognizable Crimes (IPC)": "rate_per_lakh",

    # Disposal
    "Cases for Investigation": "cases_for_investigation",
    "Cases Charge Sheeted": "chargesheeted",
    "Cases Conviction": "convicted",
    "Cases Acquittal/Discharge": "acquitted",
    "Cases Pending Investigation": "pending_investigation",
    "Cases Pending Trial": "pending_trial",
}
```

Fields mapped to `"_skip"` are ignored (serial numbers, blank columns). Unmapped column names are logged as warnings and stored in raw metadata.

Summary Row Detection

```python
SUMMARY_INDICATORS = [
    "total", "grand total", "all india", "total (all india)",
    "total states", "total uts", "total (states)",
    "total (uts)", "total (states+uts)"
]

def is_summary_row(row_values: list) -> bool:
    """Check if a row is a summary/total row."""
    first_text_cell = next(
        (str(v).strip().lower() for v in row_values
         if isinstance(v, str) and v.strip()),
        ""
    )
    return first_text_cell in SUMMARY_INDICATORS
```

Summary rows are extracted and stored separately as aggregation records. They are not included in the state-level or district-level data records.

Confidence Assignment

```python
def calculate_confidence(
    parsing_notes: list[str],
    source_type: str
) -> float:
    """
    Calculate confidence score based on parsing quality.

    Base confidence:
      Excel files: 1.0
      PDF files: 0.9

    Deductions:
      Each minor issue: -0.05 (e.g., number format conversion)
      Each moderate issue: -0.1 (e.g., ambiguous cell value)
      Each major issue: -0.2 (e.g., failed cell parsing)

    Minimum: 0.0
    """
    if source_type == "excel":
        confidence = 1.0
    elif source_type == "pdf":
        confidence = 0.9
    else:
        confidence = 0.7

    for note in parsing_notes:
        if note.startswith("MINOR:"):
            confidence -= 0.05
```

```
        elif note.startswith("MODERATE:"):
            confidence -= 0.1
        elif note.startswith("MAJOR:"):
            confidence -= 0.2

    return max(0.0, round(confidence, 2))
```

PDF Parser

The PDF parser handles metropolitan city data that is not available in the Additional Tables Excel files.

Based on Stage 1 discovery, the PDF parser:

1. Uses `pdfplumber` for text extraction
2. Identifies table boundaries on each page
3. Handles the 34 metropolitan city rows
4. Assigns base confidence of 0.9 (lower than Excel due to extraction uncertainty)
5. All 9 structured JSON files produced during Stage 1 were verified for accuracy

```
graph TD
    A[Open PDF] --> B[Iterate pages]
    B --> C[Extract tables using pdfplumber]
    C --> D{Table found?}
    D -->|Yes| E[Identify header row]
    D -->|No| F[Log: no table on page]
    E --> G[Map columns to canonical fields]
    G --> H[Extract city rows]
    H --> I[Verify against known 34 cities list]
    I --> J[Assign confidence 0.9 base]
    J --> K[Output parsed record]
```

Parse Failure Logging

Every parse failure is logged with complete context:

```
@dataclass
class ParseFailure:
    source_file: str
    source_sheet: str | None
    row_number: int
    column_name: str
    raw_value: str
    expected_type: str
    error_message: str
    timestamp: str
```

```python
    def to_dict(self) -> dict:
        return asdict(self)
```

Parse failures are collected and stored in a `parse_failures` table in the database. They are also written to a JSON log file for review.

Stage 3: Normalizer

Purpose

The normalizer transforms parsed records into canonical form by resolving geographic name variations, converting data types, standardizing null representations, handling historical boundary changes, and enriching records with source attribution metadata.

Input

- Parsed records from Stage 2
- Canonical mappings from `data/canonical-mappings/states.json`
- Quality standards from `docs/internal/QUALITY_STANDARDS.md`

Output

Fully normalized records ready for database storage:

```json
{
  "state": "Andhra Pradesh",
  "state_code": "AP",
  "entity_type": "state",
  "year": 2023,
  "category": "ipc-total",
  "category_label": "Total IPC Crimes",
  "total_cases": 123456,
  "rate_per_lakh": 157.3,
  "chargesheeted": null,
  "convicted": null,
  "acquitted": null,
  "pending_investigation": null,
  "pending_trial": null,
  "confidence": 1.0,
  "source_url": "https://www.ncrb.gov.in/crime-in-india-
        additional-table?year=2023",
  "source_file": "State-UT-wise IPC Crimes During 2023.xlsx",
  "report_name": "Crime in India 2023",
  "publishing_body": "National Crime Records Bureau",
  "collection_date": "2025-03-20T06:00:00Z",
  "normalizer_version": "1.0.0",
```

```
  "ingestion_batch_id": "uuid-v4"
}
```

Normalization Pipeline

```
graph TD
    A[Parsed Record] --> B["Step 1: State Name Resolution<br/
>(states.json lookup)"]
    B --> C["Step 2: District Name Resolution<br/>(if district-
level data)"]
    C --> D["Step 3: City Name Resolution<br/>(if city-level
data)"]
    D --> E["Step 4: Data Type Enforcement<br/>(strings → numbers,
nulls)"]
    E --> F["Step 5: Category Mapping<br/>(raw category →
canonical ID)"]
    F --> G["Step 6: Historical Annotation<br/>(Telangana, Ladakh,
bifurcations)"]
    G --> H["Step 7: Deduplication Check<br/>(against existing
records)"]
    H --> I["Step 8: Confidence Adjustment<br/>(based on
normalization quality)"]
    I --> J["Step 9: Source Attribution<br/>(attach full
provenance)"]
    J --> K["Step 10: Range Validation<br/>(flag implausible
values)"]
    K --> L[Normalized Record]
```

Step 1: State Name Resolution

```python
import json

class StateNormalizer:
    """Resolves state name variations to canonical forms."""

    def __init__(self, mappings_path: str = "data/canonical-
        mappings/states.json"):
        with open(mappings_path, "r") as f:
            self.mappings = json.load(f)

        # Build reverse lookup: variation → canonical name
        self._lookup: dict[str, str] = {}
        self._codes: dict[str, str] = {}
        for entry in self.mappings["states"]:
            canonical = entry["canonical"]
            self._codes[canonical] = entry["code"]
            for variation in entry["variations"]:
                self._lookup[variation.lower().strip()] =
        canonical

    def resolve(self, raw_name: str) -> tuple[str, str, float]:
```

```python
        """
        Resolve a raw state name to its canonical form.

        Returns:
            (canonical_name, state_code, confidence_impact)

        If the name is found in mappings: confidence_impact = 0.0
        If the name is NOT found: confidence_impact = -0.2
        """
        normalized = raw_name.lower().strip()

        if normalized in self._lookup:
            canonical = self._lookup[normalized]
            return canonical, self._codes[canonical], 0.0

        # Not found — log and pass through
        self._log_unmapped(raw_name)
        return raw_name, "UNKNOWN", -0.2

    def _log_unmapped(self, raw_name: str) -> None:
        """Append unmapped entity to unmapped-entities.json."""
        # ... implementation
```

Step 5: Category Mapping

```python
CATEGORY_MAPPINGS = {
    # Raw category descriptors → canonical category IDs
    "Total Cognizable IPC Crimes": {
        "id": "ipc-total",
        "label": "Total IPC Crimes",
    },
    "Total SLL Crimes": {
        "id": "sll-total",
        "label": "Total SLL Crimes",
    },
    "Crime Against Women": {
        "id": "crimes-against-women",
        "label": "Crimes Against Women",
    },
    "Crime Against Children": {
        "id": "crimes-against-children",
        "label": "Crimes Against Children",
    },
    "Crime/Atrocities against Scheduled Castes": {
        "id": "crimes-against-sc",
        "label": "Crimes Against Scheduled Castes",
    },
    "Crime/Atrocities against Scheduled Tribes": {
        "id": "crimes-against-st",
        "label": "Crimes Against Scheduled Tribes",
    },
```

```
    "Crime Against Senior Citizens": {
        "id": "crimes-against-senior-citizens",
        "label": "Crimes Against Senior Citizens",
    },
    "Cyber Crimes": {
        "id": "cyber-crime",
        "label": "Cyber Crimes",
    },
    # ... additional mappings
}
```

Step 6: Historical Annotations

```
def annotate_historical_boundaries(record: dict) -> dict:
    """Add boundary change annotations where applicable."""
    state = record.get("state")
    year = record.get("year")

    if state == "Telangana" and year and year < 2014:
        record["boundary_note"] = (

        "Telangana was part of Andhra Pradesh until June 2014. "
            "Pre-2014 data does not exist for Telangana as a
        separate entity."
        )

    if state == "Andhra Pradesh" and year and year < 2014:
        record["boundary_note"] = (
            "Pre-2014 data for Andhra Pradesh includes the
        Telangana region "
            "which became a separate state on June 2, 2014."
        )

    if state == "Ladakh" and year and year < 2019:
        record["boundary_note"] = (
            "Ladakh was part of Jammu & Kashmir until October
        2019. "

        "Pre-2019 data does not exist for Ladakh as a separate
        entity."
        )

    if state == "Jammu & Kashmir" and year and year >= 2019:
        record["boundary_note"] = (
            "From October 31, 2019, Jammu & Kashmir is a Union
        Territory. "
            "Ladakh was carved out as a separate Union Territory."
        )

    return record
```

Step 10: Range Validation

```python
PLAUSIBLE_RANGES = {
    "total_cases": (0, 5_000_000),
        # No state has > 5M total crimes
    "rate_per_lakh": (0, 5000),           # Rate sanity check
    "year": (2000, 2030),                  # Year sanity check
    "confidence": (0.0, 1.0),             # Score bounds
}

def validate_ranges(record: dict) -> list[str]:
    """Flag values outside plausible ranges."""
    warnings = []
    for field, (low, high) in PLAUSIBLE_RANGES.items():
        value = record.get(field)
        if value is not None and (value < low or value > high):
            warnings.append(
                f"RANGE_WARNING: {field}={value} outside "
                f"plausible range [{low}, {high}]"
            )
    return warnings
```

Stage 4: Storage

Purpose

Store normalized records in PostgreSQL with full data lineage, proper indexing for query performance, and support for future dataset expansion.

Database Schema

```sql
-- Core data sources registry
CREATE TABLE data_sources (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            TEXT NOT NULL,
    source_url      TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    data_format     TEXT NOT NULL,      -- 'excel', 'pdf', 'csv'
    update_frequency TEXT NOT NULL,     -- 'annual', 'monthly',
        etc.
    last_ingested_at TIMESTAMPTZ,
    status          TEXT NOT NULL DEFAULT 'active',
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Ingestion run log
CREATE TABLE ingestion_runs (
```

```sql
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id       UUID REFERENCES data_sources(id),
    started_at      TIMESTAMPTZ NOT NULL,
    completed_at    TIMESTAMPTZ,
    status          TEXT NOT NULL,       -- 'running', 'success',
        'failed'
    files_processed INT DEFAULT 0,
    records_extracted INT DEFAULT 0,
    records_stored  INT DEFAULT 0,
    parse_failures  INT DEFAULT 0,
    error_message   TEXT,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- State-level crime records
CREATE TABLE crime_records_state (
    id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    ingestion_run_id      UUID REFERENCES ingestion_runs(id),
    state                 TEXT NOT NULL,
    state_code            TEXT NOT NULL,
    year                  INT NOT NULL,
    category              TEXT NOT NULL,
    category_label        TEXT NOT NULL,
    total_cases           INT,
    rate_per_lakh         FLOAT,
    chargesheeted         INT,
    convicted             INT,
    acquitted             INT,
    pending_investigation INT,
    pending_trial         INT,
    confidence            FLOAT NOT NULL,
    source_url            TEXT NOT NULL,
    source_file           TEXT NOT NULL,
    report_name           TEXT NOT NULL,
    publishing_body       TEXT NOT NULL,
    collection_date       TIMESTAMPTZ NOT NULL,
    normalizer_version    TEXT NOT NULL,
    boundary_note         TEXT,
    created_at            TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT confidence_range CHECK (confidence >= 0.0 AND
        confidence <= 1.0),
    CONSTRAINT year_range CHECK (year >= 2000 AND year <= 2030)
);

-- District-level crime records
CREATE TABLE crime_records_district (
    id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    ingestion_run_id      UUID REFERENCES ingestion_runs(id),
    state                 TEXT NOT NULL,
```

```sql
    state_code            TEXT NOT NULL,
    district              TEXT NOT NULL,
    year                  INT NOT NULL,
    category              TEXT NOT NULL,
    category_label        TEXT NOT NULL,
    total_cases           INT,
    rate_per_lakh         FLOAT,
    confidence            FLOAT NOT NULL,
    source_url            TEXT NOT NULL,
    source_file           TEXT NOT NULL,
    report_name           TEXT NOT NULL,
    publishing_body       TEXT NOT NULL,
    collection_date       TIMESTAMPTZ NOT NULL,
    normalizer_version    TEXT NOT NULL,
    boundary_note         TEXT,
    created_at            TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT confidence_range CHECK (confidence >= 0.0 AND
        confidence <= 1.0)
);

-- Metropolitan city crime records
CREATE TABLE crime_records_city (
    id                    UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
    ingestion_run_id      UUID REFERENCES ingestion_runs(id),
    city                  TEXT NOT NULL,
    parent_state          TEXT NOT NULL,
    year                  INT NOT NULL,
    category              TEXT NOT NULL,
    category_label        TEXT NOT NULL,
    total_cases           INT,
    rate_per_lakh         FLOAT,
    confidence            FLOAT NOT NULL,
    source_url            TEXT NOT NULL,
    source_file           TEXT NOT NULL,
    report_name           TEXT NOT NULL,
    publishing_body       TEXT NOT NULL,
    collection_date       TIMESTAMPTZ NOT NULL,
    normalizer_version    TEXT NOT NULL,
    created_at            TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT confidence_range CHECK (confidence >= 0.0 AND
        confidence <= 1.0)
);

-- Parse failure log
CREATE TABLE parse_failures (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    source_file     TEXT NOT NULL,
    source_sheet    TEXT,
```

```sql
    row_number      INT,
    column_name     TEXT,
    raw_value       TEXT,
    expected_type   TEXT,
    error_message   TEXT NOT NULL,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Normalization decision log
CREATE TABLE normalization_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    record_id       UUID,
    field           TEXT NOT NULL,
    input_value     TEXT NOT NULL,
    output_value    TEXT NOT NULL,
    mapping_source  TEXT NOT NULL,
    confidence_impact FLOAT DEFAULT 0,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Indexes

```sql
-- State-level queries (most common)
CREATE INDEX idx_crime_state_year ON crime_records_state(state,
        year);
CREATE INDEX idx_crime_year ON crime_records_state(year);
CREATE INDEX idx_crime_category ON crime_records_state(category);
CREATE INDEX idx_crime_state_cat_year ON
        crime_records_state(state, category, year);
CREATE INDEX idx_crime_confidence ON
        crime_records_state(confidence);

-- District-level queries
CREATE INDEX idx_district_state ON crime_records_district(state);
CREATE INDEX idx_district_state_year ON
        crime_records_district(state, year);
CREATE INDEX idx_district_district ON
        crime_records_district(district);
CREATE INDEX idx_district_category ON
        crime_records_district(category);

-- City-level queries
CREATE INDEX idx_city_name ON crime_records_city(city);
CREATE INDEX idx_city_year ON crime_records_city(city, year);

-- Confidence filtering (exclude low-confidence records from API)
CREATE INDEX idx_state_confidence_filter
    ON crime_records_state(confidence)
    WHERE confidence >= 0.5;

CREATE INDEX idx_district_confidence_filter
```

```
    ON crime_records_district(confidence)
    WHERE confidence >= 0.5;
```

Storage Estimates

```
State-level records:
  36 states × ~20 categories × 3 years ≈ 2,160 records
  At ~1 KB/record (including all metadata) ≈ 2.2 MB

District-level records:
  780 districts × 10 categories × 1 year ≈ 7,800 records
  At ~1 KB/record ≈ 7.8 MB

City-level records:
  34 cities × ~15 categories × 3 years ≈ 1,530 records
  At ~1 KB/record ≈ 1.5 MB

Normalization log: ~12,000 entries ≈ 2 MB
Parse failure log: ~500 entries ≈ 0.5 MB
Ingestion metadata: ~1 MB
Indexes: ~5 MB

Total estimated: ~20 MB
Neon free tier limit: 500 MB
Utilization: ~4%
```

This leaves substantial room for growth. Adding 5 more data sources of similar size would use approximately 120 MB total, still well within the free tier limit.

Monitoring

Daily Health Check

```yaml
# .github/workflows/daily-health-check.yml
name: Daily Data Source Health Check
on:
  schedule:
    - cron: '0 6 * * *'  # 06:00 UTC daily (11:30 AM IST)
  workflow_dispatch: # Allow manual trigger

jobs:
  check-sources:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```yaml
      - name: Check NCRB source URLs
        run: python scripts/health_check.py

      - name: Create issue if check fails
        if: failure()
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: '🔴 Data Source Health Check Failed',
              body: 'Daily health check detected issues. Check the
        workflow run for details.',
              labels: ['data-source-alert']
            })
```

Weekly Quality Check

```yaml
# .github/workflows/weekly-quality-check.yml
name: Weekly Data Quality Check
on:
  schedule:
    - cron: '0 8 * * 1'  # 08:00 UTC every Monday
  workflow_dispatch:

jobs:
  quality-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run quality checks
        run: python scripts/bharatdata_quality_check.py --db-url $
        {{ secrets.DATABASE_URL }}

      - name: Create report issue
        uses: actions/github-script@v7
        with:
          script: |
            const report = require('./quality_report.json');
            github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: `📊 Weekly Quality Report: ${report.status}`,
              body: JSON.stringify(report, null, 2),
              labels: ['quality-report']
            })
```

This specification is the authoritative reference for BharatData's data pipeline. Every pipeline component must follow this specification exactly. Changes to the pipeline architecture require updating this document first.

Document 7: Normalization Guide, Agent Configuration, Content Strategy, and Brand Minimums

Given the extreme length of the previous documents and to ensure you receive the complete set, I am combining the remaining shorter documents into this final section.

```markdown

# BharatData — Data Schema & Normalization Guide

**Document ID:** DOC-07
**Version:** 1.0
**Status:** Active
**Created:** 2025-03-20

---

## Canonical Mappings File

**Location:** `data/canonical-mappings/states.json`

This file is one of the most valuable artifacts of the BharatData project.
It is a public, version-controlled JSON file that maps every known variation
of Indian state and union territory names to a single canonical form. Other
developers can use this file independently of the BharatData SDK.

### Structure

```json
{
  "version": "1.0.0",
  "last_updated": "2025-03-20",
  "entity_count": 36,
  "states": [
    {
      "canonical": "Andhra Pradesh",
      "code": "AP",
      "type": "state",
      "census_code": "28",
      "variations": [

```
        "Andhra Pradesh", "ANDHRA PRADESH", "AP", "A.P.",
        "Andhra", "Andhra Pr.", "andhra pradesh"
      ],
      "exists_from": null,
      "boundary_changes": [
        {
          "date": "2014-06-02",
          "type": "bifurcation",
          "description": "Telangana carved out as separate state",
          "note": "Pre-2014 AP data includes Telangana region"
        }
      ]
    },
    {
      "canonical": "Telangana",
      "code": "TG",
      "type": "state",
      "census_code": "36",
      "variations": [
        "Telangana", "TELANGANA", "TG", "Telengana", "telangana"
      ],
      "exists_from": "2014-06-02",
      "boundary_changes": []
    },
    {
      "canonical": "Jammu & Kashmir",
      "code": "JK",
      "type": "union_territory",
      "census_code": "01",
      "variations": [
        "Jammu & Kashmir", "JAMMU & KASHMIR", "J&K", "JK",
        "Jammu and Kashmir", "JAMMU AND KASHMIR", "J & K"
      ],
      "exists_from": null,
      "boundary_changes": [
        {
          "date": "2019-10-31",
          "type": "reorganization",
          "description": "Changed from State to Union Territory.
Ladakh carved out as separate UT.",
          "note": "Pre-2019 J&K data includes Ladakh region"
        }
      ]
    },
    {
      "canonical": "Ladakh",
      "code": "LA",
      "type": "union_territory",
      "census_code": "37",
      "variations": [
        "Ladakh", "LADAKH", "LA"
      ],
```

```
      "exists_from": "2019-10-31",
      "boundary_changes": []
    },
    {
      "canonical": "Dadra and Nagar Haveli and Daman and Diu",
      "code": "DN",
      "type": "union_territory",
      "census_code": "26",
      "variations": [
        "Dadra and Nagar Haveli and Daman and Diu",
        "D&N Haveli and Daman & Diu",
        "DNH & DD", "D & N Haveli", "Daman & Diu",
        "Dadra & Nagar Haveli", "DNH",
        "DADRA & NAGAR HAVELI AND DAMAN & DIU",
        "D&NH and D&D"
      ],
      "exists_from": "2020-01-26",
      "boundary_changes": [
        {
          "date": "2020-01-26",
          "type": "merger",
          "description": "Dadra and Nagar Haveli merged with Daman
and Diu into a single UT",
          "note": "Pre-2020 data may list these as separate UTs"
        }
      ]
    }
  ]
}
```


Resolution Algorithm

```
Input: raw state name from government file

Step 1: Trim whitespace, convert to lowercase
Step 2: Look up in variations index (case-insensitive)
Step 3a: If found → return canonical name + code +
confidence_impact=0
Step 3b: If NOT found → log to unmapped-entities.json
         → return raw name + code="UNKNOWN" +
confidence_impact=-0.2
```

All 36 Canonical Entities

```
STATES (28):
  Andhra Pradesh, Arunachal Pradesh, Assam, Bihar, Chhattisgarh,
  Goa, Gujarat, Haryana, Himachal Pradesh, Jharkhand, Karnataka,
  Kerala, Madhya Pradesh, Maharashtra, Manipur, Meghalaya,
  Mizoram, Nagaland, Odisha, Punjab, Rajasthan, Sikkim,
  Tamil Nadu, Telangana, Tripura, Uttar Pradesh, Uttarakhand,
  West Bengal
```

```
UNION TERRITORIES (8):
  Andaman & Nicobar Islands, Chandigarh,
  Dadra and Nagar Haveli and Daman and Diu,
  Delhi, Jammu & Kashmir, Ladakh, Lakshadweep, Puducherry
```

Data Type Conversion Standards

Source Format	Canonical Format	Example
`"1,23,456"` (Indian notation)	`int: 123456`	Detect by comma at position 2 from right
`"1,234,567"` (International)	`int: 1234567`	Detect by comma at position 3 from right
`"12.5%"`	`float: 12.5` + `value_type: "percentage"`	Strip % symbol
`"2.5 crore"`	`int: 25000000`	Multiply by 10,000,000
`"1.5 lakh"`	`int: 150000`	Multiply by 100,000
`""` (empty)	`null` + `original: ""`	Map to null
`"-"`	`null` + `original: "-"`	Map to null
`"NA"` / `"N/A"`	`null` + `original: "NA"`	Map to null
`"0"`	`int: 0`	Preserve as zero (NOT null)
`"2022-23"` (fiscal)	`year: 2023, year_type: "fiscal"`	Use end year
`"2023"`	`int: 2023`	Direct conversion

Confidence Scoring Detailed Rules

Scenario	Base Score	Adjustments	Final Range
Excel file, clean parse, all fields mapped	1.0	None	1.0
Excel file, timestamp prefix in filename	1.0	-0.05 (minor handling)	0.95

Scenario	Base Score	Adjustments	Final Range
Excel file, number format conversion needed	1.0	-0.05 per conversion	0.9-0.95
Excel file, unmapped state name	1.0	-0.2 (unmapped)	0.8
PDF extraction, all cells verified	0.9	None	0.9
PDF extraction, some cells ambiguous	0.9	-0.1 per ambiguous cell	0.7-0.8
Cross-file validation mismatch	varies	-0.1 (discrepancy)	varies
Statistical outlier detected	varies	-0.1 (flagged)	varies

Serving threshold: 0.5

Records at or above 0.5 are served through the API with their confidence score visible to users. Records below 0.5 are stored in the database for debugging and future improvement but are excluded from all API responses.

```markdown

# BharatData — Content & Distribution Strategy

**Document ID:** DOC-08
**Version:** 1.0
**Status:** Active
**Created:** 2025-03-20
**Visibility:** Internal (not published)

---

## Blog Post Plan

### Post 1: The Discovery Story
**Title:** "NCRB Publishes Crime Data in Two Formats — And Almost Nobody
```

Knows About the Easy One"
Draft during: Pipeline building (capture screenshots as you work)
Publish: Launch week
Platform: Dev.to + Medium
Angle: You went to extract NCRB data from their notorious PDFs and
discovered that they also publish structured downloadable files through
an "Additional Tables" page. Show the contrast between the hostile PDF
and the clean structured files. This is genuinely useful information
for journalists and researchers who may not know this access method exists.
Call to action: "I built BharatData to make this even easier. Try it."

Post 2: The Normalization Challenge
Title: "36 States, 47 Name Variations, and the Geographic Identity
Crisis of Indian Government Data"
Publish: 2 weeks after launch
Platform: Dev.to + Hashnode
Angle: Deep dive into the state name normalization problem. Show
specific examples: "AP" vs "Andhra Pradesh" vs "ANDHRA PRADESH". Show
the Telangana bifurcation challenge. Show the Dadra and Nagar Haveli
merger complexity. Make it technical enough to attract contributors.

Post 3: The Data Story
Title: "What 3 Years of NCRB Data Actually Reveals About Cyber Crime
in India"
Publish: 1 month after launch
Platform: Medium + LinkedIn
Angle: Use BharatData to produce a genuine analysis. Which states
lead in cyber crime? What are the trends? How does the conviction rate
compare to reporting rate? This demonstrates the product by using it.

Post 4: The AI Story (Stage 4)
Title: "Teaching an AI to Answer Questions About Indian Crime Data
Without Making Things Up"
Publish: When AI layer ships
Angle: Prompt engineering for domain-specific Indian civic

data.
How you prevent hallucination. What fails and why.

Post 5: The Retrospective
Title: "Building BharatData: What Worked, What Broke, What I Learned"
Publish: 6-month mark
Angle: Honest retrospective with metrics. Built from Project Journal.

Community Engagement

Communities to Join
- **DataMeet** — Indian open data community. Mailing list and Slack.
  Introduce yourself as a student building an open data tool.
- **r/developersIndia** — 330,000+ members. Post technical content.
- **Indian Data Journalism Twitter** — Follow and engage with journalists
  at IndiaSpend, The Wire, Article 14.
- **Dev.to** — Post technical articles. Engage with Indian developer tag.

Engagement Rules
- Be present in at least one community daily
- Contribute genuinely: answer questions, share insights
- Promote BharatData only when directly relevant
- Never spam, never cross-post identical messages
- Build relationships before asking for anything

Launch Checklist

Hacker News
- Title: "Show HN: Bharat Data — Open source SDK for Indian government
  crime data (TypeScript + Python)"
- First comment: Honest scope description. What data is available.
  Why you built it. What comes next.
- Respond to every comment.

Reddit
- r/developersIndia: Developer angle. Show code examples.
- r/programming: Technical angle. Focus on parsing challenge.
- r/datascience: Python angle. Show DataFrame workflow.

Twitter/X
- Thread: Problem → Screenshot of messy PDF → Clean API output →

```
  Code example → Playground link → GitHub link
- Tag relevant accounts if appropriate

### LinkedIn
- Impact angle: "Indian public data should be accessible to
everyone."
- Career angle for recruiter visibility

### Dev.to
- Full technical article (repurpose Post 1)
```

BharatData — Brand Minimums Checklist

Document ID: DOC-09
Version: 1.0
Status: Active
Created: 2025-03-20

Name

- **In prose:** Bharat Data (two words, both capitalized)
- **In code:** `bharatdata` (one word, lowercase)
- **npm package:** `bharatdata`
- **PyPI package:** `bharatdata`
- **Never:** BharatData, bharatData, Bharatdata, BHARATDATA, BD

Tagline

"Clean, queryable Indian public data"

Use everywhere: README, npm description, PyPI description, GitHub repo
description, social media bio, website header.

Disclaimer

**"Bharat Data is an independent open source project. It is not affiliated
with or endorsed by any Indian government body."**

Include on: website footer, README, API documentation, Terms of Use.

Package Descriptions

npm: "Clean, queryable Indian government data. TypeScript SDK with
full types, source attribution, and confidence scoring."

```
**PyPI:** "Clean, queryable Indian government data. Python SDK
        with
pandas integration, source attribution, and confidence scoring."

**GitHub:** "Open source SDK for clean, queryable Indian
        government data.
TypeScript + Python. REST API. Web playground."

## Social Media Bio

"Building Bharat Data — making Indian government data accessible
        for
developers, data scientists, journalists, and researchers. Open
        source. 🇮🇳"
```

Documentation Folder Structure

```
# BharatData — Documentation Folder Structure

bharatdata/
├── .agent/
│   ├── FEEDING_PROMPT.md
│   ├── agents/
│   │   ├── data-pipeline-specialist.md
│   │   ├── sdk-architect.md
│   │   ├── data-quality-auditor.md
│   │   └── bharatdata-docs-specialist.md
│   ├── skills/
│   │   ├── bharatdata-pdf-parser/
│   │   │   └── SKILL.md
│   │   ├── bharatdata-normalizer/
│   │   │   └── SKILL.md
│   │   ├── bharatdata-sdk/
│   │   │   └── SKILL.md
│   │   ├── bharatdata-data-quality/
│   │   │   └── SKILL.md
│   │   └── bharatdata-project-context/
│   │       └── SKILL.md
│   ├── workflows/
│   │   ├── bharatdata-validate.md
│   │   ├── bharatdata-ingest.md
│   │   ├── bharatdata-add-source.md
│   │   └── bharatdata-publish.md
│   └── scripts/
│       └── bharatdata_quality_check.py
│
```

```
├── data/
│   ├── raw/
│   │   └── ncrb-crime-in-india/
│   │       ├── 2021/
│   │       │   ├── states-uts/
│   │       │   ├── districts/
│   │       │   ├── metro-cities/
│   │       │   └── pdf-report/
│   │       ├── 2022/
│   │       │   └── (same structure)
│   │       └── 2023/
│   │           └── (same structure)
│   ├── structured/
│   │   └── metro-cities/
│   │       └── (9 JSON files from PDF parsing)
│   ├── canonical-mappings/
│   │   └── states.json
│   └── unmapped-entities.json
│
├── docs/
│   ├── PROJECT_OVERVIEW.md                    ← DOC-01
│   │
│   ├── internal/
│   │   ├── PRD.md                             ← DOC-02
│   │   ├── QUALITY_STANDARDS.md               ← DOC-03
│   │   ├── DATA_PIPELINE_SPEC.md              ← DOC-06
│   │   ├── CONTENT_STRATEGY.md                ← DOC-08
│   │   ├── BRAND_MINIMUMS.md                  ← DOC-09
│   │   ├── EXECUTION_STRATEGY.md              ← DOC-00
│   │   ├── LONG_TERM_VISION.md                ← DOC-00d
│   │   ├── AUDIENCE_MAP.md                    ← DOC-00c
│   │   └── JOURNAL.md                         ← DOC-00a
│   │
│   ├── sdk/
│   │   ├── DESIGN_TYPESCRIPT.md               ← DOC-04
│   │   └── DESIGN_PYTHON.md                   ← DOC-05
│   │
│   ├── data-schema/
│   │   ├── NORMALIZATION.md                   ← DOC-07
│   │   └── FIELD_MAPPINGS.md                  ← From Stage 1
│   │
│   ├── data-sources/
│   │   ├── NCRB_CRIME.md                      ← From Stage 1
│   │   ├── file-format-notes.md               ← From Stage 1
│   │   ├── column-header-inventory.md         ← From Stage 1
│   │   ├── state-name-variations.md           ← From Stage 1
│   │   ├── district-name-inventory.md         ← From Stage 1
│   │   └── data-quality-issues.md             ← From Stage 1
│   │
│   ├── legal/
│   │   └── ATTRIBUTION.md                     ← Created in Stage
        3
```

```
│   │
│   └── stages-execution-docs/
│       ├── stage-1/
│       │   └── STAGE_1_COMPLETION_REPORT.md
│       └── stage-2/
│           └── STAGE_2_COMPLETION_REPORT.md   ← Created after
        Stage 2
│
├── scripts/
│   └── stage-1/
│       ├── examine_all.py
│       ├── build_docs.py
│       ├── process_metro_data.py
│       └── probe_pdf.py
│
├── packages/
│   ├── sdk-typescript/                        ← Built in Stage 3
│   │   ├── src/
│   │   ├── package.json
│   │   └── tsconfig.json
│   ├── sdk-python/                            ← Built in Stage 3
│   │   ├── bharatdata/
│   │   ├── pyproject.toml
│   │   └── tests/
│   ├── api/                                   ← Built in Stage 3
│   │   └── src/
│   └── web/                                   ← Built in Stage 3
│       └── src/
│
├── README.md                                  ← Built in Stage 3
├── CONTRIBUTING.md                            ← Built in Stage 3
├── CODE_OF_CONDUCT.md                         ← Built in Stage 3
├── SECURITY.md                                ← Built in Stage 3
├── CHANGELOG.md                               ← Built in Stage 3
├── LICENSE                                    ← MIT, created in
        Stage 2
└── .github/
    └── workflows/
        ├── daily-health-check.yml             ← Built in Stage 3
        └── weekly-quality-check.yml           ← Built in Stage 3
```

This folder structure accommodates all 36 documents from the documentation index, all Stage 1 artifacts, all Stage 2 foundation documents, and the Stage 3 build artifacts. Each file has a clear home. No document is orphaned. The structure scales to accommodate future data sources, future SDK languages, and future documentation additions without reorganization.

```
# Stage 2 Validation and Stage 3 Complete Guide

---
```

Stage 2 Validation

Your Stage 2 execution is solid. The completion report shows systematic thinking and the right documents were created. Let me validate each element and flag what needs attention before you begin Stage 3.

What Is Correctly Done

Your core specification documents are in place. PROJECT_OVERVIEW.md, PRD_V1.md, QUALITY_STANDARDS.md, SDK_SPEC_TYPESCRIPT.md, SDK_SPEC_PYTHON.md, and PIPELINE_SPEC.md cover every decision that Stage 3 engineering needs to reference. This is the right set of documents.

Your agent configuration is functional. AGENT_ROLES.md with Architect, Builder, and Reviewer personas is a valid approach. It is a different structure than having separate agent files per role but it achieves the same purpose of giving the AI clear behavioral rules for different task types.

Your MASTER_FEEDING_PROMPT.md serves as the project DNA for AI sessions. This is critical and you have it.

Your two workflows for data validation and source ingestion cover the most important automated procedures for Stage 3.

Your key decisions are sound. Confidence threshold of 0.5 for serving. Source attribution on every record. 1 request per second rate limit. Canonical state mapping as mandatory. These are the right standards.

What Needs a Small Adjustment

There are three things I want you to verify or adjust before starting Stage 3.

First, your Stage 3 implementation plan mentions database tables named `raw_files`, `ingestion_batches`, and `observations`. Check that these table names align with what your PIPELINE_SPEC.md defines. In our earlier documents, the tables were called `data_sources`, `ingestion_runs`, `crime_records_state`, `crime_records_district`, `crime_records_city`, `parse_failures`, and `normalization_log`. If your PIPELINE_SPEC.md uses different names, that is fine as long as your Stage 3 code matches your spec. If there is a mismatch, update either the spec or the implementation plan so they agree. Do not start coding with ambiguity about table names.

Second, your implementation plan lists four scripts: ingester.py, pdf_parser.py, normalizer.py, and loader.py. This

is a clean separation but you are missing the Excel parser. Your Stage 1 discovery showed that your primary data comes from Excel files (66 of them) with PDF parsing as secondary for metro city data. Your Stage 3 plan emphasizes the PDF parser but does not explicitly mention the Excel parser which handles the majority of your data. You need five pipeline components, not four: ingester, excel_parser, pdf_parser, normalizer, and loader. Or you combine both parsers into a single parser.py that dispatches based on file type.

Third, confirm your actual folder structure. Your completion report mentions `Docs/planning-docs/` for core specs and `Docs/internal/` for agent docs. Make sure you know exactly where each file lives before Stage 3 begins because the feeding prompt references these paths. If the AI assistant looks for `Docs/planning-docs/PRD_V1.md` and the file is actually at `Docs/internal/PRD_V1.md`, the context loading fails silently and the AI generates code without your quality standards.

With those three checks done, Stage 2 is validated and you are ready for Stage 3.

Stage 3 — Data Pipeline Implementation — Complete Guide

What Stage 3 Produces

When Stage 3 is complete, you will have a working data pipeline that takes raw NCRB files from your data/raw folder, parses them into structured records, normalizes every record against your canonical mappings, stores everything in a PostgreSQL database with full source attribution and confidence scores, and exposes the data through a REST API that both SDKs and the playground will consume.

Stage 3 is the heaviest engineering stage of the entire project. Every other stage either plans or consumes. This stage builds the core engine.

Pre-Requisite Setup — Do This Before Writing Pipeline Code

Before you write your first line of pipeline code, you need three things set up and verified.

Set Up Neon PostgreSQL

Go to neon.tech and create a free account if you do not have one.

```
Create a new project called bharatdata. Neon will give you a
connection string that looks like `postgresql://
username:password@host/database?sslmode=require`.

Create a file called `.env` in your project root. Add these
variables.
```

DATABASE_URL=postgresql://username:password@host/database?sslmode=require LOG_LEVEL=INFO

```
Add `.env` to your `.gitignore` immediately. Never commit database
credentials to git.

Verify the connection by writing a quick test script.

```python
import os
from dotenv import load_dotenv
import psycopg2

load_dotenv()

conn = psycopg2.connect(os.getenv("DATABASE_URL"))
cursor = conn.cursor()
cursor.execute("SELECT version();")
print(cursor.fetchone())
cursor.close()
conn.close()
```
```

Run it. If you see the PostgreSQL version printed, your connection works.

Install Python Dependencies

Create a `requirements.txt` file in your project root.

```
pandas>=2.0.0
openpyxl>=3.0.0
pdfplumber>=0.9.0
psycopg2-binary>=2.9.0
sqlalchemy>=2.0.0
python-dotenv>=1.0.0
loguru>=0.7.0
requests>=2.28.0
```

Install them.

```
pip install -r requirements.txt
```

Verify each critical dependency works by importing it in a Python shell.

```
import pandas as pd
import pdfplumber
import psycopg2
import sqlalchemy
from loguru import logger
print("All dependencies loaded successfully")
```

Create the Database Schema

This is the first real engineering deliverable of Stage 3. The schema must match your PIPELINE_SPEC.md exactly. Write a migration script that creates all tables.

Create a file at `Scripts/stage-3/create_schema.py`.

This script connects to your Neon database and creates all tables needed for the pipeline. The tables are organized into four groups.

The metadata tables track data sources and ingestion runs. The data tables store the actual crime records at state, district, and city levels. The quality tables store parse failures and normalization decisions. The monitoring tables store health check results.

Here is the exact schema based on everything we have specified across all documents.

The `data_sources` table stores one row per registered government data source. For v1 this is just NCRB Crime in India but the schema supports future sources. It has columns for id, name, source_url, publishing_body, data_format, update_frequency, last_ingested_at, status, and created_at.

The `ingestion_runs` table stores one row per pipeline execution. Each time you run the full pipeline, a new ingestion run is created. It tracks source_id, started_at, completed_at, status (running, success, failed), files_processed count, records_extracted count, records_stored count, parse_failures count, and error_message.

The `crime_records_state` table stores normalized state-level crime records. Each row is one state, one year, one crime category. It has columns for id, ingestion_run_id, state (canonical name), state_code, year, category (canonical ID like “ipc-total”), category_label (human-readable), total_cases, rate_per_lakh, chargesheeted, convicted, acquitted, pending_investigation, pending_trial, confidence (0.0 to 1.0), source_url, source_file (original filename), report_name, publishing_body, collection_date,

normalizer_version, boundary_note, and created_at. It has check constraints ensuring confidence is between 0 and 1 and year is between 2000 and 2030.

The `crime_records_district` table has the same structure as the state table plus a district column.

The `crime_records_city` table has the same structure plus city and parent_state columns.

The `parse_failures` table logs every parsing failure with ingestion_run_id, source_file, source_sheet, row_number, column_name, raw_value, expected_type, error_message, and created_at.

The `normalization_log` table logs every normalization decision with record_id, field, input_value, output_value, mapping_source, confidence_impact, and created_at.

Create indexes on the most common query patterns. State plus year for the most common query. Year alone. Category alone. State plus category plus year for compound queries. Confidence for filtering out low-confidence records. District name and city name for their respective tables.

After writing the schema creation script, run it against your Neon database. Verify all tables exist by querying the information_schema.

```
cursor.execute("""
    SELECT table_name FROM information_schema.tables
    WHERE table_schema = 'public'
    ORDER BY table_name;
""")
for row in cursor.fetchall():
    print(row[0])
```

You should see all your tables listed. This verification confirms your database is ready for data.

Building the Pipeline — Step by Step

You build the pipeline in the exact order that data flows through it. Ingester first because everything downstream depends on downloaded files. Parser second because the normalizer needs structured records to work with. Normalizer third because the loader needs canonical records. Loader fourth because it writes to the database. Each component is built, tested in isolation, and verified before moving to the next.

Step 1 — Build the Ingester

Create `Scripts/stage-3/ingester.py`.

The ingester is responsible for one thing: getting raw files from the NCRB website to your local filesystem with complete metadata. It does not parse files. It does not normalize anything. It downloads and records what it downloaded.

The ingester operates in this sequence for each target year.

First it visits the NCRB Additional Tables page for that year. The URL pattern is `https://www.ncrb.gov.in/crime-in-india-additional-table?year={year}&category=`. It fetches this page and extracts all downloadable file links from the HTML.

Second, for each file link, it checks whether it has already downloaded this file by comparing the SHA-256 hash of the remote file against the hash stored in the database from the previous download. If the hashes match, the file has not changed and the ingester skips it with a log entry. If the hashes differ or no previous download exists, the ingester downloads the file.

Third, it stores the downloaded file in the appropriate folder under `data/raw/ncrb-crime-in-india/{year}/` based on the file type (states-uts, districts, metro-cities).

Fourth, it records the download in the `data_sources` table (if this is a new source) and creates an entry in the `ingestion_runs` table tracking this pipeline execution.

The ingester must enforce rate limiting. Between every HTTP request to the NCRB server, it waits at least one full second. This is implemented with an explicit `time.sleep(1.0)` call, not through any async mechanism that might drift.

The ingester must handle failures. If a download fails, it retries with exponential backoff: wait 60 seconds after the first failure, 300 seconds after the second, and if the third attempt also fails, it logs the failure with full details and creates a GitHub issue (or logs to a file for manual review). It does not crash. It continues with the next file.

The ingester must handle the timestamp prefix issue you discovered in Stage 1 for 2021 files. When storing files, it strips these prefixes from filenames while recording the original filename in the metadata.

How to test the ingester:

Since you already have all 75 files downloaded during Stage 1, you do not need to re-download everything from NCRB. Instead, test the ingester in two ways.

First, test the metadata recording. Point the ingester at your existing `data/raw/` files and have it calculate hashes and record metadata in the database for each file. This verifies that the hash calculation, database insertion, and metadata recording all work correctly without making any HTTP requests.

Second, test the download logic against one specific file. Pick one small States/UTs file from the NCRB website. Have the ingester download it fresh. Verify the downloaded file matches the one you already have from Stage 1. Verify the hash is recorded correctly. Run the ingester again and verify it detects “no new data” for this file.

Third, test the rate limiting. Add logging that prints the timestamp of every HTTP request. Run the ingester against three files and verify that the minimum gap between consecutive request timestamps is at least 1000 milliseconds.

Fourth, test failure handling. Point the ingester at an intentionally wrong URL. Verify it retries three times with the correct backoff intervals. Verify it logs the failure with full context. Verify it does not crash.

Write the results of these tests in your project journal. Note what worked, what needed adjustment, and what surprised you.

Step 2 — Build the Excel Parser

Create `Scripts/stage-3/excel_parser.py`.

This is the component that handles your 66 Excel files from NCRB Additional Tables. It is separate from the PDF parser because Excel and PDF parsing have fundamentally different challenges and require different libraries and logic.

The Excel parser takes a file path as input and returns a list of structured record dictionaries with confidence scores.

The parser must handle the specific challenges you documented during Stage 1.

Header detection is the first challenge. NCRB Excel files have varying header structures. Some have title text in the first few rows. Some have blank rows between the title and headers. Some have merged cells in headers. Your Stage 1 script `examine_all.py` already solves this with skiprows logic. Port that logic into the parser.

The parser reads the field mappings from your `Docs/data-schema/FIELD_MAPPINGS.md` (or a JSON version of it that you create for programmatic access). Each raw column name from the Excel file is mapped to a canonical BharatData field name. Columns that map to `_skip` (like serial numbers) are ignored. Columns that do not appear in the mappings are logged as warnings and stored in raw metadata.

For each data row in the file, the parser extracts all mapped field values, converts data types according to your quality standards (strings to numbers, null representations to actual null), assigns a confidence score based on how cleanly the parsing went, and creates a structured record dictionary.

Summary rows (Total, Grand Total, All India) are detected and handled separately. They are not included in the data records. They are stored as aggregation records that can be used for cross-validation.

The parser must handle your 133 documented data quality issues. Reference your `Docs/data-sources/data-quality-issues.md` file. Each issue should have a corresponding handling rule in the parser. Some issues reduce confidence. Some require specific data transformations. Some are simply logged as known quirks.

How to test the Excel parser:

Run the parser against all 66 Excel files. For each file, verify that the parser produces the expected number of data rows (approximately 36 for state files, approximately 780 for district files, 34 for metro city files).

For state-level files, check that exactly 36 entities appear (28 states plus 8 UTs). If you get 37 or 35, something is wrong with summary row detection or entity counting.

Pick 5 files of different types (one States/UTs IPC, one States/UTs Crimes Against Women, one District-wise, one Metro City, one Crimehead-wise). For each file, manually verify 10 random cells against the original Excel file opened in a spreadsheet application. All 50 cells across 5 files must match exactly.

Check that confidence scores are assigned correctly. Clean files should have confidence 1.0. Files where the timestamp prefix was handled should have 0.95. Files requiring any other special handling should have proportionally lower scores.

Check that all field names in the output match your canonical schema from FIELD_MAPPINGS.md. No raw column names should leak into the output.

Check that null values are represented as Python None, not as 0, empty string, dash, or “NA”.

Check that all numeric values are stored as integers or floats, not as strings.

Run your Stage 1 `examine_all.py` script after the parser to verify backward compatibility. The parser’s output should be consistent with what examine_all.py validated.

Step 3 — Build the PDF Parser

Create `Scripts/stage-3/pdf_parser.py`.

This component handles the metropolitan city PDF data that is not available in the Excel Additional Tables. You already built the foundation for this during Stage 1 with `process_metro_data.py` which produced 9 structured JSON files. Stage 3 generalizes and hardens this logic.

The PDF parser takes a PDF file path as input, identifies tables within the PDF pages, extracts structured records, and assigns confidence scores.

The PDF parser has a base confidence of 0.9 (lower than Excel’s 1.0) because PDF extraction inherently involves more uncertainty. Even verified extractions get 0.9 rather than 1.0 to signal to downstream consumers that this data came from a more error-prone extraction method.

Use pdfplumber as your primary extraction library. Based on your Stage 1 probe_pdf.py findings, you know which tables pdfplumber handles well and which require special treatment.

For the 34 metropolitan cities, the parser must verify that extracted city names match the known list of 34 cities. If an extracted name does not match, it is logged as an unmapped entity and confidence is reduced.

The PDF parser handles merged cells by detecting rows where cell values span multiple columns and resolving them to flat column names following the same pattern as the Excel parser.

How to test the PDF parser:

Compare the PDF parser output against the 9 JSON files you produced during Stage 1. The results should be identical or improved. Any differences must be investigated and explained.

Verify the Agra 2023 IPC count of 7026 that you confirmed during Stage 1. This is your ground truth benchmark.

Pick 5 additional city-metric combinations and manually verify against the source PDF. All must match.

Check that all 34 cities appear in the output for each parsed table. If any city is missing, the parser has a gap.

Step 4 — Build the Normalizer

Create `Scripts/stage-3/normalizer.py`.

The normalizer takes parsed records from either parser and transforms them into canonical form ready for database storage. It is the most intellectually important component because it encodes your domain knowledge about Indian administrative geography.

The normalizer operates as a pipeline of transformations applied sequentially to each record. The order matters because some transformations depend on the results of earlier ones.

Transformation 1: State name resolution. Load `data/canonical-mappings/states.json`. For each record's state_raw field, look up the canonical name, state code, and entity type. If the lookup succeeds, replace the raw name with the canonical name and record the resolution in the normalization log with zero confidence impact. If the lookup fails, keep the raw name, set state_code to "UNKNOWN", log the unmapped entity to `data/unmapped-entities.json`, and reduce confidence by 0.2.

Transformation 2: District name resolution (for district-level records). This is harder than state resolution because there are approximately 780 districts with extensive name variations. For v1, start with exact match resolution. If the district name from the parsed record exactly matches a known district name for the given state, accept it. If not, try case-insensitive matching. If that fails, log the unmapped district and reduce confidence by 0.2.

You do not need a complete district canonical mapping file for v1. Start with exact and case-insensitive matching which will handle the majority of cases. Build the district mapping file incrementally as you encounter unmapped districts.

Transformation 3: City name resolution (for city-level records). Match city names against the known list of 34 metropolitan cities. Handle common variations like "Bangalore" to "Bengaluru" and "Bombay" to "Mumbai".

Transformation 4: Data type enforcement. Convert all numeric fields from strings to integers or floats. Handle Indian number formatting if any slipped through the parser. Convert all null representations to Python None.

Transformation 5: Category mapping. Map raw crime category descriptions from the source files to canonical category IDs. “Total Cognizable IPC Crimes” becomes “ipc-total”. “Crime Against Women” becomes “crimes-against-women”. Use the category mapping table from your PIPELINE_SPEC.md.

Transformation 6: Historical boundary annotation. Check whether the record involves a state with boundary changes (Telangana, Ladakh, Jammu & Kashmir, Dadra and Nagar Haveli). If the record’s year is before or after a boundary change date, add an appropriate boundary_note explaining the historical context.

Transformation 7: Deduplication check. Before the record proceeds to the loader, check whether an identical record already exists in the database from a previous ingestion run. Identical means same state, year, category, and total_cases. If a duplicate exists, skip the record. If the values differ for the same state-year-category, flag this as a discrepancy for investigation.

Transformation 8: Confidence adjustment. Apply any remaining confidence adjustments based on the cumulative impact of normalization steps. If the original parser confidence was 1.0 and the normalizer reduced it by 0.2 for an unmapped state name, the final confidence is 0.8.

Transformation 9: Source attribution attachment. Add the full provenance metadata to each record: source_url, source_file, report_name, publishing_body, collection_date, normalizer_version, and ingestion_batch_id.

Transformation 10: Range validation. Check each numeric field against plausible ranges. Total_cases should be between 0 and 5,000,000. Rate_per_lakh should be between 0 and 5000. Year should be between 2000 and 2030. Confidence should be between 0.0 and 1.0. Values outside these ranges are flagged with a warning log and confidence is reduced.

How to test the normalizer:

Test state name resolution with every variation from your `state-name-variations.md` file. Every variation must resolve to the correct canonical name.

Test with intentionally wrong state names (“Nonexistent State”, “XYZABC”). Verify that the normalizer does not crash, logs the unmapped entity, and reduces confidence by 0.2.

Test historical boundary annotations. Pass a record for Telangana with year 2012. Verify that the boundary_note correctly states that Telangana was part of Andhra Pradesh before 2014.

Test data type enforcement with edge cases. Pass “1,23,456” and verify it becomes 123456. Pass “-” and verify it becomes None. Pass “0” and verify it remains 0 (not None). Pass “12.5%” and verify it becomes 12.5 with appropriate metadata.

Test deduplication by running the same dataset through the normalizer twice. The second run should produce zero new records.

Cross-validate state totals against district sums. Take the total IPC crimes for Maharashtra from the state-level data. Sum all Maharashtra districts from the district-level data. They should match within 1% tolerance. If they do not, document the discrepancy.

Step 5 — Build the Loader

Create `Scripts/stage-3/loader.py`.

The loader takes normalized records from the normalizer and writes them to the PostgreSQL database. It is the simplest component but it must handle transactions correctly to prevent partial data loads.

The loader operates within database transactions. All records from a single ingestion run are loaded within a single transaction. If any record fails to insert, the entire transaction is rolled back and the ingestion run is marked as failed. This prevents the database from containing a partial dataset from a failed run.

For each record, the loader determines which table to insert into based on the record’s geographic level. State-level records go into `crime_records_state`. District-level records go into `crime_records_district`. City-level records go into `crime_records_city`.

The loader also inserts parse failures into the `parse_failures` table and normalization decisions into the `normalization_log` table.

After all records are inserted, the loader updates the `ingestion_runs` table with the final counts: records_extracted, records_stored, parse_failures, and completed_at timestamp.

How to test the loader:

Load a small subset first. Take 5 state-level records from the normalizer output and load them. Query the database to verify all 5 are present with correct values.

Test the transaction rollback. Intentionally corrupt one record in a batch of 10 (e.g., set confidence to 2.0 which violates the check constraint). Run the loader. Verify that zero records were inserted because the transaction was rolled back. Verify the ingestion run is marked as failed.

Load all state-level data for 2023. Query the database. Verify there are approximately 36 records per category. Verify total record count matches expectations from your Stage 1 data inventory.

Load district-level data for 2023. Verify approximately 780 records per category.

Load city-level data. Verify 34 records per year per category.

Spot-check 10 random records by querying them from the database and comparing against the original Excel file values.

Step 6 — Build the Pipeline Orchestrator

Create `Scripts/stage-3/run_pipeline.py`.

The orchestrator ties all four components together into a single executable pipeline. When you run this script, it executes the complete sequence: create an ingestion run record, run the ingester for all target years, run the appropriate parser for each downloaded file, run the normalizer on all parsed records, run the loader to store everything in the database, update the ingestion run with final statistics, and log the results.

The orchestrator accepts command-line arguments for controlling which steps to run and which years to process.

```
python Scripts/stage-3/run_pipeline.py --years 2021 2022 2023 --steps all
python Scripts/stage-3/run_pipeline.py --years 2023 --steps parse normalize load
python Scripts/stage-3/run_pipeline.py --years 2023 --steps load --dry-run
```

The `--dry-run` flag runs the pipeline without writing to the database. It processes all files, parses them, normalizes them, and reports what would be loaded, but commits nothing. This is useful for validating changes to the parser or normalizer without affecting production data.

The orchestrator logs the complete pipeline execution with timings for each step, record counts at each stage, and any warnings or errors. At the end of execution, it prints a summary report.

```
Pipeline Execution Summary
============================
Ingestion Run ID: abc-123-def
Started: 2025-03-21T06:00:00Z
Completed: 2025-03-21T06:05:32Z
Duration: 5m 32s

Files processed: 75
Records extracted: 11,490
Records normalized: 11,490
Records stored: 11,490
Parse failures: 0

Confidence distribution:
  1.0:  9,200 records (80.1%)
  0.9:  1,530 records (13.3%)
  0.8:    760 records  (6.6%)
  <0.5:     0 records  (0.0%)

States mapped: 36/36
Unmapped entities: 0
```

How to test the orchestrator:

Run the full pipeline with `--dry-run` first. Verify the summary report looks correct. Check all record counts match expectations.

Then run without dry-run. Verify all records are in the database. Run the `validate-data` workflow from your `.agent/workflows/` to confirm data quality.

Query the database for total record counts by table.

```sql
SELECT 'state' as level, COUNT(*) FROM crime_records_state
UNION ALL
SELECT 'district', COUNT(*) FROM crime_records_district
UNION ALL
SELECT 'city', COUNT(*) FROM crime_records_city;
```

Compare these counts against your Stage 1 data inventory.

Step 7 — Build the Data Freshness Monitor

Create `Scripts/stage-3/health_check.py`.

This script runs as a GitHub Action on a daily schedule. It checks whether the NCRB data sources are still accessible and whether any data has changed since the last ingestion.

The health check visits each registered source URL and checks the HTTP status code. If the URL returns a non-200 status, the check fails and creates a GitHub issue with the details.

For files that are accessible, the check computes the SHA-256 hash and compares against the last known hash in the database. If the hash has changed, it means NCRB has published updated data and a new ingestion run should be triggered.

Create the GitHub Action workflow file at `.github/workflows/daily-health-check.yml`.

```yaml
name: Daily Data Source Health Check
on:
  schedule:
    - cron: '0 6 * * *'
  workflow_dispatch:

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install requests
      - run: python Scripts/stage-3/health_check.py
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
```

How to test the monitor:

Run the health check script locally. Verify it correctly reports the status of NCRB URLs. Intentionally test with a wrong URL to verify failure detection.

Push the GitHub Action workflow and trigger it manually with workflow_dispatch. Verify the Action runs successfully on GitHub.

Step 8 — Build the Quality Validation Script

Create `Scripts/stage-3/validate_data.py`.

This script runs comprehensive quality checks against the data in your database. It implements the validation procedures from your `validate-data.md` workflow.

The validation checks include:

Required fields check: Verify that no records have null values in mandatory fields (state, year, category, total_cases, confidence, source_url, collection_date).

Confidence distribution check: Calculate the average confidence across all records. Alert if average is below 0.7. Calculate the percentage of records below 0.5. Alert if more than 10% are below threshold.

Completeness check: For each year, verify that all 36 states/UTs have records. Flag any missing states.

Consistency check: Compare record counts by state between consecutive years. Flag any state where the count changed by more than 50% (which could indicate a parsing error rather than a real change).

Statistical outlier check: For each state-category combination, calculate the mean and standard deviation across years. Flag any values more than 3 standard deviations from the mean.

Cross-level validation: For states where both state-level and district-level data exist, verify that the state total matches the sum of district values within 1% tolerance.

The script outputs a JSON report with pass/fail status for each check and exits with code 0 if all checks pass or code 1 if any check fails.

How to test the validation script:

Run it against your fully loaded database. Review the report. Investigate any failures. Some failures may indicate real data quality issues in the NCRB source data rather than pipeline bugs. Document these in your data-quality-issues.md.

Step 9 — Run the Complete Pipeline End to End

This is the moment of truth. You have built and individually tested every component. Now run the full pipeline and validate the complete result.

Run the orchestrator with all years.

```
python Scripts/stage-3/run_pipeline.py --years 2021 2022 2023 --
        steps all
```

Watch the logs. Note any warnings or unexpected behavior.

After the pipeline completes, run the validation script.

```
python Scripts/stage-3/validate_data.py
```

Review the validation report. Every check should pass. If any check fails, investigate and fix the issue before proceeding.

Run the spot-check verification. Select 20 random records from the database spanning different tables, different years, and different states. For each record, open the original Excel file in a spreadsheet application and verify the values match exactly. Document the results.

If all 20 records match, your pipeline is producing accurate data. If any do not match, trace the discrepancy through the pipeline to find where the error was introduced.

After verification, commit all Stage 3 code to git and push to GitHub.

Step 10 — Write the Stage 3 Completion Report

Create `Docs/stages-execution-docs/stage-3/STAGE_3_COMPLETION_REPORT.md`.

Document what was built, what was tested, what the results were, and what issues were discovered. Include the pipeline execution summary, the validation report results, and the spot-check verification results.

This completion report serves as your quality gate. If the report shows all checks passing and all spot-checks verified, you are ready to move to building the SDK and API layers.

Stage 3 Folder Structure After Completion

```
Scripts/
└── stage-3/
    ├── create_schema.py          ← Database table creation
    ├── ingester.py               ← Downloads files from NCRB
    ├── excel_parser.py           ← Parses NCRB Excel files
    ├── pdf_parser.py             ← Parses metro city PDFs
    ├── normalizer.py             ← Canonical name resolution and
type enforcement
    ├── loader.py                 ← Writes records to PostgreSQL
    ├── run_pipeline.py           ← Orchestrator tying all
components together
    ├── health_check.py           ← Daily source URL monitoring
    ├── validate_data.py          ← Comprehensive data quality
checks
    └── utils/
        ├── rate_limiter.py       ← 1 req/sec enforcement
        ├── confidence.py         ← Confidence score calculation
        ├── field_mapper.py       ← Column name to canonical field
```

```
translation
        ├── state_resolver.py     ← State name normalization using
states.json
        ├── category_mapper.py    ← Crime category to canonical ID
mapping
        ├── null_handler.py       ← Null value detection and
standardization
        └── logger_config.py      ← Loguru configuration for
pipeline logging

.github/
└── workflows/
    └── daily-health-check.yml    ← Automated source monitoring

Docs/
└── stages-execution-docs/
    └── stage-3/
        └── STAGE_3_COMPLETION_REPORT.md
```

Stage 3 Build Order Summary

Step	Component	Test Before Moving On	Estimated Time
Pre-req	Neon database setup	Connection test passes	30 minutes
Pre-req	Python dependencies	All imports succeed	15 minutes
1	create_schema.py	All tables visible in database	1-2 hours
2	ingester.py	Metadata recorded, rate limiting verified, failure handling works	3-4 hours
3	excel_parser.py	66 files parse, 50 random cells verified, confidence correct	4-6 hours
4	pdf_parser.py	Matches Stage 1 JSON output, Agra benchmark verified	2-3 hours

Step	Component	Test Before Moving On	Estimated Time
5	normalizer.py	All state variations resolve, types enforced, boundary notes correct	4-6 hours
6	loader.py	Records in database, transaction rollback works, counts match	2-3 hours
7	run_pipeline.py	Full pipeline runs, summary report correct, dry-run works	2-3 hours
8	health_check.py	URL status checked, hash comparison works, GitHub Action runs	1-2 hours
9	validate_data.py	All quality checks pass against loaded data	2-3 hours
10	End-to-end run	Complete pipeline, validation, 20-record spot-check all pass	2-3 hours
11	Completion report	Stage 3 report written with all verification results	1 hour

The One Rule for Stage 3

Build each component. Test it thoroughly in isolation. Verify it produces correct output. Only then move to the next component. Do not build the normalizer before the parser is verified. Do not build the loader before the normalizer is verified. Each component stands on the verified output of the one before it.

If at any point a component produces wrong data, stop. Fix it. Re-verify. Then continue. A pipeline that processes data incorrectly is worse than no pipeline at all because it creates a false sense of having correct data.

Your quality standards document says wrong data served confidently is a critical failure. Stage 3 is where you prove that your pipeline never does this. Every test, every spot-check, every validation is evidence that your data is trustworthy. Build that evidence carefully.

```
I have read and internalized the complete BharatData documentation
across all five parts. Here is my confirmation of understanding,
organized by the key dimensions of the project:

## Project Identity
- **Name**: Bharat Data (prose) / `bharatdata` (code/packages)
- **Tagline**: "Clean, queryable Indian public data"
- **License**: MIT
- **Independence disclaimer**: Not affiliated with any Indian
government body

## Core Problem & Solution
BharatData bridges the gap between India's vast but practically
unusable government data (locked in hostile PDFs, inconsistent
formats, no APIs) and the developers, journalists, researchers,
students, and policy makers who need it. The solution is a four-
stage data pipeline (ingest → parse → normalize → store) serving
clean data through TypeScript and Python SDKs, a REST API, and a
web playground.

## Key Technical Specifications I'm Tracking

**Data Pipeline:**
- NCRB Crime in India as primary source (v1), using structured
Additional Tables (Excel) as the fast path, with PDF parsing for
metro city gap coverage
- 75 raw files across 3 years (2021-2023), 66 Excel + 9 PDF-parsed
JSON
```

- 133 documented data quality issues from Stage 1 exploration
- 36 state/UT entities mapped in `states.json` with all known name variations

Quality Standards:
- Confidence threshold of 0.5 for serving data through API
- Zero `any` types in TypeScript SDK public API
- Maximum 1 request/second to government servers
- Every record must have source URL, collection date, and confidence score
- Null means null — never 0, empty string, dash, or "N/A"
- Database records are immutable

TypeScript SDK: 7 query functions (`getByState`, `getByYear`, `getByDistrict`, `getByCategory`, `getByCity`, `compare`, `search`) with standard response shape (`data`, `meta`, `query`), custom error classes, and full type safety.

Python SDK: Mirrors TypeScript logically but with snake_case, pandas DataFrame integration (`to_dataframe()`), export methods (`to_csv`, `to_parquet`, `to_json`, `to_excel`), citation generation (`cite()`), optional dependency groups, and Jupyter rich display.

Database: PostgreSQL on Neon (500MB free tier), with separate tables for state/district/city crime records, plus metadata tables for data sources, ingestion runs, parse failures, and normalization logs.

Infrastructure: Free-tier only — Neon, Vercel/Cloudflare Workers, GitHub Actions, Cloudflare R2.

Execution Strategy I'm Tracking
- **Stage 1** (completed): Data exploration, file acquisition, schema mapping
- **Stage 2**: Knowledge foundation — 10 core documents (PRD, Quality Standards, both SDK specs, Pipeline spec, Normalization guide, IDE config, Content strategy, Brand minimums)
- **Stage 3**: Pipeline implementation — ingester, Excel parser, PDF parser, normalizer, loader, orchestrator, health monitoring, validation
- **Stage 4**: SDK development (TypeScript + Python), REST API, web playground
- **Stage 5**: Distribution, launch, community building
- **Phase 4** (later): AI layer — NL queries, data narration, anomaly detection

Antigravity IDE Configuration
- 4 BharatData-specific agents, 5 skills, 4 workflows, 1 quality check script
- Planning mode mandatory, never Fast mode
- Claude Opus for complex tasks, Gemini Pro for boilerplate,

```
Gemini Flash for quick edits
- FEEDING_PROMPT.md as session context loader

I'm ready to serve as your implementation reference, decision
framework, quality standard enforcer, and context maintainer. What
would you like to work on?
```

BharatData Stage 3 — Data Pipeline Implementation

Complete Guide & Folder Structure

Stage 3 Folder Structure

bharatdata/ ├── pipeline/ # All Stage 3 pipeline code lives here │ ├── **init**.py │ ├── config.py # Environment vars, DB connection, constants │ ├── run_pipeline.py # Orchestrator — entry point for full pipeline │ │ │ ├── ingester/ # Stage 1 of pipeline: File acquisition │ │ ├── **init**.py │ │ ├── ingester.py # Downloads files from NCRB website │ │ ├── rate_limiter.py # Enforces max 1 req/sec per host │ │ └── hash_tracker.py # SHA-256 comparison for change detection │ │ │ ├── parser/ # Stage 2 of pipeline: Data extraction │ │ ├── **init**.py │ │ ├── excel_parser.py # Parses 66 NCRB Additional Tables Excel files │ │ ├── pdf_parser.py # Parses metro city PDF tables │ │ ├── header_detector.py # Finds actual header row in messy Excel files │ │ ├── summary_row_detector.py # Identifies Total/Grand Total rows │ │ └── field_mapper.py # Raw column names → canonical field names │ │ │ ├── normalizer/ # Stage 3 of pipeline: Canonical transformation │ │ ├── **init**.py │ │ ├── normalizer.py # Main normalization orchestrator │ │ ├── state_resolver.py # State name → canonical using states.json │ │ ├── district_resolver.py # District name resolution │ │ ├── city_resolver.py # Metro city name resolution (34 cities) │ │ ├── category_mapper.py # Raw category → canonical category ID │ │ ├── type_enforcer.py # String→number, null standardization │ │ ├── boundary_annotator.py # Telangana/Ladakh/bifurcation notes │ │ ├── confidence_scorer.py # Calculates and adjusts confidence 0.0-1.0 │ │ └── range_validator.py # Flags implausible values │ │ │ ├── loader/ # Stage 4 of pipeline: Database storage │ │ ├── **init**.py │ │ ├── loader.py # Writes normalized records to PostgreSQL │ │ ├── schema.py # SQLAlchemy models / raw SQL schema │ │ └── deduplicator.py # Prevents duplicate records across runs │ │ │ ├── monitoring/ # Health checks and quality validation │ │ ├── **init**.py │ │ ├── health_check.py # Daily source URL monitoring │ │ ├── quality_check.py # Weekly data quality validation │ │ └── report_generator.py # Produces JSON quality reports │ │ │ ├── utils/ # Shared utilities │ │ ├── **init**.py │ │ ├──

logger_config.py # Loguru configuration │ │ ├── db_connection.py # Database connection pool management │ │ └── file_utils.py # File path resolution, hash calculation │ │ └── tests/ # Pipeline-specific tests │ ├── **init**.py │ ├── test_excel_parser.py │ ├── test_pdf_parser.py │ ├── test_normalizer.py │ ├── test_loader.py │ ├── test_state_resolver.py │ ├── test_confidence_scorer.py │ ├── test_end_to_end.py # Full pipeline integration test │ └── fixtures/ # Test data files │ ├── sample_state_ipc.xlsx │ ├── sample_district.xlsx │ └── sample_metro_city.json │ ├── data/ # Data directory (from Stage 1) │ ├── raw/ # Original downloaded files │ │ └── ncrb-crime-in-india/ │ │ ├── 2021/ │ │ ├── 2022/ │ │ └── 2023/ │ ├── structured/ # PDF-parsed JSON (from Stage 1) │ │ └── metro-cities/ │ ├── canonical-mappings/ │ │ └── states.json # 36 state/UT canonical mappings │ └── unmapped-entities.json # Names that couldn't be resolved │ ├── .github/ │ └── workflows/ │ ├── daily-health-check.yml # Automated source monitoring │ └── weekly-quality-check.yml # Automated data quality report │ ├── .env # DATABASE_URL, LOG_LEVEL (gitignored) ├── .env.example # Template without secrets ├── requirements.txt # Python dependencies └── pyproject.toml # Project metadata

---

Pre-Requisite Setup (Before Writing Pipeline Code)

Step 0A: Set Up Neon PostgreSQL

1. Go to `neon.tech`, create a free account
2. Create a new project called `bharatdata`
3. Copy the connection string from the dashboard
4. Create `.env` in project root:

DATABASE_URL=postgresql://username:password@host/database?sslmode=require LOG_LEVEL=INFO

5. Add `.env` to `.gitignore` immediately
6. Create `.env.example` without the actual credentials:

DATABASE_URL=postgresql://user:pass@host/dbname?sslmode=require LOG_LEVEL=INFO

7. Verify connection with a quick test:

```python
import os
from dotenv import load_dotenv
import psycopg2

load_dotenv()
```

```
conn = psycopg2.connect(os.getenv("DATABASE_URL"))
cursor = conn.cursor()
cursor.execute("SELECT version();")
print(cursor.fetchone())
cursor.close()
conn.close()
```

Exit condition: You see the PostgreSQL version printed. If not, fix the connection before proceeding.

Step 0B: Install Python Dependencies

Create `requirements.txt`:

```
pandas>=2.0.0
openpyxl>=3.0.0
pdfplumber>=0.9.0
psycopg2-binary>=2.9.0
sqlalchemy>=2.0.0
python-dotenv>=1.0.0
loguru>=0.7.0
requests>=2.28.0
```

Install and verify:

```
pip install -r requirements.txt
python -c "import pandas, pdfplumber, psycopg2, sqlalchemy;
        print('All dependencies OK')"
```

Exit condition: All imports succeed without errors.

Step 0C: Set Up Logging Configuration

Create `pipeline/utils/logger_config.py`:

This configures Loguru with structured logging that outputs to both console and a log file. Every pipeline component uses this logger. Log entries include timestamps, component names, and structured context fields.

Key configuration decisions: - Console output at INFO level for real-time monitoring - File output at DEBUG level for post-mortem analysis - Log rotation at 10MB to stay within free-tier storage - Structured JSON format for log files so they can be queried programmatically

Exit condition: You can import the logger in a test script and see formatted output in both console and log file.

Pipeline Implementation — Step by Step

Step 1: Create the Database Schema

File: `pipeline/loader/schema.py`

What it does: Connects to Neon and creates all tables the pipeline needs.

Tables to create (7 total):

Table	Purpose	Key Columns
`data_sources`	Registry of government data sources	name, source_url, format, update_frequency, last_ingested_at, status
`ingestion_runs`	Log of every pipeline execution	source_id, started_at, completed_at, status, records_extracted, records_stored, parse_failures
`crime_records_state`	State-level crime data	state, state_code, year, category, category_label, total_cases, rate_per_lakh, chargesheeted, convicted, acquitted, pending_investigation, pending_trial, confidence, source_url, source_file, report_name, publishing_body, collection_date, normalizer_version, boundary_note
`crime_records_district`	District-level crime data	Same as state + district column
`crime_records_city`	Metro city crime data	Same as state + city, parent_state columns
`parse_failures`	Parse error log	ingestion_run_id, source_file, row_number, column_name, raw_value,

Table	Purpose	Key Columns
		expected_type, error_message
`normalization_log`	Normalization decision audit trail	record_id, field, input_value, output_value, mapping_source, confidence_impact

Critical constraints to include: - `confidence` must be between 0.0 and 1.0 (CHECK constraint) - `year` must be between 2000 and 2030 (CHECK constraint) - All UUID primary keys using `gen_random_uuid()`

Indexes to create (11 total): - `crime_records_state(state, year)` — most common query pattern - `crime_records_state(year)` — year-based queries - `crime_records_state(category)` — category filters - `crime_records_state(state, category, year)` — compound filter - `crime_records_state(confidence) WHERE confidence >= 0.5` — partial index for serving threshold - `crime_records_district(state)` — district lookup by state - `crime_records_district(state, year)` — district by state and year - `crime_records_district(district)` — specific district lookup - `crime_records_district(category)` — district category filter - `crime_records_city(city)` — city lookup - `crime_records_city(city, year)` — city by year

How to build: 1. Write the schema as raw SQL in the `schema.py` file 2. Add a `create_all_tables()` function that executes the SQL 3. Add a `verify_tables()` function that queries `information_schema.tables` 4. Run the script

How to test:

```
# After running schema creation
cursor.execute("""
    SELECT table_name FROM information_schema.tables
    WHERE table_schema = 'public' ORDER BY table_name;
""")
for row in cursor.fetchall():
    print(row[0])
```

Exit condition: All 7 tables visible in database. Query returns exactly 7 rows.

Estimated time: 1-2 hours

Step 2: Build the Rate Limiter

File: `pipeline/ingester/rate_limiter.py`

What it does: Ensures no more than 1 HTTP request per second to any single government server. This is a non-negotiable quality standard.

Design: - Tracks the timestamp of the last request per hostname - Before each request, calculates elapsed time since last request to that host - If less than 1 second has passed, sleeps for the remaining time - Thread-safe if needed for future parallel ingestion

Key implementation detail: Use `time.monotonic()` not `time.time()` because monotonic clock is not affected by system clock adjustments.

How to test: - Make 5 requests to the same host in rapid succession - Log the timestamp of each request - Verify the gap between consecutive requests is ≥ 1.0 seconds - Verify that requests to different hosts are not throttled against each other

Exit condition: 5 consecutive requests to the same host show ≥1 second gap between each pair.

Estimated time: 30 minutes

Step 3: Build the Ingester

File: `pipeline/ingester/ingester.py`

What it does: Downloads raw files from the NCRB website and stores them with complete metadata. This is the only component that talks to external servers.

Sequence of operations: 1. Visit the NCRB Additional Tables page for the target year - URL: `https://www.ncrb.gov.in/crime-in-india-additional-table?year={year}&category=` 2. Parse the HTML to extract file download links 3. For each file link: - Check rate limiter (wait if needed) - Download the file content - Calculate SHA-256 hash - Compare against last known hash in `data_sources` table - If new: store raw file with metadata, record ingestion event - If unchanged: log “no update” and skip 4. Record the ingestion run in `ingestion_runs` table

Fault tolerance: - Attempt 1 fails → Log error details → Wait 60 seconds - Attempt 2 fails → Log error details → Wait 300 seconds (5 min) - Attempt 3 fails → Log error details → Create GitHub issue with `data-source-alert` label → Stop retrying this source

Known NCRB quirks to handle (from Stage 1): - 2021 files have timestamp prefixes in filenames that 2022/2023 don't — strip on storage, preserve original in metadata - Use `requests.Session()` to maintain cookies across downloads within a run - Some pages may require maintaining a session for access

Metadata to record per file:

```
{
    "ingestion_id": "uuid",
    "source_url": "https://www.ncrb.gov.in/...",
    "downloaded_at": "2025-03-20T06:00:00Z",
    "file_hash_sha256": "abc123...",
    "file_size_bytes": 11960,
    "file_format": "xlsx",
    "original_filename": "State-UT-wise IPC Crimes During
        2023.xlsx",
    "stored_path": "data/raw/ncrb-crime-in-india/2023/states-
        uts/...",
    "http_status": 200,
    "is_new_data": true
}
```

How to test (without hitting NCRB repeatedly):

Since you already have all 75 files from Stage 1:

1. **Metadata recording test:** Point the ingester at your existing `data/raw/` files. Have it calculate hashes and create `data_sources` and `ingestion_runs` entries. Verify entries appear in database.

2. **Single file download test:** Pick one small States/UTs file. Download it from NCRB. Compare hash against your Stage 1 copy. They should match if NCRB hasn't updated.

3. **Rate limiting test:** Add logging that prints timestamp of every HTTP request. Run ingester against 3 files. Verify ≥1 second gap between requests.

4. **Failure handling test:** Point at an intentionally wrong URL. Verify 3 retries with increasing delays. Verify error logged with full context.

Exit condition: Ingestion run appears in database with correct metadata. Rate limiting verified. Failure handling verified.

Estimated time: 3-4 hours

Step 4: Build the Excel Parser

File: `pipeline/parser/excel_parser.py`
Supporting files: `header_detector.py`, `summary_row_detector.py`, `field_mapper.py`

What it does: Takes an Excel file path as input, returns a list of structured record dictionaries with confidence scores.

Build in this order:

4A: Field Mapper (`field_mapper.py`)

Maps raw NCRB column names to canonical BharatData field names. Based on your `FIELD_MAPPINGS.md` from Stage 1.

Key mappings:

```
FIELD_MAPPINGS = {
    # State identification (multiple variations)
    "State/UT": "state_raw",
    "States/UTs": "state_raw",
    "STATE/UT": "state_raw",

    # Serial numbers (skip)
    "Sl. No.": "_skip",
    "S.No.": "_skip",
    "Sl.No": "_skip",

    # Crime counts
    "Total Cognizable IPC Crimes": "total_cases",
    "Total IPC Crimes": "total_cases",
    "Incidence": "total_cases",

    # Rates
    "Crime Rate": "rate_per_lakh",

    # Disposal pipeline
    "Cases Charge Sheeted": "chargesheeted",
    "Cases Conviction": "convicted",
    "Cases Acquittal/Discharge": "acquitted",
    "Cases Pending Investigation": "pending_investigation",
    "Cases Pending Trial": "pending_trial",
}
```

Fields mapped to `"_skip"` are ignored. Unmapped columns are logged as warnings.

4B: Header Detector (`header_detector.py`)

NCRB files have varying header positions. Some have title text in rows 0-2, blank rows, then headers. Strategy: scan rows top-down and find the first row where ≥3 cells match known column name patterns from the field mapper.

4C: Summary Row Detector (`summary_row_detector.py`)

Identifies and flags Total/Grand Total/All India rows. These are aggregations, not state-level data.

```
SUMMARY_INDICATORS = [
    "total", "grand total", "all india",
    "total (all india)", "total states", "total uts",
    "total (states)", "total (uts)", "total (states+uts)"
]
```

Summary rows are extracted separately — not included in state/district data records.

4D: Main Excel Parser (`excel_parser.py`)

The parsing flow: 1. Open Excel file with pandas/openpyxl 2. Detect header row using header detector 3. Map columns to canonical fields using field mapper 4. For each data row: - Check if it's a summary row → flag and skip for data - Extract all mapped field values - Convert data types (strings → numbers, handle Indian notation) - Handle null representations (-, NA, N/A, empty → `None`) - Calculate confidence score - Attach source file metadata 5. Return list of parsed record dictionaries

Confidence scoring for Excel: - Base: 1.0 (Excel files parse cleanly) - Minor issue (number format conversion): -0.05 - Moderate issue (ambiguous cell value): -0.10 - Major issue (failed cell parsing): -0.20 - Timestamp prefix in filename (2021 quirk): -0.05

Output per record:

```
{
    "state_raw": "ANDHRA PRADESH",
    "year": 2023,
    "total_cases": 123456,
    "rate_per_lakh": 157.3,
    "confidence": 1.0,
    "source_file": "State-UT-wise IPC Crimes During 2023.xlsx",
    "source_row": 5,
    "parsing_notes": [],
```

```
    "table_type": "state_ipc"  # Identifies which table type this
        came from
}
```

How to test: 1. Run parser against all 66 Excel files. Verify each produces the expected number of rows. 2. For state-level files: verify exactly 36 entities (28 states + 8 UTs) 3. Pick 50 random cells across 5 different file types. Manually compare parser output against original Excel file. All 50 must match. 4. Verify confidence scores: clean files = 1.0, files with timestamp prefix issue = 0.95 5. Verify all field names match canonical schema from FIELD_MAPPINGS.md 6. Verify null values are Python `None`, not 0, empty string, dash, or "NA" 7. Verify all numeric values are int or float, not strings 8. Run your Stage 1 `examine_all.py` for backward compatibility check

Exit condition: 66/66 files parse successfully. 50 random cell spot-check passes. All data types correct.

Estimated time: 4-6 hours

Step 5: Build the PDF Parser

File: `pipeline/parser/pdf_parser.py`

What it does: Handles metropolitan city data not available in Excel Additional Tables. You already built the prototype in Stage 1 (`process_metro_data.py`). This step productionizes it.

Key differences from Stage 1 prototype: - Structured output matching the same record format as Excel parser - Confidence scoring integrated (base 0.9 for PDF extraction) - Error logging through the same parse_failures system - Validation against known list of 34 metropolitan cities

How to test: 1. Compare output against Stage 1's 9 JSON files — results should be identical 2. Verify the Agra 2023 IPC count of 7026 (your ground truth benchmark from Stage 1) 3. Pick 5 additional city-metric combinations, manually verify against source PDF 4. Verify all 34 cities appear in output for each parsed table

Exit condition: Output matches Stage 1 JSON files exactly. Agra benchmark verified. All 34 cities present.

Estimated time: 2-3 hours

Step 6: Build the Normalizer

File: `pipeline/normalizer/normalizer.py`
Supporting files: All files in `normalizer/` directory

What it does: Transforms parsed records into canonical form. This is the intellectual core of the pipeline.

Build the sub-components in this order:

6A: State Resolver (`state_resolver.py`)

Loads `data/canonical-mappings/states.json`. For each record's `state_raw` field: - Trim whitespace, convert to lowercase - Look up in variations index - If found: return canonical name + state code + confidence_impact = 0 - If NOT found: log to `unmapped-entities.json`, return raw name + code "UNKNOWN" + confidence_impact = -0.2

Critical: Never drop records that can't be resolved. Pass through with raw name and reduced confidence.

6B: District Resolver (`district_resolver.py`)

For district-level records. Start with exact and case-insensitive matching. Log any unresolved districts but pass them through. You don't need complete district canonical mappings for v1.

6C: City Resolver (`city_resolver.py`)

Match city names against the known list of 34 metropolitan cities. Handle variations like "Bangalore" → "Bengaluru".

6D: Type Enforcer (`type_enforcer.py`)

Handles data type conversions based on your Stage 1 findings: - Indian notation: `"1,23,456"` → `123456` (detect by comma position pattern) - International: `"1,234,567"` → `1234567` - Percentages: `"12.5%"` → `12.5` with `value_type: "percentage"` - Lakhs/Crores: `"2.5 crore"` → `25000000` - Null standardization: `""`, `"-"`, `"NA"`, `"N/A"` → `None` (preserve original in metadata) - Zero stays zero (it's valid data, not null)

6E: Category Mapper (`category_mapper.py`)

Maps raw crime category descriptions to canonical IDs:

```
"Total Cognizable IPC Crimes" → "ipc-total"
"Crime Against Women" → "crimes-against-women"
```

```
"Cyber Crimes" → "cyber-crime"
# etc.
```

6F: Boundary Annotator (`boundary_annotator.py`)

Adds `boundary_note` field for historical boundary changes: - Telangana + year < 2014 → note about not existing separately - Andhra Pradesh + year < 2014 → note about including Telangana region - Ladakh + year < 2019 → note about being part of J&K - J&K + year ≥ 2019 → note about UT status change

6G: Confidence Scorer (`confidence_scorer.py`)

Aggregates all confidence adjustments from parsing and normalization into a final score. Enforces the 0.0-1.0 range.

6H: Range Validator (`range_validator.py`)

Flags values outside plausible ranges: - `total_cases`: 0 to 5,000,000 - `rate_per_lakh`: 0 to 5,000 - `year`: 2000 to 2030 - `confidence`: 0.0 to 1.0

6I: Main Normalizer (`normalizer.py`)

Orchestrates all sub-components in sequence:

```
Parsed Record
  → Step 1: State Name Resolution
  → Step 2: District Name Resolution (if district-level)
  → Step 3: City Name Resolution (if city-level)
  → Step 4: Data Type Enforcement
  → Step 5: Category Mapping
  → Step 6: Historical Boundary Annotation
  → Step 7: Deduplication Check
  → Step 8: Confidence Adjustment
  → Step 9: Source Attribution Attachment
  → Step 10: Range Validation
  → Normalized Record
```

Source attribution fields added in Step 9: - `source_url`: NCRB Additional Tables page URL - `source_file`: original filename - `report_name`: "Crime in India {year}" - `publishing_body`: "National Crime Records Bureau" - `collection_date`: timestamp of BharatData ingestion - `normalizer_version`: "1.0.0" - `ingestion_batch_id`: UUID of the current pipeline run

How to test: 1. Test state resolution with every variation from `state-name-variations.md` — every variation must resolve correctly 2. Test with intentionally wrong names ("Nonexistent State") — verify not dropped, logged to unmapped, confidence reduced by 0.2 3. Test boundary annotations: Telangana + year 2012 →

correct note; Ladakh + year 2018 → correct note 4. Test type enforcement edge cases: `"1,23,456"` → `123456`; `"-"` → `None`; `"0"` → `0` (not None) 5. Test deduplication: run same dataset twice, second run produces zero new records 6. **Cross-validate:** Take Maharashtra total IPC crimes from state-level file, sum all Maharashtra districts from district-level file, verify they match (within 1% tolerance)

Exit condition: All state variations resolve. Type enforcement handles all edge cases. Boundary notes correct. Cross-validation passes.

Estimated time: 4-6 hours

Step 7: Build the Loader

File: `pipeline/loader/loader.py`, `deduplicator.py`

What it does: Takes normalized records and writes them to PostgreSQL with full lineage.

Key behaviors: - Operates within database transactions — all records from one ingestion run commit together or not at all - Routes records to correct table based on geographic level: - Records with only `state` → `crime_records_state` - Records with `district` → `crime_records_district` - Records with `city` → `crime_records_city` - Also inserts parse failures into `parse_failures` table - Also inserts normalization decisions into `normalization_log` table - Updates `ingestion_runs` with final counts after all inserts

Deduplication strategy: Before inserting, check if a record with the same (state, year, category, source_file) already exists. If it does, skip insertion. This makes the pipeline idempotent — running it twice produces the same result without duplicates.

How to test: 1. Load 5 state-level records. Query database. Verify 5 rows appear with correct values. 2. Test transaction rollback: corrupt one record (e.g., confidence = 2.0 which violates CHECK constraint). Verify entire batch of 10 rolls back — 0 records inserted. 3. Load all state-level data for 2023. Verify ~36 records per category. 4. Load district-level data for 2023. Verify ~780 records per category. 5. Load city-level data. Verify 34 records per year per category. 6. Spot-check 10 random records: query from database, compare against original Excel file. 7. Run the same load again. Verify zero new records inserted (deduplication).

Exit condition: All records in database. Transaction rollback works. Deduplication works. Spot-check passes.

Estimated time: 2-3 hours

Step 8: Build the Pipeline Orchestrator

File: `pipeline/run_pipeline.py`

What it does: Ties all four components together into a single executable pipeline.

Command-line interface:

```
# Full pipeline for all years
python -m pipeline.run_pipeline --years 2021 2022 2023 --steps all

# Parse and load only (skip ingestion)
python -m pipeline.run_pipeline --years 2023 --steps parse
        normalize load

# Dry run — process everything but don't write to database
python -m pipeline.run_pipeline --years 2023 --steps all --dry-run

# Single year
python -m pipeline.run_pipeline --years 2023 --steps all
```

Execution flow: 1. Create new `ingestion_runs` record with status "running" 2. **Ingest step:** For each year, run ingester to check for new files 3. **Parse step:** For each raw file, run appropriate parser (Excel or PDF) 4. **Normalize step:** Run normalizer on all parsed records 5. **Load step:** Write all normalized records to database (or skip if dry-run) 6. Update `ingestion_runs` with final counts and status 7. Print execution summary

Summary report format:

```
═══════════════════════════════════════════
Pipeline Execution Summary
═══════════════════════════════════════════

Ingestion Run ID: abc-123-def
Started:    2025-03-21T06:00:00Z
Completed:  2025-03-21T06:05:32Z
Duration:   5m 32s

Files processed:    75
Records extracted:  11,490
Records normalized: 11,490
Records stored:     11,490
Parse failures:     0

Confidence distribution:
  1.0:  9,200 records (80.1%)
  0.9:  1,530 records (13.3%)
  0.8:    760 records  (6.6%)
  <0.5:     0 records  (0.0%)
```

```
States mapped:     36/36
Unmapped entities: 0
═══════════════════════════════════════════
```

How to test: 1. Run with `--dry-run` first. Verify summary report looks correct. Verify no records written to database. 2. Run without dry-run. Verify all records in database. 3. Verify record counts match expected values from your Stage 1 data inventory.

Exit condition: Full pipeline runs end-to-end. Summary report shows expected counts. Database contains correct data.

Estimated time: 2-3 hours

Step 9: Build the Health Check Monitor

File: `pipeline/monitoring/health_check.py`
GitHub Action: `.github/workflows/daily-health-check.yml`

What the script does: 1. For each registered source URL in `data_sources` table: - Send HTTP HEAD request (not GET, to minimize load) - Check status code is 200 - If accessible, compare page hash against last known hash 2. If URL returns non-200: log the failure with status code 3. If hash has changed: log that new data may be available 4. Exit with code 0 if all checks pass, code 1 if any fail

GitHub Action workflow:

```yaml
name: Daily Data Source Health Check
on:
  schedule:
    - cron: '0 6 * * *'  # 06:00 UTC daily (11:30 AM IST)
  workflow_dispatch:      # Allow manual trigger

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install requests psycopg2-binary python-dotenv
      - run: python -m pipeline.monitoring.health_check
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
```

How to test: 1. Run locally. Verify it reports correct status for NCRB URLs. 2. Test with intentionally wrong URL. Verify failure detected and reported. 3. Push workflow to GitHub. Trigger manually with `workflow_dispatch`. Verify it runs.

Exit condition: Health check correctly detects URL status. GitHub Action executes successfully.

Estimated time: 1-2 hours

Step 10: Build the Quality Validation Script

File: `pipeline/monitoring/quality_check.py`

What it does: Runs comprehensive quality checks against the loaded data. Implements the validation workflow from your Stage 2 specs.

Checks to implement (6 total):

Check	What It Verifies	Failure Condition
Required Fields	No nulls in state, state_code, year, category, total_cases, confidence, source_url, collection_date	Any null in mandatory field
Confidence Distribution	Average confidence across all records	Average < 0.7 or >10% records below 0.5
Completeness	All 36 states/UTs have records for each year	Missing states
Consistency	Record counts stable between consecutive years	Count changed >50% for any state
Statistical Outliers	Values within 3 standard deviations of historical mean	Flagged outliers
Data Types	Confidence 0.0-1.0, year 2000-2030, no negative total_cases	Any violation

Output: JSON report with pass/fail for each check. Exit code 0 if all pass, 1 if any fail.

How to test: 1. Run against your fully loaded database 2. Review the report — investigate any failures 3. Some failures may indicate real data issues in NCRB source data (document these as known issues)

Exit condition: Script produces a valid JSON report. All checks either pass or failures are documented as known source data issues.

Estimated time: 2-3 hours

Step 11: End-to-End Pipeline Run

This is the final verification. Run everything together and validate.

Execute:

```
# Full pipeline run
python -m pipeline.run_pipeline --years 2021 2022 2023 --steps all

# Quality validation
python -m pipeline.monitoring.quality_check
```

Manual verification (20-record spot check): 1. Query 20 random records from the database spanning: - At least 3 different years - At least 3 different crime categories - At least 2 state-level, 2 district-level, 2 city-level records 2. For each record, open the original source file (Excel or PDF) 3. Find the exact row/cell in the source 4. Verify the value matches

Database count verification:

```
SELECT 'state' as level, COUNT(*) FROM crime_records_state
UNION ALL
SELECT 'district', COUNT(*) FROM crime_records_district
UNION ALL
SELECT 'city', COUNT(*) FROM crime_records_city;
```

Compare counts against your Stage 1 data inventory expectations.

Exit condition: All 20 spot-checked records match source files. Record counts match expectations. Quality validation passes.

Estimated time: 2-3 hours

Step 12: Write Stage 3 Completion Report

File: `Docs/stages-execution-docs/stage-3/STAGE_3_COMPLETION_REPORT.md`

Document: - What was built (list all files with descriptions) - Database schema summary (tables, indexes, record counts) - Pipeline execution metrics (time, records, confidence distribution) - Quality validation results (all 6 checks) - 20-record spot-check results - Any data quality issues discovered - Known limitations

This report is your quality gate. It proves the pipeline works before you build SDKs on top of it.

Estimated time: 1 hour

Stage 3 Build Order Summary

Step	Component	Test Before Moving On	Time
0A	Neon database setup	Connection test passes	30 min
0B	Python dependencies	All imports succeed	15 min
0C	Logger configuration	Log output visible	15 min
1	Database schema	All 7 tables visible	1-2 hrs
2	Rate limiter	≥1 sec gap verified	30 min
3	Ingester	Metadata recorded, retries work	3-4 hrs
4	Excel parser	66 files parse, 50 cells verified	4-6 hrs
5	PDF parser	Matches Stage 1 JSON, Agra benchmark	2-3 hrs
6	Normalizer	State variations resolve, types enforced	4-6 hrs
7	Loader	Records in DB, rollback works, dedup works	2-3 hrs
8	Orchestrator	Full pipeline runs, summary correct	2-3 hrs
9	Health check	URL monitoring works, GitHub Action runs	1-2 hrs
10	Quality validation	All checks pass or failures documented	2-3 hrs
11	End-to-end run	20-record spot-check passes	2-3 hrs
12	Completion report	Written with all results	1 hr

Step	Component	Test Before Moving On	Time
Total			**~27-40 hrs**

The Single Rule for Stage 3

Build each component. Test it thoroughly in isolation. Verify it produces correct output. Only then move to the next component.

If at any point a component produces wrong data, stop. Fix it. Re-verify. Then continue.

A pipeline that processes 11,000 records correctly is worth infinitely more than one that processes 50,000 records with 5% errors. Your users — journalists, researchers, developers — will cite these numbers. One wrong number in a published article destroys your reputation permanently.

Database Selection, Incremental Ingestion & GitHub Workflows

Part 1: Database Selection — Supabase Wins

Why Neon Feels Slow

Neon uses a serverless architecture with cold starts. When your database hasn't been accessed for a few minutes, the compute instance suspends. The next query triggers a cold start that takes 1-3 seconds. For a pipeline inserting thousands of records, this causes noticeable lag, especially with frequent small transactions.

Comparing Your Three Options

Feature	Neon	Supabase	Convex
Database type	PostgreSQL (serverless)	PostgreSQL (always-on)	Custom reactive DB

Feature	Neon	Supabase	Convex
Cold starts	Yes (1-3 sec)	No (always running on free tier)	No
Free tier storage	500 MB	500 MB	1 GB
Free tier compute	Auto-suspend after 5 min inactivity	Always on (2 projects)	100K function calls/month
Connection pooling	Built-in but limited	PgBouncer built-in	Not applicable
SQL support	Full PostgreSQL	Full PostgreSQL	No SQL — custom query language
Dashboard	Basic	Excellent (table editor, SQL editor, logs)	Good
Your existing schema works?	Yes	Yes (zero changes)	No (complete rewrite needed)
Bulk insert speed	Slow with cold starts	Fast (persistent connection)	Different paradigm
REST API built-in	No	Yes (PostgREST auto-generated)	Yes
File storage	No	1 GB free (S3-compatible)	1 GB free
Auth built-in	No	Yes	Yes
Realtime subscriptions	No	Yes	Yes

Recommendation: Supabase

Convex is eliminated immediately. Convex is not PostgreSQL. It uses a custom reactive database with its own query language. Your entire database schema, all SQL queries, all SQLAlchemy models, and all your pipeline code would need to be completely rewritten. The pipeline spec, the loader, the quality checks — all assume PostgreSQL. Convex is a fundamentally different paradigm designed for real-time collaborative apps, not for a data pipeline serving structured government data.

Supabase beats Neon for your specific use case because:

1. **No cold starts.** Supabase keeps your database compute running on the free tier. Your pipeline inserts will not wait 1-3 seconds for the database to wake up.

2. **Built-in connection pooling via PgBouncer.** This is the single biggest speed improvement. PgBouncer maintains a pool of database connections. Instead of opening and closing a connection for every batch of records, your pipeline reuses connections from the pool. This eliminates connection overhead that slows down bulk inserts.

3. **Your existing schema works with zero changes.** Supabase is standard PostgreSQL. Every CREATE TABLE statement, every index, every CHECK constraint, every query you have designed works exactly as written.

4. **Built-in REST API (PostgREST).** Supabase automatically generates a REST API for every table. When you build the TypeScript and Python SDKs later, they can optionally use this REST API instead of requiring a custom API server. This could eliminate the need for a separate API deployment entirely.

5. **Dashboard with SQL editor.** You can visually inspect your data, run ad-hoc queries, and spot-check records directly in the browser. This accelerates debugging during pipeline development.

6. **1 GB file storage.** Supabase includes S3-compatible storage. You could store your raw NCRB files here instead of in the repository, keeping the repo size small.

Part 2: Supabase Setup Guide

Step 1: Create Supabase Project

1. Go to `https://supabase.com` and sign up with GitHub
2. Click “New Project”
3. Project name: `bharatdata`
4. Database password: generate a strong password and save it securely
5. Region: choose the closest to your users (Mumbai if available, or Singapore)
6. Click “Create new project”
7. Wait 1-2 minutes for provisioning

Step 2: Get Connection Details

After project creation, go to **Project Settings → Database**

You will see several connection strings. You need two:

Direct connection (for your pipeline — bulk inserts):

```
postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:5432/postgres
```

Connection pooler (for your API — many concurrent short queries):

```
postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:6543/postgres?pgbouncer=true
```

The difference: - Port `5432` = direct connection (use for pipeline bulk inserts) - Port `6543` = pooled connection via PgBouncer (use for API/SDK later)

Step 3: Update Your Environment File

Update your `.env`:

```
# Direct connection for pipeline (bulk operations)
DATABASE_URL=postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:5432/postgres

# Pooled connection for API (concurrent short queries) — used
later in Stage 4+
DATABASE_URL_POOLED=postgresql://postgres.[project-ref]:
[password]@aws-0-[region].pooler.supabase.com:6543/postgres?
pgbouncer=true

# Supabase specific
SUPABASE_URL=https://[project-ref].supabase.co
SUPABASE_ANON_KEY=eyJ...your-anon-key...

LOG_LEVEL=INFO
```

Update `.env.example`:

```
DATABASE_URL=postgresql://postgres.xxxxx:password@aws-0-
region.pooler.supabase.com:5432/postgres
DATABASE_URL_POOLED=postgresql://postgres.xxxxx:password@aws-0-
region.pooler.supabase.com:6543/postgres?pgbouncer=true
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_ANON_KEY=your-anon-key-here
LOG_LEVEL=INFO
```

Step 4: Verify Connection

```python
import os
from dotenv import load_dotenv
import psycopg2
import time

load_dotenv()
```

```python
# Test direct connection
start = time.time()
conn = psycopg2.connect(os.getenv("DATABASE_URL"))
cursor = conn.cursor()
cursor.execute("SELECT version();")
version = cursor.fetchone()[0]
elapsed = time.time() - start

print(f"Connected in {elapsed:.3f}s")
print(f"PostgreSQL: {version}")

cursor.close()
conn.close()
```

You should see connection time under 500ms (vs 1-3 seconds with Neon cold starts).

Step 5: Create Database Schema

You have two options for creating tables:

Option A: SQL Editor in Supabase Dashboard (recommended for first time)

1. Go to your Supabase project dashboard
2. Click “SQL Editor” in the left sidebar
3. Click “New Query”
4. Paste your complete schema SQL
5. Click “Run”
6. Go to “Table Editor” to verify all tables were created

Option B: Python script (for automated/repeatable setup)

```python
# pipeline/loader/schema.py

import os
from dotenv import load_dotenv
import psycopg2

load_dotenv()

SCHEMA_SQL = """
-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Data sources registry
CREATE TABLE IF NOT EXISTS data_sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    source_url TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
```

```
    data_format TEXT NOT NULL,
    update_frequency TEXT NOT NULL,
    last_ingested_at TIMESTAMPTZ,
    status TEXT NOT NULL DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Ingestion run log
CREATE TABLE IF NOT EXISTS ingestion_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id UUID REFERENCES data_sources(id),
    started_at TIMESTAMPTZ NOT NULL,
    completed_at TIMESTAMPTZ,
    status TEXT NOT NULL DEFAULT 'running',
    files_processed INT DEFAULT 0,
    records_extracted INT DEFAULT 0,
    records_stored INT DEFAULT 0,
    records_skipped INT DEFAULT 0,
    parse_failures INT DEFAULT 0,
    error_message TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- State-level crime records
CREATE TABLE IF NOT EXISTS crime_records_state (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    state TEXT NOT NULL,
    state_code TEXT NOT NULL,
    year INT NOT NULL,
    category TEXT NOT NULL,
    category_label TEXT NOT NULL,
    total_cases INT,
    rate_per_lakh FLOAT,
    chargesheeted INT,
    convicted INT,
    acquitted INT,
    pending_investigation INT,
    pending_trial INT,
    confidence FLOAT NOT NULL,
    source_url TEXT NOT NULL,
    source_file TEXT NOT NULL,
    report_name TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    collection_date TIMESTAMPTZ NOT NULL,
    normalizer_version TEXT NOT NULL,
    boundary_note TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT state_confidence_range
        CHECK (confidence >= 0.0 AND confidence <= 1.0),
    CONSTRAINT state_year_range
```

```
        CHECK (year >= 2000 AND year <= 2030),
    CONSTRAINT state_unique_record
        UNIQUE (state, year, category, source_file)
);

-- District-level crime records
CREATE TABLE IF NOT EXISTS crime_records_district (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    state TEXT NOT NULL,
    state_code TEXT NOT NULL,
    district TEXT NOT NULL,
    year INT NOT NULL,
    category TEXT NOT NULL,
    category_label TEXT NOT NULL,
    total_cases INT,
    rate_per_lakh FLOAT,
    confidence FLOAT NOT NULL,
    source_url TEXT NOT NULL,
    source_file TEXT NOT NULL,
    report_name TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    collection_date TIMESTAMPTZ NOT NULL,
    normalizer_version TEXT NOT NULL,
    boundary_note TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT district_confidence_range
        CHECK (confidence >= 0.0 AND confidence <= 1.0),
    CONSTRAINT district_unique_record
        UNIQUE (state, district, year, category, source_file)
);

-- City-level crime records
CREATE TABLE IF NOT EXISTS crime_records_city (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    city TEXT NOT NULL,
    parent_state TEXT NOT NULL,
    year INT NOT NULL,
    category TEXT NOT NULL,
    category_label TEXT NOT NULL,
    total_cases INT,
    rate_per_lakh FLOAT,
    confidence FLOAT NOT NULL,
    source_url TEXT NOT NULL,
    source_file TEXT NOT NULL,
    report_name TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    collection_date TIMESTAMPTZ NOT NULL,
    normalizer_version TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
```

```
    CONSTRAINT city_confidence_range
        CHECK (confidence >= 0.0 AND confidence <= 1.0),
    CONSTRAINT city_unique_record
        UNIQUE (city, year, category, source_file)
);

-- Parse failure log
CREATE TABLE IF NOT EXISTS parse_failures (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    source_file TEXT NOT NULL,
    source_sheet TEXT,
    row_number INT,
    column_name TEXT,
    raw_value TEXT,
    expected_type TEXT,
    error_message TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Normalization decision log
CREATE TABLE IF NOT EXISTS normalization_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    record_id UUID,
    field TEXT NOT NULL,
    input_value TEXT NOT NULL,
    output_value TEXT NOT NULL,
    mapping_source TEXT NOT NULL,
    confidence_impact FLOAT DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- File tracking for incremental ingestion
CREATE TABLE IF NOT EXISTS ingested_files (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    file_path TEXT NOT NULL,
    file_hash_sha256 TEXT NOT NULL,
    file_size_bytes INT,
    year INT NOT NULL,
    table_type TEXT NOT NULL,
    geographic_level TEXT NOT NULL,
    records_extracted INT DEFAULT 0,
    ingested_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT unique_file_hash
        UNIQUE (file_hash_sha256)
);

-- ===== INDEXES =====

-- State-level queries
```

```
CREATE INDEX IF NOT EXISTS idx_crime_state_year
    ON crime_records_state(state, year);
CREATE INDEX IF NOT EXISTS idx_crime_year
    ON crime_records_state(year);
CREATE INDEX IF NOT EXISTS idx_crime_category
    ON crime_records_state(category);
CREATE INDEX IF NOT EXISTS idx_crime_state_cat_year
    ON crime_records_state(state, category, year);
CREATE INDEX IF NOT EXISTS idx_state_confidence_filter
    ON crime_records_state(confidence)
    WHERE confidence >= 0.5;

-- District-level queries
CREATE INDEX IF NOT EXISTS idx_district_state
    ON crime_records_district(state);
CREATE INDEX IF NOT EXISTS idx_district_state_year
    ON crime_records_district(state, year);
CREATE INDEX IF NOT EXISTS idx_district_district
    ON crime_records_district(district);
CREATE INDEX IF NOT EXISTS idx_district_category
    ON crime_records_district(category);
CREATE INDEX IF NOT EXISTS idx_district_confidence_filter
    ON crime_records_district(confidence)
    WHERE confidence >= 0.5;

-- City-level queries
CREATE INDEX IF NOT EXISTS idx_city_name
    ON crime_records_city(city);
CREATE INDEX IF NOT EXISTS idx_city_year
    ON crime_records_city(city, year);

-- File tracking
CREATE INDEX IF NOT EXISTS idx_ingested_files_hash
    ON ingested_files(file_hash_sha256);
CREATE INDEX IF NOT EXISTS idx_ingested_files_path
    ON ingested_files(file_path);
"""


def create_schema():
    conn = psycopg2.connect(os.getenv("DATABASE_URL"))
    cursor = conn.cursor()

    cursor.execute(SCHEMA_SQL)
    conn.commit()

    # Verify tables
    cursor.execute("""
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'public'
        ORDER BY table_name;
```

```
    """)

    tables = [row[0] for row in cursor.fetchall()]
    print(f"Created {len(tables)} tables:")
    for t in tables:
        print(f"  ✓ {t}")

    cursor.close()
    conn.close()

    return tables


if __name__ == "__main__":
    create_schema()
```

Notice the critical additions compared to the original schema:

1. **`UNIQUE` constraints on data tables** — This is what prevents duplicate records. The combination `(state, year, category, source_file)` uniquely identifies a record. If you try to insert a duplicate, PostgreSQL rejects it.

2. **`ingested_files` table** — This tracks which files have already been processed, enabling incremental ingestion.

3. **`records_skipped` column in `ingestion_runs`** — Tracks how many records were skipped because they already existed.

Part 3: Incremental Ingestion — Never Re-Insert Existing Data

This solves your problem of records being re-inserted when you terminate and restart the pipeline.

The Strategy: Three Levels of Skip Detection

Level 1: File-Level Skip (Fastest)

Before parsing a file, check if it has already been fully ingested by looking up its SHA-256 hash in the `ingested_files` table.

```
# pipeline/ingester/hash_tracker.py

import hashlib
import os
import psycopg2
```

```python
class HashTracker:
    """Tracks which files have been ingested using SHA-256
        hashes."""

    def __init__(self, db_url: str):
        self.db_url = db_url

    def calculate_hash(self, file_path: str) -> str:
        """Calculate SHA-256 hash of a file."""
        sha256 = hashlib.sha256()
        with open(file_path, "rb") as f:
            for chunk in iter(lambda: f.read(8192), b""):
                sha256.update(chunk)
        return sha256.hexdigest()

    def is_file_ingested(self, file_path: str) -> bool:
        """Check if a file has already been fully ingested."""
        file_hash = self.calculate_hash(file_path)

        conn = psycopg2.connect(self.db_url)
        cursor = conn.cursor()
        cursor.execute(
            "SELECT id FROM ingested_files WHERE file_hash_sha256
        = %s",
            (file_hash,)
        )
        result = cursor.fetchone()
        cursor.close()
        conn.close()

        return result is not None

    def mark_file_ingested(
        self,
        file_path: str,
        year: int,
        table_type: str,
        geographic_level: str,
        records_extracted: int
    ):
        """Mark a file as fully ingested."""
        file_hash = self.calculate_hash(file_path)
        file_size = os.path.getsize(file_path)

        conn = psycopg2.connect(self.db_url)
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO ingested_files
                (file_path, file_hash_sha256, file_size_bytes,
                 year, table_type, geographic_level,
        records_extracted)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
```

```
            ON CONFLICT (file_hash_sha256) DO NOTHING
        """, (
            file_path, file_hash, file_size,
            year, table_type, geographic_level, records_extracted
        ))
        conn.commit()
        cursor.close()
        conn.close()

    def get_ingestion_status(self) -> dict:
        """Get summary of ingested files."""
        conn = psycopg2.connect(self.db_url)
        cursor = conn.cursor()
        cursor.execute("""
            SELECT year, geographic_level, COUNT(*),
        SUM(records_extracted)
            FROM ingested_files
            GROUP BY year, geographic_level
            ORDER BY year, geographic_level
        """)

        status = {}
        for row in cursor.fetchall():
            year, level, file_count, record_count = row
            status[f"{year}_{level}"] = {
                "files": file_count,
                "records": record_count
            }

        cursor.close()
        conn.close()
        return status
```

Level 2: Record-Level Skip (Using UNIQUE Constraints)

The `UNIQUE` constraint on each data table prevents duplicate records at the database level. When inserting, use `ON CONFLICT DO NOTHING`:

```
# In pipeline/loader/loader.py

def insert_state_record(self, record: dict, cursor) -> str:
    """Insert a state-level record. Returns 'inserted' or
        'skipped'."""
    try:
        cursor.execute("""
            INSERT INTO crime_records_state (
                ingestion_run_id, state, state_code, year,
                category, category_label, total_cases,
        rate_per_lakh,
                chargesheeted, convicted, acquitted,
                pending_investigation, pending_trial,
```

```
                confidence, source_url, source_file,
                report_name, publishing_body, collection_date,
                normalizer_version, boundary_note
            ) VALUES (
                %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
        %s,
                %s, %s, %s, %s, %s, %s, %s, %s
            )
            ON CONFLICT (state, year, category, source_file)
            DO NOTHING
            RETURNING id
        """, (
            record["ingestion_run_id"],
            record["state"],
            record["state_code"],
            record["year"],
            record["category"],
            record["category_label"],
            record.get("total_cases"),
            record.get("rate_per_lakh"),
            record.get("chargesheeted"),
            record.get("convicted"),
            record.get("acquitted"),
            record.get("pending_investigation"),
            record.get("pending_trial"),
            record["confidence"],
            record["source_url"],
            record["source_file"],
            record["report_name"],
            record["publishing_body"],
            record["collection_date"],
            record["normalizer_version"],
            record.get("boundary_note"),
        ))

        result = cursor.fetchone()
        return "inserted" if result else "skipped"

    except Exception as e:
        return f"error: {str(e)}"
```

Level 3: Batch Tracking (Resume After Termination)

Track which files within an ingestion run have been completed. If the pipeline is terminated mid-run, the next run picks up where it left off.

```
# In pipeline/run_pipeline.py — orchestrator logic

def run_pipeline(years: list, steps: list, dry_run: bool = False):
    """Main pipeline orchestrator with incremental processing."""
```

```
db_url = os.getenv("DATABASE_URL")
hash_tracker = HashTracker(db_url)

# Show what's already done
status = hash_tracker.get_ingestion_status()
if status:
    print("Previously ingested data:")
    for key, info in status.items():
        print(f"  {key}: {info['files']} files,
    {info['records']} records")
    print()

total_skipped_files = 0
total_processed_files = 0
total_inserted = 0
total_skipped_records = 0

for year in years:
    raw_dir = f"data/raw/ncrb-crime-in-india/{year}"

    # Get all files for this year
    all_files = get_files_for_year(raw_dir)

    for file_path in all_files:
        # LEVEL 1: Skip entire file if already ingested
        if hash_tracker.is_file_ingested(file_path):
            print(f"  SKIP (already ingested):
    {os.path.basename(file_path)}")
            total_skipped_files += 1
            continue

        print(f"  PROCESSING: {os.path.basename(file_path)}")
        total_processed_files += 1

        # Parse
        records = parse_file(file_path)

        # Normalize
        normalized = normalize_records(records)

        # Load (with record-level dedup via ON CONFLICT)
        if not dry_run:
            inserted, skipped = load_records(normalized,
    db_url)
            total_inserted += inserted
            total_skipped_records += skipped

            # Mark file as fully ingested
            hash_tracker.mark_file_ingested(
                file_path=file_path,
                year=year,
                table_type=detect_table_type(file_path),
```

```
                    geographic_level=detect_geo_level(file_path),
                    records_extracted=len(normalized)
                )

    # Print summary
    print(f"\n{'='*50}")
    print(f"Pipeline Summary")
    print(f"{'='*50}")
    print(f"Files skipped (already ingested):
        {total_skipped_files}")
    print(f"Files processed (new):
        {total_processed_files}")
    print(f"Records inserted:                 {total_inserted}")
    print(f"Records skipped (duplicates):
        {total_skipped_records}")
    print(f"{'='*50}")
```

Speed Optimization: Batch Inserts

Instead of inserting records one at a time, use batch inserts with `executemany` or `execute_values`:

```
# pipeline/loader/loader.py — batch insert method

from psycopg2.extras import execute_values


def bulk_insert_state_records(self, records: list[dict], cursor)
        -> tuple[int, int]:
    """
    Bulk insert state-level records using execute_values.
    Returns (inserted_count, skipped_count).
    """
    if not records:
        return 0, 0

    values = []
    for r in records:
        values.append((
            r["ingestion_run_id"],
            r["state"], r["state_code"], r["year"],
            r["category"], r["category_label"],
            r.get("total_cases"), r.get("rate_per_lakh"),
            r.get("chargesheeted"), r.get("convicted"),
            r.get("acquitted"), r.get("pending_investigation"),
            r.get("pending_trial"), r["confidence"],
            r["source_url"], r["source_file"],
            r["report_name"], r["publishing_body"],
            r["collection_date"], r["normalizer_version"],
            r.get("boundary_note"),
        ))
```

```
    # Count before insert
    cursor.execute("SELECT COUNT(*) FROM crime_records_state")
    count_before = cursor.fetchone()[0]

    # Bulk insert with conflict handling
    execute_values(
        cursor,
        """
        INSERT INTO crime_records_state (
            ingestion_run_id, state, state_code, year,
            category, category_label, total_cases, rate_per_lakh,
            chargesheeted, convicted, acquitted,
            pending_investigation, pending_trial,
            confidence, source_url, source_file,
            report_name, publishing_body, collection_date,
            normalizer_version, boundary_note
        ) VALUES %s
        ON CONFLICT (state, year, category, source_file) DO
        NOTHING
        """,
        values,
        page_size=500  # Insert 500 records per batch
    )

    # Count after insert
    cursor.execute("SELECT COUNT(*) FROM crime_records_state")
    count_after = cursor.fetchone()[0]

    inserted = count_after - count_before
    skipped = len(records) - inserted

    return inserted, skipped
```

Speed comparison: - One-at-a-time inserts: ~50-100 records/second - Batch inserts with `execute_values`: ~2,000-5,000 records/second - That's a 20-50x speedup

Connection Management for Speed

```
# pipeline/utils/db_connection.py

import os
from contextlib import contextmanager
import psycopg2
from psycopg2 import pool
from dotenv import load_dotenv

load_dotenv()

# Create a connection pool (reuse connections instead of opening
        new ones)
_pool = None
```

```python
def get_pool():
    global _pool
    if _pool is None:
        _pool = pool.SimpleConnectionPool(
            minconn=1,
            maxconn=5,
            dsn=os.getenv("DATABASE_URL")
        )
    return _pool


@contextmanager
def get_connection():
    """Get a database connection from the pool."""
    p = get_pool()
    conn = p.getconn()
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        p.putconn(conn)


@contextmanager
def get_cursor():
    """Get a database cursor with automatic connection
        management."""
    with get_connection() as conn:
        cursor = conn.cursor()
        try:
            yield cursor
        finally:
            cursor.close()
```

Part 4: GitHub Workflows — Complete Guide

Understanding GitHub Actions

GitHub Actions are automated workflows that run on GitHub's servers when triggered by events (schedule, push, manual trigger). Each workflow is a `.yml` file in `.github/workflows/`.

Workflow 1: Daily Health Check

File: `.github/workflows/daily-health-check.yml`

Purpose: Runs every day at 6:00 AM UTC (11:30 AM IST) to check if NCRB data source URLs are still accessible.

```yaml
name: Daily Data Source Health Check

# When this workflow runs
on:
  schedule:
    # Runs at 06:00 UTC every day (11:30 AM IST)
    # Cron format: minute hour day-of-month month day-of-week
    - cron: '0 6 * * *'

  # Allow manual trigger from GitHub Actions tab
  workflow_dispatch:

jobs:
  health-check:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Check out your repository code
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2: Set up Python
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      # Step 3: Install dependencies
      - name: Install dependencies
        run: |
          pip install requests psycopg2-binary python-dotenv
        loguru

      # Step 4: Run the health check script
      - name: Run health check
        id: health_check
        run: |
          python -m pipeline.monitoring.health_check
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
        continue-on-error: true

      # Step 5: Create GitHub issue if health check failed
      - name: Create issue on failure
        if: steps.health_check.outcome == 'failure'
```

```
        uses: actions/github-script@v7
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            const date = new Date().toISOString().split('T')[0];
            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: `🚨 Data Source Health Check Failed — ${date}
`,
              body: `## Health Check Failure\n\nThe daily health
check detected issues with one or more data sources.
\n\n**Date:** ${date}\n**Workflow Run:** $
{context.serverUrl}/${context.repo.owner}/$
{context.repo.repo}/actions/runs/${context.runId}
\n\nPlease check the workflow logs for details and
investigate the source availability.`,
              labels: ['data-source-alert', 'automated']
            });
```

Workflow 2: Weekly Quality Check

File: `.github/workflows/weekly-quality-check.yml`

Purpose: Runs every Monday at 8:00 AM UTC to validate data quality in the database.

```yaml
name: Weekly Data Quality Check

on:
  schedule:
    # Runs at 08:00 UTC every Monday
    - cron: '0 8 * * 1'

  workflow_dispatch:

jobs:
  quality-check:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install psycopg2-binary python-dotenv loguru
```

```yaml
- name: Run quality checks
  id: quality_check
  run: |
    python -m pipeline.monitoring.quality_check \
      --output quality_report.json
  env:
    DATABASE_URL: ${{ secrets.DATABASE_URL }}

- name: Read report
  id: read_report
  run: |
    echo "report<<EOF" >> $GITHUB_OUTPUT
    cat quality_report.json >> $GITHUB_OUTPUT
    echo "EOF" >> $GITHUB_OUTPUT

- name: Create quality report issue
  uses: actions/github-script@v7
  with:
    github-token: ${{ secrets.GITHUB_TOKEN }}
    script: |
      const fs = require('fs');
      const report =
  JSON.parse(fs.readFileSync('quality_report.json',
  'utf8'));
      const date = new Date().toISOString().split('T')[0];
      const status = report.status === 'PASSED' ? '✅' :
  '🚨';

      let body = `## Weekly Quality Report — ${date}\n\n`;
      body += `**Status:** ${status} ${report.status}\n`;
      body += `**Checks Run:** ${report.checks_run}\n`;
      body += `**Checks Passed:** ${report.checks_passed}
  \n`;
      body += `**Checks Failed:** ${report.checks_failed}
  \n\n`;

      body += `### Details\n\n`;
      for (const check of report.checks) {
        const icon = check.passed ? '✅' : '❌';
        body += `${icon} **${check.check}**\n`;
        if (check.issues && check.issues.length > 0) {
          for (const issue of check.issues) {
            body += `  - ${issue.message ||
  JSON.stringify(issue)}\n`;
          }
        }
        body += `\n`;
      }

      const labels = ['quality-report', 'automated'];
      if (report.status !== 'PASSED') {
        labels.push('needs-attention');
```

```
            }

            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: `${status} Weekly Quality Report — ${date}`,
              body: body,
              labels: labels
            });
```

Workflow 3: Pipeline Run on Demand

File: `.github/workflows/run-pipeline.yml`

Purpose: Allows you to trigger a full pipeline run from the GitHub Actions tab, useful when NCRB publishes new data.

```yaml
name: Run Data Pipeline

on:
  # Only manual trigger — never automatic for the full pipeline
  workflow_dispatch:
    inputs:
      years:
        description: 'Years to process (comma-separated)'
        required: true
        default: '2023'
        type: string
      dry_run:
        description: 'Dry run (no database writes)'
        required: false
        default: false
        type: boolean

jobs:
  run-pipeline:
    runs-on: ubuntu-latest
    timeout-minutes: 30  # Kill if it takes too long

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
```

```yaml
      - name: Run pipeline
        run: |
          YEARS="${{ github.event.inputs.years }}"
          DRY_RUN_FLAG=""
          if [ "${{ github.event.inputs.dry_run }}" = "true" ];
        then
            DRY_RUN_FLAG="--dry-run"
          fi

          # Convert comma-separated years to space-separated
          YEARS_ARGS=$(echo $YEARS | tr ',' ' ')

          python -m pipeline.run_pipeline \
            --years $YEARS_ARGS \
            --steps all \
            $DRY_RUN_FLAG
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}

      - name: Run validation after pipeline
        if: github.event.inputs.dry_run == 'false'
        run: |
          python -m pipeline.monitoring.quality_check \
            --output pipeline_validation.json
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}

      - name: Upload validation report
        if: github.event.inputs.dry_run == 'false'
        uses: actions/upload-artifact@v4
        with:
          name: pipeline-validation-report
          path: pipeline_validation.json
```

Setting Up GitHub Secrets

Before the workflows work, you must add your database credentials to GitHub:

1. Go to your GitHub repository
2. Click **Settings** → **Secrets and variables** → **Actions**
3. Click **New repository secret**
4. Add these secrets:

Secret Name	Value
`DATABASE_URL`	Your Supabase direct connection string (port 5432)

Never put database credentials in your workflow files.
Always use `${{ secrets.SECRET_NAME }}`.

Testing Your Workflows

Test Daily Health Check:

1. Push the `.yml` file to your `main` branch
2. Go to **Actions** tab in your repository
3. Click "Daily Data Source Health Check" in the left sidebar
4. Click "Run workflow" button (top right)
5. Click the green "Run workflow" button
6. Watch it execute — should take ~30 seconds
7. Check if an issue was created (only if health check detected a problem)

Test Weekly Quality Check:

Same process — trigger manually with "Run workflow" button.

Test Pipeline Run:

1. Go to Actions → "Run Data Pipeline"
2. Click "Run workflow"
3. Enter years: `2023`
4. Check "Dry run" for first test
5. Run and verify logs show correct processing without database writes

---

Part 5: The Health Check Script

File: `pipeline/monitoring/health_check.py`

```python
"""
BharatData Daily Health Check

Checks if NCRB data source URLs are still accessible.
Designed to run as a GitHub Action on a daily schedule.

Exit codes:
  0 = All sources healthy
  1 = One or more sources have issues
"""

import os
import sys
import json
import hashlib
```

```python
import requests
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()


# Data sources to check
SOURCES = [
    {
        "name": "NCRB Additional Tables 2023",
        "url": "https://www.ncrb.gov.in/crime-in-india-additional-table?year=2023&category=",
        "expected_status": 200,
    },
    {
        "name": "NCRB Additional Tables 2022",
        "url": "https://www.ncrb.gov.in/crime-in-india-additional-table?year=2022&category=",
        "expected_status": 200,
    },
    {
        "name": "NCRB Additional Tables 2021",
        "url": "https://www.ncrb.gov.in/crime-in-india-additional-table?year=2021&category=",
        "expected_status": 200,
    },
    {
        "name": "NCRB Main Website",
        "url": "https://ncrb.gov.in",
        "expected_status": 200,
    },
]


def check_source(source: dict) -> dict:
    """Check a single data source URL."""
    result = {
        "name": source["name"],
        "url": source["url"],
        "checked_at": datetime.utcnow().isoformat(),
        "healthy": False,
        "status_code": None,
        "response_time_ms": None,
        "error": None,
        "content_hash": None,
    }

    try:
        response = requests.get(
            source["url"],
            timeout=30,
```

```
            headers={
                "User-Agent": "BharatData Health Check (https://
        github.com/your-repo)"
            }
        )

        result["status_code"] = response.status_code
        result["response_time_ms"] =
        int(response.elapsed.total_seconds() * 1000)
        result["content_hash"] = hashlib.sha256(
            response.content
        ).hexdigest()[:16]

        if response.status_code == source["expected_status"]:
            result["healthy"] = True
        else:
            result["error"] = (
                f"Expected status {source['expected_status']}, "
                f"got {response.status_code}"
            )

    except requests.Timeout:
        result["error"] = "Request timed out after 30 seconds"
    except requests.ConnectionError as e:
        result["error"] = f"Connection failed: {str(e)[:200]}"
    except Exception as e:
        result["error"] = f"Unexpected error: {str(e)[:200]}"

    return result


def run_health_check() -> dict:
    """Run health checks on all registered sources."""
    print("BharatData Daily Health Check")
    print("=" * 50)

    results = []
    all_healthy = True

    for source in SOURCES:
        print(f"\nChecking: {source['name']}")
        print(f"  URL: {source['url']}")

        result = check_source(source)
        results.append(result)

        if result["healthy"]:
            print(f"  ✅ Healthy (status {result['status_code']},
        "
                  f"{result['response_time_ms']}ms)")
        else:
            print(f"  ❌ UNHEALTHY: {result['error']}")
```

```
            all_healthy = False

    report = {
        "timestamp": datetime.utcnow().isoformat(),
        "status": "HEALTHY" if all_healthy else "UNHEALTHY",
        "sources_checked": len(results),
        "sources_healthy": sum(1 for r in results if
        r["healthy"]),
        "sources_unhealthy": sum(1 for r in results if not
        r["healthy"]),
        "results": results,
    }

    print(f"\n{'=' * 50}")
    print(f"Overall: {report['status']}")
    print(f"Healthy: {report['sources_healthy']}/
        {report['sources_checked']}")

    return report


if __name__ == "__main__":
    report = run_health_check()

    # Write report to file if needed by GitHub Action
    report_path = os.getenv("HEALTH_REPORT_PATH",
        "health_report.json")
    with open(report_path, "w") as f:
        json.dump(report, f, indent=2)

    # Exit with appropriate code
    sys.exit(0 if report["status"] == "HEALTHY" else 1)
```

Part 6: The Quality Check Script

File: `pipeline/monitoring/quality_check.py`

```
"""
BharatData Weekly Quality Check

Runs data quality validation against the live database.
Designed for GitHub Actions weekly schedule or manual execution.

Usage:
    python -m pipeline.monitoring.quality_check --output
        report.json

Exit codes:
    0 = All checks passed
    1 = One or more checks failed
```

```python
"""

import os
import sys
import json
import argparse
from datetime import datetime
from dotenv import load_dotenv
import psycopg2

load_dotenv()


def check_required_fields(cursor) -> dict:
    """Verify no null values in required fields."""
    required = [
        "state", "state_code", "year", "category",
        "total_cases", "confidence", "source_url",
        "collection_date"
    ]
    issues = []

    for field in required:
        cursor.execute(
            f"SELECT COUNT(*) FROM crime_records_state WHERE
        {field} IS NULL"
        )
        null_count = cursor.fetchone()[0]
        if null_count > 0:
            issues.append({
                "field": field,
                "null_count": null_count,
                "severity": "critical"
            })

    return {
        "check": "required_fields",
        "passed": len(issues) == 0,
        "issues": issues
    }


def check_confidence_distribution(cursor) -> dict:
    """Check confidence score distribution."""
    cursor.execute("""
        SELECT
            AVG(confidence) as avg_conf,
            MIN(confidence) as min_conf,
            COUNT(*) FILTER (WHERE confidence < 0.5) as
        low_conf_count,
            COUNT(*) as total_count
        FROM crime_records_state
```

```
    """)
    row = cursor.fetchone()
    avg_conf, min_conf, low_conf_count, total_count = row

    issues = []
    if avg_conf and avg_conf < 0.7:
        issues.append({
            "message": f"Average confidence {avg_conf:.2f} below
        0.7 threshold",
            "severity": "warning"
        })

    if total_count and total_count > 0:
        low_pct = (low_conf_count / total_count) * 100
        if low_pct > 10:
            issues.append({
                "message": f"{low_pct:.1f}% records below 0.5
        confidence",
                "severity": "warning"
            })

    return {
        "check": "confidence_distribution",
        "passed": len(issues) == 0,
        "average_confidence": float(avg_conf) if avg_conf else
        None,
        "issues": issues
    }


def check_completeness(cursor, expected_states=36) -> dict:
    """Check if all expected states are present."""
    cursor.execute("""
        SELECT year, COUNT(DISTINCT state) as state_count
        FROM crime_records_state
        GROUP BY year
        ORDER BY year
    """)

    issues = []
    for row in cursor.fetchall():
        year, state_count = row
        if state_count < expected_states:
            issues.append({
                "message": f"Year {year}: only {state_count}/
        {expected_states} states",
                "severity": "warning",
                "year": year
            })

    return {
        "check": "completeness",
```

```
        "passed": len(issues) == 0,
        "issues": issues
    }


def check_data_types(cursor) -> dict:
    """Verify data type integrity."""
    issues = []

    # Confidence range
    cursor.execute("""
        SELECT COUNT(*) FROM crime_records_state
        WHERE confidence < 0 OR confidence > 1
    """)
    bad = cursor.fetchone()[0]
    if bad > 0:
        issues.append({
            "message": f"{bad} records with confidence outside
        0.0-1.0",
            "severity": "critical"
        })

    # Year range
    cursor.execute("""
        SELECT COUNT(*) FROM crime_records_state
        WHERE year < 2000 OR year > 2030
    """)
    bad = cursor.fetchone()[0]
    if bad > 0:
        issues.append({
            "message": f"{bad} records with implausible year",
            "severity": "critical"
        })

    # Negative cases
    cursor.execute("""
        SELECT COUNT(*) FROM crime_records_state
        WHERE total_cases < 0
    """)
    bad = cursor.fetchone()[0]
    if bad > 0:
        issues.append({
            "message": f"{bad} records with negative total_cases",
            "severity": "critical"
        })

    return {
        "check": "data_types",
        "passed": len(issues) == 0,
        "issues": issues
    }
```

```python
def check_record_counts(cursor) -> dict:
    """Get record counts per table for reporting."""
    cursor.execute("""
        SELECT 'state' as level, COUNT(*) FROM crime_records_state
        UNION ALL
        SELECT 'district', COUNT(*) FROM crime_records_district
        UNION ALL
        SELECT 'city', COUNT(*) FROM crime_records_city
    """)

    counts = {}
    for row in cursor.fetchall():
        counts[row[0]] = row[1]

    return {
        "check": "record_counts",
        "passed": True,
        "counts": counts,
        "issues": []
    }


def check_duplicates(cursor) -> dict:
    """Check for duplicate records."""
    cursor.execute("""
        SELECT state, year, category, source_file, COUNT(*) as cnt
        FROM crime_records_state
        GROUP BY state, year, category, source_file
        HAVING COUNT(*) > 1
        LIMIT 10
    """)

    duplicates = []
    for row in cursor.fetchall():
        duplicates.append({
            "state": row[0],
            "year": row[1],
            "category": row[2],
            "source_file": row[3],
            "count": row[4]
        })

    return {
        "check": "duplicates",
        "passed": len(duplicates) == 0,
        "duplicate_groups": len(duplicates),
        "issues": duplicates
    }


def run_all_checks() -> dict:
```

```python
    """Run all quality checks and return combined report."""
    db_url = os.getenv("DATABASE_URL")

    if not db_url:
        return {
            "timestamp": datetime.utcnow().isoformat(),
            "status": "FAILED",
            "error": "DATABASE_URL not set",
            "checks": []
        }

    try:
        conn = psycopg2.connect(db_url)
        cursor = conn.cursor()
    except Exception as e:
        return {
            "timestamp": datetime.utcnow().isoformat(),
            "status": "FAILED",
            "error": f"Database connection failed: {str(e)}",
            "checks": []
        }

    checks = [
        check_required_fields(cursor),
        check_confidence_distribution(cursor),
        check_completeness(cursor),
        check_data_types(cursor),
        check_record_counts(cursor),
        check_duplicates(cursor),
    ]

    cursor.close()
    conn.close()

    all_passed = all(c["passed"] for c in checks)

    return {
        "timestamp": datetime.utcnow().isoformat(),
        "status": "PASSED" if all_passed else "FAILED",
        "checks_run": len(checks),
        "checks_passed": sum(1 for c in checks if c["passed"]),
        "checks_failed": sum(1 for c in checks if not
        c["passed"]),
        "checks": checks
    }


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="BharatData
        Quality Check")
    parser.add_argument(
        "--output", default="quality_report.json",
```

```
        help="Output file path"
    )
    args = parser.parse_args()

    print("BharatData Weekly Quality Check")
    print("=" * 50)

    report = run_all_checks()

    # Print summary
    print(f"\nStatus: {report['status']}")
    print(f"Checks: {report.get('checks_passed', 0)}/
        {report.get('checks_run', 0)} passed")

    for check in report.get("checks", []):
        icon = "✅" if check["passed"] else "❌"
        print(f"  {icon} {check['check']}")
        for issue in check.get("issues", []):
            msg = issue.get("message", json.dumps(issue))
            print(f"      → {msg}")

    # Write report
    with open(args.output, "w") as f:
        json.dump(report, f, indent=2)

    print(f"\nReport saved to: {args.output}")

    sys.exit(0 if report["status"] == "PASSED" else 1)
```

Summary of All Changes

Problem You Had	Solution
Neon is slow (cold starts)	Switch to Supabase (always-on, connection pooling)
Records re-inserted on restart	3-level dedup: file hash tracking + UNIQUE constraints + ON CONFLICT DO NOTHING
Slow inserts	Batch inserts with `execute_values` (20-50x faster)
No monitoring	3 GitHub Actions workflows (daily health, weekly quality, on-demand pipeline)

Problem You Had	Solution
Need database guide	Complete Supabase setup with connection strings, schema, and verification

Migration Checklist (Neon → Supabase)

1. Create Supabase project
2. Update `.env` with new connection strings
3. Run `create_schema()` against Supabase
4. Add `DATABASE_URL` to GitHub repository secrets
5. Run pipeline with `--dry-run` to verify everything connects
6. Run full pipeline — observe the speed difference
7. Verify with quality check script
8. Push GitHub Action workflows and test with manual triggers
9. Delete Neon project after confirming everything works on Supabase

Database Selection, Incremental Ingestion & GitHub Workflows

Part 1: Database Selection — Supabase Wins

Why Neon Feels Slow

Neon uses a serverless architecture with cold starts. When your database hasn't been accessed for a few minutes, the compute instance suspends. The next query triggers a cold start that takes 1-3 seconds. For a pipeline inserting thousands of records, this causes noticeable lag, especially with frequent small transactions.

Comparing Your Three Options

Feature	Neon	Supabase	Convex
Database type	PostgreSQL (serverless)	PostgreSQL (always-on)	Custom reactive DB
Cold starts			No

Feature	Neon	Supabase	Convex
	Yes (1-3 sec)	No (always running on free tier)	
Free tier storage	500 MB	500 MB	1 GB
Free tier compute	Auto-suspend after 5 min inactivity	Always on (2 projects)	100K function calls/month
Connection pooling	Built-in but limited	PgBouncer built-in	Not applicable
SQL support	Full PostgreSQL	Full PostgreSQL	No SQL — custom query language
Dashboard	Basic	Excellent (table editor, SQL editor, logs)	Good
Your existing schema works?	Yes	Yes (zero changes)	No (complete rewrite needed)
Bulk insert speed	Slow with cold starts	Fast (persistent connection)	Different paradigm
REST API built-in	No	Yes (PostgREST auto-generated)	Yes
File storage	No	1 GB free (S3-compatible)	1 GB free
Auth built-in	No	Yes	Yes
Realtime subscriptions	No	Yes	Yes

Recommendation: Supabase

Convex is eliminated immediately. Convex is not PostgreSQL. It uses a custom reactive database with its own query language. Your entire database schema, all SQL queries, all SQLAlchemy models, and all your pipeline code would need to be completely rewritten. The pipeline spec, the loader, the quality checks — all assume PostgreSQL. Convex is a fundamentally different paradigm designed for real-time collaborative apps, not for a data pipeline serving structured government data.

Supabase beats Neon for your specific use case because:

1. **No cold starts.** Supabase keeps your database compute running on the free tier. Your pipeline inserts will not wait 1-3 seconds for the database to wake up.

2. **Built-in connection pooling via PgBouncer.** This is the single biggest speed improvement. PgBouncer maintains a pool of database connections. Instead of opening and closing a connection for every batch of records, your pipeline reuses connections from the pool. This eliminates connection overhead that slows down bulk inserts.

3. **Your existing schema works with zero changes.** Supabase is standard PostgreSQL. Every CREATE TABLE statement, every index, every CHECK constraint, every query you have designed works exactly as written.

4. **Built-in REST API (PostgREST).** Supabase automatically generates a REST API for every table. When you build the TypeScript and Python SDKs later, they can optionally use this REST API instead of requiring a custom API server. This could eliminate the need for a separate API deployment entirely.

5. **Dashboard with SQL editor.** You can visually inspect your data, run ad-hoc queries, and spot-check records directly in the browser. This accelerates debugging during pipeline development.

6. **1 GB file storage.** Supabase includes S3-compatible storage. You could store your raw NCRB files here instead of in the repository, keeping the repo size small.

Part 2: Supabase Setup Guide

Step 1: Create Supabase Project

1. Go to `https://supabase.com` and sign up with GitHub
2. Click “New Project”
3. Project name: `bharatdata`
4. Database password: generate a strong password and save it securely
5. Region: choose the closest to your users (Mumbai if available, or Singapore)
6. Click “Create new project”
7. Wait 1-2 minutes for provisioning

Step 2: Get Connection Details

After project creation, go to **Project Settings → Database**

You will see several connection strings. You need two:

Direct connection (for your pipeline — bulk inserts):

```
postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:5432/postgres
```

Connection pooler (for your API — many concurrent short queries):

```
postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:6543/postgres?pgbouncer=true
```

The difference: - Port `5432` = direct connection (use for pipeline bulk inserts) - Port `6543` = pooled connection via PgBouncer (use for API/SDK later)

Step 3: Update Your Environment File

Update your `.env`:

```
# Direct connection for pipeline (bulk operations)
DATABASE_URL=postgresql://postgres.[project-ref]:[password]@aws-0-
[region].pooler.supabase.com:5432/postgres

# Pooled connection for API (concurrent short queries) — used
later in Stage 4+
DATABASE_URL_POOLED=postgresql://postgres.[project-ref]:
[password]@aws-0-[region].pooler.supabase.com:6543/postgres?
pgbouncer=true

# Supabase specific
SUPABASE_URL=https://[project-ref].supabase.co
SUPABASE_ANON_KEY=eyJ...your-anon-key...

LOG_LEVEL=INFO
```

Update `.env.example`:

```
DATABASE_URL=postgresql://postgres.xxxxx:password@aws-0-
region.pooler.supabase.com:5432/postgres
DATABASE_URL_POOLED=postgresql://postgres.xxxxx:password@aws-0-
region.pooler.supabase.com:6543/postgres?pgbouncer=true
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_ANON_KEY=your-anon-key-here
LOG_LEVEL=INFO
```

Step 4: Verify Connection

```python
import os
from dotenv import load_dotenv
import psycopg2
import time

load_dotenv()
```

```python
# Test direct connection
start = time.time()
conn = psycopg2.connect(os.getenv("DATABASE_URL"))
cursor = conn.cursor()
cursor.execute("SELECT version();")
version = cursor.fetchone()[0]
elapsed = time.time() - start

print(f"Connected in {elapsed:.3f}s")
print(f"PostgreSQL: {version}")

cursor.close()
conn.close()
```

You should see connection time under 500ms (vs 1-3 seconds with Neon cold starts).

Step 5: Create Database Schema

You have two options for creating tables:

Option A: SQL Editor in Supabase Dashboard (recommended for first time)

1. Go to your Supabase project dashboard
2. Click “SQL Editor” in the left sidebar
3. Click “New Query”
4. Paste your complete schema SQL
5. Click “Run”
6. Go to “Table Editor” to verify all tables were created

Option B: Python script (for automated/repeatable setup)

```python
# pipeline/loader/schema.py

import os
from dotenv import load_dotenv
import psycopg2

load_dotenv()

SCHEMA_SQL = """
-- Enable UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Data sources registry
CREATE TABLE IF NOT EXISTS data_sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    source_url TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
```

```
    data_format TEXT NOT NULL,
    update_frequency TEXT NOT NULL,
    last_ingested_at TIMESTAMPTZ,
    status TEXT NOT NULL DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Ingestion run log
CREATE TABLE IF NOT EXISTS ingestion_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id UUID REFERENCES data_sources(id),
    started_at TIMESTAMPTZ NOT NULL,
    completed_at TIMESTAMPTZ,
    status TEXT NOT NULL DEFAULT 'running',
    files_processed INT DEFAULT 0,
    records_extracted INT DEFAULT 0,
    records_stored INT DEFAULT 0,
    records_skipped INT DEFAULT 0,
    parse_failures INT DEFAULT 0,
    error_message TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- State-level crime records
CREATE TABLE IF NOT EXISTS crime_records_state (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    state TEXT NOT NULL,
    state_code TEXT NOT NULL,
    year INT NOT NULL,
    category TEXT NOT NULL,
    category_label TEXT NOT NULL,
    total_cases INT,
    rate_per_lakh FLOAT,
    chargesheeted INT,
    convicted INT,
    acquitted INT,
    pending_investigation INT,
    pending_trial INT,
    confidence FLOAT NOT NULL,
    source_url TEXT NOT NULL,
    source_file TEXT NOT NULL,
    report_name TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    collection_date TIMESTAMPTZ NOT NULL,
    normalizer_version TEXT NOT NULL,
    boundary_note TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT state_confidence_range
        CHECK (confidence >= 0.0 AND confidence <= 1.0),
    CONSTRAINT state_year_range
```

```
        CHECK (year >= 2000 AND year <= 2030),
    CONSTRAINT state_unique_record
        UNIQUE (state, year, category, source_file)
);

-- District-level crime records
CREATE TABLE IF NOT EXISTS crime_records_district (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    state TEXT NOT NULL,
    state_code TEXT NOT NULL,
    district TEXT NOT NULL,
    year INT NOT NULL,
    category TEXT NOT NULL,
    category_label TEXT NOT NULL,
    total_cases INT,
    rate_per_lakh FLOAT,
    confidence FLOAT NOT NULL,
    source_url TEXT NOT NULL,
    source_file TEXT NOT NULL,
    report_name TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    collection_date TIMESTAMPTZ NOT NULL,
    normalizer_version TEXT NOT NULL,
    boundary_note TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT district_confidence_range
        CHECK (confidence >= 0.0 AND confidence <= 1.0),
    CONSTRAINT district_unique_record
        UNIQUE (state, district, year, category, source_file)
);

-- City-level crime records
CREATE TABLE IF NOT EXISTS crime_records_city (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    city TEXT NOT NULL,
    parent_state TEXT NOT NULL,
    year INT NOT NULL,
    category TEXT NOT NULL,
    category_label TEXT NOT NULL,
    total_cases INT,
    rate_per_lakh FLOAT,
    confidence FLOAT NOT NULL,
    source_url TEXT NOT NULL,
    source_file TEXT NOT NULL,
    report_name TEXT NOT NULL,
    publishing_body TEXT NOT NULL,
    collection_date TIMESTAMPTZ NOT NULL,
    normalizer_version TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
```

```
    CONSTRAINT city_confidence_range
        CHECK (confidence >= 0.0 AND confidence <= 1.0),
    CONSTRAINT city_unique_record
        UNIQUE (city, year, category, source_file)
);

-- Parse failure log
CREATE TABLE IF NOT EXISTS parse_failures (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ingestion_run_id UUID REFERENCES ingestion_runs(id),
    source_file TEXT NOT NULL,
    source_sheet TEXT,
    row_number INT,
    column_name TEXT,
    raw_value TEXT,
    expected_type TEXT,
    error_message TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Normalization decision log
CREATE TABLE IF NOT EXISTS normalization_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    record_id UUID,
    field TEXT NOT NULL,
    input_value TEXT NOT NULL,
    output_value TEXT NOT NULL,
    mapping_source TEXT NOT NULL,
    confidence_impact FLOAT DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- File tracking for incremental ingestion
CREATE TABLE IF NOT EXISTS ingested_files (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    file_path TEXT NOT NULL,
    file_hash_sha256 TEXT NOT NULL,
    file_size_bytes INT,
    year INT NOT NULL,
    table_type TEXT NOT NULL,
    geographic_level TEXT NOT NULL,
    records_extracted INT DEFAULT 0,
    ingested_at TIMESTAMPTZ DEFAULT NOW(),

    CONSTRAINT unique_file_hash
        UNIQUE (file_hash_sha256)
);

-- ===== INDEXES =====

-- State-level queries
```

```
CREATE INDEX IF NOT EXISTS idx_crime_state_year
    ON crime_records_state(state, year);
CREATE INDEX IF NOT EXISTS idx_crime_year
    ON crime_records_state(year);
CREATE INDEX IF NOT EXISTS idx_crime_category
    ON crime_records_state(category);
CREATE INDEX IF NOT EXISTS idx_crime_state_cat_year
    ON crime_records_state(state, category, year);
CREATE INDEX IF NOT EXISTS idx_state_confidence_filter
    ON crime_records_state(confidence)
    WHERE confidence >= 0.5;

-- District-level queries
CREATE INDEX IF NOT EXISTS idx_district_state
    ON crime_records_district(state);
CREATE INDEX IF NOT EXISTS idx_district_state_year
    ON crime_records_district(state, year);
CREATE INDEX IF NOT EXISTS idx_district_district
    ON crime_records_district(district);
CREATE INDEX IF NOT EXISTS idx_district_category
    ON crime_records_district(category);
CREATE INDEX IF NOT EXISTS idx_district_confidence_filter
    ON crime_records_district(confidence)
    WHERE confidence >= 0.5;

-- City-level queries
CREATE INDEX IF NOT EXISTS idx_city_name
    ON crime_records_city(city);
CREATE INDEX IF NOT EXISTS idx_city_year
    ON crime_records_city(city, year);

-- File tracking
CREATE INDEX IF NOT EXISTS idx_ingested_files_hash
    ON ingested_files(file_hash_sha256);
CREATE INDEX IF NOT EXISTS idx_ingested_files_path
    ON ingested_files(file_path);
"""


def create_schema():
    conn = psycopg2.connect(os.getenv("DATABASE_URL"))
    cursor = conn.cursor()

    cursor.execute(SCHEMA_SQL)
    conn.commit()

    # Verify tables
    cursor.execute("""
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'public'
        ORDER BY table_name;
```

```
    """)

    tables = [row[0] for row in cursor.fetchall()]
    print(f"Created {len(tables)} tables:")
    for t in tables:
        print(f"  ✓ {t}")

    cursor.close()
    conn.close()

    return tables


if __name__ == "__main__":
    create_schema()
```

Notice the critical additions compared to the original schema:

1. **`UNIQUE` constraints on data tables** — This is what prevents duplicate records. The combination `(state, year, category, source_file)` uniquely identifies a record. If you try to insert a duplicate, PostgreSQL rejects it.

2. **`ingested_files` table** — This tracks which files have already been processed, enabling incremental ingestion.

3. **`records_skipped` column in `ingestion_runs`** — Tracks how many records were skipped because they already existed.

Part 3: Incremental Ingestion — Never Re-Insert Existing Data

This solves your problem of records being re-inserted when you terminate and restart the pipeline.

The Strategy: Three Levels of Skip Detection

Level 1: File-Level Skip (Fastest)

Before parsing a file, check if it has already been fully ingested by looking up its SHA-256 hash in the `ingested_files` table.

```
# pipeline/ingester/hash_tracker.py

import hashlib
import os
import psycopg2
```

```python
class HashTracker:
    """Tracks which files have been ingested using SHA-256
        hashes."""

    def __init__(self, db_url: str):
        self.db_url = db_url

    def calculate_hash(self, file_path: str) -> str:
        """Calculate SHA-256 hash of a file."""
        sha256 = hashlib.sha256()
        with open(file_path, "rb") as f:
            for chunk in iter(lambda: f.read(8192), b""):
                sha256.update(chunk)
        return sha256.hexdigest()

    def is_file_ingested(self, file_path: str) -> bool:
        """Check if a file has already been fully ingested."""
        file_hash = self.calculate_hash(file_path)

        conn = psycopg2.connect(self.db_url)
        cursor = conn.cursor()
        cursor.execute(
            "SELECT id FROM ingested_files WHERE file_hash_sha256
        = %s",
            (file_hash,)
        )
        result = cursor.fetchone()
        cursor.close()
        conn.close()

        return result is not None

    def mark_file_ingested(
        self,
        file_path: str,
        year: int,
        table_type: str,
        geographic_level: str,
        records_extracted: int
    ):
        """Mark a file as fully ingested."""
        file_hash = self.calculate_hash(file_path)
        file_size = os.path.getsize(file_path)

        conn = psycopg2.connect(self.db_url)
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO ingested_files
                (file_path, file_hash_sha256, file_size_bytes,
                 year, table_type, geographic_level,
        records_extracted)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
```

```
            ON CONFLICT (file_hash_sha256) DO NOTHING
        """, (
            file_path, file_hash, file_size,
            year, table_type, geographic_level, records_extracted
        ))
        conn.commit()
        cursor.close()
        conn.close()

    def get_ingestion_status(self) -> dict:
        """Get summary of ingested files."""
        conn = psycopg2.connect(self.db_url)
        cursor = conn.cursor()
        cursor.execute("""
            SELECT year, geographic_level, COUNT(*),
        SUM(records_extracted)
            FROM ingested_files
            GROUP BY year, geographic_level
            ORDER BY year, geographic_level
        """)

        status = {}
        for row in cursor.fetchall():
            year, level, file_count, record_count = row
            status[f"{year}_{level}"] = {
                "files": file_count,
                "records": record_count
            }

        cursor.close()
        conn.close()
        return status
```

Level 2: Record-Level Skip (Using UNIQUE Constraints)

The `UNIQUE` constraint on each data table prevents duplicate records at the database level. When inserting, use `ON CONFLICT DO NOTHING`:

```
# In pipeline/loader/loader.py

def insert_state_record(self, record: dict, cursor) -> str:
    """Insert a state-level record. Returns 'inserted' or
        'skipped'."""
    try:
        cursor.execute("""
            INSERT INTO crime_records_state (
                ingestion_run_id, state, state_code, year,
                category, category_label, total_cases,
        rate_per_lakh,
                chargesheeted, convicted, acquitted,
                pending_investigation, pending_trial,
```

```
                confidence, source_url, source_file,
                report_name, publishing_body, collection_date,
                normalizer_version, boundary_note
            ) VALUES (
                %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
        %s,
                %s, %s, %s, %s, %s, %s, %s, %s
            )
            ON CONFLICT (state, year, category, source_file)
            DO NOTHING
            RETURNING id
        """, (
            record["ingestion_run_id"],
            record["state"],
            record["state_code"],
            record["year"],
            record["category"],
            record["category_label"],
            record.get("total_cases"),
            record.get("rate_per_lakh"),
            record.get("chargesheeted"),
            record.get("convicted"),
            record.get("acquitted"),
            record.get("pending_investigation"),
            record.get("pending_trial"),
            record["confidence"],
            record["source_url"],
            record["source_file"],
            record["report_name"],
            record["publishing_body"],
            record["collection_date"],
            record["normalizer_version"],
            record.get("boundary_note"),
        ))

        result = cursor.fetchone()
        return "inserted" if result else "skipped"

    except Exception as e:
        return f"error: {str(e)}"
```

Level 3: Batch Tracking (Resume After Termination)

Track which files within an ingestion run have been completed. If the pipeline is terminated mid-run, the next run picks up where it left off.

```
# In pipeline/run_pipeline.py — orchestrator logic

def run_pipeline(years: list, steps: list, dry_run: bool = False):
    """Main pipeline orchestrator with incremental processing."""
```

```
db_url = os.getenv("DATABASE_URL")
hash_tracker = HashTracker(db_url)

# Show what's already done
status = hash_tracker.get_ingestion_status()
if status:
    print("Previously ingested data:")
    for key, info in status.items():
        print(f"  {key}: {info['files']} files,
    {info['records']} records")
    print()

total_skipped_files = 0
total_processed_files = 0
total_inserted = 0
total_skipped_records = 0

for year in years:
    raw_dir = f"data/raw/ncrb-crime-in-india/{year}"

    # Get all files for this year
    all_files = get_files_for_year(raw_dir)

    for file_path in all_files:
        # LEVEL 1: Skip entire file if already ingested
        if hash_tracker.is_file_ingested(file_path):
            print(f"  SKIP (already ingested):
    {os.path.basename(file_path)}")
            total_skipped_files += 1
            continue

        print(f"  PROCESSING: {os.path.basename(file_path)}")
        total_processed_files += 1

        # Parse
        records = parse_file(file_path)

        # Normalize
        normalized = normalize_records(records)

        # Load (with record-level dedup via ON CONFLICT)
        if not dry_run:
            inserted, skipped = load_records(normalized,
    db_url)
            total_inserted += inserted
            total_skipped_records += skipped

            # Mark file as fully ingested
            hash_tracker.mark_file_ingested(
                file_path=file_path,
                year=year,
                table_type=detect_table_type(file_path),
```

```
                    geographic_level=detect_geo_level(file_path),
                    records_extracted=len(normalized)
                )

    # Print summary
    print(f"\n{'='*50}")
    print(f"Pipeline Summary")
    print(f"{'='*50}")
    print(f"Files skipped (already ingested):
        {total_skipped_files}")
    print(f"Files processed (new):
        {total_processed_files}")
    print(f"Records inserted:                 {total_inserted}")
    print(f"Records skipped (duplicates):
        {total_skipped_records}")
    print(f"{'='*50}")
```

Speed Optimization: Batch Inserts

Instead of inserting records one at a time, use batch inserts with `executemany` or `execute_values`:

```
# pipeline/loader/loader.py — batch insert method

from psycopg2.extras import execute_values


def bulk_insert_state_records(self, records: list[dict], cursor)
        -> tuple[int, int]:
    """
    Bulk insert state-level records using execute_values.
    Returns (inserted_count, skipped_count).
    """
    if not records:
        return 0, 0

    values = []
    for r in records:
        values.append((
            r["ingestion_run_id"],
            r["state"], r["state_code"], r["year"],
            r["category"], r["category_label"],
            r.get("total_cases"), r.get("rate_per_lakh"),
            r.get("chargesheeted"), r.get("convicted"),
            r.get("acquitted"), r.get("pending_investigation"),
            r.get("pending_trial"), r["confidence"],
            r["source_url"], r["source_file"],
            r["report_name"], r["publishing_body"],
            r["collection_date"], r["normalizer_version"],
            r.get("boundary_note"),
        ))
```

```
    # Count before insert
    cursor.execute("SELECT COUNT(*) FROM crime_records_state")
    count_before = cursor.fetchone()[0]

    # Bulk insert with conflict handling
    execute_values(
        cursor,
        """
        INSERT INTO crime_records_state (
            ingestion_run_id, state, state_code, year,
            category, category_label, total_cases, rate_per_lakh,
            chargesheeted, convicted, acquitted,
            pending_investigation, pending_trial,
            confidence, source_url, source_file,
            report_name, publishing_body, collection_date,
            normalizer_version, boundary_note
        ) VALUES %s
        ON CONFLICT (state, year, category, source_file) DO
        NOTHING
        """,
        values,
        page_size=500  # Insert 500 records per batch
    )

    # Count after insert
    cursor.execute("SELECT COUNT(*) FROM crime_records_state")
    count_after = cursor.fetchone()[0]

    inserted = count_after - count_before
    skipped = len(records) - inserted

    return inserted, skipped
```

Speed comparison: - One-at-a-time inserts: ~50-100 records/second - Batch inserts with `execute_values`: ~2,000-5,000 records/second - That's a 20-50x speedup

Connection Management for Speed

```
# pipeline/utils/db_connection.py

import os
from contextlib import contextmanager
import psycopg2
from psycopg2 import pool
from dotenv import load_dotenv

load_dotenv()

# Create a connection pool (reuse connections instead of opening
        new ones)
_pool = None
```

```python
def get_pool():
    global _pool
    if _pool is None:
        _pool = pool.SimpleConnectionPool(
            minconn=1,
            maxconn=5,
            dsn=os.getenv("DATABASE_URL")
        )
    return _pool


@contextmanager
def get_connection():
    """Get a database connection from the pool."""
    p = get_pool()
    conn = p.getconn()
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        p.putconn(conn)


@contextmanager
def get_cursor():
    """Get a database cursor with automatic connection
        management."""
    with get_connection() as conn:
        cursor = conn.cursor()
        try:
            yield cursor
        finally:
            cursor.close()
```

Part 4: GitHub Workflows — Complete Guide

Understanding GitHub Actions

GitHub Actions are automated workflows that run on GitHub's servers when triggered by events (schedule, push, manual trigger). Each workflow is a `.yml` file in `.github/workflows/`.

Workflow 1: Daily Health Check

File: `.github/workflows/daily-health-check.yml`

Purpose: Runs every day at 6:00 AM UTC (11:30 AM IST) to check if NCRB data source URLs are still accessible.

```yaml
name: Daily Data Source Health Check

# When this workflow runs
on:
  schedule:
    # Runs at 06:00 UTC every day (11:30 AM IST)
    # Cron format: minute hour day-of-month month day-of-week
    - cron: '0 6 * * *'

  # Allow manual trigger from GitHub Actions tab
  workflow_dispatch:

jobs:
  health-check:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Check out your repository code
      - name: Checkout repository
        uses: actions/checkout@v4

      # Step 2: Set up Python
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      # Step 3: Install dependencies
      - name: Install dependencies
        run: |
          pip install requests psycopg2-binary python-dotenv
        loguru

      # Step 4: Run the health check script
      - name: Run health check
        id: health_check
        run: |
          python -m pipeline.monitoring.health_check
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
        continue-on-error: true

      # Step 5: Create GitHub issue if health check failed
      - name: Create issue on failure
        if: steps.health_check.outcome == 'failure'
```

```
        uses: actions/github-script@v7
        with:
          github-token: ${{ secrets.GITHUB_TOKEN }}
          script: |
            const date = new Date().toISOString().split('T')[0];
            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: `🚨 Data Source Health Check Failed — ${date}
`,
              body: `## Health Check Failure\n\nThe daily health
check detected issues with one or more data sources.
\n\n**Date:** ${date}\n**Workflow Run:** $
{context.serverUrl}/${context.repo.owner}/$
{context.repo.repo}/actions/runs/${context.runId}
\n\nPlease check the workflow logs for details and
investigate the source availability.`,
              labels: ['data-source-alert', 'automated']
            });
```

Workflow 2: Weekly Quality Check

File: `.github/workflows/weekly-quality-check.yml`

Purpose: Runs every Monday at 8:00 AM UTC to validate data quality in the database.

```yaml
name: Weekly Data Quality Check

on:
  schedule:
    # Runs at 08:00 UTC every Monday
    - cron: '0 8 * * 1'

  workflow_dispatch:

jobs:
  quality-check:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install psycopg2-binary python-dotenv loguru
```

```yaml
- name: Run quality checks
  id: quality_check
  run: |
    python -m pipeline.monitoring.quality_check \
      --output quality_report.json
  env:
    DATABASE_URL: ${{ secrets.DATABASE_URL }}

- name: Read report
  id: read_report
  run: |
    echo "report<<EOF" >> $GITHUB_OUTPUT
    cat quality_report.json >> $GITHUB_OUTPUT
    echo "EOF" >> $GITHUB_OUTPUT

- name: Create quality report issue
  uses: actions/github-script@v7
  with:
    github-token: ${{ secrets.GITHUB_TOKEN }}
    script: |
      const fs = require('fs');
      const report =
  JSON.parse(fs.readFileSync('quality_report.json',
  'utf8'));
      const date = new Date().toISOString().split('T')[0];
      const status = report.status === 'PASSED' ? '✅' :
  '🚨';

      let body = `## Weekly Quality Report — ${date}\n\n`;
      body += `**Status:** ${status} ${report.status}\n`;
      body += `**Checks Run:** ${report.checks_run}\n`;
      body += `**Checks Passed:** ${report.checks_passed}
  \n`;
      body += `**Checks Failed:** ${report.checks_failed}
  \n\n`;

      body += `### Details\n\n`;
      for (const check of report.checks) {
        const icon = check.passed ? '✅' : '❌';
        body += `${icon} **${check.check}**\n`;
        if (check.issues && check.issues.length > 0) {
          for (const issue of check.issues) {
            body += `  - ${issue.message ||
  JSON.stringify(issue)}\n`;
          }
        }
        body += `\n`;
      }

      const labels = ['quality-report', 'automated'];
      if (report.status !== 'PASSED') {
        labels.push('needs-attention');
```

```
            }

            await github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: `${status} Weekly Quality Report — ${date}`,
              body: body,
              labels: labels
            });
```

Workflow 3: Pipeline Run on Demand

File: `.github/workflows/run-pipeline.yml`

Purpose: Allows you to trigger a full pipeline run from the GitHub Actions tab, useful when NCRB publishes new data.

```yaml
name: Run Data Pipeline

on:
  # Only manual trigger — never automatic for the full pipeline
  workflow_dispatch:
    inputs:
      years:
        description: 'Years to process (comma-separated)'
        required: true
        default: '2023'
        type: string
      dry_run:
        description: 'Dry run (no database writes)'
        required: false
        default: false
        type: boolean

jobs:
  run-pipeline:
    runs-on: ubuntu-latest
    timeout-minutes: 30  # Kill if it takes too long

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
```

```yaml
      - name: Run pipeline
        run: |
          YEARS="${{ github.event.inputs.years }}"
          DRY_RUN_FLAG=""
          if [ "${{ github.event.inputs.dry_run }}" = "true" ];
        then
            DRY_RUN_FLAG="--dry-run"
          fi

          # Convert comma-separated years to space-separated
          YEARS_ARGS=$(echo $YEARS | tr ',' ' ')

          python -m pipeline.run_pipeline \
            --years $YEARS_ARGS \
            --steps all \
            $DRY_RUN_FLAG
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}

      - name: Run validation after pipeline
        if: github.event.inputs.dry_run == 'false'
        run: |
          python -m pipeline.monitoring.quality_check \
            --output pipeline_validation.json
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}

      - name: Upload validation report
        if: github.event.inputs.dry_run == 'false'
        uses: actions/upload-artifact@v4
        with:
          name: pipeline-validation-report
          path: pipeline_validation.json
```

Setting Up GitHub Secrets

Before the workflows work, you must add your database credentials to GitHub:

1. Go to your GitHub repository
2. Click **Settings** → **Secrets and variables** → **Actions**
3. Click **New repository secret**
4. Add these secrets:

Secret Name	Value
`DATABASE_URL`	Your Supabase direct connection string (port 5432)

Never put database credentials in your workflow files.
Always use `${{ secrets.SECRET_NAME }}`.

Testing Your Workflows

Test Daily Health Check:

1. Push the `.yml` file to your `main` branch
2. Go to **Actions** tab in your repository
3. Click "Daily Data Source Health Check" in the left sidebar
4. Click "Run workflow" button (top right)
5. Click the green "Run workflow" button
6. Watch it execute — should take ~30 seconds
7. Check if an issue was created (only if health check detected a problem)

Test Weekly Quality Check:

Same process — trigger manually with "Run workflow" button.

Test Pipeline Run:

1. Go to Actions → "Run Data Pipeline"
2. Click "Run workflow"
3. Enter years: `2023`
4. Check "Dry run" for first test
5. Run and verify logs show correct processing without database writes

---

Part 5: The Health Check Script

File: `pipeline/monitoring/health_check.py`

```python
"""
BharatData Daily Health Check

Checks if NCRB data source URLs are still accessible.
Designed to run as a GitHub Action on a daily schedule.

Exit codes:
  0 = All sources healthy
  1 = One or more sources have issues
"""

import os
import sys
import json
import hashlib
```

```python
import requests
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()


# Data sources to check
SOURCES = [
    {
        "name": "NCRB Additional Tables 2023",
        "url": "https://www.ncrb.gov.in/crime-in-india-additional-table?year=2023&category=",
        "expected_status": 200,
    },
    {
        "name": "NCRB Additional Tables 2022",
        "url": "https://www.ncrb.gov.in/crime-in-india-additional-table?year=2022&category=",
        "expected_status": 200,
    },
    {
        "name": "NCRB Additional Tables 2021",
        "url": "https://www.ncrb.gov.in/crime-in-india-additional-table?year=2021&category=",
        "expected_status": 200,
    },
    {
        "name": "NCRB Main Website",
        "url": "https://ncrb.gov.in",
        "expected_status": 200,
    },
]


def check_source(source: dict) -> dict:
    """Check a single data source URL."""
    result = {
        "name": source["name"],
        "url": source["url"],
        "checked_at": datetime.utcnow().isoformat(),
        "healthy": False,
        "status_code": None,
        "response_time_ms": None,
        "error": None,
        "content_hash": None,
    }

    try:
        response = requests.get(
            source["url"],
            timeout=30,
```

```
            headers={
                "User-Agent": "BharatData Health Check (https://
        github.com/your-repo)"
            }
        )

        result["status_code"] = response.status_code
        result["response_time_ms"] =
        int(response.elapsed.total_seconds() * 1000)
        result["content_hash"] = hashlib.sha256(
            response.content
        ).hexdigest()[:16]

        if response.status_code == source["expected_status"]:
            result["healthy"] = True
        else:
            result["error"] = (
                f"Expected status {source['expected_status']}, "
                f"got {response.status_code}"
            )

    except requests.Timeout:
        result["error"] = "Request timed out after 30 seconds"
    except requests.ConnectionError as e:
        result["error"] = f"Connection failed: {str(e)[:200]}"
    except Exception as e:
        result["error"] = f"Unexpected error: {str(e)[:200]}"

    return result


def run_health_check() -> dict:
    """Run health checks on all registered sources."""
    print("BharatData Daily Health Check")
    print("=" * 50)

    results = []
    all_healthy = True

    for source in SOURCES:
        print(f"\nChecking: {source['name']}")
        print(f"  URL: {source['url']}")

        result = check_source(source)
        results.append(result)

        if result["healthy"]:
            print(f"  ✅ Healthy (status {result['status_code']},
        "
                  f"{result['response_time_ms']}ms)")
        else:
            print(f"  ❌ UNHEALTHY: {result['error']}")
```

```
            all_healthy = False

    report = {
        "timestamp": datetime.utcnow().isoformat(),
        "status": "HEALTHY" if all_healthy else "UNHEALTHY",
        "sources_checked": len(results),
        "sources_healthy": sum(1 for r in results if
        r["healthy"]),
        "sources_unhealthy": sum(1 for r in results if not
        r["healthy"]),
        "results": results,
    }

    print(f"\n{'=' * 50}")
    print(f"Overall: {report['status']}")
    print(f"Healthy: {report['sources_healthy']}/
        {report['sources_checked']}")

    return report


if __name__ == "__main__":
    report = run_health_check()

    # Write report to file if needed by GitHub Action
    report_path = os.getenv("HEALTH_REPORT_PATH",
        "health_report.json")
    with open(report_path, "w") as f:
        json.dump(report, f, indent=2)

    # Exit with appropriate code
    sys.exit(0 if report["status"] == "HEALTHY" else 1)
```

Part 6: The Quality Check Script

File: `pipeline/monitoring/quality_check.py`

```
"""
BharatData Weekly Quality Check

Runs data quality validation against the live database.
Designed for GitHub Actions weekly schedule or manual execution.

Usage:
    python -m pipeline.monitoring.quality_check --output
        report.json

Exit codes:
    0 = All checks passed
    1 = One or more checks failed
```

```python
"""

import os
import sys
import json
import argparse
from datetime import datetime
from dotenv import load_dotenv
import psycopg2

load_dotenv()


def check_required_fields(cursor) -> dict:
    """Verify no null values in required fields."""
    required = [
        "state", "state_code", "year", "category",
        "total_cases", "confidence", "source_url",
        "collection_date"
    ]
    issues = []

    for field in required:
        cursor.execute(
            f"SELECT COUNT(*) FROM crime_records_state WHERE
        {field} IS NULL"
        )
        null_count = cursor.fetchone()[0]
        if null_count > 0:
            issues.append({
                "field": field,
                "null_count": null_count,
                "severity": "critical"
            })

    return {
        "check": "required_fields",
        "passed": len(issues) == 0,
        "issues": issues
    }


def check_confidence_distribution(cursor) -> dict:
    """Check confidence score distribution."""
    cursor.execute("""
        SELECT
            AVG(confidence) as avg_conf,
            MIN(confidence) as min_conf,
            COUNT(*) FILTER (WHERE confidence < 0.5) as
        low_conf_count,
            COUNT(*) as total_count
        FROM crime_records_state
```

```
    """)
    row = cursor.fetchone()
    avg_conf, min_conf, low_conf_count, total_count = row

    issues = []
    if avg_conf and avg_conf < 0.7:
        issues.append({
            "message": f"Average confidence {avg_conf:.2f} below
        0.7 threshold",
            "severity": "warning"
        })

    if total_count and total_count > 0:
        low_pct = (low_conf_count / total_count) * 100
        if low_pct > 10:
            issues.append({
                "message": f"{low_pct:.1f}% records below 0.5
        confidence",
                "severity": "warning"
            })

    return {
        "check": "confidence_distribution",
        "passed": len(issues) == 0,
        "average_confidence": float(avg_conf) if avg_conf else
        None,
        "issues": issues
    }


def check_completeness(cursor, expected_states=36) -> dict:
    """Check if all expected states are present."""
    cursor.execute("""
        SELECT year, COUNT(DISTINCT state) as state_count
        FROM crime_records_state
        GROUP BY year
        ORDER BY year
    """)

    issues = []
    for row in cursor.fetchall():
        year, state_count = row
        if state_count < expected_states:
            issues.append({
                "message": f"Year {year}: only {state_count}/
        {expected_states} states",
                "severity": "warning",
                "year": year
            })

    return {
        "check": "completeness",
```

```
        "passed": len(issues) == 0,
        "issues": issues
    }


def check_data_types(cursor) -> dict:
    """Verify data type integrity."""
    issues = []

    # Confidence range
    cursor.execute("""
        SELECT COUNT(*) FROM crime_records_state
        WHERE confidence < 0 OR confidence > 1
    """)
    bad = cursor.fetchone()[0]
    if bad > 0:
        issues.append({
            "message": f"{bad} records with confidence outside
        0.0-1.0",
            "severity": "critical"
        })

    # Year range
    cursor.execute("""
        SELECT COUNT(*) FROM crime_records_state
        WHERE year < 2000 OR year > 2030
    """)
    bad = cursor.fetchone()[0]
    if bad > 0:
        issues.append({
            "message": f"{bad} records with implausible year",
            "severity": "critical"
        })

    # Negative cases
    cursor.execute("""
        SELECT COUNT(*) FROM crime_records_state
        WHERE total_cases < 0
    """)
    bad = cursor.fetchone()[0]
    if bad > 0:
        issues.append({
            "message": f"{bad} records with negative total_cases",
            "severity": "critical"
        })

    return {
        "check": "data_types",
        "passed": len(issues) == 0,
        "issues": issues
    }
```

```python
def check_record_counts(cursor) -> dict:
    """Get record counts per table for reporting."""
    cursor.execute("""
        SELECT 'state' as level, COUNT(*) FROM crime_records_state
        UNION ALL
        SELECT 'district', COUNT(*) FROM crime_records_district
        UNION ALL
        SELECT 'city', COUNT(*) FROM crime_records_city
    """)

    counts = {}
    for row in cursor.fetchall():
        counts[row[0]] = row[1]

    return {
        "check": "record_counts",
        "passed": True,
        "counts": counts,
        "issues": []
    }


def check_duplicates(cursor) -> dict:
    """Check for duplicate records."""
    cursor.execute("""
        SELECT state, year, category, source_file, COUNT(*) as cnt
        FROM crime_records_state
        GROUP BY state, year, category, source_file
        HAVING COUNT(*) > 1
        LIMIT 10
    """)

    duplicates = []
    for row in cursor.fetchall():
        duplicates.append({
            "state": row[0],
            "year": row[1],
            "category": row[2],
            "source_file": row[3],
            "count": row[4]
        })

    return {
        "check": "duplicates",
        "passed": len(duplicates) == 0,
        "duplicate_groups": len(duplicates),
        "issues": duplicates
    }


def run_all_checks() -> dict:
```

```python
    """Run all quality checks and return combined report."""
    db_url = os.getenv("DATABASE_URL")

    if not db_url:
        return {
            "timestamp": datetime.utcnow().isoformat(),
            "status": "FAILED",
            "error": "DATABASE_URL not set",
            "checks": []
        }

    try:
        conn = psycopg2.connect(db_url)
        cursor = conn.cursor()
    except Exception as e:
        return {
            "timestamp": datetime.utcnow().isoformat(),
            "status": "FAILED",
            "error": f"Database connection failed: {str(e)}",
            "checks": []
        }

    checks = [
        check_required_fields(cursor),
        check_confidence_distribution(cursor),
        check_completeness(cursor),
        check_data_types(cursor),
        check_record_counts(cursor),
        check_duplicates(cursor),
    ]

    cursor.close()
    conn.close()

    all_passed = all(c["passed"] for c in checks)

    return {
        "timestamp": datetime.utcnow().isoformat(),
        "status": "PASSED" if all_passed else "FAILED",
        "checks_run": len(checks),
        "checks_passed": sum(1 for c in checks if c["passed"]),
        "checks_failed": sum(1 for c in checks if not
        c["passed"]),
        "checks": checks
    }


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="BharatData
        Quality Check")
    parser.add_argument(
        "--output", default="quality_report.json",
```

```
        help="Output file path"
    )
    args = parser.parse_args()

    print("BharatData Weekly Quality Check")
    print("=" * 50)

    report = run_all_checks()

    # Print summary
    print(f"\nStatus: {report['status']}")
    print(f"Checks: {report.get('checks_passed', 0)}/
        {report.get('checks_run', 0)} passed")

    for check in report.get("checks", []):
        icon = "✅" if check["passed"] else "❌"
        print(f"  {icon} {check['check']}")
        for issue in check.get("issues", []):
            msg = issue.get("message", json.dumps(issue))
            print(f"      → {msg}")

    # Write report
    with open(args.output, "w") as f:
        json.dump(report, f, indent=2)

    print(f"\nReport saved to: {args.output}")

    sys.exit(0 if report["status"] == "PASSED" else 1)
```

Summary of All Changes

Problem You Had	Solution
Neon is slow (cold starts)	Switch to Supabase (always-on, connection pooling)
Records re-inserted on restart	3-level dedup: file hash tracking + UNIQUE constraints + ON CONFLICT DO NOTHING
Slow inserts	Batch inserts with `execute_values` (20-50x faster)
No monitoring	3 GitHub Actions workflows (daily health, weekly quality, on-demand pipeline)

Problem You Had	Solution
Need database guide	Complete Supabase setup with connection strings, schema, and verification

Migration Checklist (Neon → Supabase)

1. Create Supabase project
2. Update `.env` with new connection strings
3. Run `create_schema()` against Supabase
4. Add `DATABASE_URL` to GitHub repository secrets
5. Run pipeline with `--dry-run` to verify everything connects
6. Run full pipeline — observe the speed difference
7. Verify with quality check script
8. Push GitHub Action workflows and test with manual triggers
9. Delete Neon project after confirming everything works on Supabase

Stage 3 Completion Report — Verification Analysis

Overall Assessment: 3 of 5 Issues Fixed. 2 Remaining Problems.

The report shows genuine engineering progress on three of the five issues I flagged. However, two problems remain — one is a regression that is worse than before, and one critical verification step is still missing.

Issue-by-Issue Verification

Issue 1: Summary Row Contamination — PARTIALLY FIXED

What I required: Exactly 36 distinct states per year in the data tables. Zero rows matching “Total”, “Grand Total”, “All India”.

What the report shows: The fix was implemented via `is_summary_row` regex in the parser. However, the report itself admits: “99.9% for Districts (3 summary orphans remained).”

Three summary rows still exist in the district table. That means the filter is working for state-level data but has edge cases in district data where summary rows have slightly different formatting that your regex does not catch.

What to do: Run this query and fix whatever comes back:

```
SELECT DISTINCT state, district
FROM crime_records_district
WHERE LOWER(state) LIKE '%total%'
   OR LOWER(district) LIKE '%total%'
   OR LOWER(state) LIKE '%all india%'
   OR LOWER(district) LIKE '%all india%'
   OR LOWER(district) LIKE '%grand%';
```

Delete those 3 rows and update the regex to catch their specific pattern. This is 30 minutes of work.

Also, confirm the clean state count explicitly:

```
SELECT year, COUNT(DISTINCT state)
FROM crime_records_state
GROUP BY year ORDER BY year;
```

Each year must return exactly 36. If any year returns more, summary rows are still present in the state table.

Verdict: 90% fixed. Needs the final 3 orphan rows cleaned.

Issue 2: Null/Dash Handling — CORRECTLY FIXED

What I required: Context-aware conversion. Dashes in `total_cases` become 0. Dashes in disposal metrics (chargesheeted, convicted, acquitted) stay null.

What the report shows: "Imputes zeros for missing totals and converts dashes to nulls for disposition metrics."

This is exactly the nuanced approach I recommended. The language specifically distinguishes between totals (→ zero) and disposition (→ null), which means the implementation follows the correct logic.

The data health metric of "100.0% for States" on total_cases confirms the fix is working at the state level.

Verdict: Correctly implemented.

Issue 3: Confidence Scoring — CORRECTLY FIXED

What I required: Average confidence between 0.85 and 0.98. Distribution should NOT cluster at 1.0.

What the report shows: Average confidence 0.94 with "weighted graduated scale."

0.94 is within the range I specified. The shift from 99.6% average to 94% average means real deductions are being applied.

One missing detail: The report does not show the confidence distribution. I would want to see what percentage of records are at each level. Run this to verify the distribution is genuinely graduated:

```sql
SELECT
    CASE
        WHEN confidence = 1.0 THEN '1.00 (perfect)'
        WHEN confidence >= 0.95 THEN '0.95-0.99'
        WHEN confidence >= 0.90 THEN '0.90-0.94'
        WHEN confidence >= 0.80 THEN '0.80-0.89'
        WHEN confidence >= 0.50 THEN '0.50-0.79'
        ELSE 'below 0.50'
    END as confidence_band,
    COUNT(*) as record_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 1) as
        percentage
FROM crime_records_state
GROUP BY 1
ORDER BY 1 DESC;
```

If you see records spread across at least 3 different bands, the scoring is genuinely granular. If 90%+ is still in one band, the deductions are too small.

Verdict: Correctly fixed based on reported average. Confirm with distribution query.

Issue 4: City Record Count — REGRESSION. WORSE THAN BEFORE.

What I required: City records should be greater than 800. The calculation was 34 cities × 9+ category types × 3 years = ~918 minimum.

Previous count: 462 records (154 per year, roughly 4.5 categories per city)

Current count: 102 records (34 per year, exactly 1 category per city per year)

This is a significant regression. You went from 462 city records to 102. Instead of fixing the missing categories, the pipeline is now loading even fewer. The report says "Enhanced FIELD_MAPPINGS for Cognizable Crimes and regex file prefixes" and "Full City Recovery," but the numbers tell the opposite story.

34 records per year means exactly 1 record per city per year. Your Stage 1 work produced 9 separate PDF-parsed JSON files covering different crime categories for cities. Only one of those nine is making it into the database.

What happened likely: The fixes to the parser or field mapper may have inadvertently narrowed the city data intake. Or the "Enhanced FIELD_MAPPINGS" created stricter matching that rejected records that previously passed.

What to check immediately:

```sql
-- What categories exist for cities?
SELECT DISTINCT category, COUNT(*)
FROM crime_records_city
GROUP BY category
ORDER BY COUNT(*) DESC;
```

If this returns only 1 category, your city pipeline is loading only one table type.

Then check your 9 Stage 1 JSON files:

```bash
ls -la data/structured/metro-cities/
```

Count the files. Each file should correspond to a category. The pipeline should load all of them, producing 34 × number_of_files × 3 years records.

Root cause investigation: Trace the city data path through the pipeline: 1. Are all 9 JSON files being discovered by the ingester/parser? 2. Is the field mapper successfully mapping columns from all 9 file types? 3. Is the normalizer passing all city records through? 4. Is the loader inserting into `crime_records_city` for all categories?

Add logging at each stage to count records entering and leaving:

```python
# At the end of each pipeline stage for city data
print(f"Parser output: {len(city_records)} city records")
print(f"Normalizer output: {len(normalized_city)} city records")
print(f"Loader input: {len(to_load_city)} city records")
```

Verdict: Not fixed. This is worse than before and needs immediate attention. The expected count is ~918, the previous count was 462, and the current count is 102.

Estimated time to investigate and fix: 2-3 hours

Issue 5: 20-Record Manual Spot Check — NOT COMPLETED

What I required: Select 20 random records, open original source files, verify values match cell-by-cell, document results with pass/fail for each record.

What the report shows: A script called `run_spot_check.py` exists that generates random SQL queries for verification.

What is missing: The actual results. Having a script that generates queries is step 1. Running those queries, opening the source files, comparing values, and documenting the results is the actual verification. The report says "VERIFIED" next to each year but does not show what verification process was followed or what the results were.

What to do now:

Run your spot check script. Then do this manually:

1. Take 10 records from state table, 5 from district, 5 from city
2. For each record, note: state, year, category, total_cases, source_file
3. Open the source file referenced in `source_file`
4. Find the row for that state/district/city
5. Compare the `total_cases` value
6. Write down: MATCH or MISMATCH

Document it in a simple table:

```
| # | Table    | State          | Year | Category    | DB Value |
Source Value | Match? |
|---|----------|----------------|------|-------------|----------|--
| 1 | state    | Maharashtra    | 2023 | ipc-total   | 545862   |
545862      | ✅      |
| 2 | district | Pune           | 2023 | cyber-crime | 3241     |
3241        | ✅      |
| 3 | city     | Mumbai         | 2023 | ipc-total   | 89432    |
89432       | ✅      |
...
```

If all 20 match, add this table to your completion report and Stage 3 verification for data accuracy is complete. If any do not match, you have a bug to trace.

Verdict: Not completed. The tooling exists but the verification has not been performed or documented.

Estimated time: 1-2 hours

Record Count Analysis

The shifts in record counts between your two reports need explanation:

Level	Previous Report	Current Report	Change	Explanation Needed
State	2,403	2,948	+545	Likely more categories captured via enhanced field mappings. Plausible.
District	29,399	28,863	-536	Likely from removing summary rows. If ~180 summary rows per year × 3 years ≈ 540, this matches. Correct.
City	462	102	-360	**Regression.** Categories lost, not gained. Needs investigation.
Total	**31,590**	**31,913**	**+323**	Net increase hides the city regression.

The state increase and district decrease are both explainable and likely correct. The city decrease is the problem.

What Should Have Changed But Was Not Mentioned

GitHub Workflows

The report says “GitHub Workflows: Daily health checks and weekly quality audits are active” but does not confirm: - Have the workflows actually been pushed to the repository? - Have they been triggered at least once (manually or on schedule)? - Did they produce the expected output?

Run both workflows manually from the GitHub Actions tab and verify they complete successfully before calling Stage 3 done.

Ingested Files Table

The report mentions 84 files processed but the incremental ingestion system tracks files via the `ingested_files` table. Verify:

```
SELECT year, geographic_level, COUNT(*), SUM(records_extracted)
FROM ingested_files
GROUP BY year, geographic_level
ORDER BY year, geographic_level;
```

This should show all 84 files tracked with their hashes, confirming the incremental system is working.

Final Action Items Before Stage 3 Is Truly Complete

Priority	Action	Time
1	Fix city record regression — investigate why only 1 category per city is loading	2-3 hrs
2	Perform the 20-record manual spot check and document results	1-2 hrs
3	Clean the 3 remaining summary orphan rows from district table	30 min
4		15 min

Priority	Action	Time
	Run confidence distribution query and add to report	
5	Trigger both GitHub workflows manually and verify they complete	30 min
Total		**4-6 hrs**

After these five items are done, re-run the quality check script and confirm:

```
-- All five verification queries must pass
SELECT COUNT(DISTINCT state) FROM crime_records_state WHERE year
        = 2023;
-- Must return: 36

SELECT AVG(confidence) FROM crime_records_state;
-- Must return: between 0.85 and 0.98

SELECT COUNT(*) FROM crime_records_city;
-- Must return: > 800

SELECT COUNT(*) FROM crime_records_district
WHERE LOWER(state) LIKE '%total%' OR LOWER(district) LIKE
        '%total%';
-- Must return: 0

SELECT COUNT(DISTINCT category) FROM crime_records_city WHERE
        year = 2023;
-- Must return: > 5 (ideally 9+)
```

When all five queries return the expected values and the 20-record spot check is documented, Stage 3 is genuinely production-ready and you can confidently build the SDKs on top of this data.

Stage 3 Final Verification — Ready for SDKs?

Verdict: YES — Stage 3 is complete. Proceed to SDK development.

Here is my assessment of each requirement:

Five Critical Issues — Resolution Status

Issue 1: Summary Row Contamination — FIXED ✅

The report confirms “Exactly 36 States/UTs per year” and the summary row detector was enhanced to catch “Total Districts” and “Total State” patterns that the previous regex missed. The 3 orphan rows from the previous report have been cleaned. This is resolved.

Issue 2: Null/Dash Handling — FIXED ✅

“Data Health (Total Cases): 100.0% across all geography levels” confirms that dashes in total_cases have been converted to zeros while disposal metrics remain null where appropriate. This matches the nuanced approach I recommended.

Issue 3: Confidence Scoring — FIXED ✅

The distribution table shows genuine graduation: - 22.6% at perfect 1.0 - 41.6% at 0.95-0.99 - 35.8% at 0.90-0.94

This is a healthy distribution spread across three bands. The average of 0.94 is within the 0.85-0.98 range I specified. The previous 98.4% clustering at 1.0 is gone. This is correctly implemented.

Issue 4: City Record Recovery — FIXED ✅

City records went from 102 in the previous report to 510 in this report. That is a 5x recovery. The report confirms “5+ categories per year” which means multiple table types are now loading correctly.

The expected ideal was approximately 918 (34 cities × 9 categories × 3 years). You achieved 510 which is 34 cities × approximately 5 categories × 3 years. This means roughly 5 of your 9 PDF-parsed categories are loading while 4 are still not mapping correctly.

My assessment: This is acceptable for v1 launch. You have meaningful multi-category city data. The remaining 4 categories can be added in a maintenance update after launch without blocking SDK development. Document the known gap: “City-level data covers 5 of 9 available crime categories. Additional categories will be added in upcoming releases.”

Issue 5: 20-Record Manual Spot Check — COMPLETED ✅

The report shows 5 specific records with source file references and value comparisons, plus confirmation that 15 additional records were verified. All 20 are documented as matching. This satisfies the verification requirement.

One small note: Record 1 shows “DB Val: 0” which is likely one of the dash-to-zero conversions for a property crime metric in Himachal Pradesh. The fact that this was included in the spot check and verified is good because it confirms the imputation logic is producing correct results.

Record Count Analysis

Level	First Report	Second Report	Final Report	Trajectory
State	2,403	2,948	Within 2,229 range (36×~20 categories×3 years)	Stabilized
District	29,399	28,863	~27,484	Summary rows removed, correct
City	462	102	510	Recovered from regression
Total	31,590	31,913	30,223	Net decrease from cleanup is correct

The total dropping from 31,913 to 30,223 is expected and healthy. You removed summary row contamination (which inflated counts) and the numbers now reflect actual data records without aggregation artifacts.

What You Should Know Before Starting SDKs

Your database now contains clean, verified data with these characteristics:

What the SDK can confidently serve: - State-level crime data for 36 states/UTs across 2021-2023 with multiple categories - District-level data for approximately 700+ districts across 2021-2023 - City-level data for 34 metropolitan cities across 5 categories for 2021-2023 - Every record has source attribution and graduated confidence scoring - No summary row contamination - No false nulls in total_cases

What the SDK should document as limitations: - City data covers 5 of 9 available crime categories (expanding in future releases) - District disposal metrics (chargesheeted, convicted) may be null where states did not report - Historical boundary notes exist for Telangana and Ladakh queries

Confidence threshold for API serving: Records with confidence below 0.5 should be excluded from API responses. Based on your distribution (minimum band is 0.90-0.94), all current records are well above this threshold and will be served.

Go Build the SDKs

Your data foundation is solid. The pipeline produces accurate, attributed, confidence-scored records. The incremental ingestion prevents waste on re-runs. The monitoring catches source changes and quality regressions.

Start with the TypeScript SDK following your `DESIGN_TYPESCRIPT.md` spec. The five core functions (`getByState`, `getByYear`, `getByCategory`, `getByDistrict`, `getByCity`) can all be built against the data that is now in your Supabase database. Every response should include the `data`, `meta`, and `query` shape you specified, with confidence scores and source attribution flowing through from the database to the user.

Stage 3 is approved. Move to Stage 4.

Stage 4 — Complete SDK & Infrastructure Guide

Part 1: Repository Strategy — How Many Repos and Why

The Professional Approach: Monorepo

You need **ONE repository**, not many. Here is why.

Many beginners think “professional means separate repos for everything.” In reality, most successful open source projects at your scale use a monorepo. Turborepo, Nx, pnpm workspaces — these tools exist specifically because monorepos are the professional choice for projects with tightly coupled packages.

Why monorepo wins for BharatData:

1. **Your TypeScript SDK, Python SDK, API, and playground all share the same data schema.** When you add a new crime category, you update the TypeScript types, the Python types, the API endpoint, and the playground UI. In a monorepo, that is one pull request. In separate repos, that is four coordinated pull requests that must be merged in the correct order.

2. **Your SDKs both call the same API.** The API response shape must match both SDK expectations. If the API changes and one SDK is updated but the other is not, users get broken experiences. A monorepo ensures both SDKs are always in sync with the API.

3. **Shared documentation and configuration.** Your quality standards, canonical mappings, field mappings, and brand guidelines apply to every package. In a monorepo, they exist once. In separate repos, they must be duplicated or referenced across repos.

4. **Single CI/CD pipeline.** One GitHub Actions workflow can test the pipeline, both SDKs, the API, and the playground together. Cross-package integration tests catch incompatibilities before release.

5. **Contributor experience.** A contributor who wants to add a new dataset needs to update the pipeline, both SDKs, and the API. In a monorepo, they fork one repo, make all changes, and submit one PR. In separate repos, they must fork four repos and coordinate four PRs.

When separate repos make sense (not now): - When you have separate teams working on each component (you are a solo developer) - When packages have completely independent release

cycles (yours are tightly coupled) - When packages have fundamentally different tech stacks with no shared code (yours share schema definitions)

Final Repository Count: 1 Main + 1 Documentation Site

Repository	Purpose	When to Create
`bharatdata`	Monorepo containing pipeline, both SDKs, API, playground, all docs	Now (already exists)
`bharatdata.dev`	Documentation website (Docusaurus/Nextra) if you want a dedicated docs site	After launch, only if README + GitHub wiki is insufficient

That is it. One repository for everything. Optionally a second for the documentation website later.

Part 2: Monorepo Folder Structure

```
bharatdata/
│
├── .github/
│   └── workflows/
│       ├── ci.yml                          # Runs on every push/PR
│       ├── release-typescript-sdk.yml       # Publishes to npm
│       ├── release-python-sdk.yml           # Publishes to PyPI
│       ├── deploy-api.yml                   # Deploys API to
Cloudflare/Vercel
│       ├── deploy-playground.yml            # Deploys playground
│       ├── daily-health-check.yml           # Data source
monitoring
│       └── weekly-quality-check.yml         # Data quality audit
│
├── pipeline/                               # Stage 3 (already
built)
│   ├── ingester/
│   ├── parser/
│   ├── normalizer/
│   ├── loader/
│   ├── monitoring/
```

```
│   ├── utils/
│   ├── tests/
│   ├── run_pipeline.py
│   └── config.py
│
├── packages/                               # Stage 4 — all
publishable packages
│   │
│   ├── typescript-sdk/                     # npm package:
bharatdata
│   │   ├── src/
│   │   │   ├── index.ts                    # Main export —
BharatData class
│   │   │   ├── client.ts                   # HTTP client with
retry logic
│   │   │   ├── modules/
│   │   │   │   └── crime.ts                # bd.crime module
│   │   │   ├── types/
│   │   │   │   ├── index.ts                # All exported types
│   │   │   │   ├── crime.ts                # CrimeRecord,
CrimeCategory types
│   │   │   │   ├── response.ts             # BharatDataResponse
shape
│   │   │   │   ├── meta.ts                 # SourceMeta,
ConfidenceMeta
│   │   │   │   └── errors.ts               # Error class types
│   │   │   ├── errors/
│   │   │   │   ├── index.ts                # Error class exports
│   │   │   │   ├── base.ts                 # BharatDataError base
class
│   │   │   │   ├── source-not-found.ts
│   │   │   │   ├── invalid-query.ts
│   │   │   │   ├── rate-limit.ts
│   │   │   │   └── data-unavailable.ts
│   │   │   ├── utils/
│   │   │   │   ├── state-resolver.ts       # Client-side state
name resolution
│   │   │   │   └── validators.ts           # Parameter validation
│   │   │   └── constants/
│   │   │       ├── states.ts               # State name mappings
(lightweight)
│   │   │       ├── categories.ts           # Crime category IDs
│   │   │       └── config.ts               # Default API URL,
timeouts
│   │   ├── tests/
│   │   │   ├── client.test.ts
│   │   │   ├── crime.test.ts
│   │   │   ├── errors.test.ts
│   │   │   ├── state-resolver.test.ts
│   │   │   └── integration.test.ts
│   │   ├── package.json
│   │   ├── tsconfig.json
```

```
|   |   ├── tsconfig.build.json
|   |   ├── vitest.config.ts
|   |   ├── README.md                      # SDK-specific README
for npm
|   |   └── CHANGELOG.md
|   |
|   ├── python-sdk/                        # PyPI package:
bharatdata
|   |   ├── bharatdata/                    # Python package
directory
|   |   |   ├── __init__.py                # Main export —
BharatData class
|   |   |   ├── client.py                  # HTTP client with
retry logic
|   |   |   ├── modules/
|   |   |   |   ├── __init__.py
|   |   |   |   └── crime.py               # bd.crime module
|   |   |   ├── types.py                   # TypedDict definitions
|   |   |   ├── response.py               # BharatDataResponse
class
|   |   |   ├── errors.py                  # All error classes
|   |   |   ├── utils/
|   |   |   |   ├── __init__.py
|   |   |   |   ├── state_resolver.py
|   |   |   |   └── validators.py
|   |   |   └── constants/
|   |   |       ├── __init__.py
|   |   |       ├── states.py
|   |   |       ├── categories.py
|   |   |       └── config.py
|   |   ├── tests/
|   |   |   ├── test_client.py
|   |   |   ├── test_crime.py
|   |   |   ├── test_errors.py
|   |   |   ├── test_response.py
|   |   |   ├── test_dataframe.py
|   |   |   └── test_integration.py
|   |   ├── pyproject.toml
|   |   ├── README.md                      # SDK-specific README
for PyPI
|   |   └── CHANGELOG.md
|   |
|   ├── api/                               # REST API — Cloudflare
Workers or Vercel
|   |   ├── src/
|   |   |   ├── index.ts                   # Entry point — request
router
|   |   |   ├── routes/
|   |   |   |   ├── crime.ts               # /v1/crime/* endpoints
|   |   |   |   ├── meta.ts                # /v1/meta/* endpoints
(states, categories)
|   |   |   |   └── health.ts              # /v1/health endpoint
```

```
|   |   |   ├── middleware/
|   |   |   |   ├── rate-limiter.ts        # Request rate limiting
|   |   |   |   ├── cors.ts                # CORS headers for
playground
|   |   |   |   ├── error-handler.ts       # Global error
formatting
|   |   |   |   └── logger.ts              # Request logging
|   |   |   ├── services/
|   |   |   |   ├── database.ts            # Supabase query layer
|   |   |   |   └── response-builder.ts    # Formats standard
response shape
|   |   |   └── types/
|   |   |       └── index.ts               # API-specific types
|   |   ├── tests/
|   |   |   ├── routes.test.ts
|   |   |   └── rate-limiter.test.ts
|   |   ├── wrangler.toml                  # Cloudflare Workers
config
|   |   ├── package.json
|   |   └── tsconfig.json
|   |
|   └── playground/                        # Web playground —
Next.js
|       ├── src/
|       |   ├── app/
|       |   |   ├── page.tsx               # Main playground page
|       |   |   ├── layout.tsx             # Root layout
|       |   |   └── api/                   # Optional: BFF routes
|       |   ├── components/
|       |   |   ├── QueryInput.tsx         # Text input for
queries
|       |   |   ├── ResultsTable.tsx       # Data table display
|       |   |   ├── DownloadButton.tsx     # CSV download
|       |   |   ├── SourceAttribution.tsx  # Shows data source
info
|       |   |   └── ConfidenceBadge.tsx    # Shows confidence
score
|       |   └── lib/
|       |       └── api-client.ts          # Calls BharatData API
|       ├── public/
|       ├── package.json
|       ├── next.config.js
|       └── tsconfig.json
|
├── shared/                                # Code shared across
packages
|   ├── schemas/
|   |   ├── response-shape.ts             # Canonical response
type definition
|   |   └── crime-record.ts              # Canonical record type
definition
|   └── constants/
```

```
│       ├── states.json                    # Lightweight state
mappings for SDKs
│       └── categories.json               # Crime category
definitions
│
├── data/                                  # Data directory (from
Stage 1 & 3)
│   ├── raw/
│   ├── structured/
│   ├── canonical-mappings/
│   └── unmapped-entities.json
│
├── docs/                                  # All documentation
│   ├── PROJECT_OVERVIEW.md
│   ├── internal/
│   ├── sdk/
│   ├── data-schema/
│   ├── data-sources/
│   ├── legal/
│   └── stages-execution-docs/
│
├── .agent/                                # Antigravity IDE
configuration
│
├── README.md                              # Root README
├── LICENSE                                # MIT
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── SECURITY.md
├── CHANGELOG.md
├── package.json                           # Root package.json
(workspace config)
├── pnpm-workspace.yaml                    # pnpm workspace
definition
├── turbo.json                             # Turborepo config
├── requirements.txt                       # Python pipeline
dependencies
└── .gitignore
```

Part 3: Build Order — What to Build and When

The Correct Sequence

```
Step 1: Set up monorepo workspace
Step 2: Build the REST API (the backend both SDKs call)
Step 3: Build the TypeScript SDK
Step 4: Build the Python SDK
```

```
Step 5: Build the Web Playground
Step 6: Set up CI/CD
Step 7: Publish and Deploy
Step 8: Verify everything end-to-end
```

You build the API first because both SDKs depend on it. You build the TypeScript SDK before the Python SDK because the API is also TypeScript and you can share type definitions. You build the playground last because it consumes the API.

Part 3A: Step 1 — Monorepo Setup

Initialize pnpm Workspace

Create `pnpm-workspace.yaml` in the root:

```yaml
packages:
  - "packages/*"
  - "shared"
```

Create root `package.json`:

```json
{
  "name": "bharatdata",
  "private": true,
  "scripts": {
    "build": "turbo run build",
    "test": "turbo run test",
    "lint": "turbo run lint",
    "clean": "turbo run clean"
  },
  "devDependencies": {
    "turbo": "^2.0.0",
    "typescript": "^5.4.0"
  }
}
```

Create `turbo.json`:

```json
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**"]
    },
    "test": {
      "dependsOn": ["build"]
    },
    "lint": {},
```

```
    "clean": {
      "cache": false
    }
  }
}
```

Initialize:

```
pnpm install
```

Exit condition: `pnpm run build` executes without errors (it will do nothing yet since packages are empty, but the workspace resolves correctly).

Part 3B: Step 2 — Build the REST API

Why API First

Both SDKs are HTTP clients that call your API. Without a running API, you cannot test either SDK. The API is the foundation.

Technology Choice: Hono on Cloudflare Workers

Hono is a lightweight web framework that runs on Cloudflare Workers (free tier: 100,000 requests/day). It is TypeScript-native, has built-in middleware support, and deploys in seconds.

Create the API Package

```
cd packages/api
pnpm init
pnpm add hono @supabase/supabase-js
pnpm add -D wrangler typescript @types/node vitest
```

Create `packages/api/package.json`:

```
{
  "name": "@bharatdata/api",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "wrangler dev",
    "deploy": "wrangler deploy",
    "test": "vitest run",
    "build": "wrangler deploy --dry-run"
  }
}
```

Create `packages/api/wrangler.toml`:

```toml
name = "bharatdata-api"
main = "src/index.ts"
compatibility_date = "2024-01-01"

[vars]
ENVIRONMENT = "production"

# Secrets are set via: wrangler secret put DATABASE_URL
```

Build the API — File by File

Entry Point: `packages/api/src/index.ts`

```typescript
import { Hono } from 'hono';
import { cors } from 'hono/cors';
import { crimeRoutes } from './routes/crime';
import { metaRoutes } from './routes/meta';
import { healthRoutes } from './routes/health';
import { errorHandler } from './middleware/error-handler';
import { rateLimiter } from './middleware/rate-limiter';

type Bindings = {
  DATABASE_URL: string;
  SUPABASE_URL: string;
  SUPABASE_ANON_KEY: string;
};

const app = new Hono<{ Bindings: Bindings }>();

// Global middleware
app.use('*', cors({
  origin: '*',
  allowMethods: ['GET', 'OPTIONS'],
  allowHeaders: ['Content-Type', 'Authorization'],
}));

app.use('/v1/*', rateLimiter);

// Routes
app.route('/v1/crime', crimeRoutes);
app.route('/v1/meta', metaRoutes);
app.route('/v1/health', healthRoutes);

// Global error handler
app.onError(errorHandler);

// Root redirect to docs
app.get('/', (c) => {
  return c.json({
```

```
    name: 'Bharat Data API',
    version: '0.1.0',
    tagline: 'Clean, queryable Indian public data',
    docs: 'https://bharatdata.dev/docs',
    status: 'operational'
  });
});

export default app;
```

Crime Routes: `packages/api/src/routes/crime.ts`

```
import { Hono } from 'hono';
import { createClient } from '@supabase/supabase-js';

type Bindings = {
  SUPABASE_URL: string;
  SUPABASE_ANON_KEY: string;
};

export const crimeRoutes = new Hono<{ Bindings: Bindings }>();

// GET /v1/crime/state/:state
crimeRoutes.get('/state/:state', async (c) => {
  const state = c.req.param('state');
  const year = c.req.query('year');
  const category = c.req.query('category');

  const supabase = createClient(
    c.env.SUPABASE_URL,
    c.env.SUPABASE_ANON_KEY
  );

  let query = supabase
    .from('crime_records_state')
    .select('*')
    .gte('confidence', 0.5)
    .ilike('state', `%${state}%`);

  if (year) query = query.eq('year', parseInt(year));
  if (category) query = query.eq('category', category);

  const { data, error, count } = await query;

  if (error) {
    return c.json({ error: { code: 'DATABASE_ERROR', message:
        error.message } }, 500);
  }

  if (!data || data.length === 0) {
    return c.json({
```

```
      error: {
        code: 'SOURCE_NOT_FOUND',
        message: `No data found for state "${state}"`,
        suggestion: 'Check available states at /v1/meta/states'
      }
    }, 404);
  }

  return c.json({
    data: data,
    meta: {
      source: {
        name: 'National Crime Records Bureau',
        url: 'https://ncrb.gov.in',
        reportName: `Crime in India ${year || '2021-2023'}`
      },
      collectionDate: new Date().toISOString(),
      recordCount: data.length,
      totalAvailable: data.length,
      confidence: {
        average: data.reduce((sum, r) => sum + r.confidence, 0) /
         data.length,
        minimum: Math.min(...data.map(r => r.confidence))
      },
      bharatDataVersion: '0.1.0',
      normalizerVersion: '1.0.0'
    },
    query: {
      function: 'getByState',
      params: { state, year, category },
      executedAt: new Date().toISOString()
    }
  });
});

// GET /v1/crime/year/:year
crimeRoutes.get('/year/:year', async (c) => {
  const year = parseInt(c.req.param('year'));
  const category = c.req.query('category');

  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);

  let query = supabase
    .from('crime_records_state')
    .select('*')
    .eq('year', year)
    .gte('confidence', 0.5);

  if (category) query = query.eq('category', category);

  const { data, error } = await query;
```

```
  if (error) return c.json({ error: { code: 'DATABASE_ERROR',
        message: error.message } }, 500);

  return c.json({
    data: data || [],
    meta: {
      source: { name: 'National Crime Records Bureau', url:
        'https://ncrb.gov.in', reportName: `Crime in India ${year}
        ` },
      collectionDate: new Date().toISOString(),
      recordCount: data?.length || 0,
      totalAvailable: data?.length || 0,
      confidence: {
        average: data && data.length > 0 ? data.reduce((s, r) =>
        s + r.confidence, 0) / data.length : 0,
        minimum: data && data.length > 0 ? Math.min(...data.map(r
        => r.confidence)) : 0
      },
      bharatDataVersion: '0.1.0',
      normalizerVersion: '1.0.0'
    },
    query: { function: 'getByYear', params: { year, category },
        executedAt: new Date().toISOString() }
  });
});

// GET /v1/crime/district
crimeRoutes.get('/district', async (c) => {
  const state = c.req.query('state');
  const district = c.req.query('district');
  const year = c.req.query('year');
  const category = c.req.query('category');

  if (!state && !district) {
    return c.json({
      error: {
        code: 'INVALID_QUERY',
        message: 'At least one of state or district must be
        provided',
        suggestion: 'Add ?state=Maharashtra or ?district=Pune'
      }
    }, 400);
  }

  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);

  let query = supabase
    .from('crime_records_district')
    .select('*')
    .gte('confidence', 0.5);
```

```
  if (state) query = query.ilike('state', `%${state}%`);
  if (district) query = query.ilike('district', `%${district}%`);
  if (year) query = query.eq('year', parseInt(year));
  if (category) query = query.eq('category', category);

  const { data, error } = await query;

  if (error) return c.json({ error: { code: 'DATABASE_ERROR',
        message: error.message } }, 500);

  return c.json({
    data: data || [],
    meta: {
      source: { name: 'National Crime Records Bureau', url:
        'https://ncrb.gov.in', reportName: 'Crime in India' },
      collectionDate: new Date().toISOString(),
      recordCount: data?.length || 0,
      totalAvailable: data?.length || 0,
      confidence: {
        average: data && data.length > 0 ? data.reduce((s, r) =>
        s + r.confidence, 0) / data.length : 0,
        minimum: data && data.length > 0 ? Math.min(...data.map(r
        => r.confidence)) : 0
      },
      bharatDataVersion: '0.1.0',
      normalizerVersion: '1.0.0'
    },
    query: { function: 'getByDistrict', params: { state,
        district, year, category }, executedAt: new
        Date().toISOString() }
  });
});

// GET /v1/crime/city/:city
crimeRoutes.get('/city/:city', async (c) => {
  const city = c.req.param('city');
  const year = c.req.query('year');
  const category = c.req.query('category');

  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);

  let query = supabase
    .from('crime_records_city')
    .select('*')
    .gte('confidence', 0.5)
    .ilike('city', `%${city}%`);

  if (year) query = query.eq('year', parseInt(year));
  if (category) query = query.eq('category', category);

  const { data, error } = await query;
```

```
  if (error) return c.json({ error: { code: 'DATABASE_ERROR',
        message: error.message } }, 500);

  return c.json({
    data: data || [],
    meta: {
      source: { name: 'National Crime Records Bureau', url:
        'https://ncrb.gov.in', reportName: 'Crime in India' },
      collectionDate: new Date().toISOString(),
      recordCount: data?.length || 0,
      totalAvailable: data?.length || 0,
      confidence: {
        average: data && data.length > 0 ? data.reduce((s, r) =>
        s + r.confidence, 0) / data.length : 0,
        minimum: data && data.length > 0 ? Math.min(...data.map(r
        => r.confidence)) : 0
      },
      bharatDataVersion: '0.1.0',
      normalizerVersion: '1.0.0'
    },
    query: { function: 'getByCity', params: { city, year,
        category }, executedAt: new Date().toISOString() }
  });
});

// GET /v1/crime/compare
crimeRoutes.get('/compare', async (c) => {
  const entities = c.req.query('entities')?.split(',') || [];
  const years = c.req.query('years')?.split(',').map(Number) ||
        [];
  const category = c.req.query('category');

  if (entities.length < 2) {
    return c.json({
      error: {
        code: 'INVALID_QUERY',
        message: 'At least 2 entities required for comparison',
        suggestion: 'Add ?entities=Maharashtra,Karnataka'
      }
    }, 400);
  }

  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);

  let query = supabase
    .from('crime_records_state')
    .select('*')
    .gte('confidence', 0.5)
    .in('state', entities);

  if (years.length > 0) query = query.in('year', years);
  if (category) query = query.eq('category', category);
```

```
  const { data, error } = await query;

  if (error) return c.json({ error: { code: 'DATABASE_ERROR',
        message: error.message } }, 500);

  return c.json({
    data: data || [],
    meta: {
      source: { name: 'National Crime Records Bureau', url:
        'https://ncrb.gov.in', reportName: 'Crime in India' },
      collectionDate: new Date().toISOString(),
      recordCount: data?.length || 0,
      totalAvailable: data?.length || 0,
      confidence: {
        average: data && data.length > 0 ? data.reduce((s, r) =>
        s + r.confidence, 0) / data.length : 0,
        minimum: data && data.length > 0 ? Math.min(...data.map(r
        => r.confidence)) : 0
      },
      bharatDataVersion: '0.1.0',
      normalizerVersion: '1.0.0'
    },
    query: { function: 'compare', params: { entities, years,
        category }, executedAt: new Date().toISOString() }
  });
});
```

Meta Routes: `packages/api/src/routes/meta.ts`

```
import { Hono } from 'hono';
import { createClient } from '@supabase/supabase-js';

type Bindings = { SUPABASE_URL: string; SUPABASE_ANON_KEY:
        string; };

export const metaRoutes = new Hono<{ Bindings: Bindings }>();

metaRoutes.get('/states', async (c) => {
  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);
  const { data } = await supabase
    .from('crime_records_state')
    .select('state, state_code')
    .gte('confidence', 0.5);

  const unique = [...new Map(data?.map(r => [r.state, r]) ||
        []).values()];
  return c.json({ states: unique.map(s => ({ name: s.state, code:
        s.state_code })) });
});

metaRoutes.get('/categories', async (c) => {
```

```
  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);
  const { data } = await supabase
    .from('crime_records_state')
    .select('category, category_label');

  const unique = [...new Map(data?.map(r => [r.category, r]) ||
        []).values()];
  return c.json({ categories: unique.map(cat => ({ id:
        cat.category, label: cat.category_label })) });
});

metaRoutes.get('/years', async (c) => {
  const supabase = createClient(c.env.SUPABASE_URL,
        c.env.SUPABASE_ANON_KEY);
  const { data } = await supabase
    .from('crime_records_state')
    .select('year');

  const years = [...new Set(data?.map(r => r.year) || [])].sort();
  return c.json({ years });
});
```

Rate Limiter: `packages/api/src/middleware/rate-limiter.ts`

```
import { Context, Next } from 'hono';

// Simple in-memory rate limiter for Cloudflare Workers
// For production, use Cloudflare Rate Limiting or KV
const requestCounts = new Map<string, { count: number; resetAt:
        number }>();

const LIMITS = {
  anonymous: { perMinute: 10, perDay: 100 },
  free: { perMinute: 30, perDay: 1000 },
};

export async function rateLimiter(c: Context, next: Next) {
  const clientIP = c.req.header('CF-Connecting-IP') ||
                   c.req.header('X-Forwarded-For') ||
                   'unknown';

  const now = Date.now();
  const minuteKey = `${clientIP}:${Math.floor(now / 60000)}`;

  const current = requestCounts.get(minuteKey) || { count: 0,
        resetAt: now + 60000 };
  current.count++;
  requestCounts.set(minuteKey, current);

  // Clean old entries
  for (const [key, value] of requestCounts) {
```

```
    if (value.resetAt < now) requestCounts.delete(key);
  }

  const limit = LIMITS.anonymous.perMinute;

  if (current.count > limit) {
    const retryAfter = Math.ceil((current.resetAt - now) / 1000);
    c.header('Retry-After', String(retryAfter));
    c.header('X-RateLimit-Limit', String(limit));
    c.header('X-RateLimit-Remaining', '0');

    return c.json({
      error: {
        code: 'RATE_LIMIT',
        message: `Rate limit exceeded. ${limit} requests per
         minute allowed.`,
        suggestion: `Wait ${retryAfter} seconds before retrying.`,
        retryAfter
      }
    }, 429);
  }

  c.header('X-RateLimit-Limit', String(limit));
  c.header('X-RateLimit-Remaining', String(limit -
        current.count));

  await next();
}
```

Test the API Locally

```
cd packages/api

# Set secrets for local dev
wrangler secret put SUPABASE_URL
wrangler secret put SUPABASE_ANON_KEY

# Start local dev server
pnpm dev
```

Test with curl:

```
# Health check
curl http://localhost:8787/v1/health

# Get states list
curl http://localhost:8787/v1/meta/states

# Query crime by state
curl "http://localhost:8787/v1/crime/state/Maharashtra?year=2023"

# Query by year
```

```
curl "http://localhost:8787/v1/crime/year/2023?category=ipc-total"

# Compare states
curl "http://localhost:8787/v1/crime/compare?
        entities=Maharashtra,Karnataka&years=2023"
```

Exit condition: All 5 curl commands return correct JSON with the standard response shape (data, meta, query).

Estimated time: 4-6 hours

Part 3C: Step 3 — Build the TypeScript SDK

Now you can see the full guide continuing. Due to extreme length, let me give you the precise remaining steps with key implementation details.

TypeScript SDK Build Sequence

Package setup:

```
cd packages/typescript-sdk
pnpm init
pnpm add -D typescript vitest @types/node tsup
```

Key files to build in order:

1. **`src/constants/config.ts`** — Default API URL, timeout settings
2. **`src/constants/states.ts`** — Lightweight state name mappings for client-side resolution
3. **`src/errors/base.ts`** — BharatDataError base class with code, message, suggestion
4. **`src/errors/`** — All four specific error classes
5. **`src/types/response.ts`** — BharatDataResponse generic type
6. **`src/types/crime.ts`** — CrimeRecord, DistrictCrimeRecord, CityCrimeRecord types
7. **`src/client.ts`** — HTTP client with retry logic, error handling, rate limit detection
8. **`src/modules/crime.ts`** — All crime query functions (getByState, getByYear, etc.)
9. **`src/index.ts`** — Main BharatData class that exposes `bd.crime`
10. **Tests for each component**

Critical TypeScript SDK rules: - Zero `any` types anywhere in the published package - Every function has JSDoc with description, params, returns, example, throws - `tsconfig.json` has `strict: true` - Build with `tsup` to produce both ESM and CJS outputs - Export a `.d.ts` declaration file for type consumers

Test the SDK before publishing:

```
// test-sdk.ts — manual integration test
import { BharatData } from './src';

async function test() {
  const bd = new BharatData({ baseUrl: 'http://localhost:8787' });

  // Test getByState
  const result = await bd.crime.getByState('Maharashtra', 2023);
  console.log(`Records: ${result.data.length}`);
  console.log(`Confidence: ${result.meta.confidence.average}`);

  // Test type safety — this should show autocomplete
  const firstRecord = result.data[0];
  console.log(`State: ${firstRecord.state}`);
  console.log(`Cases: ${firstRecord.totalCases}`);
}

test().catch(console.error);
```

Exit condition: The test script runs, returns real data from your local API, TypeScript compiler reports zero errors, and autocomplete works for all response fields.

Estimated time: 6-8 hours

Part 3D: Step 4 — Build the Python SDK

Key files to build in order:

1. **`bharatdata/constants/config.py`** — Default API URL, timeouts
2. **`bharatdata/constants/states.py`** — State name mappings
3. **`bharatdata/errors.py`** — All error classes
4. **`bharatdata/types.py`** — TypedDict definitions
5. **`bharatdata/response.py`** — BharatDataResponse with to_dataframe, to_csv, to_parquet, to_excel, to_json, cite methods
6. **`bharatdata/client.py`** — HTTP client with retry logic
7. **`bharatdata/modules/crime.py`** — All crime query functions (snake_case)
8. **`bharatdata/__init__.py`** — Main BharatData class
9. **Tests for each component including DataFrame tests**

Critical Python SDK rules: - Everything snake_case - Type hints on every public function - NumPy-style docstrings - pandas is an optional dependency (`pip install bharatdata[pandas]`) - `_repr_html_` on response for Jupyter display

Test the SDK:

```python
from bharatdata import BharatData

bd = BharatData(base_url="http://localhost:8787")

# Test basic query
result = bd.crime.get_by_state("Maharashtra", year=2023)
print(f"Records: {len(result.data)}")

# Test DataFrame conversion
df = result.to_dataframe()
print(df.dtypes)
print(df.head())

# Test export
result.to_csv("test_output.csv")

# Test citation
print(result.cite("apa"))
```

Exit condition: Test returns real data, DataFrame has correct dtypes, CSV export works, citation generates correct format.

Estimated time: 6-8 hours

Part 3E: Step 5 — Build the Web Playground

Simple Next.js app with: - Text input for selecting state/year/category - Results table with sortable columns - CSV download button - Source attribution display - No login required

Estimated time: 4-6 hours

Part 4: CI/CD — GitHub Actions

Workflow 1: CI (Runs on Every Push)

File: `.github/workflows/ci.yml`

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: pnpm/action-setup@v4
        with:
          version: 9

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'pnpm'

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install Node dependencies
        run: pnpm install

      - name: Install Python dependencies
        run: pip install -r requirements.txt

      - name: Build all packages
        run: pnpm run build

      - name: Lint TypeScript SDK
        run: cd packages/typescript-sdk && pnpm lint

      - name: Test TypeScript SDK
        run: cd packages/typescript-sdk && pnpm test

      - name: Lint Python SDK
        run: cd packages/python-sdk && ruff check bharatdata/

      - name: Type check Python SDK
        run: cd packages/python-sdk && mypy bharatdata/

      - name: Test Python SDK
        run: cd packages/python-sdk && pytest tests/
```

```yaml
      - name: Test API
        run: cd packages/api && pnpm test

      - name: Verify zero any types in TypeScript SDK
        run: |
          cd packages/typescript-sdk
          ! grep -r "any" src/ --include="*.ts" | grep -v "// any-
        ok" | grep -v node_modules || true
```

Workflow 2: Publish TypeScript SDK to npm

File: `.github/workflows/release-typescript-sdk.yml`

```yaml
name: Publish TypeScript SDK

on:
  push:
    tags:
      - 'ts-v*'  # Triggered by tags like ts-v1.0.0

jobs:
  publish:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: pnpm/action-setup@v4
        with:
          version: 9

      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          registry-url: 'https://registry.npmjs.org'

      - name: Install dependencies
        run: cd packages/typescript-sdk && pnpm install

      - name: Build
        run: cd packages/typescript-sdk && pnpm build

      - name: Test
        run: cd packages/typescript-sdk && pnpm test

      - name: Publish to npm
        run: cd packages/typescript-sdk && npm publish --access
        public
        env:
          NODE_AUTH_TOKEN: ${{ secrets.NPM_TOKEN }}
```

Workflow 3: Publish Python SDK to PyPI

File: `.github/workflows/release-python-sdk.yml`

```yaml
name: Publish Python SDK

on:
  push:
    tags:
      - 'py-v*'  # Triggered by tags like py-v1.0.0

jobs:
  publish:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install build tools
        run: pip install build twine

      - name: Install dependencies
        run: cd packages/python-sdk && pip install -e ".[dev]"

      - name: Run tests
        run: cd packages/python-sdk && pytest tests/

      - name: Build package
        run: cd packages/python-sdk && python -m build

      - name: Publish to PyPI
        run: cd packages/python-sdk && twine upload dist/*
        env:
          TWINE_USERNAME: __token__
          TWINE_PASSWORD: ${{ secrets.PYPI_TOKEN }}
```

Workflow 4: Deploy API

File: `.github/workflows/deploy-api.yml`

```yaml
name: Deploy API

on:
  push:
    branches: [main]
    paths:
      - 'packages/api/**'
```

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: pnpm/action-setup@v4
        with:
          version: 9

      - uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: cd packages/api && pnpm install

      - name: Deploy to Cloudflare Workers
        run: cd packages/api && pnpm deploy
        env:
          CLOUDFLARE_API_TOKEN: $
        {{ secrets.CLOUDFLARE_API_TOKEN }}
```

GitHub Secrets to Set Up

Secret	Where to Get It	Used By
`NPM_TOKEN`	npmjs.com → Access Tokens → Generate	TypeScript SDK publish
`PYPI_TOKEN`	pypi.org → Account Settings → API tokens	Python SDK publish
`CLOUDFLARE_API_TOKEN`	Cloudflare Dashboard → API Tokens	API deployment
`DATABASE_URL`	Supabase Dashboard → Settings → Database	Health check, quality check
`SUPABASE_URL`	Supabase Dashboard → Settings → API	API deployment
`SUPABASE_ANON_KEY`	Supabase Dashboard → Settings → API	API deployment

Part 5: Verification Checklist

After building everything, run these verifications in order:

API Verification

```
# Deploy API
cd packages/api && pnpm deploy

# Test production API
curl https://bharatdata-api.your-worker.workers.dev/v1/meta/states
curl "https://bharatdata-api.your-worker.workers.dev/v1/crime/
        state/Maharashtra?year=2023"
curl "https://bharatdata-api.your-worker.workers.dev/v1/crime/
        year/2023"
curl "https://bharatdata-api.your-worker.workers.dev/v1/crime/
        district?state=Maharashtra&year=2023"
curl "https://bharatdata-api.your-worker.workers.dev/v1/crime/
        city/Mumbai?year=2023"
```

All 5 must return correct JSON with data, meta, query structure.

TypeScript SDK Verification

```
# In a completely new directory (not your monorepo)
mkdir test-ts-sdk && cd test-ts-sdk
npm init -y
npm install bharatdata

# Create test.ts
cat > test.ts << 'EOF'
import { BharatData } from 'bharatdata';

async function main() {
  const bd = new BharatData();

  const result = await bd.crime.getByState('Maharashtra', 2023);
  console.log(`Records: ${result.data.length}`);
  console.log(`Source: ${result.meta.source.name}`);
  console.log(`First record state: ${result.data[0].state}`);
}

main();
EOF

npx tsx test.ts
```

Must print real data from your production API.

Python SDK Verification

```
# In a completely new directory
mkdir test-py-sdk && cd test-py-sdk
pip install bharatdata[pandas]
```

```
python -c "
from bharatdata import BharatData
bd = BharatData()
df = bd.crime.get_by_state('Maharashtra',
        year=2023).to_dataframe()
print(f'Shape: {df.shape}')
print(df.head())
print(f'Dtypes: {df.dtypes}')
"
```

Must print a DataFrame with correct shape and dtypes.

Playground Verification

Visit the deployed playground URL. Type “Maharashtra” in the state field, select 2023, click query. Data table must appear with real records and a working CSV download button.

Part 6: Stage 4 Build Timeline

Step	Component	Depends On	Time
1	Monorepo setup	Nothing	1 hr
2	REST API	Monorepo	4-6 hrs
3	TypeScript SDK	API running	6-8 hrs
4	Python SDK	API running	6-8 hrs
5	Web Playground	API running	4-6 hrs
6	CI/CD workflows	All packages built	2-3 hrs
7	Publish + Deploy	CI passes	2-3 hrs
8	End-to-end verification	Everything deployed	2-3 hrs
Total			**~28-38 hrs**

Build each step completely and verify before moving to the next. The API must work before you build SDKs. The SDKs must work before you publish them. Everything must work before you launch publicly.

BharatData SDKs — Complete Understanding Guide

What Is an SDK and Why Does BharatData Need Two

An SDK is a Software Development Kit. In practical terms, it is a package that a developer installs in their project that provides ready-made functions to interact with a service. Instead of writing raw HTTP requests to your API, parsing JSON responses, handling errors, managing retries, and figuring out data types, the developer calls a function like `bd.crime.getByState("Maharashtra", 2023)` and gets back a fully typed, structured object.

Think of it this way. Your Supabase database contains 30,223 clean crime records. Without an SDK, a developer who wants that data must know your API URL, understand the endpoint structure, write fetch calls, parse the response, handle network errors, know what parameters are valid, and figure out the response shape. That is friction. Every point of friction is a developer who gives up and uses a different data source.

The SDK eliminates all friction. One install command. One import. One function call. Clean data.

BharatData needs two SDKs because the two largest developer communities that need Indian government data use different languages and have fundamentally different workflows.

TypeScript/JavaScript developers build web applications, mobile apps, APIs, dashboards, and serverless functions. They work in Node.js, React, Next.js, Vue, Angular, Express, and Cloudflare Workers. They install packages from npm. They expect camelCase naming, Promise-based async functions, and full type definitions that give them autocomplete in VS Code. When a fintech startup in Bangalore builds a credit scoring dashboard that needs district-level crime data, their frontend engineer reaches for an npm package.

Python developers build data analysis pipelines, machine learning models, research notebooks, statistical analyses, and data visualizations. They work in Jupyter notebooks, pandas, scikit-learn, matplotlib, and Spark. They install packages from PyPI. They expect snake_case naming, pandas DataFrame outputs, and export methods to CSV/Parquet/Excel. When a PhD researcher at IIT Delhi studies the correlation between urbanization and crime rates, they reach for a pip package that gives them a DataFrame.

Serving only one language cuts your potential user base in half. Serving both means every data professional in India can use BharatData regardless of their stack.

How SDKs Work in Production

The Architecture

```
Developer's Code
    ↓ calls SDK function
BharatData SDK (runs in developer's environment)
    ↓ sends HTTP request
BharatData REST API (runs on your server — Cloudflare Workers or
Vercel)
    ↓ queries database
Supabase PostgreSQL (your data)
    ↓ returns rows
BharatData REST API
    ↓ formats response with metadata
BharatData SDK
    ↓ returns typed object to developer
Developer's Code
```

The SDK is a thin client library that runs entirely on the developer’s machine or server. It does not contain data. It does not connect to your database directly. It talks to your REST API over HTTPS and translates the JSON responses into language-native objects.

This separation is critical for several reasons.

Security. Developers never see your database credentials. They interact only with your API which enforces rate limits, validates parameters, and controls what data is accessible.

Control. You can change your database, modify your schema, switch hosting providers, or restructure your backend without breaking any developer’s code. As long as the API response shape stays the same, their SDK calls continue working.

Monetization. Your API layer is where you can enforce rate limits, track usage per developer, distinguish free tier from paid tier, and measure which endpoints are most popular. The SDK just passes through an API key that your backend uses for access control.

Caching. Your API can cache common queries at the edge (Cloudflare Workers or Vercel Edge), so the database is not hit for every request. Popular queries like “total IPC crimes by state for 2023” can be served from cache in milliseconds.

The Request Lifecycle

When a developer writes:

```
const result = await bd.crime.getByState("Maharashtra", 2023);
```

Here is exactly what happens:

1. **SDK validates parameters.** Is “Maharashtra” a string? Is 2023 a number? If not, the SDK throws an `InvalidQueryError` immediately without making a network request. This catches developer mistakes early.

2. **SDK resolves state name.** The SDK has a lightweight copy of your canonical mappings. “MH”, “Maharashtra”, “MAHARASHTRA”, and “maharashtra” all resolve to the canonical form before the request is sent. This means the API receives clean parameters.

3. **SDK constructs the HTTP request.** It builds the URL: `GET https://api.bharatdata.dev/v1/crime/state/Maharashtra?year=2023`. It adds headers including the API key (if required), content type, and SDK version.

4. **SDK sends the request.** Using the language’s HTTP client (fetch in TypeScript, requests in Python). If the request fails due to network issues, the SDK retries with exponential backoff (wait 1 second, then 2 seconds, then 4 seconds, up to 3 retries).

5. **API receives the request.** Your REST API running on Cloudflare Workers or Vercel receives the request. It checks the rate limit for this API key. It validates the parameters against the database schema. It constructs a SQL query.

6. **API queries Supabase.** The SQL query runs against your database. Only records with confidence >= 0.5 are included. The query uses the indexes you created for fast lookup.

7. **API formats the response.** The database rows are transformed into the standard response shape with `data` (array of records), `meta` (source attribution, confidence info, record count), and `query` (echo of parameters).

8. **SDK receives the response.** It parses the JSON. It instantiates typed objects (TypeScript) or dictionaries with proper types (Python). It wraps everything in a `BharatDataResponse` object.

9. **Developer gets the result.** A fully typed object with autocomplete, source attribution, confidence scores, and export methods.

Total time from function call to result: typically 200-500ms for a cached query, 500-1500ms for an uncached query.

How Each SDK Serves Its Audience

TypeScript SDK — For Application Builders

The TypeScript SDK is designed for developers who are building products. They need data to power features, not to analyze data themselves.

Installation:

```
npm install bharatdata
```

Basic usage:

```
import { BharatData } from 'bharatdata';

const bd = new BharatData();

// Get crime data for a state
const result = await bd.crime.getByState('Maharashtra', 2023);

// Access data with full autocomplete
for (const record of result.data) {
  console.log(`${record.category}: ${record.totalCases} cases`);
  // TypeScript knows record.totalCases is number
  // TypeScript knows record.state is string
  // TypeScript knows record.confidence is number
}

// Access metadata
console.log(`Source: ${result.meta.source.name}`);
console.log(`Confidence: ${result.meta.confidence.average}`);
```

What makes the TypeScript SDK special:

1. **Zero `any` types.** Every function parameter, every return type, every nested object is fully typed. When a developer types `result.data[0].` in VS Code, they see every available field with

its type and description. This is not just a nice-to-have. It is the difference between a developer adopting your SDK and abandoning it. Developers judge TypeScript packages by type quality.

2. **Tree-shakeable.** If a developer only uses `bd.crime`, the census module code is not included in their bundle. This matters for frontend applications where bundle size directly impacts page load time.

3. **Works everywhere.** Node.js, browser, Deno, Bun, Cloudflare Workers, Vercel Edge Functions, AWS Lambda. The SDK uses standard fetch (or a polyfill) and has no Node-specific dependencies.

4. **Custom error classes.** Not generic Error objects but specific typed errors. A developer can write:

```
try {
  const result = await bd.crime.getByState('Nonexistent');
} catch (error) {
  if (error instanceof SourceNotFoundError) {
    console.log(error.suggestion);
    // "Check available states with bd.crime.getStates()"
  }
  if (error instanceof RateLimitError) {
    console.log(`Retry after ${error.retryAfter} seconds`);
  }
}
```

Real-world TypeScript use cases:

A fintech company building a loan risk assessment dashboard queries district-level crime data for the applicant's district. Higher crime districts may have different risk profiles. The dashboard backend calls `bd.crime.getByDistrict({ state: "Uttar Pradesh", year: 2023, category: "economic-offences" })` and uses the data to compute a risk score.

A civic tech startup building a "Know Your District" app fetches crime trends for any district across multiple years using `bd.crime.compare(["Pune", "Mumbai", "Nagpur"], { years: [2021, 2022, 2023] })` and displays the comparison as charts.

A data journalism platform auto-generates state crime report cards by querying all categories for each state and rendering them as embeddable widgets.

Python SDK — For Data Analysts and Researchers

The Python SDK is designed for people who think in DataFrames. They do not want JSON objects. They want something they can immediately pass to pandas, matplotlib, scikit-learn, or Stata.

Installation:

```
# Core (dictionary responses only)
pip install bharatdata

# With pandas DataFrame support (most users want this)
pip install bharatdata[pandas]

# With all export formats
pip install bharatdata[all]
```

Basic usage in a Jupyter notebook:

```
from bharatdata import BharatData

bd = BharatData()

# Get data as a pandas DataFrame in one line
df = bd.crime.get_by_state("Maharashtra",
        year=2023).to_dataframe()

# Now it's a regular pandas DataFrame — use all pandas tools
print(df.describe())
print(df.groupby("category")["total_cases"].sum())

# Export to CSV for sharing with colleagues
bd.crime.get_by_year(2023).to_csv("india_crime_2023.csv")

# Export to Parquet for big data pipelines
bd.crime.get_all(year_range=(2021, 2023),
        level="district").to_parquet("districts.parquet")

# Generate academic citation
result = bd.crime.get_by_state("Maharashtra", year=2023)
print(result.cite("apa"))
# Output: National Crime Records Bureau. (2023). Crime in India
        2023.
#         Ministry of Home Affairs, Government of India.
#         Retrieved via BharatData (https://bharatdata.dev) on
        2025-03-20.
```

What makes the Python SDK special:

1. **Native pandas integration.** The `to_dataframe()` method returns a proper DataFrame with correct column dtypes. Numeric columns are int64 or float64, not object. String

columns are string dtype. Year is int64. This means pandas operations like `groupby`, `pivot_table`, `merge`, and `describe` work correctly without manual type conversion.

2. **Export methods.** `to_csv()`, `to_parquet()`, `to_json()`, `to_excel()`. Each includes source attribution in the output. The Parquet export preserves types and is efficient for large datasets, making it ideal for Spark or Dask workflows.

3. **Citation generation.** The `cite()` method generates properly formatted academic citations in APA, MLA, Chicago, or BibTeX format. This single feature creates enormous goodwill with researchers because it shows you understand their workflow. Every academic paper has a methodology section that must cite data sources. Making citation generation a one-line operation removes a tedious manual step.

4. **Local caching.** When `cache=True` is set on the client, query results are cached locally. A data scientist running the same Jupyter cell twenty times during iterative analysis does not hit the API twenty times. The cache respects data freshness and invalidates when upstream data changes.

5. **Optional dependencies.** A developer who just needs dictionary responses does not have to install pandas. The `[pandas]`, `[parquet]`, and `[excel]` extras are installed only when needed. This keeps the base package lightweight.

6. **Jupyter rich display.** When a `BharatDataResponse` is the last expression in a Jupyter cell, it renders as a formatted HTML table with metadata header showing record count, source, and confidence. This makes interactive exploration natural.

Real-world Python use cases:

A PhD student at JNU studying the relationship between urbanization and crime rates uses the Python SDK to pull district-level crime data, merge it with Census population data, and run regression analysis in a Jupyter notebook. The `to_dataframe()` method gives them a clean DataFrame that integrates directly with statsmodels and scikit-learn.

A data journalist at IndiaSpend building a visualization of cyber crime trends across states uses the SDK to pull three years of data, creates a matplotlib line chart, and exports the data as CSV for their graphics team. The citation method generates the source attribution for their article.

A machine learning engineer at an insurance company building a risk model pulls district-level crime data as features. The Parquet export feeds into their Spark pipeline that processes millions of policy records.

How Users Actually Use It — The Complete Journey

Discovery

A developer finds BharatData through one of these channels: - Searching npm for “india crime data” or “indian government data” - Searching PyPI for “ncrb data” or “india crime statistics” - Reading your blog post about NCRB data challenges - Seeing your Show HN post - Finding your GitHub repository through search or trending - A colleague recommending it

First Experience

They visit your README or documentation site. Within 3 seconds they understand what BharatData is. Within 30 seconds they see the install command and a code example. They copy the example, paste it into their project, and run it. Data appears. This moment, the first successful query, is the make-or-break moment. If it works, they continue. If it fails, they leave forever.

Integration

They integrate BharatData into their project. For a TypeScript developer, this means importing the client and calling functions within their API routes or data fetching layer. For a Python developer, this means adding queries to their notebook or data pipeline.

During integration, they rely on: - TypeScript autocomplete to discover available fields - Python docstrings to understand parameters - Error messages to debug incorrect queries - The `meta` object to verify data sources and confidence

Ongoing Use

Once integrated, they use BharatData regularly. When NCRB publishes new annual data and you ingest it, their next query automatically returns the updated data without any code change on their side. This is the power of an SDK over a static dataset download. The data stays fresh.

Rate Limiting — How It Works and Why It Matters

Why Rate Limiting Exists

Rate limiting serves three purposes:

1. **Infrastructure protection.** Your Supabase database has connection limits. Your Cloudflare Workers have execution limits. Without rate limiting, one developer running a script that makes 1000 requests per second could exhaust your resources and make the service unavailable for everyone.

2. **Fair usage.** A student doing a college project and a company doing bulk data extraction should not compete for the same resources without any differentiation.

3. **Monetization foundation.** Rate limits are the mechanism that creates paid tiers. The free tier has a limit. The paid tier has a higher limit. Without rate limiting infrastructure from the start, adding monetization later requires retrofitting, which is much harder.

How Rate Limiting Works Technically

Rate limiting happens at the API layer, not in the SDK. The SDK sends requests. The API decides whether to allow them.

Implementation approach using Cloudflare Workers:

```
Developer's SDK sends request
    ↓
Cloudflare Worker receives request
    ↓
Worker extracts API key from request header (or IP address for
keyless requests)
    ↓
Worker checks rate limit counter in Cloudflare KV or Workers Rate
Limiting API
    ↓
If under limit → process request → increment counter → return data
If over limit → return 429 Too Many Requests with Retry-After
header
    ↓
SDK receives 429 → throws RateLimitError with retryAfter value
    ↓
Developer handles error or SDK auto-retries after the specified
delay
```

Rate limit tiers for v1:

Tier	Requests per Minute	Requests per Day	Cost	Who It Serves
Anonymous (no API key)	10	100	Free	Playground visitors, casual browsers
Free (with API key)	30	1,000	Free (requires email signup)	Students, individual developers, journalists
The actual limits you set should be based on real usage data after launch				

For v1 launch, you only need two tiers: anonymous (very low limit) and free-with-key (generous limit). Do not build paid tiers until you have usage data proving people want higher limits.

How the SDK Handles Rate Limits

The SDK does not enforce rate limits. It handles rate limit responses gracefully.

TypeScript:

```typescript
try {
  const result = await bd.crime.getByState("Maharashtra");
} catch (error) {
  if (error instanceof RateLimitError) {
    // error.retryAfter contains seconds to wait
    console.log(`Rate limited. Retry after ${error.retryAfter}s`);
    // SDK can auto-retry if configured:
    // const bd = new BharatData({ autoRetry: true });
  }
}
```

Python:

```python
try:
    result = bd.crime.get_by_state("Maharashtra")
except RateLimitError as e:
    print(f"Rate limited. Retry after {e.retry_after}s")
```

```
    time.sleep(e.retry_after)
    result = bd.crime.get_by_state("Maharashtra")  # retry
```

API Key Management

For the free tier with API key:

1. Developer signs up on your website (email only, no payment)
2. They receive an API key (a random string like `bd_live_abc123def456`)
3. They pass the key when creating the SDK client:

```
const bd = new BharatData({ apiKey: 'bd_live_abc123def456' });

bd = BharatData(api_key="bd_live_abc123def456")
```

1. The SDK includes this key in every request header: `Authorization: Bearer bd_live_abc123def456`
2. Your API validates the key, looks up the associated rate limit tier, and enforces accordingly

For v1, you can skip API key management entirely and use IP-based rate limiting only. Add API keys when you have enough users that IP-based limiting is insufficient. Do not over-engineer auth before you have users.

Monetization Through the SDK — When the Time Comes

You are not monetizing now. You are building for community and impact. But understanding how monetization works through an SDK helps you make correct architectural decisions today that do not need to be changed later.

The Freemium Model

The most natural monetization for a developer tool is freemium: free tier is generous enough that most users never pay, paid tier serves organizations with heavier needs.

Free tier (always free, never reduced): - 1,000 API requests per day - All public datasets - All SDK functions - Community support via GitHub issues - This tier serves: students, individual developers, journalists, researchers, hobbyists - This tier is your distribution channel. These users write blog posts about you, recommend you to colleagues, and build projects that get deployed at companies.

Pro tier (when you have proven demand): - 10,000 API requests per day - Priority support - Webhook notifications when data updates - Bulk export API (download entire datasets in one call) - Price: $15-29/month - This tier serves: professional developers, small companies, active researchers - Revenue path: a developer at a fintech company hits the free tier limit because their app makes 2000 queries per day. They expense the Pro subscription on their company card.

Enterprise tier (when you have enterprise demand): - 100,000+ API requests per day - SLA guarantee (99.9% uptime) - Dedicated support channel - Custom dataset integration (they request a specific government data source, you prioritize adding it) - On-premise deployment option (they run the stack on their infrastructure) - Price: $199-499/month - This tier serves: large fintech companies, insurance companies, consulting firms - Revenue path: an insurance company uses your district-level crime data for risk modeling across 10,000 policies per day. They need reliability guarantees.

How the SDK Enables Monetization

The SDK is the distribution mechanism. When a developer installs your package, they are one function call away from becoming a user. When they become a user, they are one rate limit away from becoming a customer. The funnel is:

```
npm install bharatdata → first query → integration into app →
hit rate limit → evaluate if worth paying → upgrade to Pro
```

Every step of this funnel happens through the SDK. The SDK makes the first step frictionless (one install, one function call). The rate limit creates the natural upgrade moment. The API key distinguishes free from paid users.

What to Build Now vs Later

Build now (v1): - IP-based rate limiting at the API level (simple, no API keys needed) - Usage counting (log how many requests each IP makes per day) - The SDK passes API keys in headers when provided, but does not require them

Build when you have 100+ weekly active users: - API key registration (simple email signup) - Per-key rate limiting - Usage dashboard showing developers their consumption

Build when you have paying demand signals: - Stripe integration for paid tiers - API key tiers (free key vs pro key vs enterprise key) - Usage-based billing infrastructure

---

How You Manage SDKs in Production

Versioning

Both SDKs follow semantic versioning (semver): - **Major version (1.0.0 → 2.0.0):** Breaking changes. A function signature changed. A response field was renamed. Developers must update their code. - **Minor version (1.0.0 → 1.1.0):** New features. A new dataset module was added. A new optional parameter was added to an existing function. Developers can update without code changes. - **Patch version (1.0.0 → 1.0.1):** Bug fixes. A parsing error was fixed. A retry logic improvement. Developers should update but nothing breaks.

The critical rule: Once you publish version 1.0.0, you cannot rename `getByState` to `fetchState` or change the response shape without a major version bump. Developers depend on your API surface. Breaking it breaks their applications.

Publishing

TypeScript to npm:

```
# Build the package
npm run build

# Test locally
npm pack
# Creates bharatdata-1.0.0.tgz
# Install it in a test project to verify

# Publish
npm publish
```

Python to PyPI:

```
# Build the package
python -m build

# Test locally
pip install dist/bharatdata-1.0.0.tar.gz

# Upload to PyPI
twine upload dist/*
```

Monitoring SDK Health

After publishing, you need to know if things are working:

1. **npm download counts:** Check weekly downloads on npmjs.com/package/bharatdata
2. **PyPI download counts:** Check on pypistats.org
3. **API error rates:** Monitor what percentage of API requests return errors
4. **GitHub issues:** Watch for bug reports from SDK users
5. **Response times:** Track p50 and p95 response times for API endpoints

Updating Both SDKs Together

When you add a new dataset or feature, both SDKs must be updated simultaneously. A user who reads your announcement "BharatData now supports Census data!" and tries `bd.census.get_by_state()` in Python should not get a "module not found" error because you only updated the TypeScript SDK.

Create a checklist for every release: 1. Update TypeScript SDK source code 2. Update Python SDK source code 3. Update TypeScript SDK version in package.json 4. Update Python SDK version in pyproject.toml 5. Update CHANGELOG.md 6. Run TypeScript tests 7. Run Python tests 8. Publish TypeScript to npm 9. Publish Python to PyPI 10. Update README with new features 11. Update API documentation

---

How the Two SDKs Stay in Sync

The TypeScript and Python SDKs serve the same API. They must have the same logical capabilities even though they use different language conventions.

Naming translation:

Concept	TypeScript	Python
Function names	`getByState()`	`get_by_state()`
Parameter names	`yearRange`	`year_range`
Response keys	`totalCases`	`total_cases`
Client creation	`new BharatData()`	`BharatData()`
Module access	`bd.crime`	`bd.crime`
Error classes	`SourceNotFoundError`	`SourceNotFoundError`

Feature parity:

Every function in the TypeScript SDK has a corresponding function in the Python SDK. Every parameter available in one is available in the other. Every error type exists in both. The response shape is logically identical, just with language-appropriate naming conventions.

Python-only features: - `to_dataframe()` — pandas integration - `to_csv()`, `to_parquet()`, `to_excel()` — export methods - `cite()` — academic citation generation - `cache=True` — local caching for Jupyter notebooks - Jupyter rich display (`_repr_html_`)

These features exist only in Python because they serve the Python data science ecosystem specifically. They do not have TypeScript equivalents because TypeScript developers do not use pandas or Jupyter.

TypeScript-only features: - Full type definitions with autocomplete - Tree-shaking support for bundle optimization - Edge runtime compatibility (Cloudflare Workers, Vercel Edge)

These exist only in TypeScript because they serve the JavaScript ecosystem specifically.

The SDK as a Product — Why Every Detail Matters

An SDK is not just a code library. It is a product with its own user experience. Every aspect of the SDK is a design decision that affects adoption:

Install size matters. A developer who sees `npm install bharatdata` download 50MB of dependencies will uninstall it. Keep dependencies minimal. The TypeScript SDK should have zero runtime dependencies if possible (use built-in fetch). The Python SDK should have only `requests` as a required dependency, with pandas/pyarrow/openpyxl as optional extras.

First query time matters. The time from `npm install` to seeing real data in the console should be under 5 minutes. This includes reading the README, copying the example, and running it. If it takes longer, the developer assumes the SDK is complicated and looks for alternatives.

Error messages matter. “Error: 404” tells the developer nothing. “State ‘Maharastra’ not found. Did you mean ‘Maharashtra’? Check available states with bd.crime.getStates()” tells the developer exactly what went wrong and how to fix it.

Documentation matters. Every public function needs a docstring/JSDoc with description, parameters, return type, example, and potential errors. Developers read documentation in their IDE through hover tooltips, not on your website.

Consistency matters. If `getByState()` returns a `BharatDataResponse`, then `getByYear()` must also return a `BharatDataResponse`. If the state parameter accepts abbreviations, the district parameter should also accept abbreviations. Any inconsistency becomes a support burden.

Backward compatibility matters. Once a developer builds their application on your SDK, you cannot change the API without breaking their code. Every function signature, every parameter name, every response field is a commitment. Get it right before v1 because changing it after v1 requires a major version bump and forces every user to update their code.

This is why you wrote the SDK design specifications during Stage 2 before writing any SDK code. The specs are contracts. The code implements the contracts. The users depend on the contracts.

Summary

The SDKs are the primary way the world interacts with BharatData. They transform your database of 30,223 clean records into a product that developers install, integrate, and depend on. The TypeScript SDK serves application builders in the JavaScript ecosystem. The Python SDK serves data scientists, researchers, and ML engineers in the Python ecosystem. Together, they cover the vast majority of professionals who need Indian government data.

Rate limiting protects your infrastructure and creates the foundation for future monetization. API keys enable per-user tracking. Semantic versioning ensures developers can trust that their integrations will not break.

Build the TypeScript SDK first because TypeScript developers are the most likely early adopters from Hacker News and the Indian developer community. Build the Python SDK immediately after because data scientists and researchers are the audience most passionate about government data. Both must be published before your public launch because missing one language cuts your addressable audience in half.

Your SDK design specifications from Stage 2 define exactly what to build. Your Supabase database contains the verified data to serve. Stage 3 proved the data is accurate. Now the SDKs make that data accessible to the world.

Stage 4A Review — Detailed Analysis

Overall Assessment: APPROVED WITH 3 CONDITIONS

The core architecture is correct. You have a working monorepo, a deployed API, and two SDKs that return real data. That is genuine progress. However, three issues need attention before moving to Stage 4B. Two are fixable quickly. One requires a judgment call from you.

What Was Done Correctly

Monorepo Architecture — Correct

Using npm workspaces with Turborepo and a shared package for state/category constants is exactly the right approach. The shared package preventing data drift between API and SDKs is the architectural decision that matters most. When you add a new state name variation or crime category, updating it once in shared and having it propagate everywhere is how professional multi-package projects work.

API on Hono/Cloudflare Workers — Correct

Sub-50ms response times for metadata queries is excellent. Using the Supabase JS SDK instead of raw TCP connections is the right choice for a serverless environment where connection pooling is not available.

The pivot from single-result to array-based results was a good engineering decision. Crime data naturally has multiple records per state/year (one per category), so returning arrays is semantically correct.

TypeScript SDK with Dual Build — Correct

ESM and CJS dual output with tsup means the SDK works in both modern and legacy JavaScript environments. Exposing 10 typed interfaces gives developers full autocomplete. This is the quality bar the spec required.

Python SDK with DataFrame Integration — Correct

The `to_dataframe()` method and `cite()` helper are the Python-specific features that differentiate your SDK from a simple API wrapper. These are working as designed.

Cross-SDK Parity — Correct

Both SDKs returning identical data with language-appropriate conventions (camelCase in TypeScript, snake_case in Python) is confirmed.

Issue 1: Category Labels Showing “Unknown” — MUST FIX

What the report says

“The current crime_records_state table has many unknown category labels. Users will find the Category filter limited.”

Why this is a problem

This is a data quality issue that directly impacts every user of both SDKs. When a developer calls `bd.crime.getByState("Maharashtra", 2023)` and gets back records where `categoryLabel` is “unknown”, they cannot distinguish between IPC crimes, crimes against women, cyber crimes, or any other category. The data is there but it is unlabeled.

Your quality standards say every record must have meaningful metadata. “Unknown” category labels violate this standard. A journalist who downloads this data sees “unknown” in their CSV and immediately loses trust.

What to investigate

Run this query against your Supabase database:

```sql
SELECT category, category_label, COUNT(*)
FROM crime_records_state
GROUP BY category, category_label
ORDER BY COUNT(*) DESC;
```

This tells you exactly how many records have proper labels versus “unknown”. Then check:

1. Is the `category` field populated correctly (like “ipc-total”, “crimes-against-women”) while only `category_label` is “unknown”?
2. Or are both `category` and `category_label` showing generic/unknown values?

If the category IDs are correct but labels are missing, the fix is straightforward. Update your category mapper in the normalizer to populate labels from your categories constant file, then re-run the pipeline for affected records.

If both fields are wrong, you need to trace the issue back to the parser to understand why the table type is not being detected correctly during ingestion.

What to do

If the category IDs (like “ipc-total”) are correct and only labels are wrong:

```sql
UPDATE crime_records_state
SET category_label = 'Total IPC Crimes'
WHERE category = 'ipc-total' AND category_label = 'unknown';
```

Do this for each category using your canonical category mappings. Then update the normalizer to prevent this from happening on future pipeline runs.

If the category IDs themselves are wrong, you need to re-run the Stage 3 pipeline with fixed category mapping. This is a bigger task but it is essential. Your SDKs are only as good as the data behind them.

Time estimate: 1-3 hours depending on root cause.

Issue 2: API Endpoint Naming Does Not Match SDK Specs — MUST VERIFY

What the report says

The API endpoints listed are: - `GET /v1/states` - `GET /v1/years` - `GET /v1/categories` - `GET /v1/crime/summary`

What the SDK specs define

Your Stage 2 SDK design specification and the Stage 4A guide defined these endpoints: - `GET /v1/meta/states` - `GET /v1/meta/years` - `GET /v1/meta/categories` - `GET /v1/crime/state/:state` - `GET /v1/crime/year/:year` - `GET /v1/crime/district` - `GET /v1/crime/city/:city` - `GET /v1/crime/compare` - `GET /v1/crime/search`

The discrepancy

The metadata endpoints are at `/v1/states` instead of `/v1/meta/states`. This is a minor naming difference but it means your SDKs must match whatever the API actually serves. If the TypeScript SDK is calling `/v1/meta/states` but the API serves `/v1/states`, every metadata request fails.

More importantly, the report mentions only one crime data endpoint (`/v1/crime/summary`) instead of the six specific endpoints (state, year, district, city, compare, search). This raises the question: does the API serve all the endpoints that both SDKs expect to call?

What to verify

Test every endpoint that your TypeScript SDK calls. For each SDK function, trace which API endpoint it hits and confirm that endpoint exists and returns the expected response shape.

```
bd.crime.getByState("Maharashtra") → GET /v1/crime/state/
Maharashtra → Does this return data?
bd.crime.getByYear(2023) → GET /v1/crime/year/2023 → Does this
return data?
bd.crime.getByDistrict({state: "Maharashtra"}) → GET /v1/crime/
district?state=Maharashtra → Does this exist?
bd.crime.getByCity("Mumbai") → GET /v1/crime/city/Mumbai → Does
this exist?
bd.crime.compare(["Maharashtra", "Karnataka"]) → GET /v1/crime/
compare?entities=... → Does this exist?
bd.crime.getStates() → GET /v1/meta/states OR /v1/states → Which
one does the SDK call?
```

If the API uses `/v1/crime/summary` as a single endpoint that handles all crime queries through query parameters, that is a valid architectural choice. But it means both SDKs must construct their requests to match this pattern. The report should clarify which pattern is used and confirm both SDKs align with it.

Time estimate: 30 minutes to verify, 1-2 hours to fix if mismatched.

Issue 3: Only 7 Crime Categories — JUDGMENT CALL

What the report says

The shared package contains 7 crime categories.

Context from your data

Your Stage 3 completion report showed that your database contains records across many more category types. Your Stage 1 data inventory included 12 state-level table types and 10 district-level table types. Your Python SDK spec defined a `CrimeCategory` literal type with 18 possible values including “ipc-total”, “sll-total”, “crimes-against-women”, “crimes-against-children”, “crimes-against-sc”, “crimes-against-st”, “crimes-against-senior-citizens”, “cyber-crime”, “economic-offences”, “ndps”, “missing-persons”, “juveniles-ipc”, “juveniles-sll”, and “property-stolen-recovered”.

7 categories out of 18 designed means the SDK exposes less than half of the data your pipeline ingested.

The judgment call

This is not necessarily wrong. You may have intentionally narrowed to 7 categories that are cleanly mapped and verified, leaving the rest for a future release. That is a valid “quality over quantity” decision that aligns with your project principles.

However, you need to confirm this was intentional, not accidental. If records exist in the database for categories that the SDK does not expose, users are getting less data than is available.

What to check

```sql
SELECT DISTINCT category FROM crime_records_state ORDER BY
        category;
```

Compare this against your 7 shared categories. If the database has more categories than the shared package exposes, decide:

Option A: Update the shared package to include all categories that exist in the database. This gives users access to everything you have ingested. This is the better option if the data quality for those categories is verified.

Option B: Keep 7 categories for now and document that additional categories will be added in upcoming releases. This is acceptable if you are uncertain about data quality for the other categories.

Either way, document the decision.

What Was NOT Mentioned But Should Be Verified

Rate limiting

The Stage 4A guide specified rate limiting as a critical middleware component. The completion report does not mention rate limiting at all. Verify that your API has rate limiting implemented and working.

Test by sending 15 rapid requests to the same endpoint. After 10 requests (your anonymous limit), you should receive a 429 response. If you do not, rate limiting is not working.

Response shape consistency

The report does not show a sample API response. Verify that every API response has the exact three top-level keys: `data`, `meta`, `query`. Both SDKs depend on this structure.

Error handling end-to-end

The report mentions the TypeScript SDK has “built-in logic to parse and throw descriptive errors” but does not demonstrate error handling working end-to-end. Verify that querying a

nonexistent state through both SDKs produces a proper SourceNotFoundError with a helpful suggestion, not a generic error or crash.

Conditions for Approval

Fix these three items and Stage 4A is approved:

Condition 1: Resolve the “unknown” category labels in the database. Either fix the labels via SQL update and normalizer patch, or document which categories have this issue and exclude them from API responses until fixed.

Condition 2: Verify that every SDK function successfully calls the correct API endpoint and receives the expected response. Document the actual endpoint-to-function mapping in a brief table.

Condition 3: Confirm whether the 7 categories is an intentional quality decision or an oversight. If intentional, document it. If oversight, expand to match available data.

Each condition can be resolved in 1-3 hours. Total additional work: 3-6 hours maximum.

After these three conditions are met, you have a green flag to proceed to Stage 4B covering the playground, CI/CD pipelines, npm/PyPI publishing, and end-to-end verification.

My Honest Assessment

You built a working API and two working SDKs in a monorepo. A developer can install your package, make a function call, and get real Indian crime data back. That is the core value proposition of BharatData delivered.

The issues I flagged are refinement issues, not architectural failures. The foundation is correct. The endpoint naming can be aligned in an hour. The category labels can be fixed with SQL updates. The category count is a scope decision, not a bug.

You are close. Fix these three items, verify, and move forward.

Stage 4A Final Review

Verdict: APPROVED. Proceed to Stage 4B.

All three conditions have been resolved correctly. Here is my verification of each:

Condition 1: Category Labels — RESOLVED ✅

You identified the root cause (filename-derived categories not mapped), wrote a targeted fix script, and bulk-updated 30,047 records across both state and district tables. The fact that you traced the issue to `category_mapper.py` and fixed the source, not just the symptoms, means future pipeline runs will not reproduce the problem. This is the correct approach.

One thing to confirm before moving on: Run this query to verify zero unknown labels remain:

```sql
SELECT COUNT(*) FROM crime_records_state WHERE category_label =
        'unknown';
SELECT COUNT(*) FROM crime_records_district WHERE category_label
        = 'unknown';
```

Both must return 0. If they do, this condition is fully closed.

Condition 2: Endpoint Naming — RESOLVED ✅

Metadata endpoints now match the spec at `/v1/meta/*`. Both SDKs are updated to call these exact paths. The consolidated `/v1/crime/summary` endpoint for crime queries is a valid architectural choice since both SDKs abstract the endpoint structure away from the user.

Rate limiting is verified with the 10-request burst test showing correct 429 responses on requests 11-15. This confirms the middleware is working.

One clarification for Stage 4B documentation: When you write the API reference documentation, explicitly document that `/v1/crime/summary` is the single crime data endpoint and explain how query parameters control what data is returned. This helps any future contributor who reads the endpoint list and wonders why there is one crime endpoint instead of six.

Condition 3: Category Expansion — RESOLVED ✅

Expanding from 7 to 18 categories was the right call. You had ingested the data. Limiting the SDK to 7 categories would have meant users could not access data that was already in the database. The shared package now reflects the full scope of ingested data, and both SDKs expose all 18 categories.

What Stage 4A Now Delivers

You have a complete consumption layer:

- A monorepo with shared constants ensuring consistency
- A deployed REST API on Cloudflare Workers with rate limiting, CORS, and standardized responses
- A TypeScript SDK with full type safety and dual ESM/CJS builds
- A Python SDK with pandas DataFrame integration, export methods, and citation generation
- 18 crime categories across 36 states/UTs for 2021-2023
- 30,047 records with proper category labels and graduated confidence scores

This is a solid foundation. Users can install either SDK, make a function call, and get clean Indian crime data with source attribution and confidence scoring.

Green Flag: Proceed to Stage 4B

Stage 4B covers:

1. **Web Playground** — Browser-based query interface for non-technical users
2. **CI/CD Workflows** — Automated testing, publishing to npm/PyPI, and API deployment
3. **Publishing** — Actually putting the packages on npm and PyPI
4. **End-to-end verification** — Fresh install test from both registries against production API

You are ready. Go build it.

Stage 4B — Playground, CI/CD, Publishing & Launch

What Stage 4B Produces

When Stage 4B is complete, you will have four new things live and verified:

1. A web playground at `bharatdata.dev` where anyone can query Indian crime data without installing anything, see results in a table, and download as CSV
2. Automated CI/CD pipelines that test every push, publish SDKs to npm and PyPI on tagged releases, and deploy the API and playground automatically
3. Both SDKs actually published on npm and PyPI, installable by anyone in the world
4. End-to-end verification proving that a fresh install from the public registries works against the production API

Step 5: Build the Web Playground

What the Playground Is

The playground is a single-page web application where non-technical users interact with BharatData. A journalist visits the URL, selects a state and year, clicks a button, and sees crime data in a table. They can download it as CSV. They never install anything, never write code, never create an account.

The playground is NOT a demo of the SDK. It is a first-class product for the roughly 80 percent of BharatData's audience who will never install an npm or pip package. Journalists, researchers, students, policy analysts, and citizens use the playground directly.

How the Playground Looks

The playground has a clean, minimal design with four visual sections stacked vertically on a single page.

Section 1: Header

At the very top, the Bharat Data logo or wordmark sits on the left. The tagline "Clean, queryable Indian public data" appears below or beside it. On the right side of the header, there are two small links: one to the GitHub repository and one to the API documentation. The header has a subtle bottom border to separate it from the query area.

The background is white or very light gray. The typography is clean and professional. No decorative elements. No hero images. No animations. The design communicates “this is a serious data tool” not “this is a startup landing page.”

Section 2: Query Builder

Below the header, the query builder occupies roughly 200 pixels of vertical space. It contains four input controls arranged horizontally on desktop and stacked vertically on mobile.

The first control is a dropdown labeled “State/UT” with all 36 states and union territories listed alphabetically. There is also an “All States” option at the top that queries all states at once. The dropdown should be searchable so a user can type “Mah” and see Maharashtra immediately.

The second control is a dropdown labeled “Year” with options 2021, 2022, 2023, and “All Years.” Default selection is the most recent year (2023).

The third control is a dropdown labeled “Category” with all 18 crime categories using their human-readable labels (not the IDs). So the user sees “Total IPC Crimes” not “ipc-total”, “Crimes Against Women” not “crimes-against-women”. There is also an “All Categories” option.

The fourth control is a prominent button labeled “Query Data” in your brand’s primary color. When clicked, it sends the request to the API.

Below the controls, a small text shows the independence disclaimer: “Bharat Data is an independent open source project. Not affiliated with any government body.”

Section 3: Results Area

When a query returns data, the results appear below the query builder in a data table.

The table has sortable column headers. Clicking a column header sorts the table by that column. Clicking again reverses the sort. The default sort is by state name alphabetically.

The columns displayed depend on what data is available but the core columns are: State, Year, Category, Total Cases, Rate per Lakh, and Confidence. Additional columns like Chargesheeted, Convicted, and Acquitted appear if the data contains them.

Above the table, a summary line shows: “Showing 36 records from National Crime Records Bureau (Crime in India 2023). Average confidence: 0.94.”

Below the table or in the top-right corner of the results area, there are two buttons: “Download CSV” and “Download JSON.” The CSV download includes all columns including source attribution. The JSON download includes the complete response with data, meta, and query objects.

Numbers in the table should be formatted with Indian comma notation for readability (12,34,567) since the primary audience is Indian. However, the CSV download should contain raw numbers without formatting.

If the query returns no results, instead of an empty table, show a helpful message: “No data found for this combination. Try selecting a different state, year, or category.”

If the API returns an error, show the error message and suggestion from the API response in a red-bordered box. Do not show raw error codes or stack traces.

Section 4: Footer

The footer contains four elements. The data source attribution: “Data sourced from the National Crime Records Bureau, Ministry of Home Affairs, Government of India.” The project links: GitHub, npm, PyPI, API Documentation. The disclaimer repeated: “Bharat Data is an independent open source project.” The license: “MIT License.”

Technology Choice: Next.js on Vercel

Next.js gives you server-side rendering (good for SEO and initial load performance), API routes if needed, and one-click deployment to Vercel’s free tier. The playground is a single page so you do not need Next.js’s routing complexity, but the deployment experience and performance optimizations make it the practical choice.

If you prefer a simpler setup, you could use plain React with Vite deployed to Cloudflare Pages or Vercel. The key requirement is: it must be a static or server-rendered page that loads fast and works on mobile.

Detailed Execution

5A: Initialize the Playground Package

Navigate to `packages/playground/`. Set it up as a Next.js project. Use the App Router (the `app/` directory structure). Install your TypeScript SDK as a dependency from the workspace so the

playground uses your SDK to talk to the API. This means the playground is also a test of your SDK. If the playground works, the SDK works.

Install a minimal UI component approach. You do not need a full component library. You need a dropdown/select component, a data table component, and buttons. You can build these with plain HTML/CSS and React, or use a lightweight library like Headless UI for accessible dropdowns. Do not install Material UI or Ant Design or any heavy framework. The playground should load in under 2 seconds on a 3G connection.

5B: Build the Query Builder Component

Create a React component that renders the four controls (state dropdown, year dropdown, category dropdown, query button).

The state dropdown should be populated from your shared constants file, not from an API call. This means the dropdown renders instantly without waiting for a network request. The 36 states are static data that rarely changes.

The category dropdown should similarly come from shared constants, showing the human-readable label to the user while storing the category ID internally.

The year dropdown can also be static for now (2021, 2022, 2023). When you add new years through pipeline runs, you update this list. Alternatively, you can fetch available years from the `/v1/meta/years` endpoint when the page loads. This adds a small network request but ensures the year list is always current. Either approach is acceptable.

When the user clicks “Query Data,” the component constructs the appropriate API call using your TypeScript SDK. The SDK handles the HTTP request, error handling, and response parsing.

During the query execution, show a loading indicator. A simple “Loading…” text or a small spinner is sufficient. Do not use skeleton screens or complex loading animations.

5C: Build the Results Table Component

Create a React component that receives an array of crime records and renders them as an HTML table.

Sortable columns: When a column header is clicked, sort the data array by that column and re-render. Track the current sort column and direction (ascending/descending) in component state. Show a small arrow indicator next to the currently sorted column header.

Number formatting: Display numbers with commas for readability. Create a simple formatting function that converts 1234567 to “12,34,567” (Indian notation). Apply this to totalCases, chargesheeted, convicted, acquitted, and other numeric columns.

Null handling: When a value is null (like chargesheeted for categories where it is not reported), display a dash “—” in the cell. Do not display “null” or “undefined” or leave the cell blank.

Confidence display: Show the confidence score as a percentage with one decimal place (e.g., “94.0%”). Optionally color-code it: green for >= 0.9, yellow for 0.7-0.89, orange for 0.5-0.69. This gives users a visual sense of data reliability.

Responsive design: On mobile screens (under 768px width), the table should be horizontally scrollable. Do not try to stack table columns vertically. Data tables are inherently wide and horizontal scrolling is the accepted mobile pattern for them.

5D: Build the Download Functionality

CSV Download: When the user clicks “Download CSV,” convert the current results array to a CSV string. Include a header row with column names. Include all data columns plus source_url and confidence. Use the File API to trigger a browser download with a filename like `bharatdata_maharashtra_2023_ipc-total.csv`. The filename should be descriptive and include the query parameters.

The CSV should use UTF-8 encoding with a BOM (byte order mark) so that Excel opens it correctly with proper character encoding. Without the BOM, Hindi text or special characters in state/district names may display incorrectly in Excel.

JSON Download: When the user clicks “Download JSON,” serialize the complete API response (data, meta, query) as a formatted JSON string with 2-space indentation. Trigger a browser download with a filename like `bharatdata_maharashtra_2023_ipc-total.json`.

Both downloads should work entirely client-side. No server round-trip needed because the data is already in the browser from the query.

5E: Build the Source Attribution Display

Below the results table, show a box with source attribution information pulled from the response’s meta object.

Display: “Source: National Crime Records Bureau — Crime in India 2023” Display: “Data accessed via Bharat Data on [date]” Display: “Average confidence: 94.0%” Display: “Records shown: 36”

This attribution box serves two purposes. It gives journalists the source information they need for their articles. And it demonstrates to all users that BharatData takes source attribution seriously, which builds trust.

5F: Add the Playground README

Create a simple landing section above the query builder (or as a collapsible “About” section) that explains what the playground does in two sentences. Something like: “Query Indian government crime data instantly. Select a state, year, and category below to see verified NCRB statistics.”

This is for users who land on the page without any context about what BharatData is.

5G: Deploy to Vercel

Connect your GitHub repository to Vercel. Configure the build settings to build only the playground package. Set the root directory to `packages/playground`. Set the build command to `next build` and the output directory to `.next`.

Vercel’s free tier gives you unlimited deployments, automatic HTTPS, and global CDN. Every push to your main branch automatically deploys the latest playground.

If you have a custom domain (bharatdata.dev), configure it in Vercel’s domain settings. Otherwise, you will get a URL like `bharatdata-playground.vercel.app`.

Verification for Step 5

5V-A: Visual Verification

Open the deployed playground URL in a desktop browser. Verify these visual elements:

1. Header shows project name and tagline
2. Three dropdowns are visible and populated (states show all 36, years show 2021-2023, categories show all 18 human-readable labels)
3. Query button is visible and clickable
4. Disclaimer text is visible

5V-B: Functional Verification

Test 1: Select Maharashtra, 2023, All Categories. Click Query. Verify: a table appears with multiple rows, each showing a different crime category for Maharashtra in 2023. Verify column headers are readable. Verify numbers are formatted with commas.

Test 2: Select All States, 2023, Total IPC Crimes. Click Query. Verify: a table appears with exactly 36 rows (one per state/UT). Verify you can sort by Total Cases column. Click the column header, verify the sort works.

Test 3: Click Download CSV. Open the downloaded file in a spreadsheet application. Verify: all columns are present, numbers are correct, source_url column exists, file opens without encoding issues.

Test 4: Click Download JSON. Open the file in a text editor. Verify: it contains data, meta, and query objects. Verify meta.source.name is “National Crime Records Bureau.”

Test 5: Select a nonexistent combination that returns no results (if possible). Verify: a helpful “no data found” message appears instead of an empty table or error.

Test 6: Open the playground on a mobile phone. Verify: the query controls are usable (dropdowns work, button is tappable). Verify: the results table scrolls horizontally.

Test 7: Check page load time. Open browser developer tools, go to Network tab, hard-refresh the page. The page should load in under 3 seconds on a decent connection. The initial render (before any query) should be under 1 second because no API calls are needed.

Expected result: All 7 tests pass. The playground is live and usable.

Estimated time for Step 5: 6-8 hours

Step 6: Set Up CI/CD

What CI/CD Accomplishes

CI (Continuous Integration) runs automated tests on every push and pull request. This catches bugs before they reach the main branch. CD (Continuous Deployment) automatically publishes packages and deploys services when you create a release.

Without CI/CD, you manually run tests, manually build packages, manually publish to npm and PyPI, and manually deploy the API. Every manual step is an opportunity for mistakes. CI/CD eliminates these mistakes by automating the entire process.

The Workflows You Need

You need five GitHub Actions workflow files. Each serves a distinct purpose and triggers on different events.

Workflow 1: CI — Runs on Every Push

File: `.github/workflows/ci.yml`

Triggers: Every push to any branch. Every pull request to the main branch.

What it does: Checks out the code. Sets up Node.js and Python. Installs dependencies for both ecosystems. Builds all TypeScript packages. Runs linting on the TypeScript SDK (eslint), Python SDK (ruff), and API. Runs the TypeScript SDK test suite. Runs the Python SDK test suite (pytest). Runs the API test suite.

The critical check: After all tests pass, run a TypeScript strict compilation check and grep for any `any` types in the SDK's public API surface. If found, the workflow fails. This enforces your zero-any rule automatically on every commit.

Why this matters: Every contributor (including future you after a long break) gets immediate feedback on whether their changes break anything. A PR that introduces a bug is caught before it merges.

Workflow 2: Publish TypeScript SDK to npm

File: `.github/workflows/release-typescript-sdk.yml`

Triggers: When you push a git tag matching the pattern `ts-v*` (like `ts-v1.0.0`, `ts-v1.1.0`).

What it does: Checks out the code. Installs dependencies. Builds the TypeScript SDK. Runs the full test suite. If tests pass, publishes to npm using `npm publish`.

How to trigger a release:

```
cd packages/typescript-sdk
# Update version in package.json to 1.0.0
git add .
git commit -m "release: typescript sdk v1.0.0"
```

```
git tag ts-v1.0.0
git push origin main --tags
```

The tag push triggers the workflow which builds, tests, and publishes.

Security: The npm auth token is stored as a GitHub secret (`NPM_TOKEN`), never in code. The workflow accesses it through `${{ secrets.NPM_TOKEN }}`.

Workflow 3: Publish Python SDK to PyPI

File: `.github/workflows/release-python-sdk.yml`

Triggers: When you push a git tag matching `py-v*` (like `py-v1.0.0`).

What it does: Checks out the code. Sets up Python 3.11. Installs build tools (build, twine). Installs the SDK with dev dependencies. Runs pytest. If tests pass, builds the source distribution and wheel. Uploads to PyPI using twine.

How to trigger:

```
cd packages/python-sdk
# Update version in pyproject.toml to 1.0.0
git add .
git commit -m "release: python sdk v1.0.0"
git tag py-v1.0.0
git push origin main --tags
```

Security: The PyPI auth token is stored as `PYPI_TOKEN` in GitHub secrets.

Workflow 4: Deploy API

File: `.github/workflows/deploy-api.yml`

Triggers: When code changes in the `packages/api/` directory are pushed to the main branch.

What it does: Checks out the code. Installs dependencies. Deploys to Cloudflare Workers using `wrangler deploy`.

Path filtering: This workflow only runs when API code changes. Pushing a README update does not trigger an API deployment. This is configured with the `paths` filter in the workflow trigger.

Security: Cloudflare API token stored as `CLOUDFLARE_API_TOKEN` in GitHub secrets.

Workflow 5: Deploy Playground

File: `.github/workflows/deploy-playground.yml`

This workflow is optional if you use Vercel's GitHub integration, because Vercel automatically deploys on every push. If you prefer explicit control, create a workflow that triggers on changes to `packages/playground/` and deploys via the Vercel CLI.

For most projects at your stage, letting Vercel handle playground deployment automatically is the simpler and correct choice. You only need an explicit workflow if you want to add pre-deployment checks (like running playground-specific tests before deploying).

Setting Up GitHub Secrets

Before any workflow can publish or deploy, you must add secrets to your GitHub repository.

Go to your GitHub repository. Click Settings. Click Secrets and variables. Click Actions. Click New repository secret. Add each secret:

Secret Name	Where to Get It
`NPM_TOKEN`	Go to npmjs.com. Log in. Go to Access Tokens. Click Generate New Token. Select Automation type. Copy the token.
`PYPI_TOKEN`	Go to pypi.org. Log in. Go to Account Settings. Scroll to API tokens. Click Add API token. Give it a name like "github-actions". Scope it to the bharatdata project (after first manual publish). Copy the token.
`CLOUDFLARE_API_TOKEN`	Go to Cloudflare Dashboard. Click My Profile. Click API Tokens. Click Create Token. Use the "Edit Cloudflare Workers" template. Copy the token.
`DATABASE_URL`	Your Supabase direct connection string. Already added from Stage 3.

Verification for Step 6

6V-A: CI Workflow Verification

Push a commit to your main branch. Go to the Actions tab in your GitHub repository. You should see the CI workflow running. Wait for it to complete. It should show a green checkmark with all steps passed.

Then intentionally break something. Create a branch, introduce a type error in the TypeScript SDK (like assigning a string to a number variable), commit and push. The CI should fail with a clear error message pointing to the type error. This proves the CI catches real issues.

Fix the error and delete the branch.

6V-B: Deployment Workflow Verification

For the API deployment workflow, push a small change to `packages/api/` (like adding a comment to the health endpoint). The deploy workflow should trigger and complete successfully. Verify the change is live by hitting the health endpoint.

For the playground, if using Vercel auto-deploy, push a small change to `packages/playground/` (like updating the footer text). Verify the change appears on the live playground URL within a few minutes.

6V-C: Publish Workflow Dry Run

You do not want to accidentally publish version 0.0.1 to npm while testing. Instead, verify the workflow file syntax by:

1. Creating the workflow files and pushing them to main
2. Checking the Actions tab to confirm they appear in the workflow list
3. Verifying they only trigger on tag pushes (they should not run on your regular commits)

The actual publish will happen in Step 7.

Expected result: CI runs on every push and catches errors. Deploy workflows trigger on path-specific changes. Publish workflows are configured and waiting for tags.

Estimated time for Step 6: 3-4 hours

Step 7: Publish and Deploy

First-Time npm Publishing

Before the GitHub Actions workflow can publish, you need to do the first publish manually. This is because npm requires you to verify your account and package name ownership on the first publish.

Navigate to `packages/typescript-sdk/`. Make sure the version in package.json is `0.1.0`. Make sure the package name is `bharatdata` (or your chosen available name).

Log in to npm from the command line:

```
npm login
```

Enter your npm username, password, and email. Then publish:

```
npm publish --access public
```

The `--access public` flag is required for unscoped packages or for the first publish of a scoped package.

After successful publish, go to `npmjs.com/package/bharatdata` and verify your package page exists. It should show your README, version 0.1.0, and the install command.

First-Time PyPI Publishing

Similarly, the first PyPI publish should be done manually.

Navigate to `packages/python-sdk/`. Build the package:

```
python -m build
```

This creates files in `dist/`. Upload to PyPI:

```
twine upload dist/*
```

Enter your PyPI username (use `__token__` as the username) and paste your API token as the password.

After successful upload, go to `pypi.org/project/bharatdata/` and verify your package page exists.

Post-Publish Tag Setup

Now that both packages are published manually for the first time, future publishes will go through GitHub Actions.

Tag the current state:

```
git tag ts-v0.1.0
git tag py-v0.1.0
git push origin --tags
```

This does not trigger a new publish (the code is already published) but it establishes the tagging convention for future releases.

Verification for Step 7

7V-A: npm Installation Test

Open a completely new terminal. Create a temporary directory. Initialize a new Node.js project. Install your published package:

```
mkdir /tmp/test-bharatdata-npm
cd /tmp/test-bharatdata-npm
npm init -y
npm install bharatdata
```

Verify: The package installs without errors. Check `node_modules/bharatdata/` exists.

Create a test file and run it:

```
const { BharatData } = require('bharatdata');

async function test() {
  const bd = new BharatData();
  const result = await bd.crime.getByState('Maharashtra', 2023);
  console.log('Records:', result.data.length);
  console.log('Source:', result.meta.source.name);
  console.log('Confidence:', result.meta.confidence.average);
  console.log('First state:', result.data[0].state);
}

test().catch(console.error);
```

Run with `node test.js`. It must print real data from your production API.

7V-B: PyPI Installation Test

Open a completely new terminal. Create a fresh virtual environment:

```
python -m venv /tmp/test-bharatdata-pypi
source /tmp/test-bharatdata-pypi/bin/activate
pip install bharatdata[pandas]
```

Verify: The package installs without errors. Run a test:

```
from bharatdata import BharatData

bd = BharatData()
result = bd.crime.get_by_state("Maharashtra", year=2023)
print(f"Records: {len(result.data)}")

df = result.to_dataframe()
print(f"DataFrame shape: {df.shape}")
print(df.head())

print(result.cite("apa"))
```

Run with `python test.py`. It must print real data, a valid DataFrame, and a proper citation.

7V-C: Playground Live Test

Open the playground URL in a browser. Query Maharashtra, 2023, Total IPC Crimes. Verify data appears. Download CSV. Open in a spreadsheet. Verify data is correct.

Expected result: All three installation/usage tests pass with real data from the production API.

Estimated time for Step 7: 2-3 hours

Step 8: End-to-End Verification

What This Proves

End-to-end verification proves that the entire BharatData system works as an integrated whole. A user discovers the project, installs the SDK, queries data, and gets accurate results. This is the final quality gate before you announce BharatData publicly.

The Verification Protocol

8A: The Developer Journey Test

Pretend you are a developer who has never heard of BharatData. You found it on npm.

1. Go to npmjs.com and search for “bharatdata”
2. Read the package README on the npm page
3. Follow the installation instructions exactly as written in the README

4. Copy the first code example from the README
5. Run it
6. Verify it works on the first try without any modification

If any step fails or requires guesswork, the README needs improvement before public launch.

Repeat this test for the Python SDK:

1. Go to pypi.org and search for “bharatdata”
2. Read the package description
3. Install with the command shown
4. Run the example code
5. Verify it works

8B: The Journalist Journey Test

Pretend you are a journalist who needs crime data for an article.

1. Go to the playground URL
2. Without reading any documentation, try to query crime data for Delhi in 2023
3. Verify the interface is intuitive enough that you can figure it out in under 30 seconds
4. Download the data as CSV
5. Open in Excel or Google Sheets
6. Verify the numbers are readable and the source attribution is included
7. Check if you can figure out where the data originally came from (the attribution should point to NCRB)

If any step is confusing, the playground UX needs improvement.

8C: The Data Accuracy Test

Pick 5 specific data points from the playground results. For each one:

1. Note the state, year, category, and total_cases from the playground
2. Go to the NCRB website and find the original source file
3. Open the source file and find the matching row
4. Verify the number matches exactly

This is the ultimate quality test. If BharatData shows Maharashtra IPC Total 2023 as 545,862 but the NCRB source file shows 545,863, you have a pipeline bug that needs to be traced and fixed.

8D: The Error Handling Test

Test that errors are handled gracefully across all three interfaces:

TypeScript SDK: Query a nonexistent state. Verify you get a SourceNotFoundError with a helpful suggestion, not a crash.

Python SDK: Query a nonexistent state. Verify you get a SourceNotFoundError with a helpful suggestion, not a traceback.

Playground: If possible, trigger an error condition (like selecting a combination with no data). Verify the playground shows a user-friendly message, not a raw error or blank screen.

8E: The Cross-SDK Consistency Test

Query the same data through all three interfaces and verify identical results:

1. TypeScript SDK: `bd.crime.getByState("Maharashtra", 2023)` — note the total_cases for IPC total
2. Python SDK: `bd.crime.get_by_state("Maharashtra", year=2023)` — note the total_cases for IPC total
3. Playground: Select Maharashtra, 2023, Total IPC Crimes — note the total_cases

All three must show the exact same number. If they differ, there is a data transformation bug in one of the SDKs or the API.

Verification Report

Create a file `docs/stages-execution-docs/stage-4/STAGE_4B_VERIFICATION.md` documenting the results of all five verification protocols. For each test, record: what you tested, what you expected, what you got, and whether it passed.

Expected result: All five verification protocols pass completely.

Estimated time for Step 8: 2-3 hours

Stage 4B Completion Criteria

Stage 4B is complete when all of the following are true:

1. The web playground is deployed and accessible at a public URL
2. The playground correctly queries and displays data for any state/year/category combination
3. CSV and JSON downloads work from the playground

4. The CI workflow runs on every push and catches real errors
5. The TypeScript SDK is published on npm and installable with `npm install bharatdata`
6. The Python SDK is published on PyPI and installable with `pip install bharatdata`
7. Both published SDKs return real data from the production API
8. The 5-point data accuracy check passes (playground values match NCRB source files)
9. Cross-SDK consistency is verified (same query returns same numbers across all three interfaces)
10. Error handling works gracefully across all three interfaces

---

Stage 4B Build Timeline

Step	Component	Time
5	Web Playground	6-8 hours
6	CI/CD Workflows	3-4 hours
7	Publishing to npm and PyPI	2-3 hours
8	End-to-End Verification	2-3 hours
Total		**13-18 hours**

Universal Data Foundation — Source-Agnostic Architecture

The Fundamental Shift in Thinking

Your current pipeline is hardcoded for NCRB. The Excel parser knows NCRB column names. The normalizer knows NCRB state name positions. The field mapper has NCRB-specific mappings. When you add Census data or CPCB air quality data, you would have to write new parsers, new mappers, and new validators for each one. At 10 data sources this is tedious. At 100 it is impossible. At 1000 it is absurd.

The solution is not building a better NCRB pipeline. The solution is building a pipeline that does not know what NCRB is. A pipeline that receives a source definition file describing where the data lives, what format it is in, what the columns mean, and what the validation rules are. The pipeline reads this definition and executes it. Adding a new data source means writing a new definition file, not new code.

This is the difference between a calculator and a spreadsheet. A calculator computes one formula at a time. A spreadsheet computes any formula you put in any cell. You are currently building calculators. You need to build a spreadsheet.

The Universal Architecture

Three Layers

```
Layer 1: SOURCE REGISTRY
    A catalog of JSON/YAML definition files.
    Each file describes ONE government data source completely:
    - Where to find it (URL, download method)
    - What format it is (Excel, CSV, PDF, HTML, API)
    - What the data looks like (column mappings, header rows)
    - How to validate it (expected row counts, total matching
rules)
    - How to normalize it (geographic columns, date columns,
numeric columns)

Layer 2: UNIVERSAL ENGINE
    A single pipeline engine that reads any source definition
    and executes the complete workflow:
    Discover → Download → Extract → Validate → Normalize → Verify
→ Load

    The engine has NO knowledge of any specific data source.
    It only understands the definition schema.

Layer 3: VERIFICATION FRAMEWORK
    Automated proof that extracted data matches the source.
    Works identically regardless of source type.
    Produces audit reports for every ingestion.
```

How It Works — Conceptual Flow

When you want to add Census demographic data, you do not write a Census parser. You write a Census source definition file that says:

“The data lives at this URL. It is an Excel file. The header row is row 3. Column A is the state name. Column B is population. Column C is literacy rate. The total row contains the text ‘All India’. The expected number of state rows is 36. The state name column should be normalized against the canonical state mappings.”

The universal engine reads this definition, downloads the file, finds row 3 as the header, maps Column A to state, Column B to population, Column C to literacy rate, finds the total row, validates that parsed rows sum to the total, normalizes state names, and loads the data. You wrote zero parsing code. You wrote a description of the data.

When you want to add CPCB air quality data from a JavaScript-rendered website, you write a different source definition that says:

“The data lives at this URL. It requires a headless browser to render. After rendering, look for an HTML table with class ‘data-table’. The header row is the first row of the table. Column 1 is city name. Column 2 is PM2.5. Column 3 is PM10. There is no total row. Expected row count is between 100 and 400. The city name column should be normalized against the canonical city mappings.”

Same engine. Different definition. Different data source. Zero new code.

The Source Definition Schema

This is the heart of the universal system. Every data source is described by a definition file that follows a standard schema. The schema must be expressive enough to describe any government data source while being simple enough that adding a new source takes 30 minutes, not 3 days.

Definition File Structure

Each source definition is a YAML or JSON file with these sections:

Identity Section. What is this data source? A unique identifier, human-readable name, publishing body, description, update frequency, geographic coverage, temporal coverage, and the license or legal status.

Acquisition Section. How do you get the raw data? This section describes the download method. It supports multiple strategies:

- **Direct download.** A URL that returns a file when you GET it. You specify the URL pattern (which may include year or date parameters), the expected file format, and any required headers or cookies.

- **Page scraping.** A URL that contains links to downloadable files. You specify the page URL, a CSS selector or regex pattern that identifies download links on the page, and the expected file format of the downloaded files.

- **JavaScript rendering.** A URL that requires a browser to render the content. You specify the URL, the wait condition (wait for a specific element to appear, wait for network idle, or wait a fixed number of seconds), and then either extract a table from the rendered page or find download links in the rendered content.

- **API call.** A structured API endpoint. You specify the base URL, the endpoint path, query parameters, pagination strategy (offset-based, cursor-based, or page-based), and the response format.

- **Manual upload.** For data sources that cannot be automatically downloaded (behind login walls, requires RTI requests). You specify a local file path pattern where manually downloaded files are placed, and the system picks them up from there.

Extraction Section. How do you turn the raw file into structured rows? This section describes the parsing strategy:

- **File format.** Excel (xls, xlsx), CSV, PDF, HTML table, JSON, or XML.

- **Header detection.** Either a fixed row number ("header is row 3") or a pattern-based detection ("header is the first row where cell A contains 'State' and cell B contains a word ending in 'Cases'"). Pattern-based detection handles files where the header position varies between years.

- **Column mapping.** A dictionary mapping raw column names (or column positions) to canonical field names. Supports regex patterns for column name matching so that "Total Cognizable IPC Crimes" and "Total IPC Crimes" and "Total Cognizable Crimes (IPC)" all map to the same field. Each mapping specifies the target field name and the expected data type (string, integer, float, date, percentage).

- **Row filtering.** Rules for identifying and excluding non-data rows. Summary rows (matching patterns like "Total", "Grand Total", "All India"), header continuation rows, footnote rows, and blank rows. Each filter specifies a column to check and a pattern to match.

- **Sheet selection.** For Excel files with multiple sheets, which sheet(s) to process. Can be by name, by index, or "all sheets."

- **Table detection.** For PDF files, how to identify tables on a page. For HTML files, a CSS selector for the data table. For multi-table files, which table to extract (by index, by header content, or by position on page).

Normalization Section. How do you standardize the extracted data? This section describes transformations applied after extraction:

- **Geographic normalization.** Which column contains the geographic entity (state, district, city). Which canonical mapping file to use for resolution. What to do with unresolved entities (reduce confidence, flag for review, or reject).

- **Temporal normalization.** Which column contains the time reference. What format it is in (calendar year, fiscal year, date, month-year). How to convert it to the canonical format.

- **Numeric normalization.** How numbers are formatted in the source (Indian comma notation, international notation, or mixed). Whether percentages include the % symbol. Whether values in lakhs or crores need multiplication.

- **Null handling.** What values represent missing data in this specific source. A mapping like: "-" means zero for count fields but null for rate fields. "NA" means null everywhere. Empty cells mean null.

- **Category assignment.** What crime/data category this file represents. Either a fixed category (the entire file is IPC crime data) or derived from a column in the file.

Validation Section. How do you prove the data is correct? This section describes verification rules:

- **Row count expectations.** Expected range of data rows. For state-level files, 30-40 rows (36 states plus possible variations). For district-level files, 700-800 rows. For city-level files, 30-40 rows.

- **Total row matching.** How to find the total/summary row in the source file. Which columns to sum and compare against the total. The acceptable tolerance (usually zero for integer columns, 0.1 for float columns due to rounding).

- **Cross-reference rules.** After loading, how to verify against other datasets. For example: "Sum of district-level total_cases for each state should match the corresponding state-level total_cases."

- **Value range rules.** Expected ranges for specific fields. Population should be positive. Crime rates should be between 0 and 10000. Percentages should be between 0 and 100.

- **Consistency rules.** Logical relationships between fields. Chargesheeted should be less than or equal to total_cases. Convicted plus acquitted should be less than or equal to chargesheeted.

The Universal Engine

The engine is a set of Python modules that read source definitions and execute them. Each module handles one stage of the pipeline and is completely source-agnostic.

Universal Downloader

The downloader reads the acquisition section of a source definition and obtains the raw files. It has four download strategies built in:

Direct download strategy. Uses `requests` to GET a URL. Handles redirects, retries with exponential backoff, rate limiting per host (configurable in the source definition, default 1 request per second), and SHA-256 hash calculation for change detection.

Page scraping strategy. Uses `requests` plus `BeautifulSoup` to load a page, find download links using CSS selectors or regex patterns from the definition, and download each linked file using the direct download strategy.

Browser rendering strategy. Uses `Playwright` (installed only when needed, not a core dependency) to load a page in a headless browser, wait for the specified condition, and then either extract a table from the rendered DOM or find download links in the rendered page. This handles JavaScript-heavy government portals like CPCB.

API strategy. Uses `requests` to call API endpoints with specified parameters and pagination. Handles response parsing for JSON and XML formats.

The downloader does not know what data it is downloading. It only knows how to follow the instructions in the acquisition section. If a source definition says “use browser rendering, wait for element with ID ‘data-table’, then extract the table,” the downloader does exactly that.

Universal Extractor

The extractor reads the extraction section and converts raw files into structured rows. It has parsers for each file format:

Excel extractor. Uses `openpyxl` for xlsx and `xlrd` for xls. Reads the header detection rules from the definition to find the header row. Applies the column mapping to translate raw headers to canonical field names. Filters out non-data rows using the row filtering rules. Extracts the total/summary row separately using the summary detection rules.

CSV extractor. Uses Python's `csv` module or `pandas`. Handles different delimiters (comma, tab, pipe) as specified in the definition. Same header detection, column mapping, and row filtering as Excel.

PDF extractor. Uses `pdfplumber`. Applies table detection rules from the definition to find tables on each page. Handles multi-page tables by detecting continuation patterns (no header row on subsequent pages). Same column mapping and row filtering.

HTML extractor. Uses `BeautifulSoup`. Applies the CSS selector from the definition to find the data table in the HTML. Parses table rows and cells. Same column mapping and row filtering.

JSON/XML extractor. Uses Python's json module or `lxml`. Navigates the document structure using a path expression specified in the definition (like JSONPath or XPath).

Every extractor produces the same output format: a list of dictionaries with canonical field names and a separately extracted total row (if the validation section defines one). The downstream stages do not know or care which extractor produced the data.

Universal Normalizer

The normalizer reads the normalization section and applies transformations. It is a sequence of pluggable transformation steps:

Geographic resolver. Takes the raw value from the geographic column, looks it up in the specified canonical mapping file, and resolves it to the canonical form. Works identically whether the column contains Indian state names, district names, city names, or any other geographic entity. The mapping file is specified in the source definition, so different sources can use different mapping files (state mappings for NCRB, city mappings for CPCB, district mappings for Census).

Type enforcer. Converts string values to the types specified in the column mapping. Handles Indian number formatting (detects comma position patterns), percentage symbols, lakh/crore textual values, and various date formats. The conversion rules are specified per column in the definition.

Null handler. Converts source-specific null representations to Python None based on the null handling rules in the definition. Different sources use different null representations. NCRB uses dashes. Census might use “N.A.” RBI might use three dots “…”. The definition specifies what each source uses.

Confidence scorer. Calculates a confidence score based on how clean the extraction was. Base confidence varies by file format (1.0 for clean Excel, 0.9 for PDF, 0.8 for HTML scraping). Deductions are applied for format conversions, fuzzy geographic matching, and null imputation. The base confidence and deduction rules can be overridden in the source definition.

Metadata attacher. Adds source attribution fields to every record: source URL, source file name, publishing body name, report name, collection timestamp, and normalizer version. All of these come from the source definition’s identity section.

Universal Validator

The validator reads the validation section and proves the data is correct before loading.

Row count validator. Checks that the number of parsed data rows falls within the expected range from the definition.

Total matcher. Sums the parsed numeric columns and compares against the extracted total row. If the definition specifies a total matching rule and the sums do not match within tolerance, the validation fails.

Schema validator. Uses Great Expectations to verify column types, value ranges, null rates, uniqueness constraints, and categorical value sets. The expectations are generated automatically from the source definition’s column mappings and validation rules. You do not write GE expectation suites manually for each source. The engine generates them from the definition.

Cross-reference validator. After loading, runs cross-reference queries specified in the definition. These queries compare data across geographic levels or across related sources.

Universal Loader

The loader writes validated data to the database. It uses a dynamic table strategy:

Table naming convention. Each dataset gets its own table named according to a pattern: `{source}_{geographic_level}_{topic}`. For example: `ncrb_state_crime`, `census_district_demographics`, `cpcb_city_air_quality`. The table is created automatically from the column mappings in the definition if it does not exist.

Schema generation. The loader reads the column mappings and creates a PostgreSQL table with appropriate column types. It adds standard metadata columns (id, ingestion_run_id, confidence, source_url, source_file, collection_date) that are present in every table regardless of the data source.

Deduplication. The loader uses UNIQUE constraints on a combination of columns specified in the definition (typically geographic entity + time period + category + source file). Uses ON CONFLICT DO NOTHING for idempotent loading.

Audit trail. Every load operation is recorded in the `ingestion_runs` table with the source definition ID, record counts, validation results, and any discrepancies found.

How Adding a New Data Source Works

Example: Adding Census Demographic Data

You do not write any Python code. You create a file called `sources/census_demographics.yaml`:

```yaml
identity:
  id: "census-demographics-2011"
  name: "Census of India 2011 — Primary Census Abstract"
  publishing_body: "Office of the Registrar General and Census
        Commissioner"
  description: "Population, literacy, and demographic data by
        state and district"
  update_frequency: "decennial"
  geographic_coverage: "state and district"
  temporal_coverage: "2011"
  legal_status: "public government publication"

acquisition:
  method: "direct_download"
  urls:
    - pattern: "https://censusindia.gov.in/2011census/PCA/
        pca_highlights/{state_code}.xlsx"
      parameters:
        state_code: ["01", "02", "03"]  # List of state codes to
        iterate
  rate_limit: 1  # seconds between requests
  file_format: "xlsx"
```

```yaml
extraction:
  header_detection:
    method: "fixed_row"
    row: 5
  column_mapping:
    "State / UT":
      field: "state"
      type: "string"
      role: "geographic_entity"
    "Total Population":
      field: "total_population"
      type: "integer"
    "Male Population":
      field: "male_population"
      type: "integer"
    "Female Population":
      field: "female_population"
      type: "integer"
    "Literacy Rate (%)":
      field: "literacy_rate"
      type: "percentage"
  row_filters:
    summary_patterns: ["total", "india", "grand total"]
  total_row:
    identifier: "India"
    columns_to_sum: ["total_population", "male_population",
        "female_population"]

normalization:
  geographic:
    column: "state"
    mapping_file: "data/canonical-mappings/states.json"
    on_unresolved: "flag_and_continue"
  numeric:
    format: "indian_comma"
    percentage_symbol: true
  null_values: ["-", "NA", "N.A.", ""]
  null_handling:
    total_population: "reject"  # Population must never be null
    literacy_rate: "null"       # Literacy rate can be null

validation:
  row_count:
    min: 30
    max: 40
  total_matching:
    tolerance: 0
  value_ranges:
    total_population: [0, 200000000]  # No state has > 200M people
    literacy_rate: [0, 100]
  consistency_rules:
```

```yaml
    - "male_population + female_population == total_population"

storage:
  table_name: "census_state_demographics"
  unique_key: ["state", "year", "source_file"]
  geographic_level: "state"
```

You run: `python -m pipeline.run_pipeline --source sources/census_demographics.yaml`

The engine reads this definition, downloads the files, extracts data using the Excel extractor with header row 5, maps columns, filters summary rows, validates totals, normalizes state names, enforces types, and loads to a dynamically created table. Zero custom code written.

Example: Adding CPCB Air Quality Data (JavaScript Website)

```yaml
identity:
  id: "cpcb-air-quality-daily"
  name: "CPCB National Air Quality Index"
  publishing_body: "Central Pollution Control Board"
  update_frequency: "daily"

acquisition:
  method: "browser_rendering"
  url: "https://app.cpcbccr.com/AQI_India/"
  wait_condition:
    type: "element_visible"
    selector: "#tblAQI"
  extract_from: "rendered_table"
  table_selector: "#tblAQI"
  rate_limit: 5  # Be extra respectful with JS-heavy sites

extraction:
  header_detection:
    method: "first_row"
  column_mapping:
    "City":
      field: "city"
      type: "string"
      role: "geographic_entity"
    "AQI":
      field: "aqi_value"
      type: "integer"
    "PM2.5":
      field: "pm25"
      type: "float"
    "PM10":
      field: "pm10"
```

```yaml
      type: "float"

normalization:
  geographic:
    column: "city"
    mapping_file: "data/canonical-mappings/cities.json"
    on_unresolved: "flag_and_continue"
  null_values: ["-", "NA", "--"]

validation:
  value_ranges:
    aqi_value: [0, 999]
    pm25: [0, 999]
    pm10: [0, 2000]

storage:
  table_name: "cpcb_city_air_quality"
  unique_key: ["city", "date", "source_file"]
  geographic_level: "city"
```

Same engine. Different definition. Completely different data source, file format, and download method. Zero new Python code.

The Source Registry

All source definitions live in a `sources/` directory in your repository. This directory is your data catalog. Each YAML file is a complete specification of one data source.

```
sources/
├── ncrb/
│   ├── crime_in_india_state.yaml
│   ├── crime_in_india_district.yaml
│   ├── crime_in_india_metro_city.yaml
│   ├── prison_statistics.yaml
│   └── accidental_deaths.yaml
├── census/
│   ├── demographics_state.yaml
│   ├── demographics_district.yaml
│   └── housing_conditions.yaml
├── cpcb/
│   ├── air_quality_daily.yaml
│   └── air_quality_annual.yaml
├── rbi/
│   ├── monetary_indicators.yaml
│   ├── banking_statistics.yaml
│   └── exchange_rates.yaml
├── agmarknet/
│   └── mandi_prices.yaml
└── _schema/
```

```
    └── source_definition_schema.yaml  # Schema that validates all
definitions
```

When you want to know what data BharatData covers, you list the files in this directory. When you want to add a new source, you create a new YAML file. When you want to update how a source is parsed (because the government changed the file format), you edit the YAML file. No Python code changes.

Updated Pipeline Architecture

Folder Structure

```
pipeline/
├── engine/                            # Universal engine (source-
agnostic)
│   ├── __init__.py
│   ├── definition_loader.py           # Reads and validates
source definitions
│   ├── downloaders/
│   │   ├── __init__.py
│   │   ├── base.py                    # Abstract downloader
interface
│   │   ├── direct.py                  # HTTP GET download
│   │   ├── page_scraper.py            # HTML page with download
links
│   │   ├── browser_renderer.py        # JavaScript-rendered pages
(Playwright)
│   │   └── api_client.py              # REST API consumption
│   ├── extractors/
│   │   ├── __init__.py
│   │   ├── base.py                    # Abstract extractor
interface
│   │   ├── excel_extractor.py         # Excel files (xlsx, xls)
│   │   ├── csv_extractor.py           # CSV files
│   │   ├── pdf_extractor.py           # PDF tables (pdfplumber)
│   │   ├── html_extractor.py          # HTML tables
(BeautifulSoup)
│   │   └── json_extractor.py          # JSON/XML responses
│   ├── normalizers/
│   │   ├── __init__.py
│   │   ├── geographic_resolver.py     # State/district/city name
resolution
│   │   ├── type_enforcer.py           # Data type conversion
│   │   ├── null_handler.py            # Null value
standardization
│   │   ├── confidence_scorer.py       # Confidence calculation
│   │   └── metadata_attacher.py       # Source attribution
│   ├── validators/
```

```
│   │   ├── __init__.py
│   │   ├── row_count_validator.py     # Expected row count check
│   │   ├── total_matcher.py           # Parsed sum vs source
total
│   │   ├── schema_validator.py        # Great Expectations
integration
│   │   ├── cross_reference.py         # Cross-level validation
│   │   └── consistency_checker.py     # Logical relationship
checks
│   ├── loaders/
│   │   ├── __init__.py
│   │   ├── dynamic_table_creator.py   # Creates tables from
definitions
│   │   ├── batch_loader.py            # High-performance batch
inserts
│   │   └── deduplicator.py            # ON CONFLICT handling
│   └── orchestrator.py                # Reads definition, runs
full pipeline
│
├── sources/                           # Source definition files
│   ├── ncrb/
│   ├── census/
│   ├── cpcb/
│   └── _schema/
│
├── canonical-mappings/                # Geographic reference
files
│   ├── states.json
│   ├── districts.json
│   ├── cities.json
│   └── countries.json                 # For trade data etc.
│
├── monitoring/
│   ├── source_watcher.py             # Detects changes across all
sources
│   ├── quality_dashboard.py          # Aggregate quality metrics
│   └── discrepancy_tracker.py        # Known data issues database
│
├── great_expectations/               # GE configuration
│   ├── great_expectations.yml
│   └── expectation_generator.py      # Auto-generates suites from
definitions
│
├── run_pipeline.py                    # CLI: python -m
pipeline.run_pipeline --source X
├── run_all.py                        # CLI: runs all registered
sources
└── config.py                         # Database connection,
global settings
```

How the Orchestrator Works

The orchestrator is the single entry point. It takes a source definition file path and executes the complete pipeline:

```
Orchestrator receives: sources/ncrb/crime_in_india_state.yaml

Step 1: Load and validate the definition file against the schema
Step 2: Select the appropriate downloader based on
acquisition.method
Step 3: Download raw files, track hashes for change detection
Step 4: Select the appropriate extractor based on
extraction.file_format
Step 5: Extract structured rows + total row using definition rules
Step 6: Run pre-normalization validation (row counts, types)
Step 7: Run normalizer chain (geographic, type, null, confidence,
metadata)
Step 8: Run post-normalization validation (totals, ranges,
consistency)
Step 9: Create or verify target table exists in database
Step 10: Load validated records with deduplication
Step 11: Run cross-reference validation if defined
Step 12: Generate verification report
Step 13: Update source registry with ingestion results
```

Every step logs its actions. If any validation step fails, the pipeline stops and generates a discrepancy report. No data reaches the production database without passing all defined validations.

Migration Strategy: From Current Pipeline to Universal Engine

You do not throw away your current pipeline. You migrate it in three phases.

Migration Phase 1: Write the NCRB Source Definition

Convert your current hardcoded NCRB pipeline logic into a source definition YAML file. Your existing `FIELD_MAPPINGS.md`, `states.json`, `category_mapper.py`, and `summary_row_detector.py` all contain knowledge that translates directly into definition file sections.

This is a documentation exercise, not a coding exercise. You are extracting the implicit knowledge in your Python code into an explicit declarative definition.

Migration Phase 2: Build the Universal Engine

Build the engine components one at a time, testing each against your NCRB definition. The Excel extractor should produce identical output to your current `excel_parser.py` for the same input files. The geographic resolver should produce identical results to your current `state_resolver.py`. Verify by running both the old and new pipeline against the same files and comparing outputs.

Migration Phase 3: Add a Second Source

Choose your second data source (Census demographics or CPCB air quality based on user demand). Write its source definition. Run it through the universal engine. If the engine handles it correctly without any code changes, the architecture is proven. If it requires code changes, the engine needs to be made more flexible.

The goal is that adding the third, fourth, and fifth sources requires zero code changes. Only new definition files.

Data Accuracy at Scale

The Verification Chain

For every record in BharatData, there must be an unbroken chain of evidence from the government source to the served data:

```
Government source file (identified by SHA-256 hash)
    ↓ hash stored in ingested_files table
Extracted total row (government's own sum)
    ↓ stored in source_file_totals table
Parsed data rows
    ↓ sum verified against total row (must match exactly)
Normalized records
    ↓ validated against Great Expectations suite
Loaded to database
    ↓ cross-referenced against other geographic levels
Served through API
    ↓ only records with confidence >= 0.5
User receives data with source_url pointing to original
```

A journalist can follow this chain backward. They see a number in the SDK response. The response includes `source_url` and `source_file`. They can download that file from NCRB and find the same number. The `source_file_totals` table proves that

BharatData's sum matches the government's own total. The `ingested_files` table proves the file BharatData parsed is the same file currently on the government website (same hash).

This chain of evidence is your data accuracy guarantee. It does not depend on the data source. It works for NCRB, Census, CPCB, RBI, or any future source because the verification framework is built into the engine, not into source-specific code.

Automated Accuracy Metrics

The quality dashboard aggregates accuracy metrics across all sources:

- Total sources registered
- Sources with 100 percent total match verification
- Sources with known discrepancies (documented)
- Sources pending verification
- Overall confidence distribution across all records
- Records by verification status (verified, unverified, flagged)

This dashboard is your proof of accuracy at scale. When someone asks "how accurate is BharatData?" you show them: "97 percent of all records are verified against government source totals with zero discrepancy. 3 percent have documented discrepancies attributed to known government data inconsistencies."

Implementation Priority

Priority	Task	What It Enables	Time
1	Design and finalize the source definition schema	Everything else depends on this	4-6 hours
2	Write the NCRB source definition file from existing code	Proves the schema works for real data	3-4 hours
3	Build the definition loader and validator	Engine can read and validate definitions	2-3 hours
4	Build the universal		

Priority	Task	What It Enables	Time
	Excel extractor	Handles NCRB and Census	4-6 hours
5	Build the total matcher verification	Proves data accuracy for every file	3-4 hours
6	Build the universal normalizer chain	Source-agnostic normalization	4-6 hours
7	Build the dynamic table creator and loader	Handles any table schema	3-4 hours
8	Integrate Great Expectations auto-generation	Automated validation from definitions	3-4 hours
9	Re-verify existing NCRB data through new engine	Proves migration did not change results	2-3 hours
10	Write Census source definition and test	Proves universality with a second source	3-4 hours
Total			**32-44 hours**

What This Changes for the Rest of the Project

SDKs and API: No Changes

The universal engine feeds data into your Supabase database. The SDKs and API read from the database. They do not care how data got there. Whether data was ingested by a hardcoded NCRB parser or a universal engine reading a YAML definition, the SDK consumer sees the same response shape.

Adding Future Datasets

When a user requests Census data through a GitHub issue, you respond: “I’ll add this. Give me a day.” You spend 30 minutes writing the Census source definition. You run it through the engine. If all validations pass, the data is live. You add a new module to the SDKs (`bd.census`) that queries the new table. Total effort: a few hours per dataset, not weeks.

Long-Term Vision: Community-Contributed Sources

Eventually, contributors can submit source definitions as pull requests. They find a government data source, write the YAML definition, test it locally, and submit a PR. You review the definition, verify it produces accurate data, and merge it. The contributor did not need to write any Python code. They wrote a data description. This dramatically lowers the barrier to contributing new data sources.

The Bottom Line

Your data is your product. Every other component — SDKs, API, playground, AI — is a delivery mechanism for the data. If the data is wrong, nothing else matters. If the data is right and provably right, everything built on top of it inherits that trust.

The universal engine with source definitions and automated verification gives you three things:

1. **Proof of accuracy** through the verification chain (hash tracking, total matching, cross-validation)
2. **Scalability** through declarative source definitions (add sources without code)
3. **Maintainability** through separation of knowledge (what the data looks like) from logic (how to process data)

Build the source definition schema first. Everything else follows from that design decision.

Stage 3 Universal Engine — Final Review

Verdict: APPROVED WITH TWO MANDATORY FIXES

The real-data verification is now complete and the results are strong. 30,223 records processed with 100% mathematical parity against government totals across all three YAML definitions. The error resolution log demonstrates genuine engineering rigor — you encountered real government data messiness (RichText objects, header shifts, unique key collisions, Crime Head mismatches) and solved each one systematically.

This is production-quality work. Two issues must be addressed before proceeding.

What Was Done Correctly

Real Data Verification — CONFIRMED

The formal verification log shows all three YAML definitions producing exact matches against their targets with zero variance. This is the proof I asked for. The engine is not just architecturally sound — it is producing verified accurate output from real NCRB files.

Error Resolution — Exceptional Quality

The four error cases documented in Section 2 are exactly the kinds of issues that surface only when processing real government data. Each one was diagnosed to root cause and fixed at the correct architectural level.

The Header Shift fix using anchored regex with look-ahead verification is particularly well-engineered. Instead of patching one file, you fixed the extraction strategy so all files benefit. This is the correct approach for a universal engine.

The RichText handling is a real-world edge case that most Indian government data projects never handle correctly. Your sanitization function that converts list-of-dict RichText objects to clean strings will save you from silent data corruption as you add more data sources.

The unique key evolution from four iterations is well-documented and the final design including source_id is correct. This prevents the cross-category collision problem permanently.

Geographic Resolution — Strong

The 4-tier resolution hierarchy (exact match, regex suffix stripping, hard-coded aliases, non-geographic bucket) is a mature approach. The specific handling of Telangana variations, J&K/Ladakh transition, and DNH merger shows real domain knowledge encoded into the system.

The 98.4% geographic resolution rate means approximately 1.6% of entity names could not be resolved to canonical forms. At 30,223 records, that is roughly 484 records with unresolved names. These records were not dropped — they were flagged and passed through with reduced confidence. This is the correct behavior per your quality standards.

Total Matcher Validation — The Most Important Achievement

The fact that the TotalMatcher caught 4 silent extraction errors that would have corrupted the database is the strongest justification for the entire universal engine rebuild. Without this validation layer, those errors would have been served to users as accurate data. With it, they were caught before reaching the database.

This single feature — automated mathematical verification against government source totals — is what separates BharatData from every other Indian government data project.

Mandatory Fix 1: Database Must Be Supabase, Not SQLite

The Problem

Section 11 references `ncrb_verification.db` and SQLite optimization including VACUUM and ANALYZE commands. Section 6 mentions “local SQLite storage.” The deduplication uses “INSERT OR IGNORE” which is SQLite syntax.

Your production database is Supabase PostgreSQL. Your SDKs and API are configured to query Supabase. If the universal engine is writing to a local SQLite database, your production Supabase database is NOT being updated by the engine.

This means one of two things is true:

Possibility A: The engine writes to SQLite for verification, and you manually or separately migrate data to Supabase. This is a fragile workflow that introduces a gap between what is verified and what is served.

Possibility B: The engine writes to SQLite and the production Supabase database still contains data from the old hardcoded pipeline. This means your SDKs are serving data that was NOT processed or verified by the universal engine.

Either possibility is a problem.

What To Fix

Update the universal engine's Loader layer to write directly to Supabase PostgreSQL instead of SQLite. The changes required are:

Connection: Replace SQLite connection with psycopg2 or SQLAlchemy connecting to your Supabase DATABASE_URL.

SQL Syntax: Replace SQLite-specific syntax with PostgreSQL equivalents. "INSERT OR IGNORE" becomes "INSERT … ON CONFLICT DO NOTHING". "INSERT OR REPLACE" becomes "INSERT … ON CONFLICT … DO UPDATE". VACUUM and ANALYZE work in PostgreSQL but with slightly different behavior.

Dynamic Table Creation: Your DynamicTableCreator currently generates SQLite CREATE TABLE statements. Update it to generate PostgreSQL-compatible DDL with proper UUID primary keys using gen_random_uuid(), TIMESTAMPTZ for dates, and CHECK constraints for confidence ranges.

Batch Loading: Your batch loader should use psycopg2.extras.execute_values for high-performance PostgreSQL batch inserts, which you already implemented in your earlier Stage 3 work.

After updating, re-run the engine against all 75 files writing directly to Supabase. Verify the record counts match (30,223 total). Then verify your SDKs return the new engine-processed data by running the same queries you used in Stage 4A verification.

Why This Cannot Be Skipped

Your entire consumption layer (API, TypeScript SDK, Python SDK, Playground) reads from Supabase. If Supabase contains old-pipeline data while your verification was done against SQLite data, the mathematical parity proof does not apply to what users actually receive. The verification chain breaks.

Time estimate: 3-4 hours to update the loader and re-run.

Mandatory Fix 2: Null Handling Decision Needs Reversal

The Problem

Section 9.2 states: “The engine maps truly empty cells to 0 for statistical aggregation.”

Your quality standards document explicitly states: “Null values represented as null, never as 0, empty string, dash, or N/A.” Your earlier Stage 3 review established the nuanced approach: dashes in total_cases become 0 (because zero crimes is meaningful), but dashes in disposal metrics (chargesheeted, convicted, acquitted) stay null (because not-reported is different from zero).

Mapping ALL empty cells to 0 collapses two distinct meanings into one. A researcher studying which states fail to report conviction data cannot distinguish “zero convictions” from “state did not report conviction data” if both show as 0.

What To Fix

Implement context-aware null handling as previously specified:

For primary count fields (total_cases, total_ipc, total_sll, and similar aggregate crime counts), empty cells and dashes should become 0 because zero crimes reported is a valid data point.

For disposal and secondary fields (chargesheeted, convicted, acquitted, pending_investigation, pending_trial), empty cells and dashes should remain null because these represent unreported data, not zero.

For rate fields (rate_per_lakh), empty cells should remain null because a rate cannot be calculated without data.

Update the TypeEnforcer or NullHandler to accept field-level null handling rules from the YAML definition. The YAML should specify per-column whether empty means zero or null:

```yaml
null_handling:
  total_cases: "zero"        # Empty means zero crimes
  chargesheeted: "null"      # Empty means not reported
  convicted: "null"          # Empty means not reported
  rate_per_lakh: "null"      # Empty means not calculable
```

This way, different data sources can define their own null semantics without changing engine code.

Time estimate: 2-3 hours to update null handler and re-process.

Observations That Are Correct But Worth Noting

Record Count Shift

Your verified total is 30,223 records. Your earlier Stage 3 had 30,223 records as well. The counts match, which suggests the universal engine produces equivalent output to the old pipeline. However, the composition changed slightly:

Previous: 2,948 state + 28,863 district + 102 city = 31,913 (before cleanup) then 30,223 after fixes.

Current: 2,785 state + 27,333 district + 105 city = 30,223.

The state count dropped from 2,948 to 2,785 (163 fewer). The district count dropped from 28,863 to 27,333 (1,530 fewer). The city count changed from 102 to 105 (3 more).

These shifts are likely explained by improved summary row filtering (removing rows that should never have been data) and improved city category recovery. But you should verify that no legitimate data records were lost in the transition. Pick a specific state-year-category combination and check whether the records present in the old database are also present in the new engine output.

Performance Is Good

11 minutes 42 seconds for 75 files producing 30,223 records is reasonable. 412MB peak memory is acceptable. The optimization from 140ms to 4ms for aggregation queries shows proper indexing.

YAML Strategy Is Validated

Using 3 YAML definitions to handle 75 diverse files proves the declarative approach works. The regex evolution across 2021/2022/2023 headers demonstrates that the YAML definitions can adapt to government format changes without engine code modifications. This is the core value proposition of the universal architecture.

Action Items Summary

Priority	Fix	What Changes	Time
1	Migrate loader from SQLite to Supabase PostgreSQL	Loader layer, connection manager, SQL syntax	3-4 hours
2	Implement context-aware null handling per YAML field rules	NullHandler/ TypeEnforcer, YAML schema	2-3 hours
3	Re-run engine against all 75 files writing to Supabase	Full pipeline execution	1 hour
4	Verify SDKs return engine-processed data	Run SDK integration tests	1 hour
Total			**7-9 hours**

After These Fixes

Once the two mandatory fixes are implemented and the engine writes verified data directly to Supabase:

1. Your SDKs serve data that has been mathematically verified against government source totals

2. Your verification chain is unbroken from NCRB source file to SDK response
3. Your null handling is semantically correct for both count fields and disposal fields
4. Your architecture is ready to add Census, CPCB, RBI, and other sources through YAML definitions alone

At that point, Stage 3 V2.0 is definitively complete and you should proceed to updating the API and SDKs to reflect any schema changes from the universal engine, then move to Stage 4B (Playground, CI/CD, Publishing) and your public launch.

The architecture is genuinely impressive. Fix the database target and the null handling, and this is approved.

Stage 3 Universal Engine — Final Review

Verdict: APPROVED WITH TWO MANDATORY FIXES

The real-data verification is now complete and the results are strong. 30,223 records processed with 100% mathematical parity against government totals across all three YAML definitions. The error resolution log demonstrates genuine engineering rigor — you encountered real government data messiness (RichText objects, header shifts, unique key collisions, Crime Head mismatches) and solved each one systematically.

This is production-quality work. Two issues must be addressed before proceeding.

What Was Done Correctly

Real Data Verification — CONFIRMED

The formal verification log shows all three YAML definitions producing exact matches against their targets with zero variance. This is the proof I asked for. The engine is not just architecturally sound — it is producing verified accurate output from real NCRB files.

Error Resolution — Exceptional Quality

The four error cases documented in Section 2 are exactly the kinds of issues that surface only when processing real government data. Each one was diagnosed to root cause and fixed at the correct architectural level.

The Header Shift fix using anchored regex with look-ahead verification is particularly well-engineered. Instead of patching one file, you fixed the extraction strategy so all files benefit. This is the correct approach for a universal engine.

The RichText handling is a real-world edge case that most Indian government data projects never handle correctly. Your sanitization function that converts list-of-dict RichText objects to clean strings will save you from silent data corruption as you add more data sources.

The unique key evolution from four iterations is well-documented and the final design including source_id is correct. This prevents the cross-category collision problem permanently.

Geographic Resolution — Strong

The 4-tier resolution hierarchy (exact match, regex suffix stripping, hard-coded aliases, non-geographic bucket) is a mature approach. The specific handling of Telangana variations, J&K/Ladakh transition, and DNH merger shows real domain knowledge encoded into the system.

The 98.4% geographic resolution rate means approximately 1.6% of entity names could not be resolved to canonical forms. At 30,223 records, that is roughly 484 records with unresolved names. These records were not dropped — they were flagged and passed through with reduced confidence. This is the correct behavior per your quality standards.

Total Matcher Validation — The Most Important Achievement

The fact that the TotalMatcher caught 4 silent extraction errors that would have corrupted the database is the strongest justification for the entire universal engine rebuild. Without this validation layer, those errors would have been served to users as accurate data. With it, they were caught before reaching the database.

This single feature — automated mathematical verification against government source totals — is what separates BharatData from every other Indian government data project.

Mandatory Fix 1: Database Must Be Supabase, Not SQLite

The Problem

Section 11 references `ncrb_verification.db` and SQLite optimization including VACUUM and ANALYZE commands. Section 6 mentions “local SQLite storage.” The deduplication uses “INSERT OR IGNORE” which is SQLite syntax.

Your production database is Supabase PostgreSQL. Your SDKs and API are configured to query Supabase. If the universal engine is writing to a local SQLite database, your production Supabase database is NOT being updated by the engine.

This means one of two things is true:

Possibility A: The engine writes to SQLite for verification, and you manually or separately migrate data to Supabase. This is a fragile workflow that introduces a gap between what is verified and what is served.

Possibility B: The engine writes to SQLite and the production Supabase database still contains data from the old hardcoded pipeline. This means your SDKs are serving data that was NOT processed or verified by the universal engine.

Either possibility is a problem.

What To Fix

Update the universal engine’s Loader layer to write directly to Supabase PostgreSQL instead of SQLite. The changes required are:

Connection: Replace SQLite connection with psycopg2 or SQLAlchemy connecting to your Supabase DATABASE_URL.

SQL Syntax: Replace SQLite-specific syntax with PostgreSQL equivalents. “INSERT OR IGNORE” becomes “INSERT … ON CONFLICT DO NOTHING”. “INSERT OR REPLACE” becomes “INSERT … ON CONFLICT … DO UPDATE”. VACUUM and ANALYZE work in PostgreSQL but with slightly different behavior.

Dynamic Table Creation: Your DynamicTableCreator currently generates SQLite CREATE TABLE statements. Update it to generate PostgreSQL-compatible DDL with proper UUID primary keys using gen_random_uuid(), TIMESTAMPTZ for dates, and CHECK constraints for confidence ranges.

Batch Loading: Your batch loader should use psycopg2.extras.execute_values for high-performance PostgreSQL batch inserts, which you already implemented in your earlier Stage 3 work.

After updating, re-run the engine against all 75 files writing directly to Supabase. Verify the record counts match (30,223 total). Then verify your SDKs return the new engine-processed data by running the same queries you used in Stage 4A verification.

Why This Cannot Be Skipped

Your entire consumption layer (API, TypeScript SDK, Python SDK, Playground) reads from Supabase. If Supabase contains old-pipeline data while your verification was done against SQLite data, the mathematical parity proof does not apply to what users actually receive. The verification chain breaks.

Time estimate: 3-4 hours to update the loader and re-run.

Mandatory Fix 2: Null Handling Decision Needs Reversal

The Problem

Section 9.2 states: “The engine maps truly empty cells to 0 for statistical aggregation.”

Your quality standards document explicitly states: “Null values represented as null, never as 0, empty string, dash, or N/A.” Your earlier Stage 3 review established the nuanced approach: dashes in total_cases become 0 (because zero crimes is meaningful), but dashes in disposal metrics (chargesheeted, convicted, acquitted) stay null (because not-reported is different from zero).

Mapping ALL empty cells to 0 collapses two distinct meanings into one. A researcher studying which states fail to report conviction data cannot distinguish “zero convictions” from “state did not report conviction data” if both show as 0.

What To Fix

Implement context-aware null handling as previously specified:

For primary count fields (total_cases, total_ipc, total_sll, and similar aggregate crime counts), empty cells and dashes should become 0 because zero crimes reported is a valid data point.

For disposal and secondary fields (chargesheeted, convicted, acquitted, pending_investigation, pending_trial), empty cells and dashes should remain null because these represent unreported data, not zero.

For rate fields (rate_per_lakh), empty cells should remain null because a rate cannot be calculated without data.

Update the TypeEnforcer or NullHandler to accept field-level null handling rules from the YAML definition. The YAML should specify per-column whether empty means zero or null:

```yaml
null_handling:
  total_cases: "zero"        # Empty means zero crimes
  chargesheeted: "null"      # Empty means not reported
  convicted: "null"          # Empty means not reported
  rate_per_lakh: "null"      # Empty means not calculable
```

This way, different data sources can define their own null semantics without changing engine code.

Time estimate: 2-3 hours to update null handler and re-process.

Observations That Are Correct But Worth Noting

Record Count Shift

Your verified total is 30,223 records. Your earlier Stage 3 had 30,223 records as well. The counts match, which suggests the universal engine produces equivalent output to the old pipeline. However, the composition changed slightly:

Previous: 2,948 state + 28,863 district + 102 city = 31,913 (before cleanup) then 30,223 after fixes.

Current: 2,785 state + 27,333 district + 105 city = 30,223.

The state count dropped from 2,948 to 2,785 (163 fewer). The district count dropped from 28,863 to 27,333 (1,530 fewer). The city count changed from 102 to 105 (3 more).

These shifts are likely explained by improved summary row filtering (removing rows that should never have been data) and improved city category recovery. But you should verify that no legitimate data records were lost in the transition. Pick a specific

state-year-category combination and check whether the records present in the old database are also present in the new engine output.

Performance Is Good

11 minutes 42 seconds for 75 files producing 30,223 records is reasonable. 412MB peak memory is acceptable. The optimization from 140ms to 4ms for aggregation queries shows proper indexing.

YAML Strategy Is Validated

Using 3 YAML definitions to handle 75 diverse files proves the declarative approach works. The regex evolution across 2021/2022/2023 headers demonstrates that the YAML definitions can adapt to government format changes without engine code modifications. This is the core value proposition of the universal architecture.

Action Items Summary

Priority	Fix	What Changes	Time
1	Migrate loader from SQLite to Supabase PostgreSQL	Loader layer, connection manager, SQL syntax	3-4 hours
2	Implement context-aware null handling per YAML field rules	NullHandler/ TypeEnforcer, YAML schema	2-3 hours
3	Re-run engine against all 75 files writing to Supabase	Full pipeline execution	1 hour
4	Verify SDKs return engine-processed data	Run SDK integration tests	1 hour
Total			

Priority	Fix	What Changes	Time
			7-9 hours

After These Fixes

Once the two mandatory fixes are implemented and the engine writes verified data directly to Supabase:

1. Your SDKs serve data that has been mathematically verified against government source totals
2. Your verification chain is unbroken from NCRB source file to SDK response
3. Your null handling is semantically correct for both count fields and disposal fields
4. Your architecture is ready to add Census, CPCB, RBI, and other sources through YAML definitions alone

At that point, Stage 3 V2.0 is definitively complete and you should proceed to updating the API and SDKs to reflect any schema changes from the universal engine, then move to Stage 4B (Playground, CI/CD, Publishing) and your public launch.

The architecture is genuinely impressive. Fix the database target and the null handling, and this is approved.

Stage 3 Universal Engine — Final Review

Verdict: APPROVED. Two observations to document, zero blocking issues.

The universal engine is now writing verified data directly to Supabase PostgreSQL. The mathematical parity is confirmed at 100%. The real-data verification is complete. The critical fixes (SQLite to Supabase migration and context-aware null handling) are implemented.

Stage 3 is complete. You can proceed.

What Is Confirmed Correct

Supabase Migration — Done

The report confirms PostgreSQL-native DDL with UUID primary keys, TIMESTAMPTZ audit columns, execute_values batch loading, and SHA-256 deduplication. The connection stability issue with SSL/pooler EOF was solved with exponential backoff retry. This is the correct production database target.

Context-Aware Null Handling — Done

Section 3.1 confirms the NullHandler applies “zero for counts, null for metadata” strategies. This matches the nuanced approach specified in the quality standards.

Mathematical Parity — Confirmed

100% parity across all three YAML definitions against government total rows. The TotalMatcher validation layer is working as designed.

Real Engineering Challenges Solved

The three technical hurdles documented (NULL mapping gap from column index shifts, RichText stringification, SSL connection stability) are genuine production issues that were diagnosed to root cause and fixed at the correct architectural level.

Observation 1: Record Count Increased Significantly — Document Why

The numbers across reports

Report	State	District	City	Total
Stage 3 V1 (old pipeline)	2,948	28,863	102	30,223
Stage 3 V2 (SQLite verification)	2,785	27,333	105	30,223
Stage 3 V2 Final (Supabase)	3,708	46,833	19	50,541

The total jumped from 30,223 to 50,541. That is a 67% increase. This is not necessarily wrong but it needs explanation because your SDKs and API were verified against the 30,223 record database.

Most likely explanation: The universal engine is now extracting more columns and categories from each file than the old pipeline did. Where the old pipeline might have extracted only `total_ipc` from a district file, the universal engine with its enhanced YAML definitions might be extracting `total_ipc`, `total_sll`, `total_cyber`, `total_sc`, `total_st` as separate columns within the same record, or it might be creating separate records per crime category per district where the old pipeline created one record per district.

Either way, this is fine as long as the mathematical parity holds (which it does). But you must update your API and SDKs to handle the new record structure and count.

What to do: Run your SDK integration tests from Stage 4A against the new 50,541 record database. If the SDKs return correct data with the new schema, no changes are needed. If the table structure or column names changed, update the API queries accordingly.

City records dropped to 19

The city count went from 105 to 19 and now covers only 2021 with 19 metropolitan cities instead of 34 cities across 3 years. The report states these are “Cities with population over 2 million.”

This means 2022 and 2023 metropolitan city data is not being ingested by the new engine. Your old pipeline had city data for all three years. Verify whether this is intentional (the YAML definition only covers 2021) or an oversight (the YAML pattern does not match 2022/2023 city files).

If the metro YAML needs to cover all three years with all 34 cities, update `ncrb_metro_unified.yaml` to include the 2022 and 2023 file patterns.

Observation 2: Year Metadata Repair Via SQL — Flag for Process Improvement

Section 5.2 describes a surgical SQL repair where 27,438 records from 2021 were missing the year tag and were fixed with a manual UPDATE statement.

This fix is correct for the immediate problem. However, it reveals that the engine's year extraction logic has a gap. The year should be injected during extraction or normalization, not repaired after loading.

What to improve: Update the orchestrator or extractor to extract the year from the file path pattern during ingestion. Your file paths follow the pattern `data/raw/ncrb-crime-in-india/{year}/districts/*.xlsx`. The year is right there in the path. The engine should parse this and attach it to every record during the extraction phase, not rely on post-hoc SQL repairs.

This is not a blocking issue because the data is correct after the repair. But for future ingestion runs (when 2024 data arrives), the year should be automatically captured, not manually patched.

What Happens Next

Your data foundation is now verified and production-ready in Supabase. The path forward is:

Immediate: Run your Stage 4A SDK integration tests against the new 50,541 record database. Verify the TypeScript SDK, Python SDK, and API all return correct data. If the table schema changed (different column names or structure), update the API route handlers to match.

Then: Proceed to Stage 4B (Playground, CI/CD, Publishing, Launch).

After launch: Add your second data source (Census or CPCB) by writing a new YAML definition and running it through the universal engine. Zero code changes required. This is the payoff of the universal architecture.

Stage 3 is approved. Go ship BharatData.

Stage 4 — Universal API, Automated CI/CD & Intelligent Reporting

---

The Core Idea

Your data layer is now universal. The pipeline reads YAML definitions and ingests any government data source without custom code. But your API layer is still hardcoded. Every time you add a new dataset, you would need to manually write new API routes, new SDK functions, and new database queries. This defeats the purpose of the universal engine.

Stage 4 makes the entire stack universal. The API reads from the same source registry that the pipeline uses. When you add a YAML file for Census data, the API automatically serves Census data through dynamic routes. When you push that YAML to GitHub, CI/CD automatically runs the pipeline, validates the data, generates an HTML report, and deploys the updated API. Zero manual intervention from YAML commit to live data.

Architecture Overview

```
You push a new YAML source definition to GitHub
    ↓
GitHub Actions detects the change in sources/ directory
    ↓
CI/CD Pipeline triggers automatically:
    Step 1: Validate the YAML against the definition schema
    Step 2: Run the universal engine (download, extract, validate,
load)
    Step 3: Run data quality checks against the new data
    Step 4: Generate HTML verification report (with LLM narration)
    Step 5: If all checks pass, deploy the updated API
    Step 6: Commit the HTML report to the repository
    ↓
API automatically discovers and serves the new dataset
    ↓
SDKs automatically expose the new dataset (via registry endpoint)
```

The key insight is that everything is driven by the source registry. The YAML files are the single source of truth. The pipeline reads them for ingestion. The API reads them for routing. The SDKs read them for function generation. The CI/CD reads them for automation triggers.

Part 1: Universal Dynamic API

The Problem With Hardcoded Routes

Your current API has routes like `/v1/crime/state/:state` and `/v1/crime/year/:year`. These routes know about crime data specifically. When you add Census data, you would need to create `/v1/census/state/:state` manually. When you add CPCB data, another set of manual routes. At 20 data sources, you have 100+ hardcoded routes.

The Universal Route Design

Instead of dataset-specific routes, build a single set of generic data routes that work with any dataset registered in your source registry.

The universal endpoint pattern:

```
GET /v1/data/:dataset/:level
```

Where: - `:dataset` is the dataset identifier from the YAML definition (like "ncrb-crime", "census-demographics", "cpcb-air-quality") - `:level` is the geographic level ("state", "district", "city", "national")

Query parameters handle all filtering: - `?state=Maharashtra` — filter by state - `?year=2023` — filter by year - `?category=ipc-total` — filter by category - `?district=Pune` — filter by district - `?city=Mumbai` — filter by city - `?limit=100&offset=0` — pagination - `?sort=total_cases&order=desc` — sorting

Examples of how this serves different datasets:

```
# Crime data for Maharashtra 2023
GET /v1/data/ncrb-crime/state?state=Maharashtra&year=2023

# Census demographics for all states
GET /v1/data/census-demographics/state

# Air quality for Delhi
GET /v1/data/cpcb-air-quality/city?city=Delhi

# Compare crime across states
GET /v1/data/ncrb-crime/state?state=Maharashtra,Karnataka,Tamil
Nadu&year=2023

# District-level crime
GET /v1/data/ncrb-crime/district?state=Maharashtra&year=2023
```

Plus metadata discovery endpoints:

```
# List all available datasets
GET /v1/registry

# Get details about a specific dataset (from YAML)
GET /v1/registry/:dataset

# List available states for a dataset
GET /v1/registry/:dataset/states

# List available years for a dataset
GET /v1/registry/:dataset/years

# List available categories for a dataset
GET /v1/registry/:dataset/categories

# List available columns/fields for a dataset
GET /v1/registry/:dataset/fields
```

How the API Discovers Datasets

The API does not hardcode dataset names or table structures. Instead, it reads from two sources of truth:

Source 1: The source registry YAML files. These tell the API what datasets exist, what their human-readable names are, what geographic levels they support, and what fields they contain.

Source 2: The database metadata. The API queries Supabase to discover which tables exist and what columns they have. This handles the case where a YAML definition exists but the data has not been ingested yet (the API returns “dataset registered but data not yet available”).

Building the Dynamic API

The Dataset Registry Service

Create a service that loads all YAML definitions from the source registry at startup. This service provides:

- List of all registered datasets with their metadata (name, description, publishing body, geographic levels, available fields)
- For each dataset, the corresponding database table name
- For each dataset, the column mapping (which database columns correspond to state, year, category, and metric fields)
- For each dataset, the validation rules and confidence thresholds

The registry service caches the YAML definitions in memory. When the API starts (or is redeployed), it scans the sources directory and loads all definitions. This means adding a new YAML and redeploying the API automatically makes the new dataset available.

The Universal Query Builder

Instead of writing SQL queries per dataset, build a query builder that constructs SQL dynamically based on the dataset's YAML definition and the request parameters.

The query builder receives: - The dataset ID (from the URL path) - The geographic level (from the URL path) - Filter parameters (from query string) - The dataset's table name and column mapping (from the registry)

It produces a Supabase query that: - Targets the correct table - Filters by the correct columns using the mapping - Applies the confidence threshold (always >= 0.5) - Handles pagination and sorting - Returns only the columns defined in the YAML

This means the query builder works for any dataset without modification. It translates generic filter parameters (state, year, category) into dataset-specific column names using the YAML mapping.

The Universal Response Builder

The response builder wraps query results in the standard BharatData response shape (data, meta, query). It reads the dataset's metadata from the YAML definition to populate: - `meta.source.name` from the YAML identity section - `meta.source.url` from the YAML acquisition section - `meta.source.reportName` from the YAML identity section - Confidence statistics calculated from the returned records - Record counts and pagination info

This response shape is identical regardless of the dataset. The SDKs can parse it without knowing what dataset they are querying.

Implementation Steps

Step 1: Build the Registry Service

Create a module that scans the `sources/` directory, loads every YAML file, validates it against your definition schema, and stores the parsed definitions in a structured format. The service exposes methods like:

- `getAllDatasets()` — returns list of all dataset summaries

- `getDataset(id)` — returns full definition for one dataset
- `getTableName(id)` — returns the database table name
- `getColumnMapping(id, level)` — returns how generic fields map to database columns
- `getAvailableLevels(id)` — returns which geographic levels this dataset supports

Step 2: Build the Universal Query Builder

Create a module that takes a dataset definition and request parameters and produces a Supabase query. The key logic is:

The request says `?state=Maharashtra`. The query builder looks up the dataset's column mapping and finds that the "state" concept maps to a column called `entity_name` (for NCRB) or `state_name` (for Census) or `city` (for CPCB city data). It constructs the filter using the correct column name.

The request says `?year=2023`. The query builder finds that "year" maps to a column called `year` in most datasets but might map to `fiscal_year` or `survey_round` in others. It uses the mapping.

The request says `?sort=total_cases&order=desc`. The query builder verifies that `total_cases` is a valid column for this dataset (checking the YAML field definitions) and applies the sort. If the column does not exist in this dataset, it returns an error explaining which columns are available.

Step 3: Build the Universal Response Builder

Create a module that takes raw database rows and the dataset definition, and produces the standard response. It handles:

- Converting database column names (snake_case) to API response names (camelCase)
- Calculating confidence statistics from the returned records
- Populating source attribution from the YAML identity section
- Building the query echo from the request parameters

Step 4: Build the Route Handlers

Create two sets of route handlers:

Registry routes that serve dataset discovery information. These are read-only endpoints that return information from the YAML definitions without querying the database.

Data routes that serve actual data. These use the query builder and response builder to serve data from any registered dataset.

Each route handler is generic. The `/v1/data/:dataset/:level` handler does not know whether it is serving crime data or census data. It:

1. Extracts the dataset ID and level from the URL
2. Looks up the dataset in the registry
3. If not found, returns 404 with available datasets listed
4. Checks that the requested level is supported by this dataset
5. Passes the request parameters to the query builder
6. Executes the query against Supabase
7. Passes results to the response builder
8. Returns the formatted response

Step 5: Backward Compatibility

Your existing SDK functions call specific endpoints like `/v1/crime/state/:state`. You have two options:

Option A: Update the SDKs to use the new universal endpoints. This is cleaner but requires an SDK update.

Option B: Create redirect routes that map old endpoints to new ones. `/v1/crime/state/:state` redirects to `/v1/data/ncrb-crime/state?state=:state`. This maintains backward compatibility while the SDKs are updated.

For v1 launch, implement both. The redirect routes ensure existing SDK installations continue working. New SDK versions use the universal endpoints directly.

Verification for Universal API

Test 1: Register a mock dataset by creating a simple YAML definition and a corresponding test table in Supabase. Query it through the universal endpoint. Verify the response shape matches the standard format.

Test 2: Query your existing NCRB data through the new universal endpoint. Compare the response against the old hardcoded endpoint response. The data must be identical.

Test 3: Query the registry endpoint. Verify it lists all registered datasets with correct metadata.

Test 4: Query a nonexistent dataset. Verify the API returns 404 with the list of available datasets.

Test 5: Query with invalid parameters (unsupported column for sorting, invalid year format). Verify helpful error messages.

Test 6: Verify rate limiting works on the new endpoints identically to the old ones.

Estimated time: 8-10 hours

Part 2: CI/CD Pipeline Automation

The Trigger: New YAML File Pushed

When you push a new or modified YAML file in the `sources/` directory to GitHub, a CI/CD workflow automatically:

1. Validates the YAML definition
2. Runs the universal pipeline for that source
3. Validates the ingested data
4. Generates an HTML verification report
5. Deploys the updated API (which now serves the new dataset)
6. Commits the report to the repository

GitHub Actions Workflow Design

File: `.github/workflows/source-ingestion.yml`

Trigger: Push to main branch with changes in `sources/` directory.

The workflow has these jobs:

Job 1: Validate Source Definition

This job runs first and fast. It checks that the new or modified YAML file is syntactically valid and conforms to your source definition schema.

Steps: - Check out the repository - Install Python with PyYAML and jsonschema - Load the YAML file - Validate it against `sources/_schema/source_definition_schema.yaml` - If validation fails, the workflow stops and creates a GitHub issue explaining what is wrong with the definition

This job takes about 30 seconds. It catches typos and schema violations before any expensive pipeline work begins.

Job 2: Run Universal Pipeline

This job only runs if Job 1 passes. It executes the full pipeline for the changed source definition.

Steps: - Check out the repository - Set up Python 3.11 with all pipeline dependencies - Identify which YAML files changed (using git diff) - For each changed YAML file, run the universal engine

orchestrator - Capture the pipeline output (record counts, validation results, timing) - Save the pipeline output as a JSON artifact

This job takes 5-40 minutes depending on the data source size.

Important configuration: The job needs access to your Supabase credentials through GitHub secrets. It also needs network access to government websites if the YAML specifies downloading new files.

For data sources that require manual file downloads (behind CAPTCHAs or login walls), the YAML should use the ManualUpload acquisition method. In this case, the files must be committed to the repository alongside the YAML definition. The CI/CD pipeline then processes the committed files.

Job 3: Data Quality Validation

This job runs after Job 2. It executes your quality checks against the newly ingested data.

Steps: - Run the Great Expectations validation suite generated from the YAML definition - Run the TotalMatcher verification - Run cross-level validation if applicable - Generate a quality report JSON

If any validation fails, the workflow stops. It does NOT deploy the API. It creates a GitHub issue with the failure details and the quality report attached.

Job 4: Generate HTML Report

This job runs after Job 3 passes. It generates a comprehensive HTML report documenting the entire ingestion process.

More on this in Part 3 below.

Job 5: Deploy API

This job runs after Job 4. It deploys the updated API to Cloudflare Workers. Since the API dynamically reads from the source registry, deploying the API with the new YAML file makes the new dataset available.

Steps: - Install Wrangler - Deploy to Cloudflare Workers - Verify the deployment by calling the registry endpoint and confirming the new dataset appears

Job 6: Commit Report

This job commits the generated HTML report to the repository so it is part of the permanent record.

Steps: - Configure git with a bot identity - Add the HTML report to `reports/` directory - Commit and push with message “automated: ingestion report for [source-name]”

The Complete Workflow

```yaml
name: Source Ingestion Pipeline

on:
  push:
    branches: [main]
    paths:
      - 'sources/**/*.yaml'
      - 'sources/**/*.yml'
  workflow_dispatch:
    inputs:
      source_file:
        description: 'Path to source definition YAML'
        required: true
        type: string

jobs:
  validate-definition:
    runs-on: ubuntu-latest
    outputs:
      changed_sources: ${{ steps.detect.outputs.sources }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 2

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install validation dependencies
        run: pip install pyyaml jsonschema

      - name: Detect changed source files
        id: detect
        run: |
          if [ "${{ github.event_name }}" = "workflow_dispatch" ];
        then
            echo "sources=${{ github.event.inputs.source_file }}"
        >> $GITHUB_OUTPUT
          else
```

```
          CHANGED=$(git diff --name-only HEAD~1 HEAD --
        'sources/**/*.yaml' 'sources/**/*.yml' | tr '\n' ',')
          echo "sources=$CHANGED" >> $GITHUB_OUTPUT
        fi

    - name: Validate YAML definitions
      run: |
        python -m pipeline.engine.definition_loader --validate $
      {{ steps.detect.outputs.sources }}

  run-pipeline:
    needs: validate-definition
    if: needs.validate-definition.outputs.changed_sources != ''
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run universal pipeline
        run: |
          IFS=',' read -ra SOURCES <<< "${{ needs.validate-
        definition.outputs.changed_sources }}"
          for source in "${SOURCES[@]}"; do
            if [ -n "$source" ]; then
              python -m pipeline.run_pipeline --source "$source"
        --output pipeline_result.json
            fi
          done
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_ANON_KEY: ${{ secrets.SUPABASE_ANON_KEY }}

      - name: Upload pipeline results
        uses: actions/upload-artifact@v4
        with:
          name: pipeline-results
          path: pipeline_result.json

  quality-check:
    needs: run-pipeline
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
```

```yaml
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Download pipeline results
        uses: actions/download-artifact@v4
        with:
          name: pipeline-results

      - name: Run quality validation
        run: |
          python -m pipeline.monitoring.quality_check \
            --output quality_report.json
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}

      - name: Upload quality report
        uses: actions/upload-artifact@v4
        with:
          name: quality-report
          path: quality_report.json

  generate-report:
    needs: quality-check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Download artifacts
        uses: actions/download-artifact@v4

      - name: Generate HTML report
        run: |
          python -m pipeline.reporting.generate_html_report \
            --pipeline-result pipeline-results/
        pipeline_result.json \
            --quality-report quality-report/quality_report.json \
            --output reports/$(date +%Y%m%d)_ingestion_report.html
        env:
          LLM_API_KEY: ${{ secrets.LLM_API_KEY }}

      - name: Upload HTML report
        uses: actions/upload-artifact@v4
```

```yaml
      with:
        name: html-report
        path: reports/*.html

  deploy-api:
    needs: quality-check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: pnpm/action-setup@v4
        with:
          version: 9

      - uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install and deploy
        run: |
          cd packages/api
          pnpm install
          pnpm deploy
        env:
          CLOUDFLARE_API_TOKEN: $
        {{ secrets.CLOUDFLARE_API_TOKEN }}

  commit-report:
    needs: generate-report
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4

      - name: Download HTML report
        uses: actions/download-artifact@v4
        with:
          name: html-report
          path: reports/

      - name: Commit report
        run: |
          git config user.name "BharatData Bot"
          git config user.email "bot@bharatdata.dev"
          git add reports/
          git commit -m "automated: ingestion report $(date +%Y-
        %m-%d)" || true
          git push
```

What Happens When You Add Census Data

You write `sources/census/demographics_state.yaml`. You commit and push to main.

GitHub Actions detects the change in `sources/`. The pipeline validates the YAML schema. It runs the universal engine which downloads Census files, extracts data, validates against totals, and loads into Supabase. Quality checks run. An HTML report is generated. The API is redeployed with the new YAML included in its registry scan. The report is committed to the repo.

Within 30-45 minutes of your git push, Census data is live and queryable through the API. No manual steps.

Part 3: HTML Report Generation with LLM

The Report Structure

After every pipeline run, the system generates a comprehensive HTML report. The report has two parts: structured data (tables, metrics, charts) and narrative analysis (generated by an LLM).

Structured Data Section

The structured section is generated purely from pipeline output data. No LLM needed. It includes:

Header: BharatData Ingestion Report, date, source name, pipeline version.

Summary Dashboard: A visual grid showing key metrics. Total records ingested. Processing time. Mathematical parity percentage. Geographic coverage percentage. Confidence distribution. Files processed. Errors encountered.

Data Flow Diagram: A visual representation of records flowing through each pipeline layer with counts at each stage. How many records entered extraction. How many survived validation. How many were loaded. How many were deduplicated.

Validation Results Table: For each validation check, show pass or fail status, expected value, actual value, and discrepancy if any. Color-coded green for pass, red for fail, yellow for warning.

Source File Inventory: Table listing every file processed with filename, year, record count, hash, total match status.

Error Log: If any errors occurred during processing, list them with severity, component, and resolution.

Geographic Coverage Map: Which states and districts have data. Which are missing. Coverage percentage.

Confidence Distribution Chart: Histogram showing how many records fall in each confidence band.

LLM Narrative Section

The narrative section uses an LLM to generate human-readable analysis of the ingestion results. This transforms raw metrics into insights that anyone can understand.

How It Works

You create a Python module that takes the structured pipeline output (a JSON blob with all metrics) and sends it to an LLM API with a carefully designed prompt. The LLM returns a narrative that is inserted into the HTML report.

The prompt design:

```
You are the BharatData quality analyst. You are writing the
narrative
section of an automated ingestion report. Based on the following
pipeline
metrics, write a clear, honest analysis in 3-5 paragraphs.

Rules:
- Every statement must be based on the provided metrics. Do not
add
  information that is not in the data.
- If there are issues, explain them clearly and suggest next
steps.
- Use specific numbers from the metrics.
- Tone: professional, honest, not promotional.
- Do not use superlatives like "excellent" or "amazing". Use
factual
  language like "achieved" or "shows".

Pipeline Metrics:
{json_blob_of_all_metrics}

Write the narrative now.
```

Which LLM to use:

For a free solution, use one of these options:

Option A: Google Gemini API. Has a generous free tier (60 requests per minute for Gemini Flash). Sufficient for generating one report per pipeline run.

Option B: Groq API with Llama. Free tier with fast inference. Good for structured text generation.

Option C: Local Ollama. Run a small model locally. No API costs. But requires local compute resources during CI/CD (not ideal for GitHub Actions).

For GitHub Actions, Option A (Gemini) or Option B (Groq) are the practical choices because they do not require local GPU resources. Store the API key as a GitHub secret.

Fallback: If the LLM API is unavailable (rate limited, down, key expired), the report generates without the narrative section. The structured data section is always present because it does not depend on any external service. The narrative is an enhancement, not a requirement.

Implementation

Create a module at `pipeline/reporting/generate_html_report.py` that:

1. Reads the pipeline result JSON (record counts, timing, validation results)
2. Reads the quality report JSON (all quality check pass/fail statuses)
3. Generates the structured HTML sections using Python string templates or Jinja2
4. Calls the LLM API with the metrics to generate the narrative
5. Inserts the narrative into the HTML
6. Writes the final HTML file

The HTML should be self-contained (all CSS inline, no external dependencies) so it renders correctly when opened from the filesystem or viewed on GitHub.

Use simple, clean HTML with a professional color scheme. Tables with alternating row colors. A summary card grid at the top. Status badges (green checkmark for pass, red X for fail). The report should look like something you would present to a data governance committee.

Estimated time for report generation: 4-6 hours

Part 4: SDK Automation — What Is Possible and What Is Not

What CAN Be Automated

Dataset discovery. The SDKs can call the `/v1/registry` endpoint to discover what datasets are available at runtime. This means when you add Census data, the SDK does not need an update. A developer can write:

```
// TypeScript
const datasets = await bd.getDatasets();
// Returns: ["ncrb-crime", "census-demographics", "cpcb-air-
        quality"]

const censusData = await bd.query("census-demographics", "state",
        {
  state: "Maharashtra",
  year: 2011
});

# Python
datasets = bd.get_datasets()
# Returns: ["ncrb-crime", "census-demographics", "cpcb-air-
        quality"]

df = bd.query("census-demographics", "state",
              state="Maharashtra", year=2011).to_dataframe()
```

This generic `query()` method works with any dataset. The SDK does not need to know what datasets exist at build time. It discovers them at runtime from the registry.

Field discovery. The SDK can call `/v1/registry/:dataset/fields` to learn what columns a dataset has. This enables generic data exploration without hardcoded field names.

Type generation. For TypeScript specifically, you could build a code generation tool that reads the registry API and generates TypeScript type definitions for each dataset. Run this tool before publishing a new SDK version, and the types are always current.

What Should Stay Manual (For Quality)

Named convenience methods. While the generic `bd.query()` works for everything, users prefer named methods like `bd.crime.getByState()`. These methods provide better autocomplete, clearer documentation, and more intuitive API design. They should be manually created for each major dataset you officially support.

The pattern is:

- `bd.query()` — universal method that works with any dataset (automated)
- `bd.crime.getByState()` — convenience method with full types and docs (manual for quality)
- `bd.census.getByState()` — same pattern for Census (manual when you add Census)

Documentation. SDK documentation including JSDoc comments, usage examples, and parameter descriptions must be written by a human (or carefully reviewed if AI-generated). Automated documentation that says "queries the census-demographics dataset" is not helpful. Good documentation says "Returns Census 2011 population and literacy data for Indian states and union territories. Note: Census 2021 data is not yet available due to COVID-19 postponement."

Type definitions. For TypeScript, the interface for a CrimeRecord is different from a CensusRecord. These types should be manually defined with proper field names, JSDoc descriptions, and nullable annotations. A CensusRecord has `population`, `literacyRate`, `sexRatio`. A CrimeRecord has `totalCases`, `chargesheeted`, `convicted`. Autogeneration can create a starting point, but the types need human review for quality.

The Recommended SDK Strategy

Build the SDK in two layers:

Layer 1: Universal (Automated)

A generic query method that works with any registered dataset. This layer is fully automated. When you add a new YAML source definition and the pipeline ingests data, the SDK can query it immediately through `bd.query()` without any SDK code changes or publishing.

This layer includes: - `bd.getDatasets()` — list all available datasets - `bd.getDatasetInfo(id)` — get metadata about a dataset - `bd.query(dataset, level, filters)` — query any dataset - Response parsing, error handling, and DataFrame conversion (Python)

Layer 2: Convenience (Manual)

Named modules for major datasets that provide typed, documented, domain-specific methods. These are created manually when you officially add a dataset to BharatData's supported collection.

This layer includes: - `bd.crime.getByState()` — with CrimeRecord types and crime-specific docs - `bd.census.getByState()` — with CensusRecord types and census-specific docs - Each method is a thin wrapper around `bd.query()` with proper types and validation

The advantage of this approach is that a new dataset is immediately queryable through Layer 1 the moment the pipeline runs. Layer 2 convenience methods can be added at your own pace without blocking data availability.

Part 5: Updated Folder Structure for Stage 4

```
packages/api/
├── src/
│   ├── index.ts                      # Entry point
│   ├── registry/
│   │   ├── registry-service.ts       # Loads and caches YAML
definitions
│   │   ├── definition-parser.ts      # Parses YAML into typed
objects
│   │   └── schema-validator.ts       # Validates definitions
against schema
│   ├── routes/
│   │   ├── data.ts                   # Universal data routes /
v1/data/:dataset/:level
│   │   ├── registry.ts              # Registry discovery
routes /v1/registry
│   │   ├── health.ts                 # Health check /v1/health
│   │   └── legacy.ts                 # Backward-compatible
redirects
│   ├── services/
│   │   ├── query-builder.ts          # Builds Supabase queries
from definitions
│   │   ├── response-builder.ts       # Formats standard response
shape
│   │   └── database.ts              # Supabase client wrapper
│   ├── middleware/
│   │   ├── rate-limiter.ts
│   │   ├── cors.ts
│   │   └── error-handler.ts
│   └── types/
│       └── index.ts
│
├── pipeline/
│   ├── reporting/                    # NEW: Report generation
│   │   ├── __init__.py
│   │   ├── generate_html_report.py   # Main report generator
│   │   ├── llm_narrator.py          # LLM narrative generation
```

```
│   │   ├── templates/
│   │   │   └── report_template.html   # Jinja2 HTML template
│   │   └── charts.py                 # Simple chart generation
(matplotlib)
│   │
│   ├── engine/                        # Universal engine
(existing)
│   ├── sources/                       # YAML definitions
(existing)
│   ├── monitoring/                    # Health and quality checks
(existing)
│   └── great_expectations/            # Validation framework
(existing)
│
├── reports/                           # Generated HTML reports
(git-tracked)
│   ├── 20260327_ncrb_crime_ingestion.html
│   └── 20260328_census_demographics_ingestion.html
│
└── .github/
    └── workflows/
        ├── ci.yml                     # Existing: test on every
push
        ├── source-ingestion.yml       # NEW: auto-ingest when
YAML changes
        ├── deploy-api.yml             # Existing: deploy on API
changes
        └── daily-health-check.yml     # Existing: monitor sources
```

Stage 4 Implementation Order

Step	Task	What It Enables	Time
1	Build Registry Service	API discovers datasets from YAML	3-4 hours
2	Build Universal Query Builder	Single query logic for all datasets	3-4 hours
3	Build Universal Response Builder	Consistent response shape for all datasets	2-3 hours
4	Build dynamic API routes	One set of routes serves all datasets	3-4 hours
5		Existing SDKs keep working	

Step	Task	What It Enables	Time
	Build backward-compatible redirects		1-2 hours
6	Test API with NCRB data through universal routes	Verify parity with old endpoints	2-3 hours
7	Build HTML report generator with LLM narration	Automated verification reports	4-6 hours
8	Build source-ingestion CI/CD workflow	Push YAML and everything runs automatically	3-4 hours
9	Update TypeScript SDK with universal query method	SDK works with any dataset	3-4 hours
10	Update Python SDK with universal query method	SDK works with any dataset	3-4 hours
11	End-to-end test: push a test YAML and verify full automation	Prove the system works	2-3 hours
Total			**30-41 hours**

Stage 4 Verification Protocol

Test 1: Universal API Verification

Query your NCRB data through both the old endpoints (if kept) and the new universal endpoints. The data returned must be identical. This proves the universal API serves the same data as the hardcoded API.

Test 2: CI/CD Automation Verification

Create a test YAML definition for a tiny mock dataset (5 records in a CSV file committed to the repo). Push it to main. Verify the entire pipeline runs automatically: validation, ingestion, quality check, report generation, API deployment, report commit. Check the generated HTML report.

Test 3: SDK Universal Query Verification

Install the updated TypeScript and Python SDKs. Use the generic `bd.query()` method to query NCRB data. Compare results against the convenience `bd.crime.getByState()` method. Results must match.

Test 4: Report Quality Verification

Open the generated HTML report in a browser. Verify it contains: summary metrics, data flow counts, validation results, source file inventory, and LLM narrative. The narrative should be factual and based only on the metrics provided. It should not contain hallucinated information.

Test 5: New Source Addition Verification

Write a real source definition YAML for a second dataset (even a small subset of Census data). Push it. Verify the full automation runs and the new dataset becomes queryable through the API and SDKs without any manual code changes.

Stage 4 Completion Criteria

Stage 4 is complete when:

1. The API serves data from any registered YAML source definition without hardcoded routes

2. Pushing a new YAML file to GitHub triggers automatic ingestion, validation, and deployment
3. Every pipeline run generates a comprehensive HTML report with structured data and LLM narrative
4. Both SDKs have a universal `query()` method that works with any registered dataset
5. The NCRB data serves identically through both old and new API patterns
6. The full automation has been tested end-to-end with at least one YAML push
7. All existing SDK integration tests still pass

Stage 4 Final Review

Verdict: APPROVED. Proceed to Extended Phase.

Your Stage 4 implementation has achieved what it set out to do. The platform has transitioned from hardcoded dataset-specific endpoints to a universal registry-driven architecture. Let me verify each requirement against your reports.

Requirement Verification

Universal API — CONFIRMED ✅

The API now serves data from any registered YAML source definition without hardcoded routes. The registry service discovers datasets automatically. Dynamic routing resolves dataset identifiers to mapped Supabase tables. The standardized response envelope is consistent across all datasets. Legacy backward-compatible redirects are verified working.

CI/CD Automation — CONFIRMED ✅

The source ingestion workflow triggers on YAML changes. The build pipeline compiles successfully for all components (API, TypeScript SDK, Python SDK). The simulated CI confirms the Worker builds correctly with the registry embedded.

HTML Reporting with LLM — CONFIRMED ✅

Sarvam-M integration generates professional narrative analysis. HTML reports are generated automatically during the ingestion pipeline. The reports include structured metrics and AI-generated narrative sections.

SDK Universal Query — CONFIRMED ✅

TypeScript SDK has the standardized `query()` method with full types. Python SDK has universal data access with native pandas support through `to_dataframe()`. Both SDKs can query any registered dataset without code changes.

End-to-End Verification — CONFIRMED ✅

The pollution-stats mock dataset was successfully discovered, ingested, deduplicated, narrated, and served through the universal API. This proves that adding a new dataset requires zero code changes to the pipeline or API.

Quality Metrics Assessment

All four key metrics hit their targets:

Metric	Result	Assessment
Functional Pass Rate	100%	All test cases passed
Build Stability	100%	All three packages compile cleanly
Data Integrity	100%	Hash-based deduplication verified
Automation Success	100%	Full pipeline runs without manual intervention

---

One Recommendation Before Extended Phase

Your Finding 01 is correct and important. The SDK README files need inline examples for the universal `query()` method. Before you publish updated SDKs in the Extended Phase, add a prominent section in both READMEs showing:

For TypeScript README: Show how to discover available datasets, how to query using the universal method, and how to use the typed convenience methods side by side. A developer reading the README should understand both approaches within 60 seconds.

For Python README: Show the same discovery and universal query pattern, plus the DataFrame conversion and export workflow. Include a Jupyter notebook example since that is where most Python users will first try the SDK.

This is not a blocking issue. It is a documentation task that should be completed during the Extended Phase before publishing to npm and PyPI.

What the Extended Phase Should Cover

Based on everything built across Stages 3 and 4, your Extended Phase should focus on:

Playground rebuild. The playground should use the universal API endpoints and the registry discovery to dynamically show available datasets in its dropdown menus. When you add Census data, the playground should automatically show a Census option without any frontend code changes.

SDK publishing. Publish the updated TypeScript SDK to npm and Python SDK to PyPI with the universal query method. Include comprehensive README documentation with examples.

Public launch preparation. Write the launch blog post. Prepare the Show HN submission. Set up the playground at your production URL.

End-to-end verification. Fresh install tests from npm and PyPI against the production API. Playground functional testing. Cross-SDK consistency verification.

My Honest Assessment

You have built something genuinely impressive across Stages 3 and 4. The journey from hardcoded NCRB scripts to a universal YAML-driven engine with automated CI/CD and LLM-generated reports is a significant architectural evolution. The system you have now can scale from 1 data source to 100 data sources with YAML files as the only addition.

The key achievements worth highlighting:

The universal engine processes real government data with mathematical verification against source totals. This is not a toy system. It proves accuracy.

The registry-driven API eliminates the need to write new routes for new datasets. This is the kind of architectural decision that pays dividends for years.

The automated CI/CD pipeline means adding a new data source is a git push, not a development sprint. This dramatically lowers the barrier to expanding coverage.

The LLM-narrated reports transform raw metrics into professional documentation that builds trust with users and stakeholders.

Stage 4 is approved. Go build the Extended Phase and launch BharatData.

BharatData Playground — Complete Design & Implementation Guide

---

What the Playground Is

The playground is the public face of BharatData for non-technical users. It is where a journalist types a question about Indian crime data and gets a verified answer with source citations. It is where a researcher explores datasets and downloads clean CSV files. It is where a student discovers real government data for their project. It is where a citizen checks a political claim against actual statistics.

The playground is not a dashboard. Dashboards display predefined metrics. The playground lets users ask their own questions and get answers in multiple formats: natural language responses from an LLM, data tables, visual charts, and downloadable files. Every answer includes source attribution so users can verify the data themselves.

The User Experience Flow

What Happens When Someone Visits

A user arrives at the playground URL. Within 2 seconds, they see a clean, professional interface with a single prominent input area. They understand immediately that they can ask a question about Indian government data. No signup. No login. No tutorial needed.

The page loads with a subtle animation of the BharatData wordmark. Below it, the tagline “Clean, queryable Indian public data” appears. Below the tagline, a large text input area invites the user to type a question or use the structured query builder.

The Two Interaction Modes

Mode 1: Natural Language (LLM-Powered)

The user types a plain English question like “Which state had the highest cyber crime in 2023?” or “Compare murder rates across southern states from 2021 to 2023” or “Show me crime trends for Maharashtra.”

The LLM processes the question, determines which dataset and filters to query, calls the BharatData API, and generates a comprehensive response that includes:

- A natural language answer with specific numbers and context
- Source citations pointing to the exact NCRB report
- A data table with the underlying records
- A visual chart if the question involves comparison or trends
- A confidence indicator showing how reliable the data is
- Download buttons for CSV and JSON

Mode 2: Structured Query Builder

For users who want precise control, a structured query builder lets them select: - Dataset (from the registry — NCRB Crime, Census Demographics, etc.) - Geographic level (State, District, City) - Specific state or district (searchable dropdown) - Year or year range - Category (crime type, demographic metric, etc.)

This mode bypasses the LLM entirely. The query goes directly to the API. The response shows the data table, chart, and download options without the narrative.

Users can switch between modes with a toggle. The natural language mode is the default because it is more accessible to non-technical users.

Page Layout — Detailed Design

Overall Structure

The page is a single vertical scroll with five distinct sections. No sidebar navigation. No multi-page routing. Everything on one page. The layout is centered with a maximum content width of 1200 pixels, creating comfortable reading margins on large screens.

Color scheme: White background with very subtle warm gray accents. Your brand primary color (choose one strong color — deep blue or deep green work well for data/government themes) is used only for interactive elements (buttons, links, active states). Text is near-black for readability. This is not a dark theme. Government data tools should feel authoritative and accessible, not trendy.

Typography: One font family throughout. Inter, IBM Plex Sans, or Source Sans Pro. These are professional, highly readable, and free. Headers in semibold weight. Body text in regular weight. Monospace for data values in tables (so numbers align properly).

Section 1: Brand Header

Height: 80 pixels on desktop, 64 pixels on mobile.

Left side: BharatData wordmark. The word “Bharat” in your brand color, “Data” in near-black. No icon or logo symbol needed for v1. A clean wordmark is more professional than a hastily designed icon. Below the wordmark in small text: “Clean, queryable Indian public data.”

Right side: Three small text links. “GitHub” linking to your repository. “API Docs” linking to your API documentation. “About” which scrolls to the footer where the full project description lives.

Bottom border: A thin line (1 pixel, light gray) separating the header from the content area.

The header is sticky. It stays visible as the user scrolls down through results. This keeps the brand presence constant and provides navigation access at all times.

Section 2: Query Area

This is the most important section. It occupies the visual center of the page when the user first arrives. Before any query, this section is vertically centered in the viewport, creating a focused, Google-search-like experience.

The Natural Language Input

A large text input area. Not a single-line input. A textarea that is 3 lines tall, suggesting that longer questions are welcome. Placeholder text reads: “Ask about Indian government data… Example: Which state had the highest crime rate in 2023?”

Below the input, three example query chips that users can click to auto-fill: - “Crime trends in Maharashtra 2021-2023” - “Compare cyber crime across top 5 states” - “District-wise crimes against women in UP 2023”

These chips serve two purposes. They show users what kinds of questions they can ask. And they provide one-click access to common queries so the user gets an instant result without typing.

The Query Button

A prominent button below the input: “Ask Bharat Data” in your brand primary color. When clicked, it sends the question to the LLM backend. While processing, the button text changes to “Analyzing…” with a subtle pulse animation.

The Mode Toggle

Below the button, a small toggle: “Switch to Structured Query” which reveals the dropdown-based query builder. When in structured mode, the toggle reads “Switch to Natural Language.” The toggle is subtle and does not dominate the visual hierarchy. Most users should default to the natural language mode.

The Structured Query Builder (Hidden by Default)

When toggled visible, four controls appear in a horizontal row:

1. **Dataset dropdown:** Lists all datasets from the registry API. Each option shows the human-readable name (“NCRB Crime in India”) not the technical ID. The dropdown is searchable.

2. **Geographic level:** Radio buttons or a small dropdown: State, District, City, National.

3. **Filters:** Dynamic dropdowns that change based on the selected dataset and level. If “NCRB Crime” is selected at “State” level, filters show: State (searchable dropdown with all

36), Year (2021/2022/2023), Category (all 18 crime categories with labels). If “Census” is selected, different filters appear based on the Census YAML definition.

4. **Query button:** “Get Data” in the same brand color.

The dynamic filter generation is powered by the registry API. When the user selects a dataset, the playground calls `/v1/registry/:dataset/fields` and `/v1/registry/:dataset/states` to populate the filter options. This means adding a new dataset automatically adds it to the playground without any frontend code changes.

Section 3: Response Area

After a query is submitted, the page smoothly scrolls down (or the content below the query area expands) to reveal the response. The response area has multiple sub-sections that appear sequentially with a subtle fade-in animation.

Sub-section 3A: LLM Narrative Response

A card with a light background and left border in your brand color. The LLM’s natural language answer appears here as formatted text.

The response is structured in paragraphs with specific numbers bolded. Every number mentioned in the narrative has a small superscript citation marker (like academic papers) that links to the source. Citations appear at the bottom of the narrative card.

Example response:

> Maharashtra reported **8,249 cyber crime cases** in 2023[1], the highest among all Indian states. This represents a **23.4% increase** from 6,685 cases in 2022[2]. Karnataka ranked second with **7,892 cases**[1], followed by Telangana with **6,341 cases**[1].
>
> When adjusted for population, the rate per lakh tells a different story. Telangana’s rate of **16.2 per lakh**[1] exceeds Maharashtra’s **6.7 per lakh**[1], suggesting that while Maharashtra has more total cases, Telangana has a higher per-capita cyber crime incidence.
>
> **Sources:** [1] NCRB Crime in India 2023, Table: State/UT-wise Cyber Crimes. Confidence: 0.95 [2] NCRB Crime in India 2022, Table: State/UT-wise Cyber Crimes. Confidence: 0.95
>
> This analysis was generated by BharatData AI from verified government statistics. Always verify critical data against primary sources.

The citation format includes the specific NCRB table name and the confidence score. This lets journalists and researchers trace every number back to its source.

Below the narrative, a small expandable section: “How was this answer generated?” that explains the LLM query interpretation, which API endpoints were called, and what data was used. This transparency builds trust.

Sub-section 3B: Data Table

Below the narrative, the actual data table appears. This is the raw evidence behind the narrative.

Table features:

- **Sortable columns.** Click any column header to sort ascending. Click again for descending. A small arrow icon indicates the current sort column and direction.
- **Column visibility toggle.** A small button “Columns” that opens a dropdown where users can show/hide specific columns. By default, show the most important columns (State, Year, Category, Total Cases, Rate per Lakh). Hide less common columns (Chargesheeted, Convicted, Acquitted) but let users reveal them.
- **Search within results.** A small search input above the table that filters rows as the user types. Useful when the table has many rows (like all districts).
- **Pagination.** For result sets over 50 rows, show pagination controls. Show 50 rows per page by default.
- **Number formatting.** Display numbers with Indian comma notation (12,34,567). This is culturally appropriate for the primary audience.
- **Null display.** Show null values as “—” (em dash) in a lighter gray color. Do not show “null” or “N/A” or leave cells blank.
- **Confidence column.** Show confidence as a percentage with a small colored dot: green for >= 0.9, yellow for 0.7-0.89, orange for 0.5-0.69. This gives users an instant visual sense of data reliability.
- **Row hover.** When hovering over a row, highlight it with a subtle background color change and show a tooltip with the source file name and collection date.

Sub-section 3C: Visual Charts

Below the data table, one or two charts visualize the data. The chart type is automatically selected based on the query:

- **Comparison queries** (multiple states, one year) → Horizontal bar chart
- **Trend queries** (one state, multiple years) → Line chart
- **Distribution queries** (all states, one year, one category) → Choropleth map of India or sorted bar chart
- **Single value queries** → No chart (the narrative is sufficient)

Charts should be clean and minimal. No 3D effects. No gradient fills. No decorative gridlines. Solid colors from your brand palette. Clear axis labels. Data labels on each bar or point showing the exact value.

Use a lightweight charting library. Recharts (React-based) or Chart.js work well. Do not use D3.js directly unless you need the India choropleth map, in which case use a pre-built India map component with D3.

The India choropleth map (if implemented) colors each state based on the queried metric value. Darker shades mean higher values. Hovering over a state shows its name and value. This is the single most visually impressive feature you can add to the playground because it makes data geographic and intuitive.

Sub-section 3D: Download Bar

A horizontal bar below the charts with download options:

- **Download CSV** — Downloads the current result set as a CSV file with source attribution columns included. Filename format: `bharatdata_{dataset}_{state}_{year}_{category}.csv`

- **Download JSON** — Downloads the complete API response (data + meta + query) as formatted JSON.

- **Download Chart** — Downloads the current chart as a PNG image. The image includes the BharatData watermark and source attribution in the bottom corner so that when journalists embed the chart in articles, the source is always visible.

- **Copy Citation** — Copies an academic citation to the clipboard. Shows a small “Copied!” confirmation toast. The citation is in APA format by default.

Section 4: Database Explorer

Below the response area (or accessible via a tab), a database explorer lets advanced users browse the raw data.

This section shows: - Available datasets (from registry) - For each dataset: available tables, record counts, date range, geographic coverage - A simple SQL-like query interface where technical users can construct more complex queries

This section is for power users. It should be visually distinct (slightly different background color) and clearly labeled “Advanced: Database Explorer” so casual users do not feel intimidated.

Section 5: Footer

The footer contains important information organized in three columns:

Column 1: About BharatData Two to three sentences describing what BharatData is. The independence disclaimer. The MIT License mention.

Column 2: Data Sources List of all currently available datasets with links to the original government sources. For each: “NCRB Crime in India — ncrb.gov.in” This helps users verify data against primary sources.

Column 3: Links & Resources - GitHub Repository - npm Package (bharatdata) - PyPI Package (bharatdata) - API Documentation - Contributing Guide - Report a Data Issue (links to GitHub issues)

Bottom bar: “Built by Yash Desai” with your personal link. “Data sourced from Government of India publications.” Copyright notice with MIT license reference.

LLM Integration — How It Works

The Architecture

```
User types question in playground
    ↓
Frontend sends question to your API
    ↓
API endpoint /v1/ai/query receives the question
    ↓
API sends question + available dataset metadata to LLM
    ↓
LLM returns a structured query plan (which dataset, filters,
aggregations)
    ↓
API executes the query plan against Supabase
```

```
    ↓
API sends query results back to LLM with a narration prompt
    ↓
LLM returns natural language narrative with citations
    ↓
API combines: narrative + raw data + chart suggestion
    ↓
Frontend renders the complete response
```

The Two LLM Calls

Call 1: Query Understanding

The first LLM call translates the user's natural language question into a structured query plan. The prompt includes your dataset registry metadata so the LLM knows what data is available.

Prompt structure:

```
You are the BharatData query engine. You translate natural
language
questions about Indian government data into structured API
queries.

Available datasets:
{registry_metadata_json}

Available states: {list_of_36_states}
Available years: [2021, 2022, 2023]
Available categories: {list_of_18_categories}

User question: "{user_question}"

Return a JSON object with:
{
  "dataset": "the dataset ID to query",
  "level": "state or district or city",
  "filters": {
    "state": "state name or null",
    "year": "year or null",
    "category": "category ID or null"
  },
  "comparison": true/false,
  "trend": true/false,
  "entities": ["list of entities if comparison"],
  "years": ["list of years if trend"],
  "chart_type": "bar or line or map or none",
  "explanation": "brief explanation of how you interpreted the
question"
}

Rules:
```

```
- Only use datasets, states, years, and categories from the
provided lists
- If the question is ambiguous, make reasonable assumptions and
note them
- If the question cannot be answered with available data, set
dataset to null
  and explain why in the explanation field
- "UP" means "Uttar Pradesh", "MH" means "Maharashtra", etc.
- "last year" means 2023 (most recent available)
- "southern states" means Karnataka, Kerala, Tamil Nadu, Andhra
Pradesh, Telangana
```

The LLM returns the structured query plan. Your API validates it (checking that the dataset, state, and category actually exist) and executes it against Supabase.

Call 2: Narrative Generation

The second LLM call takes the query results and generates the human-readable narrative.

Prompt structure:

```
You are the BharatData analyst. Generate a clear, factual
analysis
of the following Indian government data.

User's original question: "{user_question}"

Query interpretation: {query_plan_explanation}

Data returned:
{json_data_from_api}

Source information:
- Publishing body: {meta.source.name}
- Report: {meta.source.reportName}
- Confidence: {meta.confidence.average}

Rules:
- Every number you mention must come from the provided data. Do
not
  add any numbers that are not in the data.
- Use superscript citation markers (¹, ², ³) after each number.
- Provide a Sources section at the end listing each citation.
- Include the confidence score in each source citation.
- Do not speculate about causes. Only describe what the data
shows.
- Note if data has limitations (e.g., "2020 data may reflect
reduced
  reporting due to COVID-19 lockdowns").
- Use Indian number formatting (12,34,567) for readability.
```

```
- Keep the response to 2-4 paragraphs maximum.
- End with a disclaimer: "This analysis was generated from verified
  government statistics. Always verify critical data against primary sources."
```

Which LLM to Use

You mentioned using Sarvam-M in your Stage 4 report. Sarvam is an Indian AI company which is appropriate for BharatData's Indian context. However, consider these factors for the playground:

For query understanding (Call 1): Use a fast, affordable model. The query plan is a structured JSON output, not creative text. Gemini Flash, GPT-4o-mini, or Groq with Llama 3 all work well here. Speed matters because the user is waiting.

For narrative generation (Call 2): Use a higher-quality model. The narrative is user-facing text that journalists may quote. Quality matters more than speed here. Gemini Pro, GPT-4o, Claude Haiku, or Sarvam-M are good choices.

Cost management: Cache identical queries. If 10 users ask "crime in Maharashtra 2023" in the same day, make the LLM calls once and serve the cached response to the other 9. Cache keys are the normalized query plan (not the raw question text, since different phrasings can produce the same query).

Fallback: If the LLM API is unavailable (rate limited, down, quota exceeded), the playground should gracefully degrade to the structured query mode. Show the data table and charts without the narrative. Display a small notice: "AI analysis is temporarily unavailable. Showing raw data."

Handling Questions Outside Your Data

When a user asks about data you do not have ("What is the GDP of Maharashtra?"), the LLM's query understanding step should detect this and return a helpful response: "BharatData currently covers crime statistics from NCRB. GDP data is not yet available but is planned for a future release. You can explore available datasets below."

This is better than returning an error or, worse, hallucinating an answer.

How Charts Enhance Understanding

Automatic Chart Selection

The LLM's query plan includes a `chart_type` field. The frontend renders the appropriate chart based on this suggestion:

Bar chart for comparison questions. "Compare crime in Maharashtra, Karnataka, and Tamil Nadu" produces a grouped bar chart with one group per state and one bar per crime category or year.

Line chart for trend questions. "Show murder trends in Delhi from 2021 to 2023" produces a line chart with years on the x-axis and case counts on the y-axis.

Horizontal bar chart for ranking questions. "Top 10 states by cyber crime in 2023" produces a horizontal bar chart sorted by value, making it easy to compare rankings.

India map for geographic distribution questions. "Show IPC crime rates across all states in 2023" produces a choropleth map of India where each state is colored by its crime rate. This is visually striking and immediately communicates geographic patterns.

Chart Design Principles

Every chart follows these design rules:

- Clean, minimal style. No 3D effects. No gradient fills. No decorative elements.
- Your brand primary color for single-series data. A harmonious color palette for multi-series data.
- Clear axis labels with units (e.g., "Total Cases" not just "Count").
- Data labels on each bar or point showing the exact value.
- A descriptive title generated from the query (e.g., "Cyber Crime Cases by State, 2023").
- Source attribution in small text below the chart: "Source: NCRB Crime in India 2023 via BharatData"
- The BharatData wordmark in the bottom-right corner (small, subtle) so the chart is branded when downloaded or shared.

Chart Downloads

When a user clicks “Download Chart,” the chart is rendered as a PNG image at 2x resolution (for crisp display in articles and presentations). The image includes: - The chart itself - The title - Source attribution - BharatData branding

This means when a journalist drops the downloaded chart into their article, the source attribution and branding travel with it. Every chart becomes a distribution channel for BharatData.

Mobile Experience

The playground must work on mobile phones because journalists and researchers often check data from their phones.

Mobile layout adjustments:

- The query input takes full width and is taller (4 lines instead of 3) for easier typing on mobile keyboards.
- Example query chips scroll horizontally instead of wrapping.
- The structured query builder stacks controls vertically instead of horizontally.
- Data tables scroll horizontally (the accepted pattern for tables on mobile).
- Charts resize to fill the screen width with a 16:9 aspect ratio.
- Download buttons stack vertically.
- The footer collapses to a single column.

Touch interactions: - Table column sorting works with taps. - Chart tooltips appear on tap (not hover, since mobile does not have hover). - The mode toggle between natural language and structured query is large enough to tap easily.

Implementation Technology

Frontend Framework

Next.js with the App Router. This gives you server-side rendering for the initial page load (good for SEO and perceived performance), client-side interactivity for the query interface, and easy deployment to Vercel.

Key Dependencies

- **React** for the UI components

- **Recharts** or **Chart.js** for data visualization
- **Tailwind CSS** for styling (utility-first CSS that produces consistent, professional designs without writing custom CSS)
- Your **TypeScript SDK** (`bharatdata`) for API communication — the playground is also a real-world test of your SDK

Styling Approach with Tailwind

Tailwind CSS lets you build a professional design by composing utility classes. You define your brand colors, typography, and spacing in the Tailwind config. Then every component uses these consistent design tokens. This produces a cohesive look without writing hundreds of lines of custom CSS.

Define your brand in `tailwind.config.js`: - Primary color (for buttons, links, active states) - Background colors (white, warm gray variants) - Text colors (near-black, medium gray for secondary text, light gray for muted text) - Font family (Inter or your chosen font) - Border radius (small, consistent — 6px or 8px) - Shadow (subtle, used sparingly — only for the response card and header)

Branding Throughout the Playground

Consistency Elements

Every page element reinforces the BharatData brand:

- **Header wordmark** — always visible due to sticky header
- **Input placeholder** — mentions “Bharat Data” by name
- **Button text** — “Ask Bharat Data” not just “Search”
- **Response card** — branded left border in primary color
- **Chart watermark** — small BharatData wordmark on every chart
- **Download filenames** — prefixed with “bharatdata_”
- **Citation format** — includes “via BharatData (bharatdata.dev)”
- **Footer** — full branding with links and attribution
- **Browser tab** — title reads “Bharat Data — Indian Public Data Playground”
- **Favicon** — a small “BD” monogram or a simplified version of your wordmark

The Independence Disclaimer

This appears in two places:

1. Below the query input in small gray text: “Bharat Data is an independent open source project. Not affiliated with any government body.”

2. In the footer, repeated for completeness.

This is not optional. It prevents any user from mistaking BharatData for an official government service.

Performance Requirements

The playground must feel fast. Perceived performance is as important as actual performance.

Initial page load: Under 2 seconds on a 4G connection. The page should render with the query input visible and interactive before any API calls complete. The registry data (dataset list, state list) can load in the background after the initial render.

Query submission to first content: Under 3 seconds for structured queries (no LLM involved). Under 8 seconds for natural language queries (includes two LLM calls). Show a loading indicator immediately when the user clicks the button. Show the narrative text first (since the LLM generates it), then the data table (rendered from the API response), then the chart (rendered client-side from the table data).

Streaming the LLM response: If your LLM API supports streaming (most do), stream the narrative text character by character as the LLM generates it. This gives the user immediate visual feedback and makes the wait feel shorter. The cursor appears and text flows in, similar to ChatGPT’s interface.

Implementation Steps

Step	Task	Time
1	Set up Next.js project with Tailwind CSS and brand configuration	2-3 hours
2	Build the header and footer	1-2 hours

Step	Task	Time
	components with full branding	
3	Build the natural language query input with example chips	2-3 hours
4	Build the structured query builder with dynamic registry-driven dropdowns	3-4 hours
5	Build the LLM integration (query understanding and narrative generation)	4-6 hours
6	Build the data table component with sorting, search, and formatting	3-4 hours
7	Build the chart components (bar, line, horizontal bar)	3-4 hours
8	Build the download functionality (CSV, JSON, chart PNG, citation copy)	2-3 hours
9	Build the response card layout combining narrative, table, chart, downloads	2-3 hours
10	Build mobile responsiveness	2-3 hours
11	Build the database explorer section	2-3 hours
12	Performance optimization (streaming, caching, lazy loading)	2-3 hours
13	Deploy to Vercel and verify	1-2 hours
Total		**29-41 hours**

Verification Checklist

Functional Tests

1. Type “crime in Maharashtra 2023” in natural language mode. Verify narrative appears with cited numbers, data table shows correct records, chart renders, downloads work.

2. Switch to structured mode. Select NCRB Crime, State level, Maharashtra, 2023, Total IPC Crimes. Click Get Data. Verify same data appears as Test 1 but without narrative.

3. Type a comparison question. Verify grouped bar chart renders with multiple states.

4. Type a trend question. Verify line chart renders with multiple years.

5. Type a question about data that does not exist. Verify the system explains what is available instead of crashing or hallucinating.

6. Download CSV. Open in spreadsheet. Verify data is correct and source attribution columns are present.

7. Download chart PNG. Verify it includes branding and source attribution.

8. Click Copy Citation. Paste somewhere. Verify proper APA format.

9. Open on mobile phone. Verify all controls are usable and table scrolls horizontally.

10. Check page load time in browser dev tools. Verify under 2 seconds for initial render.

Brand Consistency Tests

1. Take a screenshot of the playground. Does it look professional? Does the BharatData brand feel authoritative and trustworthy?

2. Download a chart and a CSV file. Do both clearly identify BharatData as the source?

3. Read the LLM narrative response. Does it cite sources? Does it avoid speculation? Does it end with the verification disclaimer?

4. Check every interactive element (buttons, dropdowns, toggles). Do they all use the brand primary color consistently?

5. Ask someone who has never seen the playground to try it. Can they get useful data within 60 seconds without any instructions?

When all tests pass, the playground is ready for public launch.

BharatData AI Playground — Diagnosis & Fix Guide

Understanding the Root Problem

Your diagnosis is accurate. The core failure is architectural, not a bug in any single component. Let me break down exactly what is wrong and how to fix each layer.

The 0 Records Failure — Deep Analysis

When a user asks “Which district in Maharashtra reported highest crime rate,” three things need to happen:

Step 1: The LLM must understand this is a district-level query filtered by state (Maharashtra), not an entity lookup for “Maharashtra” as a district.

Step 2: The query plan must have separate fields for `state` and `district` rather than cramming both into a generic `entity` field.

Step 3: The database query must translate to something like:
```
SELECT * FROM crime_records_district WHERE state = 'Maharashtra'
ORDER BY total_cases DESC LIMIT 1
```

Your current system fails at Step 2. The `UIQueryPlan` schema forces the LLM to put “Maharashtra” into the `entity` field, which maps to the district column. Since no district is named “Maharashtra,” the query returns zero rows.

This is not a prompt engineering problem. It is a schema design problem. The fix requires changing the query plan structure, the system prompt, and the query builder together.

The Three Interconnected Fixes

Fix 1: Expand the Query Plan Schema

Your current `UIQueryPlan` has rigid fields: `entity`, `year`, `category`. This forces the LLM to squeeze multi-dimensional geographic queries into a single field.

The fix is replacing the rigid schema with a flexible filter system that matches your actual database columns.

Current schema (broken):

```
interface UIQueryPlan {
  dataset: string;
  level: string;       // "state" | "district" | "city"
  entity: string;      // This is the problem — one field for all
        geography
  year: number;
  category: string;
  chartType: string;
}
```

Fixed schema:

```
interface UIQueryPlan {
  dataset: string;
  level: string;
  filters: {
    state?: string;           // Filter by state name
    district?: string;        // Filter by district name
    city?: string;            // Filter by city name
    year?: number;            // Single year
    years?: number[];         // Multiple years for trends
    category?: string;        // Crime category
  };
  sort?: {
    field: string;            // Column to sort by
    order: "asc" | "desc";   // Sort direction
  };
  limit?: number;             // Max rows to return
  aggregation?: string;       // "sum" | "max" | "min" | "avg" |
        null
  chartType: string;          // "bar" | "line" | "table" | "none"
  explanation: string;        // LLM explains its interpretation
}
```

Why this fixes the problem: When the user asks “Which district in Maharashtra reported highest crime rate,” the LLM now generates:

```
{
  "dataset": "ncrb-crime",
  "level": "district",
  "filters": {
    "state": "Maharashtra",
    "year": 2023
  },
  "sort": {
    "field": "total_cases",
    "order": "desc"
  },
  "limit": 1,
  "chartType": "bar",
  "explanation": "Finding the district with highest crime rate in
        Maharashtra for the most recent year"
}
```

The `state` filter correctly maps to `WHERE state = 'Maharashtra'` on the district table. The `sort` and `limit` correctly return only the top district. The LLM does not need to guess which column "entity" maps to because the filter names directly correspond to database columns.

Fix 2: Update the System Prompt

Your system prompt tells the LLM how to generate the query plan. It must teach the LLM the new schema with clear examples for each query type.

The updated system prompt structure:

The prompt should have four sections: role definition, available data description, output format specification, and examples.

Role Definition:

Tell the LLM it is a query planner for Indian government data. It translates natural language questions into structured query plans. It never generates SQL. It never fabricates data. It only uses datasets, fields, states, and categories from the provided lists.

Available Data Description:

Dynamically generate this section from your registry service. For each registered dataset, include: the dataset ID, human-readable name, available geographic levels, available filter fields with their types, available sort fields, and the year range.

For NCRB crime data, the available fields section should look like:

```
Dataset: ncrb-crime
Name: NCRB Crime in India
Levels: state, district, city
```

```
Filter fields:
  - state (string): Indian state or union territory name. Accepts
full names and common abbreviations.
  - district (string): District name within a state.
  - city (string): Metropolitan city name.
  - year (integer): Single year between 2021 and 2023.
  - years (array of integers): Multiple years for trend analysis.
  - category (string): Crime category ID from this list: [ipc-
total, sll-total, crimes-against-women, ...]
Sort fields: total_cases, rate_per_lakh, chargesheeted, convicted
```

Output Format:

Show the exact JSON schema the LLM must produce. Include every field with its type and whether it is required or optional. Emphasize that the `filters` object can contain any combination of the available filter fields.

Examples:

Include at least 8 diverse examples covering every query pattern:

Example 1 — Simple state query: “Crime in Maharashtra 2023” produces filters with state and year.

Example 2 — District within a state: “Which district in UP has most cyber crime” produces level “district” with filters for state “Uttar Pradesh” and category “cyber-crime”, sorted by total_cases descending with limit 1.

Example 3 — Multi-year trend: “Show murder trends in Delhi from 2021 to 2023” produces filters with state “Delhi” and years [2021, 2022, 2023] and category “murder”, with chartType “line”.

Example 4 — Comparison: “Compare crime in Maharashtra, Karnataka, Tamil Nadu” produces level “state” with no state filter (returns all), but the LLM should note in the explanation that the UI should highlight these three states.

Example 5 — Top N ranking: “Top 5 states by cyber crime in 2023” produces level “state” with year 2023 and category “cyber-crime”, sorted by total_cases descending with limit 5.

Example 6 — Specific district: “Crime data for Pune district” produces level “district” with district “Pune”.

Example 7 — City query: “Crimes in Mumbai 2023” produces level “city” with city “Mumbai” and year 2023.

Example 8 — Broad exploration: “Show me all crime data for 2023” produces level “state” with year 2023 and no category filter (returns all categories).

These examples teach the LLM the query patterns by demonstration. The more diverse your examples, the better the LLM handles unusual questions.

Fix 3: Update the Query Builder

Your query builder receives the new flexible query plan and constructs the Supabase query. The key change is handling the `filters` object dynamically instead of mapping fixed fields.

Current logic (broken):

```
// Old: maps "entity" to a single column
query = supabase
  .from(tableName)
  .select('*')
  .eq('district', queryPlan.entity)  // This is the bug
  .eq('year', queryPlan.year);
```

Fixed logic:

The query builder should iterate over the filters object and apply each filter to the appropriate column. The column mapping comes from the dataset's YAML definition (via the registry service).

```
For each filter in queryPlan.filters:
  If filter is "state" → apply .ilike('state', filter.value)
  If filter is "district" → apply .ilike('district', filter.value)
  If filter is "city" → apply .ilike('city', filter.value)
  If filter is "year" → apply .eq('year', filter.value)
  If filter is "years" → apply .in('year', filter.value)
  If filter is "category" → apply .eq('category', filter.value)

If queryPlan.sort exists:
  Apply .order(sort.field, { ascending: sort.order === 'asc' })

If queryPlan.limit exists:
  Apply .limit(queryPlan.limit)

Always apply: .gte('confidence', 0.5)
```

The query builder validates every filter field against the dataset's registered fields (from the YAML definition). If the LLM generates a filter for a field that does not exist, the query builder rejects it with a helpful error rather than sending a broken query to the database.

For the "years" array filter, use Supabase's `.in()` method which generates a SQL `WHERE year IN (2021, 2022, 2023)` clause. This natively supports multi-year queries without any looping or multiple requests.

Improving the Narrative Quality

The Problem

Some prompts need deeper analysis than a simple data fetch and summary. When a user asks “What are the crime trends in India over the last 3 years?” they want insight, not just numbers. They want to know which categories are increasing, which states show unusual patterns, and what the data suggests about broader trends.

The Fix: Two-Tier Narrative Strategy

Tier 1: Simple queries (single state, single year, single category)

For these queries, the narrative is straightforward. The LLM receives the data and generates 2-3 paragraphs with cited numbers. This is what you have now and it works well for simple queries.

Tier 2: Complex queries (comparisons, trends, rankings, explorations)

For these queries, the narrative needs to be richer. The LLM should perform analytical reasoning on the data before generating the narrative.

The way to achieve this is by including analytical instructions in the narrative generation prompt. When the query involves multiple years, tell the LLM to calculate year-over-year changes and identify the direction of trends. When the query involves multiple states, tell the LLM to rank them and note which are above or below the national average. When the query involves a specific category, tell the LLM to provide context about what that category means in Indian law.

Updated narrative prompt for complex queries:

```
You are analyzing Indian government data for a BharatData user.

QUERY CONTEXT:
User asked: "{original_question}"
Query type: {comparison|trend|ranking|exploration}
Data retrieved: {data_json}

ANALYSIS INSTRUCTIONS:
1. Start with a direct answer to the user's question.
2. Provide key numbers with citation markers.
3. For trends: Calculate and state the percentage change between
   the first and last year for each entity.
```

```
4. For comparisons: Rank the entities and note which are
significantly
   above or below the group average.
5. For rankings: State the top and bottom performers and the gap
   between them.
6. Note any data limitations (null values, missing years,
   confidence variations).
7. End with the verification disclaimer.

RULES:
- Every number must come from the provided data.
- Use Indian number formatting (12,34,567).
- Use superscript citation markers after each cited number.
- Do not speculate about causes.
- Do not provide policy recommendations.
- Maximum 4 paragraphs.
```

This prompt structure tells the LLM what kind of analysis to perform based on the query type, rather than letting it decide on its own. This produces more consistent, higher-quality narratives.

Detecting Query Complexity

Add a `queryComplexity` field to the query plan that the LLM sets based on its interpretation:

- `simple` — single entity, single year, single category
- `comparison` — multiple entities being compared
- `trend` — same entity across multiple years
- `ranking` — sorting by a metric to find top/bottom performers
- `exploration` — broad query without specific filters

Your backend uses this field to select the appropriate narrative prompt template. Simple queries get the short prompt. Complex queries get the detailed analytical prompt.

Improving the Streaming Experience

The Problem

When the LLM generates the narrative, there is a delay between the user clicking “Ask” and seeing any response. This delay feels like the system is broken, especially for complex queries that take 5-8 seconds.

The Fix: Progressive Response Loading

Instead of waiting for the complete LLM response before showing anything, show intermediate results as they become available:

Phase 1 (Immediate, 0 seconds): Show a skeleton loading state with the message “Understanding your question…” This appears the instant the user clicks the button.

Phase 2 (0.5-1 second): Show the query interpretation. Display a small card saying: “Querying: District-level crime data for Maharashtra, 2023. Sorting by total cases.” This comes from the `explanation` field in the query plan, which is available before the database query even starts.

Phase 3 (1-2 seconds): Show the data table. The database query typically returns in under a second. Display the raw data table immediately while the narrative is still being generated. The user can start exploring the data before the AI analysis is ready.

Phase 4 (2-5 seconds): Stream the narrative. As the LLM generates text token by token through SSE, display each word as it arrives. This creates the familiar “typing” effect that users expect from chat interfaces. The narrative appears to flow in naturally.

Phase 5 (After narrative complete): Render the chart. The chart is generated client-side from the data table, so it can render as soon as the data is available. However, positioning it after the narrative creates a natural reading flow: question, interpretation, data, analysis, visualization.

This progressive loading means the user never stares at a blank screen for more than half a second. Even if the narrative takes 8 seconds to generate, the user has already seen their query interpretation, the data table, and possibly a chart before the narrative finishes streaming.

Handling Edge Cases

When the Question Cannot Be Answered

If the user asks about data that does not exist in BharatData (like GDP data or weather data), the LLM should detect this during query planning and return a response that:

1. Acknowledges what the user is looking for
2. Explains that BharatData does not currently have that dataset
3. Lists what datasets ARE available

4. Suggests where the user might find the requested data (like RBI website for economic data)

This is configured in the system prompt by including a clear instruction: “If the question requires data outside the available datasets, set dataset to null and explain what is available in the explanation field.”

The frontend detects a null dataset in the query plan and shows a friendly message instead of trying to query the database.

When the Data Has Nulls

Some queries return records where metric columns are null (like chargesheeted or convicted for states that did not report disposal data). The narrative should acknowledge this:

“Note: Disposal statistics (chargesheeted, convicted, acquitted) are not available for 5 states in this dataset. These states reported crime counts but did not provide case disposal information to NCRB.”

Add this instruction to the narrative prompt: “If the data contains null values in metric columns, note how many records have nulls and explain that null means ‘not reported by the source’ not ‘zero’.”

When the LLM Hallucinates Column Names

Your concern about the LLM generating filter names that do not match actual database columns is valid. The query builder must validate every filter against the dataset’s registered fields.

If the LLM generates `filters: { "state_name": "Maharashtra" }` but the actual column is `state`, the query builder should:

1. Check if `state_name` is a registered field
2. If not, check if a similar field exists (fuzzy matching: `state_name` is close to `state`)
3. If a close match is found, use it and log a warning
4. If no close match is found, return an error to the LLM and ask it to regenerate with valid field names

This fuzzy matching prevents hard failures when the LLM uses slightly different naming conventions than your schema defines.

When Rate Limited

If the LLM API (Sarvam-M or whichever provider) rate limits your requests during high playground traffic, the playground should:

1. Fall back to structured query mode (no LLM, just direct database query)
2. Show the data table and chart without the narrative
3. Display a small notice: “AI analysis is temporarily unavailable. Showing raw data results.”

The data is still accurate and useful without the narrative. The narrative is an enhancement, not a requirement.

The Think Tag Issue

Your report mentions “silently ripping out the `<think>` reasoning tags” from the LLM response. This is happening because Sarvam-M (or the underlying model) uses chain-of-thought reasoning wrapped in think tags.

The current approach of stripping these tags on the server before streaming to the client is correct. However, make sure the stripping is robust:

The stripper should handle think tags that span multiple SSE chunks. If one chunk ends with `<thi` and the next starts with `nk>reasoning</think>`, the simple regex replacement on individual chunks will miss it. Buffer the stream and strip complete think blocks before forwarding to the client.

Also handle the case where the model outputs the think block AFTER starting the narrative (some models reason mid-response). Strip all think content regardless of position in the response.

UX Improvements Beyond the Query Fix

Suggested Follow-Up Questions

After displaying a response, show 3 contextual follow-up questions that the user can click. Generate these based on the current query:

If the user queried crime in Maharashtra 2023, suggest: - “How does Maharashtra compare to other states?” - “Show crime trends in Maharashtra from 2021 to 2023” - “Which district in Maharashtra has the most crime?”

These follow-ups keep the user engaged and guide them toward deeper analysis. They also demonstrate what kinds of questions the playground can answer.

Query History (Session Only)

Even though you are not tracking history across sessions, keep a session-level history within the current browser tab. Show the last 5 queries as clickable chips below the input area. This lets users quickly re-run a previous query or modify it.

This is purely client-side using React state. No server storage. No cookies. When the user closes the tab, the history is gone.

Source Verification Link

For every number in the narrative, the citation should include a direct link to the source file on the NCRB website. If the user clicks the citation, they should be taken directly to the NCRB page where the source file can be downloaded.

This is possible because your database stores `source_url` and `source_file` for every record. The narrative response should include these as part of the citation metadata.

Data Freshness Indicator

Show a small indicator on the playground that tells users how current the data is. Something like: “Data last updated: March 2026. Latest available year: 2023.”

This prevents users from assuming they are looking at 2026 data when the most recent NCRB report covers 2023.

Implementation Priority

Priority	Fix	Impact	Time
1	Expand query plan schema with flexible filters	Fixes the 0 records bug for all geographic queries	2-3 hours

Priority	Fix	Impact	Time
2	Update system prompt with new schema and 8 examples	LLM generates correct query plans	2-3 hours
3	Update query builder for dynamic filter mapping	Database receives correct queries	2-3 hours
4	Add multi-year support (years array filter)	Enables trend queries	1-2 hours
5	Implement progressive response loading	Users see results faster	2-3 hours
6	Add query complexity detection and analytical prompts	Better narrative quality for complex queries	2-3 hours
7	Add suggested follow-up questions	Better user engagement	1-2 hours
8	Add edge case handling (no data, nulls, hallucinated columns)	Prevents confusing error states	2-3 hours
Total			**15-22 hours**

Fix items 1-3 first as a single coordinated change. They are interdependent. The schema change, prompt update, and query builder update must be deployed together. Test with the failing query “Which district in Maharashtra reported highest crime rate” and verify it now returns correct data.

Then fix items 4-8 incrementally, testing each one before moving to the next.

What I Need From You (If Anything Is Unclear)

The fixes above are based on your architecture description. If any of these do not match your actual implementation, let me know:

1. Is the query builder in `packages/api/src/services/query-builder.ts` or somewhere else?
2. Does the system prompt live in `packages/api/src/services/ai-service.ts` or in a separate configuration file?
3. Is the Sarvam-M API called from the Cloudflare Worker directly, or through a separate backend service?
4. Does the SSE streaming go directly from Sarvam-M through your Worker to the client, or is there an intermediate processing step?
5. What is the current rate limit on your Sarvam-M API key? This affects how much prompt detail you can include per request.

If these match what I assumed, proceed with the fixes in priority order. The 0 records bug should be resolved within the first 3 fixes.

BharatData — Project Objectives

Across Societal, Industrial, and Democratic Dimensions

Objective 1: Eliminating the Data Access Tax on Indian Innovation

The Problem

Every developer, startup, and technology company building products for the Indian market pays an invisible tax. Before they can write a single line of product code, they must spend weeks or months collecting, cleaning, and normalizing Indian government

data. This tax is not measured in rupees. It is measured in engineering hours, delayed launches, abandoned ideas, and products that never get built.

A fintech startup in Bangalore wants to build a credit risk model using district-level demographic and crime data. The data exists across NCRB, Census, and RBI publications. But accessing it requires building three separate scrapers, handling three different file formats, normalizing three different geographic coding systems, and maintaining all three scrapers as government websites change. The startup estimates 6 weeks of engineering time just for data plumbing before a single line of risk modeling code is written. With a 4-person team burning runway, 6 weeks of non-product work is the difference between launching before competitors and running out of money.

An agricultural technology company wants to build a farmer advisory app that shows real-time mandi prices alongside historical weather patterns and crop production data. The data exists across Agmarknet, IMD, and the Ministry of Agriculture. But Agmarknet uses session-based navigation that breaks automated access. IMD requires registration for historical data. The Agriculture Ministry publishes production statistics in PDF reports where column layouts change every year. The company hires a dedicated data engineer whose entire job is fighting with government data portals. That engineer costs 12-15 lakhs per year and produces zero product value. They produce data plumbing that should be infrastructure, not a competitive activity.

An insurance company wants to use district-level crime, disaster, and health data for actuarial modeling. They need NCRB crime statistics, NDMA disaster records, and CBHI health infrastructure data. Each source uses different district names. NCRB calls it “Bangalore Urban.” Census calls it “Bengaluru Urban.” The disaster database calls it “Bangalore.” The insurance company builds an internal mapping table with 780 district names across three coding systems. When new districts are created through bifurcation, the mapping table breaks and must be manually updated. This is institutional knowledge that exists inside one data engineer’s head, creating a single point of failure.

The Objective

BharatData eliminates this tax by providing a single SDK that gives instant programmatic access to clean, normalized, verified Indian government data. A developer types `npm install bharatdata` or `pip install bharatdata` and immediately has access to crime statistics, demographics, air quality, agricultural prices, and economic indicators through simple function calls.

The 6-week data plumbing effort becomes a 5-minute SDK integration. The dedicated data engineer role becomes unnecessary for basic government data access. The internal mapping table is replaced by BharatData's canonical geographic reference system that is maintained by the open source community and handles district bifurcations, state name variations, and historical boundary changes automatically.

This objective directly enables the Indian technology ecosystem to build more products, launch faster, and spend engineering resources on innovation rather than data plumbing. Every hour saved on data collection is an hour available for solving actual problems that Indian citizens face.

Objective 2: Democratizing Investigative Journalism

The Problem

India has a vibrant and essential independent media ecosystem. Organizations like IndiaSpend, The Wire, Article 14, Scroll, FactChecker, and dozens of regional outlets produce data-driven journalism that holds governments accountable and informs citizens. This journalism depends on government data. Without numbers, investigative stories are opinions. With verified numbers, they are evidence.

But accessing government data for journalism is extraordinarily expensive in time. A journalist investigating whether a state government's claim of reduced crime is true needs 10 years of NCRB data broken down by crime category for that state. This data is spread across 10 annual PDF reports, each over 400 pages, with slightly different table structures and column names each year. Manually extracting this data takes 2-3 weeks of full-time work.

Indian newsrooms are financially constrained. Most data journalists work on multiple stories simultaneously under tight deadlines. A story that requires 3 weeks of data collection will either be delayed until it is no longer relevant, simplified to use only partial data, or abandoned entirely. The stories that get abandoned are often the most important ones because they require the most comprehensive data analysis.

The consequence is a less informed public. When a Chief Minister claims "crime against women has decreased by 15% under our government," citizens have no practical way to verify this claim.

The data to verify it exists in NCRB publications. But accessing and analyzing it requires technical skills and weeks of effort that ordinary citizens and most journalists do not have.

International journalists covering India face even greater barriers. A correspondent for a European newspaper investigating air pollution in Indian cities needs CPCB air quality data. They have no familiarity with Indian government websites, no understanding of Indian administrative geography, and no ability to read Hindi text that appears in some data portals. They settle for using WHO aggregate data that lacks the city-level and temporal granularity needed for a compelling story. India's environmental crisis gets less international coverage than it deserves because the data is too hard to access from outside the country.

The Objective

BharatData provides journalists with instant access to verified government data through a web playground that requires no technical skills and no installation. A journalist types a question in plain English, receives a data-backed answer with source citations, downloads a clean CSV for their analysis, and embeds a properly attributed chart in their article. The 3-week data collection effort becomes a 3-minute playground session.

The LLM-powered natural language interface means journalists do not need to learn query syntax, understand database schemas, or write code. They ask questions in the same language they would ask a research assistant. The difference is that this assistant responds with verified government data, not opinions.

Every number served by BharatData includes a source citation pointing to the exact government report, table, and page. When a journalist publishes "Maharashtra reported 8,249 cyber crime cases in 2023 (Source: NCRB Crime in India 2023, Table 9A.1, via BharatData)," the reader can verify this number by downloading the original NCRB report. This traceability is what separates data journalism from speculation.

The playground's chart download feature includes BharatData branding and source attribution embedded in the image. When a journalist drops a downloaded chart into their article, the source information travels with the visual. Every chart becomes both a data communication tool and a trust signal.

This objective directly strengthens Indian democracy by enabling more data-driven accountability journalism, faster fact-checking of political claims, and deeper public understanding of governance outcomes.

Objective 3: Accelerating Academic Research

The Problem

Indian academic institutions produce world-class research on economics, public health, environmental science, criminology, urban planning, sociology, and public policy. Much of this research depends on Indian government data. The Census, NCRB, NFHS, NSSO, and dozens of other surveys and administrative datasets provide the empirical foundation for studying Indian society.

The data collection phase of research is disproportionately expensive. A PhD candidate studying the relationship between education spending and learning outcomes needs data from state budget documents, UDISE school enrollment data, National Achievement Survey scores, and Census demographics. Each source uses different geographic codes, different time reference periods, and different file formats. The first 6-8 months of a 3-year PhD program are spent not on research but on data infrastructure that has no intellectual contribution to the field. Every PhD student who studies an India-related topic rebuilds this infrastructure from scratch. The next student studying a related topic starts over.

This waste is not just individual. It is systemic. Indian academia collectively spends millions of person-hours per year on data collection and cleaning that adds zero intellectual value. These hours could be spent on analysis, modeling, hypothesis testing, and insight generation. The slow data collection phase also creates a selection bias in research topics. Students avoid research questions that require data from multiple government sources because the integration effort is too daunting. Important policy questions go unstudied not because they lack intellectual merit but because the data access barrier is too high.

International researchers face even higher barriers. A professor at a European university studying comparative urbanization patterns needs Indian district-level data. They have no institutional relationships with Indian government bodies, no access to data portals that require Indian institutional credentials, and no budget to hire a local research assistant for data collection. They use outdated Census 2011 data from a secondary source because accessing current data is impractical from outside India. India is understudied in global comparative research partly because Indian data is too hard to access.

The Objective

BharatData eliminates the data collection phase for Indian research. A researcher installs the Python SDK, writes three lines of code, and has a clean pandas DataFrame of any available government dataset ready for analysis.

The Python SDK is specifically designed for the research workflow. It returns pandas DataFrames with proper column types so they integrate directly with statsmodels, scikit-learn, matplotlib, and Stata. Export methods produce CSV, Parquet, and Excel files compatible with every statistical software package used in academia. The citation generation method produces properly formatted academic citations in APA, MLA, Chicago, and BibTeX formats, removing the tedious task of manually constructing source references for methodology sections.

The canonical geographic reference system means researchers can join crime data with census data with education data by state or district without building their own mapping tables. The historical boundary change handling means time-series analysis correctly accounts for the Telangana bifurcation, the Ladakh separation, and district bifurcations without manual adjustments.

This objective directly accelerates the pace of India-related academic research, enables more complex cross-dataset studies, reduces the barrier for international researchers studying India, and ensures that important policy questions are studied rather than avoided due to data access difficulty.

Objective 4: Enabling Evidence-Based Policy Making

The Problem

India’s policy making institutions, from NITI Aayog to state planning boards to district administrations, face an ironic challenge. They are surrounded by data that their own government produces but they cannot easily query across departments and datasets. A district collector who wants to understand their district’s performance across education, health, employment, crime, and infrastructure indicators must request separate reports from five different department offices. Each report arrives in a different format, at a different time, measuring different things over different periods.

NITI Aayog, India’s apex planning body, has publicly acknowledged the challenge of data integration across government departments. The Aspirational Districts Programme,

which tracks district-level development across 49 indicators, required a dedicated data integration effort because no unified data access layer existed across ministries.

State planning boards making budget allocation decisions rely on data that is months or years old because the compilation and analysis process is manual. By the time data from district health offices is aggregated at the state level, cleaned for inconsistencies, and formatted into a presentation for the planning commission, the budget cycle may have moved forward without evidence-based input.

The consequence is policy decisions made with incomplete, outdated, or inconsistent data. A state government allocating police resources across districts cannot easily correlate crime rates with police strength, population density, urbanization levels, and conviction rates because these data points live in different departmental silos with incompatible formats.

The Objective

BharatData provides a unified, queryable data layer that enables cross-departmental analysis for the first time. A policy analyst at NITI Aayog can query crime data, demographic data, education data, and health data through a single interface and receive integrated results with consistent geographic coding and temporal alignment.

The universal API serves any registered dataset through the same endpoint pattern. This means building a cross-departmental analysis tool requires calling the same API with different dataset parameters, not integrating five separate data systems.

The Agentic AI research agent (planned for a future phase) will handle complex multi-dataset queries automatically. A policy maker asking “How does air quality correlate with respiratory disease hospitalizations across states, and how has this relationship changed over the last decade?” will receive a comprehensive analysis that queries multiple datasets, aligns them temporally and geographically, calculates statistical correlations, and generates a narrative with proper caveats.

This objective directly improves governance outcomes by enabling evidence-based decision making, faster policy response to emerging trends, more efficient resource allocation, and better accountability through transparent data-driven performance measurement.

Objective 5: Empowering Civic Engagement

The Problem

Indian citizens in a democracy have the right to understand how their government performs. Public data exists precisely to enable this understanding. But the gap between data publication and citizen comprehension is vast.

When a politician claims “unemployment has decreased in our state,” a citizen has no practical way to verify this. The Periodic Labour Force Survey publishes state-level unemployment estimates. But accessing this data requires navigating the Ministry of Statistics website, downloading a PDF report, finding the right table among dozens, understanding the methodology (the difference between usual principal status and current weekly status definitions of unemployment), and performing year-over-year comparison. This requires statistical literacy that most citizens do not have and hours of effort that most citizens cannot spare.

When a parent wants to compare schools in their district using government quality metrics, they cannot. UDISE collects detailed school-level data on enrollment, infrastructure, teacher qualifications, and learning outcomes. But this data is not published in any format that allows school-level comparison by a non-technical person.

When a citizen wants to know whether air quality in their city is improving or worsening over the years, they cannot easily access historical air quality data. CPCB publishes current AQI readings but historical trend analysis requires navigating a complex web portal that serves data one station at a time, one parameter at a time.

The fundamental promise of open government data is that citizens can hold their government accountable using the government’s own numbers. This promise is broken when the data is technically public but practically inaccessible.

The Objective

BharatData’s web playground makes government data accessible to anyone with a web browser and an internet connection. A citizen types “Has crime increased in Delhi over the last 3 years?” and receives a clear, LLM-generated answer backed by verified NCRB data with source citations.

The playground does not require installation, coding skills, data analysis knowledge, or statistical literacy. It translates plain language questions into structured data queries and presents the results as natural language narratives, tables, and charts. A farmer, a teacher, a shopkeeper, or a retired government employee can query their government's data as easily as they can use a search engine.

Every response includes source attribution so the citizen knows where the data came from and can verify it. The confidence score tells them how reliable the extraction was. The known limitations section is transparent about what the data does and does not cover.

This objective directly strengthens Indian democracy by enabling informed civic participation, fact-checking of political claims by ordinary citizens, public discourse grounded in verified data rather than rhetoric, and a more accountable relationship between government and citizens.

Objective 6: Building Open Source Infrastructure for India

The Problem

India has the world's largest population, one of the largest economies, and one of the most complex administrative geographies. Yet it has no canonical, community-maintained, open source data infrastructure comparable to what exists in other countries.

The United States has the Census Bureau's API, data.gov, and dozens of well-maintained open source projects for accessing government data. The European Union has Eurostat with structured data APIs and download facilities. The UK has the Office for National Statistics with clean, machine-readable data publications.

India has data.gov.in which lists 600,000 datasets but most are links to files on other servers that frequently break. Individual projects like DataMeet have cleaned specific datasets but these are volunteer efforts that are not maintained consistently. Academic researchers at specific institutions have built one-off data processing tools for their own studies that are never shared or maintained beyond the publication of their paper.

The consequence is that every person and organization that needs Indian government data builds their own data plumbing from scratch. Thousands of scrapers, parsers, normalizers, and

mapping tables exist in private codebases across India, each solving the same problems independently. This is a massive duplication of effort that serves nobody well.

The Objective

BharatData builds the canonical open source data infrastructure for India that does not currently exist. It is not a product owned by a company. It is a public good maintained by a community.

The canonical geographic reference system (mapping every variation of every Indian state, district, and city name to a standard form, tracking historical boundary changes, and handling administrative reorganizations) is published as an open file that anyone can use independently of the BharatData SDK. This single file, maintained by community contributions, eliminates the need for thousands of private mapping tables.

The universal pipeline engine with YAML source definitions lowers the barrier to adding new data sources. A contributor who wants to add a government dataset does not need to write a parser. They write a YAML description of the data source and the engine handles the rest. This means domain experts (agricultural economists, environmental scientists, health researchers) can contribute data sources from their area of expertise without needing software engineering skills.

The MIT license ensures that anyone can use, modify, and build upon BharatData for any purpose, including commercial use. A startup can build a product on top of BharatData without licensing concerns. A government agency can adopt the infrastructure without procurement complexity. An academic can cite it in a paper without intellectual property issues.

This objective directly creates the shared data infrastructure that India needs to support its technology ecosystem, research community, journalism sector, and civic society.

Objective 7: Creating a Replicable Model for Other Countries

The Problem

The challenge of inaccessible government data is not unique to India. Every developing country faces similar issues. Brazil, Nigeria, Indonesia, South Africa, Mexico, and dozens of other nations have government data locked in hostile formats with no

unified access layer. Researchers, journalists, and developers in these countries face the same barriers that Indian data professionals face.

No open source template exists for building a national public data platform. Each country would need to start from scratch, making the same architectural decisions, solving the same parsing problems, and building the same normalization systems.

The Objective

BharatData's universal engine architecture is designed to be country-agnostic. The pipeline engine does not know it is processing Indian data. It reads YAML definitions and processes whatever data they describe. The geographic resolver uses canonical mapping files that can be replaced with any country's administrative geography. The data type handlers support international number formats alongside Indian formats.

By publishing BharatData as an open source project with comprehensive documentation of its architecture, design decisions, and operational procedures, we create a template that other countries can fork and adapt. "NigeriaData" or "BrazilData" could be built by replacing the source definitions and geographic mappings while reusing the universal engine, the SDK framework, the API architecture, and the CI/CD automation.

This objective extends BharatData's impact beyond India to the global open data movement, positioning the project as a reference implementation for national public data platforms.

Objective 8: Demonstrating AI Application in Civic Technology

The Problem

Most AI applications in 2025-2026 are focused on commercial productivity (writing emails faster), entertainment (generating images), or enterprise automation (processing invoices). Relatively few AI applications are directed at civic purposes where AI could have outsized social impact.

Government data analysis is a domain where AI can provide genuine value to society. The combination of natural language understanding (translating citizen questions into data queries), data analysis (computing trends, comparisons, and anomalies), and narrative generation (explaining statistical results in plain language) is a natural fit for large language models. But this

application requires careful engineering to prevent hallucination, ensure source attribution, and maintain the trust that government data analysis demands.

The Objective

BharatData demonstrates a responsible, production-grade application of AI in civic technology. The LLM-powered playground shows how AI can make government data accessible to non-technical citizens while maintaining verification standards that journalism and research demand.

The architecture implements multiple safeguards against AI misuse in a data context. The LLM never generates data. It only queries existing verified data. Every number in an AI-generated narrative is traced to a specific government source with a confidence score. The LLM does not speculate about causes or recommend policies. It describes what the data shows with cited evidence.

The system prompt design, the query plan validation, the progressive response loading, and the fallback to direct database queries when AI is unavailable are all replicable patterns for other civic AI applications.

This objective demonstrates that AI can be responsibly applied to strengthen democracy and civic engagement, providing a reference implementation for the emerging field of civic AI.

Summary of All Objectives

Number	Objective	Primary Beneficiary	Impact Area
1	Eliminate the data access tax on innovation	Developers and startups	Technology and economy
2	Democratize investigative journalism	Journalists and media	Democracy and accountability
3	Accelerate academic research	Researchers and students	Knowledge and policy
4	Enable evidence-based policy making	Government officials	Governance quality

Number	Objective	Primary Beneficiary	Impact Area
5	Empower civic engagement	Citizens	Democratic participation
6	Build open source infrastructure for India	Entire Indian data ecosystem	Systemic efficiency
7	Create a replicable model for other countries	Global open data movement	International impact
8	Demonstrate responsible civic AI	AI and civic tech communities	AI governance

Each objective reinforces the others. Better journalism (Objective 2) enables better civic engagement (Objective 5). Better research (Objective 3) enables better policy making (Objective 4). Open source infrastructure (Objective 6) accelerates all other objectives by creating shared tools. The AI application (Objective 8) makes all objectives accessible to non-technical users.

Together, these objectives position BharatData not as a developer tool or a data product but as civic infrastructure that serves the foundational needs of Indian democracy, economy, and society.

BharatData — Final Production Polish Guide

Part 1: Production Hardening

API Resilience

Fallback Strategy

Your API currently has a single point of contact with Supabase. If Supabase experiences downtime or slow responses, every SDK call and playground query fails. Production systems need fallback layers.

Level 1: Response Caching at the Edge

Cloudflare Workers have access to the Cache API. For queries that are frequently repeated (like “states list” or “categories list” or “Maharashtra IPC 2023”), cache the response at the edge for a configurable duration.

Metadata endpoints (states, categories, years) change rarely. Cache them for 24 hours. These endpoints are called every time the playground loads and every time an SDK initializes. Caching them eliminates thousands of unnecessary database queries per day.

Data endpoints change only when you run the pipeline (which happens when new NCRB data is published, roughly once per year). Cache these for 1 hour. This means the first user to query “Maharashtra 2023” hits the database, but the next 999 users in that hour get the cached response instantly.

When the pipeline runs and loads new data, invalidate the cache. Add a cache-busting mechanism: the pipeline’s final step calls a special API endpoint that clears the edge cache, ensuring fresh data is served immediately after ingestion.

Level 2: Stale-While-Revalidate

If Supabase is temporarily unreachable, serve the last cached response with a header indicating the data may be stale. This is better than showing an error. The response includes a `X-BharatData-Cache: stale` header so the SDK can inform the developer that the data might not be the latest.

The user sees their data. The system logs the database connection failure. When Supabase recovers, fresh data replaces the stale cache automatically.

Level 3: Static Fallback Dataset

For the absolute worst case (Supabase is down for hours and the cache has expired), maintain a static JSON snapshot of the most commonly queried data. This snapshot is bundled with the Cloudflare Worker deployment. It contains state-level summary data for the most recent year across major categories.

The static fallback is clearly marked in the response: “This data is from a cached snapshot dated [date]. Live data is temporarily unavailable.” This is honest, helpful, and prevents the playground from showing a blank error screen.

Health Check Endpoint Enhancement

Your current `/v1/health` endpoint returns a simple “ok” response. Enhance it to perform actual dependency checks:

Check 1: Can the Worker reach Supabase? Execute a simple `SELECT 1` query. Report the response time.

Check 2: Is the registry loaded? Verify that at least one dataset is registered. Report the dataset count.

Check 3: What is the cache status? Report how many cached responses exist and their average age.

Check 4: What is the current rate limit utilization? Report what percentage of the rate limit budget has been consumed in the current window.

Return all checks as a structured JSON response with individual pass/fail status for each. This endpoint is what your daily GitHub Actions health check calls. It is also useful for debugging when users report issues.

Rate Limiting Enhancement

Your current rate limiter uses in-memory counters which reset when the Worker instance is evicted. For production, enhance it with these improvements:

Add response headers on every request showing the rate limit status. The headers `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset` tell the SDK exactly how many requests remain and when the limit resets. Both SDKs should read these headers and expose them to developers.

Add a grace period. When a user hits the rate limit, the 429 response should include a `Retry-After` header with the number of seconds until the limit resets. The SDKs should read this and either auto-retry after the specified delay or expose the value to the developer.

Add separate limits for different endpoint types. Metadata endpoints (states, categories, years) should have a higher limit because they are cheap to serve from cache. Data endpoints should have a moderate limit. The AI query endpoint should have the lowest limit because it triggers expensive LLM API calls.

Request Validation

Every API endpoint should validate its inputs before touching the database. This prevents malformed requests from consuming database resources and returns helpful error messages to the developer.

Validate that year parameters are integers between 2000 and 2030. Validate that state names contain only letters, spaces, ampersands, and periods (no SQL injection characters). Validate that category parameters match the registered category list. Validate that limit parameters are positive integers not exceeding 1000. Validate that sort field parameters match actual column names in the dataset.

For every validation failure, return a 400 status with a JSON error body that includes the field that failed validation, what was expected, what was received, and a suggestion for fixing it.

Error Response Standardization

Every error response from the API must follow the exact same format regardless of what caused the error. This allows both SDKs to parse errors consistently.

The error format should always include: an HTTP status code, a machine-readable error code string, a human-readable error message, a suggestion for how to fix the problem, and a timestamp. Internal server errors should never expose database connection strings, SQL queries, stack traces, or internal file paths.

Playground Error Handling

The Error Screen

When the playground encounters an error, it should never show a blank white page, a browser error, a raw JSON dump, or a cryptic technical message. Instead, show a designed error state that matches your brand.

For API errors (the backend returned an error):

Show a card with a warm amber border (not red, which feels alarming). The card contains the error message from the API, the suggestion from the API, and a “Try Again” button. Below the card, show three alternative actions the user can try: “Try a different query,” “Use the structured query builder,” or “Check available datasets.”

For network errors (cannot reach the API):

Show a card with a gray border. The message says “Unable to connect to BharatData servers. This is usually temporary.” Include a “Retry” button that attempts the same request. Below, show: “If this persists, check the service status at [status page URL] or try again in a few minutes.”

For LLM errors (the AI component failed):

Show a card with a blue border. The message says “AI analysis is temporarily unavailable. Showing raw data results.” Below this message, show the data table and chart without the narrative. The user still gets their data. They just do not get the AI-generated analysis. This is a graceful degradation, not a failure.

For “no data” results (the query returned zero records):

This is NOT an error. Do not show an error screen. Show a friendly informational message: “No data found for this combination.” Below it, show suggestions: “Try selecting a different state,” “Try a different year (available: 2021, 2022, 2023),” “Try a broader category.” Show the list of available states and categories so the user can immediately adjust their query.

Part 2: Complete Document List

These are all the documents you need for a production-ready open source project. I am listing each document with its purpose, audience, and where it lives. Do not write them all at once. Prioritize by the order listed.

Priority 1: Must Have Before Public Launch

Document 1: Root README.md

Purpose: The first thing anyone sees when they visit your GitHub repository. Must explain what BharatData is, show installation and usage, list available data, and link to all other resources.

Audience: Everyone — developers, journalists, researchers, potential contributors, recruiters.

Location: Repository root.

Status: You already have this from the README we created. Review and update it with final production URLs, correct record counts, and verified code examples.

Document 2: TypeScript SDK README.md

Purpose: The README that appears on the npmjs.com package page. Must show installation, quick start, full API reference with all functions and types, error handling examples, and configuration options.

Audience: JavaScript and TypeScript developers.

Location: `packages/typescript-sdk/README.md`

Key content: Installation command, 3-line quick start example, table of all functions with parameters and return types, error handling code example, configuration options, link to full documentation.

Document 3: Python SDK README.md

Purpose: The README that appears on the pypi.org package page. Must show installation (with optional dependency groups), quick start, DataFrame workflow, export methods, citation generation, and Jupyter integration.

Audience: Python developers, data scientists, researchers.

Location: `packages/python-sdk/README.md`

Key content: Installation commands for core and optional extras, quick start with DataFrame example, export examples (CSV, Parquet, Excel), citation generation example, Jupyter display explanation, configuration options.

Document 4: LICENSE

Purpose: Legal permission for anyone to use, modify, and distribute BharatData.

Audience: Everyone.

Location: Repository root.

Content: Standard MIT License text. Already exists.

Document 5: CHANGELOG.md

Purpose: Version history documenting what changed in each release. Users check this to understand what is new, what broke, and what was fixed.

Audience: SDK users, API consumers, contributors.

Location: Repository root.

Format: Follow Keep a Changelog format. Sections for Added, Changed, Fixed, Removed per version. Start from version 0.1.0.

Document 6: CODE_OF_CONDUCT.md

Purpose: Community behavior standards for contributors and participants.

Audience: Contributors, community members.

Location: Repository root.

Content: Adopt the Contributor Covenant standard. Do not write a custom one.

Document 7: CONTRIBUTING.md

Purpose: How to contribute to BharatData. Development setup, how to report bugs, how to suggest data sources, how to submit code changes, how to contribute canonical mappings.

Audience: Potential contributors.

Location: Repository root.

Key content: Development environment setup steps, bug report template guidance, data source request process, canonical mapping contribution process (the easiest entry point for new contributors), code contribution process (fork, branch, PR), commit message conventions, review process and response time commitment.

Document 8: SECURITY.md

Purpose: How to report security vulnerabilities responsibly.

Audience: Security researchers, users who discover vulnerabilities.

Location: Repository root.

Key content: Dedicated security email address (not public GitHub issues), acknowledgment timeline (48 hours), fix timeline commitment, credit policy for reporters, currently supported versions.

Priority 2: Should Have Before Public Launch

Document 9: Data Attribution and Legal Compliance

Purpose: Explains where every piece of data comes from, what licenses apply, how attribution should be given, and BharatData's scraping ethics.

Audience: Users who cite BharatData data, legal reviewers, government officials.

Location: `docs/legal/ATTRIBUTION.md`

Key content: Attribution standards for every data point, recommended citation format, Government Open Data License overview, scraping ethics (rate limiting, robots.txt respect, no CAPTCHA circumvention), independence disclaimer, user responsibilities.

Document 10: Terms of Use

Purpose: Legal terms for using the API and playground.

Audience: API and playground users.

Location: Website page or `docs/legal/TERMS.md`

Key content: Data is aggregated from public sources, should be verified for critical decisions, service provided as-is, rate limits exist, attribution required when publishing, project is independent from government. Written in plain language, not legal jargon.

Document 11: Privacy Policy

Purpose: What data BharatData collects from users and how it is handled.

Audience: Users, especially in jurisdictions with data protection laws.

Location: Website page or `docs/legal/PRIVACY.md`

Key content: API usage analytics collected (endpoint, response time, status code), playground queries retained temporarily for service improvement, no personal information collected, no cookies beyond essential, no third-party sharing, data retention periods, user rights.

Document 12: API Documentation

Purpose: Complete reference for every API endpoint with request format, parameters, response shape, error codes, and examples.

Audience: Developers integrating directly with the API (those not using the SDKs).

Location: `docs/api/API_REFERENCE.md` or a dedicated documentation website.

Key content: Base URL, authentication (API key if applicable), rate limits, every endpoint with method, path, parameters, request example, response example, error responses. Include curl examples for every endpoint.

Document 13: Data Source Registry Documentation

Purpose: Detailed information about every data source BharatData serves, including source URLs, publishing bodies, coverage periods, known limitations, and data quality notes.

Audience: Users who need to understand where data comes from, researchers writing methodology sections, journalists verifying sources.

Location: `docs/data-sources/REGISTRY.md`

Key content: For each data source: full name, publishing body, source URL, data format, update frequency, coverage period, geographic coverage, fields extracted, known data quality issues, known limitations, confidence scoring methodology.

Priority 3: Should Have Within First Month After Launch

Document 14: Quick Start Guide for Journalists

Purpose: A non-technical guide specifically for journalists and media professionals explaining how to use the playground to get verified government data for their reporting.

Audience: Data journalists, reporters, editors.

Location: `docs/guides/JOURNALIST_GUIDE.md`

Key content: Step-by-step playground walkthrough with screenshots, how to interpret confidence scores, how to verify data against primary sources, how to cite BharatData in articles, how to download and use CSV exports, how to embed charts with proper attribution.

Document 15: Quick Start Guide for Researchers

Purpose: A guide for academic researchers explaining how to use the Python SDK for research workflows, including DataFrame analysis, citation generation, and reproducibility.

Audience: PhD students, professors, research analysts.

Location: `docs/guides/RESEARCHER_GUIDE.md`

Key content: Python SDK installation with pandas, Jupyter notebook workflow, DataFrame analysis examples, export to Stata/SPSS/R formats via CSV and Parquet, citation generation for methodology sections, reproducibility tips (pinning SDK version, recording query parameters).

Document 16: Quick Start Guide for Developers

Purpose: A technical guide for developers integrating BharatData into their applications.

Audience: Software engineers, startup CTOs, technical architects.

Location: `docs/guides/DEVELOPER_GUIDE.md`

Key content: SDK installation and configuration, TypeScript and Python examples side by side, error handling best practices, rate limit handling, caching strategies, webhook integration (if available), performance optimization tips, example architectures (Next.js dashboard, Flask API, data pipeline).

Document 17: Data Quality Report

Purpose: A public transparency document showing the current state of data quality across all datasets. Updated after every pipeline run.

Audience: Anyone who wants to verify BharatData's accuracy claims.

Location: `docs/quality/QUALITY_REPORT.md` or generated HTML in `reports/`

Key content: Total record counts by dataset and geographic level, mathematical parity results (parsed sums vs government totals), confidence distribution statistics, known discrepancies with explanations, data freshness (when each dataset was last updated), geographic coverage percentage.

Document 18: Architecture Overview

Purpose: Technical documentation of BharatData's system architecture for contributors and technical evaluators.

Audience: Potential contributors, technical recruiters, engineering managers evaluating BharatData for adoption.

Location: `docs/architecture/ARCHITECTURE.md`

Key content: System overview diagram, universal engine architecture, 7-layer pipeline explanation, YAML source definition schema, API architecture, SDK architecture, database schema, deployment infrastructure, technology stack with rationale for each choice.

Document 19: Public Roadmap

Purpose: Shows what BharatData is working on now, what is planned next, and what is being considered based on community feedback.

Audience: Users, community members, potential contributors.

Location: `ROADMAP.md` in repository root or a page on the website.

Key content: Three sections: Now (actively building), Next (committed for upcoming releases), Considering (community-requested features). No specific dates. Invites community input through GitHub issues with "roadmap" label.

Document 20: Brand Guidelines

Purpose: Ensures consistent visual identity across all BharatData touchpoints.

Audience: Contributors, anyone writing about BharatData, media outlets.

Location: `docs/brand/BRAND.md`

Key content: Logo usage rules, color palette with hex codes, typography specifications, name format ("Bharat Data" in prose, "bharatdata" in code), tagline, disclaimer text, voice and tone guidelines.

Part 3: Production Checklist

Before announcing BharatData publicly, verify every item on this list:

Infrastructure Checklist

Item	Check	Status
API deployed to Cloudflare Workers	Hit /v1/health and get 200	
Supabase database accessible	API can query and return data	
Edge caching configured	Second request to same endpoint is faster	
Rate limiting active	11th rapid request returns 429	
SSL/HTTPS working	All URLs use https	
CORS configured	Playground can call API from browser	
Error responses follow standard format	Every error has code, message, suggestion	
Health check runs daily	GitHub Action executes without errors	
Quality check runs weekly	GitHub Action executes without errors	

SDK Checklist

Item	Check	Status
TypeScript SDK on npm	`npm install bharatdata` works in fresh directory	
Python SDK on PyPI	`pip install bharatdata` works in fresh virtualenv	
TypeScript types are complete	Zero `any` in public API surface	

Item	Check	Status
Python type hints complete	mypy passes with zero errors	
Both SDKs return same data	Same query gives identical numbers	
Error handling works	Invalid state throws SourceNotFoundError in both	
Rate limit handling works	429 response creates RateLimitError in both	
README examples work	Copy-paste from README runs successfully	
CHANGELOG has first entry	Version 0.1.0 documented	

Playground Checklist

Item	Check	Status
Playground deployed	URL loads in browser	
Natural language query works	“Crime in Maharashtra 2023” returns results	
Structured query works	Dropdown selection returns results	
Data table renders correctly	Numbers formatted, columns sortable	
CSV download works	File opens in spreadsheet correctly	
JSON download works	File contains data, meta, query	
Chart renders correctly	Appropriate chart type for query	
Mobile responsive	Usable on phone screen	
Error states handled	No blank screens or raw errors	
LLM fallback works	Data shows even if AI is unavailable	

Item	Check	Status
Source citations present	Every number has traceable source	
Page loads under 3 seconds	Measured with browser dev tools	

Data Quality Checklist

Item	Check	Status
All records verified against totals	TotalMatcher passed for all files	
Confidence scores are graduated	Not clustered at 1.0	
No summary rows in data tables	State count is exactly 36 per year	
Null handling is context-aware	Count fields have zeros, disposal fields have nulls	
Source attribution on every record	source_url and source_file populated	
Geographic resolution complete	98%+ entities resolved to canonical names	

Documentation Checklist

Item	Check	Status
Root README complete and accurate	All code examples work	
TypeScript SDK README on npm	Visible on npmjs.com package page	
Python SDK README on PyPI	Visible on pypi.org package page	
LICENSE file present	MIT license text	
CONTRIBUTING guide present	Development setup instructions work	

Item	Check	Status
SECURITY policy present	Security email configured	
Terms of Use published	Accessible on website or repo	
Privacy Policy published	Accessible on website or repo	
Data Attribution document published	Source citations and licensing explained	

Legal and Compliance Checklist

Item	Check	Status
Independence disclaimer visible	On playground, README, API docs, footer	
Source attribution on every response	meta.source populated correctly	
Rate limiting respects government servers	Max 1 req/sec in pipeline	
No CAPTCHA circumvention	Pipeline does not bypass access controls	
Robots.txt respected	Pipeline checks robots.txt before scraping	
User data handling documented	Privacy policy covers all collected data	

Part 4: Post-Launch Monitoring

After launching, monitor these metrics daily for the first month:

API Health: Response time p50 and p95. Error rate (percentage of 5xx responses). Cache hit rate. Rate limit trigger frequency.

SDK Adoption: npm weekly downloads. PyPI weekly downloads. GitHub stars. GitHub forks.

Playground Usage: Daily unique visitors. Queries per day. Most common query patterns. Error rate on the playground.

Data Quality: Daily health check results. Weekly quality check results. User-reported data issues (GitHub issues with data-issue label).

Community Engagement: GitHub issues opened. GitHub issues responded to (track your response time). Pull requests submitted. Community questions on discussions.

Set up simple alerting: if the API error rate exceeds 5% or the health check fails for 2 consecutive days, you should be notified. GitHub Actions can send notifications through email or Slack webhooks.

Document Priority Summary

Priority	Document	Create Before Launch	Status
1	Root README.md	Yes	
2	TypeScript SDK README.md	Yes	
3	Python SDK README.md	Yes	
4	LICENSE	Yes	
5	CHANGELOG.md	Yes	
6	CODE_OF_CONDUCT.md	Yes	
7	CONTRIBUTING.md	Yes	
8	SECURITY.md	Yes	
9	Data Attribution	Yes	
10	Terms of Use	Yes	
11	Privacy Policy	Yes	
12	API Documentation	Yes	
13	Data Source Registry	Yes	
14	Journalist Guide	Within 1 month	
15	Researcher Guide	Within 1 month	
16	Developer Guide	Within 1 month	
17	Data Quality Report	Within 1 month	
18	Architecture Overview	Within 1 month	
19	Public Roadmap	Within 1 month	
20	Brand Guidelines	Within 1 month	

Write documents 1-13 before your public launch. They are essential for credibility and usability. Documents 14-20 can be written in the first month after launch as you respond to user feedback and understand which guides are most needed.

This completes the production polish guide. When every item on the infrastructure, SDK, playground, data quality, documentation, and legal checklists is checked off, BharatData is ready for public launch.